PHP — Banco de dados (MySQLi, PDO)

Manual oficial do PHP (php.net) — Português do Brasil

Fonte oficial: https://www.php.net/manual/
Gerado para o projeto doc-frog

Drivers e Plugins MySQL

PHP oferece vários drivers e plugins MySQL para acessar e lidar com o MySQL.

As diferenças e a funcionalidade das extensões MySQL estão descritas na visão geral desta seção.

As extensões listadas suportam o protocolo MySQL. Exemplos de servidores de banco de dados compatíveis são » MariaDB Server, » MySQL Server, » Percona Server for MySQL e » TiDB.

Visão geral dos drivers PHP para MySQL

Introdução

Existem muitas APIs de PHP para acessar bancos de dados MySQL. Os usuários podem escolher entre as extesões mysqli ou PDO_MySQL.

Este guia explica a terminologia usada para descrever cada API, informação sobre a escolha de qual API a ser usada, e também informação para ajudar a escolher qual biblioteca MySQL a ser usada com a API.

Visão geral de terminologia

Esta seção fornece uma introdução às opções disponíveis ao desenvolver aplicações PHP que precisem interagir com um banco de dados MySQL.

O que é uma API?

Uma Interface de Programação para Aplicações, ou API, define as classes, métodos, funções e variáveis que a aplicação precisará chamar para que consiga efetuar a tarefa desejada. No caso de aplicações PHP que precisem comunicar com banco de dados, as APIs necessárias são normalmente expostas através de extensões do PHP.

APIs podem ser procedurais ou orientadas a objeto. Com uma API procedural, pode-se chamar funções para realizar tarefas, e com uma API orientada a objeto, pode-se instanciar classes e então chamar métodos nos objetos resultantes. Entre as duas opções, a última é a interface recomendada, já que é mais moderna e leva a um código mais organizado.

Ao escrever aplicações PHP que precisam se conectar ao servidor MySQL, existem várias opções de API disponíveis. Este documento discute o que está disponível e como selecionar a melhor solução para cada aplicação.

O que é um conector?

Na documentação MySQL, o termo *conector* se refere a uma parte do software que permite à aplicação se conectar ao servidor de banco de dados MySQL. O MySQL fornece conectores para uma variedade de linguagens, incluindo o PHP.

Se uma aplicação PHP precisa se comunicar com um servidor de banco de dados será necessário escrever código PHP para realizar tais atividades como conexão ao banco de dados, consultas ao banco e outras funções relacionadas. Software é requerido para fornecer a API que a aplicação PHP irá utilizar, e também para lidar com a comunicação entre a aplicação e o servidor de banco de dados, possivelmente usando outras bibliotecas intermediárias quando necessário. Este software é conhecimento genericamente como conector, já que ele permite à aplicação se *conectar* ao servidor de banco de dados.

O que é um Driver?

Um driver é uma parte do software projetada para se comunicar com um tipo específico de servidor de banco de dados. O driver também pode chamar uma biblioteca, tal como a Bliblioteca Cliente MySQL ou o Driver Nativo MySQL. Estas bibliotecas implementam o protocolo de baixo nível usado para comunicação com o servidor de dados MySQL.

Para exemplificar, a camada de abstração de banco da dados PDO - Objetos de Dados PHP pode usar um dos vários drivers específicos para banco de dados. Um dos drivers disponíveis é o driver PDO MYSQL, que permite fazer interface com o servidor MySQL.

Algumas vezes os termos conector e driver são usados de maneira intercambiável, o que pode ser confuso. Na documentação relacionada com o MySQL o termo "driver" é reservado para o software que fornece a parte específica de banco de dados de um pacote de conector.

O que é uma Extensão?

Na documentação PHP, encontramos um outro termo - *extensão*. O código PHP consiste de um núcleo, com extensões opcionais à funcionalidade do núcleo. A extensão do PHP relacionada com o MySQL, `mysqli`, é implementada usando o framework de extensão PHP.

Uma extensão tipicamente expõe uma API para o programador PHP, para permitir que suas instalações possam ser usadas programaticamente. Entretanto, algumas extensões que usam o framework de extensão PHP não expõem uma API ao programador PHP.

A extensão do driver de PDO MySQL, por exemplo, não expõe uma API ao programador PHP, mas fornece uma interface à camada PDO acima dela.

Os termos API e extensão não podem ser considerados com o mesmo significado, já que uma extensão não necessáriamente expões uma API ao programador.

Escolhendo uma API

O PHP oferece diferentes APIs para conexão ao MySQL. Pode-se ver abaixo as APIs fornecidas pelas extensões mysqli e PDO. Cada porção de código cria uma conexão a um servidor MySQL sendo executado em "example.com" usando o usuário "user" e a senha "password". E uma consulta é executada para cumprimentar o usuário.

Exemplo #1 Comparando as APIs do MySQL

```php
<?php
// mysqli
$mysqli = new mysqli("example.com", "user", "password", "database");
$result = $mysqli->query("SELECT 'Olá, prezado usuário MySQL!' AS _message FROM DUAL");
$row = $result->fetch_assoc();
echo htmlentities($row['_message']);

// PDO
$pdo = new PDO('mysql:host=example.com;dbname=database', 'user', 'password');
$statement = $pdo->query("SELECT 'Olá, prezado usuário MySQL!' AS _message FROM DUAL");
$row = $statement->fetch(PDO::FETCH_ASSOC);
echo htmlentities($row['_message']);
```

Exemplo #2 Comparando instruções preparadas

```php
<?php
// mysqli
$mysqli = new mysqli("example.com", "user", "password", "database");
$statement = $mysqli->prepare("SELECT District FROM City WHERE Name=?");
$statement->execute(["Amersfoort"]);
$result = $statement->get_result();
$row = $result->fetch_assoc();
echo htmlentities($row['District']);

// PDO
$pdo = new PDO('mysql:host=example.com;dbname=database', 'user', 'password');
$statement = $pdo->prepare("SELECT District FROM City WHERE Name=?");
$statement->execute(["Amersfoort"]);
$row = $statement->fetch(PDO::FETCH_ASSOC);
echo htmlentities($row['District']);
```

Comparação de características

O desempenho geral de ambas as extensões é considerada aproximadamente a mesma, embora o desempenho da extensão contribui somente para uma fração do tempo total de uma requisição web do PHP. Frequentemente, o impacto é tão baixo quanto 0,1%.

	ext/mysqli	**PDO_MySQL**
Versão em que foi introduzida no PHP	5.0	5.1
Incluída no PHP 7.x e 8.x	Sim	Sim
Estado do desenvolvimento	Ativo	Ativo
Ciclo de vida	Ativo	Ativo
Recomendado para novos projetos	Sim	Sim
Interface POO	Sim	Sim
Interface Procedural	Sim	Não
API suporta consultas sem bloqueio assíncronas com mysqlnd	Sim	Não
Conexões persistentes	Sim	Sim
API suporta conjuntos de caracteres	Sim	Sim
API suporta Instruções Preparadas no lado do servidor	Sim	Sim
API suporta Instruções Preparadas no lado do cliente	Não	Sim
API suporta Procedimentos Armazenados	Sim	Sim
API suporta Múltiplas Instruções	Sim	A maioria
API suporta Transações	Sim	Sim
Transações podem ser controladas com SQL	Sim	Sim
Supporta toda a funcionalidade do MySQL 5.1+	Sim	A maioria

Escolhendo uma biblioteca

As extensões mysqli e PDO_MySQL são encapsuladores leves de uma biblioteca cliente em C. As extensões podem tanto usar a biblioteca mysqlnd quanto a biblioteca `libmysqlclient`. Escolher uma biblioteca é uma decisão a ser tomada na compilação.

A biblioteca mysqlnd é parte da distribuição PHP. Ela oferece recursos como conexões preguiçosas e cache de consultas, que não estão disponíveis com a libmysqlclient, portanto usar a biblioteca mysqlnd interna é altamente recomendado. Consulte a documentação mysqlnd para detalhes adicionais e uma lista de recursos e funcionalidade que ela oferece.

Exemplo #1 Comandos de configuração para usar mysqlnd ou libmysqlclient

```
// Recomendado, compilar com mysqlnd
$ ./configure --with-mysqli=mysqlnd --with-pdo-mysql=mysqlnd

// Alternativamente recomendado, compilar com mysqlnd
$ ./configure --with-mysqli --with-pdo-mysql

// Não recomendado, compila com libmysqlclient
$ ./configure --with-mysqli=/path/to/mysql_config --with-pdo-mysql=/path/to/mysql_config
```

Comparação de recursos de bibliotecas

É recomendado usar a biblioteca mysqlnd ao invés da biblioteca Cliente-Servidor do MySQL (libmysqlclient). Ambas são suportadas e estão constantemente sendo melhoradas.

	Driver nativo MySQL (mysqlnd)	**Biblioteca cliente-servidor MySQL (`libmysqlclient`)**
Faz parte da distribuição PHP	Sim	Não
Versão do PHP em que foi introduzida	5.3.0	N/A
Licença	Licença PHP 3.01	Dupla Licença
Estado de desenvolvimento	Ativo	Ativo
Ciclo de vida	Nenhum final anunciado	Nenhum final anunciado
Padrão de compilação (para todas as extensões MySQL)	Sim	Não
Suporte a protocolo de compressão	Sim	Sim
Suporte a SSL	Sim	Sim
Suporte a tubos nomeados	Sim	Sim
Consultas sem bloqueio assíncronas	Sim	Não
Estatísticas de desempenho	Sim	Não
LOAD LOCAL INFILE respeita a diretiva open_basedir	Sim	Não
Usa o sistema de gerenciamento de memória nativo do PHP (ex: segue os limites de memória do PHP)	Sim	Não
Retorna coluna numérica como dupla precisão (COM_QUERY)	Sim	Não
Retorna coluna numérica como string (COM_QUERY)	Sim	Sim
API tipo plugin	Sim	Limitada
Reconexão automática	Não	Opcional

Conceitos

Índice

Esses conceitos são específicos dos drivers MySQL para PHP.

Consultas com buffer e sem buffer

As consultas usam o modo de buffer por padrão. Isso significa que os resultados da consulta são imediatamente transferidos do servidor MySQL para o PHP e então mantidos na memória do processo PHP. Isso permite operações adicionais, como contar o número de linhas e mover (buscar) o ponteiro do resultado atual. Também permite emitir consultas adicionais na mesma conexão enquanto trabalha no conjunto de resultados. A desvantagem do modo buffer é que conjuntos de resultados maiores podem exigir bastante memória. A memória será mantida ocupada até que todas as referências ao conjunto de resultados sejam desativadas ou o conjunto de resultados seja explicitamente liberado, o que acontecerá automaticamente durante o término da solicitação, o mais tardar. A terminologia "armazenar resultado" também é usada para o modo buffer, pois todo o conjunto de resultados é armazenado de uma só vez.

> **Nota** :
>
> Ao usar a libmysqlclient como biblioteca, o limite de memória do PHP não contará a memória usada para conjuntos de resultados, a menos que os dados sejam buscados em variáveis PHP. Com o mysqlnd a memória contabilizada incluirá o conjunto completo de resultados.

Consultas MySQL sem buffer executam a consulta e aguardam que os dados do servidor MySQL sejam buscados. Isso usa menos memória do lado do PHP, mas pode aumentar a carga no servidor. A menos que o conjunto completo de resultados tenha sido obtido do servidor, nenhuma consulta adicional poderá ser enviada pela mesma conexão. Consultas sem buffer também podem ser chamadas de "resultado de uso". Depois que todas as linhas do conjunto de resultados forem buscadas, o conjunto de resultados desaparecerá e não poderá ser iterado novamente.

Seguindo essas características, consultas sem buffer devem ser utilizadas apenas quando se espera um grande conjunto de resultados que será processado sequencialmente. As consultas sem buffer contêm uma série de armadilhas que tornam mais difícil seu uso, por exemplo, o número de linhas no conjunto de resultados é desconhecido até que a última linha seja obtida. Consultas em buffer são a maneira mais fácil e flexível de processar conjuntos de resultados.

Como as consultas em buffer são o padrão, os exemplos abaixo demonstrarão como executar consultas sem buffer com cada API.

Exemplo #1 Exemplo de consulta sem buffer: mysqli

```php
<?php
$mysqli  = new mysqli("localhost", "my_user", "my_password", "world");
$unbufferedResult = $mysqli->query("SELECT Name FROM City", MYSQLI_USE_RESULT);

foreach ($unbufferedResult as $row) {
    echo $row['Name'] . PHP_EOL;
}
?>
```

Exemplo #2 Exemplo de consulta sem buffer: pdo_mysql

```php
<?php
$pdo = new PDO("mysql:host=localhost;dbname=world", 'my_user', 'my_password');
```

```php
$pdo->setAttribute(PDO::MYSQL_ATTR_USE_BUFFERED_QUERY, false);

$unbufferedResult = $pdo->query("SELECT Name FROM City");
foreach ($unbufferedResult as $row) {
    echo $row['Name'] . PHP_EOL;
}
?>
```

Conjuntos de caracteres

Idealmente, um conjunto de caracteres adequado será definido no nível do servidor, e isso é descrito na seção » Configuração do conjunto de caracteres do manual do servidor MySQL. Alternativamente, cada API MySQL oferece um método para definir o conjunto de caracteres em tempo de execução.

> **Cuidado**
>
> ### O conjunto de caracteres e escape de caracteres
>
> O conjunto de caracteres deve ser entendido e definido, pois afeta todas as ações e inclui implicações de segurança. Por exemplo, o mecanismo de escape (por exemplo, mysqli_real_escape_string() para mysqli e PDO::quote() para PDO_MySQL) irá aderir a esta configuração. É importante perceber que essas funções não usarão o conjunto de caracteres definido com uma consulta, portanto, por exemplo, o seguinte não terá efeito sobre elas:

Exemplo #1 Problemas ao definir o conjunto de caracteres com SQL

```php
<?php

$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

// Não afetará $mysqli->real_escape_string();
$mysqli->query("SET NAMES utf8mb4");

// Não afetará $mysqli->real_escape_string();
$mysqli->query("SET CHARACTER SET utf8mb4");

// Mas isto afetará $mysqli->real_escape_string();
$mysqli->set_charset('utf8mb4');

// Porém, isto NÃO irá afetá-la (UTF-8 vs utf8mb4) -- não use hífens aqui
$mysqli->set_charset('UTF-8');
?>
```

Abaixo estão exemplos que demonstram como alterar adequadamente o conjunto de caracteres em tempo de execução usando cada API.

> **Nota** : **Possível confusão UTF-8**
>
> Como os nomes dos conjuntos de caracteres no MySQL não contêm hífens, a string "utf8" é válida no MySQL para definir o conjunto de caracteres como UTF-8 (codificação Unicode UTF-8 de até 3 bytes). A string "UTF-8" não é válida, pois o uso de "UTF-8" não alterará o conjunto de caracteres e gerará um erro.

Exemplo #2 Exemplo de definição do conjunto de caracteres: mysqli

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

echo 'Conjunto de caracteres inicial: ' . $mysqli->character_set_name() . "\n";

if (!$mysqli->set_charset('utf8mb4')) {
    printf("Erro ao carregar o conjunto de caracteres utf8mb4: %s\n", $mysqli->error);
    exit;
}

echo 'Seu conjunto de caracteres atual é: ' . $mysqli->character_set_name() . "\n";
?>
```

Exemplo #3 Exemplo de definição do conjunto de caracteres: pdo_mysql

```php
<?php
$pdo = new PDO("mysql:host=localhost;dbname=world;charset=utf8mb4", 'my_user', 'my_pass');
?>
```

MySQLi

Introdução

A extensão `mysqli` permite a você acessar a funcionalidade provida pelo MySQL 4.1 e posterior. Maiores informações sobre o servidor de banco de dados MySQL podem ser encontradas em » http://www.mysql.com/

Uma visão geral de software disponível para usar MySQL através do PHP pode ser encontrada em Visão Geral

Documentação para o MySQL pode ser encontrada em » http://dev.mysql.com/doc/.

Partes desta documentação foram incluídas do manual do MySQL com a permissão de Oracle Corporation.

Os exemplos usam tanto o banco de dados » world quanto o banco de dados » sakila, que estão disponíveis livremente.

Visão Geral

Esta seção fornece uma introdução às opções disponíveis ao desenvolver um aplicativo PHP que precisa interagir com um banco de dados MySQL.

O que é uma API?

Uma interface de programação de aplicativos, ou API, define as classes, métodos, funções e variáveis que seu aplicativo precisará chamar para executar a tarefa desejada. No caso de aplicativos PHP que precisam se comunicar com bancos de dados, as APIs necessárias geralmente são expostas por meio de extensões PHP.

As APIs podem ser processuais ou orientadas a objetos. Com uma API procedural, você chama funções para realizar tarefas, com a API orientada a objetos você instancia classes e então chama métodos nos objetos resultantes. Das duas, a última costuma ser a interface preferida, pois é mais moderna e leva a um código mais organizado.

Ao escrever aplicativos PHP que precisam se conectar ao servidor MySQL, existem várias opções de API disponíveis. Este documento discute o que está disponível e como selecionar a melhor solução para sua aplicação.

O que é um Conector?

Na documentação do MySQL, o termo *conector* refere-se a um software que permite que seu aplicativo se conecte ao servidor de banco de dados MySQL. MySQL fornece conectores para uma variedade de linguagens, incluindo PHP.

Se seu aplicativo PHP precisar se comunicar com um servidor de banco de dados, você precisará escrever código PHP para executar atividades como conectar-se ao servidor de banco de dados, consultar o banco de dados e outras funções relacionadas ao banco de dados. O software é necessário para fornecer a API que seu aplicativo PHP usará e também para lidar com a comunicação entre seu aplicativo e o servidor de banco de dados, possivelmente usando outras bibliotecas intermediárias quando necessário. Este software é conhecido genericamente como conector, pois permite que sua aplicação se *connect* a um servidor de banco de dados.

O que é um Driver?

Um driver é um software projetado para se comunicar com um tipo específico de servidor de banco de dados. O driver também pode chamar uma biblioteca, como a MySQL Client Library ou o MySQL Native Driver. Essas bibliotecas implementam o protocolo de baixo nível usado para se comunicar com o servidor de banco de dados MySQL.

A título de exemplo, a camada de abstração do banco de dados PHP Data Objects (PDO) pode usar um dos vários drivers específicos do banco de dados. Um dos drivers disponíveis é o driver PDO MYSQL, que permite a interface com o servidor MySQL.

Às vezes, as pessoas usam os termos conector e driver de forma intercambiável, o que pode ser confuso. Na documentação relacionada ao MySQL, o termo "driver" é reservado para software que fornece a parte específica do banco de dados de um pacote de conector.

O que é uma Extensão?

Na documentação do PHP, você encontrará outro termo - *extension*. O código PHP consiste em um núcleo, com extensões opcionais para a funcionalidade principal. As extensões relacionadas ao MySQL do PHP, como a extensão mysqli e a extensão do driver PDO MySQL, são implementadas usando a estrutura de extensão do PHP.

Uma extensão normalmente expõe uma API ao programador PHP, para permitir que suas facilidades sejam usadas programaticamente. No entanto, algumas extensões que usam a estrutura de extensão PHP não expõem uma API ao programador PHP.

A extensão do driver PDO MySQL, por exemplo, não expõe uma API ao programador PHP, mas fornece uma interface para a camada PDO acima dela.

Os termos API e extensão não devem significar a mesma coisa, pois uma extensão pode não necessariamente expor uma API ao programador.

Quais são as principais ofertas de API do PHP para usar o MySQL?

Existem duas opções principais de API ao considerar a conexão com um servidor de banco de dados MySQL:

- Extensão mysqli do PHP
- Objetos de Dados PHP (PDO)

Cada um tem suas próprias vantagens e desvantagens. A discussão a seguir visa fornecer uma breve introdução aos principais aspectos de cada API.

O que é a extensão mysqli do PHP?

A extensão `mysqli`, ou como às vezes é conhecida, a extensão *improved* do MySQL, foi desenvolvida para aproveitar os novos recursos encontrados nos sistemas MySQL versões 4.1.3 e mais recentes. A extensão `mysqli` está incluída nas versões 5 e posteriores do PHP.

A extensão `mysqli` tem vários benefícios, sendo os principais aprimoramentos sobre a extensão `mysql`:

- Interface orientada a objetos
- Suporte para Declarações Preparadas
- Suporte para várias declarações
- Suporte para transações
- Recursos de depuração aprimorados

Assim como a interface orientada a objetos, a extensão também fornece uma interface processual.

A extensão `mysqli` é construída usando a e strutura de extensão PHP, seu código fonte está localizado no diretório *ext/mysqli*.

Para obter mais informações sobre a extensão `mysqli`, consulte MySQLi.

O que é PDO?

PHP Data Objects, ou PDO, é uma camada de abstração de banco de dados especificamente para aplicativos PHP. O PDO fornece uma API consistente para seu aplicativo PHP, independentemente do tipo de servidor de banco de dados ao qual seu aplicativo se conectará. Em teoria, se você estiver usando a API PDO, você pode mudar o servidor de banco de dados que você usou, digamos Firebird para MySQL, e só precisa fazer pequenas alterações em seu código PHP.

Outros exemplos de camadas de abstração de banco de dados incluem JDBC para aplicativos Java e DBI para Perl.

Embora o PDO tenha suas vantagens, como uma API limpa, simples e portátil, sua principal desvantagem é que ele não permite que você use todos os recursos avançados disponíveis nas versões mais recentes do servidor MySQL. Por exemplo, PDO não permite que você use o suporte do MySQL para Multiple Statements.

PDO é implementado usando o framework de extensão PHP, seu código fonte está localizado no diretório *ext/pdo*.

Para mais informações sobre PDO, consulte o PDO.

O que é o driver PDO MYSQL?

O driver PDO MYSQL não é uma API como tal, pelo menos da perspectiva do programador PHP. Na verdade, o driver PDO MYSQL fica na camada abaixo do próprio PDO e fornece funcionalidade específica do MySQL. O programador ainda chama a API PDO, mas o PDO usa o driver PDO MYSQL para realizar a comunicação com o servidor MySQL.

O driver PDO MYSQL é um dos vários drivers PDO disponíveis. Outros drivers PDO disponíveis incluem aqueles para os servidores de banco de dados Firebird e PostgreSQL.

O driver PDO MYSQL é implementado usando a estrutura de extensão PHP. Seu código-fonte está localizado no diretório *ext/pdo_mysql*. Ele não expõe uma API ao programador PHP.

Para obter mais informações sobre o driver PDO MYSQL, consulte Driver PDO do MySQL.

O que é o driver nativo MySQL do PHP?

Para se comunicar com o servidor de banco de dados MySQL, `mysqli` e o driver PDO MYSQL usam uma biblioteca de baixo nível que implementa o protocolo necessário. No passado, a única biblioteca disponível era a MySQL Client Library, também conhecida como `libmysqlclient`.

No entanto, a interface apresentada pelo `libmysqlclient` não foi otimizada para comunicação com aplicativos PHP, pois `libmysqlclient` foi originalmente projetado com aplicativos C em mente. Por esta razão, o MySQL Native Driver, `mysqlnd`, foi desenvolvido como uma alternativa ao `libmysqlclient` para aplicações PHP.

Tanto a extensão `mysqli` quanto o driver PDO MySQL podem ser configurados individualmente para usar `libmysqlclient`ou `mysqlnd`. Como o `mysqlnd` foi projetado especificamente para ser utilizado no sistema PHP, ele possui vários aprimoramentos de memória e velocidade em relação ao `libmysqlclient`. Você é fortemente encorajado a aproveitar essas melhorias.

O MySQL Native Driver é implementado usando a estrutura de extensão PHP. O código-fonte está localizado em *ext/mysqlnd*. Ele não expõe uma API ao programador PHP.

Comparação de recursos

A tabela a seguir compara a funcionalidade dos principais métodos de conexão ao MySQL a partir do PHP:

Comparação das opções da API do MySQL para PHP

	Extensão mysqli do PHP	**PDO (usando PDO MySQL Driver e MySQL Native Driver)**
Versão do PHP introduzida	5.0	5.0
Status de desenvolvimento do MySQL	Desenvolvimento ativo	Desenvolvimento ativo
API suporta conjuntos de caracteres	Sim	Sim
A API oferece suporte a instruções preparadas do lado do servidor	Sim	Sim
A API oferece suporte a declarações preparadas do lado do cliente	Não	Sim
API suporta procedimentos armazenados	Sim	Sim
A API oferece suporte a várias instruções	Sim	A maioria
Suporta todas as funcionalidades do MySQL 4.1+	Sim	A maioria

Guia de início rápido

Índice

Este guia de início rápido irá ajudar nas escolhas e no ganho de familiaridade com a API de MySQL do PHP.

Este início rápido dá uma visão geral da extensão mysqli. Exemplos de código são disponibilizados para todos os macro aspectos da API. Conceitos de banco de dados são explicados no grau necessário para apresentar os conceitos específicos do MySQL.

Requisito: Uma familiaridade com a linguagem de programação PHP, com a linguagem SQL, e conhecimento básico do servidor MySQL.

Interface dupla, procedural e orientada a objetos

A extensão mysqli possibilita uma dupla interface. Ela suporta o paradigma de programação procedural e orientada a objetos.

Usuários que estão migrando da extensão antiga mysql podem preferir a interface procedural. Esta interface é similar àquela da extensão antiga mysql. Em muitos casos, os nomes das funções diferem apenas pelo prefixo. Algumas funções mysqli levam um manipulador de conexão como seu primeiro argumento, enquanto as funções análogas na antiga interface mysql levam este manipulador como o último argumento, opcional.

Exemplo #1 Migração fácil da antiga extensão mysql

```
<?php
$mysqli = mysqli_connect("example.com", "user", "password", "database");
$result = mysqli_query($mysqli, "SELECT 'Por favor não use a extensão mysql descontinuada para novos
desenvolvimentos. ' AS _msg FROM DUAL");
$row = mysqli_fetch_assoc($result);
echo $row['_msg'];

$mysql = mysql_connect("example.com", "user", "password");
mysql_select_db("test");
$result = mysql_query("SELECT 'Use a extensão mysqli em seu lugar.' AS _msg FROM DUAL", $mysql);
$row = mysql_fetch_assoc($result);
echo $row['_msg'];
```

O exemplo acima produzirá:

```
Por favor não use a extensão mysql descontinuada para novos desenvolvimentos. Use a extensão mysqli
em seu lugar.
```

A interface orientada a objetos

Em adição à clássica interface procedural, usuários podem escolher usar a interface orientada a objetos. A documentação está organizada utilizando a interface orientada a objetos. Ela mostra funções agrupadas por propósito, tornando a iniciação mais fácil. A seção de referência dá exemplos para ambas as variantes de sintaxe.

Não há diferenças significativas de desempenho entre as duas interfaces. Usuários podem basear suas escolhas em preferências pessoais.

Exemplo #2 Interface orientada a objeto e procedural

```
<?php

$mysqli = mysqli_connect("example.com", "user", "password", "database");

$result = mysqli_query($mysqli, "SELECT 'Um mundo cheio de ' AS _msg FROM DUAL");
$row = mysqli_fetch_assoc($result);
echo $row['_msg'];

$mysqli = new mysqli("example.com", "user", "password", "database");

$result = $mysqli->query("SELECT 'escolhas para agradar a todos.' AS _msg FROM DUAL");
```

```php
$row = $result->fetch_assoc();
echo $row['_msg'];
```

O exemplo acima produzirá:

```
Um mundo cheio de escolhas para agradar a todos.
```

A interface orientada a objetos é usada neste guia porque a seção de referência está organizada dessa forma.

Misturando estilos

É possível trocar de estilos a qualquer momento. Misturar ambos os estilos não é recomendado por questões de clareza e estilo de código.

Exemplo #3 Estilo de código ruim

```php
<?php

$mysqli = new mysqli("example.com", "user", "password", "database");

$result = mysqli_query($mysqli, "SELECT 'Estilo possível, porém, ruim.' AS _msg FROM DUAL");

if ($row = $result->fetch_assoc()) {
    echo $row['_msg'];
}
```

O exemplo acima produzirá:

```
Estilo possível, porém, ruim.
```

Veja também

- mysqli::__construct()
- mysqli::query()
- mysqli_result::fetch_assoc()
- $mysqli::connect_errno
- $mysqli::connect_error
- $mysqli::errno
- $mysqli::error
- Resumo das Funções da Extensão MySQLi

Conexões

O servidor MySQL suporta o uso de diferentes camadas de transporte para conexões. Conexões usam TCP/IP, sockets de domínio Unix ou pipes nomeados do Windows.

O nome de servidor `localhost` tem um significado especial. Ele é vinculado ao uso de sockets de domínio Unix. Para abrir uma conexão TCP/IP ao servidor local, `127.0.0.1` deve ser usado ao invés do nome `localhost`.

Exemplo #1 Significado especial de localhost

```php
<?php

$mysqli = new mysqli("localhost", "user", "password", "database");

echo $mysqli->host_info . "\n";

$mysqli = new mysqli("127.0.0.1", "user", "password", "database", 3306);

echo $mysqli->host_info . "\n";
```

O exemplo acima produzirá:

```
Localhost via UNIX socket
127.0.0.1 via TCP/IP
```

Padrão dos parâmetros de conexão

Dependendo da função de conexão usada, alguns parâmetros podem ser omitidos. Se um parâmetro não for fornecido, a extensão tenta usar os valores padrões definidos no arquivo de configuração do PHP.

Exemplo #2 Padrões de configuração

```
mysqli.default_host=192.168.2.27
mysqli.default_user=root
mysqli.default_pw=""
mysqli.default_port=3306
mysqli.default_socket=/tmp/mysql.sock
```

Os valores dos parâmetros resultantes são então passados à biblioteca cliente que é usada pela extensão. Se a biblioteca cliente detecta parâmetros vazios ou indefinidos, ela pode usar seus próprios valores padrão internos.

Padrões de conexão internos da biblioteca

Se o valor do servidor for indefinido ou vazio, a biblioteca cliente irá considerar uma conexão socket Unix em `localhost`. Se o socket for indefinido ou vazio, e uma conexão socket Unix for solicitada, é feita tentativa de conexão ao socket padrão em `/tmp/mysql.sock`.

Em sistemas Windows, o nome de servidor '`.`' é interpretado pela biblioteca cliente como uma tentativa de se abrir uma conexão baseada em pipe nomeado do Windows. Neste caso, o parâmetro socket é interpretado como o nome do pipe. Se não informado ou vazio, o socket (nome do pipe) será igual ao padrão `\\.\pipe\MySQL`.

Se a conexão a ser estabelecida não for baseada em socket de domínio Unix nem em pipe nomeado Windows, e o parâmetro da porta estiver indefinido, a biblioteca usará o valor padrão de porta `3306`.

A biblioteca mysqlnd e a Biblioteca Cliente MySQL (libmysqlclient) implementam a mesma lógica para determinar os padrões.

Opções de conexão

Opções de conexão estão disponíveis para, por exemplo, definir comando de inicialização que são executados na conexão, ou para requisitar uso de um conjunto de caracteres específico. Opções de conexão devem ser definidas antes que uma conexão seja estabelecida.

Para definir uma opção de conexão, a operação de conexão deve ser realizada em três passos: criar um manipulador de conexão com mysqli_init() ou mysqli::__construct(), definir as opções necessárias usando mysqli::options(), e estabelecer a conexão à rede com mysqli::real_connect().

Agrupamento de conexão

A extensão mysqli suporta conexões persistentes de banco de dados, que são um tipo especial de conexões agrupadas. Por padrão, cada conexão ao banco aberta por um script é explicitamente fechada pelo usuário durante a execução ou liberada automaticamente ao final do script. Com uma conexão permanente isso não acontece. Ao invés disso, ela é colocada em um grupo para reúso posterior, se uma conexão ao mesmo servidor, usando o mesmo usuário, senha, socket, porta e banco de dados padrão, for aberta. O reúso economiza tempo e reduz a sobrecarga de conexão.

Cada processo do PHP usa seu próprio agrupamento de conexões mysqli. Dependendo do modelo de implantação do servidor web, um processo PHP pode servir a uma ou múltiplas requisições. Portanto, uma conexão agrupada pode ser usada por um ou mais scripts subsequentemente.

Conexão persistente

Se uma conexão persistente sem uso para uma certa combinação de servidor, usuário, senha, socket, porta e banco padrão não puder ser encontrada no agrupamento, a extensão mysqli abre uma nova conexão. O uso de conexões persistentes pode ser habilitado e desabilitado usando a diretiva mysqli.allow_persistent do PHP. O número total de conexões abertas por um script pode ser limitado com mysqli.max_links. O número máximo de conexões persistentes por processo PHP pode ser restringido com mysqli.max_persistent. Favor observar que o servidor web pode gerar muitos processos PHP.

Uma queixa comum sobre conexões permanentes é que seus estados não são redefinidos antes do reúso. Por exemplo, transações abertas e não terminadas não são automaticamente desfeitas. Além disso, mudanças de autorização, que aconteceram no intervalo entre a inserção da conexão no agrupamento e seu reúso, não terão efeito. Isto pode ser visto como um efeito colateral indesejado. De outro modo, o nome `persistente` pode ser entendido como uma promessa de que o estado da conexão irá persistir..

A extensão mysqli suporta ambas as interpretações de conexão persistente: persistência do estado, e redefinição do estado, antes do reúso. O padrão é redefinição. Antes que uma conexão persistente seja reutilizada, a extensão mysqli chama mysqli::change_user() implicitamente para redefinir o estado. A conexão persistente aparece ao usuário como se tivesse acabado de ser aberta. Nenhum artefato de usos anteriores é visível.

A chamada a mysqli::change_user() é uma operação custosa. Para melhor desempenho, usuários podem querer recompilar a extensão com a opção **`MYSQLI_NO_CHANGE_USER_ON_PCONNECT`** definida.

É deixado ao usuário a escolha entre comportamento seguro e melhor desempenho. Ambos são objetivos de otimização válidos. Para facilidade de uso, o comportamento seguro foi escolhido como padrão com o prejuízo do desempenho.

Veja também

- mysqli::__construct()
- mysqli_init()
- mysqli::options()

- mysqli::real_connect()
- mysqli::change_user()
- $mysqli::host_info
- Opções de Configuração MySQLi
- Conexões Persistentes de Banco de Dados

Executando instruções

Instruções podem ser executadas com mysqli::query(), mysqli::real_query() e mysqli::multi_query(). A função mysqli::query() é a mais comum, e combina a instrução em execução com uma busca em buffer de seu conjunto de resultados, se houver, em uma chamada. Chamar mysqli::query() é idêntico a chamar mysqli::real_query() seguido de mysqli::store_result().

Exemplo #1 Executando consultas

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");
```

Conjunto de resultados com buffer

Depois da execução da instrução, os resultados podem ser recebidos de uma vez ou lidos linha a linha do servidor. O buffer do conjunto de resultados do lado do cliente permite ao servidor liberar recursos associados com os resultados da instrução da forma mais antecipada possível. Em geral, clientes são consumidores lentos dos resultados. Portanto, é recomendado usar conjuntos de resultados com buffer. mysqli::query() combina buffer de execução de instrução e de conjunto de resultados.

As aplicações PHP podem navegar livremente através de resultados em buffer. A navegação é rápida porque os conjuntos de resultados são mantidos na memória do cliente. Tenha em mente que muitas vezes é mais fácil escalar por cliente do que escalar por servidor.

Exemplo #2 Navegando através de resultados em buffer

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");
$mysqli->query("INSERT INTO test(id) VALUES (1), (2), (3)");

$result = $mysqli->query("SELECT id FROM test ORDER BY id ASC");

echo "Ordem reversa...\n";
for ($row_no = $result->num_rows - 1; $row_no >= 0; $row_no--) {
    $result->data_seek($row_no);
    $row = $result->fetch_assoc();
    echo " id = " . $row['id'] . "\n";
}

echo "Ordem do conjunto de resultados...\n";
```

```php
foreach ($result as $row) {
    echo " id = " . $row['id'] . "\n";
}
```

O exemplo acima produzirá:

```
Ordem reversa...
 id = 3
 id = 2
 id = 1
Ordem do conjunto de resultados...
 id = 1
 id = 2
 id = 3
```

Conjuntos de resultado sem buffer

Se a memória do cliente é um recurso escasso e liberar recursos do servidor tão cedo quanto possível para manter a carga baixa não seja necessário, resultados sem buffer podem ser usados. Navegar por resultados sem buffer não é possível enquanto os dados não tenham sido completamente lidos.

Exemplo #3 Navegando através de resultados sem buffer

```php
<?php

$mysqli->real_query("SELECT id FROM test ORDER BY id ASC");
$result = $mysqli->use_result();

echo "Ordem do conjunto de resultados...\n";
foreach ($result as $row) {
    echo " id = " . $row['id'] . "\n";
}
```

Tipos de dados dos valores do conjunto de resultados

As funções mysqli::query(), mysqli::real_query() e mysqli::multi_query() são usadas para executar instruções não preparadas. No nível do protocolo cliente-servidor do MySQL, o comando `COM_QUERY` e o protocolo de texto são usados para execução de instrução. Com o protocolo de texto, o servidor MySQL converte todos os dados de um resultado em strings antes do envio. Esta conversão é feita independente do tipo de dados da coluna do resultado SQL. As bibliotecas do cliente mysql recebem todos os valores de colunas como strings. Nenhuma conversão adicional do lado do cliente é feita para retornar as colunas a seus tipos nativos. Ao invés disso, todos os valores são fornecidos como strings PHP.

Exemplo #4 Protocolo de texto retorna strings por padrão

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label CHAR(1))");
```

```php
$mysqli->query("INSERT INTO test(id, label) VALUES (1, 'a')");

$result = $mysqli->query("SELECT id, label FROM test WHERE id = 1");
$row = $result->fetch_assoc();

printf("id = %s (%s)\n", $row['id'], gettype($row['id']));
printf("label = %s (%s)\n", $row['label'], gettype($row['label']));
```

O exemplo acima produzirá:

```
id = 1 (string)
label = a (string)
```

É possível converter colunas de inteiros e floats de volta para números PHP configurando a opção de conexão `MYSQLI_OPT_INT_AND_FLOAT_NATIVE`, se a biblioteca mysqlnd estiver sendo usada. Se configurada, a biblioteca mysqlnd irá verificar os tipos de coluna nos metadados do conjunto de resultados e converter colunas SQL para números PHP, se a faixa de valores do tipo de dado PHP permitir. Desta forma, por exemplo, colunas INT do SQL são retornadas como inteiros.

Exemplo #5 Tipos de dados nativos com mysqlnd e opção de conexão

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$mysqli = new mysqli();
$mysqli->options(MYSQLI_OPT_INT_AND_FLOAT_NATIVE, 1);
$mysqli->real_connect("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label CHAR(1))");
$mysqli->query("INSERT INTO test(id, label) VALUES (1, 'a')");

$result = $mysqli->query("SELECT id, label FROM test WHERE id = 1");
$row = $result->fetch_assoc();

printf("id = %s (%s)\n", $row['id'], gettype($row['id']));
printf("label = %s (%s)\n", $row['label'], gettype($row['label']));
```

O exemplo acima produzirá:

```
id = 1 (integer)
label = a (string)
```

Veja também

- mysqli::__construct()

- mysqli::options()
- mysqli::real_connect()
- mysqli::query()
- mysqli::multi_query()
- mysqli::use_result()
- mysqli::store_result()

Instruções Preparadas

O banco de dados MySQL suporta instruções preparadas. Uma instrução preparada ou uma instrução parametrizada é usada para executar o mesmo comando repetidamente com alta eficiência e proteger contra injeções SQL.

Fluxo básico

A execução de uma instrução preparada consiste em dois estágios: preparação e execução. No estágio de preparação um modelo de instrução é enviado ao servidor de banco de dados. O servidor realiza uma verificação de sintaxe e inicializa os recursos internos para uso posterior.

O servidor MySQL suporta o uso de reservas de espaço posicionais anônimas com ponto de interrogação (`?`).

A preparação é seguida pela execução. Durante a execução o cliente vincula valores aos parâmetros e envia-os ao servidor. O servidor executa a instrução com os valores vinculados usando os recursos internos criados anteriormente.

Exemplo #1 Instrução preparada

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

/* Instrução não preparada */
$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label TEXT)");

/* Instrução preparada, estágio 1: prepara */
$stmt = $mysqli->prepare("INSERT INTO test(id, label) VALUES (?, ?)");

/* Instrução preparada, estágio 2: vincula e executa */
$id = 1;
$label = 'PHP';
$stmt->bind_param("is", $id, $label); // "is" significa que $id está vinculada como um inteiro e $label como uma string

$stmt->execute();
```

Execução repetida

Uma instrução preparada pode ser executada repetidas vezes. Em cada execução o valor atual da variável vinculada é avaliado e enviado ao servidor. A instrução não é analisada novamente. O modelo de instrução não é mais transferido ao servidor.

Exemplo #2 INSERT preparado uma vez, executado múltiplas vezes

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

/* Instrução não preparada */
$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label TEXT)");

/* Instrução preparada, estágio 1: prepara */
$stmt = $mysqli->prepare("INSERT INTO test(id, label) VALUES (?, ?)");

/* Instrução preparada, estágio 2: vincula e executa */
$stmt->bind_param("is", $id, $label); // "is" significa que $id está vinculada como um inteiro e $label como uma string

$data = [
    1 => 'PHP',
    2 => 'Java',
    3 => 'C++'
];
foreach ($data as $id => $label) {
    $stmt->execute();
}

$result = $mysqli->query('SELECT id, label FROM test');
var_dump($result->fetch_all(MYSQLI_ASSOC));
```

O exemplo acima produzirá:

```
array(3) {
  [0]=>
  array(2) {
    ["id"]=>
    string(1) "1"
    ["label"]=>
    string(3) "PHP"
  }
  [1]=>
  array(2) {
    ["id"]=>
    string(1) "2"
    ["label"]=>
    string(4) "Java"
  }
  [2]=>
  array(2) {
    ["id"]=>
    string(1) "3"
    ["label"]=>
    string(3) "C++"
  }
}
```

Cada instrução preparada ocupa recursos do servidor. Instruções devem ser fechadas explicitamente logo após o uso. Se não for fechada explicitamente, a instrução será fechada quando o manipulador da instrução for liberado pelo PHP.

Usar uma instrução preparada não é sempre a maneira mais eficiente de executar um comando. Uma instrução preparada executada apenas uma vez causa mais idas e voltas entre cliente e servidor do que uma não preparada. Este é o motivo pelo qual o `SELECT` não é executado como uma instrução preparada no exemplo acima.

Além disso, considere o uso da sintaxe SQL multi-INSERT do MySQL para inserção de dados. Para o exemplo, multi-INSERT requer menos idas e voltas entre o servidor e o cliente do que a instrução preparada mostrada acima.

Exemplo #3 Menos idas e voltas usando o multi-INSERT do SQL

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");

$values = [1, 2, 3, 4];

$stmt = $mysqli->prepare("INSERT INTO test(id) VALUES (?), (?), (?), (?)");
$stmt->bind_param('iiii', ...$values);
$stmt->execute();
```

Tipos de dados dos valores do conjunto de resultados

O Protocolo Cliente-Servidor do MySQL define um protocolo de transferência de dados diferente entre instruções preparadas e não preparadas. Instruções preparadas usam o chamado protocolo binário. O servidor MySQL envia os dados do conjunto de resultado como eles realmente são, em formato binário. Os resultados não são serializados em strings antes do envio. Bibliotecas cliente recebem os dados binários e tentam converter os valores para os tipos de dados apropriados do PHP. Por exemplo, resultados de uma coluna `INT` do SQL serão fornecidos como variáveis do tipo inteiro do PHP.

Exemplo #4 Tipos nativos de dados

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

/* Instrução não preparada */
$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label TEXT)");
$mysqli->query("INSERT INTO test(id, label) VALUES (1, 'PHP')");

$stmt = $mysqli->prepare("SELECT id, label FROM test WHERE id = 1");
$stmt->execute();
$result = $stmt->get_result();
$row = $result->fetch_assoc();
```

```
printf("id = %s (%s)\n", $row['id'], gettype($row['id']));
printf("label = %s (%s)\n", $row['label'], gettype($row['label']));
```

O exemplo acima produzirá:

```
id = 1 (integer)
label = PHP (string)
```

Este comportamento difere das instruções não preparadas. Por padrão, instruções não preparadas retornam todos os resultados como strings. Este padrão pode ser alterado com o uso de uma opção de conexão. Se a opção de conexão for usada, não haverá diferenças.

Recebendo resultados usando variáveis vinculadas

Resultados de instruções preparadas podem ser recebidos por vinculação a variáveis de saída, ou por requisição de um objeto mysqli_result.

Variáveis de saída devem ser vinculadas depois da execução da instrução. Uma variável deve ser vinculada a cada coluna do conjunto de resultados da instrução.

Exemplo #5 Vinculação de variável de saída

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

/* Instrução não preparada */
$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label TEXT)");
$mysqli->query("INSERT INTO test(id, label) VALUES (1, 'PHP')");

$stmt = $mysqli->prepare("SELECT id, label FROM test WHERE id = 1");
$stmt->execute();

$stmt->bind_result($out_id, $out_label);

while ($stmt->fetch()) {
    printf("id = %s (%s), label = %s (%s)\n", $out_id, gettype($out_id), $out_label,
gettype($out_label));
}
```

O exemplo acima produzirá:

```
id = 1 (integer), label = PHP (string)
```

Instruções preparadas retornam conjuntos de resultados sem buffer por padrão. Os resultados da instrução não são implicitamente recebidos e transformados do servidor para o cliente para buffer do lado do cliente. O conjunto de resultados consome recursos do servidor até que todos os resultados tenham sido recebidos pelo cliente. Por isso é recomendado consumir os resultados em tempo hábil. Se um cliente falha em receber todos os resultados, ou fecha a instrução antes de ter recebido todos os resultados, os dados têm que ser recebidos implicitamente pela `mysqli`.

Também é possível fazer buffer dos resultados de uma instrução preparada usando mysqli_stmt::store_result().

Recebendo resultados usando a interface mysqli_result

Ao invés de usar resultados vinculados, eles podem também ser recebidos através da interface mysqli_result. mysqli_stmt::get_result() retorna um conjunto de resultados em buffer.

Exemplo #6 Usando mysqli_result para receber resultados

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

/* Instrução não preparada */
$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label TEXT)");
$mysqli->query("INSERT INTO test(id, label) VALUES (1, 'PHP')");

$stmt = $mysqli->prepare("SELECT id, label FROM test WHERE id = 1");
$stmt->execute();

$result = $stmt->get_result();

var_dump($result->fetch_all(MYSQLI_ASSOC));
```

O exemplo acima produzirá:

```
array(1) {
  [0]=>
  array(2) {
    ["id"]=>
    int(1)
    ["label"]=>
    string(3) "PHP"
  }
}
```

Usar a interface mysqli_result oferece benefícios adicionais de flexibilidade de navegação pelos resultados no lado do cliente.

Exemplo #7 Conjunto de resultados com buffer para leitura flexível

```php
<?php
```

```php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

/* Instrução não preparada */
$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label TEXT)");
$mysqli->query("INSERT INTO test(id, label) VALUES (1, 'PHP'), (2, 'Java'), (3, 'C++')");

$stmt = $mysqli->prepare("SELECT id, label FROM test");
$stmt->execute();

$result = $stmt->get_result();

for ($row_no = $result->num_rows - 1; $row_no >= 0; $row_no--) {
    $result->data_seek($row_no);
    var_dump($result->fetch_assoc());
}
```

O exemplo acima produzirá:

```
array(2) {
  ["id"]=>
  int(3)
  ["label"]=>
  string(3) "C++"
}
array(2) {
  ["id"]=>
  int(2)
  ["label"]=>
  string(4) "Java"
}
array(2) {
  ["id"]=>
  int(1)
  ["label"]=>
  string(3) "PHP"
}
```

Escape e injeção SQL

Variáveis vinculadas são enviadas ao servidor separadas da consulta e, portanto, não podem interferir nela. O servidor usa estes valores diretamente no ponto de execução, depois que o modelo de instrução é analisado. Parâmetros vinculados não precisam ser escapados já que nunca são substituídos na string da consulta diretamente. Uma dica deve ser fornecida ao servidor sobre o tipo da variável vinculada, para criar a conversão apropriada. Consulte a página da função mysqli_stmt::bind_param() para mais informações.

Tal separação algumas vezes é considerada como o único recurso de segurança para evitar injeção SQL, mas o mesmo grau de segurança pode ser conseguido com instruções não preparadas, se todos os valores forem formatados corretamente. Deve ser observado que a formatação correta não é o mesmo que escapar e envolve mais lógica que um simples escape. Sendo assim, instruções preparadas são simplesmente uma abordagem mais conveniente e com menos propensão a erros a este aspecto de segurança de banco de dados.

Emulação de instrução preparada no lado do cliente

A API não inclui suporte para instrução preparada no lado do cliente.

Veja também

- mysqli::__construct()
- mysqli::query()
- mysqli::prepare()
- mysqli_stmt::prepare()
- mysqli_stmt::execute()
- mysqli_stmt::bind_param()
- mysqli_stmt::bind_result()

Procedimentos Armazenados

O banco de dados MySQL suporta procedimentos armazenados. Um procedimento armazenado é uma sub-rotina armazenada no catálogo do banco de dados. Aplicações podem chamar e executar o procedimento. A declaração `CALL` da linguagem SQL é usada para executar um procedimento armazenado.

Parâmetro

Procedimentos armazenados podem ter parâmetros `IN`, `INOUT` e `OUT`, dependendo da versão do MySQL. A interface mysqli não tem nenhuma ciência especial dos diferentes tipos de parâmetros.

Parâmetro IN

Parâmetros de entrada são providos pela declaração `CALL`. Certifique-se que os valores são corretamente escapados.

Exemplo #1 Chamando um procedimento armazenado

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");

$mysqli->query("DROP PROCEDURE IF EXISTS p");
$mysqli->query("CREATE PROCEDURE p(IN id_val INT) BEGIN INSERT INTO test(id) VALUES(id_val); END;");

$mysqli->query("CALL p(1)");

$result = $mysqli->query("SELECT id FROM test");

var_dump($result->fetch_assoc());
```

O exemplo acima produzirá:

```
array(1) {
  ["id"]=>
```

```
  string(1) "1"
}
```

Parâmetros INOUT/OUT

Os valores dos parâmetros `INOUT`/`OUT` são acessados com o uso de variáveis de sessão.

Exemplo #2 Uso de variáveis de sessão

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP PROCEDURE IF EXISTS p");
$mysqli->query('CREATE PROCEDURE p(OUT msg VARCHAR(50)) BEGIN SELECT "Hi!" INTO msg; END;');

$mysqli->query("SET @msg = ''");
$mysqli->query("CALL p(@msg)");

$result = $mysqli->query("SELECT @msg as _p_out");

$row = $result->fetch_assoc();
echo $row['_p_out'];
```

O exemplo acima produzirá:

```
Hi!
```

Desenvolvedores de aplicações e de frameworks podem ser capazes de fornecer uma API mais conveniente que use uma mistura de variáveis de sessão e inspeção de catálogos de banco de dados. Entretanto, deve ser observado o possível impacto em desempenho de uma solução customizada baseada em inspeção de catálogo.

Lidando com conjuntos de resultados

Procedimentos armazenados podem retornar conjuntos de resultados. Resultados retornados de um procedimento armazenado não podem ser recebidos corretamente usando mysqli::query(). A função mysqli::query() combina execução de instrução e recebimento do primeiro conjunto de dados em um conjunto de resultados com buffer, se houver. Porém, haverá conjuntos adicionais de resultados que estarão ocultos para o usuário, o que fará com que mysqli::query() falhe no retorno dos dados esperados.

Conjuntos de resultados retornados de um procedimento armazenado são recebidos usando-se mysqli::real_query() ou mysqli::multi_query(). Ambas as funções permitem receber qualquer número de conjuntos retornados pela instrução, como `CALL`. Falha no recebimento de todos os conjuntos retornados pelo procedimento armazenado causa um erro.

Exemplo #3 Recebendo resultados de procedimentos armazenados

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");
$mysqli->query("INSERT INTO test(id) VALUES (1), (2), (3)");

$mysqli->query("DROP PROCEDURE IF EXISTS p");
$mysqli->query('CREATE PROCEDURE p() READS SQL DATA BEGIN SELECT id FROM test; SELECT id + 1 FROM test;
END;');

$mysqli->multi_query("CALL p()");

do {
    if ($result = $mysqli->store_result()) {
        printf("---\n");
        var_dump($result->fetch_all());
        $result->free();
    }
} while ($mysqli->next_result());
```

O exemplo acima produzirá:

```
---
array(3) {
  [0]=>
  array(1) {
    [0]=>
    string(1) "1"
  }
  [1]=>
  array(1) {
    [0]=>
    string(1) "2"
  }
  [2]=>
  array(1) {
    [0]=>
    string(1) "3"
  }
}
---
array(3) {
  [0]=>
  array(1) {
    [0]=>
    string(1) "2"
  }
  [1]=>
  array(1) {
    [0]=>
    string(1) "3"
  }
  [2]=>
```

```
  array(1) {
    [0]=>
    string(1) "4"
  }
}
```

Uso de procedimentos armazenados

Nenhuma manipulação especial é necessária ao usar a interface de procedimentos armazenados para receber resultados do mesmo procedimento como no exemplo acima. As interfaces para procedimentos armazenados e não armazenados são similares. Deve ser observado que pode haver versões do servidor MYSQL que não suportem preparação da instrução `CALL` da linguagem SQL.

Exemplo #4 Procedimentos Armazenados e Instruções Preparadas

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");
$mysqli->query("INSERT INTO test(id) VALUES (1), (2), (3)");

$mysqli->query("DROP PROCEDURE IF EXISTS p");
$mysqli->query('CREATE PROCEDURE p() READS SQL DATA BEGIN SELECT id FROM test; SELECT id + 1 FROM test;
END;');

$stmt = $mysqli->prepare("CALL p()");

$stmt->execute();

do {
    if ($result = $stmt->get_result()) {
        printf("---\n");
        var_dump($result->fetch_all());
        $result->free();
    }
} while ($stmt->next_result());
```

O exemplo acima produzirá:

```
---
array(3) {
  [0]=>
  array(1) {
    [0]=>
    int(1)
  }
  [1]=>
  array(1) {
    [0]=>
    int(2)
```

```
  }
  [2]=>
  array(1) {
    [0]=>
    int(3)
  }
}
---
array(3) {
  [0]=>
  array(1) {
    [0]=>
    int(2)
  }
  [1]=>
  array(1) {
    [0]=>
    int(3)
  }
  [2]=>
  array(1) {
    [0]=>
    int(4)
  }
}
```

Obviamente, o uso da API 'bind' para recebimento de dados também é suportado.

Exemplo #5 Procedimentos Armazenados e Instruções Preparadas usando a API bind

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");
$mysqli->query("INSERT INTO test(id) VALUES (1), (2), (3)");

$mysqli->query("DROP PROCEDURE IF EXISTS p");
$mysqli->query('CREATE PROCEDURE p() READS SQL DATA BEGIN SELECT id FROM test; SELECT id + 1 FROM test;
END;');

$stmt = $mysqli->prepare("CALL p()");

$stmt->execute();

do {
    if ($stmt->store_result()) {
        $stmt->bind_result($id_out);
        while ($stmt->fetch()) {
            echo "id = $id_out\n";
        }
    }
} while ($stmt->next_result());
```

O exemplo acima produzirá:

```
id = 1
id = 2
id = 3
id = 2
id = 3
id = 4
```

Veja também

- mysqli::query()
- mysqli::multi_query()
- mysqli::next_result()
- mysqli::more_results()

Instruções Múltiplas

O MySQL opcionalmente permite múltiplas instruções em uma só string, mas isso requer uma manipulação especial.

Instruções ou consultas múltiplas devem ser executadas com mysqli::multi_query(). As instruções individuais da string são separadas por ponto-e-vírgula. Assim, todos os conjuntos de resultados retornados pelas instruções executadas precisam ser recebidas.

O servidor MySQL permite instruções que retornam dados e instruções que não retornam, em uma só instrução múltipla.

Exemplo #1 Instruções Múltiplas

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");

$sql = "SELECT COUNT(*) AS _num FROM test;
        INSERT INTO test(id) VALUES (1);
        SELECT COUNT(*) AS _num FROM test; ";

$mysqli->multi_query($sql);

do {
    if ($result = $mysqli->store_result()) {
        var_dump($result->fetch_all(MYSQLI_ASSOC));
        $result->free();
    }
} while ($mysqli->next_result());
```

O exemplo acima produzirá:

```
array(1) {
  [0]=>
  array(1) {
    ["_num"]=>
    string(1) "0"
  }
}
array(1) {
  [0]=>
  array(1) {
    ["_num"]=>
    string(1) "1"
  }
}
```

Considerações de Segurança

As funções da API mysqli::query() e mysqli::real_query() não definem uma opção de conexão necessária para ativar consultas múltiplas no servidor. Uma chamada extra da API é usada para instruções múltiplas para reduzir o estrago de ataques de injeção SQL acidentais. Um atacante pode tentar adicionar instruções como '`; DROP DATABASE mysql`' ou '`; SELECT SLEEP(999)`'. Se o atacante obtiver sucesso na adição de SQL ao string da instrução mas mysqli::multi_query() não for usada, o servidor não irá executar a instrução SQL maliciosa que foi injetada.

Exemplo #2 Injeção SQL

```php
<?php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");
$result = $mysqli->query("SELECT 1; DROP TABLE mysql.user");
?>
```

O exemplo acima produzirá:

```
PHP Fatal error:  Uncaught mysqli_sql_exception: You have an error in your SQL syntax;
check the manual that corresponds to your MySQL server version for the right syntax to
use near 'DROP TABLE mysql.user' at line 1
```

Instruções preparadas

O uso de instruções múltiplas com instruções preparadas não é suportado.

Veja também

- mysqli::query()
- mysqli::multi_query()
- mysqli::next_result()
- mysqli::more_results()

Suporte da API para transações

O servidor MySQL suporta transações dependendo do tipo de motor de armazenamento usado. Desde o MySQL 5.5, InnoDB é o motor padrão de armazenamento. InnoDB tem suporte completo a transações ACID.

Transações podem ser controladas usando-se chamadas por SQL ou pela API. É recomentado usar as chamadas via API para habilitar e desabilitar o modo `autocommit` e para enviar ou desfazer transações.

Exemplo #1 Configurando o modo `autocommit` com SQL e através da API

```php
<?php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

/* Recomendado: usando API para controlar configurações de transações */
$mysqli->autocommit(false);

/* Não será monitorado ou reconhecido pela replicação ou pelo plugin de balanceamento de carga */
$mysqli->query('SET AUTOCOMMIT = 0');
```

Pacotes de recursos opcionais, como plugins de replicação e de balanceamento de carga, podem facilmente monitorar chamadas da API. O plugin de replicação oferece balanceamento de carga com suporte a transação, se as transações forem controladas por chamadas da API. Balanceamento de carga com suporte a transação não estará disponível se instruções SQL forem usadas para configurar o modo `autocommit`, para enviar ou para desfazer transações.

Exemplo #2 Enviando e desfazendo transações

```php
<?php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");
$mysqli->autocommit(false); //desliga o modo de envio automático

$mysqli->query("INSERT INTO test(id) VALUES (1)");
$mysqli->rollback(); //desfaz a inserção acima

$mysqli->query("INSERT INTO test(id) VALUES (2)");
$mysqli->commit(); //envia (confirma) a inserção acima
```

Observe que o servidor MySQL não consegue desfazer qualquer instrução. Algumas instruções causam um envio implícito.

Veja também

- mysqli::autocommit()
- mysqli::begin_transaction()
- mysqli::commit()
- mysqli::rollback()

Metadados

Um conjunto de resultados MySQL contém metadados. Os metadados descrevem as colunas encontradas no resultado. Todos os metadados enviados pelo MySQL podem ser acessados através da interface `mysqli`. A extensão não altera, ou algumas vezes praticamente não altera, a informação que recebe. As diferenças entre as versões do servidor MySQL não estão alinhadas.

Metadados são acessados através da interface mysqli_result.

Exemplo #1 Acessando metadados do conjunto de resultados

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$result = $mysqli->query("SELECT 1 AS _one, 'Hello' AS _two FROM DUAL");
var_dump($result->fetch_fields());
```

O exemplo acima produzirá:

```
array(2) {
  [0]=>
  object(stdClass)#3 (13) {
    ["name"]=>
    string(4) "_one"
    ["orgname"]=>
    string(0) ""
    ["table"]=>
    string(0) ""
    ["orgtable"]=>
    string(0) ""
    ["def"]=>
    string(0) ""
    ["db"]=>
    string(0) ""
    ["catalog"]=>
    string(3) "def"
    ["max_length"]=>
    int(1)
    ["length"]=>
    int(1)
    ["charsetnr"]=>
    int(63)
    ["flags"]=>
    int(32897)
    ["type"]=>
    int(8)
    ["decimals"]=>
    int(0)
  }
  [1]=>
  object(stdClass)#4 (13) {
    ["name"]=>
    string(4) "_two"
    ["orgname"]=>
    string(0) ""
    ["table"]=>
    string(0) ""
    ["orgtable"]=>
    string(0) ""
    ["def"]=>
    string(0) ""
```

```
    ["db"]=>
    string(0) ""
    ["catalog"]=>
    string(3) "def"
    ["max_length"]=>
    int(5)
    ["length"]=>
    int(5)
    ["charsetnr"]=>
    int(8)
    ["flags"]=>
    int(1)
    ["type"]=>
    int(253)
    ["decimals"]=>
    int(31)
  }
}
```

Instruções preparadas

Metadados de conjuntos de resultados criados por instruções preparadas são acessados da mesma forma. Um manipulador mysqli_result adequado é retornado pelo método mysqli_stmt::result_metadata().

Exemplo #2 Metadados de instruções preparadas

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$stmt = $mysqli->prepare("SELECT 1 AS _one, 'Hello' AS _two FROM DUAL");
$stmt->execute();
$result = $stmt->result_metadata();
var_dump($result->fetch_fields());
```

Veja também

- mysqli::query()
- mysqli_result::fetch_fields()

Instalação/Configuração

Índice

Dependências

Para ter esta funções disponíveis, deve-se compilar o PHP com suporte para a extensão mysqli.

MySQL 8

Ao executar o PHP em versão anterior a 7.1.16, ou PHP 7.2 anterior a 7.2.4, defina o plugin padrão de senha do Servidor MySQL 8 para *mysql_native_password* ou surgirão erros parecidos com *The server requested authentication method unknown to the client [caching_sha2_password]* mesmo quando *caching_sha2_password* não for usado.

Isto acontece porque o padrão do MySQL 8 é caching_sha2_password, um plugin que não é reconhecido pelas versões anteriores do PHP (mysqlnd). Para resolver, altere para a configuração `default_authentication_plugin=mysql_native_password` no arquivo *my.cnf*. O plugin *caching_sha2_password* é totalmente suportado a partir do PHP 7.4.4. Para versões anteriores do PHP, a extensão mysql_xdevapi suporta o plugin.

Instalação

A extensão `mysqli` foi introduzida com o PHP versão 5.0.0. O MySQL Native Driver foi incluído no PHP na versão 5.3.0.

Instalação no Linux

As distribuições comuns do Unix incluem versões binárias do PHP que podem ser instaladas. Embora estas versões binárias sejam tipicamente construídas com suporte às extensões MySQL, as próprias bibliotecas das extensões podem precisar de pacote adicional para instalação. Verifique o gerenciador de pacote que vem com sua distribuição escolhida para disponibilidade.

Por exemplo, no Ubuntu o pacote `php5-mysql` instala as extensões do PHP ext/mysql, ext/mysqli e pdo_mysql. No CentOS, o pacote `php-mysql` também instala estas três extensões PHP.

Alternativamente, pode-se compilar esta extensão. Construir o PHP a partir do código-fonte permite que se especifique as extensões MySQL que se deseja usar, bem como pode-se escolher a biblioteca cliente para cada extensão.

O MySQL Native Driver é a opção de biblioteca cliente recomendada, e a partir do PHP 8.2.0 a única opção, pois resulta em melhor desempenho e dá acesso a recursos não disponíveis ao usar a Biblioteca Cliente MySQL. Consulte a página O que é o Driver Nativo do MySQL? para uma visão breve das vantagens do Driver Nativo MySQL.

`/path/to/mysql_config` representa a localização do programa `mysql_config` que vem com o Servidor MySQL.

Matriz de suporte ao mysqli em tempo de compilação

Versão do PHP	Padrão	Opções de Configuração: mysqlnd	Opções de Configuração: `libmysqlclient`	Registro de Alterações
8.2.0 e posteriores	mysqlnd	**--with-mysqli**	libmysqlclient não é suportada	mysqlnd é a única opção
5.4.0 a 8.1.x	mysqlnd	**--with-mysqli**	**--with-mysqli=/path/to/mysql_config**	mysqlnd é o padrão
5.3.x	libmysqlclient	**--with-mysqli=mysqlnd**	**--with-mysqli=/path/to/mysql_config**	mysqlnd é suportada
5.0.x, 5.1.x, 5.2.x	libmysqlclient	Não Disponível	**--with-mysqli=/path/to/mysql_config**	mysqlnd não é suportada

Instalação em Sistemas Windows

No Windows, a DLL *php_mysqli.dll* deve ser habilitada no *php.ini*.

Como em qualquer habilitação de extensão PHP (como *php_mysqli.dll*), a diretiva PHP extension_dir seve ser definida para o diretório onde as extensões do PHP estão localizadas. Consulte também as Instruções de Instalação Manual no Windows. Um exemplo de valor para `extension_dir` é *c:\php\ext*.

> **Nota** :
>
> Se ao iniciar o servidor web um erro parecido com este ocorrer: `"Unable to load dynamic library './php_mysqli.dll'"`, é porque *php_mysqli.dll* não pôde ser encontrada no sistema.

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de configuração do MySQLi

Nome	Padrão	Modificável	Registro de Alterações
mysqli.allow_local_infile	"0"	`INI_SYSTEM`	Antes do PHP 7.2.16 e 7.3.3 o padrão era "1".
mysqli.local_infile_directory		`INI_SYSTEM`	Disponível desde o PHP 8.1.0.
mysqli.allow_persistent	"1"	`INI_SYSTEM`	
mysqli.max_persistent	"-1"	`INI_SYSTEM`	
mysqli.max_links	"-1"	`INI_SYSTEM`	
mysqli.default_port	"3306"	`INI_ALL`	
mysqli.default_socket	NULL	`INI_ALL`	
mysqli.default_host	NULL	`INI_ALL`	
mysqli.default_user	NULL	`INI_ALL`	
mysqli.default_pw	NULL	`INI_ALL`	
mysqli.reconnect	"0"	`INI_SYSTEM`	Removido no PHP 8.2.0
mysqli.rollback_on_cached_plink	"0"	`INI_SYSTEM`	

Para maiores detalhes e definições das constantes INI_* acima, veja o capítulo sobre mudanças de configuração.

Aqui está uma breve explicação das diretivas de configuração.

`mysqli.allow_local_infile` bool

Permite acessar, direto do PHP, arquivos locais com comandos LOAD DATA

`mysqli.local_infile_directory` string

Permite restringir o carregamento de LOCAL DATA para arquivos localizados neste diretório.

`mysqli.allow_persistent` bool

Habilita a possibilidade de criar conexões persistentes usando mysqli_connect().

`mysqli.max_persistent` int

Número máximo de conexões simultâneas que podem ser estabelecidas. Definir como 0 para ilimitadas.

`mysqli.max_links` int

O número máximo de conexões por processo.

`mysqli.default_port` int

O número da porta TCP padrão para usar ao conectar ao servidor de banco de dados se não for especificada outra porta. Se não for especificado um padrão, então a porta será obtida da variável de ambiênte `MYSQL_TCP_PORT`, a entrada `mysql-tcp` em */etc/services* ou a constante em tempo de compilação `MYSQL_PORT`, nesta ordem. Em Win32 será usada apenas a constante `MYSQL_PORT`.

`mysqli.default_socket` string

O nome padrão do socket para usar ao conectar com um servidor de banco de dados local, se não for especificado outro nome de socket.

`mysqli.default_host` string

O servidor padrão para usar ao conectar com o servidor de banco de dados se não for especificado um servidor. Não se aplica em

`mysqli.default_user` string

O nome de usuário padrão para usar ao conectar com o servidor de banco de dados. Não se aplica em

`mysqli.default_pw` string

A senha padrão para usar ao conectar ao servidor de banco de dados se não for especificado outra senha. Não se aplica em

`mysqli.reconnect` int

Reconectar automaticamente ao perder a conexão.

> **Nota** :
>
> Esta configuração do *php.ini* é ignorada pelo driver mysqlnd, e foi removida no PHP 8.2.0.

`mysqli.rollback_on_cached_plink` bool

Se essa opção for ativada, o fechamento de uma conexão persistente fará o rollback de todas as transações pendentes dessa conexão antes que ela seja colocada de volta no pool de conexão persistente. Caso contrário, as transações pendentes serão revertidas apenas quando a conexão for reutilizada ou quando for realmente fechada.

Usuários não podem modificar `MYSQL_OPT_READ_TIMEOUT` via chamada de API ou configuração em tempo de execução. Note que se fosse possível haveria diferenças entre como a `libmysqlclient` e os fluxos iriam interpretar o valor de `MYSQL_OPT_READ_TIMEOUT`.

A extensão mysqli e conexões persistentes

A ideia por trás das conexões persistentes é que uma conexão entre um processo cliente e um banco de dados pode ser reutilizada por um processo cliente, em vez de ser criada e destruída várias vezes. Isso reduz a sobrecarga de criar novas conexões sempre que uma é necessária, pois as conexões não utilizadas são armazenadas em cache e estão prontas para serem reutilizadas.

Ao contrário da extensão mysql, mysqli não fornece uma função separada para abrir conexões persistentes. Para abrir uma conexão persistente, você deve preceder `p:` ao nome do host ao conectar.

O problema com conexões persistentes é que elas podem ser deixadas em estados imprevisíveis pelos clientes. Por exemplo, um bloqueio de tabela pode ser ativado antes que um cliente seja encerrado inesperadamente. Um

novo processo cliente reutilizando essa conexão persistente obterá a conexão “como está”. Qualquer limpeza precisaria ser feita pelo novo processo cliente antes que ele pudesse fazer bom uso da conexão] persistente, aumentando a carga do programador.

A conexão persistente da extensão `mysqli`, no entanto, fornece código de manipulação de limpeza integrado. A limpeza realizada pelo `mysqli` inclui:

- Reverter transações ativas
- Fechar e descartar tabelas temporárias
- Desbloquear tabelas
- Redefinir variáveis de sessão
- Fechar declarações preparadas (sempre acontece com PHP)
- Fechar manipulador
- Liberar bloqueios adquiridos com **GET_LOCK()**

Isso garante que as conexões persistentes estejam em um estado limpo ao retornar do pool de conexões, antes que o processo do cliente as use.

A extensão `mysqli` faz essa limpeza chamando automaticamente a função C-API `mysql_change_user()`.

O recurso de limpeza automática tem vantagens e desvantagens. A vantagem é que o programador não precisa mais se preocupar em adicionar código de limpeza, pois ele é chamado automaticamente. No entanto, a desvantagem é que o código pode ser um *pouco* mais lento, pois o código para executar a limpeza precisa ser executado sempre que uma conexão é retornada do pool de conexões.

É possível desligar o código de limpeza automática, compilando o PHP com **`MYSQLI_NO_CHANGE_USER_ON_PCONNECT`** definido.

> **Nota** :
>
> A extensão `mysqli` suporta conexões persistentes ao usar MySQL Native Driver ou MySQL Client Library.

Constantes predefinidas

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

`MYSQLI_READ_DEFAULT_GROUP` (int)

Lê opções do grupo nomeado em *my.cnf* ou no arquivo especificado com **`MYSQLI_READ_DEFAULT_FILE`**.

`MYSQLI_READ_DEFAULT_FILE` (int)

Lê opções do arquivo de opções informado ao invés de *my.cnf*.

`MYSQLI_OPT_CAN_HANDLE_EXPIRED_PASSWORDS` (int)

Indica ao servidor que o cliente consegue lidar com o modo "caixa de areia" para senhas expiradas. Pode ser usada com mysqli_options().

MYSQLI_OPT_LOAD_DATA_LOCAL_DIR (int)

Se habilitada, esta opção especifica o diretório a partid do qual o carregamento de dados `LOCAL` no lado do cliente é permitido em instruções `LOAD DATA LOCAL`.

MYSQLI_OPT_CONNECT_TIMEOUT (int)

Limite de tempo de conexão em segundos.

MYSQLI_OPT_READ_TIMEOUT (int)

Limite de tempo do resultado da execução do comando, em segundos. Disponível a partir do PHP 7.2.0.

MYSQLI_OPT_LOCAL_INFILE (int)

Habilita o comando `LOAD LOCAL INFILE`.

MYSQLI_OPT_INT_AND_FLOAT_NATIVE (int)

Converte colunas de inteiros e floats de volta para números PHP. Válido apenas para mysqlnd.

MYSQLI_OPT_NET_CMD_BUFFER_SIZE (int)

O tamanho do buffer interno de comando/rede. Válido apenas para mysqlnd.

MYSQLI_OPT_NET_READ_BUFFER_SIZE (int)

Máximo tamanho de porção de leitura em bytes ao ler o corpo de um pacote de comando MySQL. Válido apenas para mysqlnd.

MYSQLI_OPT_SSL_VERIFY_SERVER_CERT (int)

MYSQLI_INIT_COMMAND (int)

Comando a executar ao se conectar ao servidor MySQL. Será automaticamente re-executado na reconexão.

MYSQLI_CLIENT_CAN_HANDLE_EXPIRED_PASSWORDS (int)

Indica ao servidor que o cliente consegue lidaro com o modo "caixa de areia" para senhas expiradas. Pode ser usada com mysqli_real_connect().

MYSQLI_CLIENT_FOUND_ROWS (int)

Retorna o número de linhas correspondentes, e não o número de linhas afetadas.

MYSQLI_CLIENT_SSL_VERIFY_SERVER_CERT (int)

Verifica o certificado do servidor.

MYSQLI_CLIENT_SSL (int)

Usa SSL (protocolo criptografado). Esta opção não deve ser definida por aplicações; ela é definida internamente na biblioteca cliente MySQL.

MYSQLI_CLIENT_COMPRESS (int)

Usa protocolo de compressão.

MYSQLI_CLIENT_INTERACTIVE (int)

Permite `interactive_timeout` segundos (ao invés de `wait_timeout` segundos) de inatividade antes de fechar a conexão. A variável `wait_timeout` da sessão do cliente será definida para o valor da variável `interactive_timeout` da sessão.

MYSQLI_CLIENT_IGNORE_SPACE (int)

Permite espaços depois de nomes de função. Considera todos os nomes de função como palavras reservadas.

MYSQLI_CLIENT_NO_SCHEMA (int)

Não permite a sintaxe `db_name.tbl_name.col_name`.

MYSQLI_CLIENT_MULTI_QUERIES

Permite múltiplas consultas separadas por ponto-e-vírgula em uma única chamada a mysqli_query().

MYSQLI_STORE_RESULT (int)

Para uso em conjuntos de resultados com buffer. Tem o valor de `0`.

MYSQLI_STORE_RESULT_COPY_DATA (int)

A partir do PHP 8.1, esta constante não tem mais efeito. Antes do PHP 8.1, esta constante é usada para copiar resultados do buffer interno do `mysqlnd` para as variáveis PHP buscadas. Por padrão, o `mysqlnd` usará uma lógica de referência para evitar de copiar e duplicar os resultados mantidos em memória. Para certos resultados, por exemplo para conjuntos com muitas linhas pequenas, o método de cópia pode reduzir o uso total de memória porque as variáveis do PHP que mantêm o resultado podem ser liberadas mais cedo. Disponível somente com `mysqlnd`. Descontinuada a partir do PHP 8.4.0.

MYSQLI_USE_RESULT (int)

Para uso em conjuntos de resultados sem buffer. Tem o valor de `1`.

MYSQLI_ASSOC (int)

Colunas são retornadas no array com os nomes dos campos nas chaves.

MYSQLI_NUM (int)

Colunas são retornadas no array com índices enumerados.

MYSQLI_BOTH (int)

Colunas são retornadas no array com índice tanto numérico quanto com nomes dos campos.

MYSQLI_NOT_NULL_FLAG (int)

Indica que um campo é definido como `NOT NULL`.

MYSQLI_PRI_KEY_FLAG (int)

O campo é parte de um índice primário.

MYSQLI_UNIQUE_KEY_FLAG (int)

O campo é parte de um índice exclusivo.

MYSQLI_MULTIPLE_KEY_FLAG (int)

O campo é parte de um índice.

MYSQLI_BLOB_FLAG (int)

O campo é definido como `BLOB`.

MYSQLI_UNSIGNED_FLAG (int)

O campo é definido como `UNSIGNED`.

MYSQLI_ZEROFILL_FLAG (int)

O campo é definido como `ZEROFILL`.

MYSQLI_AUTO_INCREMENT_FLAG (int)

O campo é definido como `AUTO_INCREMENT`.

MYSQLI_TIMESTAMP_FLAG (int)

O campo é definido como `TIMESTAMP`.

MYSQLI_SET_FLAG (int)

O campo é definido como `SET`.

MYSQLI_NUM_FLAG (int)

O campo é definido como `NUMERIC`.

MYSQLI_PART_KEY_FLAG (int)

O campo é parte de um multi-índice.

MYSQLI_GROUP_FLAG (int)

O campo é parte de `GROUP BY`.

MYSQLI_NO_DEFAULT_VALUE_FLAG (int)

Uma coluna não tem cláusula `DEFAULT` em sua definição. Isto não se aplica a colunas `NULL` ou `AUTO_INCREMENT` porque tais colunas têm um valor padrão de `NULL` e um valor padrão implícito, respectivamente.

MYSQLI_TYPE_DECIMAL (int)

O campo é definido como `DECIMAL`.

MYSQLI_TYPE_NEWDECIMAL (int)

Campo de precisão matemática `DECIMAL` ou `NUMERIC`.

MYSQLI_TYPE_BIT (int)

O campo é definido como `BIT`.

MYSQLI_TYPE_TINY (int)

O campo é definido como `TINYINT`.

MYSQLI_TYPE_SHORT (int)

O campo é definido como `SMALLINT`.

MYSQLI_TYPE_LONG (int)

O campo é definido como `INT`.

MYSQLI_TYPE_FLOAT (int)

O campo é definido como `FLOAT`.

MYSQLI_TYPE_DOUBLE (int)

O campo é definido como `DOUBLE`.

MYSQLI_TYPE_NULL (int)

O campo é definido como `DEFAULT NULL`.

MYSQLI_TYPE_TIMESTAMP (int)

O campo é definido como `TIMESTAMP`.

MYSQLI_TYPE_LONGLONG (int)

O campo é definido como `BIGINT`.

MYSQLI_TYPE_INT24 (int)

O campo é definido como `MEDIUMINT`.

MYSQLI_TYPE_DATE (int)

O campo é definido como `DATE`.

MYSQLI_TYPE_TIME (int)

O campo é definido como `TIME`.

MYSQLI_TYPE_DATETIME (int)

O campo é definido como `DATETIME`.

MYSQLI_TYPE_YEAR (int)

O campo é definido como `YEAR`.

MYSQLI_TYPE_NEWDATE (int)

O campo é definido como `DATE`.

MYSQLI_TYPE_INTERVAL (int)

Sinônimo de **MYSQLI_TYPE_ENUM**. Removida a partir do PHP 8.4.0.

MYSQLI_TYPE_ENUM (int)

O campo é definido como ENUM.

MYSQLI_TYPE_SET (int)

O campo é definido como SET.

MYSQLI_TYPE_TINY_BLOB (int)

O campo é definido como TINYBLOB.

MYSQLI_TYPE_MEDIUM_BLOB (int)

O campo é definido como MEDIUMBLOB.

MYSQLI_TYPE_LONG_BLOB (int)

O campo é definido como LONGBLOB.

MYSQLI_TYPE_BLOB (int)

O campo é definido como BLOB.

MYSQLI_TYPE_VAR_STRING (int)

O campo é definido como VARCHAR.

MYSQLI_TYPE_STRING (int)

O campo é definido como CHAR ou BINARY.

MYSQLI_TYPE_CHAR (int)

O campo é definido como TINYINT. Para CHAR, veja MYSQLI_TYPE_STRING.

MYSQLI_TYPE_GEOMETRY (int)

O campo é definido como GEOMETRY

MYSQLI_TYPE_JSON (int)

O campo é definido como JSON. Válido apenas para mysqlnd e MySQL 5.7.8 ou acima.

MYSQLI_TYPE_VECTOR (int)

O campo é definido como VECTOR. Disponível a partir do PHP 8.4.0, para MySQL 9.0 e versões posteriores.

MYSQLI_NEED_DATA

Há mais dados disponíveis para vincular a variáveis.

MYSQLI_ENUM_FLAG (int)

O campo é definido como ENUM.

MYSQLI_BINARY_FLAG (int)

O campo é definido como BINARY.

MYSQLI_CURSOR_TYPE_FOR_UPDATE (int)

Removida a partir do PHP 8.4.0.

MYSQLI_CURSOR_TYPE_NO_CURSOR (int)

MYSQLI_CURSOR_TYPE_READ_ONLY (int)

MYSQLI_CURSOR_TYPE_SCROLLABLE (int)

Removida a partir do PHP 8.4.0.

MYSQLI_STMT_ATTR_CURSOR_TYPE (int)

MYSQLI_STMT_ATTR_PREFETCH_ROWS (int)

Removida a partir do PHP 8.4.0.

MYSQLI_STMT_ATTR_UPDATE_MAX_LENGTH (int)

MYSQLI_SET_CHARSET_NAME (int)

MYSQLI_REPORT_INDEX (int)

Reporta se nenhum índice ou um índice ruim foi usado em uma consulta.

MYSQLI_REPORT_ERROR (int)

Reporta erros de chamadas a funções mysqli.

MYSQLI_REPORT_STRICT (int)

Lança uma exceção `mysqli_sql_exception` para erros ao invés de gerar alertas.

MYSQLI_REPORT_ALL (int)

Define todas as opções ligadas (reportar tudo).

MYSQLI_REPORT_OFF (int)

Desliga a reportagem de erros.

MYSQLI_DEBUG_TRACE_ENABLED (int)

É definido para 1 se a funcionalidade mysqli_debug() estiver habilitada.

MYSQLI_SERVER_PUBLIC_KEY (int)

MYSQLI_REFRESH_GRANT (int)

Atualiza a tabela de permissões. Descontinuada a partir do PHP 8.4.0.

MYSQLI_REFRESH_LOG (int)

Descarrega os registros, como na execução da declaração SQL `FLUSH LOGS`. Descontinuada a partir do PHP 8.4.0.

MYSQLI_REFRESH_TABLES (int)

Descarrega o cache de tabelas, como na execução da declaração SQL FLUSH TABLES. Descontinuada a partir do PHP 8.4.0.

MYSQLI_REFRESH_HOSTS (int)

Descarrega o cache de servidores, como na execução da declaração SQL FLUSH HOSTS. Descontinuada a partir do PHP 8.4.0.

MYSQLI_REFRESH_REPLICA (int)

Apelido da constante **MYSQLI_REFRESH_SLAVE**. Disponível a partir do PHP 8.1.0. Descontinuada a partir do PHP 8.4.0.

MYSQLI_REFRESH_STATUS (int)

Redefine as variáveis de estado, como na execução da declaração SQL FLUSH STATUS. Descontinuada a partir do PHP 8.4.0.

MYSQLI_REFRESH_THREADS (int)

Descarrega o cache de threads. Descontinuada a partir do PHP 8.4.0.

MYSQLI_REFRESH_SLAVE (int)

Em um servidor de replicação escravo: redefine a informação do servidor principal, e reinicia o escravo. É como executar a declaração SQL RESET SLAVE. Descontinuada a partir do PHP 8.4.0.

MYSQLI_REFRESH_MASTER (int)

Em um servidor de replicação principal: remove os arquivos de registro binários listados no índice de registro binário, e trunca o arquivo de índice. É como executar a declaração SQL RESET MASTER. Descontinuada a partir do PHP 8.4.0.

MYSQLI_REFRESH_BACKUP_LOG (int)

Fecha e reabre os arquivos de registro reservas. Descontinuada a partir do PHP 8.4.0.

MYSQLI_TRANS_COR_AND_CHAIN (int)

Adiciona "AND CHAIN" a mysqli_commit() ou mysqli_rollback().

MYSQLI_TRANS_COR_AND_NO_CHAIN (int)

Adiciona "AND NO CHAIN" a mysqli_commit() ou mysqli_rollback().

MYSQLI_TRANS_COR_RELEASE (int)

Adiciona "RELEASE" a mysqli_commit() ou mysqli_rollback().

MYSQLI_TRANS_COR_NO_RELEASE (int)

Adiciona "NO RELEASE" a mysqli_commit() ou mysqli_rollback().

MYSQLI_TRANS_START_READ_ONLY (int)

Inicia a transação como em "START TRANSACTION READ ONLY" com mysqli_begin_transaction().

MYSQLI_TRANS_START_READ_WRITE (int)

Inicia a transação como em "START TRANSACTION READ WRITE" com mysqli_begin_transaction().

MYSQLI_TRANS_START_WITH_CONSISTENT_SNAPSHOT (int)

Inicia a transação como em "START TRANSACTION WITH CONSISTENT SNAPSHOT".

MYSQLI_CLIENT_SSL_DONT_VERIFY_SERVER_CERT (int)

Requer MySQL 5.6.5 e acima.

MYSQLI_IS_MARIADB (bool)

Informa se a extensão mysqli foi construída com uma biblioteca cliente MariaDB. Disponível a partir do PHP 8.1.2.

MYSQLI_ASYNC (int)

A consulta é realizada de forma assíncrona e nenhum resultado é retornado imediatamente. Disponível somente com `mysqlnd`.

MYSQLI_ON_UPDATE_NOW_FLAG (int)

Se um campo for atualizado é obtido o valor do horário atual.

Notas

Algumas notas de implementação:

1. Foi adicionado suporte para `MYSQL_TYPE_GEOMETRY` à extensão MySQLi no PHP 5.3.
2. Observe que existem diferentes implementações internas em `libmysqlclient` e `mysqlnd` para lidar com colunas do tipo `MYSQL_TYPE_GEOMETRY`. De um modo geral, o `mysqlnd` irá alocar significativamente menos memória. Por exemplo, se houver uma coluna `POINT` em um conjunto de resultados, `libmysqlclient` pode pré-alocar até 4 GB de RAM, embora sejam necessários menos de 50 bytes para manter uma coluna `POINT` na memória. A alocação de memória é muito menor, menos de 50 bytes, se estiver usando `mysqlnd`.

Resumo da Função de Extensão do MySQLi

Resumo dos métodos mysqli

Classe mysqli			
Interface POO	**Interface Procedural**	**Apelido (não use)**	**Descrição**
Propriedades			
$mysqli::affected_rows	mysqli_affected_rows()	N/A	Obtém o número de linhas afetadas em uma operação MySQL anterior
$mysqli::client_info	mysqli_get_client_info()	N/A	Retorna a versão do cliente MySQL como uma string
$mysqli::client_version	mysqli_get_client_version()	N/A	Retorna as informações da versão do cliente MySQL

Classe mysqli			
Interface POO	**Interface Procedural**	**Apelido (não use)**	**Descrição**
			como um número inteiro
$mysqli::connect_errno	mysqli_connect_errno()	N/A	Retorna o código de erro da última chamada de conexão
$mysqli::connect_error	mysqli_connect_error()	N/A	Retorna uma descrição de string do último erro de conexão
$mysqli::errno	mysqli_errno()	N/A	Retorna o código de erro da chamada de função mais recente
$mysqli::error	mysqli_error()	N/A	Retorna uma descrição de string do último erro
$mysqli::field_count	mysqli_field_count()	N/A	Retorna o número de colunas da consulta mais recente
$mysqli::host_info	mysqli_get_host_info()	N/A	Retorna uma string representando o tipo de conexão usada
$mysqli::protocol_version	mysqli_get_proto_info()	N/A	Retorna a versão do protocolo MySQL usado
$mysqli::server_info	mysqli_get_server_info()	N/A	Retorna a versão do servidor MySQL
$mysqli::server_version	mysqli_get_server_version()	N/A	Retorna a versão do servidor MySQL como um inteiro
$mysqli::info	mysqli_info()	N/A	Recupera informações sobre a consulta executada mais recentemente
$mysqli::insert_id	mysqli_insert_id()	N/A	Retorna o id gerado automaticamente usado na última consulta
$mysqli::sqlstate	mysqli_sqlstate()	N/A	Retorna o erro SQLSTATE da operação MySQL anterior
$mysqli::warning_count	mysqli_warning_count()	N/A	Retorna o número de avisos da última

Classe mysqli			
Interface POO	**Interface Procedural**	**Apelido (não use)**	**Descrição**
			consulta para o link fornecido
Métodos			
mysqli::autocommit()	mysqli_autocommit()	N/A	Ativa ou desativa modificações de banco de dados de confirmação automática
mysqli::change_user()	mysqli_change_user()	N/A	Altera o usuário da conexão de banco de dados especificada
mysqli::character_set_name()	mysqli_character_set_name()	N/A	Retorna o conjunto de caracteres padrão para a conexão do banco de dados
mysqli::close()	mysqli_close()	N/A	Fecha uma conexão de banco de dados aberta anteriormente
mysqli::commit()	mysqli_commit()	N/A	Confirma a transação atual
mysqli::__construct()	mysqli_connect()	N/A	Abra uma nova conexão com o servidor MySQL [Nota: método estático (ou seja, classe)]
mysqli::debug()	mysqli_debug()	N/A	Executa operações de depuração
mysqli::dump_debug_info()	mysqli_dump_debug_info()	N/A	Despejar informações de depuração no log
mysqli::get_charset()	mysqli_get_charset()	N/A	Retorna um objeto de conjunto de caracteres
mysqli::get_connection_stats()	mysqli_get_connection_stats()	N/A	Retorna estatísticas de conexão do cliente. Disponível apenas com o mysqlnd.
mysqli::get_client_info()	mysqli_get_client_info()	N/A	Retorna a versão do cliente MySQL como uma string
N/A	mysqli_get_client_stats()	N/A	Retorna as estatísticas do cliente por processo. Disponível

Classe mysqli			
Interface POO	**Interface Procedural**	**Apelido (não use)**	**Descrição**
			apenas com o mysqlnd.
mysqli::get_server_info()	mysqli_get_server_info()	N/A	Retorna uma string representando a versão do servidor MySQL ao qual a extensão MySQLi está conectada
mysqli::get_warnings()	mysqli_get_warnings()	N/A	Retorna o resultado de SHOW WARNINGS
mysqli::init()	mysqli_init()	N/A	Inicializa o MySQLi e retorna um objeto para uso com mysqli_real_connect.
mysqli::kill()	mysqli_kill()	N/A	Pede ao servidor para matar um encadeamento do MySQL
mysqli::more_results()	mysqli_more_results()	N/A	Verifica se há mais resultados de consulta de uma consulta múltipla
mysqli::multi_query()	mysqli_multi_query()	N/A	Executa uma consulta no banco de dados
mysqli::next_result()	mysqli_next_result()	N/A	Prepara o próximo resultado da multi_query
mysqli::options()	mysqli_options()	mysqli_set_opt()	Define opções
mysqli::ping()	mysqli_ping()	N/A	Faz ping em uma conexão de servidor ou tenta reconectar se a conexão cair
mysqli::prepare()	mysqli_prepare()	N/A	Prepara uma instrução SQL para execução
mysqli::query()	mysqli_query()	N/A	Executa uma consulta no banco de dados
mysqli::real_connect()	mysqli_real_connect()	N/A	Abre uma conexão com um servidor mysql
mysqli::real_escape_string(), mysqli::escape_string()	mysqli_real_escape_string()	mysqli_escape_string()	Escapa caracteres especiais em uma string para uso em uma instrução SQL,

Classe mysqli			
Interface POO	**Interface Procedural**	**Apelido (não use)**	**Descrição**
			levando em consideração o conjunto de caracteres atual da conexão
mysqli::real_query()	mysqli_real_query()	N/A	Executa uma consulta SQL
mysqli::refresh()	mysqli_refresh()	N/A	Libera tabelas ou caches ou redefine as informações do servidor de replicação
mysqli::rollback()	mysqli_rollback()	N/A	Reverte a transação atual
mysqli::select_db()	mysqli_select_db()	N/A	Seleciona o banco de dados padrão para consultas de banco de dados
mysqli::set_charset()	mysqli_set_charset()	N/A	Define o conjunto de caracteres do cliente padrão
mysqli::ssl_set()	mysqli_ssl_set()	N/A	Usado para estabelecer conexões seguras usando SSL
mysqli::stat()	mysqli_stat()	N/A	Obtém o status atual do sistema
mysqli::stmt_init()	mysqli_stmt_init()	N/A	Inicializa uma instrução e retorna um objeto para uso com mysqli_stmt_prepare
mysqli::store_result()	mysqli_store_result()	N/A	Transfere um conjunto de resultados da última consulta
mysqli::thread_id()	mysqli_thread_id()	N/A	Retorna o ID do thread para a conexão atual
mysqli::thread_safe()	mysqli_thread_safe()	N/A	Retorna se a segurança de thread é dada ou não
mysqli::use_result()	mysqli_use_result()	N/A	Inicia uma recuperação de conjunto de resultados

Resumo dos métodos mysqli_stmt

MySQL_STMT			
Interface POO	**Interface Procedural**	**Apelido (não use)**	**Descrição**
Propriedades			
$mysqli_stmt::affected_rows	mysqli_stmt_affected_rows()	N/A	Retorna o número total de linhas alteradas, excluídas ou inseridas pela última instrução executada
$mysqli_stmt::errno	mysqli_stmt_errno()	N/A	Retorna o código de erro para a chamada de instrução mais recente
$mysqli_stmt::error	mysqli_stmt_error()	N/A	Retorna uma descrição de string para o último erro de instrução
$mysqli_stmt::field_count	mysqli_stmt_field_count()	N/A	Retorna o número de campos na instrução
$mysqli_stmt::insert_id	mysqli_stmt_insert_id()	N/A	Obtenha o ID gerado da operação INSERT anterior
$mysqli_stmt::num_rows	mysqli_stmt_num_rows()	N/A	Retorna o número de linhas no conjunto de resultados da instrução
$mysqli_stmt::param_count	mysqli_stmt_param_count()	N/A	Retorna o número do parâmetro para a instrução
$mysqli_stmt::sqlstate	mysqli_stmt_sqlstate()	N/A	Retorna o erro SQLSTATE da operação anterior
Métodos			
mysqli_stmt::attr_get()	mysqli_stmt_attr_get()	N/A	Usado para obter o valor atual de um atributo de instrução
mysqli_stmt::attr_set()	mysqli_stmt_attr_set()	N/A	Usado para modificar o comportamento de uma instrução preparada
mysqli_stmt::bind_param()	mysqli_stmt_bind_param()	N/A	Vincula variáveis a uma instrução preparada como parâmetros
mysqli_stmt::bind_result()	mysqli_stmt_bind_result()	N/A	Vincula variáveis a uma instrução preparada para armazenamento de resultados
mysqli_stmt::close()	mysqli_stmt_close()	N/A	Fecha uma instrução preparada

MySQL_STMT			
Interface POO	**Interface Procedural**	**Apelido (não use)**	**Descrição**
mysqli_stmt::data_seek()	mysqli_stmt_data_seek()	N/A	Aponta para uma linha arbitrária no conjunto de resultados da instrução
mysqli_stmt::execute()	mysqli_stmt_execute()	mysqli_execute()	Executa uma consulta preparada
mysqli_stmt::fetch()	mysqli_stmt_fetch()	N/A	Busca resultados de uma instrução preparada nas variáveis vinculadas
mysqli_stmt::free_result()	mysqli_stmt_free_result()	N/A	Libera a memória de resultado armazenada para o identificador de instrução fornecido
mysqli_stmt::get_result()	mysqli_stmt_get_result()	N/A	Obtém um conjunto de resultados de uma instrução preparada. Disponível apenas com o mysqlnd.
mysqli_stmt::get_warnings()	mysqli_stmt_get_warnings()	N/A	NÃO DOCUMENTADO
mysqli_stmt::more_results()	mysqli_stmt_more_results()	N/A	Verifica se há mais resultados de consulta de uma consulta múltipla
mysqli_stmt::next_result()	mysqli_stmt_next_result()	N/A	Lê o próximo resultado de uma consulta múltipla
mysqli_stmt::num_rows()	mysqli_stmt_num_rows()	N/A	Veja também a propriedade $mysqli_stmt::num_rows
mysqli_stmt::prepare()	mysqli_stmt_prepare()	N/A	Prepara uma instrução SQL para execução
mysqli_stmt::reset()	mysqli_stmt_reset()	N/A	Redefine uma instrução preparada
mysqli_stmt::result_metadata()	mysqli_stmt_result_metadata()	N/A	Retorna os metadados do conjunto de resultados de uma instrução preparada
mysqli_stmt::send_long_data()	mysqli_stmt_send_long_data()	N/A	Envia dados em blocos
mysqli_stmt::store_result()	mysqli_stmt_store_result()	N/A	Transfere um conjunto de resultados de uma instrução preparada

Resumo dos métodos mysqli_result

mysqli_result			
Interface POO	**Interface Procedural**	**Apelido (não use)**	**Descrição**
Propriedades			
$mysqli_result::current_field	mysqli_field_tell()	N/A	Obtém o deslocamento de campo atual de um ponteiro de resultado
$mysqli_result::field_count	mysqli_num_fields()	N/A	Obtém o número de campos em um resultado
$mysqli_result::lengths	mysqli_fetch_lengths()	N/A	Retorna os comprimentos das colunas da linha atual no conjunto de resultados
$mysqli_result::num_rows	mysqli_num_rows()	N/A	Obtém o número de linhas em um resultado
Methods			
mysqli_result::data_seek()	mysqli_data_seek()	N/A	Ajusta o ponteiro de resultado para uma linha arbitrária no resultado
mysqli_result::fetch_all()	mysqli_fetch_all()	N/A	Busca todas as linhas de resultado e retorna o conjunto de resultados como um array associativo, um array numérico ou ambos. Disponível apenas com o mysqlnd.
mysqli_result::fetch_array()	mysqli_fetch_array()	N/A	Busca uma linha de resultado como um array associativo, um array numérico ou ambos
mysqli_result::fetch_assoc()	mysqli_fetch_assoc()	N/A	Busca uma linha de resultado como um array associativo
mysqli_result::fetch_column()	mysqli_fetch_column()	N/A	Busca uma única coluna da próxima linha de um conjunto de resultados
mysqli_result::fetch_field_direct()	mysqli_fetch_field_direct()	N/A	Busca metadados para um único campo
mysqli_result::fetch_field()	mysqli_fetch_field()	N/A	Retorna o próximo campo no conjunto de resultados
mysqli_result::fetch_fields()	mysqli_fetch_fields()	N/A	Retorna um array de objetos que representam os campos em um conjunto de resultados
mysqli_result::fetch_object()	mysqli_fetch_object()	N/A	Retorna a linha atual de um conjunto de resultados como um objeto

mysqli_result			
Interface POO	**Interface Procedural**	**Apelido (não use)**	**Descrição**
mysqli_result::fetch_row()	mysqli_fetch_row()	N/A	Obtém uma linha de resultado como um array enumerado
mysqli_result::field_seek()	mysqli_field_seek()	N/A	Define ponteiro de resultado para um deslocamento de campo especificado
mysqli_result::free(), mysqli_result::close, mysqli_result::free_result	mysqli_free_result()	N/A	Libera a memória associada a um resultado

Resumo dos métodos mysqli_driver

MySQL_Driver			
Interface POO	**Interface Procedural**	**Apelido (não use)**	**Descrição**
Propriedades			
$mysqli_driver::mysqli_report	mysqli_report()	N/A	Define o modo de relatório de erros do mysqli
Métodos			
mysqli_driver::embedded_server_end()	mysqli_embedded_server_end()	N/A	NÃO DOCUMENTADO
mysqli_driver::embedded_server_start()	mysqli_embedded_server_start()	N/A	NÃO DOCUMENTADO

> **Nota** :
>
> As funções apelido são fornecidas apenas para fins de compatibilidade com versões anteriores. Não os use em novos projetos.

mysqli

(PHP 5, PHP 7, PHP 8)

Introdução

Representa uma conexão entre o PHP e um banco de dados MySQL.

Resumo da classe

class **mysqli** {
/* Propriedades */
public readonly int|string *$affected_rows*;
public readonly string *$client_info*;
public readonly int *$client_version*;
public readonly int *$connect_errno*;
public readonly ?string *$connect_error*;
public readonly int *$errno*;

```
public readonly string $error;
public readonly array $error_list;
public readonly int $field_count;
public readonly string $host_info;
public readonly ?string $info;
public readonly int|string $insert_id;
public readonly string $server_info;
public readonly int $server_version;
public readonly string $sqlstate;
public readonly int $protocol_version;
public readonly int $thread_id;
public readonly int $warning_count;
/* Métodos */
public function __construct(
    ?string $hostname = null,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?string $database = null,
    ?int $port = null,
    ?string $socket = null
)
public function autocommit(bool $enable): bool
public function begin_transaction(int $flags = 0, ?string $name = null): bool
public function change_user(string $username, #[\SensitiveParameter] string $password, ?string $database): bool
public function character_set_name(): string
public function close(): true
public function commit(int $flags = 0, ?string $name = null): bool
public function connect(
    ?string $hostname = null,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?string $database = null,
    ?int $port = null,
    ?string $socket = null
): bool
public function debug(string $options): true
public function dump_debug_info(): bool
public function execute_query(string $query, ?array $params = null): mysqli_result|bool
public function get_charset(): ?object
#[\Deprecated]
public function get_client_info(): string
public function get_connection_stats(): array
public function get_server_info(): string
public function get_warnings(): mysqli_warning|false
#[\Deprecated]
public function init(): ?bool
#[\Deprecated]
public function kill(int $process_id): bool
public function more_results(): bool
public function multi_query(string $query): bool
```

```
public function next_result(): bool
public function options(int $option, string|int $value): bool
#[\Deprecated]
public function ping(): bool
public static function poll(
    ?array &$read,
    ?array &$error,
    array &$reject,
    int $seconds,
    int $microseconds = 0
): int|false
public function prepare(string $query): mysqli_stmt|false
public function query(string $query, int $result_mode = MYSQLI_STORE_RESULT): mysqli_result|bool
public function quote_string(string $string): string
public function real_connect(
    ?string $hostname = null,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?string $database = null,
    ?int $port = null,
    ?string $socket = null,
    int $flags = 0
): bool
public function real_escape_string(string $string): string
public function real_query(string $query): bool
public function reap_async_query(): mysqli_result|bool
#[\Deprecated]
public function refresh(int $flags): bool
public function release_savepoint(string $name): bool
public function rollback(int $flags = 0, ?string $name = null): bool
public function savepoint(string $name): bool
public function select_db(string $database): bool
public function set_charset(string $charset): bool
public function ssl_set(
    ?string $key,
    ?string $certificate,
    ?string $ca_certificate,
    ?string $ca_path,
    ?string $cipher_algos
): true
public function stat(): string|false
public function stmt_init(): mysqli_stmt|false
public function store_result(int $mode = 0): mysqli_result|false
public function thread_safe(): bool
public function use_result(): mysqli_result|false
}
```

Índice

mysqli::$affected_rows

mysqli::$affected_rows

mysqli_affected_rows

(PHP 5, PHP 7, PHP 8)

mysqli::$affected_rows -- mysqli_affected_rows — Retorna o número de linhas afetadas pela operação MySQL anterior

Descrição

Estilo orientado a objetos

int|string *$mysqli->affected_rows*;

Estilo procedural

function **mysqli_affected_rows**(mysqli `$mysql`): int|string

Retorna o número de linhas afetadas pelo último comando `INSERT`, `UPDATE`, `REPLACE` ou `DELETE`. Funciona como mysqli_num_rows() para consultas `SELECT`.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Um inteiro maior do que zero indica o número de linhas afetadas ou obtidas. Zero indica que nenhum registro foi atualizado por uma declaração `UPDATE`, nenhuma linha correspondeu à claúsula `WHERE` na consulta ou que nenhuma consulta foi executada ainda. `-1` indica que a consulta retornou um erro ou que **mysqli_affected_rows()** foi chamada para uma consulta `SELECT` sem buffer.

> **Nota** :
>
> Se o número de linhas afetadas for maior do que o maior valor inteiro (`PHP_INT_MAX`), o número de linhas afetadas será retornado como uma string.

Examples

Exemplo #1 Exemplo de *$mysqli->affected_rows*

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* Insert rows */
$mysqli->query("CREATE TABLE Language SELECT * from CountryLanguage");
printf("Affected rows (INSERT): %d\n", $mysqli->affected_rows);

$mysqli->query("ALTER TABLE Language ADD Status int default 0");
```

```php
/* update rows */
$mysqli->query("UPDATE Language SET Status=1 WHERE Percentage > 50");
printf("Affected rows (UPDATE): %d\n", $mysqli->affected_rows);

/* delete rows */
$mysqli->query("DELETE FROM Language WHERE Percentage < 50");
printf("Affected rows (DELETE): %d\n", $mysqli->affected_rows);

/* select all rows */
$result = $mysqli->query("SELECT CountryCode FROM Language");
printf("Affected rows (SELECT): %d\n", $mysqli->affected_rows);

/* Delete table Language */
$mysqli->query("DROP TABLE Language");
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* Insert rows */
mysqli_query($link, "CREATE TABLE Language SELECT * from CountryLanguage");
printf("Affected rows (INSERT): %d\n", mysqli_affected_rows($link));

mysqli_query($link, "ALTER TABLE Language ADD Status int default 0");

/* update rows */
mysqli_query($link, "UPDATE Language SET Status=1 WHERE Percentage > 50");
printf("Affected rows (UPDATE): %d\n", mysqli_affected_rows($link));

/* delete rows */
mysqli_query($link, "DELETE FROM Language WHERE Percentage < 50");
printf("Affected rows (DELETE): %d\n", mysqli_affected_rows($link));

/* select all rows */
$result = mysqli_query($link, "SELECT CountryCode FROM Language");
printf("Affected rows (SELECT): %d\n", mysqli_affected_rows($link));

/* Delete table Language */
mysqli_query($link, "DROP TABLE Language");
```

Os exemplos acima produzirão:

```
Affected rows (INSERT): 984
Affected rows (UPDATE): 168
Affected rows (DELETE): 815
Affected rows (SELECT): 169
```

Veja também

- mysqli_num_rows() - Obtém o número de linhas no resultado
- mysqli_info() - Recupera informação sobre a recente consulta executada

mysqli::autocommit

mysqli::autocommit

mysqli_autocommit

(PHP 5, PHP 7, PHP 8)

mysqli::autocommit -- mysqli_autocommit — Ativa ou desativa a gravação automática das modificações no banco de dados

Descrição

Estilo orientado a objetos

public function **mysqli::autocommit**(bool `$enable`): bool

Estilo procedural

function **mysqli_autocommit**(mysqli `$mysql`, bool `$enable`): bool

Ativa ou desativa o modo de gravação automática nas consultas para a conexão do banco de dados.

Para determinar o estado atual da gravação automática use o comando SQL `SELECT @@autocommit`.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`enable`

Habilitar ou não a gravação automática.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**`MYSQLI_REPORT_ERROR`**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **`MYSQLI_REPORT_STRICT`**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Exemplo de **mysqli::autocommit()**

Estilo orientado a objetos

```php
<?php

/* Solicita ao mysqli que dispare uma exceção se um erro ocorrer */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
```

```php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* O motor da tabela tem que suportar transações */
$mysqli->query("CREATE TABLE IF NOT EXISTS language (
    Code text NOT NULL,
    Speakers int(11) NOT NULL
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;");

/* Desliga a gravação automática */
$mysqli->autocommit(false);

$result = $mysqli->query("SELECT @@autocommit");
$row = $result->fetch_row();
printf("Gravação automática = %s\n", $row[0]);

try {
    /* Prepara a declaração de inserção */
    $stmt = $mysqli->prepare('INSERT INTO language(Code, Speakers) VALUES (?,?)');
    $stmt->bind_param('ss', $language_code, $native_speakers);

    /* Insere alguns valores */
    $language_code = 'DE';
    $native_speakers = 50_123_456;
    $stmt->execute();
    $language_code = 'FR';
    $native_speakers = 40_546_321;
    $stmt->execute();

    /* Grava os dados no banco. Isto não liga a gravação automática */
    $mysqli->commit();
    print "2 linhas gravadas no banco de dados\n";

    $result = $mysqli->query("SELECT @@autocommit");
    $row = $result->fetch_row();
    printf("Gravação automática = %s\n", $row[0]);

    /* Tenta inserir mais valores */
    $language_code = 'PL';
    $native_speakers = 30_555_444;
    $stmt->execute();
    $language_code = 'DK';
    $native_speakers = 5_222_444;
    $stmt->execute();

    /* Ligar a gravação automática irá disparar uma gravação */
    $mysqli->autocommit(true);

    print "2 linhas gravadas no banco de dados\n";
} catch (mysqli_sql_exception $exception) {
    $mysqli->rollback();

    throw $exception;
}
```

Estilo procedural

```php
<?php
```

```php
/* Solicita ao mysqli que dispare uma exceção se um erro ocorrer */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

/* O motor da tabela tem que suportar transações */
mysqli_query($mysqli, "CREATE TABLE IF NOT EXISTS language (
    Code text NOT NULL,
    Speakers int(11) NOT NULL
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;");

/* Desliga a gravação automática */
mysqli_autocommit($mysqli, false);

$result = mysqli_query($mysqli, "SELECT @@autocommit");
$row = mysqli_fetch_row($result);
printf("Gravação automática = %s\n", $row[0]);

try {
    /* Prepara a declaração de inserção */
    $stmt = mysqli_prepare($mysqli, 'INSERT INTO language(Code, Speakers) VALUES (?,?)');
    mysqli_stmt_bind_param($stmt, 'ss', $language_code, $native_speakers);

    /* Insere alguns valores */
    $language_code = 'DE';
    $native_speakers = 50_123_456;
    mysqli_stmt_execute($stmt);
    $language_code = 'FR';
    $native_speakers = 40_546_321;
    mysqli_stmt_execute($stmt);

    /* Grava os dados no banco. Isto não liga a gravação automática */
    mysqli_commit($mysqli);
    print "2 linhas gravadas no banco de dados\n";

    $result = mysqli_query($mysqli, "SELECT @@autocommit");
    $row = mysqli_fetch_row($result);
    printf("Gravação automática = %s\n", $row[0]);

    /* Tenta inserir mais valores */
    $language_code = 'PL';
    $native_speakers = 30_555_444;
    mysqli_stmt_execute($stmt);
    $language_code = 'DK';
    $native_speakers = 5_222_444;
    mysqli_stmt_execute($stmt);

    /* Ligar a gravação automática irá disparar uma gravação */
    mysqli_autocommit($mysqli, true);

    print "2 linhas gravadas no banco de dados\n";
} catch (mysqli_sql_exception $exception) {
    mysqli_rollback($mysqli);

    throw $exception;
}
```

Os exemplos acima produzirão:

```
Gravação automática = 0
2 linhas gravadas no banco de dados
Gravação automática = 0
2 linhas gravadas no banco de dados
Gravação automática = 0
2 linhas gravadas no banco de dados
Gravação automática = 0
2 linhas gravadas no banco de dados
```

Notas

> **Nota** :
>
> Esta função não funciona com tipos de tabelas não transacionais (como MyISAM ou ISAM).

Veja também

- mysqli_begin_transaction() - Inicia uma transação
- mysqli_commit() - Grava a transação atual
- mysqli_rollback() - Reverte a transação atual

mysqli::begin_transaction

mysqli::begin_transaction

mysqli_begin_transaction

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

mysqli::begin_transaction -- mysqli_begin_transaction — Inicia uma transação

Descrição

Estilo orientado a objetos

public function **mysqli::begin_transaction**(int `$flags` = 0, ?string `$name` = **`null`**): bool

Estilo procedural:

function **mysqli_begin_transaction**(mysqli `$mysql`, int `$flags` = 0, ?string `$name` = **`null`**): bool

Inicia uma transação. Requer o mecanismo InnoDB (é ativado por padrão). Para detalhes adicionais sobre como as transações do MySQL funcionam, consulte » http://dev.mysql.com/doc/mysql/en/commit.html.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`flags`

As flags válidas são:

- `MYSQLI_TRANS_START_READ_ONLY`: Inicia a transação como "START TRANSACTION READ ONLY". Requer MySQL 5.6 e superior.
- `MYSQLI_TRANS_START_READ_WRITE`: Inicia a transação como "START TRANSACTION READ WRITE". Requer MySQL 5.6 e superior.
- `MYSQLI_TRANS_START_WITH_CONSISTENT_SNAPSHOT`: Inicia a transação como "START TRANSACTION WITH CONSISTENT SNAPSHOT".

`name`

Nome do ponto de salvamento para a transação.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.0.0	`name` agora é anulável.

Examples

Exemplo #1 Exemplo de **mysqli::begin_transaction()**

Estilo orientado a objetos

```php
<?php

/* Diga ao mysqli para lançar uma exceção se ocorrer um erro */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* O mecanismo de tabela deve suportar transações */
$mysqli->query("CREATE TABLE IF NOT EXISTS language (
    Code text NOT NULL,
    Speakers int(11) NOT NULL
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;");

/* Inicia transação */
$mysqli->begin_transaction();

try {
    /* Insira alguns valores */
    $mysqli->query("INSERT INTO language(Code, Speakers) VALUES ('DE', 42000123)");

    /* Tente inserir valores inválidos */
    $language_code = 'FR';
    $native_speakers = 'Unknown';
    $stmt = $mysqli->prepare('INSERT INTO language(Code, Speakers) VALUES (?,?)');
    $stmt->bind_param('ss', $language_code, $native_speakers);
    $stmt->execute();
```

```
    /* Se o código atingir esse ponto sem erros, confirme os dados no banco de dados */
    $mysqli->commit();
} catch (mysqli_sql_exception $exception) {
    $mysqli->rollback();

    throw $exception;
}
```

Estilo procedural

```
<?php

/* Diga ao mysqli para lançar uma exceção se ocorrer um erro */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

/* O mecanismo de tabela deve suportar transações */
mysqli_query($mysqli, "CREATE TABLE IF NOT EXISTS language (
    Code text NOT NULL,
    Speakers int(11) NOT NULL
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;");

/* Inicia transação */
mysqli_begin_transaction($mysqli);

try {
    /* Insira alguns valores */
    mysqli_query($mysqli, "INSERT INTO language(Code, Speakers) VALUES ('DE', 42000123)");

    /* Tente inserir valores inválidos */
    $language_code = 'FR';
    $native_speakers = 'Unknown';
    $stmt = mysqli_prepare($mysqli, 'INSERT INTO language(Code, Speakers) VALUES (?,?)');
    mysqli_stmt_bind_param($stmt, 'ss', $language_code, $native_speakers);
    mysqli_stmt_execute($stmt);

    /* Se o código atingir esse ponto sem erros, confirme os dados no banco de dados */
    mysqli_commit($mysqli);
} catch (mysqli_sql_exception $exception) {
    mysqli_rollback($mysqli);

    throw $exception;
}
```

Notas

> **Nota** :
>
> Esta função não funciona com tipos de tabelas não transacionais (como MyISAM ou ISAM).

Veja também

- mysqli_autocommit() - Ativa ou desativa a gravação automática das modificações no banco de dados
- mysqli_commit() - Grava a transação atual

- mysqli_rollback() - Reverte a transação atual

mysqli::change_user

mysqli::change_user

mysqli_change_user

(PHP 5, PHP 7, PHP 8)

mysqli::change_user -- mysqli_change_user — Altera o usuário da conexão com o banco de dados

Descrição

Estilo orientado a objetos

public function **mysqli::change_user**(string `$username`, #[\SensitiveParameter] string `$password`, ?string `$database`): bool

Estilo procedural

function **mysqli_change_user**(
    mysqli `$mysql`,
    string `$username`,
    #[\SensitiveParameter] string `$password`,
    ?string `$database`
): bool

Tenta conexão ao banco de dados especificado usando as credenciais fornecidas.

Em comparação ao método mysqli::connect(), este método não não fechará a conexão atual se a nova conexão não puder ser aberta.

Para conseguir mudar usuários com sucesso, os parâmetors `username` e `password` fornecidos devem ser válidos e este usuário deve ter permissões suficientes para acessar o banco de dados desejado. Se por qualquer motivo a autorização falhar, a autenticação do usuário atual irá permanecer.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`username`

O nome do usuário MySQL.

`password`

A senha MySQL.

`database`

O nome do banco de dados. Se `null` ou uma string vazia forem passados, a conexão ao servidor será aberta sem banco de dados padrão.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (`MYSQLI_REPORT_ERROR`) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como `MYSQLI_REPORT_STRICT`, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Redefinindo a sessão da conexão

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "meu_usuario", "minha_senha", "teste");

$mysqli->query("SET @a:=1");

$mysqli->change_user("meu_usuario", "minha_senha", "mundo");

$result = $mysqli->query("SELECT DATABASE()");
$row = $result->fetch_row();
printf("Banco de dados padrão: %s\n", $row[0]);

$result = $mysqli->query("SELECT @a");
$row = $result->fetch_row();
if ($row[0] === null) {
    printf("Valor da variável 'a' é NULL\n");
}
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "meu_usuario", "minha_senha", "teste");

mysqli_query($link, "SET @a:=1");

mysqli_change_user($link, "meu_usuario", "minha_senha", "mundo");

$result = mysqli_query($link, "SELECT DATABASE()");
$row = mysqli_fetch_row($result);
printf("Banco de dados padrão: %s\n", $row[0]);

$result = mysqli_query($link, "SELECT @a");
$row = mysqli_fetch_row($result);
if ($row[0] === null) {
    printf("Valor da variável 'a' é NULL\n");
}
```

Os exemplos acima produzirão:

```
```

```
Banco de dados padrão: mundo
Valor da variável 'a' é NULL
```

Exemplo #2 Se `database` for `null` a conexão é aberta sem selecionar nenhum banco de dados padrão

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "meu_usuario", "minha_senha", "teste");

$mysqli->change_user("meu_usuario", "minha_senha", null);

$result = $mysqli->query("SELECT DATABASE()");
$row = $result->fetch_row();
printf("Banco de dados padrão: %s\n", $row[0]);
```

Os exemplos acima produzirão:

```
Banco de dados padrão:
```

Notas

> **Nota** :
>
> Usar este comando sempre irá fazer com que a conexão atual se comporte como se fosse uma conexão totalmente nova, não importando se a operação foi completada com sucesso. Esta reinicialização inclui realizar uma reversão de dados em quaisquer transações ativas, fechando todas as tabelas temporárias e destravando todas as tabelas travadas.

Veja também

- mysqli_connect() - Sinônimo de mysqli::__construct
- mysqli_select_db() - Seleciona o banco de dados padrão para consultas de banco de dados

mysqli::character_set_name

mysqli::character_set_name

mysqli_character_set_name

(PHP 5, PHP 7, PHP 8)

mysqli::character_set_name -- mysqli_character_set_name — Retorna o conjunto de caracteres atual da conexão com o banco de dados

Descrição

Estilo orientado a objetos

public function **mysqli::character_set_name**(): string

Estilo procedural

function **mysqli_character_set_name**(mysqli `$mysql`): string

Retorna o conjunto de caracteres atual da conexão com o banco de dados.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

O conjunto de caracteres atual da conexão

Examples

Exemplo #1 Exemplo de **mysqli::character_set_name()**

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "meu_usuario", "minha_senha", "mundo");

/* Define o conjunto padrão de caracteres */
$mysqli->set_charset('utf8mb4');

/* Mostra o conjunto atual de caracteres */
$charset = $mysqli->character_set_name();
printf("Conjunto atual de caracteres é %s\n", $charset);
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "meu_usuario", "minha_senha", "mundo");

/* Define o conjunto padrão de caracteres */
mysqli_set_charset($mysqli, 'utf8mb4');

/* Mostra o conjunto atual de caracteres */
$charset = mysqli_character_set_name($mysqli);
printf("Conjunto atual de caracteres é %s\n", $charset);
```

Os exemplos acima produzirão:

```
Conjunto atual de caracteres é utf8mb4
```

Veja também

- mysqli_set_charset() - Define o conjunto de caracteres do cliente
- mysqli_real_escape_string() - Escape caracteres especiais em uma string para uso em uma instrução SQL, levando em consideração o conjunto de caracteres atual da conexão

mysqli::close

mysqli::close

mysqli_close

(PHP 5, PHP 7, PHP 8)

mysqli::close -- mysqli_close — Fecha uma conexão ao banco de dados previamente aberta

Descrição

Estilo orientado a objetos

public function **mysqli::close**(): true

Estilo procedural

function **mysqli_close**(mysqli `$mysql`): true

Fecha uma conexão ao banco de dados previamente aberta.

Conexões MySQL não persistentes e conjuntos de resultados são automaticamente fechados quando seus objetos são destruídos. Fechar as conexões e liberar os conjuntos de resultados explicitamente é opcional. Entretanto, é uma boa prática fechar a conexão assim que o script termina de executar todas as suas operações de banco de dados, se ele ainda tiver muito processamento a realizar depois de obter os resultados.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função sempre retorna `true`. Anteriormente retornava `false` em caso de falha.

Examples

Exemplo #1 Exemplo de mysqli::close()

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "meu_usuario", "minha_senha", "mundo");

$result = $mysqli->query("SELECT Nome, CodigoPais FROM Cidade ORDER BY ID LIMIT 3");

/* Fecha a conexão assim que não for mais necessária */
$mysqli->close();

foreach ($result as $row) {
    /* Processando os dados obtidos do banco */
}
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "meu_usuario", "minha_senha", "mundo");

$result = mysqli_query($mysqli, "SELECT Nome, CodigoPais FROM Cidade ORDER BY ID LIMIT 3");

/* Fecha a conexão assim que não for mais necessária */
mysqli_close($mysqli);

foreach ($result as $row) {
    /* Processando os dados obtidos do banco */
}
```

Notas

> **Nota** :
>
> **mysqli_close()** não fechará conexões persistentes. Para detalhes adicionais, consulte a página do manual sobre conexões persistentes.

Veja também

- mysqli::__construct() - Abre uma nova conexão com o servidor MySQL
- mysqli_init() - Inicializa o MySQLi e retorna um objeto a ser utilizado com mysqli_real_connect()
- mysqli_real_connect() - Abre uma conexão a um servidor MySQL
- mysqli_free_result() - Libera memória associada a um resultado

mysqli::commit

mysqli::commit

mysqli_commit

(PHP 5, PHP 7, PHP 8)

mysqli::commit -- mysqli_commit — Grava a transação atual

Descrição

Estilo orientado a objetos

public function **mysqli::commit**(int `$flags` = 0, ?string `$name` = **`null`**): bool

Estilo procedural

function **mysqli_commit**(mysqli `$mysql`, int `$flags` = 0, ?string `$name` = **`null`**): bool

Grava a transação atual para a conexão do banco de dados.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`flags`

Uma máscara de bits de constantes **`MYSQLI_TRANS_COR_*`**.

`name`

Se fornecido, `COMMIT/*name*/` é executado.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**`MYSQLI_REPORT_ERROR`**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **`MYSQLI_REPORT_STRICT`**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Registro de Alterações

Versão	Descrição
8.0.0	`name` agora pode ser nulo.

Examples

Consulto o exemplo de mysqli::begin_transaction().

Notas

> **Nota** :
>
> Esta função não funciona com tipos de tabelas não transacionais (como MyISAM ou ISAM).

Veja também

- mysqli_autocommit() - Ativa ou desativa a gravação automática das modificações no banco de dados
- mysqli_begin_transaction() - Inicia uma transação
- mysqli_rollback() - Reverte a transação atual
- mysqli_savepoint() - Define um ponto de salvamento de transação nomeado

mysqli::$connect_errno

mysqli::$connect_errno

mysqli_connect_errno

(PHP 5, PHP 7, PHP 8)

mysqli::$connect_errno -- mysqli_connect_errno — Retorna o código de erro da última chamada de conexão

Descrição

Estilo orientado a objetos

int *$mysqli->connect_errno*;

Estilo procedural

function **mysqli_connect_errno**(): int

Retorna o código de erro da última tentativa de conexão.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um código de erro para a última tentativa de conexão, se tiver falhado. Zero significa que nenhum erro ocorreu.

Retorna o último código de erro da conexão independente da instância na qual o método for chamado.

Examples

Exemplo #1 Exemplo de *$mysqli->connect_errno*

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_OFF);
/* @ é usado para suprimir avisos */
$mysqli = @new mysqli('localhost', 'usuario_falso', 'senha_errada', 'nao_existe');
if ($mysqli->connect_errno) {
    /* Use seu método preferido de registro de erro aqui */
    error_log('Erro de conexão: ' . $mysqli->connect_errno);
}
```

Estilo procedural

```php
<?php
```

```php
mysqli_report(MYSQLI_REPORT_OFF);
/* @ é usado para suprimir avisos */
$link = @mysqli_connect('localhost', 'usuario_falso', 'senha_errada', 'nao_existe');
if (!$link) {
    /* Use seu método preferido de registro de erro aqui */
    error_log('Erro de conexão: ' . mysqli_connect_errno());
}
```

Veja também

- mysqli_connect() - Sinônimo de mysqli::__construct
- mysqli_connect_error() - Retorna uma descrição do último erro de conexão
- mysqli_errno() - Retorna o código de erro para a chamada de função mais recente
- mysqli_error() - Retorna uma string descrevendo o último erro
- mysqli_sqlstate() - Retorna o erro SQLSTATE da operação MySQL anterior

mysqli::$connect_error

mysqli::$connect_error

mysqli_connect_error

(PHP 5, PHP 7, PHP 8)

mysqli::$connect_error -- mysqli_connect_error — Retorna uma descrição do último erro de conexão

Descrição

Estilo orientado a objetos

?string *$mysqli->connect_error*;

Estilo procedural

function **mysqli_connect_error**(): ?string

Retorna a mensagem de erro da última tentativa de conexão.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Uma string que descreve o erro. `null` é retornado se nenhum erro tiver ocorrido.

Retorna o último erro da conexão independente da instância na qual o método for chamado.

Examples

Exemplo #1 Exemplo de *$mysqli->connect_error*

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_OFF);
/* @ é usado para suprimir avisos */
```

```php
$mysqli = @new mysqli('localhost', 'usuario_falso', 'senha_errada', 'nao_existe');
if ($mysqli->connect_error) {
    /* Use seu método preferido de registro de erro aqui */
    error_log('Erro de conexão: ' . $mysqli->connect_error);
}
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_OFF);
/* @ é usado para suprimir avisos */
$link = @mysqli_connect('localhost', 'usuario_falso', 'senha_errada', 'nao_existe');
if (!$link) {
    /* Use seu método preferido de registro de erro aqui */
    error_log('Erro de conexão: ' . mysqli_connect_error());
}
```

Veja também

- mysqli_connect() - Sinônimo de mysqli::__construct
- mysqli_connect_errno() - Retorna o código de erro da última chamada de conexão
- mysqli_errno() - Retorna o código de erro para a chamada de função mais recente
- mysqli_error() - Retorna uma string descrevendo o último erro
- mysqli_sqlstate() - Retorna o erro SQLSTATE da operação MySQL anterior

mysqli::__construct

mysqli::__construct

mysqli::connect

mysqli_connect

(PHP 5, PHP 7, PHP 8)

mysqli::__construct -- mysqli::connect -- mysqli_connect — Abre uma nova conexão com o servidor MySQL

Descrição

Estilo orientado a objetos

public function **mysqli::__construct**(
  ?string $hostname = null,
  ?string $username = null,
  #[\SensitiveParameter] ?string $password = null,
  ?string $database = null,
  ?int $port = null,
  ?string $socket = null
)
public function **mysqli::connect**(
  ?string $hostname = null,
  ?string $username = null,
  #[\SensitiveParameter] ?string $password = null,
  ?string $database = null,

```
    ?int $port = null,
    ?string $socket = null
): bool
```

Estilo procedural

```
function mysqli_connect(
    ?string $hostname = null,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?string $database = null,
    ?int $port = null,
    ?string $socket = null
): mysqli|false
```

Abre uma conexão com o servidor MySQL.

Parâmetros

`hostname`

Pode ser um nome de host ou um endereço IP. Ao passar `null`, o valor é recuperado de mysqli.default_host. Quando possível, serão usados pipes em vez do protocolo TCP/IP. O protocolo TCP/IP é usado se um nome de host e um número de porta forem fornecidos juntos, por ex. `localhost:3308`.

Prefixar o host com `p:` abre uma conexão persistente. mysqli_change_user() é chamado automaticamente em conexões abertas a partir do pool de conexões.

`username`

O nome de usuário do MySQL ou `null` para assumir o nome de usuário com base na opção ini mysqli.default_user.

`password`

A senha do MySQL ou `null` para assumir a senha com base na opção ini mysqli.default_pw.

`database`

O banco de dados padrão a ser utilizado ao realizar consultas ou `null`.

`port`

O número da porta para tentar conectar ao servidor MySQL ou `null` para assumir a porta com base na opção ini mysqli.default_port.

`socket`

O socket ou pipe nomeado que deve ser usado ou `null` para assumir o socket com base na opção ini mysqli.default_socket.

> **Nota** :
>
> A especificação do parâmetro `socket` não determinará explicitamente o tipo de conexão a ser usada ao conectar-se ao servidor MySQL. A maneira com a qual a conexão é feita com o banco de dados

> MySQL é determinada pelo parâmetro `hostname`.

Valor Retornado

mysqli::__construct() sempre retorna um objeto que representa a conexão a um servidor MySQL, independente se teve sucesso ou não.

mysqli_connect() retorna um objeto que representa a conexão com um servidor MySQL, ou `false` em caso de falha.

mysqli::connect() retorna `true` em caso de sucesso ou `false` em caso de falha. Antes do PHP 8.1.0, ele retornava `null` em caso de sucesso.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (`MYSQLI_REPORT_ERROR`) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como `MYSQLI_REPORT_STRICT`, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Registro de Alterações

Versão	Descrição
8.1.0	Agora, o método **mysqli::connect()** retorna `true` em vez de `null` em caso de sucesso.
7.4.0	Todos os parâmetros podem ser null.

Examples

Exemplo #1 Exemplo de **mysqli::__construct()**

Estilo orientado a objetos

```php
<?php

/* O relatório de erros para mysqli deve ser ativado antes de tentar fazer uma conexão */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$mysqli = new mysqli('localhost', 'my_user', 'my_password', 'my_db');

/* Defina o conjunto de caracteres desejado após estabelecer uma conexão */
$mysqli->set_charset('utf8mb4');

printf("Success... %s\n", $mysqli->host_info);
```

Estilo procedural

```php
<?php

/* O relatório de erros para mysqli deve ser ativado antes de tentar fazer uma conexão */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$mysqli = mysqli_connect('localhost', 'my_user', 'my_password', 'my_db');

/* Defina o conjunto de caracteres desejado após estabelecer uma conexão */
mysqli_set_charset($mysqli, 'utf8mb4');
```

```php
printf("Sucesso... %s\n", mysqli_get_host_info($mysqli));
```

Os exemplos acima produzirão algo semelhante a:

```
Sucesso... localhost via TCP/IP
```

Exemplo #2 Estendendo a classe mysqli

```php
<?php

class FooMysqli extends mysqli {
    public function __construct($host, $user, $pass, $db, $port, $socket, $charset) {
        mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
        parent::__construct($host, $user, $pass, $db, $port, $socket);
        $this->set_charset($charset);
    }
}

$db = new FooMysqli('localhost', 'my_user', 'my_password', 'my_db', 3306, null, 'utf8mb4');
```

Exemplo #3 Tratamento manual de erros

Se o relatório de erros estiver desabilitado, o desenvolvedor é responsável por verificar e tratar as falhas

Estilo orientado a objetos

```php
<?php

error_reporting(0);
mysqli_report(MYSQLI_REPORT_OFF);
$mysqli = new mysqli('localhost', 'my_user', 'my_password', 'my_db');
if ($mysqli->connect_errno) {
    throw new RuntimeException('mysqli connection error: ' . $mysqli->connect_error);
}

/* Defina o conjunto de caracteres desejado após estabelecer uma conexão */
$mysqli->set_charset('utf8mb4');
if ($mysqli->errno) {
    throw new RuntimeException('mysqli error: ' . $mysqli->error);
}
```

Estilo procedural

```php
<?php

error_reporting(0);
mysqli_report(MYSQLI_REPORT_OFF);
$mysqli = mysqli_connect('localhost', 'my_user', 'my_password', 'my_db');
if (mysqli_connect_errno()) {
    throw new RuntimeException('mysqli connection error: ' . mysqli_connect_error());
```

```
}

/* Defina o conjunto de caracteres desejado após estabelecer uma conexão */
mysqli_set_charset($mysqli, 'utf8mb4');
if (mysqli_errno($mysqli)) {
    throw new RuntimeException('mysqli error: ' . mysqli_error($mysqli));
}
```

Notas

> **Nota** :
>
> O MySQLnd sempre assume o conjunto de caracteres padrão do servidor. Este conjunto de caracteres é enviado durante a criação/autenticação da conexão e será usado pelo mysqlnd.
>
> A libmysqlclient usa o conjunto de caracteres padrão definido no arquivo *my.cnf* ou definido por uma chamada explícita a mysqli_options() antes de chamar mysqli_real_connect(), mas depois de chamar mysqli_init().

> **Nota** :
>
> Estilo orientado a objetos apenas: Se a conexão falhar, um objeto ainda será retornado. Para verificar se a conexão falhou, use a função mysqli_connect_error() ou a propriedade mysqli->connect_error como nos exemplos anteriores.

> **Nota** :
>
> Se for necessário definir opções, como o tempo limite de conexão, mysqli_real_connect() deve ser usado.

> **Nota** :
>
> Chamar o construtor sem parâmetros é o mesmo que chamar mysqli_init().

> **Nota** :
>
> O erro "Can't create TCP/IP socket (10106)" geralmente significa que a diretiva de configuração variables_order não contém o caractere E. No Windows, se o ambiente não for copiado, a variável de ambiente SYSTEMROOT não estará disponível e o PHP terá problemas para carregar o Winsock.

Veja também

- mysqli_real_connect() - Abre uma conexão a um servidor MySQL
- mysqli_options() - Define opções
- mysqli_connect_errno() - Retorna o código de erro da última chamada de conexão
- mysqli_connect_error() - Retorna uma descrição do último erro de conexão
- mysqli_close() - Fecha uma conexão ao banco de dados previamente aberta

mysqli::debug

mysqli::debug

mysqli_debug

(PHP 5, PHP 7, PHP 8)

mysqli::debug -- mysqli_debug — Realiza operações de depuração

Descrição

Estilo orientado a objetos

public function **mysqli::debug**(string `$options`): true

Estilo procedural

function **mysqli_debug**(string `$options`): true

Realiza operações de depuração usando a biblioteca Fred Fish.

Parâmetros

`options`

Uma string representando a operação de depuração a ser executada

A string de opções de depuração é uma sequência de campos separados por dois pontos (:) desta maneira:

```
<campo_1>:<campo_2>:<campo_N>
```

Cada campo consiste de um caractere de opção mandatório seguido por uma `,` opcional e uma lista de modificadores: separados por vírgula: `opção[,modificador,modificador,...,modificador]`

Caracteres de Opção Reconhecidos

Caractere de `options`	**Descrição**
O	**`MYSQLND_DEBUG_FLUSH`**
A/a	**`MYSQLND_DEBUG_APPEND`**
F	**`MYSQLND_DEBUG_DUMP_FILE`**
i	**`MYSQLND_DEBUG_DUMP_PID`**
L	**`MYSQLND_DEBUG_DUMP_LINE`**
m	**`MYSQLND_DEBUG_TRACE_MEMORY_CALLS`**
n	**`MYSQLND_DEBUG_DUMP_LEVEL`**
o	saída para arquivo
T	**`MYSQLND_DEBUG_DUMP_TIME`**
t	**`MYSQLND_DEBUG_DUMP_TRACE`**
x	**`MYSQLND_DEBUG_PROFILE_CALLS`**

Valor Retornado

Sempre retorna **`true`**.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função agora sempre retorna `true`. Anteriormente retornava `false` em caso de falha.

Examples

Exemplo #1 Gerando um Arquivo de Rastreamento

```
<?php

/* Cria um arquivo de rastreamento em '/tmp/client.trace' na máquina local (do cliente): */
mysqli_debug("d:t:o,/tmp/client.trace");

?>
```

Notas

> **Nota** :
>
> Para usar a função **mysqli_debug()** deve-se compilar a biblioteca do cliente MySQL com suporte a depuração.

Veja também

- mysqli_dump_debug_info() - Descarrega informação de debug no log
- mysqli_report() - Sinônimo de mysqli_driver->report_mode

mysqli::dump_debug_info

mysqli::dump_debug_info

mysqli_dump_debug_info

(PHP 5, PHP 7, PHP 8)

mysqli::dump_debug_info -- mysqli_dump_debug_info — Descarrega informação de debug no log

Descrição

Estilo orientado a objetos

public function **mysqli::dump_debug_info**(): bool

Estilo procedural

function **mysqli_dump_debug_info**(mysqli `$mysql`): bool

Esta função é projetada para ser executada por um usuário com super privilégio e é usada para descarregar informações de depuração no log, para o servidor MySQL, relacionadas à conexão.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Veja também

- mysqli_debug() - Realiza operações de depuração

mysqli->errno

mysqli->errno

mysqli_errno

(PHP 5, PHP 7, PHP 8)

mysqli->errno -- mysqli_errno — Retorna o código de erro para a chamada de função mais recente

Descrição

Estilo orientado a objetos

int *$mysqli->errno*;

Estilo procedural

function **mysqli_errno**(mysqli `$mysql`): int

Retorna o último código de erro para a chamada de função MySQLi mais recente que pode ter sucesso ou falhar.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Um valor de código de erro para a última chamada, se ela falhou. `0` significa que nenhum erro ocorreu.

Examples

Exemplo #1 Exemplo de *$mysqli->errno*

Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if ($mysqli->connect_errno) {
    printf("Conexão falhou: %s\n", $mysqli->connect_error);
    exit();
}

if (!$mysqli->query("SET a=1")) {
```

```
    printf("Errorcode: %d\n", $mysqli->errno);
}

/* close connection */
$mysqli->close();
?>
```

Estilo procedural

```
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

if (!mysqli_query($link, "SET a=1")) {
    printf("Errorcode: %d\n", mysqli_errno($link));
}

/* close connection */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Errorcode: 1193
```

Veja também

- mysqli_connect_errno() - Retorna o código de erro da última chamada de conexão
- mysqli_connect_error() - Retorna uma descrição do último erro de conexão
- mysqli_error() - Retorna uma string descrevendo o último erro
- mysqli_sqlstate() - Retorna o erro SQLSTATE da operação MySQL anterior

mysqli->error

mysqli->error

mysqli_error

(PHP 5, PHP 7, PHP 8)

mysqli->error -- mysqli_error — Retorna uma string descrevendo o último erro

Descrição

Estilo orientado a objetos

string *$mysqli->error*;

Estilo procedural

function **mysqli_error**(mysqli `$mysql`): string

Retorna a última mensagem de erro da chamada de função MySQLi mais recente que possa ter sucesso ou falhar.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Uma string que descreve o erro. Uma string vazia se não houve erro.

Examples

Exemplo #1 Exemplo de *$mysqli->error*

Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if ($mysqli->connect_errno) {
    printf("Conexão falhou: %s\n", $mysqli->connect_error);
    exit();
}

if (!$mysqli->query("SET a=1")) {
    printf("Error message: %s\n", $mysqli->error);
}

/* close connection */
$mysqli->close();
?>
```

Estilo procedural

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

if (!mysqli_query($link, "SET a=1")) {
    printf("Error message: %s\n", mysqli_error($link));
}

/* close connection */
```

```
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Error message: Unknown system variable 'a'
```

Veja também

- mysqli_connect_errno() - Retorna o código de erro da última chamada de conexão
- mysqli_connect_error() - Retorna uma descrição do último erro de conexão
- mysqli_errno() - Retorna o código de erro para a chamada de função mais recente
- mysqli_sqlstate() - Retorna o erro SQLSTATE da operação MySQL anterior

mysqli::$error_list

mysqli::$error_list

mysqli_error_list

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

mysqli::$error_list -- mysqli_error_list — Retorna uma lista de erros do último comando executado

Descrição

Estilo orientado a objetos

array *$mysqli->error_list*;

Estilo procedural

function **mysqli_error_list**(mysqli `$mysql`): array

Retorna um array de erros para a chamada de função MySQLi mais recente que pode ser bem-sucedida ou falhar.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Uma lista de erros, cada um como um array associativo contendo errno, error e sqlstate.

Examples

Exemplo #1 Exemplo de *$mysqli->error_list*

Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "nobody", "");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

if (!$mysqli->query("SET a=1")) {
    print_r($mysqli->error_list);
}

/* fecha a conexão */
$mysqli->close();
?>
```

Estilo procedural

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

if (!mysqli_query($link, "SET a=1")) {
    print_r(mysqli_error_list($link));
}

/* fecha a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Array
(
    [0] => Array
        (
            [errno] => 1193
            [sqlstate] => HY000
            [error] => Unknown system variable 'a'
        )

)
```

Veja também

- mysqli_connect_errno() - Retorna o código de erro da última chamada de conexão
- mysqli_connect_error() - Retorna uma descrição do último erro de conexão
- mysqli_error() - Retorna uma string descrevendo o último erro
- mysqli_sqlstate() - Retorna o erro SQLSTATE da operação MySQL anterior

mysqli::escape_string

mysqli::escape_string

mysqli_escape_string

(PHP 5, PHP 7, PHP 8)

mysqli::escape_string -- mysqli_escape_string — Sinônimo de mysqli_real_escape_string()

Descrição

Esta função é um apelido para: mysqli_real_escape_string().

mysqli::execute_query

mysqli::execute_query

mysqli_execute_query

(PHP 8 >= 8.2.0)

mysqli::execute_query -- mysqli_execute_query — Prepara, vincula parâmetros e executa a instrução SQL

Descrição

Estilo orientado a objetos

public function **mysqli::execute_query**(string `$query`, ?array `$params` = `null`): mysqli_result|bool

Estilo procedural

function **mysqli_execute_query**(mysqli `$mysql`, string `$query`, ?array `$params` = `null`): mysqli_result|bool

Prepara a consulta SQL, associa parâmetros e a executa. O método **mysqli::execute_query()** é um atalho para mysqli::prepare(), mysqli_stmt::bind_param(), mysqli_stmt::execute() e mysqli_stmt::get_result().

O modelo de instrução pode conter zero ou mais marcadores de parâmetro de ponto de interrogação (`?`) também chamados de espaços reservados. Os valores dos parâmetros devem ser fornecidos como um array usando o parâmetro `params`.

Uma instrução preparada é criada, mas nunca é exposta fora da função. É impossível acessar as propriedades da instrução como faria com o objeto mysqli_stmt. Devido a essa limitação, as informações de status são copiadas para o objeto mysqli e estão disponíveis usando seus métodos, por exemplo mysqli_affected_rows() ou mysqli_error().

> **Nota** :

> No caso em que uma instrução é passada para **mysqli_execute_query()** que é maior que `max_allowed_packet` do servidor, os códigos de erro retornados são diferentes dependendo do sistema operacional. O comportamento é o seguinte:
>
> - No Linux retorna um código de erro de 1153. A mensagem de erro significa "que o pacote é maior que `max_allowed_packet` bytes."
> - No Windows retorna um código de erro 2006. Essa mensagem de erro significa "que o servidor foi desativado".

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`query`

A consulta, como uma string. Deve consistir em uma única instrução SQL.

A instrução SQL pode conter zero ou mais marcadores de parâmetro representados por caracteres de ponto de interrogação (`?`) nas posições apropriadas.

> **Nota** :
>
> Os marcadores são válidos apenas em determinados locais nas instruções SQL. Por exemplo, eles são permitidos na lista `VALUES()` de uma instrução `INSERT` (para especificar valores de coluna para uma linha) ou em uma comparação com uma coluna em uma cláusula `WHERE` para especificar um valor de comparação. No entanto, eles não são permitidos para identificadores (como nomes de tabelas ou colunas).

`params`

Um array opcional com tantos elementos quantos forem os parâmetros vinculados na instrução SQL que está sendo executada. Cada valor é tratado como uma string.

Valor Retornado

Retorna `false` em caso de falha. Para consultas bem-sucedidas que produzem um conjunto de resultados, como `SELECT, SHOW, DESCRIBE` ou `EXPLAIN`, retorna um objeto mysqli_result. Para outras consultas bem-sucedidas, retorna `true`.

Examples

Exemplo #1 Exemplo de **mysqli::execute_query()**

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli('localhost', 'my_user', 'my_password', 'world');

$query = 'SELECT Name, District FROM City WHERE CountryCode=? ORDER BY Name LIMIT 5';
```

```php
$result = $mysqli->execute_query($query, ['DEU']);
foreach ($result as $row) {
    printf("%s (%s)\n", $row["Name"], $row["District"]);
}
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = 'SELECT Name, District FROM City WHERE CountryCode=? ORDER BY Name LIMIT 5';
$result = mysqli_execute_query($link, $query, ['DEU']);
foreach ($result as $row) {
    printf("%s (%s)\n", $row["Name"], $row["District"]);
}
```

Os exemplos acima produzirão algo semelhante a:

```
Aachen (Nordrhein-Westfalen)
Augsburg (Baijeri)
Bergisch Gladbach (Nordrhein-Westfalen)
Berlin (Berliini)
Bielefeld (Nordrhein-Westfalen)
```

Veja também

- mysqli_prepare() - Prepara uma instrução SQL para execução
- mysqli_stmt_execute() - Executa uma instrução preparada
- mysqli_stmt_bind_param() - Liga variáveis a uma declaração preparada como parâmetros
- mysqli_stmt_get_result() - Obtém um resultado de uma instrução preparada como um objeto mysqli_result

mysqli::$field_count

mysqli::$field_count

mysqli_field_count

(PHP 5, PHP 7, PHP 8)

mysqli::$field_count -- mysqli_field_count — Retorna o número de campos para a consulta mais recente

Descrição

Estilo orientado a objetos

int *$mysqli->field_count*;

Estilo procedural

function **mysqli_field_count**(mysqli `$mysql`): int

Retorna o número de campos para a consulta mais recente na conexão representada pelo parâmetro `mysql`. Esta função pode ser útil quando a função mysqli_store_result() é usada para determinar se a consulta deveria ou não ter produzido um conjunto não vazio de resultados sem conhecer a natureza da consulta.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Um inteiro representando o número de campos em um conjunto de resultados.

Examples

Exemplo #1 Exemplo de *$mysqli->field_count*

Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "meu_usuario", "minha_senha", "teste");

$mysqli->query( "DROP TABLE IF EXISTS amigos");
$mysqli->query( "CREATE TABLE amigos (id int, nome varchar(20))");

$mysqli->query( "INSERT INTO amigos VALUES (1,'Luiz'), (2, 'Ricardo')");


$mysqli->real_query("SELECT * FROM amigos");

if ($mysqli->field_count) {
    /* esta foi uma consulta de select/show ou describe */
    $result = $mysqli->store_result();

    /* processa o conjunto de resultados */
    $row = $result->fetch_row();

    /* libera o conjunto de resultados */
    $result->close();
}

/* fecha a conexão */
$mysqli->close();
?>
```

Estilo procedural

```php
<?php
$link = mysqli_connect("localhost", "meu_usuario", "minha_senha", "teste");

mysqli_query($link, "DROP TABLE IF EXISTS amigos");
mysqli_query($link, "CREATE TABLE amigos (id int, nome varchar(20))");

mysqli_query($link, "INSERT INTO amigos VALUES (1,'Luiz'), (2, 'Ricardo')");
```

```php
mysqli_real_query($link, "SELECT * FROM amigos");

if (mysqli_field_count($link)) {
    /* esta foi uma consulta de select/show ou describe */
    $result = mysqli_store_result($link);

    /* processa o conjunto de resultados */
    $row = mysqli_fetch_row($result);

    /* libera o conjunto de resultados */
    mysqli_free_result($result);
}

/* fecha a conexão */
mysqli_close($link);
?>
```

mysqli::get_charset

mysqli::get_charset

mysqli_get_charset

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

mysqli::get_charset -- mysqli_get_charset — Retorna um objeto de conjunto de caracteres

Descrição

Estilo orientado a objetos

public function **mysqli::get_charset**(): ?object

Estilo procedural

function **mysqli_get_charset**(mysqli `$mysql`): ?object

Retorna um objeto de conjunto de caracteres fornecendo várias propriedades do conjunto de caracteres ativo atual.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

A função retorna um objeto de conjunto de caracteres com as seguintes propriedades:

`charset`

Nome do conjunto de caracteres

`collation`

Nome do agrupamento

`dir`

Diretório do qual a descrição do conjunto de caracteres foi obtida ou "" para conjuntos de caracteres integrados

`min_length`

Comprimento mínimo de caracteres em bytes

`max_length`

Comprimento máximo de caracteres em bytes

`number`

Número interno do conjunto de caracteres

`state`

A partir do PHP 8.2.0, é sempre `1`

Examples

Exemplo #1 Exemplo de **mysqli::get_charset()**

Estilo orientado a objetos

```
<?php
  $db = mysqli_init();
  $db->real_connect("localhost","root","","test");
  $db->set_charset('latin1');
  var_dump($db->get_charset());
?>
```

Estilo procedural

```
<?php
  $db = mysqli_init();
  mysqli_real_connect($db, "localhost","root","","test");
  mysqli_set_charset($db, 'latin1');
  var_dump(mysqli_get_charset($db));
?>
```

Os exemplos acima produzirão:

```
object(stdClass)#2 (7) {
  ["charset"]=>
  string(6) "latin1"
  ["collation"]=>
  string(17) "latin1_swedish_ci"
  ["dir"]=>
  string(0) ""
  ["min_length"]=>
  int(1)
  ["max_length"]=>
```

```
  int(1)
  ["number"]=>
  int(8)
  ["state"]=>
  int(1)
}
```

Veja também

- mysqli_character_set_name() - Retorna o conjunto de caracteres atual da conexão com o banco de dados
- mysqli_set_charset() - Define o conjunto de caracteres do cliente

mysqli::$client_info

mysqli::$client_info

mysqli::get_client_info

mysqli_get_client_info

(PHP 5, PHP 7, PHP 8)

mysqli::$client_info -- mysqli::get_client_info -- mysqli_get_client_info — Obtém informação do cliente MySQL

Descrição

Estilo orientado a objetos

string *$mysqli->client_info*;
#[\Deprecated]
public function **mysqli::get_client_info**(): string

Estilo procedural

function **mysqli_get_client_info**(?mysqli `$mysql` = null): string

Retorna uma string que representa a versão de biblioteca do cliente MySQL.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Uma string que representa a versão da biblioteca do cliente MySQL.

Registro de Alterações

Versão	Descrição
8.1.0	Chamar **mysqli_get_client_info()** com o argumento `mysql` foi descontinuado. Esta função nunca requereu um parâmetro, mas incorretamente permitia este parâmetro opcional.
8.1.0	O método orientado a objeto **mysqli::get_client_info()** foi descontinuado.

Examples

Exemplo #1 mysqli_get_client_info

```php
<?php

/* Não é necessária uma conexão para determinar
   a versão da biblioteca do cliente mysql */

printf("Client library version: %s\n", mysqli_get_client_info());
?>
```

Veja também

- mysqli_get_client_version() - Retorna a versão do cliente MySQL como um inteiro
- mysqli_get_server_info() - Retorna a versão do servidor MySQL
- mysqli_get_server_version() - Retorna a versão do servidor MySQL como um inteiro

mysqli::$client_version

mysqli::$client_version

mysqli_get_client_version

(PHP 5, PHP 7, PHP 8)

mysqli::$client_version -- mysqli_get_client_version — Retorna a versão do cliente MySQL como um inteiro

Descrição

Estilo orientado a objetos

int *$mysqli->client_version*;

Estilo procedural

function **mysqli_get_client_version**(): int

Retorna a versão do cliente como um inteiro.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um número que representa a versão da biblioteca de cliente MySQL no formato: `versão_principal*10000 + versão_menor *100 + sub_versão`. Por exemplo, 4.1.0 é retornado como 40100.

Isto é útil para determinar rapidamente a versão da biblioteca de cliente para saber se alguma funcionalidade existe.

Examples

Exemplo #1 mysqli_get_client_version

```php
<?php

/* Não é necessária uma conexão para determinar
   a versão da biblioteca do cliente mysql */

printf("Client library version: %d\n", mysqli_get_client_version());
?>
```

Veja também

- mysqli_get_client_info() - Obtém informação do cliente MySQL
- mysqli_get_server_info() - Retorna a versão do servidor MySQL
- mysqli_get_server_version() - Retorna a versão do servidor MySQL como um inteiro

mysqli::get_connection_stats

mysqli::get_connection_stats

mysqli_get_connection_stats

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli::get_connection_stats -- mysqli_get_connection_stats — Retorna estatísticas sobre a conexão do cliente

Descrição

Estilo orientado a objetos

public function **mysqli::get_connection_stats**(): array

Estilo procedural

function **mysqli_get_connection_stats**(mysqli `$mysql`): array

Retorna estatísticas sobre a conexão do cliente.

> **Nota** :
>
> Disponível apenas com o mysqlnd.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Retorna um array com estatísticas de conexão.

Examples

Exemplo #1 Exemplo de mysqli_get_connection_stats()

```php
<?php
$link = mysqli_connect();
print_r(mysqli_get_connection_stats($link));
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [bytes_sent] => 43
    [bytes_received] => 80
    [packets_sent] => 1
    [packets_received] => 2
    [protocol_overhead_in] => 8
    [protocol_overhead_out] => 4
    [bytes_received_ok_packet] => 11
    [bytes_received_eof_packet] => 0
    [bytes_received_rset_header_packet] => 0
    [bytes_received_rset_field_meta_packet] => 0
    [bytes_received_rset_row_packet] => 0
    [bytes_received_prepare_response_packet] => 0
    [bytes_received_change_user_packet] => 0
    [packets_sent_command] => 0
    [packets_received_ok] => 1
    [packets_received_eof] => 0
    [packets_received_rset_header] => 0
    [packets_received_rset_field_meta] => 0
    [packets_received_rset_row] => 0
    [packets_received_prepare_response] => 0
    [packets_received_change_user] => 0
    [result_set_queries] => 0
    [non_result_set_queries] => 0
    [no_index_used] => 0
    [bad_index_used] => 0
    [slow_queries] => 0
    [buffered_sets] => 0
    [unbuffered_sets] => 0
    [ps_buffered_sets] => 0
    [ps_unbuffered_sets] => 0
    [flushed_normal_sets] => 0
    [flushed_ps_sets] => 0
    [ps_prepared_never_executed] => 0
    [ps_prepared_once_executed] => 0
    [rows_fetched_from_server_normal] => 0
    [rows_fetched_from_server_ps] => 0
    [rows_buffered_from_client_normal] => 0
    [rows_buffered_from_client_ps] => 0
    [rows_fetched_from_client_normal_buffered] => 0
    [rows_fetched_from_client_normal_unbuffered] => 0
    [rows_fetched_from_client_ps_buffered] => 0
    [rows_fetched_from_client_ps_unbuffered] => 0
    [rows_fetched_from_client_ps_cursor] => 0
    [rows_skipped_normal] => 0
    [rows_skipped_ps] => 0
    [copy_on_write_saved] => 0
    [copy_on_write_performed] => 0
```

```
    [command_buffer_too_small] => 0
    [connect_success] => 1
    [connect_failure] => 0
    [connection_reused] => 0
    [reconnect] => 0
    [pconnect_success] => 0
    [active_connections] => 1
    [active_persistent_connections] => 0
    [explicit_close] => 0
    [implicit_close] => 0
    [disconnect_close] => 0
    [in_middle_of_command_close] => 0
    [explicit_free_result] => 0
    [implicit_free_result] => 0
    [explicit_stmt_close] => 0
    [implicit_stmt_close] => 0
    [mem_emalloc_count] => 0
    [mem_emalloc_ammount] => 0
    [mem_ecalloc_count] => 0
    [mem_ecalloc_ammount] => 0
    [mem_erealloc_count] => 0
    [mem_erealloc_ammount] => 0
    [mem_efree_count] => 0
    [mem_malloc_count] => 0
    [mem_malloc_ammount] => 0
    [mem_calloc_count] => 0
    [mem_calloc_ammount] => 0
    [mem_realloc_count] => 0
    [mem_realloc_ammount] => 0
    [mem_free_count] => 0
    [proto_text_fetched_null] => 0
    [proto_text_fetched_bit] => 0
    [proto_text_fetched_tinyint] => 0
    [proto_text_fetched_short] => 0
    [proto_text_fetched_int24] => 0
    [proto_text_fetched_int] => 0
    [proto_text_fetched_bigint] => 0
    [proto_text_fetched_decimal] => 0
    [proto_text_fetched_float] => 0
    [proto_text_fetched_double] => 0
    [proto_text_fetched_date] => 0
    [proto_text_fetched_year] => 0
    [proto_text_fetched_time] => 0
    [proto_text_fetched_datetime] => 0
    [proto_text_fetched_timestamp] => 0
    [proto_text_fetched_string] => 0
    [proto_text_fetched_blob] => 0
    [proto_text_fetched_enum] => 0
    [proto_text_fetched_set] => 0
    [proto_text_fetched_geometry] => 0
    [proto_text_fetched_other] => 0
    [proto_binary_fetched_null] => 0
    [proto_binary_fetched_bit] => 0
    [proto_binary_fetched_tinyint] => 0
    [proto_binary_fetched_short] => 0
    [proto_binary_fetched_int24] => 0
    [proto_binary_fetched_int] => 0
    [proto_binary_fetched_bigint] => 0
    [proto_binary_fetched_decimal] => 0
```

```
    [proto_binary_fetched_float] => 0
    [proto_binary_fetched_double] => 0
    [proto_binary_fetched_date] => 0
    [proto_binary_fetched_year] => 0
    [proto_binary_fetched_time] => 0
    [proto_binary_fetched_datetime] => 0
    [proto_binary_fetched_timestamp] => 0
    [proto_binary_fetched_string] => 0
    [proto_binary_fetched_blob] => 0
    [proto_binary_fetched_enum] => 0
    [proto_binary_fetched_set] => 0
    [proto_binary_fetched_geometry] => 0
    [proto_binary_fetched_other] => 0
)
```

Veja também

- Stats description

mysqli::$host_info

mysqli::$host_info

mysqli_get_host_info

(PHP 5, PHP 7, PHP 8)

mysqli::$host_info -- mysqli_get_host_info — Retorna uma string representando o tipo de conexão utilizada

Descrição

Estilo orientado a objetos

string *$mysqli->host_info*;

Estilo procedural

function **mysqli_get_host_info**(mysqli `$mysql`): string

Retorna uma string descrevendo a conexão representada pelo parâmetro `mysql` (incluindo o nome de host servidor).

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Uma string de caracteres representando o nome de host do servidor e o tipo de conexão.

Examples

Exemplo #1 Exemplo de *$mysqli->host_info*

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* mostra informação do host */
printf("Host info: %s\n", $mysqli->host_info);
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* mostra informação do host */
printf("Host info: %s\n", mysqli_get_host_info($link));
```

Os exemplos acima produzirão:

```
Host info: Localhost via UNIX socket
```

Veja também

- mysqli_get_proto_info() - Retorna a versão do protocolo MySQL usada

mysqli::$protocol_version

mysqli::$protocol_version

mysqli_get_proto_info

(PHP 5, PHP 7, PHP 8)

mysqli::$protocol_version -- mysqli_get_proto_info — Retorna a versão do protocolo MySQL usada

Descrição

Estilo orientado a objetos

int *$mysqli->protocol_version*;

Estilo procedural

function **mysqli_get_proto_info**(mysqli `$mysql`): int

Retorna um inteiro representando a versão do protocolo MySQL usada pela conexão representada pelo parâmetro `mysql`.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Retorna um inteiro representando a versão do protocolo.

Examples

Exemplo #1 Exemplo de *$mysqli->protocol_version*

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password");

/* mostra a versão do protocolo */
printf("Versão do protocolo: %d\n", $mysqli->protocol_version);
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password");

/* mostra a versão do protocolo */
printf("Versão do protocolo: %d\n", mysqli_get_proto_info($link));
```

Os exemplos acima produzirão:

```
Versão do protocolo: 10
```

Veja também

- mysqli_get_host_info() - Retorna uma string representando o tipo de conexão utilizada

mysqli::$server_info

mysqli::$server_info

mysqli::get_server_info

mysqli_get_server_info

(PHP 5, PHP 7, PHP 8)

mysqli::$server_info -- mysqli::get_server_info -- mysqli_get_server_info — Retorna a versão do servidor MySQL

Descrição

Estilo orientado a objetos

string *$mysqli->server_info*;
public function **mysqli::get_server_info**(): string

Estilo procedural

function **mysqli_get_server_info**(mysqli `$mysql`): string

Retorna uma string representando a versão do servidor MySQL ao qual a extensão MySQLi está conectada.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Uma string de caracteres representando a versão do servidor.

Examples

Exemplo #1 Exemplo de *$mysqli->server_info*

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password");

/* print server version */
printf("Server version: %s\n", $mysqli->server_info);
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password");

/* print server version */
printf("Server version: %s\n", mysqli_get_server_info($link));
```

Os exemplos acima produzirão algo semelhante a:

```
Server version: 8.0.21
```

Veja também

- mysqli_get_client_info() - Obtém informação do cliente MySQL
- mysqli_get_client_version() - Retorna a versão do cliente MySQL como um inteiro
- mysqli_get_server_version() - Retorna a versão do servidor MySQL como um inteiro

mysqli::$server_version

mysqli::$server_version

mysqli_get_server_version

(PHP 5, PHP 7, PHP 8)

mysqli::$server_version -- mysqli_get_server_version — Retorna a versão do servidor MySQL como um inteiro

Descrição

Estilo orientado a objetos

int *$mysqli->server_version*;

Estilo procedural

function **mysqli_get_server_version**(mysqli `$mysql`): int

A função **mysqli_get_server_version()** retorna a versão do servidor ao qual se está conectado (representado pelo parâmetro `mysql`) como um inteiro.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Um inteiro representando a versão do servidor.

A forma deste número de versão é `versão_principal * 10000 + versão_menor * 100 + sub_versão` (ex. a versão 4.1.0 é 40100).

Examples

Exemplo #1 Exemplo de *$mysqli->server_version*

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password");

/* print server version */
printf("Server version: %d\n", $mysqli->server_version);
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password");

/* print server version */
printf("Server version: %d\n", mysqli_get_server_version($link));
```

Os exemplos acima produzirão algo semelhante a:

```
Server version: 80021
```

Veja também

- mysqli_get_client_info() - Obtém informação do cliente MySQL
- mysqli_get_client_version() - Retorna a versão do cliente MySQL como um inteiro
- mysqli_get_server_info() - Retorna a versão do servidor MySQL

mysqli::get_warnings

mysqli::get_warnings

mysqli_get_warnings

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

mysqli::get_warnings -- mysqli_get_warnings — Obtém o resultado de SHOW WARNINGS

Descrição

Estilo orientado a objetos

public function **mysqli::get_warnings**(): mysqli_warning|false

Estilo procedural

function **mysqli_get_warnings**(mysqli `$mysql`): mysqli_warning|false

Retorna uma lista vinculada individualmente composta por mysqli_warning ou **`false`** se não houver alertas. Cada objeto na lista corresponde a uma única linha do resultado de `SHOW WARNINGS`. Chamar mysqli_warning::next() irá recarregar o objeto com os valores da próxima linha.

> **Nota** :
>
> Para recuperar mensagens de alerta, é recomendado usar o comando SQL `SHOW WARNINGS [limit row_count]` em vez desta função.

> **Aviso**
>
> A lista vinculada não pode ser retrocedida ou recuperada novamente.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Retorna uma lista vinculada individualmente composta por mysqli_warning ou `false` se não houver alertas.

Examples

Exemplo #1 Percorrendo a lista vinculada para buscar todos os alertas

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$mysqli->query("SELECT 1/0, CAST('NULL' AS UNSIGNED)");

if ($mysqli->warning_count > 0) {
    $warning = $mysqli->get_warnings();
    if ($warning !== false) {
        do {
            printf("Número do erro: %s\n", $warning->errno);
            printf("Mensagem: %s\n", $warning->message);
        } while ($warning->next());
    }
}
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "user", "password", "test");

mysqli_query($link, "SELECT 1/0, CAST('NULL' AS UNSIGNED)");

if (mysqli_warning_count($link) > 0) {
    $warning = mysqli_get_warnings($link);
    if ($warning !== false) {
        do {
            printf("Número do erro: %s\n", $warning->errno);
            printf("Mensagem: %s\n", $warning->message);
        } while ($warning->next());
    }
}
```

Os exemplos acima produzirão:

```
Número do erro: 1365
Mensagem: Division by 0
Número do erro: 1292
Mensagem: Truncated incorrect INTEGER value: 'NULL'
```

mysqli::$info

mysqli::$info

mysqli_info

(PHP 5, PHP 7, PHP 8)

mysqli::$info -- mysqli_info — Recupera informação sobre a recente consulta executada

Descrição

Estilo orientado a objetos

?string *$mysqli->info*;

Estilo procedural

function **mysqli_info**(mysqli `$mysql`): ?string

A função **mysqli_info()** retorna uma string contendo informação sobre a última consulta executada. A natureza desta string é fornecida abaixo:

Possíveis valores de retorno de mysqli_info

Tipo de consulta	Exemplo da string de resultado
INSERT INTO...SELECT...	Records: 100 Duplicates: 0 Warnings: 0
INSERT INTO...VALUES (...),(...),(...)	Records: 3 Duplicates: 0 Warnings: 0
LOAD DATA INFILE ...	Records: 1 Deleted: 0 Skipped: 0 Warnings: 0
ALTER TABLE ...	Records: 3 Duplicates: 0 Warnings: 0
UPDATE ...	Rows matched: 40 Changed: 40 Warnings: 0

> **Nota** :
>
> Consultas que não se encaixam em um dos formatos acima não são suportadas. Nesta situação, **mysqli_info()** irá retornar uma string vazia.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Uma string de caracteres representando informação adicional sobre a mais recente consulta executada.

Examples

Exemplo #1 Exemplo de *$mysqli->info*

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$mysqli->query("CREATE TEMPORARY TABLE t1 LIKE City");

/* INSERT INTO ... SELECT */
$mysqli->query("INSERT INTO t1 SELECT * FROM City ORDER BY ID LIMIT 150");
printf("%s\n", $mysqli->info);
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

mysqli_query($link, "CREATE TEMPORARY TABLE t1 LIKE City");

/* INSERT INTO ... SELECT */
mysqli_query($link, "INSERT INTO t1 SELECT * FROM City ORDER BY ID LIMIT 150");
printf("%s\n", mysqli_info($link));
```

Os exemplos acima produzirão:

```
Records: 150  Duplicates: 0  Warnings: 0
```

Veja também

- mysqli_affected_rows() - Retorna o número de linhas afetadas pela operação MySQL anterior
- mysqli_warning_count() - Retorna o número de alertas gerados pela consulta mais recente
- mysqli_num_rows() - Obtém o número de linhas no resultado

mysqli::init

mysqli::init

mysqli_init

(PHP 5, PHP 7, PHP 8)

mysqli::init -- mysqli_init — Inicializa o MySQLi e retorna um objeto a ser utilizado com mysqli_real_connect()

Descrição

Estilo orientado a objetos

#[\Deprecated]
public function **mysqli::init**(): ?bool

Estilo procedural

function **mysqli_init**(): mysqli|false

Aloca ou inicializa um objeto MYSQL adequado para as funções mysqli_options() e mysqli_real_connect().

> **Nota** :
>
> Chamadas subsequentes a quaisquer funções mysqli (exceto mysqli_options() e mysqli_ssl_set()) irão falhar até que mysqli_real_connect() seja chamada.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

mysqli::init() retorna `null` em caso de sucesso, ou `false` em caso de falha. **mysqli_init()** retorna um objeto em caso de sucesso, ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.1.0	O método **mysqli::init()** com estilo orientado a objeto foi descontinuado. Substitua as chamadas a **parent::init()** por **parent::__construct()**.

Examples

Consulte mysqli_real_connect().

Veja também

- mysqli_options() - Define opções
- mysqli_close() - Fecha uma conexão ao banco de dados previamente aberta
- mysqli_real_connect() - Abre uma conexão a um servidor MySQL
- mysqli_connect() - Sinônimo de mysqli::__construct

mysqli::$insert_id

mysqli::$insert_id

mysqli_insert_id

(PHP 5, PHP 7, PHP 8)

mysqli::$insert_id -- mysqli_insert_id — Retorna o valor gerado para uma coluna AUTO_INCREMENT pela última consulta

Descrição

Estilo orientado a objetos

int|string *$mysqli->insert_id*;

Estilo procedural

function **mysqli_insert_id**(mysqli `$mysql`): int|string

Retorna o ID gerado por uma consulta `INSERT` ou `UPDATE` em uma tabela que tenha o atributo `AUTO_INCREMENT`. No caso de uma declaração `INSERT` de múltiplas linhas, ela retorna o primeiro valor gerado automaticamente que foi inserido com sucesso.

Realizar uma declaração `INSERT` ou `UPDATE` usando a função MYSQL `LAST_INSERT_ID()` também irá modificar o valor retornado por **mysqli_insert_id()**. Se `LAST_INSERT_ID(expr)` foi usado para gerar o valor de `AUTO_INCREMENT`, ela retorna o valor da última `expr` no lugar do valor `AUTO_INCREMENT` gerado.

Retorna `0` se a declaração anterior não modificou um valor `AUTO_INCREMENT`. **mysqli_insert_id()** deve ser chamada imediatamente após a declaração que gerou o valor.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

O valor do campo `AUTO_INCREMENT` que foi atualizado pela consulta anterior. Retorna zero se não houve consulta anterior na conexão ou se a consulta não atualizou um valor `AUTO_INCREMENT`.

Apenas declarações emitidas com a conexão atual afetam o valor de retorno. O valor não é afetado por declarações emitidas usando outras conexões ou clientes.

> **Nota** :
>
> Se o número é maior que o máximo valor de um inteiro, ele será retornado como uma string.

Examples

Exemplo #1 Exemplo de *$mysqli->insert_id*

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$mysqli->query("CREATE TABLE myCity LIKE City");
```

```php
$query = "INSERT INTO myCity VALUES (NULL, 'Stuttgart', 'DEU', 'Stuttgart', 617000)";
$mysqli->query($query);

printf("Novo registro tem ID %d.\n", $mysqli->insert_id);

/* deleta a tabela */
$mysqli->query("DROP TABLE myCity");
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

mysqli_query($link, "CREATE TABLE myCity LIKE City");

$query = "INSERT INTO myCity VALUES (NULL, 'Stuttgart', 'DEU', 'Stuttgart', 617000)";
mysqli_query($link, $query);

printf("Novo registro tem ID %d.\n", mysqli_insert_id($link));

/* deleta a tabela */
mysqli_query($link, "DROP TABLE myCity");
```

Os exemplos acima produzirão:

```
Novo registro tem ID 1.
```

mysqli::kill

mysqli::kill

mysqli_kill

(PHP 5, PHP 7, PHP 8)

mysqli::kill -- mysqli_kill — Solicita ao servidor o encerramento de um thread do MySQL

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.4.0. O uso desta função é fortemente desencorajado.

Descrição

Estilo orientado a objetos

#[\Deprecated]
public function **mysqli::kill**(int `$process_id`): bool

Estilo procedural

#[\Deprecated]
function **mysqli_kill**(mysqli `$mysql`, int `$process_id`): bool

Esta função é usada para solicitar ao servidor o encerramento de uma thread do MySQL especificada pelo parâmetro `process_id`. Este valor deve ser recuperado chamando a função mysqli_thread_id().

Para encerrar a execução de uma consulta, você deve usar o comando SQL `KILL QUERY processid`.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**`MYSQLI_REPORT_ERROR`**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **`MYSQLI_REPORT_STRICT`**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Registro de Alterações

Versão	Descrição
8.4.0	O método **mysqli::kill()** e a função **mysqli_kill()** foram descontinuados. No lugar deles, deve ser usado o comando SQL `KILL`.

Examples

Exemplo #1 Exemplo de **mysqli::kill()**

Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* verifica conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

/* determina o identificador da thread */
$thread_id = $mysqli->thread_id;

/* Encerra a conexão */
$mysqli->kill($thread_id);

/* Isto deve gerar um erro */
if (!$mysqli->query("CREATE TABLE myCity LIKE City")) {
    printf("Error: %s\n", $mysqli->error);
    exit;
```

```
}

/* fecha a conexão */
$mysqli->close();
?>
```

Estilo procedural

```
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* verifica conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

/* determina o identificador da thread */
$thread_id = mysqli_thread_id($link);

/* Encerra a conexão */
mysqli_kill($link, $thread_id);

/* Isto deve gerar um erro */
if (!mysqli_query($link, "CREATE TABLE myCity LIKE City")) {
    printf("Error: %s\n", mysqli_error($link));
    exit;
}

/* fecha a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Error: MySQL server has gone away
```

Veja também

- mysqli_thread_id() - Retorna o ID da thread para a conexão atual

mysqli::more_results

mysqli::more_results

mysqli_more_results

(PHP 5, PHP 7, PHP 8)

mysqli::more_results -- mysqli_more_results — Verifica se há mais algum resultado de uma consulta múltipla

Descrição

Estilo orientado a objetos

public function **mysqli::more_results**(): bool

Estilo procedural

function **mysqli_more_results**(mysqli `$mysql`): bool

Indica se um ou mais conjuntos de resultados estão disponíveis de uma chamada anterior a mysqli_multi_query().

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Retorna **`true`** se um ou mais conjuntos de resultados (incluindo erros) estão disponíveis de uma chamada anterior a mysqli_multi_query(), caso contrário **`false`**.

Examples

Consulte mysqli_multi_query().

Veja também

- mysqli_multi_query() - Executa uma ou mais consultas no banco de dados
- mysqli_next_result() - Prepara o próximo resultado de multi_query
- mysqli_store_result() - Transfere um conjunto de resultados da última consulta
- mysqli_use_result() - Inicia uma recuperação de conjunto de resultados

mysqli::multi_query

mysqli::multi_query

mysqli_multi_query

(PHP 5, PHP 7, PHP 8)

mysqli::multi_query -- mysqli_multi_query — Executa uma ou mais consultas no banco de dados

Descrição

Estilo orientado a objetos

public function **mysqli::multi_query**(string `$query`): bool

Estilo procedural

function **mysqli_multi_query**(mysqli `$mysql`, string `$query`): bool

Executa uma ou múltiplas consultas que são concatenadas com ponto-e-vírgula.

> **Aviso**
>
> **Alerta de segurança: injeção de SQL**

> Se a consulta contiver qualquer entrada de variável, instruções preparadas parametrizadas devem ser usadas. Alternativamente, os dados devem ser formatados corretamente e todas as strings devem ser escapadas usando a função mysqli_real_escape_string().

Consultas são enviadas em uma chamada única ao banco de dados e processadas sequencialmente. **mysqli_multi_query()** espera pela finalização da primeira consulta antes de retornar o controle ao PHP. Enquanto isso, o servidor MySQL continuará processando as consultas remanescentes de forma assíncrona a partir do PHP e disponibilizará os resultados para busca.

É recomendado usar do-while para processar consultas múltiplas. A conexão estará ocupada enquanto todas as consultas tiverem concluído e seus resultados enviados ao PHP. Nenhuma outra declaração pode ser emitida na mesma conexão até que todas as consultas sejam processadas. Para proceder à consulta seguinte, use mysqli_next_result(). Se a consulta seguinte ainda não estiver pronta, mysqli esperará pela resposta do servidor MySQL. Para verificar se há mais resultados, use mysqli_more_results().

Para consultas que produzem um conjnto de resultados, como em `SELECT, SHOW, DESCRIBE` ou `EXPLAIN`, mysqli_use_result() ou mysqli_store_result() podem ser usadas para recuperar o conjunto. Para consultas que não produzem um conjunto de resultados, as mesmas funções podem ser usadas para recuperar informação como o número de linhas afetadas.

> **Dica**
>
> Executar declarações `CALL` para procedimentos armazenados pode produzir múltiplos conjuntos de resultados. Se o procedimento armazenado contiver declarações `SELECT`, os conjuntos são retornados na ordem em que são produzidos enquanto o procedimento é executado. Em geral, a função que chama não sabe quantos conjuntos de resultados um procedimento irá retornar e deve estar preparada para recuperar múltiplos conjuntos. O resultado final do procedimento é um resultado de estado que não inclui nenhum conjunto. O estado indica se o procedimento teve sucesso ou se ocorreu um erro.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`query`

Uma string contendo as consultas que serão executadas. Consultas múltiplas devem ser separadas por ponto-e-vírgula (;).

Valor Retornado

Retorna **`false`** se a primeira consulta falhar. Para recuperar erros subsequentes de outras consultas, deve-se chamar mysqli_next_result() primeiro.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**`MYSQLI_REPORT_ERROR`**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **`MYSQLI_REPORT_STRICT`**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Exemplo de mysqli::multi_query()

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT CURRENT_USER();";
$query .= "SELECT Name FROM City ORDER BY ID LIMIT 20, 5";

/* executa as consultas múltiplas */
$mysqli->multi_query($query);
do {
    /* armazena o conjunto de resultados no PHP */
    if ($result = $mysqli->store_result()) {
        while ($row = $result->fetch_row()) {
            printf("%s\n", $row[0]);
        }
    }
    /* divisor de impressão */
    if ($mysqli->more_results()) {
        printf("-----------------\n");
    }
} while ($mysqli->next_result());
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT CURRENT_USER();";
$query .= "SELECT Name FROM City ORDER BY ID LIMIT 20, 5";

/* executa as consultas múltiplas */
mysqli_multi_query($link, $query);
do {
    /* armazena o conjunto de resultados no PHP */
    if ($result = mysqli_store_result($link)) {
        while ($row = mysqli_fetch_row($result)) {
            printf("%s\n", $row[0]);
        }
    }
    /* divisor de impressão */
    if (mysqli_more_results($link)) {
        printf("-----------------\n");
    }
} while (mysqli_next_result($link));
```

Os exemplos acima produzirão algo semelhante a:

```
my_user@localhost
-----------------
Amersfoort
Maastricht
Dordrecht
Leiden
Haarlemmermeer
```

Veja também

- mysqli_query() - Executa uma consulta no banco de dados
- mysqli_use_result() - Inicia uma recuperação de conjunto de resultados
- mysqli_store_result() - Transfere um conjunto de resultados da última consulta
- mysqli_next_result() - Prepara o próximo resultado de multi_query
- mysqli_more_results() - Verifica se há mais algum resultado de uma consulta múltipla

mysqli::next_result

mysqli::next_result

mysqli_next_result

(PHP 5, PHP 7, PHP 8)

mysqli::next_result -- mysqli_next_result — Prepara o próximo resultado de multi_query

Descrição

Estilo orientado a objetos

public function **mysqli::next_result**(): bool

Estilo procedural

function **mysqli_next_result**(mysqli `$mysql`): bool

Prepara o próximo conjunto de resultado de uma chamada anterior a mysqli_multi_query() que pode ser recuperado por mysqli_store_result() ou mysqli_use_result().

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha. Também retorna **`false`** se a declaração seguinte resultou em erro, diferentemente de mysqli_more_results().

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**MYSQLI_REPORT_ERROR**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **MYSQLI_REPORT_STRICT**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Consulte mysqli_multi_query().

Veja também

- mysqli_multi_query() - Executa uma ou mais consultas no banco de dados
- mysqli_more_results() - Verifica se há mais algum resultado de uma consulta múltipla
- mysqli_store_result() - Transfere um conjunto de resultados da última consulta
- mysqli_use_result() - Inicia uma recuperação de conjunto de resultados

mysqli::options

mysqli::options

mysqli_options

(PHP 5, PHP 7, PHP 8)

mysqli::options -- mysqli_options — Define opções

Descrição

Estilo orientado a objetos

public function **mysqli::options**(int `$option`, string|int `$value`): bool

Estilo procedural

function **mysqli_options**(mysqli `$mysql`, int `$option`, string|int `$value`): bool

Usado para definir opções extras de conexão e afetar o comportamento de uma conexão.

Esta função pode ser chamada várias vezes para definir várias opções.

mysqli_options() deve ser chamado depois de mysqli_init() e antes de mysqli_real_connect().

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`option`

A opção que você deseja definir. Pode ser um dos seguintes valores:

Valid options

Name	Description
MYSQLI_OPT_CONNECT_TIMEOUT	Tempo limite de conexão em segundos
MYSQLI_OPT_READ_TIMEOUT	Tempo limite do resultado da execução do comando em segundos. Disponível a partir do PHP 7.2.0.

Name	Description
MYSQLI_OPT_LOCAL_INFILE	Ativar/desativar o uso de LOAD LOCAL INFILE
MYSQLI_OPT_LOAD_DATA_LOCAL_DIR	Diretório a ser usado para LOAD DATA LOCAL INFILE.
MYSQLI_INIT_COMMAND	Comando para executar após estabelecer conexão ao servidor MySQL
MYSQLI_SET_CHARSET_NAME	O conjunto de caracteres a ser definido como padrão..
MYSQLI_READ_DEFAULT_FILE	Lê as opções do arquivo de opções nomeadas em vez de *my.cnf* Não suportado por mysqlnd.
MYSQLI_READ_DEFAULT_GROUP	Leia as opções do grupo nomeado de *my.cnf* ou o arquivo especificado com **MYSQL_READ_DEFAULT_FILE**. Não suportado pelo mysqlnd.
MYSQLI_SERVER_PUBLIC_KEY	Arquivo de chave pública RSA usado com a autenticação baseada em SHA-256.
MYSQLI_OPT_NET_CMD_BUFFER_SIZE	O tamanho do buffer interno de comando/rede. Válido apenas para mysqlnd.
MYSQLI_OPT_NET_READ_BUFFER_SIZE	Tamanho máximo do bloco de leitura em bytes ao ler o corpo de um pacote de comando do MySQL. Válido apenas para mysqlnd.
MYSQLI_OPT_INT_AND_FLOAT_NATIVE	Converte colunas de números inteiros e pontos flutuantes de volta para números PHP ao usar instruções não preparadas. Válido apenas para mysqlnd.
MYSQLI_OPT_SSL_VERIFY_SERVER_CERT	Informa se o certificado do servidor deve ser verificado.
MYSQLI_OPT_CAN_HANDLE_EXPIRED_PASSWORDS	Informa se o cliente deve aceitar senhas expiradas.
1	Informa se deve ser usada compressão na comunicação de rede.

`value`

O valor da opção.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (`MYSQLI_REPORT_ERROR`) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como `MYSQLI_REPORT_STRICT`, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Veja mysqli_real_connect().

Notas

> **Nota** :

> O MySQLnd sempre assume o conjunto de caracteres padrão do servidor. Este conjunto de caracteres é enviado durante a criação/autenticação da conexão e será usado pelo mysqlnd.
>
> A libmysqlclient usa o conjunto de caracteres padrão definido no arquivo *my.cnf* ou definido por uma chamada explícita a **mysqli_options()** antes de chamar mysqli_real_connect(), mas depois de chamar mysqli_init().

Veja também

- mysqli_init() - Inicializa o MySQLi e retorna um objeto a ser utilizado com mysqli_real_connect()
- mysqli_real_connect() - Abre uma conexão a um servidor MySQL

mysqli::ping

mysqli::ping

mysqli_ping

(PHP 5, PHP 7, PHP 8)

mysqli::ping -- mysqli_ping — Faz ping em uma conexão de servidor ou tenta reconectar se a conexão cair

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.4.0. O uso desta função é fortemente desencorajado.

Descrição

Estilo orientado a objetos

```
#[\Deprecated]
public function mysqli::ping(): bool
```

Estilo procedural

```
#[\Deprecated]
function mysqli_ping(mysqli $mysql): bool
```

Verifica se a conexão com o servidor está funcionando. Se ele estiver inativo e a opção global mysqli.reconnect estiver habilitada, uma reconexão automática será tentada.

> **Nota** :
>
> A configuração mysqli.reconnect do *php.ini* é ignorada pelo driver mysqlnd, então a reconexão automática nunca é tentada.

Esta função pode ser utilizada por clientes que ficam muito tempo ociosos, para verificar se o servidor encerrou a conexão e reconectar se necessário.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (`MYSQLI_REPORT_ERROR`) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como `MYSQLI_REPORT_STRICT`, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Registro de Alterações

Versão	Descrição
8.4.0	O método **mysqli::ping()** e a função **mysqli_ping()** foram descontinuados. A partir do PHP 8.2.0, o recurso de `reconnect` (reconexão) não está mais disponível, tornando esta função obsoleta.

Examples

Exemplo #1 Exemplo de mysqli::ping()

Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "meu_usuario", "minha_senha", "mundo");

/* verifica a conexão */
if ($mysqli->connect_errno) {
    printf("Falha na conexão: %s\n", $mysqli->connect_error);
    exit();
}

/* verifica se o servidor está ativo */
if ($mysqli->ping()) {
    printf ("Nossa conexão está ok!\n");
} else {
    printf ("Erro: %s\n", $mysqli->error);
}

/* encerra a conexão */
$mysqli->close();
?>
```

Estilo procedural

```php
<?php
$link = mysqli_connect("localhost", "meu_usuario", "minha_senha", "mundo");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Falha na conexão: %s\n", mysqli_connect_error());
    exit();
}

/* verifica se o servidor está ativo */
if (mysqli_ping($link)) {
```

```
    printf ("Nossa conexão está ok!\n");
} else {
    printf ("Erro: %s\n", mysqli_error($link));
}

/* encerra a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Nossa conexão está ok!
```

mysqli::poll

mysqli::poll

mysqli_poll

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli::poll -- mysqli_poll — Sondagem de conexões

Descrição

Estilo orientado a objetos

public static function **mysqli::poll**(
    ?array &$read,
    ?array &$error,
    array &$reject,
    int $seconds,
    int $microseconds = 0
): int|false

Estilo procedural

function **mysqli_poll**(
    ?array &$read,
    ?array &$error,
    array &$reject,
    int $seconds,
    int $microseconds = 0
): int|false

Faz sondagem de conexões. Este método pode ser usado como static.

> **Nota** :

> Disponível apenas com o mysqlnd.

Parâmetros

`read`

Lista de conexões a verificar por resultados marcantes que possam ser lidos.

`error`

Lista de conexões nas quais um erro ocorreu, por exemplo, falha na consulta ou conexão perdida.

`reject`

Lista de conexões rejeitadas porque nenhuma consulta assícrona foi executada para que a função pudesse sondar os resultados.

`seconds`

Número máximo de segundos a esperar, deve ser não negativo.

`microseconds`

Número máximo de microssegundos a esperar, deve ser não negativo.

Valor Retornado

Retorna o número de conexões prontas em caso de sucesso, `false` caso contrário.

Erros/Exceções

Uma exceção ValueError é lançada quando nem o parâmetro `read` nem o parâmetro `error` são passados.

Registro de Alterações

Versão	Descrição
8.3.0	Agora lança uma exceção ValueError quando nem o parâmetro `read` nem o parâmetro `error` são passados.

Examples

Exemplo #1 Um exemplo de **mysqli_poll()**

```
<?php
$link1 = mysqli_connect();
$link1->query("SELECT 'test'", MYSQLI_ASYNC);
$all_links = array($link1);
$processed = 0;
do {
    $links = $errors = $reject = array();
    foreach ($all_links as $link) {
        $links[] = $errors[] = $reject[] = $link;
    }
    if (!mysqli_poll($links, $errors, $reject, 1)) {
        continue;
    }
```

```
    foreach ($links as $link) {
        if ($result = $link->reap_async_query()) {
            print_r($result->fetch_row());
            if (is_object($result))
                mysqli_free_result($result);
        } else die(sprintf("MySQLi Error: %s", mysqli_error($link)));
        $processed++;
    }
} while ($processed < count($all_links));
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => test
)
```

Veja também

- mysqli_query() - Executa uma consulta no banco de dados
- mysqli_reap_async_query() - Obtenha o resultado da consulta assíncrona

mysqli::prepare

mysqli::prepare

mysqli_prepare

(PHP 5, PHP 7, PHP 8)

mysqli::prepare -- mysqli_prepare — Prepara uma instrução SQL para execução

Descrição

Estilo orientado a objetos

public function **mysqli::prepare**(string `$query`): mysqli_stmt|false

Estilo procedural

function **mysqli_prepare**(mysqli `$mysql`, string `$query`): mysqli_stmt|false

Prepara a consulta SQL e retorna um identificador de instrução a ser usado para outras operações na instrução. A consulta deve consistir em uma única instrução SQL.

O modelo de instrução pode conter zero ou mais marcadores de parâmetro de ponto de interrogação (`?`) também chamados de espaços reservados. Os marcadores de parâmetro devem ser vinculados às variáveis do aplicativo usando mysqli_stmt_bind_param() antes de executar a instrução.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`query`

A consulta, como uma string. Deve consistir em uma única instrução SQL.

A instrução SQL pode conter zero ou mais marcadores de parâmetro representados por caracteres de ponto de interrogação (`?`) nas posições apropriadas.

> **Nota** :
>
> Os marcadores são válidos apenas em determinados locais nas instruções SQL. Por exemplo, eles são permitidos na lista `VALUES()` de uma instrução `INSERT` (para especificar valores de coluna para uma linha) ou em uma comparação com uma coluna em uma cláusula `WHERE` para especificar um valor de comparação. No entanto, eles não são permitidos para identificadores (como nomes de tabelas ou colunas).

Valor Retornado

mysqli_prepare() retorna um objeto de instrução ou `false` se ocorreu um erro.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (`MYSQLI_REPORT_ERROR`) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como `MYSQLI_REPORT_STRICT`, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Exemplo de **mysqli::prepare()**

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$city = "Amersfoort";

/* criar uma declaração preparada */
$stmt = $mysqli->prepare("SELECT District FROM City WHERE Name=?");

/* parâmetros de ligação para marcadores */
$stmt->bind_param("s", $city);

/* executar consulta */
$stmt->execute();

/* vincular variáveis de resultado */
$stmt->bind_result($district);

/* buscar valor */
$stmt->fetch();
```

```
printf("%s está no distrito %s\n", $city, $district);
```

Estilo procedural

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$city = "Amersfoort";

/* criar uma declaração preparada */
$stmt = mysqli_prepare($link, "SELECT District FROM City WHERE Name=?");

/* parâmetros de ligação para marcadores */
mysqli_stmt_bind_param($stmt, "s", $city);

/* executar consulta */
mysqli_stmt_execute($stmt);

/* vincular variáveis de resultado */
mysqli_stmt_bind_result($stmt, $district);

/* buscar valor */
mysqli_stmt_fetch($stmt);

printf("%s está no distrito %s\n", $city, $district);
```

Os exemplos acima produzirão:

```
Amersfoort está no distrito de Utrecht
```

Veja também

- mysqli_stmt_execute() - Executa uma instrução preparada
- mysqli_stmt_fetch() - Recebe resultados de uma instrução preparada nas variáveis vinculadas
- mysqli_stmt_bind_param() - Liga variáveis a uma declaração preparada como parâmetros
- mysqli_stmt_bind_result() - Liga variáveis a uma declaração preparada para armazenamento de resultado
- mysqli_stmt_get_result() - Obtém um resultado de uma instrução preparada como um objeto mysqli_result
- mysqli_stmt_close() - Fecha uma instrução preparada

mysqli::query

mysqli::query

mysqli_query

(PHP 5, PHP 7, PHP 8)

mysqli::query -- mysqli_query — Executa uma consulta no banco de dados

Descrição

Estilo orientado a objetos

public function **mysqli::query**(string `$query`, int `$result_mode` = **`MYSQLI_STORE_RESULT`**): mysqli_result|bool

Estilo procedural

function **mysqli_query**(mysqli `$mysql`, string `$query`, int `$result_mode` = **`MYSQLI_STORE_RESULT`**): mysqli_result|bool

Executa uma `query` no banco de dados.

> **Aviso**
>
> ### Alerta de segurança: injeção de SQL
>
> Se a consulta contiver qualquer entrada de variável, instruções preparadas parametrizadas devem ser usadas. Alternativamente, os dados devem ser formatados corretamente e todas as strings devem ser escapadas usando a função mysqli_real_escape_string().

Para consultas não-DML (não INSERT, UPDATE ou DELETE), esta função é semelhante a chamar mysqli_real_query() seguido por mysqli_use_result() ou mysqli_store_result().

> **Nota** :
>
> No caso em que uma instrução é passada para **mysqli_query()** que é maior que `max_allowed_packet` do servidor, os códigos de erro retornados são diferentes dependendo se você está usando MySQL Native Driver (`mysqlnd`)ou MySQL Client Library (`libmysqlclient`). O comportamento é o seguinte:
>
> - `mysqlnd` no Linux retorna um código de erro de 1153. A mensagem de erro significa que "recebi um pacote maior que `max_allowed_packet` bytes".
> - `mysqlnd` no Windows retorna um código de erro 2006. Esta mensagem de erro significa que o "server has gone away".
> - `libmysqlclient` em todas as plataformas retorna um código de erro 2006. Essa mensagem de erro significa que o "servidor foi desativado".

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`query`

A string de consulta.

`result_mode`

O modo de resultado pode ser uma das 3 constantes que indicam como o resultado será retornado do servidor MySQL.

`MYSQLI_STORE_RESULT` (padrão) - retorna um objeto mysqli_resultcom conjunto de resultados em buffer.

`MYSQLI_USE_RESULT` - retorna um objeto mysqli_result com conjunto de resultados sem buffer. Enquanto houver registros pendentes esperando para serem buscados, a linha de conexão estará ocupada e todas as chamadas subseqüentes retornarão o erro `Commands out of sync`. Para evitar o erro, todos os registros devem ser buscados no servidor ou o conjunto de resultados deve ser descartado chamando mysqli_free_result().

`MYSQLI_ASYNC` (disponível com mysqlnd) - a consulta é executada de forma assíncrona e nenhum conjunto de resultados é retornado imediatamente. mysqli_poll() é então usado para obter resultados de tais consultas. Usado em combinação com a constante `MYSQLI_STORE_RESULT` ou `MYSQLI_USE_RESULT`.

Valor Retornado

Retorna `false` em caso de falha. Para consultas bem-sucedidas que produzem um conjunto de resultados, como `SELECT`, `SHOW`, `DESCRIBE` ou `EXPLAIN`, **mysqli_query()** retornará um objeto mysqli_result. Para outras consultas bem-sucedidas, **mysqli_query()** retornará `true`.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (`MYSQLI_REPORT_ERROR`) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como `MYSQLI_REPORT_STRICT`, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Exemplo de **mysqli::query()**

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* Criar tabela não retorna um conjunto de resultados */
$mysqli->query("CREATE TEMPORARY TABLE myCity LIKE City");
printf("Tabela myCity criada com sucesso.\n");

/* Consultas selecionadas retornam um conjunto de resultados */
$result = $mysqli->query("SELECT Name FROM City LIMIT 10");
printf("Select retornou %d linhas.\n", $result->num_rows);

/* Se tivermos que recuperar uma grande quantidade de dados, usamos MYSQLI_USE_RESULT */
$result = $mysqli->query("SELECT * FROM City", MYSQLI_USE_RESULT);

/* Observe que não podemos executar nenhuma função que interaja com o
servidor até que todos os registros tenham sido totalmente recuperados ou o resultado
conjunto foi fechado. Todas as chamadas retornarão um erro 'fora de sincronia' */
$mysqli->query("SET @a:='isso não vai funcionar'");
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");
```

```php
/* Criar tabela não retorna um conjunto de resultados */
mysqli_query($link, "CREATE TEMPORARY TABLE myCity LIKE City");
printf("Tabela myCity criada com sucesso.\n");

/* Consultas de seleção retornam um conjunto de resultados */
$result = mysqli_query($link, "SELECT Name FROM City LIMIT 10");
printf("Select retornou %d linhas.\n", mysqli_num_rows($result));

/* Se tivermos que recuperar uma grande quantidade de dados, usamos MYSQLI_USE_RESULT */
$result = mysqli_query($link, "SELECT * FROM City", MYSQLI_USE_RESULT);

/* Observe que não podemos executar nenhuma função que interaja com o
servidor até que todos os registros tenham sido totalmente recuperados ou o resultado
conjunto foi fechado. Todas as chamadas retornarão um erro 'fora de sincronia' */
mysqli_query($link, "SET @a:='isso não vai funcionar'");
```

Os exemplos acima produzirão algo semelhante a:

```
Tabela myCity criada com sucesso.
Select retornou 10 linhas.

Fatal error: Uncaught mysqli_sql_exception: Commands out of sync; you can't run this command now
in...
```

Veja também

- mysqli_execute_query() - Prepara, vincula parâmetros e executa a instrução SQL
- mysqli_real_query() - Executa uma consulta SQL
- mysqli_multi_query() - Executa uma ou mais consultas no banco de dados
- mysqli_prepare() - Prepara uma instrução SQL para execução
- mysqli_free_result() - Libera memória associada a um resultado

mysqli::quote_string

mysqli::quote_string

mysqli_quote_string

(PHP 8 >= 8.6.0)

mysqli::quote_string -- mysqli_quote_string — Quotes and escapes a string for use in an SQL statement

Descrição

Estilo orientado a objetos

public function **mysqli::quote_string**(string `$string`): string

Estilo procedural

function **mysqli_quote_string**(mysqli `$mysql`, string `$string`): string

Creates a legal SQL string literal that can be used in an SQL statement. The given string is escaped in the same way as mysqli::real_escape_string(), taking into account the current character set of the connection, and the result is wrapped in single quotes. The returned value must therefore be inserted into the query without adding any further quotes around it.

Unlike mysqli::real_escape_string(), the result is also safe when the `NO_BACKSLASH_ESCAPES` SQL mode is enabled on the connection.

> **Cuidado**
>
> **Security: the default character set**
>
> The character set must be set either at the server level, or with the API function mysqli_set_charset() for it to affect **mysqli_quote_string()**. See the concepts section on character sets for more information.

> **Nota** :
>
> Quoting is only suitable for string literals. It does not make identifiers, such as table or column names, safe. Whenever possible, use prepared statements instead of building queries manually.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`string`

The string to be quoted and escaped.

Characters encoded are `NUL (ASCII 0)`, `\n`, `\r`, `\`, `'`, `"`, and `CTRL+Z`. If the `NO_BACKSLASH_ESCAPES` SQL mode is enabled on the connection, only `'` is encoded, by doubling it.

Valor Retornado

Returns the escaped string enclosed in single quotes.

Examples

Exemplo #1 **mysqli::quote_string()** example

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$city = "'s-Hertogenbosch";

/* this query with quoted $city will work */
$query = sprintf("SELECT CountryCode FROM City WHERE name=%s", $mysqli->quote_string($city));
$result = $mysqli->query($query);
printf("Select returned %d rows.\n", $result->num_rows);
```

```
/* this query will fail, because we didn't quote and escape $city */
$query = sprintf("SELECT CountryCode FROM City WHERE name='%s'", $city);
$result = $mysqli->query($query);
```

Estilo procedural

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$city = "'s-Hertogenbosch";

/* this query with quoted $city will work */
$query = sprintf("SELECT CountryCode FROM City WHERE name=%s", mysqli_quote_string($mysqli, $city));
$result = mysqli_query($mysqli, $query);
printf("Select returned %d rows.\n", mysqli_num_rows($result));

/* this query will fail, because we didn't quote and escape $city */
$query = sprintf("SELECT CountryCode FROM City WHERE name='%s'", $city);
$result = mysqli_query($mysqli, $query);
```

Os exemplos acima produzirão algo semelhante a:

```
Select returned 1 rows.

Fatal error: Uncaught mysqli_sql_exception: You have an error in your SQL syntax; check the manual
that corresponds to your MySQL server version for the right syntax to use near 's-Hertogenbosch'' at
line 1 in...
```

Veja também

- mysqli::real_escape_string() - Escape caracteres especiais em uma string para uso em uma instrução SQL, levando em consideração o conjunto de caracteres atual da conexão
- mysqli::set_charset() - Define o conjunto de caracteres do cliente

mysqli::real_connect

mysqli::real_connect

mysqli_real_connect

(PHP 5, PHP 7, PHP 8)

mysqli::real_connect -- mysqli_real_connect — Abre uma conexão a um servidor MySQL

Descrição

Estilo orientado a objetos

```
public function mysqli::real_connect(
    ?string $hostname = null,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?string $database = null,
    ?int $port = null,
    ?string $socket = null,
    int $flags = 0
): bool
```

Estilo procedural

```
function mysqli_real_connect(
    mysqli $mysql,
    ?string $hostname = null,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?string $database = null,
    ?int $port = null,
    ?string $socket = null,
    int $flags = 0
): bool
```

Abre uma conexão a um servidor de banco de dados MySQL com várias opções de conexão.

Esta função difere de mysqli_connect():

- **mysqli_real_connect()** precisa de um objeto válido que deve ser criado por mysqli_init().
- Existe um parâmetro `flags`.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`hostname`

Pode ser um nome de host ou um endereço IP. Ao passar `null`, o valor é recuperado de mysqli.default_host. Quando possível, serão usados pipes em vez do protocolo TCP/IP. O protocolo TCP/IP é usado se um nome de host e um número de porta forem fornecidos juntos, por ex. `localhost:3308`.

`username`

O nome de usuário do MySQL, ou `null` para assumir o nome de usuário com base na opção ini mysqli.default_user.

`password`

A senha do MySQL, ou `null` para assumir a senha com base na opção ini mysqli.default_pw.

`database`

O banco de dados padrão a ser usado ao realizar consultas ou `null`.

`port`

O número da porta para tentar conectar ao servidor MySQL ou **`null`** para assumir a porta com base na opção mysqli.default_port.

`socket`

O socket ou pipe nomeado que deve ser usado ou **`null`** para assumir o socket com base na opção ini mysqli.default_socket.

> **Nota** :
>
> A especificação do parâmetro `socket` não determinará explicitamente o tipo de conexão a ser usada ao conectar-se ao servidor MySQL. A forma como a conexão é feita com o banco de dados MySQL é determinada pelo parâmetro `hostname`.

`flags`

Com o parâmetro `flags`, pode-se definir diferentes opções de conexão:

Opções suportadas

Nome	Descrição
`MYSQLI_CLIENT_COMPRESS`	Usa protocolo de compressão
`MYSQLI_CLIENT_FOUND_ROWS`	Retorna o número de linhas correspondentes, não o número de linhas afetadas
`MYSQLI_CLIENT_IGNORE_SPACE`	Permite espaços após os nomes das funções. Faz com que todos os nomes de funções sejam palavras reservadas.
`MYSQLI_CLIENT_INTERACTIVE`	Permite segundos de `interactive_timeout` (em vez de segundos de `wait_timeout`) de inatividade antes de fechar a conexão
`MYSQLI_CLIENT_SSL`	Usa SSL (criptografia)
`MYSQLI_CLIENT_SSL_DONT_VERIFY_SERVER_CERT`	Como **`MYSQLI_CLIENT_SSL`**, mas desabilita a validação do certificado SSL fornecido. Isso é apenas para instalações usando o driver nativo do MySQL e MySQL 5.6 ou posterior.

> **Nota** :
>
> Por motivos de segurança, a opção **`MULTI_STATEMENT`** não é suportado no PHP. Se for desejado executar várias consultas, use a função mysqli_multi_query().

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**`MYSQLI_REPORT_ERROR`**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **`MYSQLI_REPORT_STRICT`**, uma exceção

mysqli_sql_exception será lançada em vez do aviso.

Registro de Alterações

Versão	Descrição
7.4.0	Todos os parâmetros agora são anuláveis.

Examples

Exemplo #1 Exemplo de **mysqli::real_connect()**

Estilo orientado a objetos

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_init();

$mysqli->options(MYSQLI_INIT_COMMAND, 'SET AUTOCOMMIT = 0');
$mysqli->options(MYSQLI_OPT_CONNECT_TIMEOUT, 5);

$mysqli->real_connect('localhost', 'my_user', 'my_password', 'my_db', null, null,
MYSQLI_CLIENT_COMPRESS|MYSQLI_CLIENT_FOUND_ROWS);

echo 'Sucesso... ' . $mysqli->host_info . "\n";
```

Estilo orientado a objetos ao estender a classe mysqli

```
<?php

class foo_mysqli extends mysqli {
    public function __construct($host, $user, $pass, $db)
    {
        parent::__construct();

        parent::options(MYSQLI_INIT_COMMAND, 'SET AUTOCOMMIT = 0');
        parent::options(MYSQLI_OPT_CONNECT_TIMEOUT, 5);

        parent::real_connect($host, $user, $pass, $db, null, null,
MYSQLI_CLIENT_COMPRESS|MYSQLI_CLIENT_FOUND_ROWS);
    }
}

$db = new foo_mysqli('localhost', 'my_user', 'my_password', 'my_db');

echo 'Sucesso... ' . $db->host_info . "\n";
```

Estilo procedural

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_init();

mysqli_options($link, MYSQLI_INIT_COMMAND, 'SET AUTOCOMMIT = 0');
mysqli_options($link, MYSQLI_OPT_CONNECT_TIMEOUT, 5);
```

```php
mysqli_real_connect($link, 'localhost', 'my_user', 'my_password', 'my_db', null, null,
MYSQLI_CLIENT_COMPRESS|MYSQLI_CLIENT_FOUND_ROWS);

echo 'Sucesso... ' . mysqli_get_host_info($link) . "\n";
```

Os exemplos acima produzirão:

```
Sucesso... MySQL host info: localhost via TCP/IP
```

Notas

> **Nota** :
>
> O MySQLnd sempre assume o conjunto de caracteres padrão do servidor. Este conjunto de caracteres é enviado durante a criação/autenticação da conexão e será usado pelo mysqlnd.
>
> A libmysqlclient usa o conjunto de caracteres padrão definido no arquivo *my.cnf* ou definido por uma chamada explícita a mysqli_options() antes de chamar **mysqli_real_connect()**, mas depois de chamar mysqli_init().

Veja também

- mysqli_connect() - Sinônimo de mysqli::__construct
- mysqli_init() - Inicializa o MySQLi e retorna um objeto a ser utilizado com mysqli_real_connect()
- mysqli_options() - Define opções
- mysqli_ssl_set() - Usado para estabelecer conexões seguras usando SSL
- mysqli_close() - Fecha uma conexão ao banco de dados previamente aberta

mysqli::real_escape_string

mysqli::real_escape_string

mysqli_real_escape_string

(PHP 5, PHP 7, PHP 8)

mysqli::real_escape_string -- mysqli_real_escape_string — Escape caracteres especiais em uma string para uso em uma instrução SQL, levando em consideração o conjunto de caracteres atual da conexão

Descrição

Estilo orientado a objetos

public function **mysqli::real_escape_string**(string `$string`): string

Estilo procedural

function **mysqli_real_escape_string**(mysqli `$mysql`, string `$string`): string

Esta função é usada para criar uma string SQL legal que você pode usar em uma instrução SQL. A string fornecida é codificada para produzir uma string SQL com escape, levando em consideração o conjunto de caracteres atual da conexão.

> **Cuidado**
>
> **Segurança: o conjunto de caracteres padrão**
>
> O conjunto de caracteres deve ser definido no nível do servidor ou com a função API mysqli_set_charset() para que afete **mysqli_real_escape_string()**. Consulte a seção de conceitos sobre conjuntos de caracteres para obter mais informações.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`string`

A string a ser escapada.

Os caracteres codificados são `NUL (ASCII 0)`, `\n`, `\r`, `\`, `'`, `"`, e `CTRL+Z`.

Valor Retornado

Retorna uma string com escape.

Examples

Exemplo #1 Exemplo de mysqli::real_escape_string()

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$city = "'s-Hertogenbosch";

/* esta consulta com escape $city funcionará */
$query = sprintf("SELECT CountryCode FROM City WHERE name='%s'",
$mysqli->real_escape_string($city));
$result = $mysqli->query($query);
printf("Select retornou %d linhas.\n", $result->num_rows);

/* esta consulta falhará, porque não escapamos de $city */
$query = sprintf("SELECT CountryCode FROM City WHERE name='%s'", $city);
$result = $mysqli->query($query);
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
```

```php
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$city = "'s-Hertogenbosch";

/* esta consulta com escape $city funcionará */
$query = sprintf("SELECT CountryCode FROM City WHERE name='%s'",
mysqli_real_escape_string($mysqli, $city));
$result = mysqli_query($mysqli, $query);
printf("Select retornou %d linhas.\n", mysqli_num_rows($result));

/* esta consulta falhará, porque não escapamos de $city*/
$query = sprintf("SELECT CountryCode FROM City WHERE name='%s'", $city);
$result = mysqli_query($mysqli, $query);
```

Os exemplos acima produzirão algo semelhante a:

```
Select retornou 1 linhas.

Fatal error: Uncaught mysqli_sql_exception: You have an error in your SQL syntax; check the manual
that corresponds to your MySQL server version for the right syntax to use near 's-Hertogenbosch'' at
line 1 in...
```

Veja também

- mysqli_set_charset() - Define o conjunto de caracteres do cliente

mysqli::real_query

mysqli::real_query

mysqli_real_query

(PHP 5, PHP 7, PHP 8)

mysqli::real_query -- mysqli_real_query — Executa uma consulta SQL

Descrição

Estilo orientado a objetos

public function **mysqli::real_query**(string `$query`): bool

Estilo procedural

function **mysqli_real_query**(mysqli `$mysql`, string `$query`): bool

Executa uma única consulta no banco de dados cujo resultado pode ser recuperado ou armazenado usando as funções mysqli_store_result() ou mysqli_use_result().

> **Aviso**
>
> **Alerta de segurança: injeção de SQL**

> Se a consulta contiver qualquer entrada de variável, instruções preparadas parametrizadas devem ser usadas. Alternativamente, os dados devem ser formatados corretamente e todas as strings devem ser escapadas usando a função mysqli_real_escape_string().

Para determinar se uma determinada consulta deve retornar um conjunto de resultados ou não, consulte mysqli_field_count().

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`query`

A string de consulta.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**`MYSQLI_REPORT_ERROR`**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **`MYSQLI_REPORT_STRICT`**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Veja também

- mysqli_query() - Executa uma consulta no banco de dados
- mysqli_store_result() - Transfere um conjunto de resultados da última consulta
- mysqli_use_result() - Inicia uma recuperação de conjunto de resultados

mysqli::reap_async_query

mysqli::reap_async_query

mysqli_reap_async_query

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli::reap_async_query -- mysqli_reap_async_query — Obtenha o resultado da consulta assíncrona

Descrição

Estilo orientado a objetos

public function **mysqli::reap_async_query**(): mysqli_result|bool

Estilo procedural

function **mysqli_reap_async_query**(mysqli `$mysql`): mysqli_result|bool

Obtenha o resultado da consulta assíncrona.

> **Nota** :
>
> Disponível apenas com o mysqlnd.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Retorna `false` em caso de falha. Para consultas bem-sucedidas que produzem um conjunto de resultados, como `SELECT, SHOW, DESCRIBE` ou `EXPLAIN`, **mysqli_reap_async_query()** retornará um objeto mysqli_result. Para outras consultas bem-sucedidas, **mysqli_reap_async_query()** retornará `true`.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (`MYSQLI_REPORT_ERROR`) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como `MYSQLI_REPORT_STRICT`, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Veja também

- mysqli_poll() - Sondagem de conexões

mysqli::refresh

mysqli::refresh

mysqli_refresh

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli::refresh -- mysqli_refresh — Atualiza

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.4.0. O uso desta função é fortemente desencorajado.

Descrição

Estilo orientado a objetos

#[\Deprecated]
public function **mysqli::refresh**(int `$flags`): bool

Estilo procedural

#[\Deprecated]
function **mysqli_refresh**(mysqli `$mysql`, int `$flags`): bool

Libera tabelas ou caches ou redefine as informações do servidor de replicação.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`flags`

As opções para atualizar, usando as constantes MYSQLI_REFRESH_* conforme documentado na documentação das MySQLi constantes.

Veja também a documentação oficial do » MySQL Refresh.

Valor Retornado

`true` se a atualização foi bem-sucedida, caso contrário, `false`

Registro de Alterações

Versão	Descrição
8.4.0	O método **mysqli::refresh()** e a função **mysqli_refresh()** foram descontinuados. No lugar deles, use o comando SQL `FLUSH`.

Veja também

- mysqli_poll() - Sondagem de conexões

mysqli::release_savepoint

mysqli::release_savepoint

mysqli_release_savepoint

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

mysqli::release_savepoint -- mysqli_release_savepoint — Remove o ponto de salvamento nomeado do conjunto de pontos de salvamento da transação atual

Descrição

Estilo orientado a objetos

public function **mysqli::release_savepoint**(string `$name`): bool

Estilo procedural:

function **mysqli_release_savepoint**(mysqli `$mysql`, string `$name`): bool

Esta função é idêntica à execução de `$mysqli->query("RELEASE SAVEPOINT `$name`");`. Esta função não aciona commit ou rollback.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`name`

O identificador do ponto de salvamento.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Veja também

- mysqli_savepoint() - Define um ponto de salvamento de transação nomeado

mysqli::rollback

mysqli::rollback

mysqli_rollback

(PHP 5, PHP 7, PHP 8)

mysqli::rollback -- mysqli_rollback — Reverte a transação atual

Descrição

Estilo orientado a objetos

public function **mysqli::rollback**(int `$flags` = 0, ?string `$name` = **`null`**): bool

Estilo procedural

function **mysqli_rollback**(mysqli `$mysql`, int `$flags` = 0, ?string `$name` = **`null`**): bool

Reverte a transação atual para o banco de dados.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`flags`

Uma máscara de bits das constantes **`MYSQLI_TRANS_COR_*`**.

`name`

Se fornecido, então `ROLLBACK/*name*/` é executado.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**`MYSQLI_REPORT_ERROR`**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **`MYSQLI_REPORT_STRICT`**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Registro de Alterações

Versão	Descrição
8.0.0	`name` agora é anulável.

Examples

Veja o exemplo mysqli::begin_transaction().

Notas

> **Nota** :
>
> Esta função não funciona com tipos de tabelas não transacionais (como MyISAM ou ISAM).

Veja também

- mysqli_begin_transaction() - Inicia uma transação
- mysqli_commit() - Grava a transação atual
- mysqli_autocommit() - Ativa ou desativa a gravação automática das modificações no banco de dados
- mysqli_release_savepoint() - Remove o ponto de salvamento nomeado do conjunto de pontos de salvamento da transação atual

mysqli::savepoint

mysqli::savepoint

mysqli_savepoint

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

mysqli::savepoint -- mysqli_savepoint — Define um ponto de salvamento de transação nomeado

Descrição

Estilo orientado a objetos

public function **mysqli::savepoint**(string `$name`): bool

Estilo procedural:

function **mysqli_savepoint**(mysqli `$mysql`, string `$name`): bool

Esta função é idêntica à execução de `$mysqli->query("SAVEPOINT `$name`");`

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`name`

O identificador do ponto de salvamento.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Veja também

- mysqli_release_savepoint() - Remove o ponto de salvamento nomeado do conjunto de pontos de salvamento da transação atual

mysqli::select_db

mysqli::select_db

mysqli_select_db

(PHP 5, PHP 7, PHP 8)

mysqli::select_db -- mysqli_select_db — Seleciona o banco de dados padrão para consultas de banco de dados

Descrição

Estilo orientado a objetos

public function **mysqli::select_db**(string `$database`): bool

Estilo procedural

function **mysqli_select_db**(mysqli `$mysql`, string `$database`): bool

Seleciona o banco de dados padrão a ser usado ao executar consultas na conexão do banco de dados.

> **Nota** :
>
> Esta função deve ser usada apenas para alterar o banco de dados padrão para a conexão. Você pode selecionar o banco de dados padrão com o 4º parâmetro em mysqli_connect().

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`database`

O nome do banco de dados.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (`MYSQLI_REPORT_ERROR`) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como `MYSQLI_REPORT_STRICT`, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Exemplo de **mysqli::select_db()**

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "test");

/* obtem o nome do banco de dados padrão atual */
$result = $mysqli->query("SELECT DATABASE()");
$row = $result->fetch_row();
printf("O banco de dados padrão é %s.\n", $row[0]);

/* altere o banco de dados padrão para "mundo" */
$mysqli->select_db("world");

/* obtem o nome do banco de dados padrão atual */
$result = $mysqli->query("SELECT DATABASE()");
$row = $result->fetch_row();
printf("O banco de dados padrão é %s.\n", $row[0]);
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "test");

/* obter o nome do banco de dados padrão atual */
$result = mysqli_query($link, "SELECT DATABASE()");
$row = mysqli_fetch_row($result);
printf("O banco de dados padrão és %s.\n", $row[0]);

/* altere o banco de dados padrão para "mundo" */
mysqli_select_db($link, "world");

/* obtem o nome do banco de dados padrão atual */
$result = mysqli_query($link, "SELECT DATABASE()");
$row = mysqli_fetch_row($result);
printf("O banco de dados padrão é %s.\n", $row[0]);
```

Os exemplos acima produzirão:

```
O banco de dados padrão é test.
O banco de dados padrão é world.
```

Veja também

- mysqli_connect() - Sinônimo de mysqli::__construct
- mysqli_real_connect() - Abre uma conexão a um servidor MySQL

mysqli::set_charset

mysqli::set_charset

mysqli_set_charset

(PHP 5 >= 5.0.5, PHP 7, PHP 8)

mysqli::set_charset -- mysqli_set_charset — Define o conjunto de caracteres do cliente

Descrição

Estilo orientado a objetos

public function **mysqli::set_charset**(string `$charset`): bool

Estilo procedural

function **mysqli_set_charset**(mysqli `$mysql`, string `$charset`): bool

Define o conjunto de caracteres a ser usado ao enviar dados de e para o servidor de banco de dados.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`charset`

O conjunto de caracteres desejado.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**`MYSQLI_REPORT_ERROR`**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **`MYSQLI_REPORT_STRICT`**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Exemplo de **mysqli::set_charset()**

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "test");

printf("Conjunto de caracteres inicial: %s\n", $mysqli->character_set_name());
```

```php
/* alterar conjunto de caracteres para utf8mb4 */
$mysqli->set_charset("utf8mb4");

printf("Conjunto de caracteres atual: %s\n", $mysqli->character_set_name());
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect('localhost', 'my_user', 'my_password', 'test');

printf("Conjunto de caracteres inicial: %s\n", mysqli_character_set_name($link));

/* alterar conjunto de caracteres para utf8mb4 */
mysqli_set_charset($link, "utf8mb4");

printf("Conjunto de caracteres atual: %s\n", mysqli_character_set_name($link));
```

Os exemplos acima produzirão algo semelhante a:

```
Conjunto de caracteres inicial: latin1
Conjunto de caracteres atual: utf8mb4
```

Notas

> **Nota** :
>
> Esta é a maneira preferida de alterar o conjunto de caracteres. Usar mysqli_query() para configurá-lo (como `SET NAMES utf8`) não é recomendado. Consulte a seção de conceitos do conjunto de caracteres do MySQL para obter mais informações.

Veja também

- mysqli_character_set_name() - Retorna o conjunto de caracteres atual da conexão com o banco de dados
- mysqli_real_escape_string() - Escape caracteres especiais em uma string para uso em uma instrução SQL, levando em consideração o conjunto de caracteres atual da conexão
- Conceitos do conjunto de caracteres do MySQL
- » Lista de conjuntos de caracteres que o MySQL suporta

mysqli::set_opt

mysqli::set_opt

mysqli_set_opt

(PHP 5, PHP 7, PHP 8)

mysqli::set_opt -- mysqli_set_opt — Sinônimo de mysqli_options()

Descrição

Esta função é um apelido para: mysqli_options().

mysqli::$sqlstate

mysqli::$sqlstate

mysqli_sqlstate

(PHP 5, PHP 7, PHP 8)

mysqli::$sqlstate -- mysqli_sqlstate — Retorna o erro SQLSTATE da operação MySQL anterior

Descrição

Estilo orientado a objetos

string *$mysqli->sqlstate*;

Estilo procedural

function **mysqli_sqlstate**(mysqli `$mysql`): string

Retorna uma string contendo o código de erro SQLSTATE para o último erro. O código de erro consiste em cinco caracteres. `'00000'` significa nenhum erro. Os valores são especificados por ANSI SQL e ODBC. Para obter uma lista de valores possíveis, consulte » http://dev.mysql.com/doc/mysql/en/error-handling.html.

> **Nota** :
>
> Observe que nem todos os erros do MySQL ainda estão mapeados para SQLSTATE. O valor `HY000` (erro geral) é usado para erros não mapeados.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Retorna uma string contendo o código de erro SQLSTATE para o último erro. O código de erro consiste em cinco caracteres. `'00000'` significa nenhum erro.

Examples

Exemplo #1 Exemplo de *$mysqli->sqlstate*

Estilo orientado a objetos

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* Table City já existe, então devemos receber um erro */
```

```
try {
    $mysqli->query("CREATE TABLE City (ID INT, Name VARCHAR(30))");
} catch (mysqli_sql_exception) {
    printf("Erro - SQLSTATE %s.\n", $mysqli->sqlstate);
}
```

Estilo procedural

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* Table City já existe, então devemos receber um erro */
try {
    mysqli_query($link, "CREATE TABLE City (ID INT, Name VARCHAR(30))");
} catch (mysqli_sql_exception) {
    printf("Erro - SQLSTATE %s.\n", mysqli_sqlstate($link));
}
```

Os exemplos acima produzirão:

```
Erro - SQLSTATE 42S01.
```

Veja também

- mysqli_errno() - Retorna o código de erro para a chamada de função mais recente
- mysqli_error() - Retorna uma string descrevendo o último erro

mysqli::ssl_set

mysqli::ssl_set

mysqli_ssl_set

(PHP 5, PHP 7, PHP 8)

mysqli::ssl_set -- mysqli_ssl_set — Usado para estabelecer conexões seguras usando SSL

Descrição

Estilo orientado a objetos

public function **mysqli::ssl_set**(
    ?string $key,
    ?string $certificate,
    ?string $ca_certificate,
    ?string $ca_path,
    ?string $cipher_algos
): true

Estilo procedural

```
function mysqli_ssl_set(
    mysqli $mysql,
    ?string $key,
    ?string $certificate,
    ?string $ca_certificate,
    ?string $ca_path,
    ?string $cipher_algos
): true
```

Usado para estabelecer conexões seguras usando SSL. Deve ser chamado antes de mysqli_real_connect(). Esta função não faz nada, a menos que o suporte OpenSSL esteja ativado.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`key`

O nome do caminho para o arquivo de chave.

`certificate`

O nome do caminho para o arquivo de certificado.

`ca_certificate`

O nome do caminho para o arquivo de autoridade de certificação.

`ca_path`

O nome do caminho para um diretório que contém certificados SSL CA confiáveis no formato PEM.

`cipher_algos`

Uma lista de cifras permitidas a serem usadas para criptografia SSL.

Valor Retornado

Sempre retorna **`true`**. Se a configuração do SSL estiver incorreta, mysqli_real_connect() retornará um erro quando você tentar se conectar.

Veja também

- mysqli_options() - Define opções
- mysqli_real_connect() - Abre uma conexão a um servidor MySQL

mysqli::stat

mysqli::stat

mysqli_stat

(PHP 5, PHP 7, PHP 8)

mysqli::stat -- mysqli_stat — Obtém o status atual do sistema

Descrição

Estilo orientado a objetos

public function **mysqli::stat**(): string|false

Estilo procedural

function **mysqli_stat**(mysqli `$mysql`): string|false

mysqli_stat() retorna uma string contendo informações semelhantes àquelas fornecidas pelo comando 'mysqladmin status'. Isso inclui o tempo de atividade em segundos e o número de threads em execução, perguntas, recarregamentos e tabelas abertas.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Uma string que descreve o status do servidor. `false` se ocorreu um erro.

Examples

Exemplo #1 Exemplo de **mysqli::stat()**

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

printf ("Status do sistema: %s\n", $mysqli->stat());
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

printf("System status: %s\n", mysqli_stat($link));
```

Os exemplos acima produzirão:

```
Status do sistema: Uptime: 272  Threads: 1  Questions: 5340  Slow queries: 0
Opens: 13  Flush tables: 1  Open tables: 0  Queries per second avg: 19.632
Memory in use: 8496K  Max memory used: 8560K
```

Veja também

- mysqli_get_server_info() - Retorna a versão do servidor MySQL

mysqli::stmt_init

mysqli::stmt_init

mysqli_stmt_init

(PHP 5, PHP 7, PHP 8)

mysqli::stmt_init -- mysqli_stmt_init — Inicializa uma instrução e retorna um objeto para uso com mysqli_stmt_prepare

Descrição

Estilo orientado a objetos

public function **mysqli::stmt_init**(): mysqli_stmt|false

Estilo procedural

function **mysqli_stmt_init**(mysqli `$mysql`): mysqli_stmt|false

Aloca e inicializa um objeto de instrução adequado para mysqli_stmt_prepare().

> **Nota** :
>
> Qualquer chamada subseqüente para qualquer método mysqli_stmt falhará até que mysqli_stmt_prepare() seja chamado.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Retorna um objeto mysqli_stmt.

Veja também

- mysqli_stmt_prepare() - Prepara uma declaração SQL para execução

mysqli::store_result

mysqli::store_result

mysqli_store_result

(PHP 5, PHP 7, PHP 8)

mysqli::store_result -- mysqli_store_result — Transfere um conjunto de resultados da última consulta

Descrição

Estilo orientado a objetos

public function **mysqli::store_result**(int `$mode` = 0): mysqli_result|false

Estilo procedural

function **mysqli_store_result**(mysqli `$mysql`, int `$mode` = 0): mysqli_result|false

Transfere o conjunto de resultados da última consulta na conexão do banco de dados representada pelo parâmetro `mysql` para ser usado com a função mysqli_data_seek().

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`mode`

A opção que você deseja definir. A partir do PHP 8.1, este parâmetro não tem efeito. Pode ser um dos seguintes valores:

Valid options

Nome	Descrição
`MYSQLI_STORE_RESULT_COPY_DATA`	Copia os resultados do buffer mysqlnd interno nas variáveis PHP buscadas. Por padrão, o mysqlnd usará uma lógica de referência para evitar copiar e duplicar os resultados mantidos na memória. Para certos conjuntos de resultados, por exemplo, conjuntos de resultados com muitas linhas pequenas, a abordagem de cópia pode reduzir o uso geral de memória porque as variáveis PHP que contêm resultados podem ser liberadas mais cedo (disponível apenas com mysqlnd)

Valor Retornado

Retorna um objeto de resultado armazenado em buffer ou `false` se ocorreu um erro.

> **Nota** :
>
> **mysqli_store_result()** retorna `false` caso a consulta não retorne um conjunto de resultados (se a consulta for, por exemplo, uma instrução INSERT). Esta função também retorna `false` se a leitura do conjunto de resultados falhou. Você pode verificar se houve um erro verificando se mysqli_error() não retorna uma string vazia, se mysqli_errno() retorna um valor diferente de zero ou se mysqli_field_count() retorna um valor diferente de zero. Além disso, a possível razão para esta função retornar `false` após uma chamada bem-sucedida para mysqli_query() pode ser um conjunto de resultados muito grande (a memória para ele não pode ser alocada). Se mysqli_field_count() retornar um valor diferente de zero, a instrução deve ter produzido um conjunto de resultados não vazio.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (`MYSQLI_REPORT_ERROR`) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como `MYSQLI_REPORT_STRICT`, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Registro de Alterações

Versão	Descrição
8.4.0	Passar o parâmetro `mode` foi descontinuado. O parâmetro já não tem efeito a partir do PHP 8.1.0.

Examples

Veja mysqli_multi_query().

Notas

> **Nota** :
>
> Embora seja sempre uma boa prática liberar a memória usada pelo resultado de uma consulta usando a função mysqli_free_result(), ao transferir grandes conjuntos de resultados usando **mysqli_store_result()** isso se torna particularmente importante.

Veja também

- mysqli_real_query() - Executa uma consulta SQL
- mysqli_use_result() - Inicia uma recuperação de conjunto de resultados

mysqli::$thread_id

mysqli::$thread_id

mysqli_thread_id

(PHP 5, PHP 7, PHP 8)

mysqli::$thread_id -- mysqli_thread_id — Retorna o ID da thread para a conexão atual

Descrição

Estilo orientado a objetos

int *$mysqli->thread_id*;

Estilo procedural

function **mysqli_thread_id**(mysqli `$mysql`): int

A função **mysqli_thread_id()** retorna o ID da thread para a conexão atual que pode então ser eliminada usando a função mysqli_kill(). Se a conexão for perdida e você reconectar com mysqli_ping(), o ID do thread será diferente. Portanto, o ID do thread deve ser obtido somente quando necessário.

> **Nota** :
>
> O ID do encadeamento é atribuído conexão por conexão. Portanto, se a conexão for interrompida e restabelecida, um novo ID de thread será atribuído.

> Para eliminar uma consulta em execução, pode ser usado o comando SQL `KILL QUERY id_processo`.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Retorna o ID do thread para a conexão atual.

Examples

Exemplo #1 Exemplo de *$mysqli->thread_id*

Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* verifique a conexão */
if (mysqli_connect_errno()) {
    printf("Falha na conexão: %s\n", mysqli_connect_error());
    exit();
}

/* determine nosso id de thread */
$thread_id = $mysqli->thread_id;

/* Matar conexão */
$mysqli->kill($thread_id);

/* Isso deve produzir um erro */
if (!$mysqli->query("CREATE TABLE myCity LIKE City")) {
    printf("Erro: %s\n", $mysqli->error);
    exit;
}

/* fecha a conexão */
$mysqli->close();
?>
```

Estilo procedural

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* verifique a conexão*/
if (mysqli_connect_errno()) {
    printf("Falha na conexão: %s\n", mysqli_connect_error());
    exit();
}

/* determine nosso id de thread */
$thread_id = mysqli_thread_id($link);
```

```php
/* Matar conexão */
mysqli_kill($link, $thread_id);

/* Isso deve produzir um erro */
if (!mysqli_query($link, "CREATE TABLE myCity LIKE City")) {
    printf("Erro: %s\n", mysqli_error($link));
    exit;
}

/* fecha a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Erro: MySQL server has gone away
```

Veja também

- mysqli_kill() - Solicita ao servidor o encerramento de um thread do MySQL

mysqli::thread_safe

mysqli::thread_safe

mysqli_thread_safe

(PHP 5, PHP 7, PHP 8)

mysqli::thread_safe -- mysqli_thread_safe — Retorna se a segurança de thread é dada ou não

Descrição

Estilo orientado a objetos

public function **mysqli::thread_safe**(): bool

Estilo procedural

function **mysqli_thread_safe**(): bool

Informa se a biblioteca cliente é compilada como thread-safe.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se a biblioteca cliente for thread-safe, caso contrário, `false`.

mysqli::use_result

mysqli::use_result

mysqli_use_result

(PHP 5, PHP 7, PHP 8)

mysqli::use_result -- mysqli_use_result — Inicia uma recuperação de conjunto de resultados

Descrição

Estilo orientado a objetos

public function **mysqli::use_result**(): mysqli_result|false

Estilo procedural

function **mysqli_use_result**(mysqli `$mysql`): mysqli_result|false

Usado para iniciar a recuperação de um conjunto de resultados da última consulta executada usando a função mysqli_real_query() na conexão com o banco de dados.

Esta ou a função mysqli_store_result() deve ser chamada antes que os resultados de uma consulta possam ser recuperados, e uma ou outra deve ser chamada para evitar que a próxima consulta nessa conexão com o banco de dados falhe.

> **Nota** :
>
> A função **mysqli_use_result()** não transfere todo o conjunto de resultados do banco de dados e, portanto, não pode ser usada com funções como mysqli_data_seek() para mover para uma linha específica dentro do conjunto. Para usar esta funcionalidade, o conjunto de resultados deve ser armazenado usando mysqli_store_result(). Não se deve usar **mysqli_use_result()** se for feito muito processamento no lado do cliente, pois isso irá sobrecarregar o servidor e evitar que outras threads atualizem quaisquer tabelas das quais os dados estão sendo buscados.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um objeto de resultado sem buffer ou `false` se ocorreu um erro.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (`MYSQLI_REPORT_ERROR`) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como `MYSQLI_REPORT_STRICT`, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Exemplo de **mysqli::use_result()**

Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* verifique a conexão */
if (mysqli_connect_errno()) {
    printf("Falha na conexão: %s\n", mysqli_connect_error());
    exit();
}

$query  = "SELECT CURRENT_USER();";
$query .= "SELECT Name FROM City ORDER BY ID LIMIT 20, 5";

/* executar várias consultas */
if ($mysqli->multi_query($query)) {
    do {
        /* armazenar primeiro conjunto de resultados */
        if ($result = $mysqli->use_result()) {
            while ($row = $result->fetch_row()) {
                printf("%s\n", $row[0]);
            }
            $result->close();
        }
        /* divisor de impressão */
        if ($mysqli->more_results()) {
            printf("-----------------\n");
        }
    } while ($mysqli->next_result());
}

/* fecha a conexão */
$mysqli->close();
?>
```

Estilo procedural

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* verifique a conexão */
if (mysqli_connect_errno()) {
    printf("Falha na conexão: %s\n", mysqli_connect_error());
    exit();
}

$query  = "SELECT CURRENT_USER();";
$query .= "SELECT Name FROM City ORDER BY ID LIMIT 20, 5";

/* executar várias consultas */
if (mysqli_multi_query($link, $query)) {
    do {
        /* armazenar primeiro conjunto de resultados */
        if ($result = mysqli_use_result($link)) {
            while ($row = mysqli_fetch_row($result)) {
                printf("%s\n", $row[0]);
            }
            mysqli_free_result($result);
        }
        /* divisor de impressão */
```

```
        if (mysqli_more_results($link)) {
            printf("-----------------\n");
        }
    } while (mysqli_next_result($link));
}

/* fecha a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
my_user@localhost
-----------------
Amersfoort
Maastricht
Dordrecht
Leiden
Haarlemmermeer
```

Veja também

- mysqli_real_query() - Executa uma consulta SQL
- mysqli_store_result() - Transfere um conjunto de resultados da última consulta

mysqli::$warning_count

mysqli::$warning_count

mysqli_warning_count

(PHP 5, PHP 7, PHP 8)

mysqli::$warning_count -- mysqli_warning_count — Retorna o número de alertas gerados pela consulta mais recente

Descrição

Estilo orientado a objetos

int *$mysqli->warning_count*;

Estilo procedural

function **mysqli_warning_count**(mysqli `$mysql`): int

Retorna o número de alertas gerados pela consulta mais recente.

> **Nota** :

> Para recuperar mensagens de alerta, o comando SQL a seguir pode ser usado: `SHOW WARNINGS [limit row_count]`.

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

Valor Retornado

Número de avisos ou zero se não houver avisos.

Examples

Exemplo #1 Exemplo de *$mysqli->warning_count*

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$mysqli->query("SELECT 42/0");

if ($mysqli->warning_count > 0) {
    $result = $mysqli->query("SHOW WARNINGS");
    $row = $result->fetch_row();
    printf("%s (%d): %s\n", $row[0], $row[1], $row[2]);
}
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

mysqli_query($link, "SELECT 42/0");

if (mysqli_warning_count($link) > 0) {
    $result = mysqli_query($link, "SHOW WARNINGS");
    $row = mysqli_fetch_row($result);
    printf("%s (%d): %s\n", $row[0], $row[1], $row[2]);
}
```

Os exemplos acima produzirão:

```
Warning (1365): Division by 0
```

Veja também

- mysqli_errno() - Retorna o código de erro para a chamada de função mais recente
- mysqli_error() - Retorna uma string descrevendo o último erro
- mysqli_sqlstate() - Retorna o erro SQLSTATE da operação MySQL anterior

mysqli_stmt

(PHP 5, PHP 7, PHP 8)

Introdução

Representa uma declaração preparada.

Resumo da classe

class **mysqli_stmt** {
/* Propriedades */
public readonly int|string *$affected_rows*;
public readonly int|string *$insert_id*;
public readonly int|string *$num_rows*;
public readonly int *$param_count*;
public readonly int *$field_count*;
public readonly int *$errno*;
public readonly string *$error*;
public readonly array *$error_list*;
public readonly string *$sqlstate*;
public int *$id*;
/* Métodos */
public function __construct(mysqli `$mysql`, ?string `$query` = **`null`**)
public function attr_get(int `$attribute`): int
public function attr_set(int `$attribute`, int `$value`): bool
public function bind_param(string `$types`, mixed `&$var`, mixed `&...$vars`): bool
public function bind_result(mixed `&$var`, mixed `&...$vars`): bool
public function close(): true
public function data_seek(int `$offset`): void
public function execute(?array `$params` = **`null`**): bool
public function fetch(): ?bool
public function free_result(): void
public function get_result(): mysqli_result|false
public function get_warnings(): mysqli_warning|false
public function more_results(): bool
public function next_result(): bool
public function num_rows(): int|string
public function prepare(string `$query`): bool
public function reset(): bool
public function result_metadata(): mysqli_result|false
public function send_long_data(int `$param_num`, string `$data`): bool
public function store_result(): bool
}

Propriedades

id

Armazena o ID da declaração.

Índice

mysqli_stmt::$affected_rows

mysqli_stmt::$affected_rows

mysqli_stmt_affected_rows

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$affected_rows -- mysqli_stmt_affected_rows — Retorna o número total de linhas modificadas, apagadas, inseridas ou correspondidas pela última instrução executada

Descrição

Estilo orientado a objetos

int|string *$mysqli_stmt->affected_rows*;

Estilo procedural

function **mysqli_stmt_affected_rows**(mysqli_stmt `$statement`): int|string

Retorna o número de linhas afetadas por consultas `INSERT`, `UPDATE` ou `DELETE`. Funciona como mysqli_stmt_num_rows() para instruções `SELECT`.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Um inteiro maior que zero indica o número de linhas afetadas ou recebidas. Zero indica que nenhum registro foi atualizado para uma instrução `UPDATE`, nenhuma linha correspondeu à cláusula `WHERE` na consulta ou que nenhuma consulta foi executada. `-1` indica que a consulta retornou um erro ou que, para uma consulta `SELECT`, **mysqli_stmt_affected_rows()** foi chamada antes de mysqli_stmt_store_result().

> **Nota** :
>
> Se o número de linhas afetadas for maior que o máximo valor inteiro do PHP, o número de linhas afetadas será retornado como uma string.

Examples

Exemplo #1 Exemplo de mysqli_stmt_affected_rows()

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* cria tabela temporária */
$mysqli->query("CREATE TEMPORARY TABLE myCountry LIKE Country");

$query = "INSERT INTO myCountry SELECT * FROM Country WHERE Code LIKE ?";

/* prepara a instrução */
$stmt = $mysqli->prepare($query);

/* Vincula variável para o espaço reservado */
$code = 'A%';
$stmt->bind_param("s", $code);

/* executa a instrução */
$stmt->execute();

printf("Linhas inseridas: %d\n", $stmt->affected_rows);
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* cria tabela temporária */
mysqli_query($link, "CREATE TEMPORARY TABLE myCountry LIKE Country");

$query = "INSERT INTO myCountry SELECT * FROM Country WHERE Code LIKE ?";

/* prepara a instrução */
$stmt = mysqli_prepare($link, $query);

/* Vincula variável para o espaço reservado */
$code = 'A%';
mysqli_stmt_bind_param($stmt, "s", $code);

/* executa a instrução */
mysqli_stmt_execute($stmt);

printf("Linhas inseridas: %d\n", mysqli_stmt_affected_rows($stmt));
```

Os exemplos acima produzirão:

```
Linhas inseridas: 17
```

Veja também

- mysqli_stmt_num_rows() - Retorna o número de linhas recebidas pelo servidor
- mysqli_stmt_store_result() - Armazena um conjunto de resultados em um buffer interno

mysqli_stmt::attr_get

mysqli_stmt::attr_get

mysqli_stmt_attr_get

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::attr_get -- mysqli_stmt_attr_get — Usado para obter o valor atual de um atributo de instrução

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::attr_get**(int `$attribute`): int

Estilo procedural

function **mysqli_stmt_attr_get**(mysqli_stmt `$statement`, int `$attribute`): int

Obtém o valor atual de um atributo de instrução.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

`attribute`

O atributo que se deseja obter.

Valor Retornado

Retorna o valor do atributo.

mysqli_stmt::attr_set

mysqli_stmt::attr_set

mysqli_stmt_attr_set

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::attr_set -- mysqli_stmt_attr_set — Usado para modificar o comportamento de uma instrução preparada

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::attr_set**(int `$attribute`, int `$value`): bool

Estilo procedural

function **mysqli_stmt_attr_set**(mysqli_stmt `$statement`, int `$attribute`, int `$value`): bool

Usada para modificar o comportamento de uma instrução preparada. Esta função pode ser chamada múltiplas vezes para definir vários atributos.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

`attribute`

O atributo que se deseja definir. Pode ser um dos seguintes valores:

Valores de atributos

Caractere	Descrição
MYSQLI_STMT_ATTR_UPDATE_MAX_LENGTH	Definir para **`true`** faz com que mysqli_stmt_store_result() atualize o valor do metadado `MYSQL_FIELD->max_length`.
MYSQLI_STMT_ATTR_CURSOR_TYPE	Tipo de cursor a ser aberto para a instrução quando mysqli_stmt_execute() for chamada. `value` pode ser **`MYSQLI_CURSOR_TYPE_NO_CURSOR`** (o padrão) ou **`MYSQLI_CURSOR_TYPE_READ_ONLY`**.
MYSQLI_STMT_ATTR_PREFETCH_ROWS	Número de linhas a receber do servidor de cada vez ao usar um cursor. `value` pode estar na faixa de 1 até o máximo valor de um inteiro longo sem sinal. O padrão é 1. Removida a partir do PHP 8.4.0.

Se a opção **`MYSQLI_STMT_ATTR_CURSOR_TYPE`** for usada com **`MYSQLI_CURSOR_TYPE_READ_ONLY`**, um cursor é aberto para a instrução quando mysqli_stmt_execute() é chamada. Se já houver um cursor aberto de uma chamada anterior de mysqli_stmt_execute(), o cursor é fechado antes que um novo seja aberto. mysqli_stmt_reset() também fecha qualquer cursor aberto antes de preparar a instrução para re-execução. mysqli_stmt_free_result() fecha qualquer cursor aberto.

Se for aberto um cursor para uma instrução preparada, a função mysqli_stmt_store_result() é desnecessária.

`value`

O valor a designar para o atributo.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**`MYSQLI_REPORT_ERROR`**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **`MYSQLI_REPORT_STRICT`**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Veja também

- » Conector/MySQL mysql_stmt_attr_set()

mysqli_stmt::bind_param

mysqli_stmt::bind_param

mysqli_stmt_bind_param

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::bind_param -- mysqli_stmt_bind_param — Liga variáveis a uma declaração preparada como parâmetros

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::bind_param**(string `$types`, mixed `&$var`, mixed `&...$vars`): bool

Estilo procedural

function **mysqli_stmt_bind_param**(
  mysqli_stmt `$statement`,
  string `$types`,
  mixed `&$var`,
  mixed `&...$vars`
): bool

Liga variáveis para os marcadores de parâmetros na declaração SQL preparada por mysqli_prepare() ou mysqli_stmt_prepare().

> **Nota** :
>
> Se o tamanho de dados de um variável exceder o tamanho de pacote máximo permitido (max_allowed_packet), deve-se especificar `b` em `types` e usar mysqli_stmt_send_long_data() para enviar os dados em pacotes.

> **Nota** :
>
> Deve-se ter cuidado ao usar **mysqli_stmt_bind_param()** em conjunto com call_user_func_array(). Note que **mysqli_stmt_bind_param()** requer que os parâmetros sejam passados por referência, enquanto que call_user_func_array() pode aceitar como parâmetro uma lista de variáveis que podem representar referências ou valores.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

`types`

Uma string que contém um ou mais caracteres que especificam os tipos para as variáveis correspondentes ligadas:

Caracteres de especificação de tipo

Caractere	Descrição
i	variável correspondente tem tipo int
d	variável correspondente tem tipo float
s	variável correspondente tem tipo string
b	variável correspondente é um blob e será enviada em pacotes

`var`
`vars`

O número de variáveis e o comprimento da string que o parâmetro `types` deve corresponder aos parâmetros na declaração.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**`MYSQLI_REPORT_ERROR`**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **`MYSQLI_REPORT_STRICT`**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Exemplo de mysqli_stmt::bind_param()

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli('localhost', 'my_user', 'my_password', 'world');

$stmt = $mysqli->prepare("INSERT INTO CountryLanguage VALUES (?, ?, ?, ?)");
$stmt->bind_param('sssd', $code, $language, $official, $percent);

$code = 'DEU';
$language = 'Bavarian';
$official = "F";
$percent = 11.2;

$stmt->execute();

printf("%d linha inserida.\n", $stmt->affected_rows);

/* Limpa tabela CountryLanguage */
$mysqli->query("DELETE FROM CountryLanguage WHERE Language='Bavarian'");
printf("%d linha apagada.\n", $mysqli->affected_rows);
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
```

```php
$link = mysqli_connect('localhost', 'my_user', 'my_password', 'world');

$stmt = mysqli_prepare($link, "INSERT INTO CountryLanguage VALUES (?, ?, ?, ?)");
mysqli_stmt_bind_param($stmt, 'sssd', $code, $language, $official, $percent);

$code = 'DEU';
$language = 'Bavarian';
$official = "F";
$percent = 11.2;

mysqli_stmt_execute($stmt);

printf("%d linha inserida.\n", mysqli_stmt_affected_rows($stmt));

/* Limpa tabela CountryLanguage */
mysqli_query($link, "DELETE FROM CountryLanguage WHERE Language='Bavarian'");
printf("%d linha apagada.\n", mysqli_affected_rows($link));
```

Os exemplos acima produzirão:

```
1 linha inserida.
1 linha apagada.
```

Exemplo #2 Usando ... para fornecer argumentos

O operador ... pode ser usado para fornecer uma lista de argumentos de comprimento variável, como por exemplo em uma cláusula `WHERE IN`.

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli('localhost', 'my_user', 'my_password', 'world');

$stmt = $mysqli->prepare("SELECT Language FROM CountryLanguage WHERE CountryCode IN (?, ?)");
/* Usando ... para fornecer argumentos */
$stmt->bind_param('ss', ...['DEU', 'POL']);
$stmt->execute();
$stmt->store_result();

printf("%d linhas encontradas.\n", $stmt->num_rows());
```

Os exemplos acima produzirão:

```
10 linhas encontradas.
```

Veja também

- mysqli_stmt_bind_result() - Liga variáveis a uma declaração preparada para armazenamento de resultado
- mysqli_stmt_execute() - Executa uma instrução preparada
- mysqli_stmt_fetch() - Recebe resultados de uma instrução preparada nas variáveis vinculadas
- mysqli_prepare() - Prepara uma instrução SQL para execução
- mysqli_stmt_send_long_data() - Envia dados em blocos
- mysqli_stmt_errno() - Retorna o código de erro para a chamada de instrução mais recente
- mysqli_stmt_error() - Retorna uma descrição em string para o último erro de instrução

mysqli_stmt::bind_result

mysqli_stmt::bind_result

mysqli_stmt_bind_result

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::bind_result -- mysqli_stmt_bind_result — Liga variáveis a uma declaração preparada para armazenamento de resultado

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::bind_result**(mixed `&$var`, mixed `&...$vars`): bool

Estilo procedural

function **mysqli_stmt_bind_result**(mysqli_stmt `$statement`, mixed `&$var`, mixed `&...$vars`): bool

Liga colunas no conjunto de resultados a variáveis.

Quando mysqli_stmt_fetch() é chamada para obter dados, o protocolo cliente/servidor MySQL coloca os dados das colunas ligadas nas variáveis expecificadas `var`/`vars`.

Uma coluna pode ser ligada ou religada a qualquer tempo, mesmo depois que um conjunto de resultados tenha sido parcialmente recebido. A nova ligação tem efeito na próxima vez que mysqli_stmt_fetch() for chamada.

> **Nota** :
>
> Todas as colunas precisam ser ligadas depois da chamada a mysqli_stmt_execute() e antes da chamada a mysqli_stmt_fetch().

> **Nota** :
>
> Dependendo dos tipos das colunas, variáveis ligadas podem silenciosamente ser alteradas para O tipo PHP correspondente.

> **Dica**

> Esta função é útil para resultados simples. Para receber um conjunto de resultados iterável ou obter cada linha como um array ou objeto, use a função mysqli_stmt_get_result().

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

`var`

A primeira variável a ser ligada.

`vars`

Mais variáveis a serem ligadas.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* prepara a declaração */
$stmt = $mysqli->prepare("SELECT Code, Name FROM Country ORDER BY Name LIMIT 5");
$stmt->execute();

/* liga as variáveis à declaração preparada */
$stmt->bind_result($col1, $col2);

/* obtém os valores */
while ($stmt->fetch()) {
    printf("%s %s\n", $col1, $col2);
}
```

Exemplo #2 Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* prepara a declaração */
$stmt = mysqli_prepare($link, "SELECT Code, Name FROM Country ORDER BY Name LIMIT 5");
mysqli_stmt_execute($stmt);

/* liga as variáveis à declaração preparada */
mysqli_stmt_bind_result($stmt, $col1, $col2);

/* obtém os valores */
```

```php
while (mysqli_stmt_fetch($stmt)) {
    printf("%s %s\n", $col1, $col2);
}
```

Os exemplos acima produzirão algo semelhante a:

```
AFG Afghanistan
ALB Albania
DZA Algeria
ASM American Samoa
AND Andorra
```

Veja também

- mysqli_stmt_get_result() - Obtém um resultado de uma instrução preparada como um objeto mysqli_result
- mysqli_stmt_bind_param() - Liga variáveis a uma declaração preparada como parâmetros
- mysqli_stmt_execute() - Executa uma instrução preparada
- mysqli_stmt_fetch() - Recebe resultados de uma instrução preparada nas variáveis vinculadas
- mysqli_prepare() - Prepara uma instrução SQL para execução
- mysqli_stmt_prepare() - Prepara uma declaração SQL para execução

mysqli_stmt::close

mysqli_stmt::close

mysqli_stmt_close

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::close -- mysqli_stmt_close — Fecha uma instrução preparada

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::close**(): true

Estilo procedural

function **mysqli_stmt_close**(mysqli_stmt `$statement`): true

Fecha uma instrução preparada. **mysqli_stmt_close()** também desaloca da memória o manipulador da instrução. Se a instrução atual tem resultados pendentes ou não lidos, esta função as cancela de forma que a próxima consulta possa ser executada.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Sempre retorna `true`.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função agora sempre retorna `true`. Anteriormente retornava `false` em caso de falha.

Veja também

- mysqli_prepare() - Prepara uma instrução SQL para execução

mysqli_stmt::__construct

mysqli_stmt::__construct

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::__construct — Constrói um novo objeto mysqli_stmt

Descrição

public function **mysqli_stmt::__construct**(mysqli `$mysql`, ?string `$query` = `null`)

Este método constrói um novo objeto mysqli_stmt.

Parâmetros

`link`

Om objeto mysqli válido.

`query`

A consulta, como uma string. Se este parâmetro for `null`, o construtor se comporta de forma idêntica a mysqli_stmt_init(), caso contrário se comporta como mysqli_prepare().

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (`MYSQLI_REPORT_ERROR`) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como `MYSQLI_REPORT_STRICT`, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Registro de Alterações

Versão	Descrição
8.0.0	`query` agora pode ser nulo.

Veja também

- mysqli_prepare() - Prepara uma instrução SQL para execução
- mysqli_stmt_init() - Inicializa uma instrução e retorna um objeto para uso com mysqli_stmt_prepare

mysqli_stmt::data_seek

mysqli_stmt::data_seek

mysqli_stmt_data_seek

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::data_seek -- mysqli_stmt_data_seek — Ajusta o ponteiro de resultado para uma linha arbitrária no resultado com buffer

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::data_seek**(int `$offset`): void

Estilo procedural

function **mysqli_stmt_data_seek**(mysqli_stmt `$statement`, int `$offset`): void

Esta função move o ponteiro do conjunto de resultados com buffer para uma linha arbitrária especificada pelo parâmetro `offset`.

Esta função funciona somente com o conjunto de resultados interno com buffer. mysqli_stmt_store_result() precisa ser chamada antes de **mysqli_stmt_data_seek()**.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

`offset`

Precisa ser entre zero e o número total de linhas menos um (0.. mysqli_stmt_num_rows() - 1).

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name";
$stmt = $mysqli->prepare($query);
$stmt->execute();

$stmt->bind_result($name, $code);

$stmt->store_result();

/* aponta para a linha núm. 400 */
$stmt->data_seek(399);
```

```
$stmt->fetch();

printf("City: %s  Countrycode: %s\n", $name, $code);
```

Exemplo #2 Estilo procedural

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name";
$stmt = mysqli_prepare($link, $query);

mysqli_stmt_execute($stmt);

mysqli_stmt_bind_result($stmt, $name, $code);

mysqli_stmt_store_result($stmt);

/* aponta para a linha núm. 400 */
mysqli_stmt_data_seek($stmt, 399);

mysqli_stmt_fetch($stmt);

printf("City: %s  Countrycode: %s\n", $name, $code);
```

Os exemplos acima produzirão:

```
City: Benin City  Countrycode: NGA
```

Veja também

- mysqli_prepare() - Prepara uma instrução SQL para execução
- mysqli_stmt_store_result() - Armazena um conjunto de resultados em um buffer interno
- mysqli_stmt_num_rows() - Retorna o número de linhas recebidas pelo servidor
- mysqli_data_seek() - Ajusta o ponteiro do resultado para uma linha arbitrária no resultado

mysqli_stmt::$errno

mysqli_stmt::$errno

mysqli_stmt_errno

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$errno -- mysqli_stmt_errno — Retorna o código de erro para a chamada de instrução mais recente

Descrição

Estilo orientado a objetos

int *$mysqli_stmt->errno*;

Estilo procedural

function **mysqli_stmt_errno**(mysqli_stmt `$statement`): int

Retorna o código de erro para a função de instrução mais recentemente chamada, que possa ter sucesso ou falhar.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Um valor de código de erro. Zero significa que não houve erro.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php
/* Abre uma conexão */
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* Verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

$mysqli->query("CREATE TABLE myCountry LIKE Country");
$mysqli->query("INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = $mysqli->prepare($query)) {

    /* apaga a tabela */
    $mysqli->query("DROP TABLE myCountry");

    /* executa a instrução */
    $stmt->execute();

    printf("Erro: %d.\n", $stmt->errno);

    /* fecha a instrução */
    $stmt->close();
}

/* fecha a conexão */
$mysqli->close();
?>
```

Exemplo #2 Estilo procedural

```php
<?php
/* Abre uma conexão */
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* Verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

mysqli_query($link, "CREATE TABLE myCountry LIKE Country");
mysqli_query($link, "INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = mysqli_prepare($link, $query)) {

    /* apaga a tabela */
    mysqli_query($link, "DROP TABLE myCountry");

    /* executa a instrução */
    mysqli_stmt_execute($stmt);

    printf("Erro: %d.\n", mysqli_stmt_errno($stmt));

    /* fecha a instrução */
    mysqli_stmt_close($stmt);
}

/* close connection */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Erro: 1146.
```

Veja também

- mysqli_stmt_error() - Retorna uma descrição em string para o último erro de instrução
- mysqli_stmt_sqlstate() - Retorna o erro SQLSTATE da operação da instrução anterior

mysqli_stmt::$error

mysqli_stmt::$error

mysqli_stmt_error

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$error -- mysqli_stmt_error — Retorna uma descrição em string para o último erro de instrução

Descrição

Estilo orientado a objetos

string *$mysqli_stmt->error*;

Estilo procedural

function **mysqli_stmt_error**(mysqli_stmt `$statement`): string

Retorna uma string contendo a mensagem de erro para a função de instrução mais recentemente chamada, que possa ter sucesso ou falhar.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Uma string que descreve o erro. Uma string vazia se nenhum erro ocorreu.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php
/* Abre uma conexão */
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* Verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

$mysqli->query("CREATE TABLE myCountry LIKE Country");
$mysqli->query("INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = $mysqli->prepare($query)) {

    /* apaga a tabela */
    $mysqli->query("DROP TABLE myCountry");

    /* executa a instrução */
    $stmt->execute();

    printf("Erro: %s.\n", $stmt->error);

    /* fecha a instrução */
    $stmt->close();
}
```

```
/* fecha a conexão */
$mysqli->close();
?>
```

Exemplo #2 Estilo procedural

```
<?php
/* Abre uma conexão */
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* Verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

mysqli_query($link, "CREATE TABLE myCountry LIKE Country");
mysqli_query($link, "INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = mysqli_prepare($link, $query)) {

    /* apaga a tabela */
    mysqli_query($link, "DROP TABLE myCountry");

    /* executa a instrução */
    mysqli_stmt_execute($stmt);

    printf("Erro: %s.\n", mysqli_stmt_error($stmt));

    /* fecha a instrução */
    mysqli_stmt_close($stmt);
}

/* fecha a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Erro: Table 'world.myCountry' doesn't exist.
```

Veja também

- mysqli_stmt_errno() - Retorna o código de erro para a chamada de instrução mais recente
- mysqli_stmt_sqlstate() - Retorna o erro SQLSTATE da operação da instrução anterior

mysqli_stmt::$error_list

mysqli_stmt::$error_list

mysqli_stmt_error_list

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

mysqli_stmt::$error_list -- mysqli_stmt_error_list — Retorna uma lista de erros da última instrução executada

Descrição

Estilo orientado a objetos

array *$mysqli_stmt->error_list*;

Estilo procedural

function **mysqli_stmt_error_list**(mysqli_stmt `$statement`): array

Retorna um array de erros para a função de instrução mais recente que possa ter sucesso ou falhar.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Uma lista de erros, cada um como um array associativo contendo o número do erro (errno), o erro (error) e o estado SQL (sqlstate).

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php
/* Abre uma conexão */
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* Verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

$mysqli->query("CREATE TABLE myCountry LIKE Country");
$mysqli->query("INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = $mysqli->prepare($query)) {

    /* apaga a tabela */
    $mysqli->query("DROP TABLE myCountry");

    /* executa a instrução */
    $stmt->execute();

    echo "Erro:\n";
    print_r($stmt->error_list);
```

```
    /* fecha a instrução */
    $stmt->close();
}

/* fecha a conexão */
$mysqli->close();
?>
```

Exemplo #2 Estilo procedural

```
<?php
/* Abre uma conexão */
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* Verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

mysqli_query($link, "CREATE TABLE myCountry LIKE Country");
mysqli_query($link, "INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = mysqli_prepare($link, $query)) {

    /* apaga a tabela */
    mysqli_query($link, "DROP TABLE myCountry");

    /* executa a instrução */
    mysqli_stmt_execute($stmt);

    echo "Erro:\n";
    print_r(mysqli_stmt_error_list($stmt));

    /* fecha a instrução */
    mysqli_stmt_close($stmt);
}

/* fecha a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Erro:
Array
(
    [0] => Array
        (
            [errno] => 1146
            [sqlstate] => 42S02
            [error] => Table 'world.myCountry' doesn't exist
        )
```

```
)
```

Veja também

- mysqli_stmt_error() - Retorna uma descrição em string para o último erro de instrução
- mysqli_stmt_errno() - Retorna o código de erro para a chamada de instrução mais recente
- mysqli_stmt_sqlstate() - Retorna o erro SQLSTATE da operação da instrução anterior

mysqli_stmt::execute

mysqli_stmt::execute

mysqli_stmt_execute

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::execute -- mysqli_stmt_execute — Executa uma instrução preparada

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::execute**(?array `$params = null`): bool

Estilo procedural

function **mysqli_stmt_execute**(mysqli_stmt `$statement`, ?array `$params = null`): bool

Executa uma instrução previamente preparada. A instrução precisa ser preparada com sucesso antes da execução, usando a função mysqli_prepare() ou a função mysqli_stmt_prepare(), ou ainda passando o segundo argumento para o método mysqli_stmt::__construct().

Se a instrução for `UPDATE`, `DELETE` ou `INSERT`, o número total de linhas afetadas pode ser determinado usando-se a função mysqli_stmt_affected_rows(). Se a consulta retorna um resultado, ele pode ser recebido pela função mysqli_stmt_get_result() ou linha por linha diretamente da instrução usando a função mysqli_stmt_fetch().

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

`params`

Um array opcional listando tantos elementos quantos forem os parâmetros vinculados na instrução SQL sendo executada. Cada valor é tratado como uma string.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (`MYSQLI_REPORT_ERROR`) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como `MYSQLI_REPORT_STRICT`, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Registro de Alterações

Versão	Descrição
8.1.0	O parâmetro optional `params` foi adicionado.

Examples

Exemplo #1 Executa a instrução preparada com variáveis vinculadas

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$mysqli->query("CREATE TEMPORARY TABLE myCity LIKE City");

/* Prepara uma instrução de inserção */
$stmt = $mysqli->prepare("INSERT INTO myCity (Name, CountryCode, District) VALUES (?,?,?)");

/* Vincula variáveis aos parâmetros */
$stmt->bind_param("sss", $val1, $val2, $val3);

$val1 = 'Stuttgart';
$val2 = 'DEU';
$val3 = 'Baden-Wuerttemberg';

/* Executa a instrução */
$stmt->execute();

$val1 = 'Bordeaux';
$val2 = 'FRA';
$val3 = 'Aquitaine';

/* Executa a instrução */
$stmt->execute();

/* Recebe todas as linhas da tabela myCity */
$query = "SELECT Name, CountryCode, District FROM myCity";
$result = $mysqli->query($query);
while ($row = $result->fetch_row()) {
    printf("%s (%s,%s)\n", $row[0], $row[1], $row[2]);
}
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

mysqli_query($link, "CREATE TEMPORARY TABLE myCity LIKE City");
```

```php
/* Prepara uma instrução de inserção */
$stmt = mysqli_prepare($link, "INSERT INTO myCity (Name, CountryCode, District) VALUES (?,?,?)");

/* Vincula variáveis aos parâmetros */
mysqli_stmt_bind_param($stmt, "sss", $val1, $val2, $val3);

$val1 = 'Stuttgart';
$val2 = 'DEU';
$val3 = 'Baden-Wuerttemberg';

/* Executa a instrução */
mysqli_stmt_execute($stmt);

$val1 = 'Bordeaux';
$val2 = 'FRA';
$val3 = 'Aquitaine';

/* Executa a instrução */
mysqli_stmt_execute($stmt);

/* Recebe todas as linhas da tabela myCity */
$query = "SELECT Name, CountryCode, District FROM myCity";
$result = mysqli_query($link, $query);
while ($row = mysqli_fetch_row($result)) {
    printf("%s (%s,%s)\n", $row[0], $row[1], $row[2]);
}
```

Os exemplos acima produzirão:

```
Stuttgart (DEU,Baden-Wuerttemberg)
Bordeaux (FRA,Aquitaine)
```

Exemplo #2 Executa uma isntrução preparada com um array de valores

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli('localhost', 'my_user', 'my_password', 'world');

$mysqli->query('CREATE TEMPORARY TABLE myCity LIKE City');

/* Prepara uma instrução de inserção */
$stmt = $mysqli->prepare('INSERT INTO myCity (Name, CountryCode, District) VALUES (?,?,?)');

/* Executa a instrução */
$stmt->execute(['Stuttgart', 'DEU', 'Baden-Wuerttemberg']);

/* Recebe todas as linhas da tabela myCity */
$query = 'SELECT Name, CountryCode, District FROM myCity';
```

```
$result = $mysqli->query($query);
while ($row = $result->fetch_row()) {
    printf("%s (%s,%s)\n", $row[0], $row[1], $row[2]);
}
```

Estilo procedural

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect('localhost', 'my_user', 'my_password', 'world');

mysqli_query($link, 'CREATE TEMPORARY TABLE myCity LIKE City');

/* Prepara uma instrução de inserção */
$stmt = mysqli_prepare($link, 'INSERT INTO myCity (Name, CountryCode, District) VALUES (?,?,?)');

/* Executa a instrução */
mysqli_stmt_execute($stmt, ['Stuttgart', 'DEU', 'Baden-Wuerttemberg']);

/* Recebe todas as linhas da tabela myCity */
$query = 'SELECT Name, CountryCode, District FROM myCity';
$result = mysqli_query($link, $query);
while ($row = mysqli_fetch_row($result)) {
    printf("%s (%s,%s)\n", $row[0], $row[1], $row[2]);
}
```

Os exemplos acima produzirão:

```
Stuttgart (DEU,Baden-Wuerttemberg)
```

Veja também

- mysqli_execute_query() - Prepara, vincula parâmetros e executa a instrução SQL
- mysqli_prepare() - Prepara uma instrução SQL para execução
- mysqli_stmt_bind_param() - Liga variáveis a uma declaração preparada como parâmetros
- mysqli_stmt_get_result() - Obtém um resultado de uma instrução preparada como um objeto mysqli_result

mysqli_stmt::fetch

mysqli_stmt::fetch

mysqli_stmt_fetch

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::fetch -- mysqli_stmt_fetch — Recebe resultados de uma instrução preparada nas variáveis vinculadas

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::fetch**(): ?bool

Estilo procedural

function **mysqli_stmt_fetch**(mysqli_stmt `$statement`): ?bool

Recebe o resultado de uma instrução preparada nas variáveis vinculadas por mysqli_stmt_bind_result().

> **Nota** :
>
> Observe que todas as colunas precisam estar vinculadas pela aplicação antes de chamar a função **mysqli_stmt_fetch()**.

> **Nota** :
>
> Os dados são transferidos sem buffer quando não se chama a função mysqli_stmt_store_result(), o que pode reduzir o desempenho (mas reduz o consumo de memória).

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Valor Retornado

Valor	Descrição
`true`	Sucesso. Dados foram recebidos
`false`	Error occurred
`null`	Não há mais linhas/dados ou ocorreu um truncamento de dados

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (`MYSQLI_REPORT_ERROR`) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como `MYSQLI_REPORT_STRICT`, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}
```

```php
$query = "SELECT Name, CountryCode FROM City ORDER by ID DESC LIMIT 150,5";

if ($stmt = $mysqli->prepare($query)) {

    /* executa a instrução */
    $stmt->execute();

    /* vincula as variáveis do resultado */
    $stmt->bind_result($name, $code);

    /* recebe os valores */
    while ($stmt->fetch()) {
        printf ("%s (%s)\n", $name, $code);
    }

    /* fecha a instrução */
    $stmt->close();
}

/* fecha a conexão */
$mysqli->close();
?>
```

Exemplo #2 Estilo procedural

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, CountryCode FROM City ORDER by ID DESC LIMIT 150,5";

if ($stmt = mysqli_prepare($link, $query)) {

    /* executa a instrução */
    mysqli_stmt_execute($stmt);

    /* vincula as variáveis do resultado */
    mysqli_stmt_bind_result($stmt, $name, $code);

    /* recebe os valores */
    while (mysqli_stmt_fetch($stmt)) {
        printf ("%s (%s)\n", $name, $code);
    }

    /* fecha a instrução */
    mysqli_stmt_close($stmt);
}

/* fecha a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Rockford (USA)
Tallahassee (USA)
Salinas (USA)
Santa Clarita (USA)
Springfield (USA)
```

Veja também

- mysqli_prepare() - Prepara uma instrução SQL para execução
- mysqli_stmt_errno() - Retorna o código de erro para a chamada de instrução mais recente
- mysqli_stmt_error() - Retorna uma descrição em string para o último erro de instrução
- mysqli_stmt_bind_result() - Liga variáveis a uma declaração preparada para armazenamento de resultado

mysqli_stmt::$field_count

mysqli_stmt::$field_count

mysqli_stmt_field_count

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$field_count -- mysqli_stmt_field_count — Retorna o número de colunas na instrução informada

Descrição

Estilo orientado a objetos

int *$mysqli_stmt->field_count*;

Estilo procedural

function **mysqli_stmt_field_count**(mysqli_stmt `$statement`): int

Retorna o número de colunas na instrução preparada.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Retorna um inteiro que representa o número de colunas.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$code = 'FR';

$stmt = $mysqli->prepare("SELECT Name FROM Country WHERE Code=?");
$stmt->bind_param('s', $code);
$stmt->execute();
$row = $stmt->get_result()->fetch_row();
for ($i = 0; $i < $stmt->field_count; $i++) {
    printf("Valor da coluna número %d é %s", $i, $row[$i]);
}
```

Exemplo #2 Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$code = 'FR';

$stmt = mysqli_prepare($mysqli, "SELECT Name FROM Country WHERE Code=?");
mysqli_stmt_bind_param($stmt, 's', $code);
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);
$row = mysqli_fetch_row($result);
for ($i = 0; $i < mysqli_stmt_field_count($stmt); $i++) {
    printf("Valor da coluna número %d é %s", $i, $row[$i]);
}
```

Os exemplos acima produzirão algo semelhante a:

```
Valor da coluna número 0 é France
```

Veja também

- mysqli_stmt_num_rows() - Retorna o número de linhas recebidas pelo servidor

mysqli_stmt::free_result

mysqli_stmt::free_result

mysqli_stmt_free_result

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::free_result -- mysqli_stmt_free_result — Libera memória do resultado armazenado para o manipulador de instrução informado

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::free_result**(): void

Estilo procedural

function **mysqli_stmt_free_result**(mysqli_stmt `$statement`): void

Libera a memória do resultado associado à instrução, que foi alocada por mysqli_stmt_store_result().

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Nenhum valor é retornado.

Veja também

- mysqli_stmt_store_result() - Armazena um conjunto de resultados em um buffer interno

mysqli_stmt::get_result

mysqli_stmt::get_result

mysqli_stmt_get_result

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli_stmt::get_result -- mysqli_stmt_get_result — Obtém um resultado de uma instrução preparada como um objeto mysqli_result

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::get_result**(): mysqli_result|false

Estilo procedural

function **mysqli_stmt_get_result**(mysqli_stmt `$statement`): mysqli_result|false

Recebe um resultado de uma instrução preparada como um objeto mysqli_result. Os dados serão transportados do servidor MySQL para o PHP. Este método deve ser chamado somente para consultas que produzem um conjunto de resultados.

> **Nota** :
>
> Disponível apenas com o mysqlnd.

> **Nota** :
>
> Esta função não pode ser usada junto com a função mysqli_stmt_store_result(). Ambas estas funções recebem o conjunto completo de resultados do servidor MySQL.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Retorna **`false`** em caso de falha. Para consultas bem sucedidas que produzem um resultado, como `SELECT, SHOW, DESCRIBE` ou `EXPLAIN`, **mysqli_stmt_get_result()** retornará um objeto mysqli_result. Para outras consultas bem sucedidas, **mysqli_stmt_get_result()** retornará **`false`**. A função mysqli_stmt_errno() pode ser usada para distinguir entre os dois motivos para **`false`**; devido a um problema de software, antes do PHP 7.4.13, mysqli_errno() tinha de ser usada para este propósito.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**`MYSQLI_REPORT_ERROR`**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **`MYSQLI_REPORT_STRICT`**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, Population, Continent FROM Country WHERE Continent=? ORDER BY Name LIMIT 1";

$stmt = $mysqli->prepare($query);
$stmt->bind_param("s", $continent);

$continentList = array('Europe', 'Africa', 'Asia', 'North America');

foreach ($continentList as $continent) {
    $stmt->execute();
    $result = $stmt->get_result();
    while ($row = $result->fetch_array(MYSQLI_NUM)) {
        foreach ($row as $r) {
            print "$r ";
        }
        print "\n";
    }
}
```

Exemplo #2 Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, Population, Continent FROM Country WHERE Continent=? ORDER BY Name LIMIT 1";

$stmt = mysqli_prepare($link, $query);
mysqli_stmt_bind_param($stmt, "s", $continent);

$continentList= array('Europe', 'Africa', 'Asia', 'North America');

foreach ($continentList as $continent) {
    mysqli_stmt_execute($stmt);
    $result = mysqli_stmt_get_result($stmt);
    while ($row = mysqli_fetch_array($result, MYSQLI_NUM)) {
        foreach ($row as $r) {
            print "$r ";
        }
        print "\n";
    }
}
```

Os exemplos acima produzirão algo semelhante a:

```
Albania 3401200 Europe
Algeria 31471000 Africa
Afghanistan 22720000 Asia
Anguilla 8000 North America
```

Veja também

- mysqli_prepare() - Prepara uma instrução SQL para execução
- mysqli_stmt_result_metadata() - Retorna os metadados de resultado de uma instrução preparada
- mysqli_stmt_fetch() - Recebe resultados de uma instrução preparada nas variáveis vinculadas
- mysqli_fetch_array() - Recebe a próxima linha de resultado como array associativo, numérico, ou ambos
- mysqli_stmt_store_result() - Armazena um conjunto de resultados em um buffer interno

mysqli_stmt::get_warnings

mysqli_stmt::get_warnings

mysqli_stmt_get_warnings

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

mysqli_stmt::get_warnings -- mysqli_stmt_get_warnings — Obtém o resultado de SHOW WARNINGS

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::get_warnings**(): mysqli_warning|false

Estilo procedural

function **mysqli_stmt_get_warnings**(mysqli_stmt `$statement`): mysqli_warning|false

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

mysqli_stmt::$insert_id

mysqli_stmt::$insert_id

mysqli_stmt_insert_id

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$insert_id -- mysqli_stmt_insert_id — Obtém o ID gerado pela operação INSERT anterior

Descrição

Estilo orientado a objetos

int|string *$mysqli_stmt->insert_id*;

Estilo procedural

function **mysqli_stmt_insert_id**(mysqli_stmt `$statement`): int|string

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

mysqli_stmt::more_results

mysqli_stmt::more_results

mysqli_stmt_more_results

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli_stmt::more_results -- mysqli_stmt_more_results — Verifica se há mais resultados de uma consulta múltipla

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::more_results**(): bool

Estilo procedural:

function **mysqli_stmt_more_results**(mysqli_stmt `$statement`): bool

Verifica se há mais resultados de uma consulta múltipla.

> **Nota** :
>
> Disponível apenas com o mysqlnd.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Retorna `true` se houver mais resultados, caso contrário retorna `false`.

Veja também

- mysqli_stmt::next_result() - Lê o próximo resultado de uma consulta múltipla
- mysqli::multi_query() - Executa uma ou mais consultas no banco de dados

mysqli_stmt::next_result

mysqli_stmt::next_result

mysqli_stmt_next_result

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli_stmt::next_result -- mysqli_stmt_next_result — Lê o próximo resultado de uma consulta múltipla

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::next_result**(): bool

Estilo procedural:

function **mysqli_stmt_next_result**(mysqli_stmt `$statement`): bool

Lê o próximo resultado de uma consulta múltipla.

> **Nota** :
>
> Antes do PHP 8.1.0, disponível somente com mysqlnd.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (`MYSQLI_REPORT_ERROR`) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como `MYSQLI_REPORT_STRICT`, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Registro de Alterações

Versão	Descrição
8.1.0	Agora também disponível com a biblioteca libmysqlclient.

Veja também

- mysqli_stmt::more_results() - Verifica se há mais resultados de uma consulta múltipla
- mysqli::multi_query() - Executa uma ou mais consultas no banco de dados

mysqli_stmt::$num_rows

mysqli_stmt::$num_rows

mysqli_stmt::num_rows

mysqli_stmt_num_rows

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$num_rows -- mysqli_stmt::num_rows -- mysqli_stmt_num_rows — Retorna o número de linhas recebidas pelo servidor

Descrição

Estilo orientado a objetos

int|string *$mysqli_stmt->num_rows*;
public function **mysqli_stmt::num_rows**(): int|string

Estilo procedural

function **mysqli_stmt_num_rows**(mysqli_stmt `$statement`): int|string

Retorna o número de linhas em buffer na instrução. Esta função somente funcionará depois que mysqli_stmt_store_result() for chamara para fazer buffer do conjunto inteiro de resultados no manipulador da instrução.

Esta função retorna `0` a menos que todas as linhas tenham sido recebidas do servidor.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Um int que representa o número de linhas em buffer. Retorna `0` em modo sem buffer a menos que todas as linhas tenham sido recebidas do servidor.

> **Nota** :
>
> Se o número de linhas for maior que `PHP_INT_MAX`, o número será retornado como uma string.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name LIMIT 20";
$stmt = $mysqli->prepare($query);
$stmt->execute();

/* armazena o resultado em um buffer interno */
$stmt->store_result();

printf("Número de linhas: %d.\n", $stmt->num_rows);
```

Exemplo #2 Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name LIMIT 20";
$stmt = mysqli_prepare($link, $query);
mysqli_stmt_execute($stmt);

/* armazena o resultado em um buffer interno */
mysqli_stmt_store_result($stmt);

printf("Número de linhas: %d.\n", mysqli_stmt_num_rows($stmt));
```

Os exemplos acima produzirão:

```
Número de linhas: 20.
```

Veja também

- mysqli_stmt_store_result() - Armazena um conjunto de resultados em um buffer interno
- mysqli_stmt_affected_rows() - Retorna o número total de linhas modificadas, apagadas, inseridas ou correspondidas pela última instrução executada
- mysqli_prepare() - Prepara uma instrução SQL para execução

mysqli_stmt::$param_count

mysqli_stmt::$param_count

mysqli_stmt_param_count

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$param_count -- mysqli_stmt_param_count — Retorna o número de parâmetros para uma instrução fornecida

Descrição

Estilo orientado a objetos

int *$mysqli_stmt->param_count*;

Estilo procedural

function **mysqli_stmt_param_count**(mysqli_stmt `$statement`): int

Retorna o número de marcadores de parâmetros presentes na instrução preparada.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Retorna um inteiro que representa o número de parâmetros.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

if ($stmt = $mysqli->prepare("SELECT Name FROM Country WHERE Name=? OR Code=?")) {

    $marker = $stmt->param_count;
    printf("Instrução tem %d marcadores.\n", $marker);

    /* fecha a instrução */
    $stmt->close();
}

/* fecha a conexão */
$mysqli->close();
?>
```

Exemplo #2 Estilo procedural

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

if ($stmt = mysqli_prepare($link, "SELECT Name FROM Country WHERE Name=? OR Code=?")) {

    $marker = mysqli_stmt_param_count($stmt);
    printf("Instrução tem %d marcadores.\n", $marker);

    /* fecha a instrução */
    mysqli_stmt_close($stmt);
}

/* fecha a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Instrução tem 2 marcadores.
```

Veja também

- mysqli_prepare() - Prepara uma instrução SQL para execução

mysqli_stmt::prepare

mysqli_stmt::prepare

mysqli_stmt_prepare

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::prepare -- mysqli_stmt_prepare — Prepara uma declaração SQL para execução

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::prepare**(string `$query`): bool

Estilo procedural

function **mysqli_stmt_prepare**(mysqli_stmt `$statement`, string `$query`): bool

Prepara uma declaração para execução. A consulta deve consistir de uma única declaração SQL.

O modelo de declaração pode conter zero ou mais marcadores de parâmetro que são os pontos de interrogação (`?`), também chamados de espaços reservados. Os marcadores de parâmetros devem ser ligados às variáveis da aplicação usando mysqli_stmt_bind_param() antes da execução da declaração.

> **Nota** :
>
> No caso de uma declaração passada a **mysqli_stmt_prepare()** ser mais longa que o parâmetro `max_allowed_packet` do servidor, os códigos de erro retornados serão diferentes dependendo se está sendo utilizando o driver nativo MySQL (`mysqlnd`) ou a biblioteca cliente do MySQL (`libmysqlclient`). O comportamento é o seguinte:
>
> - `mysqlnd` no Linux retorna o código de erro 1153. A mensagem de erro diz "got a packet bigger than (recebido pacote maior que) `max_allowed_packet` bytes".
> - `mysqlnd` no Windos retorna o código de erro 2006. Esta mensagem de erro diz "server has gone away" (servidor se foi).
> - `libmysqlclient` em todas as plataformas a retorna o código de erro 2006. Esta mensagem de erro diz "server has gone away" (servidor se foi).

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

`query`

A consulta, como uma string. Deve consistir de uma única declaração SQL.

A declaração SQL pode conter zero ou mais marcadores de parâmetros representados por caracteres de ponto de interrogação (`?`) nas posições apropriadas.

> **Nota** :
>
> Os marcadores são permitidos somente em alguns lugares das declarações SQL. Por exemplo, eles são permitidos na lista `VALUES()` de Uma declaração `INSERT` (para especificar valores de colunas para uma linha), ou em uma comparação com uma coluna em uma cláusula `WHERE` para especificar um valor de comparação. Entretanto, não são permitidos para identificadores (como tabelas ou nomes de colunas).

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**`MYSQLI_REPORT_ERROR`**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **`MYSQLI_REPORT_STRICT`**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Exemplo de mysqli_stmt::prepare()

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$city = "Amersfoort";

/* cria uma declaração preparada */
$stmt = $mysqli->stmt_init();
$stmt->prepare("SELECT District FROM City WHERE Name=?");

/* liga parâmetros para os marcadores */
$stmt->bind_param("s", $city);

/* executa a consulta */
$stmt->execute();

/* liga o resultado à variável */
$stmt->bind_result($district);

/* obtém o valor */
$stmt->fetch();

printf("%s está no distrito %s\n", $city, $district);
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$city = "Amersfoort";

/* cria uma declaração preparada */
$stmt = mysqli_stmt_init($link);
mysqli_stmt_prepare($stmt, "SELECT District FROM City WHERE Name=?");

/* liga parâmetros para os marcadores */
mysqli_stmt_bind_param($stmt, "s", $city);

/* executa a consulta */
mysqli_stmt_execute($stmt);

/* liga o resultado à variável */
mysqli_stmt_bind_result($stmt, $district);

/* obtém o valor */
mysqli_stmt_fetch($stmt);

printf("%s está no distrito %s\n", $city, $district);
```

Os exemplos acima produzirão:

```
Amersfoort está no distrito Utrecht
```

Veja também

- mysqli_stmt_init() - Inicializa uma instrução e retorna um objeto para uso com mysqli_stmt_prepare
- mysqli_stmt_execute() - Executa uma instrução preparada
- mysqli_stmt_fetch() - Recebe resultados de uma instrução preparada nas variáveis vinculadas
- mysqli_stmt_bind_param() - Liga variáveis a uma declaração preparada como parâmetros
- mysqli_stmt_bind_result() - Liga variáveis a uma declaração preparada para armazenamento de resultado
- mysqli_stmt_get_result() - Obtém um resultado de uma instrução preparada como um objeto mysqli_result
- mysqli_stmt_close() - Fecha uma instrução preparada

mysqli_stmt::reset

mysqli_stmt::reset

mysqli_stmt_reset

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::reset -- mysqli_stmt_reset — Redefine uma instrução preparada

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::reset**(): bool

Estilo procedural

function **mysqli_stmt_reset**(mysqli_stmt `$statement`): bool

Redefine uma instrução preparada no cliente e no servidor para o estado pós-preparação.

A função redefine a instrução no servidor, os dados enviados por mysqli_stmt_send_long_data(), os resultados sem buffer e os erros atuais. Ela não limpa vinculações de variáveis ou resultados armazenados. Resultados armazenados serão limpos ao executar a instrução preparada (ou ao fechá-la).

Para preparar uma instrução com outra consulta, utilize a função mysqli_stmt_prepare().

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Veja também

- mysqli_prepare() - Prepara uma instrução SQL para execução

mysqli_stmt::result_metadata

mysqli_stmt::result_metadata

mysqli_stmt_result_metadata

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::result_metadata -- mysqli_stmt_result_metadata — Retorna os metadados de resultado de uma instrução preparada

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::result_metadata**(): mysqli_result|false

Estilo procedural

function **mysqli_stmt_result_metadata**(mysqli_stmt `$statement`): mysqli_result|false

Se uma instrução passada a mysqli_prepare() for do tipo que produz um conjunto de resultados, **mysqli_stmt_result_metadata()** retornará o objeto de resultado que pode ser usado para processar a meta informação como o número total de campos e informações individual dos campos.

Esta função retorna um objeto vazio mysqli_result que pode ser usado para acessar informações de metadados da instrução preparada sem precisar buscar as linhas de dados reais. Não é necessário usar esta função ao usar mysqli_stmt_get_result() para recuperar todo o conjunto de resultados de uma instrução preparada como um objeto de resultado.

> **Nota** :
>
> Este objeto de conjunto de resultados pode ser passado como argumento apenas para as funções baseadas em campo que processam metadados de resultados, como:
>
> - mysqli_num_fields()
> - mysqli_fetch_field()
> - mysqli_fetch_field_direct()
> - mysqli_fetch_fields()
> - mysqli_field_count()
> - mysqli_field_seek()
> - mysqli_field_tell()
> - mysqli_free_result()

> **Nota** :

> O conjunto de resultados retornado por **mysqli_stmt_result_metadata()** contém apenas metadados. Não contém nenhum resultado de linha. As linhas são obtidas chamando mysqli_stmt_get_result() no identificador da instrução ou com mysqli_stmt_fetch().

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Retorna um objeto de resultado ou **`false`** se um erro ocorrer. Se a instrução não produzir um conjunto de resultados, **`false`** também será retornado.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**`MYSQLI_REPORT_ERROR`**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **`MYSQLI_REPORT_STRICT`**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "test");

$mysqli->query("DROP TABLE IF EXISTS friends");
$mysqli->query("CREATE TABLE friends (id int, name varchar(20))");

$mysqli->query("INSERT INTO friends VALUES (1,'Hartmut'), (2, 'Ulf')");

$stmt = $mysqli->prepare("SELECT id, name FROM friends");
$stmt->execute();

/* obtém o resultado de metadados */
$result = $stmt->result_metadata();

/* obtém informação do campo a partir do resultado de metadados */
$field = $result->fetch_field();

printf("Nome do campo: %s\n", $field->name);
?>
```

Exemplo #2 Estilo procedural

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "test");

mysqli_query($link, "DROP TABLE IF EXISTS friends");
mysqli_query($link, "CREATE TABLE friends (id int, name varchar(20))");

mysqli_query($link, "INSERT INTO friends VALUES (1,'Hartmut'), (2, 'Ulf')");

$stmt = mysqli_prepare($link, "SELECT id, name FROM friends");
```

```php
mysqli_stmt_execute($stmt);

/* obtém o resultado de metadados */
$result = mysqli_stmt_result_metadata($stmt);

/* obtém informação do campo a partir do resultado de metadados */
$field = mysqli_fetch_field($result);

printf("Nome do campo: %s\n", $field->name);
?>
```

Veja também

- mysqli_prepare() - Prepara uma instrução SQL para execução
- mysqli_free_result() - Libera memória associada a um resultado
- mysqli_stmt_get_result() - Obtém um resultado de uma instrução preparada como um objeto mysqli_result

mysqli_stmt::send_long_data

mysqli_stmt::send_long_data

mysqli_stmt_send_long_data

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::send_long_data -- mysqli_stmt_send_long_data — Envia dados em blocos

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::send_long_data**(int `$param_num`, string `$data`): bool

Estilo procedural

function **mysqli_stmt_send_long_data**(mysqli_stmt `$statement`, int `$param_num`, string `$data`): bool

Permite enviar dados de parâmetros ao servidor em pedaços (ou blocos), por exemplo, se o tamanho de um BLOB exceder o tamanho de `max_allowed_packet`. Esta função pode ser chamada múltiplas vezes para enviar as partes de um valor de dados binários ou de caracteres para uma coluna, que precisa ser do tipo TEXT ou BLOB.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

`param_num`

Indica a qual parâmetro os dados serão associados. Parêmetros são numerados a partir de 0.

`data`

Uma string contendo os dados que devem ser enviados.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php
$stmt = $mysqli->prepare("INSERT INTO messages (message) VALUES (?)");
$null = NULL;
$stmt->bind_param("b", $null);
$fp = fopen("messages.txt", "r");
while (!feof($fp)) {
    $stmt->send_long_data(0, fread($fp, 8192));
}
fclose($fp);
$stmt->execute();
?>
```

Veja também

- mysqli_prepare() - Prepara uma instrução SQL para execução
- mysqli_stmt_bind_param() - Liga variáveis a uma declaração preparada como parâmetros

mysqli_stmt::$sqlstate

mysqli_stmt::$sqlstate

mysqli_stmt_sqlstate

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$sqlstate -- mysqli_stmt_sqlstate — Retorna o erro SQLSTATE da operação da instrução anterior

Descrição

Estilo orientado a objetos

string *$mysqli_stmt->sqlstate*;

Estilo procedural

function **mysqli_stmt_sqlstate**(mysqli_stmt `$statement`): string

Retorna uma string contento o código de erro SQLSTATE para a função de instrução preparada mais recentemente chamada, que possa ter sucesso ou falhar. O código de erro consiste de cinco caracteres. `'00000'` significa que não houve erro. Os valores são especificados pela ANSI SQL e pela ODBC. Para uma lista de valores possíveis, consulte » http://dev.mysql.com/doc/mysql/en/error-handling.html.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Retorna uma string contendo o código de erro SQLSTATE para o último erro. O código de erro consiste de cinco caracteres. `'00000'` significa que não houve erro.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php
/* abre uma conexão */
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

$mysqli->query("CREATE TABLE myCountry LIKE Country");
$mysqli->query("INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = $mysqli->prepare($query)) {

    /* apaga a tabela */
    $mysqli->query("DROP TABLE myCountry");

    /* executa a instrução */
    $stmt->execute();

    printf("Erro: %s.\n", $stmt->sqlstate);

    /* fecha a instrução */
    $stmt->close();
}

/* fecha a conexão */
$mysqli->close();
?>
```

Exemplo #2 Estilo procedural

```php
<?php
/* abre uma conexão */
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

mysqli_query($link, "CREATE TABLE myCountry LIKE Country");
mysqli_query($link, "INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = mysqli_prepare($link, $query)) {

    /* apaga a tabela */
    mysqli_query($link, "DROP TABLE myCountry");
```

```
    /* executa a instrução */
    mysqli_stmt_execute($stmt);

    printf("Error: %s.\n", mysqli_stmt_sqlstate($stmt));

    /* fecha a instrução */
    mysqli_stmt_close($stmt);
}

/* fecha a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Erro: 42S02.
```

Notas

> **Nota** :
>
> Observe que nem todos os erros do MySQL são mapeados para SQLSTATEs. O valor `HY000` (erro geral) é usado para erros não mapeados.

Veja também

- mysqli_stmt_errno() - Retorna o código de erro para a chamada de instrução mais recente
- mysqli_stmt_error() - Retorna uma descrição em string para o último erro de instrução

mysqli_stmt::store_result

mysqli_stmt::store_result

mysqli_stmt_store_result

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::store_result -- mysqli_stmt_store_result — Armazena um conjunto de resultados em um buffer interno

Descrição

Estilo orientado a objetos

public function **mysqli_stmt::store_result**(): bool

Estilo procedural

function **mysqli_stmt_store_result**(mysqli_stmt `$statement`): bool

Esta função deve ser chamada, para consultas que produzem com sucesso um conjunto de resultados (ex.: `SELECT, SHOW, DESCRIBE, EXPLAIN`), seomente se o resultado completo precisa ser mantido em buffer no PHP. Cada chamada subsequente a mysqli_stmt_fetch() retornará dados em buffer.

> **Nota** :
>
> É desnecessário chamar **mysqli_stmt_store_result()** para outras conultas, mas se isto for feito, não irá causar dano ou qualquer perda notável de desempenho em qualquer caso. Pode-se detectar se a consulta produziu resultado verificando se a função mysqli_stmt_result_metadata() retorna **`false`**.

Parâmetros

`statement`

Somente no estilo procedural: Um objeto mysqli_stmt retornado por mysqli_stmt_init().

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**`MYSQLI_REPORT_ERROR`**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **`MYSQLI_REPORT_STRICT`**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name LIMIT 20";
$stmt = $mysqli->prepare($query);
$stmt->execute();

/* armazena o resultado em um buffer interno */
$stmt->store_result();

printf("Número de linhas: %d.\n", $stmt->num_rows);
```

Exemplo #2 Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name LIMIT 20";
$stmt = mysqli_prepare($link, $query);
mysqli_stmt_execute($stmt);
```

```php
/* armazena o resultado em um buffer interno */
mysqli_stmt_store_result($stmt);

printf("Número de linhas: %d.\n", mysqli_stmt_num_rows($stmt));
```

Os exemplos acima produzirão:

```
Número de linhas: 20.
```

Veja também

- mysqli_prepare() - Prepara uma instrução SQL para execução
- mysqli_stmt_result_metadata() - Retorna os metadados de resultado de uma instrução preparada
- mysqli_stmt_fetch() - Recebe resultados de uma instrução preparada nas variáveis vinculadas

mysqli_result

(PHP 5, PHP 7, PHP 8)

Introdução

Representa o conjunto de resultados da consulta feita ao banco de dados.

Resumo da classe

class **mysqli_result** implements IteratorAggregate {
/* Propriedades */
public readonly int *$current_field*;
public readonly int *$field_count*;
public readonly ?array *$lengths*;
public readonly int|string *$num_rows*;
public int *$type*;
/* Métodos */
public function __construct(mysqli `$mysql`, int `$result_mode` = **`MYSQLI_STORE_RESULT`**)
public function data_seek(int `$offset`): bool
public function fetch_all(int `$mode` = **`MYSQLI_NUM`**): array
public function fetch_array(int `$mode` = **`MYSQLI_BOTH`**): array|null|false
public function fetch_assoc(): array|null|false
public function fetch_column(int `$column` = 0): null|int|float|string|false
public function fetch_field(): object|false
public function fetch_field_direct(int `$index`): object|false
public function fetch_fields(): array
public function fetch_object(string `$class` = "stdClass", array `$constructor_args` = []): object|null|false
public function fetch_row(): array|null|false
public function field_seek(int `$index`): true
public function free(): void
public function close(): void
public function free_result(): void

public function getIterator(): Iterator
}

Propriedades

type

Armazena informação se o o resultado é com ou sem buffer, como um int (**MYSQLI_STORE_RESULT** ou **MYSQLI_USE_RESULT**, respectivamente).

Registro de Alterações

Versão	Descrição
8.0.0	**mysqli_result** agora implementa IteratorAggregate. Anteriormente, Traversable era implementado.

Índice

mysqli_result::__construct

mysqli_result::__construct

(PHP 5, PHP 7, PHP 8)

mysqli_result::__construct — Constrói um objeto mysqli_result

Descrição

public function **mysqli_result::__construct**(mysqli `$mysql`, int `$result_mode` = **MYSQLI_STORE_RESULT**)

Este método constrói um novo objeto mysqli_result.

Pode ser usado para criar o objeto mysqli_result após a chamada à função mysqli_real_query() ou mysqli_multi_query(). Construir o objeto manualmente é equivalente a chamar a função mysqli_store_result() ou mysqli_use_result().

Parâmetros

`mysql`

Somente no estilo procedural: Um objeto mysqli retornado por mysqli_connect() ou mysqli_init()

`result_mode`

O modo de resultado pode ser uma de 2 constantes que indicam como o resultado será retornado do servidor MySQL.

MYSQLI_STORE_RESULT (padrão) - cria um objeto mysqli_result com resultado em buffer.

MYSQLI_USE_RESULT - cria um objetomysqli_result com resultado sem buffer. Enquanto houver registros pendentes esperando ser recebidos, a linha de conexão estará ocupada e todas as chamadas subsequentes retornarão o erro `Commands out of sync` (comandos fora de sincronismo). Para evitar o erro, todos os registros devem ser recebidos do servidor ou o resultado deve ser descartado através da função mysqli_free_result(). A conexão precisa permanecer aberta para que as linhas sejam recebidas.

Erros/Exceções

Se o relatório de erros da extensão mysqli estiver habilitado (**MYSQLI_REPORT_ERROR**) e a operação solicitada falhar, um aviso será gerado. Se, além disso, o modo for definido como **MYSQLI_REPORT_STRICT**, uma exceção mysqli_sql_exception será lançada em vez do aviso.

Examples

Exemplo #1 Criação de um objeto mysqli_result

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* Consultas SELECT retornam um resultado */
$mysqli->real_query("SELECT Name FROM City LIMIT 10");

$result = new mysqli_result($mysqli);
printf("SELECT retornou %d linhas.\n", $result->num_rows);
```

Os exemplos acima produzirão algo semelhante a:

```
SELECT retornou 10 linhas.
```

Veja também

- mysqli_multi_query() - Executa uma ou mais consultas no banco de dados
- mysqli_real_query() - Executa uma consulta SQL
- mysqli_store_result() - Transfere um conjunto de resultados da última consulta
- mysqli_use_result() - Inicia uma recuperação de conjunto de resultados

mysqli_result::$current_field

mysqli_result::$current_field

mysqli_field_tell

(PHP 5, PHP 7, PHP 8)

mysqli_result::$current_field -- mysqli_field_tell — Obtém o deslocamento atual de campo de um ponteiro de resultado

Descrição

Estilo orientado a objetos

int *$mysqli_result->current_field*;

Estilo procedural

function **mysqli_field_tell**(mysqli_result `$result`): int

Retorna a posição do cursor de campo usado na última chamada a mysqli_fetch_field(). Este valor pode ser usado como um argumento para mysqli_field_seek().

Parâmetros

`result`

Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

Valor Retornado

Retorna o deslocamento atual do cursor de campos.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Code LIMIT 5";

if ($result = $mysqli->query($query)) {

    /* Obtém informação de campo para todas as colunas */
    while ($finfo = $result->fetch_field()) {

        /* obtém o deslocamento do ponteiro de campos */
        $currentfield = $result->current_field;

        printf("Column %d:\n", $currentfield);
        printf("Name:     %s\n", $finfo->name);
        printf("Table:    %s\n", $finfo->table);
        printf("max. Len: %d\n", $finfo->max_length);
        printf("Flags:    %d\n", $finfo->flags);
        printf("Type:     %d\n\n", $finfo->type);
    }
    $result->close();
}

/* fecha a conexão */
$mysqli->close();
?>
```

Exemplo #2 Estilo procedural

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
```

```
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Code LIMIT 5";

if ($result = mysqli_query($link, $query)) {

    /* Obtém informação de campo para todas as colunas */
    while ($finfo = mysqli_fetch_field($result)) {

        /* obtém o deslocamento do ponteiro de campos */
        $currentfield = mysqli_field_tell($result);

        printf("Column %d:\n", $currentfield);
        printf("Name:     %s\n", $finfo->name);
        printf("Table:    %s\n", $finfo->table);
        printf("max. Len: %d\n", $finfo->max_length);
        printf("Flags:    %d\n", $finfo->flags);
        printf("Type:     %d\n\n", $finfo->type);
    }
    mysqli_free_result($result);
}

/* fecha a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Column 1:
Name:     Name
Table:    Country
max. Len: 11
Flags:    1
Type:     254

Column 2:
Name:     SurfaceArea
Table:    Country
max. Len: 10
Flags:    32769
Type:     4
```

Veja também

- mysqli_fetch_field() - Retorna o próximo campo no conjunto de resultados
- mysqli_field_seek() - Move o ponteiro do resultado para um campo especificado

mysqli_result::data_seek

mysqli_result::data_seek

mysqli_data_seek

(PHP 5, PHP 7, PHP 8)

mysqli_result::data_seek -- mysqli_data_seek — Ajusta o ponteiro do resultado para uma linha arbitrária no resultado

Descrição

Estilo orientado a objetos

public function **mysqli_result::data_seek**(int `$offset`): bool

Estilo procedural

function **mysqli_data_seek**(mysqli_result `$result`, int `$offset`): bool

A função **mysqli_data_seek()** move o ponteiro para uma posição arbitrária especificada pelo parâmetro `offset` no conjunto de resultados.

Parâmetros

`result`

> Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

`offset`

> O deslocamento de linha. deve estar entre zero e o número total de linhas menos uma (0..mysqli_num_rows() - 1).

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo de mysqli::data_seek()

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name";
$result = $mysqli->query($query);

/* Vai até a linha 401 */
$result->data_seek(400);

/* Recebe uma única linha */
$row = $result->fetch_row();

printf("Cidade: %s  Código do país: %s\n", $row[0], $row[1]);
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name";

$result = mysqli_query($link, $query);

/* Vai até a linha 401 */
mysqli_data_seek($result, 400);

/* Recebe uma única linha */
$row = mysqli_fetch_row($result);

printf ("Cidade: %s  Código do país: %s\n", $row[0], $row[1]);
```

Os exemplos acima produzirão:

```
Cidade: Benin City  Código do país: NGA
```

Exemplo #2 Ajustando o ponteiro de resultado na iteração

Esta função pode ser útil quando iterar sobre o conjunto de resultados para impor uma ordem customizada ou para retroceder o conjunto de resultados ao iterar múltiplas vezes.

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name LIMIT 15,4";
$result = $mysqli->query($query);

/* Itera o conjunto de resultados na ordem inversa */
for ($row_no = $result->num_rows - 1; $row_no >= 0; $row_no--) {
    $result->data_seek($row_no);

    /* Recebe uma linha */
    $row = $result->fetch_row();

    printf("Cidade: %s  Código do país: %s\n", $row[0], $row[1]);
}

/* Redefine o ponteiro para o início do conjunto de resultados */
$result->data_seek(0);

print "\n";

/* Itera o mesmo conjunto novamente */
```

```
while ($row = $result->fetch_row()) {
    printf("Cidade: %s  Código do país: %s\n", $row[0], $row[1]);
}
```

Os exemplos acima produzirão:

```
Cidade: Acmbaro  Código do país: MEX
Cidade: Abuja  Código do país: NGA
Cidade: Abu Dhabi  Código do país: ARE
Cidade: Abottabad  Código do país: PAK

Cidade: Abottabad  Código do país: PAK
Cidade: Abu Dhabi  Código do país: ARE
Cidade: Abuja  Código do país: NGA
Cidade: Acmbaro  Código do país: MEX
```

Notas

> **Nota** :
>
> Esta função somente pode ser usada com resultados em buffer obtidos do uso das funções mysqli_store_result(), mysqli_query() ou mysqli_stmt_get_result().

Veja também

- mysqli_store_result() - Transfere um conjunto de resultados da última consulta
- mysqli_fetch_row() - Obtém a próxima linha de um resultado como um array enumerado
- mysqli_fetch_array() - Recebe a próxima linha de resultado como array associativo, numérico, ou ambos
- mysqli_fetch_assoc() - Obtém a próxima linha do conjunto de resultados como um array associativo
- mysqli_fetch_object() - Recebe a próxima linha de um resultado como um objeto
- mysqli_query() - Executa uma consulta no banco de dados
- mysqli_num_rows() - Obtém o número de linhas no resultado

mysqli_result::fetch_all

mysqli_result::fetch_all

mysqli_fetch_all

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli_result::fetch_all -- mysqli_fetch_all — Recebe todas as linhas de resultado como array associativo, numérico, ou ambos

Descrição

Estilo orientado a objetos

public function **mysqli_result::fetch_all**(int `$mode` = `MYSQLI_NUM`): array

Estilo procedural

function **mysqli_fetch_all**(mysqli_result `$result`, int `$mode` = `MYSQLI_NUM`): array

Retorna um array bidimensional com todas as linhas do resultado como um array associativo, numérico, ou ambos.

> **Nota** :
>
> Antes do PHP 8.1.0, disponível somente com mysqlnd.

Parâmetros

`result`

Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

`mode`

Este parâmetro opcional é uma constante que indica que tipo de array deve ser produzido a partir dos dados da linha atual. Os valores possíveis para este parâmetro são as constantes `MYSQLI_ASSOC`, `MYSQLI_NUM` ou `MYSQLI_BOTH`.

Valor Retornado

Retorna um array de arrays associativos ou numéricos com as linhas do resultado.

Registro de Alterações

Versão	Descrição
8.1.0	Agora também disponível com a biblioteca libmysqlclient.

Examples

Exemplo #1 Exemplo de **mysqli_result::fetch_all()**

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$result = $mysqli->query("SELECT Name, CountryCode FROM City ORDER BY ID LIMIT 3");

$rows = $result->fetch_all(MYSQLI_ASSOC);
foreach ($rows as $row) {
    printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);
}
```

Estilo procedural

```php
<?php
```

```php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$result = mysqli_query($mysqli, "SELECT Name, CountryCode FROM City ORDER BY ID LIMIT 3");

$rows = mysqli_fetch_all($result, MYSQLI_ASSOC);
foreach ($rows as $row) {
    printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);
}
```

Os exemplos acima produzirão:

```
Kabul (AFG)
Qandahar (AFG)
Herat (AFG)
```

Veja também

- mysqli_fetch_array() - Recebe a próxima linha de resultado como array associativo, numérico, ou ambos
- mysqli_fetch_column() - Recebe uma única coluna da próxima linha de um resultado
- mysqli_query() - Executa uma consulta no banco de dados

mysqli_result::fetch_array

mysqli_result::fetch_array

mysqli_fetch_array

(PHP 5, PHP 7, PHP 8)

mysqli_result::fetch_array -- mysqli_fetch_array — Recebe a próxima linha de resultado como array associativo, numérico, ou ambos

Descrição

Estilo orientado a objetos

public function **mysqli_result::fetch_array**(int `$mode` = **`MYSQLI_BOTH`**): array|null|false

Estilo procedural

function **mysqli_fetch_array**(mysqli_result `$result`, int `$mode` = **`MYSQLI_BOTH`**): array|null|false

Recebe uma linha de dados do conjunto de resultados e retorna-a como um array. Cada chamada subsequente a esta função retornará a linha seguinte dentro do conjunto de resultados, ou **`null`** se não houver mais linhas.

Além de armazenar os dados nos índices numéricos do array resultante, esta função também pode armazenar os dados em índices associativos usando os nomes dos campos do conjunto de resultados como chaves.

Se duas ou mais colunas do resultado tiverem o mesmo nome, a última coluna terá precedência e irá sobrescrever os dados anteriores. Para acessar múltiplas colunas com o mesmo nome, deve ser usada a versão

da linha indexada numericamente.

> **Nota** :
>
> Os nomes de campos retornados por esta função *diferenciam maiúsculas e minúsculas*.

> **Nota** :
>
> Esta função define campos NULL como o valor `null` do PHP.

Parâmetros

`result`

Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

`mode`

O segundo argumento é uma constante que indica qual tipo de array deve ser produzido a partir dos dados da linha atual. Os valores possíveis para este parâmetro são as constantes `MYSQLI_ASSOC`, `MYSQLI_NUM` ou `MYSQLI_BOTH`.

Ao usar a constante `MYSQLI_ASSOC`, esta função irá se comportar de modo idêntico à função mysqli_fetch_assoc(), enquanto `MYSQLI_NUM` irá se comportar de modo idêntico à função mysqli_fetch_row(). A opção final `MYSQLI_BOTH` irá criar um único array com os atributos de ambas.

Valor Retornado

Retorna um array que representa a linha recebida, `null` se não houver mais linhas no resultado, ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de mysqli_result::fetch_array()

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY ID LIMIT 3";
$result = $mysqli->query($query);

/* array com índice numérico */
$row = $result->fetch_array(MYSQLI_NUM);
printf("%s (%s)\n", $row[0], $row[1]);

/* array associativo */
$row = $result->fetch_array(MYSQLI_ASSOC);
printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);
```

```php
/* array com índice numérico e associativo */
$row = $result->fetch_array(MYSQLI_BOTH);
printf("%s (%s)\n", $row[0], $row["CountryCode"]);
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER by ID LIMIT 3";
$result = mysqli_query($mysqli, $query);

/* array com índice numérico */
$row = mysqli_fetch_array($result, MYSQLI_NUM);
printf("%s (%s)\n", $row[0], $row[1]);

/* array associativo */
$row = mysqli_fetch_array($result, MYSQLI_ASSOC);
printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);

/* array com índice numérico e associativo */
$row = mysqli_fetch_array($result, MYSQLI_BOTH);
printf("%s (%s)\n", $row[0], $row["CountryCode"]);
```

Os exemplos acima produzirão algo semelhante a:

```
Kabul (AFG)
Qandahar (AFG)
Herat (AFG)
```

Veja também

- mysqli_fetch_assoc() - Obtém a próxima linha do conjunto de resultados como um array associativo
- mysqli_fetch_column() - Recebe uma única coluna da próxima linha de um resultado
- mysqli_fetch_row() - Obtém a próxima linha de um resultado como um array enumerado
- mysqli_fetch_object() - Recebe a próxima linha de um resultado como um objeto
- mysqli_query() - Executa uma consulta no banco de dados
- mysqli_data_seek() - Ajusta o ponteiro do resultado para uma linha arbitrária no resultado

mysqli_result::fetch_assoc

mysqli_result::fetch_assoc

mysqli_fetch_assoc

(PHP 5, PHP 7, PHP 8)

mysqli_result::fetch_assoc -- mysqli_fetch_assoc — Obtém a próxima linha do conjunto de resultados como um array associativo

Descrição

Estilo orientado a objetos

public function **mysqli_result::fetch_assoc**(): array|null|false

Estilo procedural

function **mysqli_fetch_assoc**(mysqli_result `$result`): array|null|false

Obtém uma linha de dados do conjunto de resultados e retorna-o como um array associativo. Cada chamada subsequente desta função retornará a linha seguinte do conjunto de resultados, ou **`null`** se não houver mais linhas.

Se duas ou mais colunas do resultado tiverem o mesmo nome, a última coluna terá precedência e irá sobrescrever qualquer dado anterior. Para acessar múltiplas colunas com o mesmo nome, mysqli_fetch_row() pode ser usada para obter o array indexado numericamente, ou apelidos podem ser usados na lista de seleção da consulta SQL para dar nomes diferentes às colunas.

> **Nota** :
>
> Os nomes de campos retornados por esta função *diferenciam maiúsculas e minúsculas*.

> **Nota** :
>
> Esta função define campos NULL como o valor **`null`** do PHP.

Parâmetros

`result`

> Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

Valor Retornado

Retorna um array associativo que representa a linha obtida, onde cada chave no array representa o nome de uma das colunas do conjunto de resultados; **`null`** se não houver mais linhas no conjunto de resultados, ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo de **mysqli_result::fetch_assoc()**

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY ID DESC";

$result = $mysqli->query($query);
```

```php
/* obtém o array associativo */
while ($row = $result->fetch_assoc()) {
    printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);
}
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY ID DESC";

$result = mysqli_query($mysqli, $query);

/* obtém o array associativo */
while ($row = mysqli_fetch_assoc($result)) {
    printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);
}
```

Os exemplos acima produzirão algo semelhante a:

```
Pueblo (USA)
Arvada (USA)
Cape Coral (USA)
Green Bay (USA)
Santa Clara (USA)
```

Exemplo #2 Comparação de mysqli_result iterator e uso de mysqli_result::fetch_assoc()

mysqli_result pode ser iterada usando-se `foreach`. O conjunto de resultados será iterado sempre a partir da primeira linha, independente da posição atual do ponteiro.

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = 'SELECT Name, CountryCode FROM City ORDER BY ID DESC';

// Usando iterardores
$result = $mysqli->query($query);
foreach ($result as $row) {
    printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);
}

echo "\n==================\n";

// Não usando iteradores
$result = $mysqli->query($query);
```

```php
while ($row = $result->fetch_assoc()) {
    printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);
}
```

O exemplo acima produzirá algo semelhante a:

```
Pueblo (USA)
Arvada (USA)
Cape Coral (USA)
Green Bay (USA)
Santa Clara (USA)

==================
Pueblo (USA)
Arvada (USA)
Cape Coral (USA)
Green Bay (USA)
Santa Clara (USA)
```

Veja também

- mysqli_fetch_array() - Recebe a próxima linha de resultado como array associativo, numérico, ou ambos
- mysqli_fetch_column() - Recebe uma única coluna da próxima linha de um resultado
- mysqli_fetch_row() - Obtém a próxima linha de um resultado como um array enumerado
- mysqli_fetch_object() - Recebe a próxima linha de um resultado como um objeto
- mysqli_query() - Executa uma consulta no banco de dados
- mysqli_data_seek() - Ajusta o ponteiro do resultado para uma linha arbitrária no resultado

mysqli_result::fetch_column

mysqli_result::fetch_column

mysqli_fetch_column

(PHP 8 >= 8.1.0)

mysqli_result::fetch_column -- mysqli_fetch_column — Recebe uma única coluna da próxima linha de um resultado

Descrição

Estilo orientado a objetos

public function **mysqli_result::fetch_column**(int `$column` = 0): null|int|float|string|false

Estilo procedural

function **mysqli_fetch_column**(mysqli_result `$result`, int `$column` = 0): null|int|float|string|false

Recebe uma linha de dados do resultado e retorna a coluna indexada a partir de 0. Cada chamada subsequente desta função retornará o valor da linha seguinte dentro do conjunto de resultados, ou **`false`** se não houver mais

linhas.

> **Nota** :
>
> Esta função define campos NULL como o valor `null` do PHP.

Parâmetros

`result`

Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

`column`

Número da coluna que se deseja receber da linha, sendo que a primeira é indexada em 0. Se nenhum valor for fornecido, a primeira coluna será retornada.

Valor Retornado

Retorna uma única coluna da próxima linha de um conjunto de resultados ou `false` se não houver mais linhas.

> **Aviso**
>
> Não há como retornar outra coluna da mesma linha se esta função for usada para obter dados.

Examples

Exemplo #1 Exemplo de mysqli_result::fetch_column()

Estilo orientado a objetos

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT CountryCode, Name FROM City ORDER BY ID DESC LIMIT 5";

$result = $mysqli->query($query);

/* recebe um único valor da segunda coluna */
while ($Name = $result->fetch_column(1)) {
    printf("%s\n", $Name);
}
```

Estilo procedural

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT CountryCode, Name FROM City ORDER BY ID DESC LIMIT 5";
```

```php
$result = mysqli_query($mysqli, $query);

/* recebe um único valor da segunda coluna */
while ($Name = mysqli_fetch_column($result, 1)) {
    printf("%s\n", $Name);
}
```

Os exemplos acima produzirão algo semelhante a:

```
Rafah
Nablus
Jabaliya
Hebron
Khan Yunis
```

Veja também

- mysqli_fetch_all() - Recebe todas as linhas de resultado como array associativo, numérico, ou ambos
- mysqli_fetch_array() - Recebe a próxima linha de resultado como array associativo, numérico, ou ambos
- mysqli_fetch_assoc() - Obtém a próxima linha do conjunto de resultados como um array associativo
- mysqli_fetch_object() - Recebe a próxima linha de um resultado como um objeto
- mysqli_fetch_row() - Obtém a próxima linha de um resultado como um array enumerado
- mysqli_data_seek() - Ajusta o ponteiro do resultado para uma linha arbitrária no resultado

mysqli_result::fetch_field

mysqli_result::fetch_field

mysqli_fetch_field

(PHP 5, PHP 7, PHP 8)

mysqli_result::fetch_field -- mysqli_fetch_field — Retorna o próximo campo no conjunto de resultados

Descrição

Estilo orientado a objetos

public function **mysqli_result::fetch_field**(): object|false

Estilo procedural

function **mysqli_fetch_field**(mysqli_result `$result`): object|false

Retorna a definição de uma coluna de um conjunto de resultados como um objeto. Utilize esta função repetidamente para obter informação sobre todas as colunas no conjunto de resultados.

Parâmetros

`result`

Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

Valor Retornado

Retorna um objeto que contém informações sobre a definição do campo ou `false` se nenhuma informação de campos estiver disponível.

Propriedades dos objetos

Propriedade	Descrição
name	O nome da coluna
orgname	O nome original da coluna se foi especificado um apelido
table	O nome da tabela à qual o campo pertence (se não calculado)
orgtable	Nome da tabela original se foi especificado um apelido
def	Não utilizado. Sempre uma string vazia
db	O nome do banco de dados
catalog	Não utilizado. Sempre `"def"`
max_length	A largura máxima do campo para o conjunto de resultados. A partir do PHP 8.1, este valor é sempre `0`.
length	O comprimento do campo em bytes. Para colunas de strings, o valor do comprimento depende do conjunto de caracteres. Por exemplo, se o conjunto de caracteres for `latin1`, um conjunto de um byte, o valor de comprimento para uma consulta `SELECT 'abc'` será 3. Se o conjunto for `utf8mb4`, um conjunto multi-bytes onde os caracteres podem ter até 4 bytes, o valor de comprimento será 12.
charsetnr	O número do conjunto de caracteres do campo.
flags	Um inteiro representando bits de opções para o campo.
type	O tipo de dados usado para este campo
decimals	O número de decimais para campos numéricos, e a precisão em segundos fracionários para campos temporais.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Code LIMIT 5";

if ($result = $mysqli->query($query)) {

    /* Obtém informação do campo para todos os campos */
    while ($finfo = $result->fetch_field()) {
```

```
        printf("Name:     %s\n", $finfo->name);
        printf("Table:    %s\n", $finfo->table);
        printf("max. Len: %d\n", $finfo->max_length);
        printf("Flags:    %d\n", $finfo->flags);
        printf("Type:     %d\n\n", $finfo->type);
    }
    $result->close();
}

/* fecha a conexão */
$mysqli->close();
?>
```

Exemplo #2 Estilo procedural

```
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Code LIMIT 5";

if ($result = mysqli_query($link, $query)) {

    /* Obtém informação do campo para todos os campos */
    while ($finfo = mysqli_fetch_field($result)) {

        printf("Name:     %s\n", $finfo->name);
        printf("Table:    %s\n", $finfo->table);
        printf("max. Len: %d\n", $finfo->max_length);
        printf("Flags:    %d\n", $finfo->flags);
        printf("Type:     %d\n\n", $finfo->type);
    }
    mysqli_free_result($result);
}

/* fecha a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Name:     Name
Table:    Country
max. Len: 11
Flags:    1
Type:     254

Name:     SurfaceArea
Table:    Country
max. Len: 10
```

```
Flags:    32769
Type:     4
```

Veja também

- mysqli_num_fields() - Obtém o número de campos no conjunto de resultados
- mysqli_fetch_field_direct() - Recebe metadados para um único campo
- mysqli_fetch_fields() - Retorna um array de objetos representando os campos em um conjunto de resultados
- mysqli_field_seek() - Move o ponteiro do resultado para um campo especificado

mysqli_result::fetch_field_direct

mysqli_result::fetch_field_direct

mysqli_fetch_field_direct

(PHP 5, PHP 7, PHP 8)

mysqli_result::fetch_field_direct -- mysqli_fetch_field_direct — Recebe metadados para um único campo

Descrição

Estilo orientado a objetos

public function **mysqli_result::fetch_field_direct**(int `$index`): object|false

Estilo procedural

function **mysqli_fetch_field_direct**(mysqli_result `$result`, int `$index`): object|false

Retorna um objeto que contém informação de definição de campo a partir do conjunto de resultados especificado.

Parâmetros

`result`

Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

`index`

O número do campo. Este valor deve estar na faixa de `0` a `número de campos - 1`.

Valor Retornado

Retorna um objeto que contém informação de definição de campo ou **`false`** se nenhuma informação de campo para o valor especificado em `index` estiver disponível.

Propriedades dos objetos

Propriedade	Descrição
name	O nome da coluna

Propriedade	Descrição
orgname	O nome original da coluna se foi especificado um apelido
table	O nome da tabela à qual o campo pertence (se não calculado)
orgtable	Nome da tabela original se foi especificado um apelido
def	Não utilizado. Sempre uma string vazia
db	O nome do banco de dados
catalog	Não utilizado. Sempre `"def"`
max_length	A largura máxima do campo para o conjunto de resultados. A partir do PHP 8.1, este valor é sempre `0`.
length	O comprimento do campo em bytes. Para colunas de strings, o valor do comprimento depende do conjunto de caracteres. Por exemplo, se o conjunto de caracteres for `latin1`, um conjunto de um byte, o valor de comprimento para uma consulta `SELECT 'abc'` será 3. Se o conjunto for `utf8mb4`, um conjunto multi-bytes onde os caracteres podem ter até 4 bytes, o valor de comprimento será 12.
charsetnr	O número do conjunto de caracteres do campo.
flags	Um inteiro representando bits de opções para o campo.
type	O tipo de dados usado para este campo
decimals	O número de decimais para campos numéricos, e a precisão em segundos fracionários para campos temporais.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Name LIMIT 5";

if ($result = $mysqli->query($query)) {

    /* Obtém informação de campo para a coluna 'SurfaceArea' */
    $finfo = $result->fetch_field_direct(1);

    printf("Name:     %s\n", $finfo->name);
    printf("Table:    %s\n", $finfo->table);
    printf("max. Len: %d\n", $finfo->max_length);
    printf("Flags:    %d\n", $finfo->flags);
    printf("Type:     %d\n", $finfo->type);

    $result->close();
}

/* fecha a conexão */
```

```php
$mysqli->close();
?>
```

Exemplo #2 Estilo procedural

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Name LIMIT 5";

if ($result = mysqli_query($link, $query)) {

    /* Obtém informação de campo para a coluna 'SurfaceArea' */
    $finfo = mysqli_fetch_field_direct($result, 1);

    printf("Name:     %s\n", $finfo->name);
    printf("Table:    %s\n", $finfo->table);
    printf("max. Len: %d\n", $finfo->max_length);
    printf("Flags:    %d\n", $finfo->flags);
    printf("Type:     %d\n", $finfo->type);

    mysqli_free_result($result);
}

/* fecha a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Name:     SurfaceArea
Table:    Country
max. Len: 10
Flags:    32769
Type:     4
```

Veja também

- mysqli_num_fields() - Obtém o número de campos no conjunto de resultados
- mysqli_fetch_field() - Retorna o próximo campo no conjunto de resultados
- mysqli_fetch_fields() - Retorna um array de objetos representando os campos em um conjunto de resultados

mysqli_result::fetch_fields

mysqli_result::fetch_fields

mysqli_fetch_fields

(PHP 5, PHP 7, PHP 8)

mysqli_result::fetch_fields -- mysqli_fetch_fields — Retorna um array de objetos representando os campos em um conjunto de resultados

Descrição

Estilo orientado a objetos

public function **mysqli_result::fetch_fields**(): array

Estilo procedural

function **mysqli_fetch_fields**(mysqli_result `$result`): array

Esta função serve para o mesmo propósito que a função mysqli_fetch_field() com uma simples diferença, ao invés de retornar um objeto de cada vez para cada campo, as colunas são retornadas como um array de objetos.

Parâmetros

`result`

> Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

Valor Retornado

Retorna um array de objetos contendo informações de definições de campo.

Propriedades dos objetos

Propriedade	Descrição
name	O nome da coluna
orgname	O nome original da coluna se foi especificado um apelido
table	O nome da tabela à qual o campo pertence (se não calculado)
orgtable	Nome da tabela original se foi especificado um apelido
def	Não utilizado. Sempre uma string vazia
db	O nome do banco de dados
catalog	Não utilizado. Sempre `"def"`
max_length	A largura máxima do campo para o conjunto de resultados. A partir do PHP 8.1, este valor é sempre `0`.
length	O comprimento do campo em bytes. Para colunas de strings, o valor do comprimento depende do conjunto de caracteres. Por exemplo, se o conjunto de caracteres for `latin1`, um conjunto de um byte, o valor de comprimento para uma consulta `SELECT 'abc'` será 3. Se o conjunto for `utf8mb4`, um conjunto multi-bytes onde os caracteres podem ter até 4 bytes, o valor de comprimento será 12.
charsetnr	O número do conjunto de caracteres do campo.
flags	Um inteiro representando bits de opções para o campo.

Propriedade	Descrição
type	O tipo de dados usado para este campo
decimals	O número de decimais para campos numéricos, e a precisão em segundos fracionários para campos temporais.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("127.0.0.1", "root", "foofoo", "sakila");

/*verifica a conexão */
if ($mysqli->connect_errno) {
    printf("Conexão falhou: %s\n", $mysqli->connect_error);
    exit();
}

foreach (array('latin1', 'utf8') as $charset) {

    // Define o conjunto de caracteres, para mostrar seu impacto em alguns valores (ex.: comprimento em bytes)
    $mysqli->set_charset($charset);

    $query = "SELECT actor_id, last_name from actor ORDER BY actor_id";

    echo "=====================\n";
    echo "Conjunto de Caracteres: $charset\n";
    echo "=====================\n";

    if ($result = $mysqli->query($query)) {

        /* Obtém informação de campo para todas as colunas */
        $finfo = $result->fetch_fields();

        foreach ($finfo as $val) {
            printf("Nome:        %s\n",   $val->name);
            printf("Tabela:      %s\n",   $val->table);
            printf("Comp. máx:   %d\n",   $val->max_length);
            printf("Comprimento: %d\n",   $val->length);
            printf("Id conj. carac: %d\n",   $val->charsetnr);
            printf("Opções:      %d\n",   $val->flags);
            printf("Tipo:        %d\n\n", $val->type);
        }
        $result->free();
    }
}
$mysqli->close();
?>
```

Exemplo #2 Estilo procedural

```php
<?php
$link = mysqli_connect("127.0.0.1", "my_user", "my_password", "sakila");

/*verifica a conexão */
```

```php
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

foreach (array('latin1', 'utf8') as $charset) {

    // Define o conjunto de caracteres, para mostrar seu impacto em alguns valores (ex.: comprimento em bytes)
    mysqli_set_charset($link, $charset);

    $query = "SELECT actor_id, last_name from actor ORDER BY actor_id";

    echo "=====================\n";
    echo "Conjunto de Caracteres: $charset\n";
    echo "=====================\n";

    if ($result = mysqli_query($link, $query)) {

        /* Obtém informação de campo para todas as colunas */
        $finfo = mysqli_fetch_fields($result);

        foreach ($finfo as $val) {
            printf("Nome:        %s\n",   $val->name);
            printf("Tabela:      %s\n",   $val->table);
            printf("Comp. máx:   %d\n",   $val->max_length);
            printf("Comprimento: %d\n",   $val->length);
            printf("Id conj. carac: %d\n",   $val->charsetnr);
            printf("Opções:      %d\n",   $val->flags);
            printf("Tipo:        %d\n\n", $val->type);
        }
        mysqli_free_result($result);
    }
}

mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
=====================
Conjunto de Caracteres: latin1
=====================
Nome:        actor_id
Tabela:      actor
Comp. máx:   3
Comprimento: 5
Id conj. carac: 63
Opções:      49699
Tipo:        2

Nome:        last_name
Tabela:      actor
Comp. máx:   12
Comprimento: 45
Id conj. carac: 8
```

```
Opções:      20489
Tipo:        253

======================
Conjunto de Caracteres: utf8
======================
Nome:        actor_id
Tabela:      actor
Comp. máx:   3
Comprimento: 5
Id conj. carac: 63
Opções:      49699
Tipo:        2

Nome:        last_name
Tabela:      actor
Comp. máx:   12
Comprimento: 135
Id conj. carac: 33
Opções:      20489
```

Veja também

- mysqli_num_fields() - Obtém o número de campos no conjunto de resultados
- mysqli_fetch_field_direct() - Recebe metadados para um único campo
- mysqli_fetch_field() - Retorna o próximo campo no conjunto de resultados

mysqli_result::fetch_object

mysqli_result::fetch_object

mysqli_fetch_object

(PHP 5, PHP 7, PHP 8)

mysqli_result::fetch_object -- mysqli_fetch_object — Recebe a próxima linha de um resultado como um objeto

Descrição

Estilo orientado a objetos

public function **mysqli_result::fetch_object**(string `$class` = "stdClass", array `$constructor_args` = []): object|null|false

Estilo procedural

function **mysqli_fetch_object**(mysqli_result `$result`, string `$class` = "stdClass", array `$constructor_args` = []): object|null|false

Recebe uma linha de dados do resultado e retorna-a como um objeto, onde cada propriedade representa o nome da coluna do resultado. Cada chamada subsequente a esta função retornará a linha seguinte dentro do conjunto de resultados, ou `null` se não houver mais linhas.

Se duas ou mais colunas do resultado tiverem o mesmo nome, a última coluna terá precedência e irá sobrescrever os dados anteriores. Para acessar múltiplas colunas com o mesmo nome, mysqli_fetch_row() pode ser usada para receber o array indexado numericamente, ou apelidos podem ser usados na lista de seleção da consulta SQL para dar nomes diferentes às colunas.

> **Nota** :
>
> Esta função define as propriedades do objeto antes de chamar o construtor do objeto.

> **Nota** :
>
> Os nomes de campos retornados por esta função *diferenciam maiúsculas e minúsculas*.

> **Nota** :
>
> Esta função define campos NULL como o valor `null` do PHP.

Parâmetros

`result`

Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

`class`

O nome da classe a instanciar, definir propriedades e retornar. Se não especificada, um objeto stdClass é retornado.

`constructor_args`

Um array opcional de parâmetros para passar ao construtor de objetos da classe definida em `class`.

Valor Retornado

Retorna um objeto que representa a linha recebida, onde cada propriedade representa o nome da coluna do resultado, `null` se não houver mais colunas no resultado, ou `false` em caso de falha.

Erros/Exceções

Um exceção ValueError é lançada quando o parâmetro `constructor_args` é não vazio com a classe não possuindo um construtor.

Registro de Alterações

Versão	Descrição
8.3.0	Agora lança uma exceção ValueError quando o parâmetro `constructor_args` é não vazio com a classe não possuindo um construtor; anteriormente uma Exception era lançada.
8.0.0	`constructor_args` agora aceita `[]` para construtores sem parâmetros; anteriormente, uma exceção era lançada.

Examples

Exemplo #1 Exemplo de **mysqli_result::fetch_object()**

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY ID DESC";

$result = $mysqli->query($query);

while ($obj = $result->fetch_object()) {
    printf("%s (%s)\n", $obj->Name, $obj->CountryCode);
}
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY ID DESC";

$result = mysqli_query($link, $query);

while ($obj = mysqli_fetch_object($result)) {
    printf("%s (%s)\n", $obj->Name, $obj->CountryCode);
}
```

Os exemplos acima produzirão algo semelhante a:

```
Pueblo (USA)
Arvada (USA)
Cape Coral (USA)
Green Bay (USA)
Santa Clara (USA)
```

Veja também

- mysqli_fetch_array() - Recebe a próxima linha de resultado como array associativo, numérico, ou ambos
- mysqli_fetch_assoc() - Obtém a próxima linha do conjunto de resultados como um array associativo
- mysqli_fetch_column() - Recebe uma única coluna da próxima linha de um resultado
- mysqli_fetch_row() - Obtém a próxima linha de um resultado como um array enumerado
- mysqli_query() - Executa uma consulta no banco de dados
- mysqli_data_seek() - Ajusta o ponteiro do resultado para uma linha arbitrária no resultado

mysqli_result::fetch_row

mysqli_result::fetch_row

mysqli_fetch_row

(PHP 5, PHP 7, PHP 8)

mysqli_result::fetch_row -- mysqli_fetch_row — Obtém a próxima linha de um resultado como um array enumerado

Descrição

Estilo orientado a objetos

public function **mysqli_result::fetch_row**(): array|null|false

Estilo procedural

function **mysqli_fetch_row**(mysqli_result `$result`): array|null|false

Obtém uma linha de dados do conjunto de resultado e retorna-o como um array enumerado, onde cada coluna é armazenada em um deslocamento do array iniciando de 0 (zero). Cada chamada subsequente a esta função retornará a próxima linha no conjunto de resultados, ou `null` se não houver mais linhas.

> **Nota** :
>
> Esta função define campos NULL como o valor `null` do PHP.

Parâmetros

`result`

> Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

Valor Retornado

Retorna um array representando a linha obtida, `null` se não houver mais linhas no conjunto de resultados, ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de **mysqli_result::fetch_row()**

Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY ID DESC";

$result = $mysqli->query($query);
```

```php
/* obtém array de objetos */
while ($row = $result->fetch_row()) {
    printf("%s (%s)\n", $row[0], $row[1]);
}
```

Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY ID DESC";

$result = mysqli_query($mysqli, $query);

/* obtém array de objetos */
while ($row = mysqli_fetch_row($result)) {
    printf("%s (%s)\n", $row[0], $row[1]);
}
```

Os exemplos acima produzirão algo semelhante a:

```
Pueblo (USA)
Arvada (USA)
Cape Coral (USA)
Green Bay (USA)
Santa Clara (USA)
```

Veja também

- mysqli_fetch_array() - Recebe a próxima linha de resultado como array associativo, numérico, ou ambos
- mysqli_fetch_assoc() - Obtém a próxima linha do conjunto de resultados como um array associativo
- mysqli_fetch_column() - Recebe uma única coluna da próxima linha de um resultado
- mysqli_fetch_object() - Recebe a próxima linha de um resultado como um objeto
- mysqli_query() - Executa uma consulta no banco de dados
- mysqli_data_seek() - Ajusta o ponteiro do resultado para uma linha arbitrária no resultado

mysqli_result::$field_count

mysqli_result::$field_count

mysqli_num_fields

(PHP 5, PHP 7, PHP 8)

mysqli_result::$field_count -- mysqli_num_fields — Obtém o número de campos no conjunto de resultados

Descrição

Estilo orientado a objetos

int *$mysqli_result->field_count*;

Estilo procedural

function **mysqli_num_fields**(mysqli_result `$result`): int

Obtém o número de campos no conjunto de resultados.

Parâmetros

`result`

Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

Valor Retornado

Um valor do tipo int que representa o número de campos.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$result = $mysqli->query("SELECT Name, CountryCode, District, Population FROM City ORDER BY ID LIMIT
1");

/* Obtém o número de campos do resultado */
$field_cnt = $result->field_count;

printf("O conjunto de resultados tem %d campos.\n", $field_cnt);
```

Exemplo #2 Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$result = mysqli_query($link, "SELECT Name, CountryCode, District, Population FROM City ORDER BY ID
LIMIT 1");

/* Obtém o número de campos do resultado */
$field_cnt = mysqli_num_fields($result);

printf("O conjunto de resultados tem %d campos.\n", $field_cnt);
```

Os exemplos acima produzirão:

```
O conjunto de resultados tem 4 campos.
```

Veja também

- mysqli_fetch_field() - Retorna o próximo campo no conjunto de resultados

mysqli_result::field_seek

mysqli_result::field_seek

mysqli_field_seek

(PHP 5, PHP 7, PHP 8)

mysqli_result::field_seek -- mysqli_field_seek — Move o ponteiro do resultado para um campo especificado

Descrição

Estilo orientado a objetos

public function **mysqli_result::field_seek**(int `$index`): true

Estilo procedural

function **mysqli_field_seek**(mysqli_result `$result`, int `$index`): true

Define o cursor de campo para o índice especificado. A próxima chamada a mysqli_fetch_field() retornará a definição de campo da coluna associada com este índice.

> **Nota** :
>
> Para mover o ponteiro para o início de uma linha, passe um índice de valor zero.

Parâmetros

`result`

Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

`index`

O número do campo. Este valor precisa estar num intervalo de `0` a `número de campos - 1`.

Valor Retornado

Sempre retorna **`true`**.

Registro de Alterações

Versão	Descrição
8.0.0	Esta função agora sempre retorna **`true`**. Anteriormente retornava **`false`** em caso de falha.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Code LIMIT 5";

if ($result = $mysqli->query($query)) {

    /* Obtém informação de campo para a 2ª coluna */
    $result->field_seek(1);
    $finfo = $result->fetch_field();

    printf("Name:     %s\n", $finfo->name);
    printf("Table:    %s\n", $finfo->table);
    printf("max. Len: %d\n", $finfo->max_length);
    printf("Flags:    %d\n", $finfo->flags);
    printf("Type:     %d\n\n", $finfo->type);

    $result->close();
}

/* fecha a conexão */
$mysqli->close();
?>
```

Exemplo #2 Estilo procedural

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Code LIMIT 5";

if ($result = mysqli_query($link, $query)) {

    /* Obtém informação de campo para a 2ª coluna */
    mysqli_field_seek($result, 1);
    $finfo = mysqli_fetch_field($result);

    printf("Name:     %s\n", $finfo->name);
    printf("Table:    %s\n", $finfo->table);
    printf("max. Len: %d\n", $finfo->max_length);
    printf("Flags:    %d\n", $finfo->flags);
    printf("Type:     %d\n\n", $finfo->type);

    mysqli_free_result($result);
```

```
}

/* fecha a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Name:     SurfaceArea
Table:    Country
max. Len: 10
Flags:    32769
Type:     4
```

Veja também

- mysqli_fetch_field() - Retorna o próximo campo no conjunto de resultados

mysqli_result::free

mysqli_result::free

mysqli_result::close

mysqli_result::free_result

mysqli_free_result

(PHP 5, PHP 7, PHP 8)

mysqli_result::free -- mysqli_result::close -- mysqli_result::free_result -- mysqli_free_result — Libera memória associada a um resultado

Descrição

Estilo orientado a objetos

public function **mysqli_result::free**(): void
public function **mysqli_result::close**(): void
public function **mysqli_result::free_result**(): void

Estilo procedural

function **mysqli_free_result**(mysqli_result `$result`): void

Libera a memória associada ao resultado.

Parâmetros

`result`

Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

Valor Retornado

Nenhum valor é retornado.

Veja também

- mysqli_query() - Executa uma consulta no banco de dados
- mysqli_stmt_get_result() - Obtém um resultado de uma instrução preparada como um objeto mysqli_result
- mysqli_store_result() - Transfere um conjunto de resultados da última consulta
- mysqli_use_result() - Inicia uma recuperação de conjunto de resultados

mysqli_result::getIterator

mysqli_result::getIterator

(PHP 8)

mysqli_result::getIterator — Obtém um iterador externo

Descrição

public function **mysqli_result::getIterator**(): Iterator

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

mysqli_result::$lengths

mysqli_result::$lengths

mysqli_fetch_lengths

(PHP 5, PHP 7, PHP 8)

mysqli_result::$lengths -- mysqli_fetch_lengths — Retorna os comprimentos das colunas da linha atual do resultado

Descrição

Estilo orientado a objetos

?array *$mysqli_result->lengths*;

Estilo procedural

function **mysqli_fetch_lengths**(mysqli_result `$result`): array|false

A função **mysqli_fetch_lengths()** retorna um array que contém os comprimentos de cada coluna da linha atual dentro do conjunto de resultados.

Parâmetros

`result`

Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

Valor Retornado

Um array de números inteiros que representam o tamanho de cada coluna (não incluindo caracteres nulos de terminação). `false` se ocorrer um erro.

mysqli_fetch_lengths() é válida apenas para a linha atual do resultado. Retorna `false` se for chamada antes de se chamar mysqli_fetch_row, mysqli_fetch_array, mysqli_fetch_object ou após ter recebido todas as linhas do resultado.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT * from Country ORDER BY Code LIMIT 1";

if ($result = $mysqli->query($query)) {

    $row = $result->fetch_row();

    /* mostra os comprimentos das colunas */
    foreach ($result->lengths as $i => $val) {
        printf("Campo %2d tem comprimento %2d\n", $i+1, $val);
    }
    $result->close();
}

/* fecha a conexão */
$mysqli->close();
?>
```

Exemplo #2 Estilo procedural

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* verifica a conexão */
if (mysqli_connect_errno()) {
    printf("Conexão falhou: %s\n", mysqli_connect_error());
```

```
    exit();
}

$query = "SELECT * from Country ORDER BY Code LIMIT 1";

if ($result = mysqli_query($link, $query)) {

    $row = mysqli_fetch_row($result);

    /* mostra os comprimentos das colunas */
    foreach (mysqli_fetch_lengths($result) as $i => $val) {
        printf("Campo %2d tem comprimento %2d\n", $i+1, $val);
    }
    mysqli_free_result($result);
}

/* fecha a conexão */
mysqli_close($link);
?>
```

Os exemplos acima produzirão:

```
Campo  1 tem comprimento  3
Campo  2 tem comprimento  5
Campo  3 tem comprimento 13
Campo  4 tem comprimento  9
Campo  5 tem comprimento  6
Campo  6 tem comprimento  1
Campo  7 tem comprimento  6
Campo  8 tem comprimento  4
Campo  9 tem comprimento  6
Campo 10 tem comprimento  6
Campo 11 tem comprimento  5
Campo 12 tem comprimento 44
Campo 13 tem comprimento  7
Campo 14 tem comprimento  3
Campo 15 tem comprimento  2
```

mysqli_result::$num_rows

mysqli_result::$num_rows

mysqli_num_rows

(PHP 5, PHP 7, PHP 8)

mysqli_result::$num_rows -- mysqli_num_rows — Obtém o número de linhas no resultado

Descrição

Estilo orientado a objetos

int|string *$mysqli_result->num_rows*;

Estilo procedural

function **mysqli_num_rows**(mysqli_result `$result`): int|string

Retorna o número de linhas no resultado.

O comportamento de **mysqli_num_rows()** depende se estão sendo usados resultados com buffer ou sem buffer. A função retorna `0` para resultado sem buffer a menos que todas as linhas já tiverem sido recebidas do servidor.

Parâmetros

`result`

Somente no estilo procedural: Um objeto mysqli_result retornado por mysqli_query(), mysqli_store_result(), mysqli_use_result() ou mysqli_stmt_get_result().

Valor Retornado

Um int que representa o número de linhas recebidas. Retorna `0` em modo sem buffer a menos que todas as linhas tenham sido recebidas do servidor.

> **Nota** :
>
> Se o número de linhas for maior que **PHP_INT_MAX**, o número será retornado como uma string.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$result = $mysqli->query("SELECT Code, Name FROM Country ORDER BY Name");

/* Obtém o número de linhas no resultado */
$row_cnt = $result->num_rows;

printf("Resultado contém %d linhas.\n", $row_cnt);
```

Exemplo #2 Estilo procedural

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$result = mysqli_query($link, "SELECT Code, Name FROM Country ORDER BY Name");

/* Obtém o número de linhas no resultado */
$row_cnt = mysqli_num_rows($result);
```

```
printf("Resultado contém %d linhas.\n", $row_cnt);
```

Os exemplos acima produzirão:

```
Resultado contém 239 linhas.
```

Notas

> **Nota** :
>
> Em contraste à função mysqli_stmt_num_rows(), esta função não tem uma variante de método orientado a objetos. No estilo de linguagem orientada a objetos, use __get().

Veja também

- mysqli_affected_rows() - Retorna o número de linhas afetadas pela operação MySQL anterior
- mysqli_store_result() - Transfere um conjunto de resultados da última consulta
- mysqli_use_result() - Inicia uma recuperação de conjunto de resultados
- mysqli_query() - Executa uma consulta no banco de dados
- mysqli_stmt_num_rows() - Retorna o número de linhas recebidas pelo servidor

mysqli_driver

(PHP 5, PHP 7, PHP 8)

Introdução

A classe **mysqli_driver** é uma instância do padrão monostate, ou seja, existe apenas um driver que pode ser acessado por meio de uma quantidade arbitrária de instâncias **mysqli_driver**.

Resumo da classe

final class **mysqli_driver** {
/* Propriedades */
public readonly string *$client_info*;
public readonly int *$client_version*;
public readonly int *$driver_version*;
public int *$report_mode*;
}

Propriedades

client_info

A versão do cabeçalho da API do cliente

client_version

A versão do cliente

driver_version

A versão do driver MySQLi

> **Aviso**
>
> Esta propriedade foi *descontinuada* a partir do PHP 8.1.0. Confiar nesta propriedade é altamente desencorajado.

embedded

Se o suporte MySQLi Embedded está ativado

> **Aviso**
>
> Esta propriedade foi *removida* a partir do PHP 8.0.0.

reconnect

Permitir ou impedir a reconexão (consulte a diretiva mysqli.reconnect INI)

> **Aviso**
>
> Esta propriedade foi *removida* juntamente com a diretiva INI mysqli.reconnect a partir do PHP 8.2.0.

report_mode

Defina como `MYSQLI_REPORT_OFF`, `MYSQLI_REPORT_ALL` ou qualquer combinação de `MYSQLI_REPORT_STRICT` (lança exceções para erros), `MYSQLI_REPORT_ERROR` (reporta erros) e `MYSQLI_REPORT_INDEX` (erros relacionados a índices). Veja também mysqli_report().

Registro de Alterações

Versão	Descrição
8.2.0	mysqli_driver::$reconnect foi removido.
8.1.0	mysqli_driver::$driver_version foi descontinuada.
8.0.0	mysqli_driver::$embedded foi removido.
7.4.0	mysqli_driver::embedded_server_start() e **mysqli_driver:embedded_server_end()** foram removidos.

Índice

mysqli_driver::embedded_server_end

mysqli_driver::embedded_server_end

mysqli_embedded_server_end

(PHP 5 >= 5.1.0, PHP 7 < 7.4.0)

mysqli_driver::embedded_server_end -- mysqli_embedded_server_end — Para o servidor incorporado

> **Aviso**
>
> Esta função foi *REMOVIDA* a partir do PHP 7.4.0.

Descrição

Estilo orientado a objetos

public function **mysqli_driver::embedded_server_end**(): void

Estilo procedural

function **mysqli_embedded_server_end**(): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

mysqli_driver::embedded_server_start

mysqli_driver::embedded_server_start

mysqli_embedded_server_start

(PHP 5 >= 5.1.0, PHP 7 < 7.4.0)

mysqli_driver::embedded_server_start -- mysqli_embedded_server_start — Inicializa e dispara o servidor incorporado

> **Aviso**
>
> Esta função foi *REMOVIDA* a partir do PHP 7.4.0.

Descrição

Estilo orientado a objetos

public function **mysqli_driver::embedded_server_start**(int `$start`, array `$arguments`, array `$groups`): bool

Estilo procedural

function **mysqli_embedded_server_start**(int `$start`, array `$arguments`, array `$groups`): bool

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

mysqli_driver::$report_mode

mysqli_driver::$report_mode

mysqli_report

(PHP 5, PHP 7, PHP 8)

mysqli_driver::$report_mode -- mysqli_report — Define o modo de relatório de erros do mysqli

Descrição

Estilo orientado a objetos

int *$mysqli_driver->report_mode*;

Estilo procedural

function mysqli_report(int `$flags`): true

Dependendo das flags, ele define o modo de relatório de erros do mysqli como exceção, aviso ou nenhum. Quando definido como **`MYSQLI_REPORT_ALL`** ou **`MYSQLI_REPORT_INDEX`**, ele também informará sobre as consultas que não usam um índice (ou usam um índice incorreto).

A partir do PHP 8.1.0, a configuração padrão é `MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT`. Anteriormente, era **`MYSQLI_REPORT_OFF`**.

Parâmetros

`flags`

Flags suportadas

Nome	Descrição
`MYSQLI_REPORT_OFF`	Desativa os relatórios
`MYSQLI_REPORT_ERROR`	Relatar erros de chamadas de função mysqli
`MYSQLI_REPORT_STRICT`	Lance mysqli_sql_exception para erros em vez de avisos
`MYSQLI_REPORT_INDEX`	Informa se nenhum índice ou índice inválido foi usado em uma consulta
`MYSQLI_REPORT_ALL`	Define todas as opções (relatar tudo)

Valor Retornado

Sempre retorna **`true`**.

Registro de Alterações

Versão	Descrição
8.1.0	O valor padrão agora é `MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT`. Anteriormente, era **`MYSQLI_REPORT_OFF`**.

Examples

Exemplo #1 Estilo orientado a objetos

```php
<?php

/* ativar relatórios */
$driver = new mysqli_driver();
$driver->report_mode = MYSQLI_REPORT_ALL;

try {
    /* se a conexão falhar, um mysqli_sql_exception será lançado */
    $mysqli = new mysqli("localhost", "my_user", "my_password", "my_db");
```

```
    /* esta consulta deve relatar um erro */
    $result = $mysqli->query("SELECT Name FROM Nonexistingtable WHERE population > 50000");

    /* esta consulta deve relatar um índice ruim se a população da coluna não tiver um índice */
    $result = $mysqli->query("SELECT Name FROM City WHERE population > 50000");
} catch (mysqli_sql_exception $e) {
    error_log($e->__toString());
}
```

Exemplo #2 Estilo procedural

```
<?php

/* ativar relatórios */
mysqli_report(MYSQLI_REPORT_ALL);

try {
    /* se a conexão falhar, um mysqli_sql_exception será lançado */
    $link = mysqli_connect("localhost", "my_user", "my_password", "my_db");

    /* esta consulta deve relatar um erro */
    $result = mysqli_query($link, "SELECT Name FROM Nonexistingtable WHERE population > 50000");

    /* esta consulta deve relatar um índice ruim se a população da coluna não tiver um índice */
    $result = mysqli_query($link, "SELECT Name FROM City WHERE population > 50000");
} catch (mysqli_sql_exception $e) {
    error_log($e->__toString());
}
```

Exemplo #3 Relatório de erros, exceto erros de índice ruim

```
<?php

/* ativar relatórios */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

try {
    /* se a conexão falhar, um mysqli_sql_exception será lançado */
    $mysqli = new mysqli("localhost", "my_user", "my_password", "my_db");

    /* esta consulta deve relatar um erro */
    $result = $mysqli->query("SELECT Name FROM Nonexistingtable WHERE population > 50000");

    /* isso NÃO relatará nenhum erro, mesmo que o índice não esteja disponível */
    $result = $mysqli->query("SELECT Name FROM City WHERE population > 50000");
} catch (mysqli_sql_exception $e) {
    error_log($e->__toString());
}
```

Veja também

- mysqli_sql_exception
- set_exception_handler() - Define uma função de manipulação de exceção definida pelo usuário
- error_reporting() - Define quais erros do PHP são relatados

mysqli_warning

(PHP 5, PHP 7, PHP 8)

Introdução

Representa um aviso do MySQL.

Resumo da classe

```
final class mysqli_warning {
/* Propriedades */
public string $message;
public string $sqlstate;
public int $errno;
/* Métodos */
private function __construct()
public function next(): bool
}
```

Propriedades

message

string da Mensagem

sqlstate

Estado SQL

errno

Número do erro

Índice

mysqli_warning::__construct

mysqli_warning::__construct

(PHP 5, PHP 7, PHP 8)

mysqli_warning::__construct — Construtor privado para proibir instanciação direta

Descrição

private function **mysqli_warning::__construct**()

Parâmetros

Esta função não possui parâmetros.

mysqli_warning::next

mysqli_warning::next

(PHP 5, PHP 7, PHP 8)

mysqli_warning::next — Busca o próximo aviso

Descrição

public function **mysqli_warning::next**(): bool

Altere as informações de aviso para o próximo aviso, se possível.

Depois que o aviso for definido para o próximo aviso, novos valores de propriedades `message`, `sqlstate` e `errno` de mysqli_warning estarão disponíveis.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna **`true`** se o próximo aviso foi obtido com sucesso. Se não houver mais avisos, retornará **`false`**

mysqli_sql_exception

(PHP 5, PHP 7, PHP 8)

Introdução

A classe de manipulação de exceção mysqli.

Resumo da classe

final class **mysqli_sql_exception** extends RuntimeException {
/* Propriedades */
protected string *$sqlstate* = "00000";
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos */
public function getSqlState(): string
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

Propriedades

sqlstate

O estado sql com o erro.

Índice

mysqli_sql_exception::getSqlState

mysqli_sql_exception::getSqlState

(PHP 8 >= 8.1.2)

mysqli_sql_exception::getSqlState — Retorna o código de erro SQLSTATE

Descrição

public function **mysqli_sql_exception::getSqlState**(): string

Retorna uma string contendo o código de erro SQLSTATE para o último erro. O código de erro consiste em cinco caracteres. Os valores são especificados por ANSI SQL e ODBC. Para obter uma lista de valores possíveis, consulte » http://dev.mysql.com/doc/mysql/en/error-handling.html.

> **Nota** :
>
> Observe que nem todos os erros do MySQL ainda estão mapeados para SQLSTATEs. O valor `HY000` (erro geral) é usado para erros não mapeados.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna uma string contendo o código de erro SQLSTATE para o último erro. O código de erro consiste em cinco caracteres.

Apelidos e Funções descontinuadas da Mysqli

Índice

mysqli_connect

mysqli_connect

(PHP 5, PHP 7, PHP 8)

mysqli_connect — Sinônimo de mysqli::__construct()

Descrição

Esta função é um apelido para: mysqli::__construct()

> **Nota** :

> Se o modo de exceção mysqli não estiver habilitado e uma conexão falhar, então **mysqli_connect()** retornará `false` ao invés de um objeto. A função mysqli_connect_error() pode ser usada para buscar o erro de conexão.

mysqli::escape_string

mysqli::escape_string

mysqli_escape_string

(PHP 5, PHP 7, PHP 8)

mysqli::escape_string -- mysqli_escape_string — Sinônimo de mysqli_real_escape_string()

Descrição

Esta função é um apelido para: mysqli_real_escape_string().

mysqli_execute

mysqli_execute

(PHP 5, PHP 7, PHP 8)

mysqli_execute — Sinônimo de mysqli_stmt_execute()

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

Esta função é um apelido para: mysqli_stmt_execute().

Veja também

- mysqli_stmt_execute() - Executa uma instrução preparada

mysqli_get_client_stats

mysqli_get_client_stats

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli_get_client_stats — Retorna as estatísticas do cliente por processo

Descrição

function **mysqli_get_client_stats**(): array

Retorna as estatísticas do cliente por processo.

> **Nota** :
>
> Disponível apenas com o mysqlnd.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um array com estatísticas do cliente.

Examples

Exemplo #1 Exemplo de **mysqli_get_client_stats()**

```php
<?php
$link = mysqli_connect();
print_r(mysqli_get_client_stats());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [bytes_sent] => 43
    [bytes_received] => 80
    [packets_sent] => 1
    [packets_received] => 2
    [protocol_overhead_in] => 8
    [protocol_overhead_out] => 4
    [bytes_received_ok_packet] => 11
    [bytes_received_eof_packet] => 0
    [bytes_received_rset_header_packet] => 0
    [bytes_received_rset_field_meta_packet] => 0
    [bytes_received_rset_row_packet] => 0
    [bytes_received_prepare_response_packet] => 0
    [bytes_received_change_user_packet] => 0
    [packets_sent_command] => 0
    [packets_received_ok] => 1
    [packets_received_eof] => 0
    [packets_received_rset_header] => 0
    [packets_received_rset_field_meta] => 0
    [packets_received_rset_row] => 0
    [packets_received_prepare_response] => 0
    [packets_received_change_user] => 0
    [result_set_queries] => 0
    [non_result_set_queries] => 0
    [no_index_used] => 0
    [bad_index_used] => 0
    [slow_queries] => 0
    [buffered_sets] => 0
    [unbuffered_sets] => 0
    [ps_buffered_sets] => 0
    [ps_unbuffered_sets] => 0
    [flushed_normal_sets] => 0
    [flushed_ps_sets] => 0
    [ps_prepared_never_executed] => 0
    [ps_prepared_once_executed] => 0
    [rows_fetched_from_server_normal] => 0
```

```
    [rows_fetched_from_server_ps] => 0
    [rows_buffered_from_client_normal] => 0
    [rows_buffered_from_client_ps] => 0
    [rows_fetched_from_client_normal_buffered] => 0
    [rows_fetched_from_client_normal_unbuffered] => 0
    [rows_fetched_from_client_ps_buffered] => 0
    [rows_fetched_from_client_ps_unbuffered] => 0
    [rows_fetched_from_client_ps_cursor] => 0
    [rows_skipped_normal] => 0
    [rows_skipped_ps] => 0
    [copy_on_write_saved] => 0
    [copy_on_write_performed] => 0
    [command_buffer_too_small] => 0
    [connect_success] => 1
    [connect_failure] => 0
    [connection_reused] => 0
    [reconnect] => 0
    [pconnect_success] => 0
    [active_connections] => 1
    [active_persistent_connections] => 0
    [explicit_close] => 0
    [implicit_close] => 0
    [disconnect_close] => 0
    [in_middle_of_command_close] => 0
    [explicit_free_result] => 0
    [implicit_free_result] => 0
    [explicit_stmt_close] => 0
    [implicit_stmt_close] => 0
    [mem_emalloc_count] => 0
    [mem_emalloc_ammount] => 0
    [mem_ecalloc_count] => 0
    [mem_ecalloc_ammount] => 0
    [mem_erealloc_count] => 0
    [mem_erealloc_ammount] => 0
    [mem_efree_count] => 0
    [mem_malloc_count] => 0
    [mem_malloc_ammount] => 0
    [mem_calloc_count] => 0
    [mem_calloc_ammount] => 0
    [mem_realloc_count] => 0
    [mem_realloc_ammount] => 0
    [mem_free_count] => 0
    [proto_text_fetched_null] => 0
    [proto_text_fetched_bit] => 0
    [proto_text_fetched_tinyint] => 0
    [proto_text_fetched_short] => 0
    [proto_text_fetched_int24] => 0
    [proto_text_fetched_int] => 0
    [proto_text_fetched_bigint] => 0
    [proto_text_fetched_decimal] => 0
    [proto_text_fetched_float] => 0
    [proto_text_fetched_double] => 0
    [proto_text_fetched_date] => 0
    [proto_text_fetched_year] => 0
    [proto_text_fetched_time] => 0
    [proto_text_fetched_datetime] => 0
    [proto_text_fetched_timestamp] => 0
    [proto_text_fetched_string] => 0
    [proto_text_fetched_blob] => 0
```

```
    [proto_text_fetched_enum] => 0
    [proto_text_fetched_set] => 0
    [proto_text_fetched_geometry] => 0
    [proto_text_fetched_other] => 0
    [proto_binary_fetched_null] => 0
    [proto_binary_fetched_bit] => 0
    [proto_binary_fetched_tinyint] => 0
    [proto_binary_fetched_short] => 0
    [proto_binary_fetched_int24] => 0
    [proto_binary_fetched_int] => 0
    [proto_binary_fetched_bigint] => 0
    [proto_binary_fetched_decimal] => 0
    [proto_binary_fetched_float] => 0
    [proto_binary_fetched_double] => 0
    [proto_binary_fetched_date] => 0
    [proto_binary_fetched_year] => 0
    [proto_binary_fetched_time] => 0
    [proto_binary_fetched_datetime] => 0
    [proto_binary_fetched_timestamp] => 0
    [proto_binary_fetched_string] => 0
    [proto_binary_fetched_blob] => 0
    [proto_binary_fetched_enum] => 0
    [proto_binary_fetched_set] => 0
    [proto_binary_fetched_geometry] => 0
    [proto_binary_fetched_other] => 0
)
```

Veja também

- Descrição das estatísticas

mysqli_get_links_stats

mysqli_get_links_stats

(PHP 5 >= 5.6.0, PHP 7, PHP 8)

mysqli_get_links_stats — Retorna informações sobre links abertos e em cache

Descrição

function **mysqli_get_links_stats**(): array

mysqli_get_links_stats() retorna informações sobre links MySQL abertos e em cache.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

mysqli_get_links_stats() retorna um array associativo com três elementos, digitados da seguinte forma:

`total`

Um int que indica o número total de links abertos em qualquer estado.

`active_plinks`

Um int representando o número de conexões persistentes ativas.

`cached_plinks`

Um int representando o número de conexões persistentes inativas.

mysqli_report

mysqli_report

(PHP 5, PHP 7, PHP 8)

mysqli_report — Sinônimo de mysqli_driver->report_mode

Descrição

Esta função é um apelido para: mysqli_driver->report_mode

mysqli::set_opt

mysqli::set_opt

mysqli_set_opt

(PHP 5, PHP 7, PHP 8)

mysqli::set_opt -- mysqli_set_opt — Sinônimo de mysqli_options()

Descrição

Esta função é um apelido para: mysqli_options().

Registro de Alterações

As alterações a seguir foram feitas em classes/funções/métodos desta extensão.

Version	Function	Description
8.4.0	mysqli::kill	O método mysqli::kill e a função mysqli_kill foram descontinuados. No lugar deles, deve ser usado o comando SQL KILL.
	mysqli::ping	O método mysqli::ping e a função mysqli_ping foram descontinuados. A partir do PHP 8.2.0, o recurso de reconnect (reconexão) não está mais disponível, tornando esta função obsoleta.
	mysqli::refresh	O método mysqli::refresh e a função mysqli_refresh foram descontinuados. No lugar deles, use o comando SQL FLUSH.
	mysqli::store_result	Passar o parâmetro mode foi descontinuado. O parâmetro já não tem efeito a partir do PHP 8.1.0.
8.3.0	mysqli_result::fetch_object	Agora lança uma exceção ValueError quando o parâmetro constructor_args é não vazio com a classe não possuindo um construtor; anteriormente uma Exception era lançada.

Version	Function	Description
	mysqli::poll	Agora lança uma exceção ValueError quando nem o parâmetro read nem o parâmetro error são passados.
8.1.0	mysqli_driver::$report_mode	O valor padrão agora é MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT. Anteriormente, era MYSQLI_REPORT_OFF.
	mysqli_result::fetch_all	Agora também disponível com a biblioteca libmysqlclient.
	mysqli_stmt::execute	O parâmetro optional params foi adicionado.
	mysqli_stmt::next_result	Agora também disponível com a biblioteca libmysqlclient.
	mysqli::__construct	Agora, o método mysqli::connect retorna true em vez de null em caso de sucesso.
	mysqli::$client_info	Chamar mysqli_get_client_info com o argumento mysql foi descontinuado. Esta função nunca requereu um parâmetro, mas incorretamente permitia este parâmetro opcional.
	mysqli::$client_info	O método orientado a objeto mysqli::get_client_info foi descontinuado.
	mysqli::init	O método mysqli::init com estilo orientado a objeto foi descontinuado. Substitua as chamadas a parent::init por parent::__construct.
8.0.0	mysqli_result::fetch_object	constructor_args agora aceita [] para construtores sem parâmetros; anteriormente, uma exceção era lançada.
	mysqli_result::field_seek	Esta função agora sempre retorna true. Anteriormente retornava false em caso de falha.
	mysqli_stmt::close	Esta função agora sempre retorna true. Anteriormente retornava false em caso de falha.
	mysqli_stmt::__construct	query agora pode ser nulo.
	mysqli::begin_transaction	name agora é anulável.
	mysqli::close	Esta função sempre retorna true. Anteriormente retornava false em caso de falha.
	mysqli::commit	name agora pode ser nulo.
	mysqli::debug	Esta função agora sempre retorna true. Anteriormente retornava false em caso de falha.
	mysqli::rollback	name agora é anulável.
7.4.0	mysqli::__construct	Todos os parâmetros podem ser null.
	mysqli::real_connect	Todos os parâmetros agora são anuláveis.

Mysql_xdevapi

Introdução

Esta extensão fornece acesso ao MySQL Document Store através da X DevAPI. A X DevAPI é uma API comum fornecida por múltiplos conectores MySQL que permitem acesso fácil a tabelas relacionais assim como a coleções de documentos, que são representadas em JSON, a partir de uma API com operações em estilo CRUD.

A X DevAPI usa do protocolo X, o protocolo cliente-servidor de nova geração do servidor MySQL 8.0.

Para informação geral sobre o MySQL Document Store, favor se referir ao capítulo » MySQL Document Store no manual do MySQL.

Instalação/Configuração

Índice

Dependências

Esta extensão requer um Servidor MySQL 8+ com o plugin X habilitado (padrão).

Bibliotecas que são pré-requisitos para esta extensão: Boost (1.53.0 ou superior), OpenSSL e Protobuf.

Instalação

Esta extensão » PECL não é distribuída com o PHP.

Um procedimento de exemplo de instalação no Ubuntu 18.04 com PHP 7.2:

```
// Dependências
$ apt install build-essential libprotobuf-dev libboost-dev openssl protobuf-compiler liblz4-tool zstd

// PHP com as extensões desejadas; php7.2-dev é requerido para compilação
$ apt install php7.2-cli php7.2-dev php7.2-mysql php7.2-pdo php7.2-xml

// Compila a extensão
$ pecl install mysql_xdevapi
```

O comando `pecl install` não habilita as extensões PHP (por padrão) e as habilitações de extensões podem ser feitas de várias formas. Um outro exemplo do PHP 7.2 no Ubuntu 18.04:

```
// Cria o seu próprio arquivo ini
$ echo "extension=mysql_xdevapi.so" > /etc/php/7.2/mods-available/mysql_xdevapi.ini

// Usa o comando 'phpenmod' (nota: específico para Debian/Ubuntu)
$ phpenmod -v 7.2 -s ALL mysql_xdevapi

// Uma alternativa ao 'phpenmod' é criar uma ligação simbólica manualmente
// $ ln -s /etc/php/7.2/mods-available/mysql_xdevapi.ini /etc/php/7.2/cli/conf.d/20-mysql_xdevapi.ini

// Vejamos agora que extensões do MySQL estão habilitadas
$ php -m |grep mysql

mysql_xdevapi
mysqli
mysqlnd
pdo_mysql
```

Informações para instalar esta extensão PECL podem ser encontradas no capítulo Instalação de Extensões PECL do manual. Informações adicionais, como novas versões, downloads, arquivos fontes, informações do mantenedor e um CHANGELOG, podem ser encontradas aqui: » https://pecl.php.net/package/mysql_xdevapi.

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de Configuração de mysql_xdevapi

Nome	Padrão	Modificável	Registro de Alterações
xmysqlnd.collect_memory_statistics	0	INI_SYSTEM	
xmysqlnd.collect_statistics	1	INI_ALL	
xmysqlnd.debug		INI_SYSTEM	
xmysqlnd.mempool_default_size	16000	INI_ALL	
xmysqlnd.net_read_timeout	31536000	INI_SYSTEM	
xmysqlnd.trace_alloc		INI_SYSTEM	

Aqui está uma breve explicação das diretivas de configuração.

`xmysqlnd.collect_memory_statistics` int

`xmysqlnd.collect_statistics` int

`xmysqlnd.debug` string

`xmysqlnd.mempool_default_size` int

`xmysqlnd.net_read_timeout` int

`xmysqlnd.trace_alloc` string

Construindo / Compilando a Partir da Fonte

Considerations for compiling this extension from source.

- O nome da extensão é 'mysql_xdevapi', portanto use `--enable-mysql-xdevapi`.
- Boost: requerido, opcionalmente use a opção de configuração --with-boost=DIR ou defina a variável de ambiente MYSQL_XDEVAPI_BOOST_ROOT. Somente os arquivos de cabeçalho Boost são necessários, não os binários.
- Google Protocol Buffers (protobuf): requerido, opcionalmente use a opção de configuração --with-protobuf=DIR ou defina a variável de ambiente MYSQL_XDEVAPI_PROTOBUF_ROOT.

  Opcionamente, use `make protobufs` para gerar arquivos protobuf (*.pb.cc/.h), e `make clean-protobufs` para remover os arquivos protobuf gerados.

  Nota do protobuf específica para Windows: dependendo do ambiente, a biblioteca estática com um DLL multi-thread pode ser necessária. Para preparar, use as seguintes opções: *-Dprotobuf_MSVC_STATIC_RUNTIME=OFF -Dprotobuf_BUILD_SHARED_LIBS=OFF*
- Google Protocol Buffers / compilador de protocolo (protoc): requerido, deve-se garantir que o 'protoc' apropriado esteja disponível na variável de ambiente PATH durante a construção. É especialmente importante porque os scripts em lote do Windows PHP SDK podem sobrescrever o ambiente.
- Bison: requerido, e disponível a partir do PATH.

  Nota do bison específica para o Windows: recomenda-se fortemente o uso do bison fornecido com o SDK do PHP escolhido, ou pode resultar em um erro similar a "zend_globals_macros.h(39): error C2375: 'zendparse': redefinition; different linkage Zend/zend_language_parser.h(214): note: see declaration of 'zendparse'". Além disso, os scripts em lote do Windows PHP SDK podem sobrescrever o ambiente.

- Notas específicas para Windows: Para preparar o ambiente, consulte a documentação de compilação oficial do Windows para » o SDK atual.

  Recomenda-se usar a barra invertida '\\' ao invés da barra normal '/' para todos os caminhos.

Constantes predefinidas

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

MYSQLX_CLIENT_SSL (int)

MYSQLX_TYPE_DECIMAL (int)

MYSQLX_TYPE_TINY (int)

MYSQLX_TYPE_SHORT (int)

MYSQLX_TYPE_SMALLINT (int)

MYSQLX_TYPE_MEDIUMINT (int)

MYSQLX_TYPE_INT (int)

MYSQLX_TYPE_BIGINT (int)

MYSQLX_TYPE_LONG (int)

MYSQLX_TYPE_FLOAT (int)

MYSQLX_TYPE_DOUBLE (int)

MYSQLX_TYPE_NULL (int)

MYSQLX_TYPE_TIMESTAMP (int)

MYSQLX_TYPE_LONGLONG (int)

MYSQLX_TYPE_INT24 (int)

MYSQLX_TYPE_DATE (int)

MYSQLX_TYPE_TIME (int)

MYSQLX_TYPE_DATETIME (int)

MYSQLX_TYPE_YEAR (int)

MYSQLX_TYPE_NEWDATE (int)

MYSQLX_TYPE_ENUM (int)

MYSQLX_TYPE_SET (int)

MYSQLX_TYPE_TINY_BLOB (int)

MYSQLX_TYPE_MEDIUM_BLOB (int)

MYSQLX_TYPE_LONG_BLOB (int)

`MYSQLX_TYPE_BLOB` (int)

`MYSQLX_TYPE_VAR_STRING` (int)

`MYSQLX_TYPE_STRING` (int)

`MYSQLX_TYPE_CHAR` (int)

`MYSQLX_TYPE_BYTES` (int)

`MYSQLX_TYPE_INTERVAL` (int)

`MYSQLX_TYPE_GEOMETRY` (int)

`MYSQLX_TYPE_JSON` (int)

`MYSQLX_TYPE_NEWDECIMAL` (int)

`MYSQLX_TYPE_BIT` (int)

`MYSQLX_LOCK_DEFAULT` (int)

`MYSQLX_LOCK_NOWAIT` (int)

`MYSQLX_LOCK_SKIP_LOCKED` (int)

Registro de Alterações

As alterações a seguir foram feitas em classes/funções/métodos desta extensão.

Registro Geral Alterações para a extensão ext/mysql_xdevapi

Este registro de alterações referencia a extensão ext/mysql_xdevapi.

Alterações globais da ext/mysql_xdevapi

Versão	Descrição
8.0.28	Removido suporte a TLSv1 e TLSv1.1. Os valores "TLSv1", "TLSv1.0", e "TLSv1.1" agora são considerados inválidos para o atributo de conexão "tls-versions".
8.0.26	Suporte a TLSv1 e TLSv1.1 foi descontinuado.

Examples

O ponto de entrada central para a X DevAPI é a função **mysql_xdevapi\getSession()**, que recebe uma URI para um servidor MySQL 8.0 e retorna um objeto mysql_xdevapi\Session.

Exemplo #1 Conectando a um Servidor MySQL

```
<?php
try {
    $session = mysql_xdevapi\getSession("mysqlx://usuario:senha@servidor");
} catch(Exception $e) {
    die("Não foi possível estabelecer conexão: " . $e->getMessage());
```

```
}

// ... use $session
?>
```

A sessão fornece acesso completo à API. Para uma nova instalação do Servidor MySQL, o primeiro passo é criar um esquema com uma coleção para armazenar dados:

Exemplo #2 Criando um Esquema e uma Coleção no Servidor MySQL

```
<?php
$schema = $session->createSchema("teste");
$collection = $schema->createCollection("exemplo");
?>
```

Ao armazenar dados, normalmente json_encode() é usada para codificar os dados em JSON, que assim podem ser armazenados em uma coleção.

O exemplo a seguir armazena dados na coleção criada anteriormente, e depois recupera partes dela novamente.

Exemplo #3 Armazenando e Recuperando Dados

```
<?php
$marco = [
  "nome" => "Marco",
  "idade"  => 19,
  "emprego"  => "Programador"
];
$mike = [
  "nome" => "Mike",
  "idade"  => 39,
  "emprego"  => "Gerente"
];

$schema = $session->getSchema("teste");
$collection = $schema->getCollection("exemplo");

$collection->add($marco, $mike)->execute();

var_dump($collection->find("nome = 'Mike'")->execute()->fetchOne());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(4) {
  ["_id"]=>
  string(28) "00005ad66aaf0000000000000003"
  ["idade"]=>
  int(39)
  ["emprego"]=>
  string(7) "Gerente"
  ["nome"]=>
  string(4) "Mike"
}
```

O exemplo demonstra que o Servidor MySQL adiciona um campo extra de nome `_id`, que serve como chave primária para o documento.

O exemplo também demonstra que os dados recuperados são ordenados alfabeticamente, Esta ordem específica vem do armazenamento binário eficiente dentro do servidor MySQL, mas isso nem sempre é confiável. Refira-se à documentação do tipo de dados JSON do MySQL para detalhes.

Opcionalmente, use os iteradores do PHP para buscar múltiplos documentos:

Exemplo #4 Buscando e Iterando sobre Múltiplos Documentos

```php
<?php
$result = $collection->find()->execute();
foreach ($result as $doc) {
  echo "{$doc["nome"]} é um {$doc["emprego"]}.\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Marco é um Programador.
Mike é um Gerente.
```

Funções de Mysql_xdevapi

Índice

expression

expression

(No version information available, might only be in Git)

expression — Vincula as variáveis de instrução preparada como parâmetros

Descrição

function **mysql_xdevapi\expression**(string `$expression`): object

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`expression`

Valor Retornado

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Expression()**

```php
<?php
$expression = mysql_xdevapi\Expression("[idade,emprego]");

$res  = $coll->find("idade > 30")->fields($expression)->limit(3)->execute();
$data = $res->fetchAll();

print_r($data);
?>
```

O exemplo acima produzirá algo semelhante a:

```
<?php
```

getSession

getSession

(No version information available, might only be in Git)

getSession — Conecta a um servidor MySQL

Descrição

function **mysql_xdevapi\getSession**(string `$uri`): mysql_xdevapi\Session

Conecta a um servidor MySQL.

Parâmetros

`uri`

A URI para o servidor MySQL server, como `mysqlx://user:password@host`.

Formato da URI:

`scheme://[user[:[password]]@]target[:port][?attribute1=value1&attribute2=value2...`

- `scheme`: requerido, o protocolo de conexão

  Na mysql_xdevapi é sempre 'mysqlx' (para o Protocolo X)

- `user`: opcional, a conta de usuário do MySQL para autenticação
- `password`: opcional, a senha do usuário do MySQL para autenticação
- `target`: requerido, a instância do servidor à qual a conexão se refere:

  * Conexão TCP (nome de servidor, endereço IPv4 ou endereço IPv6)

* Caminho do soquete Unix (caminho de arquivo local)

* Pipe nomeado do Windows (caminho de arquivo local)

- `port`: opcional, porta de rede do servidor MySQL.

  Por padrão a porta do Protocolo X é 33060

- `?attribute=value`: este elemento é opcional e especifica um dicionário de dados que contém diferentes opções, incluindo:

  - O atributo `auth` (mecanismo de autenticação) relacionado a conexões criptografadas. Para informação adicional, consulte » Opções de Comando para Conexões Criptografadas. Os seguintes valores de 'auth' são suportados: `plain`, `mysql41`, `external` e `sha256_mem`.

  - O atributo `connect-timeout` afeta a conexão e não as operações subsequentes. É definido por conexão, seja em servidor único ou em múltiplos servidores.

    Passar um número inteiro positivo define o tempo limite da conexão em segundos, passar 0 (zero) desabilita o tempo limite (infinito). Não definir o tempo limite de conexão usa o valor padrão de 10.

    Relacionado, as variáveis de ambiente MYSQLX_CONNECTION_TIMEOUT (tempo limite em segundos) e MYSQLX_TEST_CONNECTION_TIMEOUT (usadas durante a execução de testes) podem ser definidas e usadas em vez do tempo limite de conexão na URI. A opção de URI de tempo limite de conexão tem precedência sobre essas variáveis de ambiente.

  - O atributo opcional `compression` aceita estes valores: `preferred` (o cliente negocia com o servidor para encontrar um algoritmo suportado; a conexão é descompactada se um algoritmo mutuamente suportado não for encontrado), `required` (como "preferred", mas a conexão é terminada se um algoritmo mutuamente suportado não for encontrado), ou `disabled` (conexão é descompactada). O padrão é `preferred`.

    Esta opção foi adicionada na versão 8.0.20.

  - O atributo opcional `compression-algorithms` define os algortimos de compressão desejados (e sua ordem preferida de uso): `zstd_stream` (sinônimo: zstd), `lz4_message` (sinônimo: lz4), ou `deflate_stream` (sinônimos: deflate ou zlib). Por padrão, a ordem usada (dependendo da disponibilidade do sistema) é lz4_message, zstd_stream e depois deflate_stream. Por exemplo, passar compression-algorithms=[lz4,zstd_stream] usa lz4 se estiver disponível, caso contrário, zstd_stream é usado. Se ambos estiverem indisponíveis, o comportamento dependerá do valor da compactação, por exemplo, se compression=required então falhará com um erro.

    Esta opção foi adicionada na versão 8.0.22.

Exemplo #1 Exemplos de URI

```
mysqlx://foobar
mysqlx://root@localhost?socket=%2Ftmp%2Fmysqld.sock%2F
mysqlx://foo:bar@localhost:33060
mysqlx://foo:bar@localhost:33160?ssl-mode=disabled
mysqlx://foo:bar@localhost:33260?ssl-mode=required
mysqlx://foo:bar@localhost:33360?ssl-mode=required&auth=mysql41
mysqlx://foo:bar@(/path/to/socket)
mysqlx://foo:bar@(/path/to/socket)?auth=sha256_mem
```

```
mysqlx://foo:bar@[localhost:33060, 127.0.0.1:33061]
mysqlx://foobar?ssl-ca=(/path/to/ca.pem)&ssl-crl=(/path/to/crl.pem)
mysqlx://foo:bar@[localhost:33060, 127.0.0.1:33061]?ssl-mode=disabled
mysqlx://foo:bar@localhost:33160/?connect-timeout=0
mysqlx://foo:bar@localhost:33160/?connect-timeout=10&compression=required
mysqlx://foo:bar@localhost:33160/?connect-timeout=10&compression=required&compression-algorithms=
[lz4,zstd_stream]
```

Para informações relacionadas, consulte a documentação sobre » Conectando com uma String de URI do Shell do MySQL.

Valor Retornado

Um objeto **Session**.

Erros/Exceções

Uma falha de conexão lança uma Exception.

Examples

Exemplo #2 Exemplo de **mysql_xdevapi\getSession()**

```php
<?php
try {
    $session = mysql_xdevapi\getSession("mysqlx://user:password@host");
} catch(Exception $e) {
    die("A conexão não pode ser estabelecida: " . $e->getMessage());
}

$schemas = $session->getSchemas();
print_r($schemas);

$mysql_version = $session->getServerVersion();
print_r($mysql_version);

var_dump($collection->find("name = 'Alfred'")->execute()->fetchOne());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => mysql_xdevapi\Schema Object
        (
            [name] => helloworld
        )
    [1] => mysql_xdevapi\Schema Object
        (
            [name] => information_schema
        )
    [2] => mysql_xdevapi\Schema Object
        (
            [name] => mysql
        )
```

```
    [3] => mysql_xdevapi\Schema Object
        (
            [name] => performance_schema
        )
    [4] => mysql_xdevapi\Schema Object
        (
            [name] => sys
        )
)

80012

array(4) {
  ["_id"]=>
  string(28) "00005ad66abf0001000400000003"
  ["age"]=>
  int(42)
  ["job"]=>
  string(7) "Butler"
  ["name"]=>
  string(4) "Alfred"
}
```

mysql_xdevapi\BaseResult

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\BaseResult** {
/* Métodos */
abstract public function **getWarnings**(): array
abstract public function **getWarningsCount**(): int
}

Índice

BaseResult::getWarnings

BaseResult::getWarnings

(No version information available, might only be in Git)

BaseResult::getWarnings — Buscar alertas da última operação

Descrição

abstract public function **mysql_xdevapi\BaseResult::getWarnings**(): array

Busca alertas gerados pela última operação do servidor MySQL.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array de objetos Warning da última operação. Cada objeto define uma mensagem ('message') de erro, um nível ('level') de erro e um código ('code') de erro. Um array vazio será retornado se nenhum erro estiver presente.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\RowResult::getWarnings()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();

$schema = $session->getSchema("foo");
$table  = $schema->getTable("test_table");

$table->insert(['x'])->values([1])->values([2])->execute();

$res = $table->select(['x/0 as bad_x'])->execute();
$warnings = $res->getWarnings();

print_r($warnings);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => mysql_xdevapi\Warning Object
        (
            [message] => Division by 0
            [level] => 2
            [code] => 1365
        )
    [1] => mysql_xdevapi\Warning Object
        (
            [message] => Division by 0
            [level] => 2
            [code] => 1365
        )
)
```

BaseResult::getWarningsCount

BaseResult::getWarningsCount

(No version information available, might only be in Git)

BaseResult::getWarningsCount — Busca contagem de alertas da última operação

Descrição

abstract public function **mysql_xdevapi\BaseResult::getWarningsCount**(): int

Retorna o número de alertas gerados pela última operação. Especificamente, esses alertas são gerados pelo servidor MySQL.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número de alertas da última operação.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\RowResult::getWarningsCount()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS foo")->execute();
$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();

$schema = $session->getSchema("foo");
$table  = $schema->getTable("test_table");

$table->insert(['x'])->values([1])->values([2])->execute();

$res = $table->select(['x/0 as bad_x'])->execute();

echo $res->getWarningsCount();
?>
```

O exemplo acima produzirá algo semelhante a:

```
2
```

mysql_xdevapi\Client

(No version information available, might only be in Git)

Introdução

Fornece acesso ao pool de conexões.

Resumo da classe

class **mysql_xdevapi\Client** {
/* Métodos */
public function close(): bool
public function **getSession**(): mysql_xdevapi\Session
}

Índice

mysql_xdevapi\Client::close

mysql_xdevapi\Client::close

(No version information available, might only be in Git)

mysql_xdevapi\Client::close — Fecha o cliente

Descrição

public function **mysql_xdevapi\Client::close**(): bool

Fecha todas as conexões de cliente com o servidor.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se as conexões forem fechadas.

Client::__construct

Client::__construct

(No version information available, might only be in Git)

Client::__construct — Construtor de Client

Descrição

private function **mysql_xdevapi\Client::__construct**()

Constrói um objeto Client.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Client::__construct()

```
<?php
$pooling_options = '{
  "enabled": true,
    "maxSize": 10,
    "maxIdleTime": 3600,
```

```
    "queueTimeOut": 1000
}';
$client = mysql_xdevapi\getClient($connection_uri, $pooling_options);
$session = $client->getSession();
```

Client::getClient

Client::getClient

(No version information available, might only be in Git)

Client::getClient — Obtém a sessão do cliente

Descrição

public function **mysql_xdevapi\Client::getSession**(): mysql_xdevapi\Session

Obtém a sessão associada com o cliente.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto Session.

mysql_xdevapi\Collection

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\Collection** implements mysql_xdevapi\SchemaObject {
/* Propriedades */
public *$name*;
/* Métodos */
public function **add**(mixed `$document`): mysql_xdevapi\CollectionAdd
public function **addOrReplaceOne**(string `$id`, string `$doc`): mysql_xdevapi\Result
public function **count**(): int
public function **createIndex**(string `$index_name`, string `$index_desc_json`): void
public function **dropIndex**(string `$index_name`): bool
public function **existsInDatabase**(): bool
public function **find**(string `$search_condition = ?`): mysql_xdevapi\CollectionFind
public function **getName**(): string
public function **getOne**(string `$id`): Document
public function **getSchema**(): Schema Object
public function **getSession**(): Session
public function **modify**(string `$search_condition`): mysql_xdevapi\CollectionModify
public function **remove**(string `$search_condition`): mysql_xdevapi\CollectionRemove
public function **removeOne**(string `$id`): mysql_xdevapi\Result
public function **replaceOne**(string `$id`, string `$doc`): mysql_xdevapi\Result
}

Propriedades

name

Índice

Collection::add

Collection::add

(No version information available, might only be in Git)

Collection::add — Adiciona um documento na coleção

Descrição

public function **mysql_xdevapi\Collection::add**(mixed `$document`): mysql_xdevapi\CollectionAdd

Dispara a inserção do(s) documento(s) informado(s) na coleção, múltiplas variantes deste método são suportadas. As opções incluem:

1. Adiciona um único documento como uma string JSON.
2. Adiciona um único documento como um array no formato: `[ 'field' => 'value', 'field2' => 'value2' ... ]`
3. Uma combinação de ambos, múltiplos documentos podem ser adicionados na mesma operação.

Parâmetros

`document`

Um ou múltiplos documentos, pode ser um JSON ou um array de campos com seus valores associados. Não pode ser um array vazio.

O servidor MySQL gera automaticamente um valor único de `_id` para cada documento (recomendado), embora também possa ser adicionado manualmente. Este valor precisa ser único, caso contrário a operação de adição falhará.

Valor Retornado

Um objeto CollectionAdd. Use execute() para retornar um Result que possa ser usado para consultar o número de itens afetados, o número de alertas gerados pela operação, ou para buscar uma lista de IDs gerados para os documentos inseridos.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Collection::add()**

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$collection = $schema->getCollection("people");
```

```php
// Adiciona dois documentos
$collection->add('{"name": "Fred",  "age": 21, "job": "Construction"}')->execute();
$collection->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')->execute();

// Adiciona dois documentos usando um único objeto JSON
$result = $collection->add(
  '{"name": "Bernie",
    "jobs": [{"title":"Cat Herder","Salary":42000}, {"title":"Father","Salary":0}],
    "hobbies": ["Sports","Making cupcakes"]}',
  '{"name": "Jane",
    "jobs": [{"title":"Scientist","Salary":18000}, {"title":"Mother","Salary":0}],
    "hobbies": ["Walking","Making pies"]}')->execute();

// Busca uma lista de IDs gerados pelo último add()
$ids = $result->getGeneratedIds();
print_r($ids);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => 00005b6b53610000000000000056
    [1] => 00005b6b53610000000000000057
)
```

Notas

> **Nota** :
>
> Um _id único é gerado pelo MySQL Server 8.0 ou superior, como demonstrado no exemplo. O campo _id precisa ser definido manualmente se o MySQL Server 5.7 estiver sendo usado.

Collection::addOrReplaceOne

Collection::addOrReplaceOne

(No version information available, might only be in Git)

Collection::addOrReplaceOne — Adiciona ou substitui um documento na coleção

Descrição

public function **mysql_xdevapi\Collection::addOrReplaceOne**(string `$id`, string `$doc`): mysql_xdevapi\Result

Adiciona um novo documento, ou substitui um documento se ele já existir.

Aqui estão vários cenários para este método:

- Se nem o id nem qualquer valor único de chave conflitar com algum documento na coleção, o documento é adicionado.
- Se o id não corresponder a nenhum documento mas uma ou mais valores únicos de chave conflitarem com um documento na coleção, um erro é emitido.
- Se o id corresponder a um documento existente e não houver chaves únicas definidas para a coleção, o documento é substituído.
- Se o id corresponder a um documento existente e, todas as chaves únicas no documento substituto corresponderem ao mesmo documento ou não conflitarem com nenhum outro documento na coleção, o documento é substituído.
- Se o id corresponder a um documento existente e uma ou mais chaves únicas corresponderem a um documento diferente na coleção, um erro é emitido.

Parâmetros

`id`

Este é o id de filtro. Se este id ou qualquer outro campo que tiver um índice único já existir na coleção, o documento correspondente será atualizado.

Por padrão, este id é gerado automaticamente pelo servidor MySQL quando o registro é adicionado e é referenciado como um campo com o nome '_id'.

`doc`

Este é o documento a ser inserido ou substituído, que é uma string JSON.

Valor Retornado

Um objeto Result.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Collection::addOrReplaceOne()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$collection = $schema->getCollection("people");

// Usando add()
$result = $collection->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')->execute();

// Usando addOrReplaceOne()
// Nota: aqui está sendo passado um valor conhecido de _id
$result = $collection->addOrReplaceOne('00005b6b536100000000000000056', '{"name": "Fred",  "age": 21,
"job": "Construction"}');

?>
```

Collection::__construct

Collection::__construct

(No version information available, might only be in Git)

Collection::__construct — Construtor de Collection

Descrição

private function **mysql_xdevapi\Collection::__construct**()

Constrói um objeto Collection.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Collection::getOne()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection->add('{"name": "Alfred", "age": 42, "job": "Butler"}')->execute();

// Um _id único é gerado pelo servidor MySQL (por padrão, e recomendado)
// Aqui os ids gerados são recuperados; somente um neste exemplo, $ids[0]
$ids        = $result->getGeneratedIds();
$alfreds_id = $ids[0];

// ...

print_r($alfreds_id);
print_r($collection->getOne($alfreds_id));
?>
```

O exemplo acima produzirá algo semelhante a:

```
00005b6b536100000000000000b1

Array
(
    [_id] => 00005b6b536100000000000000b1
    [age] => 42
    [job] => Butler
    [name] => Alfred
)
```

Collection::count

Collection::count

(No version information available, might only be in Git)

Collection::count — Obtém a contagem de documentos

Descrição

public function **mysql_xdevapi\Collection::count**(): int

Esta funcionalidade é similar à operação SQL `SELECT COUNT(*)` no servidor MySQL para o esquema e coleção atuais. Em outras palavras, ela conta o número de documentos na coleção.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número de documentos na coleção.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Collection::count()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$collection = $schema->getCollection("people");

$result = $collection
  ->add(
  '{"name": "Bernie",
    "jobs": [
      {"title":"Cat Herder","Salary":42000},
      {"title":"Father","Salary":0}
    ],
    "hobbies": ["Sports","Making cupcakes"]}',
  '{"name": "Jane",
    "jobs": [
      {"title":"Scientist","Salary":18000},
      {"title":"Mother","Salary":0}
    ],
    "hobbies": ["Walking","Making pies"]}')
  ->execute();

var_dump($collection->count());
?>
```

O exemplo acima produzirá:

```
int(2)
```

Collection::createIndex

Collection::createIndex

(No version information available, might only be in Git)

Collection::createIndex — Cria índice na coleção

Descrição

public function **mysql_xdevapi\Collection::createIndex**(string `$index_name`, string `$index_desc_json`): void

Cria um índice na coleção.

Uma exceção é lançada se um índice com o mesmo nome já existir, ou se a definição do índice não estiver formada corretamente.

Parâmetros

`index_name`

O nome do índice a ser criado. O nome precisa ser válido da mesma forma que os nomes aceitos pela consulta SQL `CREATE INDEX`.

`index_desc_json`

Definição do índice a ser criado. Contém um array de objetos IndexField, e cada objeto descreve um único documento membro para incluir no índice e uma string opcional para o tipo de índice, que pode ser INDEX (padrão) ou SPATIAL.

Uma descrição de um único IndexField consiste nos campos a seguir:

- `field`: string, o caminho completo para o documento membro ou campo a ser indexado.
- `type`: string, um dos tipos de coluna suportados pelo SQL para mapear o campo. Para tipos numéricos, a palavra-chave opcional UNSIGNED pode ser incluída na sequência. Para o tipo TEXT, o comprimento a considerar para a indexação pode ser incluído.
- `required`: booleano, (opcional) TRUE se for necessário que o campo exista no documento. O padrão é **`false`**, exceto para `GEOJSON` onde o padrão é **`true`**.
- `options`: inteiro, (opcional) opções especiais para uso durante a decodificação de dados `GEOJSON`.
- `srid`: inteiro, (opcional) valor srid para uso durante a decodificação de dados `GEOJSON`.

É um erro incluir outros campos não descritos acima em documentos IndexDefinition ou IndexField.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Collection::createIndex()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

// Criando um índice de texto
$collection->createIndex(
  'myindex1',
  '{"fields": [{
    "field": "$.name",
    "type": "TEXT(25)",
    "required": true}],
    "unique": false}'
);

// Um índice espacial
$collection->createIndex(
  'myindex2',
  '{"fields": [{
    "field": "$.home",
    "type": "GEOJSON",
    "required": true}],
    "type": "SPATIAL"}'
);

// Índice com múltiplos campos
$collection->createIndex(
  'myindex3',
  '{"fields": [
    {
      "field": "$.name",
      "type": "TEXT(20)",
      "required": true
    },
    {
      "field": "$.age",
      "type": "INTEGER"
    },
    {
      "field": "$.job",
      "type": "TEXT(30)",
      "required": false
    }
  ],
  "unique": true
  }'
);
```

Collection::dropIndex

Collection::dropIndex

(No version information available, might only be in Git)

Collection::dropIndex — Remove um índice da coleção

Descrição

public function **mysql_xdevapi\Collection::dropIndex**(string `$index_name`): bool

Remove um índice da coleção.

Esta operação não gera um erro se o índice não existir, mas neste caso é retornado `false`.

Parâmetros

`index_name`

Nome do índice da coleção a ser removido.

Valor Retornado

`true` se a operação DROP INDEX for bem sucedida, caso contrário `false`.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Collection::dropIndex()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

// ...

$collection = $schema->getCollection("people");

$collection->createIndex(
  'myindex',
  '{"fields": [{"field": "$.name", "type": "TEXT(25)", "required": true}], "unique": false}'
);

// ...

if ($collection->dropIndex('myindex')) {
    echo "Um índice de nome 'myindex' foi encontrado e removido.";
}
?>
```

O exemplo acima produzirá:

```
Um índice de nome 'myindex' foi encontrado e removido.
```

Collection::existsInDatabase

Collection::existsInDatabase

(No version information available, might only be in Git)

Collection::existsInDatabase — Verifica se uma coleção existe no banco de dados

Descrição

public function **mysql_xdevapi\Collection::existsInDatabase**(): bool

Verifica se o objeto Collection se refere a uma coleção no banco de dados (esquema).

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se a coleção existir no banco de dados, `false` se não existir.

Uma tabela definida com duas colunas (doc e _id) é considerada uma coleção, e uma terceira coluna _json_schema a partir do MySQL 8.0.21. Inserir uma coluna adicional significa que existsInDatabase() não mais a verá como uma coleção.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Collection::existsInDatabase()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

// ...

$collection = $schema->getCollection("people");

// ...

if (!$collection->existsInDatabase()) {
    echo "A coleção não existe mais no banco de dados addressbook. O que aconteceu?";
}
?>
```

Collection::find

Collection::find

(No version information available, might only be in Git)

Collection::find — Pesquisa por um documento

Descrição

public function **mysql_xdevapi\Collection::find**(string `$search_condition` = ?): mysql_xdevapi\CollectionFind

Pesquisa uma coleção do banco de dados por um documento ou conjunto de documentos. Os documentos encontrados são retornados como um objeto CollectionFind que servirá para modificação ou para busca de resultados.

Parâmetros

`search_condition`

> Embora seja opcional, normalmente uma condição é definida para limitar os resultados a um subconjunto de documentos.
>
> Múltiplos elementos podem fazer parte da condição e a sintaxe suporta vinculação de parâmetros. A expressão usada como condição de pesquisa precisa ser uma expressão SQL válida. Se nenhuma condição de pesquisa for informada (campo vazio), será assumida a expressão find('true').

Valor Retornado

Um objeto CollectionFind para verificar a operação, ou buscar os documentos encontrados.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Collection::find()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$collection->add('{"name": "Alfred",     "age": 18, "job": "Butler"}')->execute();
$collection->add('{"name": "Bob",        "age": 19, "job": "Swimmer"}')->execute();
$collection->add('{"name": "Fred",       "age": 20, "job": "Construction"}')->execute();
$collection->add('{"name": "Wilma",      "age": 21, "job": "Teacher"}')->execute();
$collection->add('{"name": "Suki",       "age": 22, "job": "Teacher"}')->execute();

$find   = $collection->find('job LIKE :job AND age > :age');
$result = $find
  ->bind(['job' => 'Teacher', 'age' => 20])
  ->sort('age DESC')
  ->limit(2)
  ->execute();

print_r($result->fetchAll());
?>
```

O exemplo acima produzirá:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b536100000000000000a8
            [age] => 22
            [job] => Teacher
            [name] => Suki
        )
    [1] => Array
        (
            [_id] => 00005b6b536100000000000000a7
            [age] => 21
            [job] => Teacher
            [name] => Wilma
        )
)
```

Collection::getName

Collection::getName

(No version information available, might only be in Git)

Collection::getName — Obtém o nome da coleção

Descrição

public function **mysql_xdevapi\Collection::getName**(): string

Recupera o nome da coleção.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O nome da coleção, como uma string.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Collection::getName()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");


// ...
```

```php
var_dump($collection->getName());
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(6) "people"
```

Collection::getOne

Collection::getOne

(No version information available, might only be in Git)

Collection::getOne — Obtém um documento

Descrição

public function **mysql_xdevapi\Collection::getOne**(string `$id`): Document

Busca um documento da coleção.

É um atalho para: `Collection.find("_id = :id").bind("id", id).execute().fetchOne();`

Parâmetros

`id`

O _id do documento na coleção.

Valor Retornado

O objeto Collection, ou `null` se o _id não corresponder a um documento.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Collection::getOne()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection->add('{"name": "Alfred", "age": 42, "job": "Butler"}')->execute();

// Um _id único é gerado pelo servidor MySQL (por padrão, e recomendado)
// Isto recupera os ids gerados; somente um neste exemplo, $ids[0]
$ids        = $result->getGeneratedIds();
```

```php
$alfreds_id = $ids[0];

// ...

print_r($alfreds_id);
print_r($collection->getOne($alfreds_id));
?>
```

O exemplo acima produzirá algo semelhante a:

```
00005b6b536100000000000000b1

Array
(
    [_id] => 00005b6b536100000000000000b1
    [age] => 42
    [job] => Butler
    [name] => Alfred
)
```

Collection::getSchema

Collection::getSchema

(No version information available, might only be in Git)

Collection::getSchema — Obtém o objeto de esquema

Descrição

public function **mysql_xdevapi\Collection::getSchema**(): Schema Object

Recupera o objeto de esquema que contém a coleção.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O objeto de esquema em caso de sucesso, ou `null` se o objeto não puder ser recuperado para a coleção informada.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Collection::getSchema()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
```

```php
$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

var_dump($collection->getSchema());
?>
```

O exemplo acima produzirá algo semelhante a:

```
object(mysql_xdevapi\Schema)#9 (1) {
  ["name"]=>
  string(11) "addressbook"
}
```

Collection::getSession

Collection::getSession

(No version information available, might only be in Git)

Collection::getSession — Obtém o objeto de sessão

Descrição

public function **mysql_xdevapi\Collection::getSession**(): Session

Obtém um novo objeto Session do objeto Collection.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto Session.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Collection::getSession()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

// ...

$newsession = $collection->getSession();
```

```
var_dump($session);
var_dump($newsession);
?>
```

O exemplo acima produzirá algo semelhante a:

```
object(mysql_xdevapi\Session)#1 (0) {
}
object(mysql_xdevapi\Session)#4 (0) {
}
```

Collection::modify

Collection::modify

(No version information available, might only be in Git)

Collection::modify — Modifica documentos da coleção

Descrição

public function **mysql_xdevapi\Collection::modify**(string `$search_condition`): mysql_xdevapi\CollectionModify

Modifica os documentos de uma coleção que se encaixam em condições específicas de pesquisa. Múltiplas operações são permitidas e a vinculação de parâmetros é suportada.

Parâmetros

`search_condition`

> Precisa ser uma expressão SQL válida usada para corresponder aos documentos que serão modificados. Esta expressão pode ser tão simples como `true`, que corresponde a todos os documentos, ou pode conter funções e operações como `'CAST(_id AS SIGNED) >= 10'`, `'age MOD 2 = 0 OR age MOD 3 = 0'` ou `'_id IN ["2","5","7","10"]'`.

Valor Retornado

Se a operação não for executada, a função retornará um objeto Modify que pode ser usado para inserir operações adicionais de modificação.

Se a operação de modificação for executada, o objeto retornado conterá o resultado da operação.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Collection::modify()**

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
```

```php
$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$collection->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$collection->add('{"name": "Bob",    "age": 19, "job": "Painter"}')->execute();

// Adiciona dois novos 'jobs' para todos os 'Painters': 'Artist' e 'Crafter'
$collection
  ->modify("job in ('Butler', 'Painter')")
  ->arrayAppend('job', 'Artist')
  ->arrayAppend('job', 'Crafter')
  ->execute();

// Remove o campo 'beer' de todos os documentos com a idade (age) menor que 21
$collection
  ->modify('age < 21')
  ->unset(['beer'])
  ->execute();
?>
```

Collection::remove

Collection::remove

(No version information available, might only be in Git)

Collection::remove — Remove documentos da coleção

Descrição

public function **mysql_xdevapi\Collection::remove**(string `$search_condition`): mysql_xdevapi\CollectionRemove

Remove documentos de uma coleção que se encaixam em condições específicas de pesquisa. Múltiplas operações são permitidas e a vinculação de parâmetros é suportada.

Parâmetros

`search_condition`

> Precisa ser uma expressão SQL válida usada para corresponder aos documentos que serão modificados. Esta expressão pode ser tão simples como `true`, que corresponde a todos os documentos, ou pode conter funções e operações como `'CAST(_id AS SIGNED) >= 10'`, `'age MOD 2 = 0 OR age MOD 3 = 0'` ou `'_id IN ["2","5","7","10"]'`.

Valor Retornado

Se a operação não for executada, a função retornará um objeto Remove que pode ser usado para inserir operações de remoção adicionais.

Se a operação de remoção for executada, o objeto retornado conterá o resultado da operação.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Collection::remove()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
```

```php
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$collection->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$collection->add('{"name": "Bob",    "age": 19, "job": "Painter"}')->execute();

// Remove todos os 'painters'
$collection
  ->remove("job in ('Painter')")
  ->execute();

// Remove o 'butler' mais antigo (maior 'age')
$collection
  ->remove("job in ('Butler')")
  ->sort('age desc')
  ->limit(1)
  ->execute();

// Remove o registro com maior 'age'
$collection
  ->remove('true')
  ->sort('age desc')
  ->limit(1)
  ->execute();
?>
```

Collection::removeOne

Collection::removeOne

(No version information available, might only be in Git)

Collection::removeOne — Remove um documento da coleção

Descrição

public function **mysql_xdevapi\Collection::removeOne**(string `$id`): mysql_xdevapi\Result

Remove da coleção o documento com o ID correspondente. É um atalho para `Collection.remove("_id = :id").bind("id", id).execute()`.

Parâmetros

`id`

> O ID do documento da coleção a ser removido. Tipicamente é o _id gerado pelo servidor MySQL quando o registro foi inserido.

Valor Retornado

Um objeto Result que pode ser usado para consultar o número de itens afetados ou o número de alertas gerados pela operação.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Collection::removeOne()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();

// Normalmente o _id é conhecido por outros meios,
// mas para este exemplo vamos buscar o id gerado e usá-lo
$ids       = $result->getGeneratedIds();
$alfred_id = $ids[0];

$result = $collection->removeOne($alfred_id);

if(!$result->getAffectedItemsCount()) {
    echo "Alfred com id $alfred_id não foi removido.";
} else {
    echo "Tchau, Alfred, pode levar o _id $alfred_id com você.";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Tchau, Alfred, pode levar o _id 00005b6b536100000000000000cb com você.
```

Collection::replaceOne

Collection::replaceOne

(No version information available, might only be in Git)

Collection::replaceOne — Substitui um documento na coleção

Descrição

public function **mysql_xdevapi\Collection::replaceOne**(string `$id`, string `$doc`): mysql_xdevapi\Result

Atualiza (ou substitui) o documento identificado pelo ID, se existir.

Parâmetros

`id`

ID do documento para substituir ou atualizar. Tipicamente é o _id que foi gerado pelo servidor MySQL quando o registro foi inserido.

doc

Documento da coleção para atualizar ou substituir o documento que corresponde ao parâmetro **id**.

Este documento pode ser um objeto de documento ou uma string JSON válida descrevendo o novo documento.

Valor Retornado

Um objeto Result que pode ser usado para consultar o número de itens afetados e o número de alertas gerados pela operação.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Collection::replaceOne()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();

// Normalmente o _id é conhecido por outros meios,
// mas para este exemplo vamos buscar o id gerado e usá-lo
$ids       = $result->getGeneratedIds();
$alfred_id = $ids[0];

// ...

$alfred = $collection->getOne($alfred_id);
$alfred['age'] = 81;
$alfred['job'] = 'Guru';

$collection->replaceOne($alfred_id, $alfred);

?>
```

mysql_xdevapi\CollectionAdd

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\CollectionAdd** implements mysql_xdevapi\Executable {
/* Métodos */
public function **execute**(): mysql_xdevapi\Result
}

Índice

CollectionAdd::__construct

CollectionAdd::__construct

(No version information available, might only be in Git)

CollectionAdd::__construct — Construtor de CollectionAdd

Descrição

private function **mysql_xdevapi\CollectionAdd::__construct**()

Use para adicionar um documento à coleção; chamado a partir de um objeto Collection.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionAdd::__construct()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$collection = $schema->getCollection("people");

// Adiciona dois documentos
$collection
  ->add('{"name": "Fred",  "age": 21, "job": "Construction"}')
  ->execute();

$collection
  ->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')
  ->execute();

// Adiciona dois documentos usando um único objeto JSON
$result = $collection
  ->add(
    '{"name": "Bernie",
      "jobs": [{"title":"Cat Herder","Salary":42000}, {"title":"Father","Salary":0}],
      "hobbies": ["Sports","Making cupcakes"]}',
    '{"name": "Jane",
      "jobs": [{"title":"Scientist","Salary":18000}, {"title":"Mother","Salary":0}],
      "hobbies": ["Walking","Making pies"]}')
  ->execute();

// Busca uma lista de IDs gerados pelo último add()
$ids = $result->getGeneratedIds();
print_r($ids);

?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => 00005b6b53610000000000000056
    [1] => 00005b6b53610000000000000057
)
```

Notas

> **Nota** :
>
> Um _id único é gerado pelo MySQL Server 8.0 ou superior, como demonstrado no exemplo. O campo _id precisa ser definido manualmente se o MySQL Server 5.7 estiver sendo usado.

CollectionAdd::execute

CollectionAdd::execute

(No version information available, might only be in Git)

CollectionAdd::execute — Executa a instrução

Descrição

public function **mysql_xdevapi\CollectionAdd::execute**(): mysql_xdevapi\Result

O método de execução é requerido para enviar a requisição de operação CRUD ao servidor MySQL.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto Result que pode ser usado para verificar o status da operação, como o número de linhas afetadas.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\CollectionAdd::execute()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$collection = $schema->getCollection("people");

// Adiciona dois documentos
$collection
```

```
  ->add('{"name": "Fred",  "age": 21, "job": "Construction"}')
  ->execute();

$collection
  ->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')
  ->execute();

// Adiciona dois documentos usando um único objeto JSON
$result = $collection
  ->add(
    '{"name": "Bernie",
      "jobs": [{"title":"Cat Herder","Salary":42000}, {"title":"Father","Salary":0}],
      "hobbies": ["Sports","Making cupcakes"]}',
    '{"name": "Jane",
      "jobs": [{"title":"Scientist","Salary":18000}, {"title":"Mother","Salary":0}],
      "hobbies": ["Walking","Making pies"]}')
  ->execute();

// Busca uma lista de IDs gerados no último add()
$ids = $result->getGeneratedIds();
print_r($ids);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => 00005b6b53610000000000000056
    [1] => 00005b6b53610000000000000057
)
```

mysql_xdevapi\CollectionFind

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\CollectionFind** implements mysql_xdevapi\Executable, mysql_xdevapi\CrudOperationBindable, mysql_xdevapi\CrudOperationLimitable, mysql_xdevapi\CrudOperationSortable {
/* Métodos */
public function **bind**(array `$placeholder_values`): mysql_xdevapi\CollectionFind
public function **execute**(): mysql_xdevapi\DocResult
public function **fields**(string `$projection`): mysql_xdevapi\CollectionFind
public function **groupBy**(string `$sort_expr`): mysql_xdevapi\CollectionFind
public function **having**(string `$sort_expr`): mysql_xdevapi\CollectionFind
public function **limit**(int `$rows`): mysql_xdevapi\CollectionFind
public function **lockExclusive**(int `$lock_waiting_option` = ?): mysql_xdevapi\CollectionFind
public function **lockShared**(int `$lock_waiting_option` = ?): mysql_xdevapi\CollectionFind

public function **offset**(int `$position`): mysql_xdevapi\CollectionFind
public function **sort**(string `$sort_expr`): mysql_xdevapi\CollectionFind
}

Índice

CollectionFind::bind

CollectionFind::bind

(No version information available, might only be in Git)

CollectionFind::bind — Vincula valor a marcador na consulta

Descrição

public function **mysql_xdevapi\CollectionFind::bind**(array `$placeholder_values`): mysql_xdevapi\CollectionFind

Permite ao usuário vincular um parâmetro ao marcador na condição de pesquisa da operação de busca. O marcador tem a forma de :NOME onde ':' é um prefixo comum que precisa sempre existir antes de qualquer NOME, o nome real do marcador. A função de vinculação aceita uma lista de marcadores se múltiplas entidades precisarem ser substituídas na condição de pesquisa.

Parâmetros

`placeholder_values`

> Valores a substituir na condição de pesquisa; múltiplos valores são permitidos e são passados como um array onde "NOME_MARCADOR => VALOR_MARCADOR".

Valor Retornado

Um objeto CollectionFind, ou encadeado com execute() para retornar um objeto Result.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionFind::bind()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");
$result = $create
  ->add('{"name": "Alfred", "age": 18, "job": "Butler"}')
  ->execute();

// ...

$collection = $schema->getCollection("people");

$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->execute();
```

```
var_dump($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1) {
  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000cf"
    ["age"]=>
    int(18)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(6) "Alfred"
  }
}
```

CollectionFind::__construct

CollectionFind::__construct

(No version information available, might only be in Git)

CollectionFind::__construct — Construtor de CollectionFind

Descrição

private function **mysql_xdevapi\CollectionFind::__construct**()

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de CollectionFind

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");
$result = $create
```

```
  ->add('{"name": "Alfred", "age": 18, "job": "Butler"}')
  ->execute();

// ...

$collection = $schema->getCollection("people");

$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->execute();

var_dump($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1) {
  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000cf"
    ["age"]=>
    int(18)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(6) "Alfred"
  }
}
```

CollectionFind::execute

CollectionFind::execute

(No version information available, might only be in Git)

CollectionFind::execute — Executa a instrução

Descrição

public function **mysql_xdevapi\CollectionFind::execute**(): mysql_xdevapi\DocResult

Executa a operação de busca; esta funcionalidade permite encadeamento de métodos.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto mysql_xdevapi\DocResult que pode ser usado para buscar resultados ou para consultar o status da operação.

Examples

Exemplo #1 Exemplo de CollectionFind

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$create
  ->add('{"name": "Alfred", "age": 18, "job": "Butler"}')
  ->execute();

// ...

$collection = $schema->getCollection("people");

$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->execute();

var_dump($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1) {
  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000cf"
    ["age"]=>
    int(18)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(6) "Alfred"
  }
}
```

CollectionFind::fields

CollectionFind::fields

(No version information available, might only be in Git)

CollectionFind::fields — Define filtro de campo de documento

Descrição

public function **mysql_xdevapi\CollectionFind::fields**(string `$projection`): mysql_xdevapi\CollectionFind

Define as colunas para a consulta retornar. Se não definidas, todas as colunas são usadas.

Parâmetros

`projection`

> Pode ser uma única string ou um array de strings identificando as colunas a serem retornadas para cada documento que corresponder à condição de pesquisa.

Valor Retornado

Um objeto CollectionFind que pode ser usado para processamento adicional.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\CollectionFind::fields()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$create
  ->add('{"name": "Alfred", "age": 18, "job": "Butler"}')
  ->execute();

// ...

$collection = $schema->getCollection("people");

$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->fields('name')
  ->execute();

var_dump($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1) {
  [0]=>
  array(1) {
    ["name"]=>
    string(6) "Alfred"
```

```
  }
}
```

CollectionFind::groupBy

CollectionFind::groupBy

(No version information available, might only be in Git)

CollectionFind::groupBy — Define critérios de agrupamento

Descrição

public function **mysql_xdevapi\CollectionFind::groupBy**(string `$sort_expr`): mysql_xdevapi\CollectionFind

Esta função pode ser usada para agrupar o conjunto de resultados por uma ou mais colunas. É frequentemente usado com funções de agragação como `COUNT`, `MAX`, `MIN`, `SUM` etc.

Parâmetros

`sort_expr`

> A coluna ou colunas que serão usadas para a operação de agrupamento, pode ser uma string única ou um array de strings, uma para cada coluna.

Valor Retornado

Um objeto CollectionFind que pode ser usado para processamento adicional.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionFind::groupBy()

```php
<?php

// Assumindo que $coll é um objeto Collection válido

// Extrai todos os documentos da coleção e agrupa os resultados pelo campo 'nome'
$res = $coll->find()->groupBy('nome')->execute();

?>
```

CollectionFind::having

CollectionFind::having

(No version information available, might only be in Git)

CollectionFind::having — Define condição para funções de agregação

Descrição

public function **mysql_xdevapi\CollectionFind::having**(string `$sort_expr`): mysql_xdevapi\CollectionFind

Esta função pode ser usada depois da operação 'field' para fazer uma seleção nos documentos a extrair.

Parâmetros

`sort_expr`

Precisa ser uma expressão SQL válida, o uso de funções de agregação é permitido.

Valor Retornado

Objeto CollectionFind que pode ser usado para processamento adicional.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionFind::having()

```php
<?php

// Assumindo que $coll é um objeto Collection válido

// Encontra todos os documentos para os quais a 'idade' é maior que 40,
// Somente as colunas 'nome' e 'idade' são retornadas no objeto Result
$res = $coll->find()->fields(['nome','idade'])->having('idade > 40')->execute();

?>
```

CollectionFind::limit

CollectionFind::limit

(No version information available, might only be in Git)

CollectionFind::limit — Limita o número de documentos retornados

Descrição

public function **mysql_xdevapi\CollectionFind::limit**(int `$rows`): mysql_xdevapi\CollectionFind

Define o número máximo de documentos a retornar.

Parâmetros

`rows`

Número máximo de documentos.

Valor Retornado

Um objeto CollectionFind que pode ser usado para processamento adicional; pode ser encadeado com o método execute() para retornar um objeto DocResult.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionFind::limit()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
```

```php
$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");
$create
  ->add('{"name": "Alfred", "age": 18, "job": "Butler"}')
  ->execute();
$create
  ->add('{"name": "Reginald", "age": 42, "job": "Butler"}')
  ->execute();


// ...

$collection = $schema->getCollection("people");

$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->sort('age desc')
  ->limit(1)
  ->execute();

var_dump($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1) {
  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000f3"
    ["age"]=>
    int(42)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(8) "Reginald"
  }
}
```

CollectionFind::lockExclusive

CollectionFind::lockExclusive

(No version information available, might only be in Git)

CollectionFind::lockExclusive — Executa a operação com EXCLUSIVE LOCK

Descrição

public function **mysql_xdevapi\CollectionFind::lockExclusive**(int `$lock_waiting_option = ?`): mysql_xdevapi\CollectionFind

Trava o documento de forma exclusiva. Enquanto o documento estiver travado, outras transações não podem atualizá-lo, use `SELECT ... LOCK IN SHARE MODE`, ou leia os dados em certos níveis de isolamento de transação. Leituras consistentes ignoram qualquer trava definida em registros que existem na visualização de leitura.

Para evitar problemas de concorrência, faz sentido usar esta função com o método **mysql_xdevapi\Collection::modify()** Essencialmente, esta função usa travas de registros para serializar acesso a eles.

Parâmetros

`lock_waiting_option`

Parâmetro opcional de espera. Por padrão é `MYSQLX_LOCK_DEFAULT`. Valores válidos são estas constantes:

- `MYSQLX_LOCK_DEFAULT`
- `MYSQLX_LOCK_NOWAIT`
- `MYSQLX_LOCK_SKIP_LOCKED`

Valor Retornado

Retorna um objeto CollectionFind que pode ser usado para processamento adicional.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionFind::lockExclusive()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$session->startTransaction();

$result = $collection
  ->find("age > 50")
  ->lockExclusive()
  ->execute();

// ... faz uma operação com o objeto

// Completa a transação e destrava o documento
$session->commit();
?>
```

CollectionFind::lockShared

CollectionFind::lockShared

(No version information available, might only be in Git)

CollectionFind::lockShared — Executa a operação com SHARED LOCK

Descrição

public function **mysql_xdevapi\CollectionFind::lockShared**(int `$lock_waiting_option = ?`): mysql_xdevapi\CollectionFind

Permite compartilhar os documentos entre múltiplas transações que estão travando em modo compartilhado.

Outras sessões podem ler as linhas, mas não podem modificá-las até que a transação seja efetivamente executada.

Se quaisquer destas linhas tiverem sido alteradas por outra transação que não tenha sido efetivamente executada, a consulta irá esperar até que a transação tenha terminado para ser capaz de usar os valores mais recentes.

Parâmetros

`lock_waiting_option`

Parâmetro opcional de espera. Por padrão é `MYSQLX_LOCK_DEFAULT`. Valores válidos são estas constantes:

- `MYSQLX_LOCK_DEFAULT`
- `MYSQLX_LOCK_NOWAIT`
- `MYSQLX_LOCK_SKIP_LOCKED`

Valor Retornado

Um objeto CollectionFind que pode ser usado para processamento adicional.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionFind::lockShared()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$session->startTransaction();

$result = $collection
  ->find("age > 50")
  ->lockShared()
  ->execute();

// ... lê o objeto em modo compartilhado

// Completa a transação e destrava o documento
$session->commit();
?>
```

CollectionFind::offset

CollectionFind::offset

(No version information available, might only be in Git)

CollectionFind::offset — Pula um número de elementos a serem retornados

Descrição

public function **mysql_xdevapi\CollectionFind::offset**(int `$position`): mysql_xdevapi\CollectionFind

Pula (desloca) este número de elementos que de outra forma seriam retornados pela operação de pesquisa. Use com o método limit().

Definir um deslocamento maior que o tamanho do conjunto de resultados resulta em um conjunto vazio.

Parâmetros

`position`

Número de elementos a pular para a operação limit().

Valor Retornado

Um objeto CollectionFind que pode ser usado para processamento adicional.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionFind::offset()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");
$create
  ->add('{"name": "Alfred", "age": 18, "job": "Butler"}')
  ->execute();
$create
  ->add('{"name": "Reginald", "age": 42, "job": "Butler"}')
  ->execute();

// ...

$collection = $schema->getCollection("people");

$result = $collection
  ->find()
  ->sort('age asc')
  ->offset(1)
  ->limit(1)
  ->execute();

var_dump($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1) {
  [0]=>
  array(4) {
```

```
    ["_id"]=>
    string(28) "00005b6b536100000000000000f3"
    ["age"]=>
    int(42)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(8) "Reginald"
  }
}
```

CollectionFind::sort

CollectionFind::sort

(No version information available, might only be in Git)

CollectionFind::sort — Define os critérios de ordenação

Descrição

public function **mysql_xdevapi\CollectionFind::sort**(string `$sort_expr`): mysql_xdevapi\CollectionFind

Ordena o conjunto de resultados pelo campo selecionado no argumento sort_expr. As ordens suportadas são ASC (ascendente) ou DESC (descendente). Esta operação é equivalente à 'ORDER BY' do SQL e segue o mesmo conjunto de regras.

Parâmetros

`sort_expr`

> Uma ou mais expressões podem ser fornecidas. A avaliação é da esquerda para a direita, e as expressões são separadas por vírgula.

Valor Retornado

Um objeto CollectionFind que pode ser usado para executar o comando, ou para inserir operações adicionais.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\CollectionFind::sort()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");
$create
  ->add('{"name": "Alfred", "age": 18, "job": "Butler"}')
  ->execute();
$create
  ->add('{"name": "Reginald", "age": 42, "job": "Butler"}')
  ->execute();
```

```php
// ...

$collection = $schema->getCollection("people");

$result = $collection
  ->find()
  ->sort('job desc', 'age asc')
  ->execute();

var_dump($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(2) {
  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000106"
    ["age"]=>
    int(18)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(6) "Alfred"
  }
  [1]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000107"
    ["age"]=>
    int(42)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(8) "Reginald"
  }
}
```

mysql_xdevapi\CollectionModify

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\CollectionModify** implements mysql_xdevapi\Executable, mysql_xdevapi\CrudOperationBindable, mysql_xdevapi\CrudOperationLimitable, mysql_xdevapi\CrudOperationSkippable, mysql_xdevapi\CrudOperationSortable {
/* Métodos */

public function **arrayAppend**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify
public function **arrayInsert**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify
public function **bind**(array `$placeholder_values`): mysql_xdevapi\CollectionModify
public function **execute**(): mysql_xdevapi\Result
public function **limit**(int `$rows`): mysql_xdevapi\CollectionModify
public function **patch**(string `$document`): mysql_xdevapi\CollectionModify
public function **replace**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify
public function **set**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify
public function **skip**(int `$position`): mysql_xdevapi\CollectionModify
public function **sort**(string `$sort_expr`): mysql_xdevapi\CollectionModify
public function **unset**(array `$fields`): mysql_xdevapi\CollectionModify
}

Índice

CollectionModify::arrayAppend

CollectionModify::arrayAppend

(No version information available, might only be in Git)

CollectionModify::arrayAppend — Anexa elemento a um array de campos

Descrição

public function **mysql_xdevapi\CollectionModify::arrayAppend**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify

Adiciona um elemento a um array de campos do documento, já que múltiplos documentos de um campo são representados como um array. Diferentemente de arrayInsert(), arrayAppend() sempre anexa o novo elemento ao final do array, enquanto que arrayInsert() pode definir a localização.

Parâmetros

`collection_field`

O identificador do campo onde o novo elemento é inserido.

`expression_or_literal`

O novo elemento a inserir no final do array de campos do documento.

Valor Retornado

Um objeto CollectionModify que pode ser usado para executar o comando, ou para operações adicionais.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionModify::arrayAppend()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
```

```php
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection
  ->add(
  '{"name":   "Bernie",
    "traits": ["Friend", "Brother", "Human"]}')
  ->execute();

$collection
  ->modify("name in ('Bernie', 'Jane')")
  ->arrayAppend('traits', 'Happy')
  ->execute();

$result = $collection
  ->find()
  ->execute();

print_r($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b5361000000000000010c
            [name] => Bernie
            [traits] => Array
                (
                    [0] => Friend
                    [1] => Brother
                    [2] => Human
                    [3] => Happy
                )
        )
)
```

CollectionModify::arrayInsert

CollectionModify::arrayInsert

(No version information available, might only be in Git)

CollectionModify::arrayInsert — Insere elemento em um array de campos

Descrição

public function **mysql_xdevapi\CollectionModify::arrayInsert**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify

Adiciona um elemento a um campo do documento, já que múltiplos documentos de um campo são representados como um array. Diferentemente de **mysql_xdevapi\CollectionModify::arrayAppend()** este método permite especificar onde o novo elemento é inserido definindo que item o precede, enquanto que **mysql_xdevapi\CollectionModify::arrayAppend()** sempre anexa o novo elemento no final do array.

Parâmetros

`collection_field`

Identifica o item no array após o qual o novo elemento será inserido. O formato deste parâmetro é `FIELD_NAME[ INDEX ]` onde FIELD_NAME é o nome do campo do documento ao qual o elemento será adicionado, e INDEX é o índice do elemento dentro do campo.

O campo INDEX inicia em zero, portanto o primeiro item do array tem um índice de 0.

`expression_or_literal`

O novo elemento a inserir após FIELD_NAME[INDEX]

Valor Retornado

Um objeto CollectionModify que pode ser usado para executar o comando, ou para inserir operações adicionais.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionModify::arrayInsert()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection
  ->add(
  '{"name":   "Bernie",
    "traits": ["Friend", "Brother", "Human"]}')
  ->execute();

$collection
  ->modify("name in ('Bernie', 'Jane')")
  ->arrayInsert('traits[1]', 'Happy')
  ->execute();

$result = $collection
  ->find()
  ->execute();

print_r($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b5361000000000000010d
            [name] => Bernie
            [traits] => Array
                (
                    [0] => Friend
                    [1] => Happy
                    [2] => Brother
                    [3] => Human
                )
        )
)
```

CollectionModify::bind

CollectionModify::bind

(No version information available, might only be in Git)

CollectionModify::bind — Vincula valor ao marcador da consulta

Descrição

public function **mysql_xdevapi\CollectionModify::bind**(array `$placeholder_values`): mysql_xdevapi\CollectionModify

Vincula um parâmetro ao marcador na condição de pesquisa da operação de modificação.

O marcador tem a forma :NOME onde ':' é um prefixo comum que deve sempre existir antes de qualquer NOME de marcador. O método de vinculação aceita uma lista de marcadores se múltiplas entidades tiverem que ser substituídas na condição de pesquisa da operação de modificação.

Parâmetros

`placeholder_values`

> Valores de marcadores para substituir na condição de pesquisa. Valores múltiplos são permitidos e devem ser passados como um array de mapeamentos NOME_DO_MARCADOR=>VALOR_DO_MARCADOR.

Valor Retornado

Um objeto CollectionModify que pode ser usado para executar o comando, ou para inserir operações adicionais.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\CollectionModify::bind()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
```

```
$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection
  ->add(
  '{"name":   "Bernie",
    "traits": ["Friend", "Brother", "Human"]}')
  ->execute();

$collection
  ->modify("name = :name")
  ->bind(['name' => 'Bernie'])
  ->arrayAppend('traits', 'Happy')
  ->execute();

$result = $collection
  ->find()
  ->execute();

print_r($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b53610000000000000110
            [name] => Bernie
            [traits] => Array
                (
                    [0] => Friend
                    [1] => Brother
                    [2] => Human
                    [3] => Happy
                )
        )
)
```

CollectionModify::__construct

CollectionModify::__construct

(No version information available, might only be in Git)

CollectionModify::__construct — Construtor de CollectionModify

Descrição

private function **mysql_xdevapi\CollectionModify::__construct**()

Modifica (atualiza) uma coleção e é instanciada pelo método **mysql_xdevapi\Collection::modify()**.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionModify::__construct()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection
  ->add(
  '{"name":   "Bernie",
    "traits": ["Friend", "Brother", "Human"]}')
  ->execute();

$collection
  ->modify("name in ('Bernie', 'Jane')")
  ->arrayAppend('traits', 'Happy')
  ->execute();

$result = $collection
  ->find()
  ->execute();

print_r($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b536100000000000010c
            [name] => Bernie
            [traits] => Array
                (
                    [0] => Friend
                    [1] => Brother
                    [2] => Human
                    [3] => Happy
                )
        )
)
```

CollectionModify::execute

CollectionModify::execute

(No version information available, might only be in Git)

CollectionModify::execute — Executa operação de modificação

Descrição

public function **mysql_xdevapi\CollectionModify::execute**(): mysql_xdevapi\Result

O método de execução é requerido para enviar a requisição de operação CRUD ao servidor MySQL.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto Result que pode ser usado para verificar o status da operação, como o número de linhas afetadas.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionModify::execute()

```
<?php

/* ... */

?>
```

CollectionModify::limit

CollectionModify::limit

(No version information available, might only be in Git)

CollectionModify::limit — Limita o número de documentos modificados

Descrição

public function **mysql_xdevapi\CollectionModify::limit**(int `$rows`): mysql_xdevapi\CollectionModify

Limita o número de documentos modificados por esta operação. Opcionalmente pode ser combinada com skip() para definir um valor de deslocamento.

Parâmetros

`rows`

O número máximo de documentos a modificar.

Valor Retornado

Um objeto CollectionModify.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\CollectionModify::limit()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$collection->add('{"name": "Fred",  "age": 21, "job": "Construction"}')->execute();
$collection->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')->execute();
$collection->add('{"name": "Betty", "age": 24, "job": "Teacher"}')->execute();

$collection
  ->modify("job = :job")
  ->bind(['job' => 'Teacher'])
  ->set('job', 'Principal')
  ->limit(1)
  ->execute();

$result = $collection
  ->find()
  ->execute();

print_r($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b53610000000000000118
            [age] => 21
            [job] => Construction
            [name] => Fred
        )
    [1] => Array
        (
            [_id] => 00005b6b53610000000000000119
            [age] => 23
            [job] => Principal
            [name] => Wilma
        )
    [2] => Array
        (
            [_id] => 00005b6b5361000000000000011a
            [age] => 24
            [job] => Teacher
            [name] => Betty
```

```
        )
)
```

CollectionModify::patch

CollectionModify::patch

(No version information available, might only be in Git)

CollectionModify::patch — Corrige documento

Descrição

public function **mysql_xdevapi\CollectionModify::patch**(string `$document`): mysql_xdevapi\CollectionModify

Aplica um objeto de correção a um ou mais documentos, podendo atualizar várias propriedades do documento.

Parâmetros

`document`

Um documento com as propriedades a serem aplicadas aos documentos correspondentes.

Valor Retornado

Um objeto CollectionModify.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\CollectionModify::patch()**

```php
<?php

$res = $coll->modify('"Programmatore" IN job')->patch('{"Hobby" : "Programmare"}')->execute();

?>
```

CollectionModify::replace

CollectionModify::replace

(No version information available, might only be in Git)

CollectionModify::replace — Substitui campo de documento

Descrição

public function **mysql_xdevapi\CollectionModify::replace**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify

Substitui (atualiza) um valor de campo fornecido com um novo.

Parâmetros

`collection_field`

O caminho do documento do item a definir.

`expression_or_literal`

O valor a ser definido no atributo especificado.

Valor Retornado

Um objeto CollectionModify.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionModify::replace()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection
  ->add(
  '{"name":   "Bernie",
    "traits": ["Friend", "Brother", "Human"]}')
  ->execute();

$collection
  ->modify("name = :name")
  ->bind(['name' => 'Bernie'])
  ->replace("name", "Bern")
  ->execute();

$result = $collection
  ->find()
  ->execute();

print_r($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b536100000000000011b
            [name] => Bern
            [traits] => Array
                (
                    [0] => Friend
                    [1] => Brother
                    [2] => Human
                )
```

```
        )
)
```

CollectionModify::set

CollectionModify::set

(No version information available, might only be in Git)

CollectionModify::set — Define atributo de documento

Descrição

public function **mysql_xdevapi\CollectionModify::set**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify

Define ou atualiza atributos em documentos de uma coleção.

Parâmetros

`collection_field`

O caminho (nome) do documento do item a ser definido.

`expression_or_literal`

O valor para o qual será definido.

Valor Retornado

Um objeto CollectionModify.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionModify::set()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection
  ->add(
  '{"name":   "Bernie",
    "traits": ["Friend", "Brother", "Human"]}')
  ->execute();

$collection
  ->modify("name = :name")
  ->bind(['name' => 'Bernie'])
  ->set("name", "Bern")
  ->execute();
```

```php
$result = $collection
  ->find()
  ->execute();

print_r($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b53610000000000000111
            [name] => Bern
            [traits] => Array
                (
                    [0] => Friend
                    [1] => Brother
                    [2] => Human
                )
        )
)
```

CollectionModify::skip

CollectionModify::skip

(No version information available, might only be in Git)

CollectionModify::skip — Pula elementos

Descrição

public function **mysql_xdevapi\CollectionModify::skip**(int `$position`): mysql_xdevapi\CollectionModify

Pula os primeiros N elementos que seriam de outra forma retornados por uma operação de pesquisa. Se o número de elementos pulados for maior que o tamanho do conjunto de resultados, a operação de pesquisa retorna um conjunto vazio.

Parâmetros

`position`

Número de elementos a pular.

Valor Retornado

Um objeto CollectionModify para uso em processamento adicional.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionModify::skip()

```php
<?php

$coll->modify('age > :age')->sort('age desc')->unset(['age'])->bind(['age' => 20])->limit(4)->skip(1)->execute();

?>
```

CollectionModify::sort

CollectionModify::sort

(No version information available, might only be in Git)

CollectionModify::sort — Define os critérios de ordenação

Descrição

public function **mysql_xdevapi\CollectionModify::sort**(string `$sort_expr`): mysql_xdevapi\CollectionModify

Ordena o conjunto de resultados pelo campo selecionado no argumento sort_expr. As ordens permitidas são ASC (ascendente) ou DESC (descendente). Esta operação é equivalente à 'ORDER BY' do SQL e segue o mesmo conjunto de regras.

Parâmetros

`sort_expr`

> Uma ou mais expressões podem ser informadas. A avaliação é da esquerda para a direita e cada expressão precisa ser separada por uma vírgula.

Valor Retornado

Objeto CollectionModify que pode ser usado para processamentos adicionais.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionModify::sort()

```php
<?php

$res = $coll->modify('true')->sort('name desc', 'age asc')->limit(4)->set('Married', 'NO')->execute();

?>
```

CollectionModify::unset

CollectionModify::unset

(No version information available, might only be in Git)

CollectionModify::unset — Remove o valor de campos do documento

Descrição

public function **mysql_xdevapi\CollectionModify::unset**(array `$fields`): mysql_xdevapi\CollectionModify

Remove atributos de documentos em uma coleção.

Parâmetros

`fields`

Os atributos a remover dos documentos em uma coleção.

Valor Retornado

Objeto CollectionModify que pode ser usado para processamento adicional.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionModify::unset()

```
<?php

$res = $coll->modify('job like :job_name')->unset(["age", "name"])->bind(['job_name' => 'Plumber'])->execute();

?>
```

mysql_xdevapi\CollectionRemove

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\CollectionRemove** implements mysql_xdevapi\Executable, mysql_xdevapi\CrudOperationBindable, mysql_xdevapi\CrudOperationLimitable, mysql_xdevapi\CrudOperationSortable {
/* Métodos */
public function **bind**(array `$placeholder_values`): mysql_xdevapi\CollectionRemove
public function **execute**(): mysql_xdevapi\Result
public function **limit**(int `$rows`): mysql_xdevapi\CollectionRemove
public function **sort**(string `$sort_expr`): mysql_xdevapi\CollectionRemove
}

Índice

CollectionRemove::bind

CollectionRemove::bind

(No version information available, might only be in Git)

CollectionRemove::bind — Vincula valor ao marcador

Descrição

public function **mysql_xdevapi\CollectionRemove::bind**(array `$placeholder_values`): mysql_xdevapi\CollectionRemove

Vincula um parâmetro ao marcador na condição de pesquisa da operação de remoção.

O marcador tem a forma :NOME onde ':' é um prefixo comum que deve sempre existir antes de qualquer NOME de marcador. O método de vinculação aceita uma lista de marcadores se múltiplas entidades tiverem que ser substituídas na condição de pesquisa da operação de remoção.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`placeholder_values`

Valores de marcadores para substituir na condição de pesquisa. Valores múltiplos são permitidos e devem ser passados como um array de mapeamentos NOME_DO_MARCADOR=>VALOR_DO_MARCADOR.

Valor Retornado

Um objeto CollectionRemove que pode ser usado para executar o comando, ou para inserir operações adicionais.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionRemove::bind()

```
<?php

$res = $coll->remove('age > :age_from and age < :age_to')->bind(['age_from' => 20, 'age_to' => 50])->limit(7)->execute();

?>
```

CollectionRemove::__construct

CollectionRemove::__construct

(No version information available, might only be in Git)

CollectionRemove::__construct — Construtor de CollectionRemove

Descrição

private function **mysql_xdevapi\CollectionRemove::__construct**()

Remove documentos de coleções e é instanciado pelo método **mysql_xdevapi\Collection::remove()**.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Collection::remove()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
```

```php
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$collection->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$collection->add('{"name": "Bob",    "age": 19, "job": "Painter"}')->execute();

// Remove todos os 'Painters'
$collection
  ->remove("job in ('Painter')")
  ->execute();

// Remove o 'Butler' com maior 'age'
$collection
  ->remove("job in ('Butler')")
  ->sort('age desc')
  ->limit(1)
  ->execute();

// Remove registro com maior 'age'
$collection
  ->remove('true')
  ->sort('age desc')
  ->limit(1)
  ->execute();
?>
```

CollectionRemove::execute

CollectionRemove::execute

(No version information available, might only be in Git)

CollectionRemove::execute — Executa operação de remoção

Descrição

public function **mysql_xdevapi\CollectionRemove::execute**(): mysql_xdevapi\Result

O método de execução precisa ser chamado para disparar o envio da requisição de operação CRUD do cliente para o servidor MySQL.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Objeto Result.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionRemove::execute()

```
<?php

$res = $coll->remove('true')->sort('age desc')->limit(2)->execute();

?>
```

CollectionRemove::limit

CollectionRemove::limit

(No version information available, might only be in Git)

CollectionRemove::limit — Limita o número de documentos a remover

Descrição

public function **mysql_xdevapi\CollectionRemove::limit**(int `$rows`): mysql_xdevapi\CollectionRemove

Define o número máximo de documentos a remover.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`rows`

O número máximo de documentos a remover.

Valor Retornado

Retorna um objeto CollectionRemove que pode ser usado para executar o comando, ou para inserir operações adicionais.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionRemove::limit()

```
<?php

$res = $coll->remove('job in (\'Barista\', \'Programmatore\', \'Ballerino\', \'Programmatrice\')')->limit(5)->sort(['age desc', 'name asc'])->execute();

?>
```

CollectionRemove::sort

CollectionRemove::sort

(No version information available, might only be in Git)

CollectionRemove::sort — Define os critérios de ordenação

Descrição

public function **mysql_xdevapi\CollectionRemove::sort**(string `$sort_expr`): mysql_xdevapi\CollectionRemove

Ordena o conjunto de resultados pelo campo selecionado no argumento sort_expr. As ordens permitidas são ASC (ascendente) ou DESC (descendente). Esta operação é equivalente à 'ORDER BY' do SQL e segue o mesmo conjunto de regras.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`sort_expr`

Uma ou mais expressões podem ser informadas. A avaliação é da esquerda para a direita e cada expressão precisa ser separada por uma vírgula.

Valor Retornado

Um objeto CollectionRemove que pode ser usado para executar o comando, ou para processamentos adicionais.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CollectionRemove::sort()

```php
<?php

$res = $coll->remove('true')->sort('age desc')->limit(2)->execute();

?>
```

mysql_xdevapi\ColumnResult

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\ColumnResult** {
/* Métodos */
public function **getCharacterSetName**(): string
public function **getCollationName**(): string
public function **getColumnLabel**(): string
public function **getColumnName**(): string
public function **getFractionalDigits**(): int
public function **getLength**(): int
public function **getSchemaName**(): string
public function **getTableLabel**(): string
public function **getTableName**(): string
public function **getType**(): int
public function **isNumberSigned**(): int
public function **isPadded**(): int

}

Índice

ColumnResult::__construct

ColumnResult::__construct

(No version information available, might only be in Git)

ColumnResult::__construct — Construtor de ColumnResult

Descrição

private function **mysql_xdevapi\ColumnResult::__construct**()

Recupera os metadados de coluna, como seu conjunto de caracteres; é instanciado pelo método **mysql_xdevapi\RowResult::getColumns()**.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\ColumnResult::__construct()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS nonsense")->execute();
$session->sql("CREATE DATABASE nonsense")->execute();
$session->sql("CREATE TABLE nonsense.numbers (hello int, world float unsigned)")->execute();
$session->sql("INSERT INTO  nonsense.numbers values (42, 42)")->execute();

$schema = $session->getSchema("nonsense");
$table  = $schema->getTable("numbers");

$result1 = $table->select('hello','world')->execute();

// Retorna um array de objetos ColumnResult
$columns = $result1->getColumns();

foreach ($columns as $column) {
    echo "\nColuna de nome " , $column->getColumnLabel();
    echo " é do tipo "       , $column->getType();
    echo " e é ", ($column->isNumberSigned() === 0) ? "sem sinal." : "com sinal.";
}

// Alternativamente
$result2 = $session->sql("SELECT * FROM nonsense.numbers")->execute();

// Retorna um array de objetos FieldMetadata
print_r($result2->getColumns());
```

O exemplo acima produzirá algo semelhante a:

```
Coluna de nome hello é do tipo 19 e é com sinal.
Coluna de nome world é do tipo 4 e é sem sinal.

Array
(
    [0] => mysql_xdevapi\FieldMetadata Object
        (
            [type] => 1
            [type_name] => SINT
            [name] => hello
            [original_name] => hello
            [table] => numbers
            [original_table] => numbers
            [schema] => nonsense
            [catalog] => def
            [collation] => 0
            [fractional_digits] => 0
            [length] => 11
            [flags] => 0
            [content_type] => 0
        )
    [1] => mysql_xdevapi\FieldMetadata Object
        (
            [type] => 6
            [type_name] => FLOAT
            [name] => world
            [original_name] => world
            [table] => numbers
            [original_table] => numbers
            [schema] => nonsense
            [catalog] => def
            [collation] => 0
            [fractional_digits] => 31
            [length] => 12
            [flags] => 1
            [content_type] => 0
        )
)
```

ColumnResult::getCharacterSetName

ColumnResult::getCharacterSetName

(No version information available, might only be in Git)

ColumnResult::getCharacterSetName — Obtém o conjunto de caracteres

Descrição

public function **mysql_xdevapi\ColumnResult::getCharacterSetName**(): string

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\ColumnResult::getCharacterSetName()

```
<?php

/* ... */

?>
```

ColumnResult::getCollationName

ColumnResult::getCollationName

(No version information available, might only be in Git)

ColumnResult::getCollationName — Obtém o nome da colação

Descrição

public function **mysql_xdevapi\ColumnResult::getCollationName**(): string

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\ColumnResult::getCollationName()

```
<?php

/* ... */

?>
```

ColumnResult::getColumnLabel

ColumnResult::getColumnLabel

(No version information available, might only be in Git)

ColumnResult::getColumnLabel — Obtém o label da coluna

Descrição

public function **mysql_xdevapi\ColumnResult::getColumnLabel**(): string

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\ColumnResult::getColumnLabel()

```
<?php

/* ... */

?>
```

ColumnResult::getColumnName

ColumnResult::getColumnName

(No version information available, might only be in Git)

ColumnResult::getColumnName — Obtém o nome da coluna

Descrição

public function **mysql_xdevapi\ColumnResult::getColumnName**(): string

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\ColumnResult::getColumnName()

```
<?php

/* ... */

?>
```

ColumnResult::getFractionalDigits

ColumnResult::getFractionalDigits

(No version information available, might only be in Git)

ColumnResult::getFractionalDigits — Obtém o comprimento dos dígitos fracionários

Descrição

public function **mysql_xdevapi\ColumnResult::getFractionalDigits**(): int

Busca o número de dígitos fracionários para a coluna.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\ColumnResult::getFractionalDigits()

```
<?php

/* ... */

?>
```

ColumnResult::getLength

ColumnResult::getLength

(No version information available, might only be in Git)

ColumnResult::getLength — Obtém o comprimento do campo da coluna

Descrição

public function **mysql_xdevapi\ColumnResult::getLength**(): int

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\ColumnResult::getLength()

```
<?php

/* ... */

?>
```

ColumnResult::getSchemaName

ColumnResult::getSchemaName

(No version information available, might only be in Git)

ColumnResult::getSchemaName — Obtám o nome do esquema

Descrição

public function **mysql_xdevapi\ColumnResult::getSchemaName**(): string

Obtám o nome do esquema onde a coluna é armazenada.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\ColumnResult::getSchemaName()

```
<?php

/* ... */

?>
```

ColumnResult::getTableLabel

ColumnResult::getTableLabel

(No version information available, might only be in Git)

ColumnResult::getTableLabel — Obtém o label da tabela

Descrição

public function **mysql_xdevapi\ColumnResult::getTableLabel**(): string

> **Aviso**

> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\ColumnResult::getTableLabel()

```
<?php

/* ... */

?>
```

ColumnResult::getTableName

ColumnResult::getTableName

(No version information available, might only be in Git)

ColumnResult::getTableName — Obtém o nome da tabela

Descrição

public function **mysql_xdevapi\ColumnResult::getTableName**(): string

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Nome da tabela para a coluna.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\ColumnResult::getTableName()

```
<?php

/* ... */

?>
```

ColumnResult::getType

ColumnResult::getType

(No version information available, might only be in Git)

ColumnResult::getType — Obtém o tipo de coluna

Descrição

public function **mysql_xdevapi\ColumnResult::getType**(): int

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\ColumnResult::getType()

```
<?php

/* ... */

?>
```

ColumnResult::isNumberSigned

ColumnResult::isNumberSigned

(No version information available, might only be in Git)

ColumnResult::isNumberSigned — Verifica se é do tipo com sinal

Descrição

public function **mysql_xdevapi\ColumnResult::isNumberSigned**(): int

Recupera informação da coluna e é instanciado pelo método **mysql_xdevapi\RowResult::getColumns()**.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se a coluna informada tem um tipo com sinal.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\ColumnResult::isNumberSigned()

```php
<?php

/* ... */

?>
```

ColumnResult::isPadded

ColumnResult::isPadded

(No version information available, might only be in Git)

ColumnResult::isPadded — Verifica se é preenchida à esquerda

Descrição

public function **mysql_xdevapi\ColumnResult::isPadded**(): int

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se a coluna informada é preenchida à esquerda.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\ColumnResult::isPadded()

```php
<?php

/* ... */

?>
```

mysql_xdevapi\CrudOperationBindable

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\CrudOperationBindable** {
/* Métodos */
abstract public function **bind**(array `$placeholder_values`): mysql_xdevapi\CrudOperationBindable
}

Índice

CrudOperationBindable::bind

CrudOperationBindable::bind

(No version information available, might only be in Git)

CrudOperationBindable::bind — Vincula valor ao marcador

Descrição

abstract public function **mysql_xdevapi\CrudOperationBindable::bind**(array `$placeholder_values`): mysql_xdevapi\CrudOperationBindable

Vincula um valor a um marcador específico.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`placeholder_values`

Os nomes dos marcadores e os valores a vincular.

Valor Retornado

Um objeto CrudOperationBindable.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CrudOperationBindable::bind()

```
<?php

$res = $coll->modify('name like :name')->arrayInsert('job[0]', 'Calciatore')->bind(['name' =>
'ENTITY'])->execute();
$res = $table->delete()->orderby('age desc')->where('age < 20 and age > 12 and name != :name')-
>bind(['name' => 'Tierney'])->limit(2)->execute();

?>
```

mysql_xdevapi\CrudOperationLimitable

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\CrudOperationLimitable** {
/* Métodos */
abstract public function **limit**(int `$rows`): mysql_xdevapi\CrudOperationLimitable
}

Índice

CrudOperationLimitable::limit

CrudOperationLimitable::limit

(No version information available, might only be in Git)

CrudOperationLimitable::limit — Define o limite do resultado

Descrição

abstract public function **mysql_xdevapi\CrudOperationLimitable::limit**(int `$rows`): mysql_xdevapi\CrudOperationLimitable

Define o número máximo de registros ou documentos a retornar.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`rows`

O número máximo de registros ou documentos.

Valor Retornado

Um objeto CrudOperationLimitable.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CrudOperationLimitable::limit()

```
<?php

$res = $coll->find()->fields(['name as n','age as a','job as j'])->groupBy('j')->limit(11)->execute();
$res = $table->update()->set('age',69)->where('age > 15 and age < 22')->limit(4)->orderby(['age
asc','name desc'])->execute();

?>
```

mysql_xdevapi\CrudOperationSkippable

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\CrudOperationSkippable** {
/* Métodos */
abstract public function **skip**(int `$skip`): mysql_xdevapi\CrudOperationSkippable
}

Índice

CrudOperationSkippable::skip

CrudOperationSkippable::skip

(No version information available, might only be in Git)

CrudOperationSkippable::skip — Número de operações a pular

Descrição

abstract public function **mysql_xdevapi\CrudOperationSkippable::skip**(int `$skip`): mysql_xdevapi\CrudOperationSkippable

Pula este número de registros na operação retornada.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`skip`

Número de elementos a pular.

Valor Retornado

Um objeto CrudOperationSkippable.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CrudOperationSkippable::skip()

```
<?php

$res = $coll->find('job like \'Programmatore\'')->limit(1)->skip(3)->sort('age asc')->execute();

?>
```

mysql_xdevapi\CrudOperationSortable

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\CrudOperationSortable** {
/* Métodos */
abstract public function **sort**(string `$sort_expr`): mysql_xdevapi\CrudOperationSortable
}

Índice

CrudOperationSortable::sort

CrudOperationSortable::sort

(No version information available, might only be in Git)

CrudOperationSortable::sort — Ordena resultados

Descrição

abstract public function **mysql_xdevapi\CrudOperationSortable::sort**(string `$sort_expr`): mysql_xdevapi\CrudOperationSortable

Ordena o conjunto de resultados pelo campo selecionado no argumento sort_expr. As ordens permitidas são ASC (ascendente) ou DESC (descendente). Esta operação é equivalente à 'ORDER BY' do SQL e segue o mesmo conjunto de regras.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`sort_expr`

Uma ou mais expressões podem ser informadas. A avaliação é da esquerda para a direita e cada expressão precisa ser separada por uma vírgula.

Valor Retornado

Um objeto CrudOperationSortable.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\CrudOperationSortable::sort()

```
<?php

$res = $coll->find('job like \'Cavia\'')->sort('age desc', '_id desc')->execute();

?>
```

mysql_xdevapi\DatabaseObject

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\DatabaseObject** {
/* Métodos */
abstract public function **existsInDatabase**(): bool
abstract public function **getName**(): string
abstract public function **getSession**(): mysql_xdevapi\Session
}

Índice

DatabaseObject::existsInDatabase

DatabaseObject::existsInDatabase

(No version information available, might only be in Git)

DatabaseObject::existsInDatabase — Verifica se um objeto existe em um banco de dados

Descrição

abstract public function **mysql_xdevapi\DatabaseObject::existsInDatabase**(): bool

Verifica se o objeto se refere a um que exista no banco de dados.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se o objeto existe no banco de dados ou `false` se não existe.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\DatabaseObject::existsInDatabase()

```php
<?php

$existInDb = $dbObj->existsInDatabase();

?>
```

DatabaseObject::getName

DatabaseObject::getName

(No version information available, might only be in Git)

DatabaseObject::getName — Obtém o nome do objeto

Descrição

abstract public function **mysql_xdevapi\DatabaseObject::getName**(): string

Busca o nome deste objeto de banco de dados.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O nome deste objeto de banco de dados.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\DatabaseObject::getName()

```php
<?php

$dbObjName = $dbObj->getName();

?>
```

DatabaseObject::getSession

DatabaseObject::getSession

(No version information available, might only be in Git)

DatabaseObject::getSession — Obtém o nome da sessão

Descrição

abstract public function **mysql_xdevapi\DatabaseObject::getSession**(): mysql_xdevapi\Session

Busca a sessão associada ao objeto de banco de dados.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O objeto Session.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\DatabaseObject::getSession()

```php
<?php

$session = $dbObj->getSession();

?>
```

mysql_xdevapi\DocResult

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\DocResult** implements mysql_xdevapi\BaseResult, Traversable {
/* Métodos */
public function **fetchAll**(): array
public function **fetchOne**(): array

public function **getWarnings**(): Array
public function **getWarningsCount**(): int
}

Índice

DocResult::__construct

DocResult::__construct

(No version information available, might only be in Git)

DocResult::__construct — Construtor de DocResult

Descrição

private function **mysql_xdevapi\DocResult::__construct**()

Busca os resultados de documentos e alertas, e é instanciado por CollectionFind.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Um exemplo de DocResult

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$create->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$create->add('{"name": "Reginald", "age": 42, "job": "Butler"}')->execute();

// ...

$collection = $schema->getCollection("people");

// Gera um objeto DocResult
$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->sort('age desc')
  ->limit(1)
  ->execute();

var_dump($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1) {
  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000f3"
    ["age"]=>
    int(42)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(8) "Reginald"
  }
}
```

DocResult::fetchAll

DocResult::fetchAll

(No version information available, might only be in Git)

DocResult::fetchAll — Obtém todas as linhas

Descrição

public function **mysql_xdevapi\DocResult::fetchAll**(): array

Busca todos os resultados de um conjunto.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array numérico com todos os resultados da consulta; cada resultado é um array associativo. Um array vazio é retornado se nenhuma linha estiver presente.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\DocResult::fetchAll()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$create->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$create->add('{"name": "Reginald", "age": 42, "job": "Butler"}')->execute();

// ...

$collection = $schema->getCollection("people");
```

```php
// Gera um objeto DocResult
$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->sort('age desc')
  ->execute();

var_dump($result->fetchAll());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(2) {

  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000123"
    ["age"]=>
    int(42)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(8) "Reginald"
  }

  [1]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000122"
    ["age"]=>
    int(18)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(6) "Alfred"
  }

}
```

DocResult::fetchOne

DocResult::fetchOne

(No version information available, might only be in Git)

DocResult::fetchOne — Obtém uma linha

Descrição

public function **mysql_xdevapi\DocResult::fetchOne**(): array

Busca um resultado de um conjunto de resultados.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O resultado, como um array associativo ou `null` se nenhum resultado estiver presente.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\DocResult::fetchOne()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$create->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$create->add('{"name": "Reginald", "age": 42, "job": "Butler"}')->execute();

// ...

$collection = $schema->getCollection("people");

// Gera um objeto DocResult
$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->sort('age desc')
  ->execute();

var_dump($result->fetchOne());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(4) {
  ["_id"]=>
  string(28) "00005b6b536100000000000000125"
  ["age"]=>
  int(42)
  ["job"]=>
  string(6) "Butler"
  ["name"]=>
  string(8) "Reginald"
}
```

DocResult::getWarnings

DocResult::getWarnings

(No version information available, might only be in Git)

DocResult::getWarnings — Obtém alertas da última operação

Descrição

public function **mysql_xdevapi\DocResult::getWarnings**(): Array

Busca alertas gerados pela última operação do servidor MySQL.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array contendo objetos Warning da última operação. Cada objeto define uma mensagem de erro ('message'), um nível de erro ('level') e um código de erro ('code'). Um array vazio é retornado se nenhum erro tiver ocorrido.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\DocResult::getWarnings()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$create->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$create->add('{"name": "Reginald", "age": 42, "job": "Butler"}')->execute();

// ...

$collection = $schema->getCollection("people");

// Gera um objeto DocResult
$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->sort('age desc')
  ->execute();

if (!$result->getWarningsCount()) {
    echo "Ocorreu um erro:\n";
    print_r($result->getWarnings());
    exit;
}

var_dump($result->fetchOne());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Ocorreu um erro:

Array
(
    [0] => mysql_xdevapi\Warning Object
        (
            [message] => Algo de errado
            [level] => 2
            [code] => 1365
        )
    [1] => mysql_xdevapi\Warning Object
        (
            [message] => Algo de errado
            [level] => 2
            [code] => 1365
        )
)
```

DocResult::getWarningsCount

DocResult::getWarningsCount

(No version information available, might only be in Git)

DocResult::getWarningsCount — Obtém a contagem de alertas da última operação

Descrição

public function **mysql_xdevapi\DocResult::getWarningsCount**(): int

Retorna o número de alertas gerados pela última operação. Especificamente, estes alertas são gerados pelo servidor MySQL.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número de alertas gerados pela última operação.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\DocResult::getWarningsCount()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
```

```php
$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$create->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$create->add('{"name": "Reginald", "age": 42, "job": "Butler"}')->execute();

// ...

$collection = $schema->getCollection("people");

// Gera um objeto DocResult
$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->sort('age desc')
  ->execute();

if (!$result->getWarningsCount()) {
    echo "Ocorreu um erro:\n";
    print_r($result->getWarnings());
    exit;
}

var_dump($result->fetchOne());
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(4) {
  ["_id"]=>
  string(28) "00005b6b536100000000000000135"
  ["age"]=>
  int(42)
  ["job"]=>
  string(6) "Butler"
  ["name"]=>
  string(8) "Reginald"
}
```

mysql_xdevapi\Exception

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\Exception** extends RuntimeException implements Throwable {
}

mysql_xdevapi\Executable

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\Executable** {
/* Métodos */
abstract public function **execute**(): mysql_xdevapi\Result
}

Índice

Executable::execute

Executable::execute

(No version information available, might only be in Git)

Executable::execute — Executa instrução

Descrição

abstract public function **mysql_xdevapi\Executable::execute**(): mysql_xdevapi\Result

Executa a instrução a partir de uma operação de coleção ou uma consulta de tabela; esta funcionalidade permite encadeamento de métodos.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um dos objetos Result, tal como Result ou SqlStatementResult.

Examples

Exemplo #1 Exemplos de execute()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$result_sql = $session->sql("CREATE DATABASE addressbook")->execute();

var_dump($result_sql);


$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("humans");

$result_collection = $collection->add(
  '{"name": "Jane",
    "jobs": [{"title":"Scientist","Salary":18000}, {"title":"Mother","Salary":0}],
    "hobbies": ["Walking","Making pies"]}');

$result_collection_executed = $result_collection->execute();
```

```
var_dump($result_collection);
var_dump($result_collection_executed);
?>
```

O exemplo acima produzirá algo semelhante a:

```
object(mysql_xdevapi\SqlStatementResult)#3 (0) {
}

object(mysql_xdevapi\CollectionAdd)#5 (0) {
}

object(mysql_xdevapi\Result)#7 (0) {
}
```

mysql_xdevapi\ExecutionStatus

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\ExecutionStatus** {
/* Propriedades */
public *$affectedItems*;
public *$matchedItems*;
public *$foundItems*;
public *$lastInsertId*;
public *$lastDocumentId*;
/* Constructor */
private function **__construct**()
}

Propriedades

affectedItems

matchedItems

foundItems

lastInsertId

lastDocumentId

Índice

ExecutionStatus::__construct

ExecutionStatus::__construct

(No version information available, might only be in Git)

ExecutionStatus::__construct — Construtor de ExecutionStatus

Descrição

private function **mysql_xdevapi\ExecutionStatus::__construct**()

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\ExecutionStatus::__construct()

```
<?php

/* ... */

?>
```

mysql_xdevapi\Expression

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\Expression** {
/* Propriedades */
public *$name*;
/* Constructor */
public function **__construct**(string `$expression`)
}

Propriedades

name

Índice

Expression::__construct

Expression::__construct

(No version information available, might only be in Git)

Expression::__construct — Construtor de Expression

Descrição

public function **mysql_xdevapi\Expression::__construct**(string `$expression`)

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`expression`

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Expression::__construct()

```
<?php

/* ... */

?>
```

mysql_xdevapi\Result

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\Result** implements mysql_xdevapi\BaseResult, Traversable {
/* Métodos */
public function **getAffectedItemsCount**(): int
public function **getAutoIncrementValue**(): int
public function **getGeneratedIds**(): array
public function **getWarnings**(): array
public function **getWarningsCount**(): int
}

Índice

Result::__construct

Result::__construct

(No version information available, might only be in Git)

Result::__construct — Construtor de Result

Descrição

private function **mysql_xdevapi\Result::__construct**()

Um objeto que recupera IDs geradas, valores de AUTO_INCREMENT e alertas para um conjuntos de resultados.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Result::__construct()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("
  CREATE TABLE addressbook.names
    (id INT NOT NULL AUTO_INCREMENT, name VARCHAR(30), age INT, PRIMARY KEY (id))
  ")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->insert("name", "age")->values(["Suzanne", 31],["Julie", 43])->execute();
$result = $table->insert("name", "age")->values(["Suki", 34])->execute();

$ai = $result->getAutoIncrementValue();
var_dump($ai);
?>
```

O exemplo acima produzirá:

```
int(3)
```

Result::getAffectedItemsCount

Result::getAffectedItemsCount

(No version information available, might only be in Git)

Result::getAffectedItemsCount — Obtém contagem de linhas afetadas

Descrição

public function **mysql_xdevapi\Result::getAffectedItemsCount**(): int

Obtém o número de linhas afetadas pela operação anterior.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número (como um inteiro) de linhas afetadas.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Result::getAffectedItemsCount()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$collection = $schema->getCollection("people");

$result = $collection->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')->execute();

var_dump( $res->getAffectedItemsCount() );
?>
```

O exemplo acima produzirá:

```
int(1)
```

Result::getAutoIncrementValue

Result::getAutoIncrementValue

(No version information available, might only be in Git)

Result::getAutoIncrementValue — Obtém valor de incremento automático

Descrição

public function **mysql_xdevapi\Result::getAutoIncrementValue**(): int

Obtém o último valor de AUTO_INCREMENT (último id de inserção).

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O último valor de AUTO_INCREMENT.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Result::getAutoIncrementValue()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
```

```php
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("
  CREATE TABLE addressbook.names
    (id INT NOT NULL AUTO_INCREMENT, name VARCHAR(30), age INT, PRIMARY KEY (id))
  ")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->insert("name", "age")->values(["Suzanne", 31],["Julie", 43])->execute();
$result = $table->insert("name", "age")->values(["Suki", 34])->execute();

$ai = $result->getAutoIncrementValue();
var_dump($ai);
?>
```

O exemplo acima produzirá:

```
int(3)
```

Result::getGeneratedIds

Result::getGeneratedIds

(No version information available, might only be in Git)

Result::getGeneratedIds — Obtém os IDs gerados

Descrição

public function **mysql_xdevapi\Result::getGeneratedIds**(): array

Busca os valores de _id gerados na última operação. O campo de _id único é gerado pelo servidor MySQL.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array de IDs gerados na última operação, ou um array vazio se não houve nenhum.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Result::getGeneratedIds()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
```

```php
$create = $schema->createCollection("people");

$collection = $schema->getCollection("people");

$result = $collection->add(
  '{"name": "Bernie",
    "jobs": [{"title":"Cat Herder","Salary":42000}, {"title":"Father","Salary":0}],
    "hobbies": ["Sports","Making cupcakes"]}',
  '{"name": "Jane",
    "jobs": [{"title":"Scientist","Salary":18000}, {"title":"Mother","Salary":0}],
    "hobbies": ["Walking","Making pies"]}')->execute();

$ids = $result->getGeneratedIds();
var_dump($ids);
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(2) {
  [0]=>
  string(28) "00005b6b53610000000000000064"
  [1]=>
  string(28) "00005b6b53610000000000000065"
}
```

Result::getWarnings

Result::getWarnings

(No version information available, might only be in Git)

Result::getWarnings — Obtém alertas da última operação

Descrição

public function **mysql_xdevapi\Result::getWarnings**(): array

Recupera alertas da última operação de Result.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array com objetos Warning da última operação. Cada objeto define uma mensagem de erro (chave 'message'), um nível de erro (chave 'level') e um código de erro (chave 'code'). Um array vazio é retornado se nenhum erro estiver presente.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\RowResult::getWarnings()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();

$schema = $session->getSchema("foo");
$table  = $schema->getTable("test_table");

$table->insert(['x'])->values([1])->values([2])->execute();

$res = $table->select(['x/0 as bad_x'])->execute();
$warnings = $res->getWarnings();

print_r($warnings);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => mysql_xdevapi\Warning Object
        (
            [message] => Division by 0
            [level] => 2
            [code] => 1365
        )
    [1] => mysql_xdevapi\Warning Object
        (
            [message] => Division by 0
            [level] => 2
            [code] => 1365
        )
)
```

Result::getWarningsCount

Result::getWarningsCount

(No version information available, might only be in Git)

Result::getWarningsCount — Obtém contagem de alertas da última operação

Descrição

public function **mysql_xdevapi\Result::getWarningsCount**(): int

Obtém o número de alertas da última operação Result.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número de alertas gerados pela última operação.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\RowResult::getWarningsCount()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS foo")->execute();
$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();

$schema = $session->getSchema("foo");
$table  = $schema->getTable("test_table");

$table->insert(['x'])->values([1])->values([2])->execute();

$res = $table->select(['x/0 as bad_x'])->execute();

echo $res->getWarningsCount();
?>
```

O exemplo acima produzirá algo semelhante a:

```
2
```

mysql_xdevapi\RowResult

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\RowResult** implements mysql_xdevapi\BaseResult, Traversable {
/* Métodos */
public function **fetchAll**(): array
public function **fetchOne**(): array
public function **getColumnsCount**(): int
public function **getColumnNames**(): array
public function **getColumns**(): array
public function **getWarnings**(): array
public function **getWarningsCount**(): int
}

Índice

RowResult::__construct

RowResult::__construct

(No version information available, might only be in Git)

RowResult::__construct — Construtor de RowResult

Descrição

private function **mysql_xdevapi\RowResult::__construct**()

Representa o conjunto de resultados obtido a partir de consulta ao banco de dados.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\RowResult::__construct()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$row = $table->select('name', 'age')->where('age > 18')->execute()->fetchAll();

print_r($row);
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
    [1] => Array
        (
            [name] => Sam
            [age] => 33
        )
)
```

RowResult::fetchAll

RowResult::fetchAll

(No version information available, might only be in Git)

RowResult::fetchAll — Obtém todas as linhas do resultado

Descrição

public function **mysql_xdevapi\RowResult::fetchAll**(): array

Busca todas as linhas do conjunto de resultados.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array numérico com todos os resultados da consulta; cada resultado é um array associativo. Um array vazio é retornado se nenhuma linha estiver presente.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\RowResult::fetchAll()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$row = $table->select('name', 'age')->execute()->fetchAll();

print_r($row);
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
    [1] => Array
        (
            [name] => Sam
            [age] => 33
        )
)
```

RowResult::fetchOne

RowResult::fetchOne

(No version information available, might only be in Git)

RowResult::fetchOne — Obtém linha do resultado

Descrição

public function **mysql_xdevapi\RowResult::fetchOne**(): array

Busca um resultado do conjunto de resultados.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O resultado, como um array associativo ou `null` se nenhum resultado estiver presente.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\RowResult::fetchOne()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$row = $table->select('name', 'age')->where('age < 40')->execute()->fetchOne();

print_r($row);
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [name] => Sam
    [age] => 33
)
```

RowResult::getColumnsCount

RowResult::getColumnsCount

(No version information available, might only be in Git)

RowResult::getColumnsCount — Obtém a contagem de colunas

Descrição

public function **mysql_xdevapi\RowResult::getColumnsCount**(): int

Recupera a contagem de colunas presentes no conjunto de resultados.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número de colunas; 0 se não houver nenhuma.

Registro de Alterações

Versão	Descrição
8.0.14	Método renomeado de getColumnCount() para getColumnsCount().

Examples

Exemplo #1 Exemplo de mysql_xdevapi\RowResult::getColumnsCount()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE addressbook")->execute();
$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$sql = $session->sql("SELECT * from addressbook.names")->execute();

echo $sql->getColumnsCount();
```

O exemplo acima produzirá algo semelhante a:

```
2
```

RowResult::getColumnNames

RowResult::getColumnNames

(No version information available, might only be in Git)

RowResult::getColumnNames — Obtém o nome de todas as colunas

Descrição

public function **mysql_xdevapi\RowResult::getColumnNames**(): array

Recupera os nomes das colunas presentes no conjunto de resultados.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array numérico de nomes de colunas da tabela, ou um array vazio se o conjunto de resultados estiver vazio.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\RowResult::getColumnNames()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE addressbook")->execute();
$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$sql = $session->sql("SELECT * from addressbook.names")->execute();

$colnames = $sql->getColumnNames();

print_r($colnames);
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => name
    [1] => age
)
```

RowResult::getColumns

RowResult::getColumns

(No version information available, might only be in Git)

RowResult::getColumns — Obtém metadados de coluna

Descrição

public function **mysql_xdevapi\RowResult::getColumns**(): array

Recupera metadados das colunas presentes no conjunto de resultados.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array de objetos FieldMetadata representando as colunas no resultado, ou um array vazio se o conjunto de resultados estiver vazio.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\RowResult::getColumns()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE addressbook")->execute();
$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$sql = $session->sql("SELECT * from addressbook.names")->execute();

$cols = $sql->getColumns();

print_r($cols);
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => mysql_xdevapi\FieldMetadata Object
        (
            [type] => 7
            [type_name] => BYTES
            [name] => name
            [original_name] => name
            [table] => names
            [original_table] => names
```

```
            [schema] => addressbook
            [catalog] => def
            [collation] => 255
            [fractional_digits] => 0
            [length] => 65535
            [flags] => 0
            [content_type] => 0
        )
    [1] => mysql_xdevapi\FieldMetadata Object
        (
            [type] => 1
            [type_name] => SINT
            [name] => age
            [original_name] => age
            [table] => names
            [original_table] => names
            [schema] => addressbook
            [catalog] => def
            [collation] => 0
            [fractional_digits] => 0
            [length] => 11
            [flags] => 0
            [content_type] => 0
        )
)
```

RowResult::getWarnings

RowResult::getWarnings

(No version information available, might only be in Git)

RowResult::getWarnings — Obtém os alertas da última operação

Descrição

public function **mysql_xdevapi\RowResult::getWarnings**(): array

Recupera os alertas da última operação de RowResult.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array de objetos Warning da última operação. Cada objeto define uma mensagem de erro (chave 'message'), um nível de erro (chave 'level') e um código de erro (chave 'code'). Um array vazio é retornado se nenhum erro estiver presente.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\RowResult::getWarnings()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();

$schema = $session->getSchema("foo");
$table  = $schema->getTable("test_table");

$table->insert(['x'])->values([1])->values([2])->execute();

$res = $table->select(['x/0 as bad_x'])->execute();
$warnings = $res->getWarnings();

print_r($warnings);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => mysql_xdevapi\Warning Object
        (
            [message] => Division by 0
            [level] => 2
            [code] => 1365
        )
    [1] => mysql_xdevapi\Warning Object
        (
            [message] => Division by 0
            [level] => 2
            [code] => 1365
        )
)
```

RowResult::getWarningsCount

RowResult::getWarningsCount

(No version information available, might only be in Git)

RowResult::getWarningsCount — Obtém a contagem de alertas da última operação

Descrição

public function **mysql_xdevapi\RowResult::getWarningsCount**(): int

Recupera o número de alertas da última operação de RowResult.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número de alertas gerados pela última operação.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\RowResult::getWarningsCount()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS foo")->execute();
$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();

$schema = $session->getSchema("foo");
$table  = $schema->getTable("test_table");

$table->insert(['x'])->values([1])->values([2])->execute();

$res = $table->select(['x/0 as bad_x'])->execute();

echo $res->getWarningsCount();
?>
```

O exemplo acima produzirá algo semelhante a:

```
2
```

mysql_xdevapi\Schema

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\Schema** implements mysql_xdevapi\DatabaseObject {
/* Propriedades */
public *$name*;
/* Métodos */
public function **createCollection**(string `$name`, string `$validate = ?`): mysql_xdevapi\Collection
public function **dropCollection**(string `$collection_name`): bool
public function **existsInDatabase**(): bool
public function **getCollection**(string `$name`): mysql_xdevapi\Collection
public function **getCollectionAsTable**(string `$name`): mysql_xdevapi\Table
public function **getCollections**(): array
public function **getName**(): string
public function **getSession**(): mysql_xdevapi\Session
public function **getTable**(string `$name`): mysql_xdevapi\Table

public function **getTables**(): array
}

Propriedades

name

Índice

Schema::__construct

Schema::__construct

(No version information available, might only be in Git)

Schema::__construct — Construtor de Schema

Descrição

private function **mysql_xdevapi\Schema::__construct**()

O objeto Schema fornece acesso completo ao esquema (banco de dados).

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Schema::__construct()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS food")->execute();
$session->sql("CREATE DATABASE food")->execute();
$session->sql("CREATE TABLE food.fruit(name text, rating text)")->execute();

$schema = $session->getSchema("food");
$schema->createCollection("trees");

print_r($schema->gettables());
print_r($schema->getcollections());
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [fruit] => mysql_xdevapi\Table Object
        (
            [name] => fruit
        )
)
Array
(
    [trees] => mysql_xdevapi\Collection Object
```

```
        (
            [name] => trees
        )
)
```

Schema::createCollection

Schema::createCollection

(No version information available, might only be in Git)

Schema::createCollection — Adiciona coleção ao esquema

Descrição

public function **mysql_xdevapi\Schema::createCollection**(string `$name`, string `$validate = ?`): mysql_xdevapi\Collection

Cria uma coleção dentro do esquema.

Parâmetros

`name`

Nome da coleção.

`validate`

Definição de validação, como um objeto JSON.

Valor Retornado

O objeto Collection.

Registro de Alterações

Versão	Descrição
8.0.20	Adicionado o parâmetro opcional de validação.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Schema::createCollection()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS food")->execute();
$session->sql("CREATE DATABASE food")->execute();
$session->sql("CREATE TABLE food.fruit(name text, rating text)")->execute();

$schema = $session->getSchema("food");
$schema->createCollection("trees");
```

```
print_r($schema->gettables());
print_r($schema->getcollections());
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [fruit] => mysql_xdevapi\Table Object
        (
            [name] => fruit
        )
)
Array
(
    [trees] => mysql_xdevapi\Collection Object
        (
            [name] => trees
        )
)
```

Exemplo #2 Exemplo de mysql_xdevapi\Schema::createCollection()

```
<?php
 $collection = $schema->createCollection("minha_coleção", '{
    "validation": {
        "level": "strict",
        "schema": {
            "id": "http://json-schema.org/geo",
            "description": "Uma coordenada geográfica",
            "type": "object",
            "properties": {
                "latitude": {
                    "type": "number"
                },
                "longitude": {
                    "type": "number"
                }
            },
            "required": ["latitude", "longitude"]
        }
    }
}');
// Irá funcionar
$collection->add('{"latitude": 10, "longitude": 20}')->execute();

// Irá falhar, tipos inválidos (não são números)
$collection->add('{"latitude": "lat", "longitude": "long"}')->execute();
```

Schema::dropCollection

Schema::dropCollection

(No version information available, might only be in Git)

Schema::dropCollection — Remove coleção do esquema

Descrição

public function **mysql_xdevapi\Schema::dropCollection**(string `$collection_name`): bool

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`collection_name`

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Schema::dropCollection()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS food")->execute();
$session->sql("CREATE DATABASE food")->execute();
$session->sql("CREATE TABLE food.fruit(name text, rating text)")->execute();

$schema = $session->getSchema("food");

$schema->createCollection("trees");
$schema->dropCollection("trees");
$schema->createCollection("buildings");

print_r($schema->gettables());
print_r($schema->getcollections());
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [fruit] => mysql_xdevapi\Table Object
        (
            [name] => fruit
        )
)
Array
(
    [buildings] => mysql_xdevapi\Collection Object
        (
            [name] => buildings
        )
)
```

Schema::existsInDatabase

Schema::existsInDatabase

(No version information available, might only be in Git)

Schema::existsInDatabase — Verifica se existe no banco de dados

Descrição

public function **mysql_xdevapi\Schema::existsInDatabase**(): bool

Verifica se o objeto atual (esquema, tabela, coleção ou visualização) existe no objeto Schema.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se o esquema, tabela, coleção ou visualização ainda existirem no banco de dados, senão `false`.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Schema::existsInDatabase()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS food")->execute();
$session->sql("CREATE DATABASE food")->execute();
$session->sql("CREATE TABLE food.fruit(name text, rating text)")->execute();

$schema = $session->getSchema("food");
$schema->createCollection("trees");

// ...

$trees = $schema->getCollection("trees");

// ...

// Esta coleção ainda existe no banco de dados (esquema)?
if ($trees->existsInDatabase()) {
    echo "Sim, a coleção 'trees' ainda está presente.";
}
```

O exemplo acima produzirá algo semelhante a:

```
Sim, a coleção 'trees' ainda está presente.
```

Schema::getCollection

Schema::getCollection

(No version information available, might only be in Git)

Schema::getCollection — Obtém coleção do esquema

Descrição

public function **mysql_xdevapi\Schema::getCollection**(string `$name`): mysql_xdevapi\Collection

Obtém uma coleção do esquema.

Parâmetros

`name`

Nome da coleção a ser obtida.

Valor Retornado

O objeto Collection para a coleção selecionada.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Schema::getCollection()**

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS food")->execute();
$session->sql("CREATE DATABASE food")->execute();

$schema = $session->getSchema("food");
$schema->createCollection("trees");

// ...

$trees = $schema->getCollection("trees");

var_dump($trees);
```

O exemplo acima produzirá algo semelhante a:

```
object(mysql_xdevapi\Collection)#3 (1) {
  ["name"]=>
  string(5) "trees"
}
```

Schema::getCollectionAsTable

Schema::getCollectionAsTable

(No version information available, might only be in Git)

Schema::getCollectionAsTable — Obtém uma coleção como um objeto Table

Descrição

public function **mysql_xdevapi\Schema::getCollectionAsTable**(string `$name`): mysql_xdevapi\Table

Obtém uma coleção, porém como objeto Table ao invés de objeto Collection.

Parâmetros

`name`

Nome da coleção a ser instanciada como objeto Table.

Valor Retornado

Um objeto mysql_xdevapi\Table para a coleção.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Schema::getCollectionAsTable()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema  = $session->getSchema("addressbook");
$collect = $schema->createCollection("people");
$collect->add('{"name": "Fred",  "age": 21, "job": "Construction"}')->execute();
$collect->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')->execute();

$table      = $schema->getCollectionAsTable("people");
$collection = $schema->getCollection("people");

var_dump($table);
var_dump($collection);
```

O exemplo acima produzirá algo semelhante a:

```
object(mysql_xdevapi\Table)#4 (1) {
  ["name"]=>
  string(6) "people"
}

object(mysql_xdevapi\Collection)#5 (1) {
  ["name"]=>
  string(6) "people"
}
```

Schema::getCollections

Schema::getCollections

(No version information available, might only be in Git)

Schema::getCollections — Obtém todas as coleções do esquema

Descrição

public function **mysql_xdevapi\Schema::getCollections**(): array

Busca uma lista de coleções deste esquema.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Array de todas as coleções neste esquema, onde cada elemento tem o valor de um objeto Collection com o nome da coleção como chave.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Schema::getCollections()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema  = $session->getSchema("addressbook");
$collect = $schema->createCollection("people");
$collect->add('{"name": "Fred",  "age": 21, "job": "Construction"}')->execute();
$collect->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')->execute();

$collections = $schema->getCollections();
var_dump($collections);
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1) {
  ["people"]=>
  object(mysql_xdevapi\Collection)#4 (1) {
    ["name"]=>
    string(6) "people"
  }
}
```

Schema::getName

Schema::getName

(No version information available, might only be in Git)

Schema::getName — Obtém o nome do esquema

Descrição

public function **mysql_xdevapi\Schema::getName**(): string

Obtém o nome do esquema.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O nome do esquema conectado ao objeto Schema, como uma string.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Schema::getName()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema  = $session->getSchema("addressbook");

// ...

var_dump($schema->getName());
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(11) "addressbook"
```

Schema::getSession

Schema::getSession

(No version information available, might only be in Git)

Schema::getSession — Obtém a sessão do esquema

Descrição

public function **mysql_xdevapi\Schema::getSession**(): mysql_xdevapi\Session

Obtém um novo objeto Session a partir do objeto Schema.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto Session.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Schema::getSession()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema  = $session->getSchema("addressbook");

// ...

$newsession = $schema->getSession();

var_dump($session);
var_dump($newsession);
?>
```

O exemplo acima produzirá algo semelhante a:

```
object(mysql_xdevapi\Session)#1 (0) {
}

object(mysql_xdevapi\Session)#3 (0) {
}
```

Schema::getTable

Schema::getTable

(No version information available, might only be in Git)

Schema::getTable — Obtém tabela do esquema

Descrição

public function **mysql_xdevapi\Schema::getTable**(string `$name`): mysql_xdevapi\Table

Busca um objeto Table para a tabela informada do esquema.

Parâmetros

`name`

Nome da tabela.

Valor Retornado

Um objeto Table.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Schema::getTable()**

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$row = $table->select('name', 'age')->execute()->fetchAll();

print_r($row);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
    [1] => Array
        (
            [name] => Sam
            [age] => 33
        )
)
```

Schema::getTables

Schema::getTables

(No version information available, might only be in Git)

Schema::getTables — Obtém tabelas do esquema

Descrição

public function **mysql_xdevapi\Schema::getTables**(): array

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Array de todas as tabelas do esquema, onde cada elemento tem um objeto Table como valor e o nome da tabela como chave.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Schema::getTables()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$session->sql("CREATE TABLE addressbook.cities(name text, population int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('Portland', 639863), ('Seattle', 704352)")-
>execute();

$schema = $session->getSchema("addressbook");
$tables = $schema->getTables();

var_dump($tables);
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(2) {
  ["cities"]=>
  object(mysql_xdevapi\Table)#3 (1) {
    ["name"]=>
    string(6) "cities"
  }

  ["names"]=>
  object(mysql_xdevapi\Table)#4 (1) {
```

```
    ["name"]=>
    string(5) "names"
  }
}
```

mysql_xdevapi\SchemaObject

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\SchemaObject** implements mysql_xdevapi\DatabaseObject {
/* Métodos */
abstract function **getSchema**(): mysql_xdevapi\Schema
}

Índice

SchemaObject::getSchema

SchemaObject::getSchema

(No version information available, might only be in Git)

SchemaObject::getSchema — Obtém o objeto do esquema

Descrição

abstract function **mysql_xdevapi\SchemaObject::getSchema**(): mysql_xdevapi\Schema

Usado por outros objetos para recuperar um objeto de esquema.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O objeto Schema atual.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Session::getSchema()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$schema  = $session->getSchema("addressbook");

print_r($schema);
```

O exemplo acima produzirá algo semelhante a:

```
mysql_xdevapi\Schema Object
(
    [name] => addressbook
)
```

mysql_xdevapi\Session

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\Session** {
/* Métodos */
public function **close**(): bool
public function **commit**(): Object
public function **createSchema**(string `$schema_name`): mysql_xdevapi\Schema
public function **dropSchema**(string `$schema_name`): bool
public function **generateUUID**(): string
public function **getDefaultSchema**(): ?mysql_xdevapi\Schema
public function **getSchema**(string `$schema_name`): mysql_xdevapi\Schema
public function **getSchemas**(): array
public function **getServerVersion**(): int
public function **listClients**(): array
public function **quoteName**(string `$name`): string
public function **releaseSavepoint**(string `$name`): void
public function **rollback**(): void
public function **rollbackTo**(string `$name`): void
public function **setSavepoint**(string `$name = ?`): string
public function **sql**(string `$query`): mysql_xdevapi\SqlStatement
public function **startTransaction**(): void
}

Índice

Session::close

Session::close

(No version information available, might only be in Git)

Session::close — Fecha sessão

Descrição

public function **mysql_xdevapi\Session::close**(): bool

Fecha a sessão com o servidor.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se a sessão for fechada.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Session::close()**

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$session->close();
```

Session::commit

Session::commit

(PHP 4 >= 4.4.0, PHP 5, PHP 7, PHP 8)

Session::commit — Confirma a transação

Descrição

public function **mysql_xdevapi\Session::commit**(): Object

Confirma a transação.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto SqlStatementResult.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Session::commit()**

```
<?php
$session    = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$collection = $session->getSchema("addressbook")->getCollection("friends");

$session->startTransaction();

$collection->add('{"John":42, "Sam":33}')->execute();
$savepoint = $session->setSavepoint();

$session->commit();
$session->close();
```

Session::__construct

Session::__construct

(No version information available, might only be in Git)

Session::__construct — Construtor de Description

Descrição

private function **mysql_xdevapi\Session::__construct**()

Um objeto Session, iniciado por getSession().

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Session::__construct()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->close();
?>
```

Session::createSchema

Session::createSchema

(No version information available, might only be in Git)

Session::createSchema — Cria novo esquema

Descrição

public function **mysql_xdevapi\Session::createSchema**(string `$schema_name`): mysql_xdevapi\Schema

Cria um novo esquema.

Parâmetros

`schema_name`

Nome do esquema a ser criado.

Valor Retornado

Um objeto Schema em caso de sucesso e emite uma exceção em caso de falha.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Session::createSchema()

```
<?php
$uri  = 'mysqlx://happyuser:password@127.0.0.1:33060/';
$sess = mysql_xdevapi\getSession($uri);
```

```php
try {

    if ($schema = $sess->createSchema('fruit')) {
        echo "Info: Eu criei um esquema de nome 'fruit'\n";
    }

} catch (Exception $e) {

   echo $e->getMessage();

}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Info: Eu criei um esquema de nome 'fruit'
```

Session::dropSchema

Session::dropSchema

(No version information available, might only be in Git)

Session::dropSchema — Remove um esquema

Descrição

public function **mysql_xdevapi\Session::dropSchema**(string `$schema_name`): bool

Remove um esquema (banco de dados).

Parâmetros

`schema_name`

Nome do esquema a remover.

Valor Retornado

`true` se o esquema for removido ou `false` se ele não exisitir ou não puder ser removido.

Um erro de nível `E_WARNING` é gerado se o esquema não existir.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Session::dropSchema()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->dropSchema("addressbook");
```

```
$session->close();
?>
```

Session::generateUUID

Session::generateUUID

(No version information available, might only be in Git)

Session::generateUUID — Obtém novo UUID

Descrição

public function **mysql_xdevapi\Session::generateUUID**(): string

Gera um Identificador Único Universal (UUID) de acordo com a » RFC 4122.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O UUID; uma string com comprimento de 32 bytes.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Session::generateUuid()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$uuid = $session->generateUuid();

var_dump($uuid);
```

O exemplo acima produzirá algo semelhante a:

```
string(32) "484B18AC7980F8D4FE84613CDA5EE84B"
```

Session::getDefaultSchema

Session::getDefaultSchema

(No version information available, might only be in Git)

Session::getDefaultSchema — Obtém o nome de esquema padrão

Descrição

public function **mysql_xdevapi\Session::getDefaultSchema**(): ?mysql_xdevapi\Schema

Recupera o esquema padrão que é tipicamente definido na URI de conexão.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Esquema padrão definido pela conexão ou `null` se nenhum foi definido.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Session::getSchema()

```
<?php
$uri = "mysqlx://testuser:testpasswd@localhost:33160/testx?ssl-mode=disabled";
$session = mysql_xdevapi\getSession($uri);

$schema = $session->getDefaultSchema();
echo $schema->getName();
?>
```

O exemplo acima produzirá:

```
testx
```

Session::getSchema

Session::getSchema

(No version information available, might only be in Git)

Session::getSchema — Obtém um novo objeto de esquema

Descrição

public function **mysql_xdevapi\Session::getSchema**(string `$schema_name`): mysql_xdevapi\Schema

Um novo objeto Schema para o nome de esquema informado.

Parâmetros

`schema_name`

Nome do esquema (banco de dados) para buscar um objeto Schema.

Valor Retornado

Um objeto Schema.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Session::getSchema()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$schema  = $session->getSchema("addressbook");

print_r($schema);
```

O exemplo acima produzirá algo semelhante a:

```
mysql_xdevapi\Schema Object
(
    [name] => addressbook
)
```

Session::getSchemas

Session::getSchemas

(No version information available, might only be in Git)

Session::getSchemas — Obtém os esquemas

Descrição

public function **mysql_xdevapi\Session::getSchemas**(): array

Obtém os objetos de esquema para todos os esquemas disponíveis para a sessão.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array contendo objetos que representam todos os esquemas disponíveis para a sessão.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Session::getSchemas()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$schemas  = $session->getSchemas();

print_r($schemas);
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => mysql_xdevapi\Schema Object
        (
            [name] => addressbook
        )
    [1] => mysql_xdevapi\Schema Object
        (
            [name] => information_schema
        )
    ...
```

Session::getServerVersion

Session::getServerVersion

(No version information available, might only be in Git)

Session::getServerVersion — Obtém a versão do servidor

Descrição

public function **mysql_xdevapi\Session::getServerVersion**(): int

Recupera a versão do servidor MySQL para a sessão.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A versão do servidor MySQL para a sessão, como um inteiro tipo "80012".

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Session::getServerVersion()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$version = $session->getServerVersion();

var_dump($version);
```

O exemplo acima produzirá algo semelhante a:

```
int(80012)
```

Session::listClients

Session::listClients

(No version information available, might only be in Git)

Session::listClients — Obtém a lista de clientes

Descrição

public function **mysql_xdevapi\Session::listClients**(): array

Obtém uma lista de conexões de clientes ao servidor MySQL da sessão.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array contendo os clientes atualmente registrados. Os elementos do array são "client_id", "user", "host", e "sql_session".

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Session::listClients()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$ids = $session->listClients();

var_dump($ids);
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(1) {
  [0]=>
  array(4) {
    ["client_id"]=>
    int(61)
    ["user"]=>
    string(4) "root"
    ["host"]=>
    string(9) "localhost"
    ["sql_session"]=>
    int(72)
  }
}
```

Session::quoteName

Session::quoteName

(No version information available, might only be in Git)

Session::quoteName — Adiciona escape

Descrição

public function **mysql_xdevapi\Session::quoteName**(string `$name`): string

Uma função para escapar nomes e identificadores SQL. Escapa o identificador fornecido de acordo com as configurações da conexão atual. Esta função de escape não deve ser usada para escapar valores.

Parâmetros

`name`

A string a ser escapada.

Valor Retornado

A string escapada.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Session::quoteName()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$first = "MySQL's test";
var_dump($first);
var_dump($session->quoteName($first));

$second = 'Another `test` "like" `this`';
var_dump($second);
var_dump($session->quoteName($second));
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(12) "MySQL's test"
string(14) "`MySQL's test`"

string(28) "Another `test` "like" `this`"
string(34) "`Another ``test`` "like" ``this```"
```

Session::releaseSavepoint

Session::releaseSavepoint

(No version information available, might only be in Git)

Session::releaseSavepoint — Libera o ponto de gravação definido

Descrição

public function **mysql_xdevapi\Session::releaseSavepoint**(string `$name`): void

Libera um ponto de gravação previamente definido.

Parâmetros

`name`

Nome do ponto de gravação a liberar.

Valor Retornado

Um objeto SqlStatementResult.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Session::releaseSavepoint()

```php
<?php
$session    = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$collection = $session->getSchema("addressbook")->getCollection("friends");

$session->startTransaction();
$collection->add( '{"test1":1, "test2":2}' )->execute();

$savepoint = $session->setSavepoint();

$collection->add( '{"test3":3, "test4":4}' )->execute();

$session->releaseSavepoint($savepoint);
$session->rollback();
?>
```

Session::rollback

Session::rollback

(No version information available, might only be in Git)

Session::rollback — Reverte a transação

Descrição

public function **mysql_xdevapi\Session::rollback**(): void

Reverte a transação.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto SqlStatementResult.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Session::rollback()

```
<?php
$session    = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$collection = $session->getSchema("addressbook")->getCollection("names");

$session->startTransaction();
$collection->add( '{"test1":1, "test2":2}' )->execute();

$savepoint = $session->setSavepoint();

$collection->add( '{"test3":3, "test4":4}' )->execute();

$session->releaseSavepoint($savepoint);
$session->rollback();
?>
```

Session::rollbackTo

Session::rollbackTo

(No version information available, might only be in Git)

Session::rollbackTo — Reverte a transação para um ponto de gravação

Descrição

public function **mysql_xdevapi\Session::rollbackTo**(string `$name`): void

Reverte a transação para o ponto de gravação.

Parâmetros

`name`

Nome do ponto de gravação para o qual a reversão ocorrerá; insensível a maiúsculas/minúsculas.

Valor Retornado

Um objeto SqlStatementResult.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Session::rollbackTo()

```
<?php
$session    = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$collection = $session->getSchema("addressbook")->getCollection("names");

$session->startTransaction();
$collection->add( '{"test1":1, "test2":2}' )->execute();

$savepoint1 = $session->setSavepoint();

$collection->add( '{"test3":3, "test4":4}' )->execute();
```

```php
$savepoint2 = $session->setSavepoint();

$session->rollbackTo($savepoint1);
?>
```

Session::setSavepoint

Session::setSavepoint

(No version information available, might only be in Git)

Session::setSavepoint — Cria ponto de gravação

Descrição

public function **mysql_xdevapi\Session::setSavepoint**(string `$name` = ?): string

Cria um novo ponto de gravação para a transação.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`name`

O nome do ponto de gravação. O nome é gerado automaticamente se o parâmetro `name` opcional não for definido, como 'SAVEPOINT1', 'SAVEPOINT2' e assim por diante.

Valor Retornado

O nome do ponto de gravação.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Session::setSavepoint()**

```php
<?php
$session    = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$collection = $session->getSchema("addressbook")->getCollection("names");

$session->startTransaction();
$collection->add( '{"test1":1, "test2":2}' )->execute();

$savepoint = $session->setSavepoint();

$collection->add( '{"test3":3, "test4":4}' )->execute();

$session->releaseSavepoint($savepoint);
$session->rollback();
?>
```

Session::sql

Session::sql

(No version information available, might only be in Git)

Session::sql — Cria consulta SQL

Descrição

public function **mysql_xdevapi\Session::sql**(string `$query`): mysql_xdevapi\SqlStatement

Cria uma instrução SQL nativa. Marcadores são suportados usando a sintaxe nativa "?". Use o método `execute` para executar a instrução SQL.

Parâmetros

`query`

Instrução SQL a executar.

Valor Retornado

Um objeto SqlStatement.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Session::sql()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("CREATE DATABASE addressbook")->execute();
?>
```

Session::startTransaction

Session::startTransaction

(No version information available, might only be in Git)

Session::startTransaction — Inicia transação

Descrição

public function **mysql_xdevapi\Session::startTransaction**(): void

Inicia uma nova transação.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto SqlStatementResult.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Session::startTransaction()

```php
<?php
$session    = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$collection = $session->getSchema("addressbook")->getCollection("friends");

$session->startTransaction();
$collection->add( '{"test1":1, "test2":2}' )->execute();

$savepoint = $session->setSavepoint();

$collection->add( '{"test3":3, "test4":4}' )->execute();

$session->releaseSavepoint($savepoint);
$session->rollback();
?>
```

mysql_xdevapi\SqlStatement

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\SqlStatement** {
/* Constantes */
const int *EXECUTE_ASYNC* = 1;
const int *BUFFERED* = 2;
/* Propriedades */
public *$statement*;
/* Métodos */
public function **bind**(string `$param`): mysql_xdevapi\SqlStatement
public function **execute**(): mysql_xdevapi\Result
public function **getNextResult**(): mysql_xdevapi\Result
public function **getResult**(): mysql_xdevapi\Result
public function **hasMoreResults**(): bool
}

Propriedades

statement

Constantes predefinidas

`mysql_xdevapi\SqlStatement::EXECUTE_ASYNC`

`mysql_xdevapi\SqlStatement::BUFFERED`

Índice

SqlStatement::bind

SqlStatement::bind

(No version information available, might only be in Git)

SqlStatement::bind — Vincula parâmetros da instrução

Descrição

public function **mysql_xdevapi\SqlStatement::bind**(string `$param`): mysql_xdevapi\SqlStatement

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

`param`

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatement::bind()

```
<?php

/* ... */

?>
```

SqlStatement::__construct

SqlStatement::__construct

(No version information available, might only be in Git)

SqlStatement::__construct — Construtor de Description

Descrição

private function **mysql_xdevapi\SqlStatement::__construct**()

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatement::__construct()

```
<?php

/* ... */

?>
```

SqlStatement::execute

SqlStatement::execute

(No version information available, might only be in Git)

SqlStatement::execute — Executa a operação

Descrição

public function **mysql_xdevapi\SqlStatement::execute**(): mysql_xdevapi\Result

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatement::execute()

```
<?php

/* ... */

?>
```

SqlStatement::getNextResult

SqlStatement::getNextResult

(No version information available, might only be in Git)

SqlStatement::getNextResult — Obtém o próximo resultado

Descrição

public function **mysql_xdevapi\SqlStatement::getNextResult**(): mysql_xdevapi\Result

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatement::getNextResult()

```php
<?php

/* ... */

?>
```

SqlStatement::getResult

SqlStatement::getResult

(No version information available, might only be in Git)

SqlStatement::getResult — Obtém resultado

Descrição

public function **mysql_xdevapi\SqlStatement::getResult**(): mysql_xdevapi\Result

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\SqlStatement::getResult()**

```php
<?php

/* ... */

?>
```

SqlStatement::hasMoreResults

SqlStatement::hasMoreResults

(No version information available, might only be in Git)

SqlStatement::hasMoreResults — Verifica por mais resultados

Descrição

public function **mysql_xdevapi\SqlStatement::hasMoreResults**(): bool

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se o conjunto de resultados tiver mais objetos a serem buscados.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatement::hasMoreResults()

```
<?php

/* ... */

?>
```

mysql_xdevapi\SqlStatementResult

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\SqlStatementResult** implements mysql_xdevapi\BaseResult, Traversable {
/* Métodos */
public function **fetchAll**(): array
public function **fetchOne**(): array
public function **getAffectedItemsCount**(): int
public function **getColumnsCount**(): int
public function **getColumnNames**(): array
public function **getColumns**(): Array
public function **getGeneratedIds**(): array
public function **getLastInsertId**(): String
public function **getWarnings**(): array
public function **getWarningCounts**(): int
public function **hasData**(): bool
public function **nextResult**(): mysql_xdevapi\Result
}

Índice

SqlStatementResult::__construct

SqlStatementResult::__construct

(No version information available, might only be in Git)

SqlStatementResult::__construct — Construtor de SqlStatementResult

Descrição

private function **mysql_xdevapi\SqlStatementResult::__construct**()

> **Aviso**

> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatementResult::__construct()

```
<?php

/* ... */

?>
```

SqlStatementResult::fetchAll

SqlStatementResult::fetchAll

(No version information available, might only be in Git)

SqlStatementResult::fetchAll — Obtém todas as linhas do resultado

Descrição

public function **mysql_xdevapi\SqlStatementResult::fetchAll**(): array

Busca todas as linhas do conjunto de resultados.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array numérico com todos os resultados da consulta; cada resultado é um array associativo. Um array vazio é retornado se nenhuma linha estiver presente.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatementResult::fetchAll()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS dbtest")->execute();
$session->sql("CREATE DATABASE dbtest")->execute();
$session->sql("CREATE TABLE dbtest.workers(name text, age int, job text)")->execute();
$session->sql("INSERT INTO dbtest.workers values ('John', 42, 'bricklayer'), ('Sam', 33,
'carpenter')")->execute();

$schema = $session->getSchema("dbtest");
```

```php
$table  = $schema->getTable("workers");

$rows = $session->sql("SELECT * FROM dbtest.workers")->execute()->fetchAll();

print_r($rows);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
    [1] => Array
        (
            [name] => Sam
            [age] => 33
        )
)
```

SqlStatementResult::fetchOne

SqlStatementResult::fetchOne

(No version information available, might only be in Git)

SqlStatementResult::fetchOne — Obtém uma única linha

Descrição

public function **mysql_xdevapi\SqlStatementResult::fetchOne**(): array

Busca uma linha do conjunto de resultados.

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O resultado, como um array associativo. Caso não haja resultado, será retornado null.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatementResult::fetchOne()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS dbtest")->execute();
$session->sql("CREATE DATABASE dbtest")->execute();
$session->sql("CREATE TABLE dbtest.workers(name text, age int, job text)")->execute();
$session->sql("INSERT INTO dbtest.workers values ('John', 42, 'bricklayer'), ('Sam', 33,
'carpenter')")->execute();

$schema = $session->getSchema("dbtest");
$table  = $schema->getTable("workers");

$rows = $session->sql("SELECT * FROM dbtest.workers")->execute()->fetchOne();

print_r($rows);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [name] => John
    [age] => 42
    [job] => bricklayer
)
```

SqlStatementResult::getAffectedItemsCount

SqlStatementResult::getAffectedItemsCount

(No version information available, might only be in Git)

SqlStatementResult::getAffectedItemsCount — Obtém a contagem de linhas afetadas

Descrição

public function **mysql_xdevapi\SqlStatementResult::getAffectedItemsCount**(): int

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatementResult::getAffectedItemsCount()

```
<?php

/* ... */

?>
```

SqlStatementResult::getColumnsCount

SqlStatementResult::getColumnsCount

(No version information available, might only be in Git)

SqlStatementResult::getColumnsCount — Obtém a contagem de colunas

Descrição

public function **mysql_xdevapi\SqlStatementResult::getColumnsCount**(): int

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número de colunas; 0 se não houver nenhuma.

Registro de Alterações

Versão	Descrição
8.0.14	Método renomeado de getColumnCount() para getColumnsCount().

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatementResult::getColumnsCount()

```
<?php

/* ... */

?>
```

SqlStatementResult::getColumnNames

SqlStatementResult::getColumnNames

(No version information available, might only be in Git)

SqlStatementResult::getColumnNames — Obtém nomes de colunas

Descrição

public function **mysql_xdevapi\SqlStatementResult::getColumnNames**(): array

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatementResult::getColumnNames()

```
<?php

/* ... */

?>
```

SqlStatementResult::getColumns

SqlStatementResult::getColumns

(No version information available, might only be in Git)

SqlStatementResult::getColumns — Obtém colunas

Descrição

public function **mysql_xdevapi\SqlStatementResult::getColumns**(): Array

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatementResult::getColumns()

```
<?php

/* ... */

?>
```

SqlStatementResult::getGeneratedIds

SqlStatementResult::getGeneratedIds

(No version information available, might only be in Git)

SqlStatementResult::getGeneratedIds — Obtém IDs gerados

Descrição

public function **mysql_xdevapi\SqlStatementResult::getGeneratedIds**(): array

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array de IDs gerados na última operação, ou um array vazio se não houver nenhum.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatementResult::getGeneratedIds()

```
<?php

/* ... */

?>
```

SqlStatementResult::getLastInsertId

SqlStatementResult::getLastInsertId

(No version information available, might only be in Git)

SqlStatementResult::getLastInsertId — Obtém o ID da última inserção

Descrição

public function **mysql_xdevapi\SqlStatementResult::getLastInsertId**(): String

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O ID para a última operação de inserção.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatementResult::getLastInsertId()

```
<?php

/* ... */

?>
```

SqlStatementResult::getWarnings

SqlStatementResult::getWarnings

(No version information available, might only be in Git)

SqlStatementResult::getWarnings — Obtém alertas da última operação

Descrição

public function **mysql_xdevapi\SqlStatementResult::getWarnings**(): array

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um array de objetos Warning da última operação. Cada objeto define uma mensagem de erro ('message'), nível de erro ('level') e código de erro ('code'). Um array vazio é retornado se nenhum erro estiver presente.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatementResult::getWarnings()

```
<?php

/* ... */

?>
```

SqlStatementResult::getWarningsCount

SqlStatementResult::getWarningsCount

(No version information available, might only be in Git)

SqlStatementResult::getWarningsCount — Obtém contagem de alertas da última operação

Descrição

public function **mysql_xdevapi\SqlStatementResult::getWarningCounts**(): int

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número de alertas emitidos durante a última operação CRUD.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatementResult::getWarningsCount()

```php
<?php

/* ... */

?>
```

SqlStatementResult::hasData

SqlStatementResult::hasData

(No version information available, might only be in Git)

SqlStatementResult::hasData — Verifica se o resultado tem dados

Descrição

public function **mysql_xdevapi\SqlStatementResult::hasData**(): bool

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se o conjunto de resultado tiver dados.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatementResult::hasData()

```php
<?php

/* ... */

?>
```

SqlStatementResult::nextResult

SqlStatementResult::nextResult

(No version information available, might only be in Git)

SqlStatementResult::nextResult — Obtém o próximo resultado

Descrição

public function **mysql_xdevapi\SqlStatementResult::nextResult**(): mysql_xdevapi\Result

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O próximo objeto Result da conjunto de resultados.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\SqlStatementResult::nextResult()

```
<?php

/* ... */

?>
```

mysql_xdevapi\Statement

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\Statement** {
/* Constantes */
const int *EXECUTE_ASYNC* = 1;
const int *BUFFERED* = 2;
/* Métodos */
public function **getNextResult**(): mysql_xdevapi\Result
public function **getResult**(): mysql_xdevapi\Result
public function **hasMoreResults**(): bool
}

Constantes predefinidas

mysql_xdevapi\Statement::EXECUTE_ASYNC

`mysql_xdevapi\Statement::BUFFERED`

Índice

Statement::__construct

Statement::__construct

(No version information available, might only be in Git)

Statement::__construct — Construtor de Statement

Descrição

private function **mysql_xdevapi\Statement::__construct**()

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Statement::__construct()

```
<?php

/* ... */

?>
```

Statement::getNextResult

Statement::getNextResult

(No version information available, might only be in Git)

Statement::getNextResult — Obtém o próximo resultado

Descrição

public function **mysql_xdevapi\Statement::getNextResult**(): mysql_xdevapi\Result

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Statement::getNextResult()

```
<?php

/* ... */

?>
```

Statement::getResult

Statement::getResult

(No version information available, might only be in Git)

Statement::getResult — Obtém resultado

Descrição

public function **mysql_xdevapi\Statement::getResult**(): mysql_xdevapi\Result

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Statement::getResult()

```
<?php

/* ... */

?>
```

Statement::hasMoreResults

Statement::hasMoreResults

(No version information available, might only be in Git)

Statement::hasMoreResults — Verifica se há mais resultados

Descrição

public function **mysql_xdevapi\Statement::hasMoreResults**(): bool

> **Aviso**

> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Statement::hasMoreResults()

```
<?php

/* ... */

?>
```

mysql_xdevapi\Table

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Fornece acesso à tabela através das instruções INSERT/SELECT/UPDATE/DELETE.

Resumo da classe

class **mysql_xdevapi\Table** implements mysql_xdevapi\SchemaObject {
/* Propriedades */
public *$name*;
/* Métodos */
public function **count**(): int
public function **delete**(): mysql_xdevapi\TableDelete
public function **existsInDatabase**(): bool
public function **getName**(): string
public function **getSchema**(): mysql_xdevapi\Schema
public function **getSession**(): mysql_xdevapi\Session
public function **insert**(mixed `$columns`, mixed `...$more_columns`): mysql_xdevapi\TableInsert
public function **isView**(): bool
public function **select**(mixed `$columns`, mixed `...$more_columns`): mysql_xdevapi\TableSelect
public function **update**(): mysql_xdevapi\TableUpdate
}

Propriedades

name

Índice

Table::__construct

Table::__construct

(No version information available, might only be in Git)

Table::__construct — Construtor de Table

Descrição

private function **mysql_xdevapi\Table::__construct**()

Constrói um objeto de tabela.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Table::__construct()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");
?>
```

Table::count

Table::count

(No version information available, might only be in Git)

Table::count — Obtém contagem de linhas

Descrição

public function **mysql_xdevapi\Table::count**(): int

Busca o número de linhas na tabela.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O número total de linhas na tabela.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Table::count()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();
```

```php
$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

var_dump($table->count());
?>
```

O exemplo acima produzirá:

```
int(2)
```

Table::delete

Table::delete

(No version information available, might only be in Git)

Table::delete — Deleta linhas da tabela

Descrição

public function **mysql_xdevapi\Table::delete**(): mysql_xdevapi\TableDelete

Deleta linhas da tabela.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto TableDelete; use o método execute() para executar a instrução de deleção.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Table::delete()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->delete()->where("name = :name")->orderby("age DESC")->limit(1)->bind(['name' => 'John'])-
>execute();
?>
```

Table::existsInDatabase

Table::existsInDatabase

(No version information available, might only be in Git)

Table::existsInDatabase — Verifica se uma tabela existe no banco de dados

Descrição

public function **mysql_xdevapi\Table::existsInDatabase**(): bool

Verifica se esta tabela existe no banco de dados.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se a tabela existir no banco de dados, ou `false` se não existir.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Table::existsInDatabase()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

if ($table->existsInDatabase()) {
  echo "Sim, esta tabela ainda existe no esquema da sessão.";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Sim, esta tabela ainda existe no esquema da sessão.
```

Table::getName

Table::getName

(No version information available, might only be in Git)

Table::getName — Obtém nome da tabela

Descrição

public function **mysql_xdevapi\Table::getName**(): string

Retorna o nome deste objeto de banco de dados.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

O nome deste objeto de banco de dados.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Table::getName()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

var_dump($table->getName());
?>
```

O exemplo acima produzirá algo semelhante a:

```
string(5) "names"
```

Table::getSchema

Table::getSchema

(No version information available, might only be in Git)

Table::getSchema — Obtém esquema da tabela

Descrição

public function **mysql_xdevapi\Table::getSchema**(): mysql_xdevapi\Schema

Busca o esquema associado com a tabela.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto Schema.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Table::getSchema()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

var_dump($table->getSchema());
?>
```

O exemplo acima produzirá algo semelhante a:

```
object(mysql_xdevapi\Schema)#9 (1) {
  ["name"]=>
  string(11) "addressbook"
}
```

Table::getSession

Table::getSession

(No version information available, might only be in Git)

Table::getSession — Obtém a sessão da tabela

Descrição

public function **mysql_xdevapi\Table::getSession**(): mysql_xdevapi\Session

Obtém a sessão associada com a tabela.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto Session.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Table::getSession()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

var_dump($table->getSession());
?>
```

O exemplo acima produzirá algo semelhante a:

```
object(mysql_xdevapi\Session)#9 (0) {
}
```

Table::insert

Table::insert

(No version information available, might only be in Git)

Table::insert — Insere linhas na tabela

Descrição

public function **mysql_xdevapi\Table::insert**(mixed `$columns`, mixed `...$more_columns`): mysql_xdevapi\TableInsert

Insere linhas na tabela.

Parâmetros

`columns`

As colunas para inserir os dados. Pode ser um array com um ou mais valores, ou uma string.

`more_columns`

Definições de colunas adicionais.

Valor Retornado

Um objeto TableInsert; use o método execute() para executar a instrução de inserção.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Table::insert()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table ->insert("name", "age")
  ->values(["Suzanne", 31],["Julie", 43])
  ->execute();
?>
```

Table::isView

Table::isView

(No version information available, might only be in Git)

Table::isView — Verifica se a tabela é uma visualização

Descrição

public function **mysql_xdevapi\Table::isView**(): bool

Determina se o objeto subjacente é ou não uma visualização.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

`true` se o objeto subjacente for uma visualização, caso contrário `false`.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Table::isView()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

if ($table->isView()) {
    echo "Isto é uma visualização.";
} else {
```

```
    echo "Isto não é uma visualização.";
}
?>
```

O exemplo acima produzirá:

```
Isto não é uma visualização.
```

Table::select

Table::select

(No version information available, might only be in Git)

Table::select — Seleciona linhas da tabela

Descrição

public function **mysql_xdevapi\Table::select**(mixed `$columns`, mixed `...$more_columns`): mysql_xdevapi\TableSelect

Busca dados de uma tabela.

Parâmetros

`columns`

As colunas das quais os dados serão selecionados. Pode ser um array com um ou mais valores, ou uma string.

`more_columns`

Definições de colunas adicionais.

Valor Retornado

Um objeto TableSelect; use o método execute() para executar a seleção e retornar um objeto RowResult.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Table::count()**

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");
```

```php
$row = $table->select('name', 'age')->execute()->fetchAll();

print_r($row);
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
    [1] => Array
        (
            [name] => Sam
            [age] => 33
        )
)
```

Table::update

Table::update

(No version information available, might only be in Git)

Table::update — Atualiza linhas na tabela

Descrição

public function **mysql_xdevapi\Table::update**(): mysql_xdevapi\TableUpdate

Atualiza linhas na tabela.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto TableUpdate; use o método execute() para executar a instrução de atualização.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\Table::update()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
```

```php
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->update()->set('age',34)->where('name = "Sam"')->limit(1)->execute();
?>
```

mysql_xdevapi\TableDelete

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Uma instrução para operações de deleção em uma tabela.

Resumo da classe

class **mysql_xdevapi\TableDelete** implements mysql_xdevapi\Executable {
/* Métodos */
public function **bind**(array `$placeholder_values`): mysql_xdevapi\TableDelete
public function **execute**(): mysql_xdevapi\Result
public function **limit**(int `$rows`): mysql_xdevapi\TableDelete
public function **orderby**(string `$orderby_expr`): mysql_xdevapi\TableDelete
public function **where**(string `$where_expr`): mysql_xdevapi\TableDelete
}

Índice

TableDelete::bind

TableDelete::bind

(No version information available, might only be in Git)

TableDelete::bind — Vincula parâmetros na instrução de deleção

Descrição

public function **mysql_xdevapi\TableDelete::bind**(array `$placeholder_values`): mysql_xdevapi\TableDelete

Vincula um valor a um marcador específico.

Parâmetros

`placeholder_values`

O nome do marcador e o valor a vincular.

Valor Retornado

Um objeto TableDelete.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableDelete::bind()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->delete()
  ->where("name = :name")
  ->bind(['name' => 'John'])
  ->orderby("age DESC")
  ->limit(1)
  ->execute();

?>
```

TableDelete::__construct

TableDelete::__construct

(No version information available, might only be in Git)

TableDelete::__construct — Construtor de TableDelete

Descrição

private function **mysql_xdevapi\TableDelete::__construct**()

Iniciado pelo uso do método delete().

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableDelete::__construct()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->delete()
  ->where("name = :name")
  ->bind(['name' => 'John'])
  ->orderby("age DESC")
  ->limit(1)
```

```
  ->execute();

?>
```

TableDelete::execute

TableDelete::execute

(No version information available, might only be in Git)

TableDelete::execute — Executa a instrução de deleção

Descrição

public function **mysql_xdevapi\TableDelete::execute**(): mysql_xdevapi\Result

Executa a instrução de deleção.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto Result.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\TableDelete::execute()**

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->delete()
  ->where("name = :name")
  ->bind(['name' => 'John'])
  ->orderby("age DESC")
  ->limit(1)
  ->execute();

?>
```

TableDelete::limit

TableDelete::limit

(No version information available, might only be in Git)

TableDelete::limit — Limita as linhas deletadas

Descrição

public function **mysql_xdevapi\TableDelete::limit**(int `$rows`): mysql_xdevapi\TableDelete

Define o número máximo de registros ou documentos a deletar.

Parâmetros

`rows`

O número máximo de registros ou documentos a deletar.

Valor Retornado

Objeto TableDelete.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableDelete::limit()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->delete()
  ->where("name = :name")
  ->bind(['name' => 'John'])
  ->orderby("age DESC")
  ->limit(1)
  ->execute();

?>
```

TableDelete::orderby

TableDelete::orderby

(No version information available, might only be in Git)

TableDelete::orderby — Define os critérios de ordenação na deleção

Descrição

public function **mysql_xdevapi\TableDelete::orderby**(string `$orderby_expr`): mysql_xdevapi\TableDelete

Define as opções de ordenação para um conjunto de resultados.

Parâmetros

`orderby_expr`

A definição de ordenação.

Valor Retornado

Um objeto TableDelete.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\TableDelete::orderBy()**

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->delete()
  ->where("age = :age")
  ->bind(['age' => 42])
  ->orderby("name DESC")
  ->limit(1)
  ->execute();

?>
```

TableDelete::where

TableDelete::where

(No version information available, might only be in Git)

TableDelete::where — Define condição de pesquisa na deleção

Descrição

public function **mysql_xdevapi\TableDelete::where**(string `$where_expr`): mysql_xdevapi\TableDelete

Define a condição de pesquisa para filtrar.

Parâmetros

`where_expr`

Define a condição de pesquisa para filtrar documentos ou registros.

Valor Retornado

Objeto TableDelete.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\TableDelete::where()**

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->delete()
```

```
  ->where("id = :id")
  ->bind(['id' => 42])
  ->limit(1)
  ->execute();

?>
```

mysql_xdevapi\TableInsert

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Uma instrução para operações de inserção em uma tabela.

Resumo da classe

class **mysql_xdevapi\TableInsert** implements mysql_xdevapi\Executable {
/* Métodos */
public function **execute**(): mysql_xdevapi\Result
public function **values**(array `$row_values`): mysql_xdevapi\TableInsert
}

Índice

TableInsert::__construct

TableInsert::__construct

(No version information available, might only be in Git)

TableInsert::__construct — Construtor de TableInsert

Descrição

private function **mysql_xdevapi\TableInsert::__construct**()

Iniciado pelo uso do método insert().

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableInsert::__construct()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");
```

```
$table
  ->insert("name", "age")
  ->values(["Suzanne", 31],["Julie", 43])
  ->execute();
?>
```

TableInsert::execute

TableInsert::execute

(No version information available, might only be in Git)

TableInsert::execute — Executa instrução de inserção

Descrição

public function **mysql_xdevapi\TableInsert::execute**(): mysql_xdevapi\Result

Executa a instrução.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto Result.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableInsert::execute()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table
  ->insert("name", "age")
  ->values(["Suzanne", 31],["Julie", 43])
  ->execute();
?>
```

TableInsert::values

TableInsert::values

(No version information available, might only be in Git)

TableInsert::values — Adiciona valores para inserção

Descrição

public function **mysql_xdevapi\TableInsert::values**(array `$row_values`): mysql_xdevapi\TableInsert

Define os valores para inserção.

Parâmetros

`row_values`

Valores (como array) de colunas para inserção.

Valor Retornado

Um objeto TableInsert.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\TableInsert::values()**

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table
  ->insert("name", "age")
  ->values(["Suzanne", 31],["Julie", 43])
  ->execute();
?>
```

mysql_xdevapi\TableSelect

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Uma instrução para operações de recuperação de registros em uma tabela.

Resumo da classe

class **mysql_xdevapi\TableSelect** implements mysql_xdevapi\Executable {
/* Métodos */
public function **bind**(array `$placeholder_values`): mysql_xdevapi\TableSelect
public function **execute**(): mysql_xdevapi\RowResult
public function **groupBy**(mixed `$sort_expr`): mysql_xdevapi\TableSelect
public function **having**(string `$sort_expr`): mysql_xdevapi\TableSelect
public function **limit**(int `$rows`): mysql_xdevapi\TableSelect
public function **lockExclusive**(int `$lock_waiting_option` = ?): mysql_xdevapi\TableSelect
public function **lockShared**(int `$lock_waiting_option` = ?): mysql_xdevapi\TableSelect
public function **offset**(int `$position`): mysql_xdevapi\TableSelect
public function **orderby**(mixed `$sort_expr`, mixed `...$sort_exprs`): mysql_xdevapi\TableSelect

public function **where**(string `$where_expr`): mysql_xdevapi\TableSelect
}

Índice

TableSelect::bind

TableSelect::bind

(No version information available, might only be in Git)

TableSelect::bind — Vincula parâmetros de consulta de seleção

Descrição

public function **mysql_xdevapi\TableSelect::bind**(array `$placeholder_values`): mysql_xdevapi\TableSelect

Vincula um valor a um marcador específico.

Parâmetros

`placeholder_values`

O nome do marcador, e o valor a vincular.

Valor Retornado

Um objeto TableSelect.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\TableSelect::bind()**

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('name','age')
  ->where('name like :name and age > :age')
  ->bind(['name' => 'John', 'age' => 42])
  ->execute();

$row = $result->fetchAll();
print_r($row);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
```

```
        )
)
```

TableSelect::__construct

TableSelect::__construct

(No version information available, might only be in Git)

TableSelect::__construct — Construtor TableSelect

Descrição

private function **mysql_xdevapi\TableSelect::__construct**()

Um objeto retornado pelo método select(); use execute() para executar a consulta.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableSelect::__construct()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('name','age')
  ->where('name like :name and age > :age')
  ->bind(['name' => 'John', 'age' => 42])
  ->orderBy('age desc')
  ->execute();

$row = $result->fetchAll();
print_r($row);

?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
```

```
        (
            [name] => John
            [age] => 42
        )
)
```

TableSelect::execute

TableSelect::execute

(No version information available, might only be in Git)

TableSelect::execute — Executa instrução de seleção

Descrição

public function **mysql_xdevapi\TableSelect::execute**(): mysql_xdevapi\RowResult

Executa a instrução de seleção encadeando-a com o método execute().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto RowResult.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableSelect::execute()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('name','age')
  ->where('name like :name and age > :age')
  ->bind(['name' => 'John', 'age' => 42])
  ->orderBy('age desc')
  ->execute();

$row = $result->fetchAll();
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
```

```
        (
            [name] => John
            [age] => 42
        )
)
```

TableSelect::groupBy

TableSelect::groupBy

(No version information available, might only be in Git)

TableSelect::groupBy — Define critérios de agrupamento de seleção

Descrição

public function **mysql_xdevapi\TableSelect::groupBy**(mixed `$sort_expr`): mysql_xdevapi\TableSelect

Define critérios de agrupamento para o conjunto de resultados.

Parâmetros

`sort_expr`

Os critérios de agrupamento.

Valor Retornado

Um objeto TableSelect.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\TableSelect::groupBy()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 42)")->execute();
$session->sql("INSERT INTO addressbook.names values ('Suki', 31)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('count(*) as count', 'age')
  ->groupBy('age')->orderBy('age asc')
  ->execute();

$row = $result->fetchAll();
print_r($row);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [count] => 1
            [age] => 31
        )
    [1] => Array
        (
            [count] => 2
            [age] => 42
        )
)
```

TableSelect::having

TableSelect::having

(No version information available, might only be in Git)

TableSelect::having — Definir condição para função de agregação na seleção

Descrição

public function **mysql_xdevapi\TableSelect::having**(string `$sort_expr`): mysql_xdevapi\TableSelect

Define uma condição para registros a serem considerados nas operações de funções de agregação.

Parâmetros

`sort_expr`

Uma condição nas funções de agregação usadas nos critérios de agrupamento.

Valor Retornado

Um objeto TableSelect.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\TableSelect::having()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 42)")->execute();
$session->sql("INSERT INTO addressbook.names values ('Suki', 31)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");
```

```php
$result = $table->select('count(*) as count', 'age')
  ->groupBy('age')->orderBy('age asc')
  ->having('count > 1')
  ->execute();

$row = $result->fetchAll();
print_r($row);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [count] => 2
            [age] => 42
        )
)
```

TableSelect::limit

TableSelect::limit

(No version information available, might only be in Git)

TableSelect::limit — Limita as linhas selecionadas

Descrição

public function **mysql_xdevapi\TableSelect::limit**(int `$rows`): mysql_xdevapi\TableSelect

Define o número máximo de registros ou documentos a retornar.

Parâmetros

`rows`

O número máximo de registros ou documentos.

Valor Retornado

Um objeto TableSelect.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableSelect::limit()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
```

```php
$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('name', 'age')
  ->limit(1)
  ->execute();

$row = $result->fetchAll();
print_r($row);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
)
```

TableSelect::lockExclusive

TableSelect::lockExclusive

(No version information available, might only be in Git)

TableSelect::lockExclusive — Executa EXCLUSIVE LOCK

Descrição

public function **mysql_xdevapi\TableSelect::lockExclusive**(int `$lock_waiting_option = ?`): mysql_xdevapi\TableSelect

Executa uma operação de leitura com EXCLUSIVE LOCK. Somente uma trava pode estar ativa no mesmo momento.

Parâmetros

`lock_waiting_option`

Espera opcional cujo padrão é **`MYSQLX_LOCK_DEFAULT`**. Valores válidos são:

- **`MYSQLX_LOCK_DEFAULT`**
- **`MYSQLX_LOCK_NOWAIT`**
- **`MYSQLX_LOCK_SKIP_LOCKED`**

Valor Retornado

Objeto TableSelect.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableSelect::lockExclusive()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$session->startTransaction();

$result = $table->select('name', 'age')
  ->lockExclusive(MYSQLX_LOCK_NOWAIT)
  ->execute();

$session->commit();

$row = $result->fetchAll();
print_r($row);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
    [1] => Array
        (
            [name] => Sam
            [age] => 42
        )
)
```

TableSelect::lockShared

TableSelect::lockShared

(No version information available, might only be in Git)

TableSelect::lockShared — Executa SHARED LOCK

Descrição

public function **mysql_xdevapi\TableSelect::lockShared**(int `$lock_waiting_option = ?`): mysql_xdevapi\TableSelect

Executa uma operação de leitura com SHARED LOCK. Somente uma trava pode estar ativa no mesmo momento.

Parâmetros

`lock_waiting_option`

Espera opcional cujo padrão é `MYSQLX_LOCK_DEFAULT`. Valores válidos são:

- `MYSQLX_LOCK_DEFAULT`
- `MYSQLX_LOCK_NOWAIT`
- `MYSQLX_LOCK_SKIP_LOCKED`

Valor Retornado

Um objeto TableSelect.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableSelect::lockShared()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$session->startTransaction();

$result = $table->select('name', 'age')
  ->lockShared(MYSQLX_LOCK_NOWAIT)
  ->execute();

$session->commit();

$row = $result->fetchAll();
print_r($row);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
    [1] => Array
        (
            [name] => Sam
            [age] => 42
        )
)
```

TableSelect::offset

TableSelect::offset

(No version information available, might only be in Git)

TableSelect::offset — Define o deslocamento limite

Descrição

public function **mysql_xdevapi\TableSelect::offset**(int `$position`): mysql_xdevapi\TableSelect

Pula o número informado de linhas no resultado.

Parâmetros

`position`

O deslocamento limite.

Valor Retornado

Um objeto TableSelect.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\TableSelect::offset()**

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 42)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('name', 'age')
  ->limit(1)
  ->offset(1)
  ->execute();

$row = $result->fetchAll();
print_r($row);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
```

```
    [0] => Array
        (
            [name] => Sam
            [age] => 42
        )
)
```

TableSelect::orderby

TableSelect::orderby

(No version information available, might only be in Git)

TableSelect::orderby — Define critérios de ordenamento na seleção

Descrição

public function **mysql_xdevapi\TableSelect::orderby**(mixed `$sort_expr`, mixed `...$sort_exprs`): mysql_xdevapi\TableSelect

Define a ordem por critérios.

Parâmetros

`sort_expr`

> As expressões que definem a ordenação por critério. Pode ser um array com uma ou mais expressões ou uma string.

`sort_exprs`

> Parâmetros sort_expr adicionais.

Valor Retornado

A TableSelect object.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableSelect::orderBy()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('name', 'age')
  ->orderBy('name desc')
  ->execute();

$row = $result->fetchAll();
print_r($row);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [name] => Sam
            [age] => 42
        )
    [1] => Array
        (
            [name] => John
            [age] => 42
        )
)
```

TableSelect::where

TableSelect::where

(No version information available, might only be in Git)

TableSelect::where — Define condição de pesquisa na seleção

Descrição

public function **mysql_xdevapi\TableSelect::where**(string `$where_expr`): mysql_xdevapi\TableSelect

Define condição de pesquisa para filtrar.

Parâmetros

`where_expr`

Define a condição de pesquisa para filtrar documentos ou registros.

Valor Retornado

Um objeto TableSelect.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\TableSelect::where()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('name','age')
  ->where('name like :name and age > :age')
  ->bind(['name' => 'John', 'age' => 42])
```

```
  ->execute();

$row = $result->fetchAll();
print_r($row);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
)
```

mysql_xdevapi\TableUpdate

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Uma instrução para operações de atualização de registros em uma tabela.

Resumo da classe

class **mysql_xdevapi\TableUpdate** implements mysql_xdevapi\Executable {
/* Métodos */
public function **bind**(array `$placeholder_values`): mysql_xdevapi\TableUpdate
public function **execute**(): mysql_xdevapi\TableUpdate
public function **limit**(int `$rows`): mysql_xdevapi\TableUpdate
public function **orderby**(mixed `$orderby_expr`, mixed `...$orderby_exprs`): mysql_xdevapi\TableUpdate
public function **set**(string `$table_field`, string `$expression_or_literal`): mysql_xdevapi\TableUpdate
public function **where**(string `$where_expr`): mysql_xdevapi\TableUpdate
}

Índice

TableUpdate::bind

TableUpdate::bind

(No version information available, might only be in Git)

TableUpdate::bind — Vincula parâmetros de consulta de atualização

Descrição

public function **mysql_xdevapi\TableUpdate::bind**(array `$placeholder_values`): mysql_xdevapi\TableUpdate

Vincula um valor a um marcador específico.

Parâmetros

`placeholder_values`

O nome do marcador e o valor a vincular, definidos como um array JSON.

Valor Retornado

Um objeto TableUpdate.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableUpdate::bind()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->update()
  ->set('status', 'admin')
  ->where('name = :name and age > :age')
  ->bind(['name' => 'Bernie', 'age' => 2000])
  ->execute();

?>
```

TableUpdate::__construct

TableUpdate::__construct

(No version information available, might only be in Git)

TableUpdate::__construct — Construtor de TableUpdate

Descrição

private function **mysql_xdevapi\TableUpdate::__construct**()

Iniciado pelo uso do método update().

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableUpdate::__construct()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");
```

```
$res = $table->update()
  ->set('level', 3)
  ->where('age > 15 and age < 22')
  ->limit(4)
  ->orderby(['age asc','name desc'])
  ->execute();

?>
```

TableUpdate::execute

TableUpdate::execute

(No version information available, might only be in Git)

TableUpdate::execute — Executa instrução de atualização

Descrição

public function **mysql_xdevapi\TableUpdate::execute**(): mysql_xdevapi\TableUpdate

Executa a instrução de atualização.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Um objeto TableUpdate.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableUpdate::execute()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$res = $table->update()
  ->set('level', 3)
  ->where('age > 15 and age < 22')
  ->limit(4)
  ->orderby(['age asc','name desc'])
  ->execute();

?>
```

TableUpdate::limit

TableUpdate::limit

(No version information available, might only be in Git)

TableUpdate::limit — Limita contagem de atualização de linhas

Descrição

public function **mysql_xdevapi\TableUpdate::limit**(int `$rows`): mysql_xdevapi\TableUpdate

Define o número máximo de registros ou documentos atualizados.

Parâmetros

`rows`

O número máximo de registros ou documentos a atualizar.

Valor Retornado

Um objeto TableUpdate.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\TableUpdate::limit()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$res = $table->update()
  ->set('level', 3)
  ->where('age > 15 and age < 22')
  ->limit(4)
  ->orderby(['age asc','name desc'])
  ->execute();

?>
```

TableUpdate::orderby

TableUpdate::orderby

(No version information available, might only be in Git)

TableUpdate::orderby — Define critérios de ordenação

Descrição

public function **mysql_xdevapi\TableUpdate::orderby**(mixed `$orderby_expr`, mixed `...$orderby_exprs`): mysql_xdevapi\TableUpdate

Define os critérios de ordenação.

Parâmetros

`orderby_expr`

As expressões que definem a ordenação por critério. Pode ser um array com uma ou mais expressões ou uma string.

`orderby_exprs`

Parâmetros sort_expr adicionais.

Valor Retornado

Um objeto TableUpdate.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableUpdate::orderby()

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$res = $table->update()
  ->set('level', 3)
  ->where('age > 15 and age < 22')
  ->limit(4)
  ->orderby(['age asc','name desc'])
  ->execute();
?>
```

TableUpdate::set

TableUpdate::set

(No version information available, might only be in Git)

TableUpdate::set — Adiciona campo a ser atualizado

Descrição

public function **mysql_xdevapi\TableUpdate::set**(string `$table_field`, string `$expression_or_literal`): mysql_xdevapi\TableUpdate

Atualiza valor da coluna nos registros em uma tabela.

Parâmetros

`table_field`

O nome da coluna a ser atualizada.

`expression_or_literal`

O valor a ser definido na coluna especificada.

Valor Retornado

Um objeto TableUpdate.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\TableUpdate::set()

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$res = $table->update()
  ->set('level', 3)
  ->where('age > 15 and age < 22')
  ->limit(4)
  ->orderby(['age asc','name desc'])
  ->execute();

?>
```

TableUpdate::where

TableUpdate::where

(No version information available, might only be in Git)

TableUpdate::where — Define filtro de pesquisa

Descrição

public function **mysql_xdevapi\TableUpdate::where**(string `$where_expr`): mysql_xdevapi\TableUpdate

Define a condição de pesquisa para filtrar.

Parâmetros

`where_expr`

A condição de pesquisa para filtrar documentos ou registros.

Valor Retornado

Um objeto TableUpdate.

Examples

Exemplo #1 Exemplo de **mysql_xdevapi\TableUpdate::where()**

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$res = $table->update()
  ->set('level', 3)
  ->where('age > 15 and age < 22')
  ->limit(4)
  ->orderby(['age asc','name desc'])
  ->execute();

?>
```

mysql_xdevapi\Warning

(PECL mysql-xdevapi >= 8.0.11)

Introdução

Resumo da classe

class **mysql_xdevapi\Warning** {
/* Propriedades */
public *$message*;
public *$level*;
public *$code*;
/* Constructor */
private function **__construct**()
}

Propriedades

message

level

code

Índice

Warning::__construct

Warning::__construct

(No version information available, might only be in Git)

Warning::__construct — Construtor de Warning

Descrição

private function **mysql_xdevapi\Warning::__construct**()

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Examples

Exemplo #1 Exemplo de mysql_xdevapi\Warning::__construct()

```
<?php

/* ... */

?>
```

MySQL (Original)

Introdução

Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida a partir do PHP 7.0.0. No lugar dela, tanto a extensão mysqli quanto a extensão PDO_MySQL devem ser usadas. Consulte também a Visão Geral da API MySQL para ajuda adicional na escolha de uma API MySQL.

Estas funções permitem acessar servidores de banco de dados MySQL. Mais informação sobre o MySQL podem ser encontradas em » http://www.mysql.com/.

Documentação para o MySQL pode ser encontrada em » http://dev.mysql.com/doc/.

Instalação/Configuração

Índice

Dependências

Para ter estas funções disponíveis, você deve compilar o PHP com suporte ao MySQL.

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:

Instalação

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:

Para compilar, simplesmente use a opção de configuração **--with-mysql[=DIR]** onde o parâmetro opcional `[DIR]` aponta para o diretório onde o MySQL está instalado.

Embora esta esta extensão MySQL seja compatível com o MySQL 4.1.0 e posterior, ela não suporta a funcionalidade extra que estas versões fornecem. Para isto, use a extensão MySQLi.

Se você quiser instalar a extensão mysql junto com a extensão mysqli você deve usar a mesma biblioteca de cliente para evitar quaisquer conflitos.

Instalação em Sistemas Linux

Nota: `[DIR]` é o caminho para os arquivos da biblioteca de cliente MySQL (*cabeçalhos e bibliotecas*), que podem ser obtidos do site do » MySQL.

Matriz de suporte em tempo de compilação da ext/mysql

Versão do PHP	Padrão	Opções de Configuração: mysqlnd	Opções de Configuração: `libmysqlclient`	Registro de Alterações

Versão do PHP	Padrão	Opções de Configuração: mysqlnd	Opções de Configuração: `libmysqlclient`	Registro de Alterações
4.x.x	libmysqlclient	Não Disponível	**--without-mysql** para desabilitar	MySQL habilitado por padrão, bibliotecas de clientes MySQL vêm embutidas
5.0.x, 5.1.x, 5.2.x	libmysqlclient	Não Disponível	**--with-mysql=[DIR]**	O MySQL não é mais habilitado por padrão, e as bibliotecas de clientes MySQL não vêm mais embutidas
5.3.x	libmysqlclient	**--with-mysql=mysqlnd**	**--with-mysql=[DIR]**	mysqlnd agora está disponível
5.4.x	mysqlnd	**--with-mysql**	**--with-mysql=[DIR]**	mysqlnd agora é o padrão

Instalação em Sistemas Windows

PHP 5.0.x, 5.1.x, 5.2.x

O MySQL não é mais habilitado por padrão, então a DLL *php_mysql.dll* deve estar habilitada dentro do *php.ini*. Além disso, o PHP precisa de acesso à biblioteca de cliente MySQL. Um arquivo chamado *libmysql.dll* é incluído na distribuição PHP do Windows, e para que o PHP possa conversar com o MySQL este arquivo precisa estar disponível no *PATH* dos sistemas Windows. Veja a pergunta entitulada "Como adicionar meu diretório PHP ao *PATH* do Windows" para informações sobre como fazer isso. Embora copiar o arquivo *libmysql.dll* para o diretório de sistema do Windows também funcione (porque o diretório de sistema está por padrão no *PATH* do sistema), isto não é recomendado.

Assim como ao habilitar qualquer extensão PHP (como *php_mysql.dll*), a diretiva PHP extension_dir deve apontar para o diretório onde as extensões PHP estão localizadas. Veja também as Instruções para Instalação Manual no Windows. Um exemplo de valor para extension_dir no PHP 5 é `c:\php\ext`

> **Nota** :
>
> Se ao iniciar o servidor web um erro similar ao seguinte ocorrer: `"Unable to load dynamic library './php_mysql.dll'"`, é porque o arquivo *php_mysql.dll* e/ou *libmysql.dll* não pode ser encontrado pelo sistema.

PHP 5.3.0+

O Driver Nativo MySQL está habilitado por padrão. Inclui *php_mysql.dll*, mas *libmysql.dll* não é mais necessário ou usado.

Notas de Instalação do MySQL

> **Aviso**
>
> Falhas e problemas de inicialização do PHP podem ocorrer ao carregar esta extensão em conjunto com a extensão recode. Veja a extensão recode para maiores informações.

> **Nota** :

> Se for necessário usar conjuntos de caracteres diferentes de *latin* (padrão), deve-se instalar uma libmysqlclient externa (não incluída) com o suporte ao conjunto de caracteres compilado.

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de configuração do MySQL

Nome	Padrão	Modificável	Registro de Alterações
mysql.allow_local_infile	"1"	`INI_SYSTEM`	
mysql.allow_persistent	"1"	`INI_SYSTEM`	
mysql.max_persistent	"-1"	`INI_SYSTEM`	
mysql.max_links	"-1"	`INI_SYSTEM`	
mysql.trace_mode	"0"	`INI_ALL`	
mysql.default_port	NULL	`INI_ALL`	
mysql.default_socket	NULL	`INI_ALL`	
mysql.default_host	NULL	`INI_ALL`	
mysql.default_user	NULL	`INI_ALL`	
mysql.default_password	NULL	`INI_ALL`	
mysql.connect_timeout	"60"	`INI_ALL`	

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`mysql.allow_local_infile` int

Permite acessar, da perspectiva do PHP, arquivos locais com declarações LOAD DATA

`mysql.allow_persistent` bool

Se deve-se permitir conexões persistentes para MySQL.

`mysql.max_persistent` int

O número máximo de conexões persistentes com o MySQL por processo.

`mysql.max_links` int

O número máximo de conexões com o MYSQL por processo, incluindo conexões persistentes.

`mysql.trace_mode` bool

Modo de rastreamento. Quando `mysql.trace_mode` está ativado, alertas para pesquisas em tabelas/índices, conjuntos de resultados não liberados, e erros de SQL serão exibidos.

`mysql.default_port` string

O número padrão da porta TCP a usar quando conectar com o servidor do banco de dados se nenhuma outra porta for especificada. Se nenhum padrão for especificado, a porta será obtida da variável de

ambiente *MYSQL_TCP_PORT*, da entrada `mysql-tcp` em */etc/services* ou da constante da compilação **`MYSQL_PORT`**, nesta ordem. Windows irá usar somente a constante **`MYSQL_PORT`**.

`mysql.default_socket` string

O nome padrão do socket para usar quando conectar com um servidor de banco de dados local se outro nome de socket não for especificado.

`mysql.default_host` string

O servidor padrão para usar quando conectar com um servidor de banco de dados se outro servidor não for encontrado. Não se aplica em modo seguro do SQL.

`mysql.default_user` string

O nome padrão de usuário para conectar ao servidor de banco de dados se outro nome não for especificado. Não se aplica em modo seguro do SQL.

`mysql.default_password` string

A senha padrão para usar ao conectar ao servidor de banco de dados se outra senha não for especificada. Não se aplica em modo seguro do SQL.

`mysql.connect_timeout` int

Limite de tempo da conexão em segundos. No Linux este limite é também usado para a espera da primeira resposta do servidor.

Tipos de Recurso

Existem dois tipos de recursos usados no módulo MySQL. O primeiro é o identificador de conexão com o banco de dados, e o segundo é o recurso que guarda o resultado de uma consulta.

Registro de Alterações

As alterações a seguir foram feitas em classes/funções/métodos desta extensão.

Registro Geral de Modificações para a extensão ext/mysql

Este registro de modificações referencia a extensão ext/mysql.

Modificações ext/mysql globais

Uma lista de modificações à extensão ext/mysql está apresentada a seguir.

Versão	Descrição
7.0.0	Esta extensão foi removida do PHP. Para detalhes, consulte Escolhendo uma API.

Modificações a funções existentes

A lista a seguir é uma compilação de entradas de modificações das funções ext/mysql.

Version	Function	Description

Constantes predefinidas

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

É possível especificar opções de cliente adicionais para as funções mysql_connect() e mysql_pconnect(). As seguintes constantes estão definidas:

Constantes do cliente MySQL

Constante	Descrição
MYSQL_CLIENT_COMPRESS	Usar protocolo de compressão
MYSQL_CLIENT_IGNORE_SPACE	Permitir espaços após nomes de funções
MYSQL_CLIENT_INTERACTIVE	Permitir interactive_timeout segundos (ao invés de **wait_timeout**) de inatividade antes de fechar a conexão.
MYSQL_CLIENT_SSL	Usar criptografia SSL. Esta opção está disponível somente com a versão 4.x ou posterior da biblioteca cliente MySQL. A versão 3.23.x é incluída com o PHP 4 e com binários Windows do PHP 5.

A função mysql_fetch_array() usa uma constante para os diferentes tipos de arrays de resultados. As constantes a seguir estão definidas:

Constantes de busca MySQL

Constante	Descrição
MYSQL_ASSOC	Colunas são retornadas no array contendo o nome do campo como as chaves do array.
MYSQL_BOTH	Colunas são retornadas no array contendo chaves numéricas e chaves com o nome do campo.
MYSQL_NUM	Colunas são retornadas no array contendo chaves numéricas para os campos. A chave inicia em 0 para o primeiro campo do resultado.

Examples

Índice

Exemplo de uso da extensão MySQL

Este exemplo simples mostra como conectar, executar uma consulta, mostrar as linhas do resultado e disconectar de um banco de dados MySQL.

Exemplo #1 Exemplo de uso da extensão MySQL

```
<?php
// Conectando e selecionando o banco de dados
$link = mysql_connect('mysql_host', 'mysql_user', 'mysql_password')
    or die('Não foi possível conectar: ' . mysql_error());
echo 'Conectado com sucesso';
mysql_select_db('my_database') or die('Não foi possível selecionar o banco de dados');

// Realizado a consulta SQL
$query = 'SELECT * FROM my_table';
$result = mysql_query($query) or die('A consulta falhou: ' . mysql_error());

// Exebindo os resultados em HTML
echo "<table>\n";
while ($line = mysql_fetch_array($result, MYSQL_ASSOC)) {
    echo "\t<tr>\n";
```

```
    foreach ($line as $col_value) {
        echo "\t\t<td>$col_value</td>\n";
    }
    echo "\t</tr>\n";
}
echo "</table>\n";

// Libera o conjunto de resultados
mysql_free_result($result);

// Fecha a conexão
mysql_close($link);
?>
```

Funções da MySQL

Notas

> **Nota** :
>
> A maioria das funções MySQL aceitam `link_identifier` como o último parâmetro opcional. Se este não for dados, a última conexão aberta é usada. Se não existir, é tentado estabelecer uma conexão com os valores padrão definidos no *php.ini*. Se não for possível, a função retornará `false`.

Índice

mysql_affected_rows

mysql_affected_rows

(PHP 4, PHP 5)

mysql_affected_rows — Obtém o número de linhas afetadas na operação anterior do MySQL

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_affected_rows()
> - PDOStatement::rowCount()

Descrição

function **mysql_affected_rows**(resource `$link_identifier` = NULL): int

Obtém o número de linhas afetadas pelo último INSERT, UPDATE, REPLACE ou DELETE associado ao `link_identifier`.

Parâmetros

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

Retorna o número de linas afetadas em caso de sucesso, e -1 se a última consulta falhou.

Se a última consulta foi um DELETE sem a claúsula WHERE, todos os registros terão sido excluídos da tabela mas esta função irá retornar zero em versões do MySQL anteriores a 4.1.2.

Ao usar UPDATE, o MySQL não irá atualizar colunas aonde o novo valor é o mesmo que o valor anterior. Isto cria a possibilidade de que **mysql_affected_rows()** possa não ser igual ao número de linhas encontradas, apenas o número de linhas que literalmente foram afetadas pela consulta.

O comando REPLACE primeiro exclui o registro com a mesma chave primária e então insere o novo registro. Esta função retorna o número de registros excluídos mais o número de registros inseridos.

No caso de consultas "INSERT ... ON DUPLICATE KEY UPDATE", o valor retornado será `1` se um insert foi executado, ou `2` para um update de uma linha existente.

Examples

Exemplo #1 Exemplo mysql_affected_rows()

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}
mysql_select_db('mydb');

/* isso deve retornar os números corretos de registros excluídos */
mysql_query('DELETE FROM mytable WHERE id < 10');
printf("Registro excluídos: %d\n", mysql_affected_rows());

/* com uma cláusula where que nunca é verdadeira, ela deve retornar 0 */
mysql_query('DELETE FROM mytable WHERE 0');
printf("Registro excluídos: %d\n", mysql_affected_rows());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Registro excluídos: 10
Registro excluídos: 0
```

Exemplo #2 Exemplo de mysql_affected_rows() usando transações

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
```

```php
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}
mysql_select_db('mydb');

/* Atualiza registros */
mysql_query("UPDATE mytable SET used=1 WHERE id < 10");
printf ("Registros atualizados: %d\n", mysql_affected_rows());
mysql_query("COMMIT");
?>
```

O exemplo acima produzirá algo semelhante a:

```
Registros atualizados: 10
```

Notas

> **Nota** : **Transações**
>
> Se você estiver usando transações, você deve utilizar **mysql_affected_rows()** depois da sua consulta INSERT, UPDATE, ou DELETE, não depois de COMMIT.

> **Nota** : **Comandos SELECT**
>
> Para obter o número de linhas retornados por um SELECT, é possível usar mysql_num_rows().

> **Nota** : **Chaves estrangeiras em cascata**
>
> **mysql_affected_rows()** não conta linhas afetadas implicitamente pelo uso de ON DELETE CASCADE e/ou ON UPDATE CASCADE em chaves estrangeiras.

Veja também

- mysql_num_rows() - Obtém o número de linhas em um resultado
- mysql_info() - Obtém informação sobre a consulta mais recente

mysql_client_encoding

mysql_client_encoding

(PHP 4 >= 4.3.0, PHP 5)

mysql_client_encoding — Retorna o nome do conjunto de caracteres

> **Aviso**

> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_character_set_name()

Descrição

function **mysql_client_encoding**(resource `$link_identifier` = NULL): string

Obtém a variável `character_set` do MySQL.

Parâmetros

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

Retorna o nome do conjunto de caracteres padrão para a conexão atual.

Examples

Exemplo #1 Exemplo de mysql_client_encoding()

```php
<?php
$link    = mysql_connect('localhost', 'mysql_user', 'mysql_password');
$charset = mysql_client_encoding($link);

echo "O conjunto de caracteres atual é: $charset\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
O conjunto de caracteres atual é: latin1
```

Veja também

- mysql_set_charset() - Define o conjunto de caracteres do cliente
- mysql_real_escape_string() - Escapa os caracteres especiais em uma string para uso em uma instrução SQL

mysql_close

mysql_close

(PHP 4, PHP 5)

mysql_close — Fecha a conexão MySQL

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_close()
> - PDO: Atribuir o valor `null` ao objeto PDO

Descrição

function **mysql_close**(resource `$link_identifier` = NULL): bool

mysql_close() fecha a conexão não persistente ao servidor MySQL que está associado ao identificador de conexão especificado. Se `link_identifier` não for especificado, a última conexão aberta é usada.

Conexões MySQL não persistentes abertas e conjuntos de resultados são automaticamente destruídos quando um script PHP termina sua execução. Portanto, embora seja opcional fechar conexões abertas e liberar conjuntos de resultados, isso é recomendado. Isso retornará imediatamente os recursos usados ao PHP e MySQL, o que pode melhorar o desempenho. Para obter informações relacionadas, consulte a página liberando recursos.

Parâmetros

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo **mysql_close()**

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}
echo 'Conectado com sucesso';
mysql_close($link);
?>
```

O exemplo acima produzirá:

```
Conectado com sucesso
```

Notas

> **Nota** :
>
> **mysql_close()** não irá fechar conexões persistentes criadas por mysql_pconnect(). Para detalhes adicionais, consulte a seção sobre conexões persistentes.

Veja também

- mysql_connect() - Abre uma conexão a um servidor MySQL
- mysql_free_result() - Libera memória usada pelo resultado

mysql_connect

mysql_connect

(PHP 4, PHP 5)

mysql_connect — Abre uma conexão a um servidor MySQL

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_connect()
> - PDO::__construct()

Descrição

```
function mysql_connect(
    string $server = ini_get("mysql.default_host"),
    string $username = ini_get("mysql.default_user"),
    string $password = ini_get("mysql.default_password"),
    bool $new_link = false,
    int $client_flags = 0
): resource|false
```

Abre ou reutiliza uma conexão a um servidor MySQL.

Parâmetros

`server`

O servidor MySQL. Também pode incluir um número de porta, por exemplo, "servidor:porta" ou um caminho para um soquete local, por exemplo ":/caminho/para/socquete" para o servidor local.

Se a diretiva do PHP mysql.default_host não estiver definida (padrão), o valor padrão é 'localhost:3306'. No modo seguro do SQL, este parâmetro é ignorado e o valor 'localhost:3306' é sempre usado.

`username`

O nome de usuário. O valor padrão é definido pela diretiva mysql.default_user. No modo seguro do SQL, este parâmetro é ignorado e o nome do usuário proprietário do processo do servidor é usado.

`password`

A senha. O valor padrão é definido pela diretiva mysql.default_password. No modo seguro do SQL, este parâmetro é ignorado e uma senha vazia é usada.

`new_link`

Se uma segunda chamada for feita a **mysql_connect()** com os mesmos argumentos, não é estabelecida uma nova conexão, mas ao invés disso, o identificador da conexão que já estiver aberta será retornado. O parâmetro `new_link` modifica este comportamento e faz com que **mysql_connect()** sempre abra uma nova conexão, mesmo se **mysql_connect()** tiver sido chamada antes com os mesmos parâmetros. No modo seguro do SQL, este parâmetro é ignorado.

`client_flags`

O parâmetro `client_flags` pode ser uma combinação das seguintes constantes: 128 (habilita uso de `LOAD DATA LOCAL`), **`MYSQL_CLIENT_SSL`**, **`MYSQL_CLIENT_COMPRESS`**, **`MYSQL_CLIENT_IGNORE_SPACE`** ou **`MYSQL_CLIENT_INTERACTIVE`**. Leia a seção sobre Constantes do cliente MySQL para mais informações. No modo seguro do SQL, este parâmetro é ignorado.

Valor Retornado

Retorna um identificador de conexão MySQL em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo de **mysql_connect()**

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}
echo 'Conexão bem sucedida';
mysql_close($link);
?>
```

Exemplo #2 Exemplo de **mysql_connect()** usando a a sintaxe `servidor:porta`

```php
<?php
// Conexão a example.com na porta 3307
$link = mysql_connect('example.com:3307', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}
echo 'Conexão bem sucedida';
mysql_close($link);

// Conexão com servidor local na porta 3307
$link = mysql_connect('127.0.0.1:3307', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}
echo 'Conexão bem sucedida';
```

```
mysql_close($link);
?>
```

Exemplo #3 Exemplo de mysql_connect() usando a sintaxe ":/caminho/para/soquete"

```
<?php
// Conexão ao servidor local e soquete, ex.: /tmp/mysql.sock

// Variação 1: omitindo localhost
$link = mysql_connect('/tmp/mysql', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}
echo 'Conexão bem sucedida';
mysql_close($link);


// Variação 2: com localhost
$link = mysql_connect('localhost:/tmp/mysql.sock', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}
echo 'Conexão bem sucedida';
mysql_close($link);
?>
```

Notas

> **Nota** :
>
> Em qualquer lugar que for especificado "localhost" ou "localhost:porta" como servidor, a biblioteca cliente do MySQL irá sobrescrever isso e tentar se conectar a um soquete local (pipe nomeado no Windows). Se for desejado usar TCP/IP, deve-se usar "127.0.0.1" ao invés de "localhost". Se a biblioteca cliente do MySQL tentar se conectar com o soquete local errado, o caminho correto deve ser definido como mysql.default_host no *php.ini* e o campo do servidor deve ser deixado em branco.

> **Nota** :
>
> A conexão com o servidor será fechada assim que a execução do script terminar, a menos que tenha sido fechada anteriormente chamando-se mysql_close() explicitamente.

> **Nota** :
>
> O erro "Can't create TCP/IP socket (10106)" normalmente significa que a diretiva de configuração variables_order não contém o caractere E. No Windows, se o ambiente não for copiado, a variável de ambiente SYSTEMROOT não estará disponível e o PHP terá problemas para carregar o Winsock.

Veja também

- mysql_pconnect() - Abre uma conexão persistente a um servidor MySQL
- mysql_close() - Fecha a conexão MySQL

mysql_create_db

mysql_create_db

(PHP 4, PHP 5)

mysql_create_db — Cria um banco de dados MySQL

> **Aviso**
>
> Esta função foi descontinuada a partir do PHP 4.3.0 e ela e a extensão MySQL original foram removidas a partir do PHP 7.0.0. Em vez disso, use as extensões MySQLi ou PDO_MySQL, que são desenvolvidas ativamente. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_query()
> - PDO::query()

Descrição

function **mysql_create_db**(string `$database_name`, resource `$link_identifier` = NULL): bool

mysql_create_db() tenta criar um novo banco de dados no servidor associado com o identificador de conexão especificado.

Parâmetros

`database_name`

O nome do banco de dados a ser criado.

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo alternativo de mysql_create_db()

A função **mysql_create_db()** foi descontinuada. É preferível usar a função mysql_query() para executar o comando `CREATE DATABASE`.

```
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}

$sql = 'CREATE DATABASE my_db';
if (mysql_query($sql, $link)) {
    echo "Banco de dados my_db criado com sucesso\n";
```

```
} else {
    echo 'Erro ao criar banco de dados: ' . mysql_error() . "\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Banco de dados my_db criado com sucesso
```

Notas

> **Nota** :
>
> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado: **mysql_createdb()**

> **Nota** :
>
> Esta função não estará disponível se a extensão MySQL foi compilada com a versão MySQL 4.x da biblioteca de cliente.

Veja também

- mysql_query() - Envia uma consulta MySQL
- mysql_select_db() - Seleciona um banco de dados MySQL

mysql_data_seek

mysql_data_seek

(PHP 4, PHP 5)

mysql_data_seek — Move o ponteiro interno do resultado

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_data_seek()
> - `PDO::FETCH_ORI_ABS`

Descrição

function **mysql_data_seek**(resource `$result`, int `$row_number`): bool

mysql_data_seek() move o ponteiro interno da linha de um resultado MySQL associado ao identificador de resultado informado para o número da linha especificada. A próxima chamada a uma função MySQL que retorna resultado, como mysql_fetch_assoc(), irá retornar esta linha.

`row_number` começa em 0. O `row_number` deve ser um valor no intervalo de 0 a mysql_num_rows() - 1. Entretanto, se o conjunto de resultados for vazio (mysql_num_rows() == 0), uma busca para 0 irá falhar com um **`E_WARNING`** e **mysql_data_seek()** iá retornar **`false`**.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

`row_number`

O número de linha desejado para o ponteiro de resultado.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo **mysql_data_seek()**

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}
$db_selected = mysql_select_db('sample_db');
if (!$db_selected) {
    die('Não foi possível selecionar o banco de dados: ' . mysql_error());
}
$query = 'SELECT last_name, first_name FROM friends';
$result = mysql_query($query);
if (!$result) {
    die('A consulta falhou: ' . mysql_error());
}
/* Busca linhas na ordem inversa */
for ($i = mysql_num_rows($result) - 1; $i >= 0; $i--) {
    if (!mysql_data_seek($result, $i)) {
        echo "Não foi possível apontar para a linha $i: " . mysql_error() . "\n";
        continue;
    }

    if (!($row = mysql_fetch_assoc($result))) {
        continue;
    }

    echo $row['last_name'] . ' ' . $row['first_name'] . "<br />\n";
}

mysql_free_result($result);
?>
```

Notas

> **Nota** :
>
> A função **mysql_data_seek()** pode ser usada apenas em conjunto com a função mysql_query(), não com mysql_unbuffered_query().

Veja também

- mysql_query() - Envia uma consulta MySQL
- mysql_num_rows() - Obtém o número de linhas em um resultado
- mysql_fetch_row() - Obtém uma linha como um array numérico
- mysql_fetch_assoc() - Obtém uma linha do resultado como um array associativo
- mysql_fetch_array() - Obtém uma linha de resultado como um array associativo, numérico, ou ambos
- mysql_fetch_object() - Obtém o resultado de uma linha como um objeto

mysql_db_name

mysql_db_name

(PHP 4, PHP 5)

mysql_db_name — Obtém o nome do banco de dados a partir da chamada à mysql_list_dbs()

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - Query: `SELECT DATABASE()`

Descrição

function **mysql_db_name**(resource `$result`, int `$row`, mixed `$field` = NULL): string

Obtém o nome do banco de dados a partir de uma chamada à mysql_list_dbs().

Parâmetros

`result`

O ponteiro de resultado de uma chamada à mysql_list_dbs().

`row`

O índice dentro do conjunto de resultados.

`field`

O nome do campo.

Valor Retornado

Retorna o nome do banco de dados em caso de sucesso, e **`false`** em caso de falha. Se for retornado **`false`**, utilize mysql_error() para determinar a natureza do erro.

Examples

Exemplo #1 Exemplo mysql_db_name()

```php
<?php
error_reporting(E_ALL);

$link = mysql_connect('dbhost', 'username', 'password');
$db_list = mysql_list_dbs($link);

$i = 0;
$cnt = mysql_num_rows($db_list);
while ($i < $cnt) {
    echo mysql_db_name($db_list, $i) . "\n";
    $i++;
}
?>
```

Notas

> **Nota** :
>
> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado: **mysql_dbname()**

Veja também

- mysql_list_dbs() - Lista os bancos de dados disponíveis em um servidor MySQL
- mysql_tablename() - Obtém o nome da tabela do campo

mysql_db_query

mysql_db_query

(PHP 4, PHP 5)

mysql_db_query — Seleciona um banco de dados e executa uma consulta nele

> **Aviso**
>
> Esta função foi descontinuada a partir do PHP 5.3.0 e ela e a extensão MySQL original foram removidas a partir do PHP 7.0.0. Em vez disso, use as extensões MySQLi ou PDO_MySQL, que são desenvolvidas ativamente. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_select_db() e depois a consulta
> - PDO::__construct()

Descrição

function **mysql_db_query**(string `$database`, string `$query`, resource `$link_identifier` = NULL): resource|bool

mysql_db_query() seleciona um banco de dados e executa uma consulta nele.

Parâmetros

`database`

O nome do banco de dados que será selecionado.

`query`

A consulta MySQL.

Os dados dentro da consulta devem estar adequadamente escapados.

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível **`E_WARNING`** será gerado.

Valor Retornado

Retorna um recurso de resultado MySQL com o resultado da consulta, ou **`false`** em caso de erro. A função também retorna **`true`**/**`false`** para consultas `INSERT`/`UPDATE`/`DELETE` para indicar sucesso/falha.

Examples

Exemplo #1 Exemplo alternativo de **mysql_db_query()**

```php
<?php

if (!$link = mysql_connect('mysql_host', 'mysql_user', 'mysql_password')) {
    echo 'Não foi possível conectar ao mysql';
    exit;
}

if (!mysql_select_db('mysql_dbname', $link)) {
    echo 'Não foi possível selecionar o banco de dados';
    exit;
}

$sql    = 'SELECT foo FROM bar WHERE id = 42';
$result = mysql_query($sql, $link);

if (!$result) {
    echo "Erro do banco de dados, não foi possível consultar o banco de dados\n";
    echo 'Erro MySQL: ' . mysql_error();
    exit;
}

while ($row = mysql_fetch_assoc($result)) {
    echo $row['foo'];
}

mysql_free_result($result);

?>
```

Notas

> **Nota** :
>
> Deve-se ter atenção pois esta função **NÃO** muda de volta para o banco de dados que estava conectado anteriormente. Em outras palavras, esta função não pode ser usada executar *temporariamente* uma consulta SQL em outro banco de dados, o retorno ao banco anterior deve ser feito manualmente. É fortemente recomendado usar a sintaxe `bancodedados.tabela` nas consultas SQL ou mysql_select_db() ao invés desta função.

Veja também

- mysql_query() - Envia uma consulta MySQL
- mysql_select_db() - Seleciona um banco de dados MySQL

mysql_drop_db

mysql_drop_db

(PHP 4, PHP 5)

mysql_drop_db — Drop (exclui) um banco de dados MySQL

> **Aviso**
>
> Esta função foi descontinuada a partir do PHP 4.3.0 e ela e a extensão MySQL original foram removidas a partir do PHP 7.0.0. Em vez disso, use as extensões MySQLi ou PDO_MySQL, que são desenvolvidas ativamente. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - Executa um query `DROP DATABASE`

Descrição

function **mysql_drop_db**(string `$database_name`, resource `$link_identifier` = NULL): bool

mysql_drop_db() tenta executar um drop (remover) todo um banco de dados do servidor associado ao identificador de conexão especificado. Esta função foi descontinuada, é preferível usar mysql_query() para realilar um comando `DROP DATABASE`.

Parâmetros

`database_name`

O nome do banco de dados que será excluído.

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível **`E_WARNING`** será gerado.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo alternativo de mysql_drop_db()

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}

$sql = 'DROP DATABASE my_db';
if (mysql_query($sql, $link)) {
    echo "Banco de dados my_db foi excluído com sucesso\n";
} else {
    echo 'Erro ao excluir o banco de dados: ' . mysql_error() . "\n";
}
?>
```

Notas

> **Aviso**
>
> Esta função não estará disponível se a extensão MySQL foi compilada com a versão da biblioteca de cliente MySQL 4.x.

> **Nota** :
>
> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado: **mysql_dropdb()**

Veja também

- mysql_query() - Envia uma consulta MySQL

mysql_errno

mysql_errno

(PHP 4, PHP 5)

mysql_errno — Retorna o valor numérico da mensagem de erro da operação MySQL anterior

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_errno()
> - PDO::errorCode()

Descrição

function **mysql_errno**(resource `$link_identifier` = NULL): int

Retorna o número do erro da última função MySQL.

Erros vindos do banco de dados MySQL não geram mais avisos. Ao invés disso, use **mysql_errno()** para obter o código do erro. Note que esta função retorna o código apenas da função MySQL mais recentemente usada (não incluindo mysql_error() e **mysql_errno()**), assim se você quiser usá-la, tenha certeza de verificar o valor antes de executar outra função MySQL.

Parâmetros

`link_identifier`

> A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

Retorna o número de erro da última função MySQL, ou `0` (zero) se não houve erro.

Examples

Exemplo #1 Exemplo **mysql_errno()**

```php
<?php
$link = mysql_connect("localhost", "mysql_user", "mysql_password");

if (!mysql_select_db("nonexistentdb", $link)) {
    echo mysql_errno($link) . ": " . mysql_error($link). "\n";
}

mysql_select_db("kossu", $link);
if (!mysql_query("SELECT * FROM nonexistenttable", $link)) {
    echo mysql_errno($link) . ": " . mysql_error($link) . "\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
1049: Unknown database 'nonexistentdb'
1146: Table 'kossu.nonexistenttable' doesn't exist
```

Veja também

- mysql_error() - Retorna o texto da mensagem de erro da operação MySQL anterior
- » códigos de erro MySQL

mysql_error

mysql_error

(PHP 4, PHP 5)

mysql_error — Retorna o texto da mensagem de erro da operação MySQL anterior

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_error()
> - PDO::errorInfo()

Descrição

function **mysql_error**(resource `$link_identifier` = NULL): string

Retorna o texto do último erro de funções MySQL. Erros vindos do banco de dados MySQL não geram mais avisos. Ao invés, use **mysql_error()** para obter o texto do erro. Note que esta função apenas retorna o texto do erro da função MySQL mais recentemente executada (não incluindo **mysql_error()** e mysql_errno()), assim se você quiser usa-la, tenha certeza de obter o valor antes de utilizar outra função MySQL.

Parâmetros

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

Retorna o texto do erro da ultima função MySQL, ou `''` (uma string vazia) se não houve erro.

Examples

Exemplo #1 Exemplo **mysql_error()**

```php
<?php
$link = mysql_connect("localhost", "mysql_user", "mysql_password");

mysql_select_db("nonexistentdb", $link);
echo mysql_errno($link) . ": " . mysql_error($link). "\n";

mysql_select_db("kossu", $link);
mysql_query("SELECT * FROM nonexistenttable", $link);
echo mysql_errno($link) . ": " . mysql_error($link) . "\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
1049: Unknown database 'nonexistentdb'
```

```
1146: Table 'kossu.nonexistenttable' doesn't exist
```

Veja também

- mysql_errno() - Retorna o valor numérico da mensagem de erro da operação MySQL anterior
- » códigos de erro MySQL

mysql_escape_string

mysql_escape_string

(PHP 4 >= 4.0.3, PHP 5)

mysql_escape_string — Escapa uma string para usá-la em uma função mysql_query

> **Aviso**
>
> Esta função foi descontinuada a partir do PHP 4.3.0 e ela e a extensão MySQL original foram removidas a partir do PHP 7.0.0. Em vez disso, use as extensões MySQLi ou PDO_MySQL, que são desenvolvidas ativamente. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_escape_string()
> - PDO::quote()

Descrição

function **mysql_escape_string**(string `$unescaped_string`): string

Esta função escapará a string `unescaped_string`, de modo que seja seguro usá-la em uma função mysql_query(). Esta função foi descontinuada.

Esta função é idêntica à mysql_real_escape_string() exceto que mysql_real_escape_string() recebe um manipulador de conexão e escapa a string de acordo com o conjunto de caracteres atual. **mysql_escape_string()** não recebe um manipulador de conexão e não respeita a configuração atual do conjunto de caracteres.

Parâmetros

`unescaped_string`

A string que deve ser escapada.

Valor Retornado

Retorna a string escapada.

Examples

Exemplo #1 Exemplo de mysql_escape_string()

```php
<?php
$item = "Zak's Laptop";
$escaped_item = mysql_escape_string($item);
```

```
printf("String escapada: %s\n", $escaped_item);
?>
```

O exemplo acima produzirá:

```
String escapada: Zak\'s Laptop
```

Notas

> **Nota** :
>
> **mysql_escape_string()** não escapa % e _.

Veja também

- mysql_real_escape_string() - Escapa os caracteres especiais em uma string para uso em uma instrução SQL

mysql_fetch_array

mysql_fetch_array

(PHP 4, PHP 5)

mysql_fetch_array — Obtém uma linha de resultado como um array associativo, numérico, ou ambos

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_fetch_array()
> - PDOStatement::fetch()

Descrição

function **mysql_fetch_array**(resource `$result`, int `$result_type` = MYSQL_BOTH): array

Retorna um array que corresponde à linha obtida e move adiante o ponteiro interno dos dados.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

`result_type`

O tipo de array que deve ser obtido. É uma constante e aceita os seguintes valores: **MYSQL_ASSOC**, **MYSQL_NUM** e **MYSQL_BOTH**.

Valor Retornado

Retorna um array que corresponde à linha obtida ou **false** se não houver mais linhas. O tipo da array retornado depende de como `result_type` estiver definido. Usando **MYSQL_BOTH** (padrão), será obtido um array com ambos os índices, numérico e associativo. Usando **MYSQL_ASSOC**, somente índices associativos serão obtidos (da mesma forma que mysql_fetch_assoc() funciona), e usando **MYSQL_NUM**, somente os índices numéricos (como mysql_fetch_row() funciona).

Se duas ou mais colunas do resultado tiverem os mesmos nomes de campos, a última coluna terá precedência. Para acessar a(s) outra(s) coluna(s) de mesmo nome, o índice numérico da coluna deve ser usado ou um apelido para a coluna deve ser especificado. Para colunas com apelidos, o conteúdo não poderá ser acessado com o nome original da coluna.

Examples

Exemplo #1 Consulta com nomes de campos duplicados usando apelidos

```
SELECT table1.field AS foo, table2.field AS bar FROM table1, table2
```

Exemplo #2 mysql_fetch_array() com MYSQL_NUM

```
<?php
mysql_connect("localhost", "mysql_user", "mysql_password") or
    die("Não foi possível conectar: " . mysql_error());
mysql_select_db("mydb");

$result = mysql_query("SELECT id, name FROM mytable");

while ($row = mysql_fetch_array($result, MYSQL_NUM)) {
    printf("ID: %s  Nome: %s", $row[0], $row[1]);
}

mysql_free_result($result);
?>
```

Exemplo #3 mysql_fetch_array() com MYSQL_ASSOC

```
<?php
mysql_connect("localhost", "mysql_user", "mysql_password") or
    die("Não foi possível conectar: " . mysql_error());
mysql_select_db("mydb");

$result = mysql_query("SELECT id, name FROM mytable");

while ($row = mysql_fetch_array($result, MYSQL_ASSOC)) {
    printf("ID: %s  Nome: %s", $row["id"], $row["name"]);
}

mysql_free_result($result);
?>
```

Exemplo #4 mysql_fetch_array() com MYSQL_BOTH

```php
<?php
mysql_connect("localhost", "mysql_user", "mysql_password") or
    die("Não foi possível conectar: " . mysql_error());
mysql_select_db("mydb");

$result = mysql_query("SELECT id, name FROM mytable");

while ($row = mysql_fetch_array($result, MYSQL_BOTH)) {
    printf ("ID: %s  Nome: %s", $row[0], $row["name"]);
}

mysql_free_result($result);
?>
```

Notas

> **Nota** : **Desempenho**
>
> É importante observar que **mysql_fetch_array()** *não é significativamente* mais lento do que mysql_fetch_row(), enquanto que a primeira fornece um valor agregado significativo.

> **Nota** :
>
> Os nomes de campos retornados por esta função *diferenciam maiúsculas e minúsculas*.

> **Nota** :
>
> Esta função define campos NULL como o valor `null` do PHP.

Veja também

- mysql_fetch_row() - Obtém uma linha como um array numérico
- mysql_fetch_assoc() - Obtém uma linha do resultado como um array associativo
- mysql_data_seek() - Move o ponteiro interno do resultado
- mysql_query() - Envia uma consulta MySQL

mysql_fetch_assoc

mysql_fetch_assoc

(PHP 4 >= 4.0.3, PHP 5)

mysql_fetch_assoc — Obtém uma linha do resultado como um array associativo

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_fetch_assoc()

> - PDOStatement::fetch() com `mode` como **`PDO::FETCH_ASSOC`**

Descrição

function **mysql_fetch_assoc**(resource `$result`): array

Retorna um array associativo que corresponde a linha obtida e move o ponteiro interno dos dados adiante. **mysql_fetch_assoc()** é equivalente a utilizar mysql_fetch_array() com MYSQL_ASSOC para o segunto parâmetro, que é opcional. Ela apenas retorna um array associativo.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

Valor Retornado

Retorna um array associativo de strings que corresponde à linha obtida, ou **`false`** se não houver mais linhas.

Se duas ou mais colunas do resultado tiverem o mesmo nome de campo, a ultima coluna terá precedencia. Para acessar as outras colunas com o mesmo nome, é necessário acessar os outros resultados usando índices numéricos com mysql_fetch_row() ou adicionar apelidos. Veja o exemplo em mysql_fetch_array() para uma descrição sobre apelidos.

Examples

Exemplo #1 Um exemplo expandido de mysql_fetch_assoc()

```php
<?php

$conn = mysql_connect("localhost", "mysql_user", "mysql_password");

if (!$conn) {
    echo "Não foi possível conectar ao DB: " . mysql_error();
    exit;
}

if (!mysql_select_db("mydbname")) {
    echo "Não foi possível selecionar mydbname: " . mysql_error();
    exit;
}

$sql = "SELECT id as userid, fullname, userstatus
        FROM   sometable
        WHERE  userstatus = 1";

$result = mysql_query($sql);

if (!$result) {
    echo "Não foi possível executar a consulta ($sql) no DB: " . mysql_error();
    exit;
}

if (mysql_num_rows($result) == 0) {
    echo "Nenhuma linha encontrada, nada para exibir, saindo...";
    exit;
```

```
}

// Enquanto uma linha de dados existir, coloque essa linha em $row como um array associativo
// Observação: Se for esperada apenas uma linha, não é necessário usar um loop
// Observação: Se for colocado extract($row); dentro do loop a seguir, serão
// criados $userid, $fullname e $userstatus
while ($row = mysql_fetch_assoc($result)) {
    echo $row["userid"];
    echo $row["fullname"];
    echo $row["userstatus"];
}

mysql_free_result($result);

?>
```

Notas

> **Nota** : **Performance**
>
> Uma coisa importante para notar é que usar **mysql_fetch_assoc()** *não é significativamente* mais lento do que usar mysql_fetch_row(), enquanto provê um valor agregado significante.

> **Nota** :
>
> Os nomes de campos retornados por esta função *diferenciam maiúsculas e minúsculas*.

> **Nota** :
>
> Esta função define campos NULL como o valor `null` do PHP.

Veja também

- mysql_fetch_row() - Obtém uma linha como um array numérico
- mysql_fetch_array() - Obtém uma linha de resultado como um array associativo, numérico, ou ambos
- mysql_data_seek() - Move o ponteiro interno do resultado
- mysql_query() - Envia uma consulta MySQL
- mysql_error() - Retorna o texto da mensagem de erro da operação MySQL anterior

mysql_fetch_field

mysql_fetch_field

(PHP 4, PHP 5)

mysql_fetch_field — Obtém informações de coluna de um resultado e retorna como um objeto

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:

> - mysqli_fetch_field()
> - PDOStatement::getColumnMeta()

Descrição

function **mysql_fetch_field**(resource `$result`, int `$field_offset` = 0): object

Retorna um objeto contendo informações sobre campo. Esta função pode ser usada para obter informações sobre campos no resultado de consulta informado.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

`field_offset`

O índice numérico do campo. Se o índice não for especificado, o próximo campo que ainda não tenha sido obtido por esta função é recuperado. `field_offset` inicia em `0`.

Valor Retornado

Retorna um object contendo informação sobre o campo. As propriedades do objeto são:

- name - nome da coluna
- table - a tabela à qual o objeto pertence, que é o nome do apelido se um estiver definido
- max_length - comprimento máximo da coluna
- not_null - 1 se a coluna não puder ser `null`
- primary_key - 1 se a coluna for uma chave primária
- unique_key - 1 se a coluna for uma chave exclusiva
- multiple_key - 1 se a coluna for uma chave não-exclusiva
- numeric - 1 se a coluna for numérica
- blob - 1 se a coluna for um BLOB
- type - o tipo da coluna
- unsigned - 1 se a coluna for sem sinal
- zerofill - 1 se a coluna for prenchida com zero

Examples

Exemplo #1 Exemplo de **mysql_fetch_field()**

```php
<?php
$conn = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$conn) {
    die('Não foi possível conectar: ' . mysql_error());
}
```

```php
mysql_select_db('database');
$result = mysql_query('select * from table');
if (!$result) {
    die('A consulta falhou: ' . mysql_error());
}
/* Obtém as informações da coluna */
$i = 0;
while ($i < mysql_num_fields($result)) {
    echo "Informação para a coluna $i:<br />\n";
    $meta = mysql_fetch_field($result, $i);
    if (!$meta) {
        echo "Sem informação disponível<br />\n";
    }
    echo "<pre>
blob:         $meta->blob
max_length:   $meta->max_length
multiple_key: $meta->multiple_key
name:         $meta->name
not_null:     $meta->not_null
numeric:      $meta->numeric
primary_key:  $meta->primary_key
table:        $meta->table
type:         $meta->type
unique_key:   $meta->unique_key
unsigned:     $meta->unsigned
zerofill:     $meta->zerofill
</pre>";
    $i++;
}
mysql_free_result($result);
?>
```

Notas

> **Nota** :
>
> Os nomes de campos retornados por esta função *diferenciam maiúsculas e minúsculas*.

> **Nota** :
>
> Se campos ou nomes de tabelas forem apelidados na consulta SQL, o nome do apelido será retornado. O nome original pode ser recuperado, por exemplo, usando mysqli_result::fetch_field().

Veja também

- mysql_field_seek() - Define o ponteiro do resultado para o índice de campo especificado

mysql_fetch_lengths

mysql_fetch_lengths

(PHP 4, PHP 5)

mysql_fetch_lengths — Obtém o tamanho de cada saída de um resultado

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_fetch_lengths()
> - PDOStatement::getColumnMeta()

Descrição

function **mysql_fetch_lengths**(resource `$result`): array|false

Retorna um array que corresponde aos tamanhos de cada campo na ultima linha obtida pelo MySQL.

mysql_fetch_lengths() guarda o tamanho de cada coluna do resultado na última linha retornada por mysql_fetch_row(), mysql_fetch_assoc(), mysql_fetch_array(), e mysql_fetch_object() em um array, começando no índice 0.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

Valor Retornado

Um array de tamanhos em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo **mysql_fetch_lengths()**

```php
<?php
$result = mysql_query("SELECT id,email FROM people WHERE id = '42'");
if (!$result) {
    echo 'Não foi possível executar a consulta: ' . mysql_error();
    exit;
}
$row     = mysql_fetch_assoc($result);
$lengths = mysql_fetch_lengths($result);

print_r($row);
print_r($lengths);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [id] => 42
    [email] => user@example.com
)
Array
```

```
(
    [0] => 2
    [1] => 16
)
```

Veja também

- mysql_field_len() - Retorna o tamanho do campo especificado
- mysql_fetch_row() - Obtém uma linha como um array numérico
- strlen() - Retorna o tamanho de uma string

mysql_fetch_object

mysql_fetch_object

(PHP 4, PHP 5)

mysql_fetch_object — Obtém o resultado de uma linha como um objeto

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_fetch_object()
> - PDOStatement::fetch() com `mode` como **`PDO::FETCH_OBJ`**

Descrição

function **mysql_fetch_object**(resource `$result`, string `$class_name = ?`, array `$params = ?`): object

Retorna um objeto com propriedades que correspondem a linha obtida e move o ponteiro interno dos dados adiante.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

`class_name`

O nome de classe para instanciar, definir as propriedades e retornar. Se não for especificado, um objeto stdClass é retornado.

`params`

Um array opcional de parâmetros para passar para o construtor do objeto `class_name`.

Valor Retornado

Retorna um object com propriedades strings que correspondem a linha obtida, ou `false` se não houverem mais linhas.

Examples

Exemplo #1 Exemplo mysql_fetch_object()

```php
<?php
mysql_connect("hostname", "user", "password");
mysql_select_db("mydb");
$result = mysql_query("select * from mytable");
while ($row = mysql_fetch_object($result)) {
    echo $row->user_id;
    echo $row->fullname;
}
mysql_free_result($result);
?>
```

Exemplo #2 Exemplo mysql_fetch_object()

```php
<?php
class foo {
    public $name;
}

mysql_connect("hostname", "user", "password");
mysql_select_db("mydb");

$result = mysql_query("select name from mytable limit 1");
$obj = mysql_fetch_object($result, 'foo');
var_dump($obj);
?>
```

Notas

> **Nota** : **Performance**
>
> Em termos de velocidade, a função é idêntica à mysql_fetch_array(), e quase tão rápida quanto mysql_fetch_row() (a diferença é insignificante).

> **Nota** :
>
> **mysql_fetch_object()** é similar a mysql_fetch_array(), com uma diferença - um objeto é retornado, ao invés de um array. Indiretamente, isto significa que você pode acessar os dados apenas pelo nome dos campos, e não por seus índices (números são ilegais como nomes de propriedades).

> **Nota** :
>
> Os nomes de campos retornados por esta função *diferenciam maiúsculas e minúsculas*.

> **Nota** :

> Esta função define campos NULL como o valor `null` do PHP.

Veja também

- mysql_fetch_array() - Obtém uma linha de resultado como um array associativo, numérico, ou ambos
- mysql_fetch_assoc() - Obtém uma linha do resultado como um array associativo
- mysql_fetch_row() - Obtém uma linha como um array numérico
- mysql_data_seek() - Move o ponteiro interno do resultado
- mysql_query() - Envia uma consulta MySQL

mysql_fetch_row

mysql_fetch_row

(PHP 4, PHP 5)

mysql_fetch_row — Obtém uma linha como um array numérico

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_fetch_row()
> - PDOStatement::fetch() com `mode` como `PDO::FETCH_NUM`

Descrição

function **mysql_fetch_row**(resource `$result`): array

Retorna um array numérico que corresponde a linha obtida e move o ponteiro interno dos dados adiante.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

Valor Retornado

Retorna um array numérico de strings que corresponde linha obtida, ou `false` se não houverem mais linhas.

mysql_fetch_row() obtém uma linha de dados do resultado associado ao identificador do resultado especificado. A linha é retornada como um array. Cada coluna do resultado é armazenado em um índice do array, começando no índice 0.

Examples

Exemplo #1 Obtendo uma linha com mysql_fetch_row()

```
<?php
$result = mysql_query("SELECT id,email FROM people WHERE id = '42'");
if (!$result) {
    echo 'Não foi possível executar a consulta: ' . mysql_error();
```

```
    exit;
}
$row = mysql_fetch_row($result);

echo $row[0]; // 42
echo $row[1]; // o valor do campo email
?>
```

Notas

> **Nota** :
>
> Esta função define campos NULL como o valor `null` do PHP.

Veja também

- mysql_fetch_array() - Obtém uma linha de resultado como um array associativo, numérico, ou ambos
- mysql_fetch_assoc() - Obtém uma linha do resultado como um array associativo
- mysql_fetch_object() - Obtém o resultado de uma linha como um objeto
- mysql_data_seek() - Move o ponteiro interno do resultado
- mysql_fetch_lengths() - Obtém o tamanho de cada saída de um resultado
- mysql_result() - Retorna dados do resultado

mysql_field_flags

mysql_field_flags

(PHP 4, PHP 5)

mysql_field_flags — Obtém as opções associadas ao campo especificado em um resultado

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_fetch_field_direct() [opções]
> - PDOStatement::getColumnMeta() [opções]

Descrição

function **mysql_field_flags**(resource `$result`, int `$field_offset`): string|false

mysql_field_flags() retorna as opções do campo especificado. As opções são mostradas como uma única palavra por opção separada por um único espaço, de forma a permitir a divisão do valor retornado com o uso da função explode().

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

`field_offset`

O deslocamento numérico do campo. `field_offset` começa em `0`. Se `field_offset` não existir, um erro de nível `E_WARNING` também será emitido.

Valor Retornado

Retorna uma string com as opções associadas ao resultado ou `false` em caso de falha.

As seguintes opções são reportadas, se a versão do MySQL for atual o suficiente para suportá-las: `"not_null"`, `"primary_key"`, `"unique_key"`, `"multiple_key"`, `"blob"`, `"unsigned"`, `"zerofill"`, `"binary"`, `"enum"`, `"auto_increment"` e `"timestamp"`.

Examples

Exemplo #1 Um exemplo de mysql_field_flags()

```php
<?php
$result = mysql_query("SELECT id,email FROM people WHERE id = '42'");
if (!$result) {
    echo 'Não foi possível executar a consulta: ' . mysql_error();
    exit;
}
$flags = mysql_field_flags($result, 0);

echo $flags;
print_r(explode(' ', $flags));
?>
```

O exemplo acima produzirá algo semelhante a:

```
not_null primary_key auto_increment
Array
(
    [0] => not_null
    [1] => primary_key
    [2] => auto_increment
)
```

Notas

> **Nota** :
>
> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado: **mysql_fieldflags()**

Veja também

- mysql_field_type() - Obtém o tipo do campo especificado em um resultado

- mysql_field_len() - Retorna o tamanho do campo especificado

mysql_field_len

mysql_field_len

(PHP 4, PHP 5)

mysql_field_len — Retorna o tamanho do campo especificado

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_fetch_field_direct() [length]
> - PDOStatement::getColumnMeta() [len]

Descrição

function **mysql_field_len**(resource `$result`, int `$field_offset`): int|false

mysql_field_len() retorna o tamanho do campo especificado.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

`field_offset`

O deslocamento numérico do campo. `field_offset` começa em `0`. Se `field_offset` não existir, um erro de nível `E_WARNING` também será emitido.

Valor Retornado

O tamanho do campo especificado em sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de mysql_field_len()

```
<?php
$result = mysql_query("SELECT id,email FROM people WHERE id = '42'");
if (!$result) {
    echo 'Não foi possível executar a consulta: ' . mysql_error();
    exit;
}

// Obtendo o comprimento do campo 'id' como especificado no
// esquema do banco de dados.
$length = mysql_field_len($result, 0);
echo $length;
?>
```

Notas

> **Nota** :
>
> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado: **mysql_fieldlen()**

Veja também

- mysql_fetch_lengths() - Obtém o tamanho de cada saída de um resultado
- strlen() - Retorna o tamanho de uma string

mysql_field_name

mysql_field_name

(PHP 4, PHP 5)

mysql_field_name — Obtém o nome do campo especificado em um resultado

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_fetch_field_direct() [name] ou [orgname]
> - PDOStatement::getColumnMeta() [name]

Descrição

function **mysql_field_name**(resource `$result`, int `$field_offset`): string|false

mysql_field_name() retorna o nome do campo do índice especificado.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

`field_offset`

O deslocamento numérico do campo. `field_offset` começa em `0`. Se `field_offset` não existir, um erro de nível `E_WARNING` também será emitido.

Valor Retornado

O nome do campo do índice especificado emj caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo mysql_field_name()

```php
<?php
/* A tabela users consiste em três campos:
 *   user_id
 *   username
 *   password.
 */
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar ao servidor MySQL: ' . mysql_error());
}
$dbname = 'mydb';
$db_selected = mysql_select_db($dbname, $link);
if (!$db_selected) {
    die("Não foi possível selecionar $dbname: " . mysql_error());
}
$res = mysql_query('select * from users', $link);

echo mysql_field_name($res, 0) . "\n";
echo mysql_field_name($res, 2);
?>
```

O exemplo acima produzirá:

```
user_id
password
```

Notas

> **Nota** :
>
> Os nomes de campos retornados por esta função *diferenciam maiúsculas e minúsculas*.

> **Nota** :
>
> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado:
> **mysql_fieldname()**

Veja também

- mysql_field_type() - Obtém o tipo do campo especificado em um resultado
- mysql_field_len() - Retorna o tamanho do campo especificado

mysql_field_seek

mysql_field_seek

(PHP 4, PHP 5)

mysql_field_seek — Define o ponteiro do resultado para o índice de campo especificado

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_field_seek()
> - PDOStatement::fetch() usando os parâmetros `cursor_orientation` e `offset`

Descrição

function **mysql_field_seek**(resource `$result`, int `$field_offset`): bool

Move para o índice do campo especificado. Se a próxima chamada a mysql_fetch_field() não incluir um índice, o índice de campo especificado em **mysql_field_seek()** será retornado.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

`field_offset`

O deslocamento numérico do campo. `field_offset` começa em `0`. Se `field_offset` não existir, um erro de nível `E_WARNING` também será emitido.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Veja também

- mysql_fetch_field() - Obtém informações de coluna de um resultado e retorna como um objeto

mysql_field_table

mysql_field_table

(PHP 4, PHP 5)

mysql_field_table — Obtém o nome da tabela na qual o campo especificado esta

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_fetch_field_direct() [table] ou [orgtable]
> - PDOStatement::getColumnMeta() [table]

Descrição

function **mysql_field_table**(resource `$result`, int `$field_offset`): string

Retorna o nome da tabela na qual o campo especificado está.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

`field_offset`

O deslocamento numérico do campo. `field_offset` começa em `0`. Se `field_offset` não existir, um erro de nível `E_WARNING` também será emitido.

Valor Retornado

O nome da tabela em caso de sucesso.

Examples

Exemplo #1 Exemplo de mysql_field_table()

```php
<?php

$query = "SELECT account.*, country.* FROM account, country WHERE country.name = 'Portugal' AND
account.country_id = country.id";

// obtém resultado do banco de dados
$result = mysql_query($query);

// Lista o nome da tabela e depois o nome do campo
for ($i = 0; $i < mysql_num_fields($result); ++$i) {
    $table = mysql_field_table($result, $i);
    $field = mysql_field_name($result, $i);

    echo  "$table: $field\n";
}

?>
```

Notas

> **Nota** :
>
> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado: **mysql_fieldtable()**

Veja também

- mysql_list_tables() - Lista as tabelas em um banco de dados MySQL

mysql_field_type

mysql_field_type

(PHP 4, PHP 5)

mysql_field_type — Obtém o tipo do campo especificado em um resultado

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_fetch_field_direct() [type]
> - PDOStatement::getColumnMeta() [driver:decl_type] ou [pdo_type]

Descrição

function **mysql_field_type**(resource `$result`, int `$field_offset`): string

mysql_field_type() é similar a função mysql_field_name(). Os argumentos são idênticos, mas é o tipo do campo que é retornado.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

`field_offset`

O deslocamento numérico do campo. `field_offset` começa em `0`. Se `field_offset` não existir, um erro de nível `E_WARNING` também será emitido.

Valor Retornado

O tipo de campo retornado será um de `"int"`, `"real"`, `"string"`, `"blob"` e outros como detalhado na » documentação do MySQL.

Examples

Exemplo #1 Exemplo de **mysql_field_type()**

```php
<?php
mysql_connect("localhost", "mysql_username", "mysql_password");
mysql_select_db("mysql");
$result = mysql_query("SELECT * FROM func");
$fields = mysql_num_fields($result);
$rows   = mysql_num_rows($result);
$table  = mysql_field_table($result, 0);
echo "Sua tabela '" . $table . "' tem " . $fields . " campos e " . $rows . " registro(s)\n";
echo "A tabela tem os seguintes campos:\n";
for ($i=0; $i < $fields; $i++) {
    $type  = mysql_field_type($result, $i);
    $name  = mysql_field_name($result, $i);
    $len   = mysql_field_len($result, $i);
    $flags = mysql_field_flags($result, $i);
    echo $type . " " . $name . " " . $len . " " . $flags . "\n";
}
mysql_free_result($result);
```

```
mysql_close();
?>
```

O exemplo acima produzirá algo semelhante a:

```
Sua tabela 'func' tem 4 campos e 1 registro(s)
A tabela tem os seguintes campos:
string name 64 not_null primary_key binary
int ret 1 not_null
string dl 128 not_null
string type 9 not_null enum
```

Notas

> **Nota** :
>
> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado: **mysql_fieldtype()**

Veja também

- mysql_field_name() - Obtém o nome do campo especificado em um resultado
- mysql_field_len() - Retorna o tamanho do campo especificado

mysql_free_result

mysql_free_result

(PHP 4, PHP 5)

mysql_free_result — Libera memória usada pelo resultado

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_free_result()
> - Atribuir o valor `null` ao objeto PDO ou PDOStatement::closeCursor()

Descrição

function **mysql_free_result**(resource `$result`): bool

mysql_free_result() liberará toda a memória associada ao identificador de resultado `result`.

mysql_free_result() apenas precisa ser chamada se houver preocupação sobre a quantidade de memória usada em consultas que retornam grandes conjuntos de resultados. Toda a memória associada ao resultado é liberada

automaticamente no final da execução do script.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Se for usado algo que não seja um recurso para o parâmetro `result`, um erro de nível E_WARNING será emitido. É importante observar que mysql_query() somente retorna um resource para consultas SELECT, SHOW, EXPLAIN e DESCRIBE.

Examples

Exemplo #1 Um exemplo de mysql_free_result()

```php
<?php
$result = mysql_query("SELECT id,email FROM people WHERE id = '42'");
if (!$result) {
    echo 'Não foi possível executar a consulta: ' . mysql_error();
    exit;
}
/* Usa o resultado assumindo que não vamos precisar mais dele depois */
$row = mysql_fetch_assoc($result);

/* Agora a memória do resultado é liberada e a execução do script continua */
mysql_free_result($result);

echo $row['id'];
echo $row['email'];
?>
```

Notas

> **Nota** :
>
> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado: **mysql_freeresult()**

Veja também

- mysql_query() - Envia uma consulta MySQL
- is_resource() - Verifica se uma variável é um recurso

mysql_get_client_info

mysql_get_client_info

(PHP 4 >= 4.0.5, PHP 5)

mysql_get_client_info — Obtém informações do cliente MySQL

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_get_client_info()
> - PDO::getAttribute() com `attribute` como **`PDO::ATTR_CLIENT_VERSION`**

Descrição

function **mysql_get_client_info**(): string

mysql_get_client_info() retorna uma string que representa a versão da biblioteca do cliente.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

A versão do cliente MySQL.

Examples

Exemplo #1 Exemplo mysql_get_client_info()

```php
<?php
printf("MySQL client info: %s\n", mysql_get_client_info());
?>
```

O exemplo acima produzirá algo semelhante a:

```
MySQL client info: 3.23.39
```

Veja também

- mysql_get_host_info() - Obtém informações do servidor MySQL
- mysql_get_proto_info() - Obtém informações do protocolo MySQL
- mysql_get_server_info() - Obtém informações do servidor MySQL

mysql_get_host_info

mysql_get_host_info

(PHP 4 >= 4.0.5, PHP 5)

mysql_get_host_info — Obtém informações do servidor MySQL

> **Aviso**

> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_get_host_info()
> - PDO::getAttribute() com `attribute` como **`PDO::ATTR_CONNECTION_STATUS`**

Descrição

function **mysql_get_host_info**(resource `$link_identifier` = NULL): string|false

Descreve o tipo de conexão em uso para a conexão, incluindo o nome do servidor.

Parâmetros

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível **`E_WARNING`** será gerado.

Valor Retornado

Retorna uma string descrevendo o tipo da conexão MySQL em uso para a conexão ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo **mysql_get_host_info()**

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}
printf("MySQL host info: %s\n", mysql_get_host_info());
?>
```

O exemplo acima produzirá algo semelhante a:

```
MySQL host info: Localhost via UNIX socket
```

Veja também

- mysql_get_client_info() - Obtém informações do cliente MySQL
- mysql_get_proto_info() - Obtém informações do protocolo MySQL
- mysql_get_server_info() - Obtém informações do servidor MySQL

mysql_get_proto_info

mysql_get_proto_info

(PHP 4 >= 4.0.5, PHP 5)

mysql_get_proto_info — Obtém informações do protocolo MySQL

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_get_proto_info()

Descrição

function **mysql_get_proto_info**(resource `$link_identifier` = NULL): int|false

Obtém o protocolo MySQL.

Parâmetros

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

Retorna o protocolo MySQL em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo mysql_get_proto_info()

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}
printf("MySQL protocol version: %s\n", mysql_get_proto_info());
?>
```

O exemplo acima produzirá algo semelhante a:

```
MySQL protocol version: 10
```

Veja também

- mysql_get_client_info() - Obtém informações do cliente MySQL
- mysql_get_host_info() - Obtém informações do servidor MySQL
- mysql_get_server_info() - Obtém informações do servidor MySQL

mysql_get_server_info

mysql_get_server_info

(PHP 4 >= 4.0.5, PHP 5)

mysql_get_server_info — Obtém informações do servidor MySQL

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_get_server_info()
> - PDO::getAttribute() com `attribute` como **`PDO::ATTR_SERVER_VERSION`**

Descrição

function **mysql_get_server_info**(resource `$link_identifier` = NULL): string|false

Obtém a versão do servidor MySQL.

Parâmetros

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível **`E_WARNING`** será gerado.

Valor Retornado

Retorna a versão do servidor MySQL em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo mysql_get_server_info()

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}
printf("Versão do servidor MySQL: %s\n", mysql_get_server_info());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Versão do servidor MySQL: 4.0.1-alpha
```

Veja também

- mysql_get_client_info() - Obtém informações do cliente MySQL
- mysql_get_host_info() - Obtém informações do servidor MySQL
- mysql_get_proto_info() - Obtém informações do protocolo MySQL
- phpversion() - Obtém a versão atual do PHP

mysql_info

mysql_info

(PHP 4 >= 4.3.0, PHP 5)

mysql_info — Obtém informação sobre a consulta mais recente

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_info()

Descrição

function **mysql_info**(resource `$link_identifier` = NULL): string

Retorna informação detalhada sobre a última consulta.

Parâmetros

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

Retorna informação sobre o comando em caso de sucesso, ou `false` em caso de falha. Confira o exemplo abaixo para ver quais comandos disponibilizam a informação e qual pode ser o valor retornado. Comandos que não estão listados irão retornar `false`.

Examples

Exemplo #1 Comandos MySQL relevantes

Comandos que retornam valores de string. Os números são apenas para proposito de ilustração; seus valores irão corresponder à consulta.

```
INSERT INTO ... SELECT ...
String format: Records: 23 Duplicates: 0 Warnings: 0
INSERT INTO ... VALUES (...),(...),(...)...
String format: Records: 37 Duplicates: 0 Warnings: 0
LOAD DATA INFILE ...
String format: Records: 42 Deleted: 0 Skipped: 0 Warnings: 0
ALTER TABLE
String format: Records: 60 Duplicates: 0 Warnings: 0
UPDATE
String format: Rows matched: 65 Changed: 65 Warnings: 0
```

Notas

> **Nota** :
>
> **mysql_info()** retornará um valor diferente de `false` para comandos INSERT ... VALUES apenas se múltiplas listas de valores forem especificadas no comando.

Veja também

- mysql_affected_rows() - Obtém o número de linhas afetadas na operação anterior do MySQL
- mysql_insert_id() - Obtém o ID gerado pela operação INSERT anterior
- mysql_stat() - Obtém o status atual do sistema

mysql_insert_id

mysql_insert_id

(PHP 4, PHP 5)

mysql_insert_id — Obtém o ID gerado pela operação INSERT anterior

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_insert_id()
> - PDO::lastInsertId()

Descrição

function **mysql_insert_id**(resource `$link_identifier` = NULL): int

Obtém o ID gerado para uma coluna AUTO_INCREMENT pela consulta anterior (normalmente INSERT).

Parâmetros

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma

como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

O ID gerado para uma coluna AUTO_INCREMENT pela consulta anterior em caso de sucesso, `0` se a consulta anterior não gerou um valor AUTO_INCREMENT, ou `false` se não foi estabelecido conexão com o MySQL.

Examples

Exemplo #1 Exemplo mysql_insert_id()

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}
mysql_select_db('mydb');

mysql_query("INSERT INTO mytable (product) values ('kossu')");
printf("Last inserted record has id %d\n", mysql_insert_id());
?>
```

Notas

> **Cuidado**
>
> **mysql_insert_id()** irá converter o tipo de retorno da API nativa em C do MySQL `mysql_insert_id()` para um tipo `long` (chamado int no PHP). Se a sua coluna AUTO_INCREMENT for uma coluna do tipo BIGINT (64 bits), a conversão pode resultar em um valor incorreto. Ao invés disto, use a função LAST_INSERT_ID() interna do MySQL em uma consulta SQL. Para mais informações sobre valores máximos de inteiros do PHP, visite a documentação integer.

> **Nota** :
>
> Devido **mysql_insert_id()** agir sobre a última consulta realizada, tenha certeza de chamar **mysql_insert_id()** imediatamente após a consulta que gerou o valor.

> **Nota** :
>
> O valor da função SQL do MySQL `LAST_INSERT_ID()` sempre contém o valor AUTO_INCREMENT mais recentemente gerado e não é reiniciado entre as consultas.

Veja também

- mysql_query() - Envia uma consulta MySQL
- mysql_info() - Obtém informação sobre a consulta mais recente

mysql_list_dbs

mysql_list_dbs

(PHP 4, PHP 5)

mysql_list_dbs — Lista os bancos de dados disponíveis em um servidor MySQL

> **Aviso**
>
> Esta função foi descontinuada a partir do PHP 5.4.0 e ela e a extensão MySQL original foram removidas a partir do PHP 7.0.0. Em vez disso, use as extensões MySQLi ou PDO_MySQL, que são desenvolvidas ativamente. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - SQL Query: `SHOW DATABASES`

Descrição

function **mysql_list_dbs**(resource `$link_identifier` = NULL): resource

Retorna um ponteiro de resultado contendo os bancos de dados disponíveis atualmente no servidor MySQL.

Parâmetros

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível **`E_WARNING`** será gerado.

Valor Retornado

Retorna um ponteiro de resultado resource em caso de sucesso, ou **`false`** em caso de falha. Use a função mysql_tablename() para lidar com este ponteiro de resultado, ou qualquer função para tabelas com resultados, como mysql_fetch_array().

Examples

Exemplo #1 Exemplo **mysql_list_dbs()**

```php
<?php
// Uso sem mysql_list_dbs()
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
$res = mysql_query("SHOW DATABASES");

while ($row = mysql_fetch_assoc($res)) {
    echo $row['Database'] . "\n";
}

// Descontinuado a partir do PHP 5.4.0
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
$db_list = mysql_list_dbs($link);

while ($row = mysql_fetch_object($db_list)) {
     echo $row->Database . "\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
database1
database2
database3
```

Notas

> **Nota** :
>
> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado: **mysql_listdbs()**

Veja também

- mysql_db_name() - Obtém o nome do banco de dados a partir da chamada à mysql_list_dbs
- mysql_select_db() - Seleciona um banco de dados MySQL

mysql_list_fields

mysql_list_fields

(PHP 4, PHP 5)

mysql_list_fields — Lista os campos de uma tabela MySQL

> **Aviso**
>
> Esta função foi descontinuada a partir do PHP 5.4.0 e ela e a extensão MySQL original foram removidas a partir do PHP 7.0.0. Em vez disso, use as extensões MySQLi ou PDO_MySQL, que são desenvolvidas ativamente. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - Consulta SQL: `SHOW COLUMNS FROM sometable`

Descrição

function **mysql_list_fields**(string `$database_name`, string `$table_name`, resource `$link_identifier` = NULL): resource

Obtém informações sobre a tabela informada.

Esta função foi descontinuada. É preferrível usar mysql_query() para realizar uma consulta SQL `SHOW COLUMNS FROM table [LIKE 'name']`.

Parâmetros

`database_name`

O nome do banco de dados que está sendo consultado.

`table_name`

O nome da tabela que esta sendo consultada.

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível **`E_WARNING`** será gerado.

Valor Retornado

Um ponteiro de recurso resource em caso de sucesso, ou **`false`** em caso de falha.

O ponteiro de resultado pode ser usado com mysql_field_flags(), mysql_field_len(), mysql_field_name() e mysql_field_type().

Examples

Exemplo #1 Alternativa para a função descontinuada mysql_list_fields()

```php
<?php
$result = mysql_query("SHOW COLUMNS FROM sometable");
if (!$result) {
    echo 'Não foi possível executar a consulta: ' . mysql_error();
    exit;
}
if (mysql_num_rows($result) > 0) {
    while ($row = mysql_fetch_assoc($result)) {
        print_r($row);
    }
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [Field] => id
    [Type] => int(7)
    [Null] =>
    [Key] => PRI
    [Default] =>
    [Extra] => auto_increment
)
Array
(
    [Field] => email
    [Type] => varchar(100)
    [Null] =>
    [Key] =>
    [Default] =>
    [Extra] =>
)
```

Notas

> **Nota** :
>
> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado: **mysql_listfields()**

Veja também

- mysql_field_flags() - Obtém as opções associadas ao campo especificado em um resultado
- mysql_info() - Obtém informação sobre a consulta mais recente

mysql_list_processes

mysql_list_processes

(PHP 4 >= 4.3.0, PHP 5)

mysql_list_processes — Lista os processos MySQL

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_thread_id()

Descrição

function **mysql_list_processes**(resource `$link_identifier` = NULL): resource|false

Obtém as threads atuais do servidor MySQL.

Parâmetros

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

Um ponteiro de resultado resource em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo mysql_list_processes()

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');

$result = mysql_list_processes($link);
while ($row = mysql_fetch_assoc($result)){
    printf("%s %s %s %s %s\n", $row["Id"], $row["Host"], $row["db"],
        $row["Command"], $row["Time"]);
}
mysql_free_result($result);
?>
```

O exemplo acima produzirá algo semelhante a:

```
1 localhost test Processlist 0
4 localhost mysql sleep 5
```

Veja também

- mysql_thread_id() - Retorna o ID da thread atual
- mysql_stat() - Obtém o status atual do sistema

mysql_list_tables

mysql_list_tables

(PHP 4, PHP 5)

mysql_list_tables — Lista as tabelas em um banco de dados MySQL

> **Aviso**
>
> Esta função foi descontinuada a partir do PHP 4.3.0 e ela e a extensão MySQL original foram removidas a partir do PHP 7.0.0. Em vez disso, use as extensões MySQLi ou PDO_MySQL, que são desenvolvidas ativamente. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - SQL Query: `SHOW TABLES FROM dbname`

Descrição

function **mysql_list_tables**(string `$database`, resource `$link_identifier` = NULL): resource|false

Obtém uma lista de nomes de tabelas de um banco de dados MySQL.

Esta função foi preterida em favor de mysql_query() para emitir uma instrução SQL `SHOW TABLES [FROM db_name] [LIKE 'pattern']` em seu lugar.

Parâmetros

`database`

O nome do banco de dados

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

Um ponteiro de resultado do tipo resource em caso de sucesso ou `false` em caso de falha.

A função mysql_tablename() deve ser usada para navegar pelo ponteiro de resultados, ou qualquer função para tabelas de resultados, como mysql_fetch_array().

Examples

Exemplo #1 Exemplo alternativo de mysql_list_tables()

```php
<?php
$dbname = 'mysql_dbname';

if (!mysql_connect('mysql_host', 'mysql_user', 'mysql_password')) {
    echo 'Não foi possível conectar ao MySQL';
    exit;
}

$sql = "SHOW TABLES FROM $dbname";
$result = mysql_query($sql);

if (!$result) {
    echo "Erro de BD, não foi possível listar as tabelas\n";
    echo 'Erro do MySQL: ' . mysql_error();
    exit;
}

while ($row = mysql_fetch_row($result)) {
    echo "Tabela: {$row[0]}\n";
}

mysql_free_result($result);
?>
```

Notas

> **Nota** :
>
> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado: **mysql_listtables()**

Veja também

- mysql_list_dbs() - Lista os bancos de dados disponíveis em um servidor MySQL
- mysql_tablename() - Obtém o nome da tabela do campo

mysql_num_fields

mysql_num_fields

(PHP 4, PHP 5)

mysql_num_fields — Obtém o numero de campos em um resultado

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_num_fields()
> - PDOStatement::columnCount()

Descrição

function **mysql_num_fields**(resource `$result`): int|false

Obtém o numero de campos de uma consulta.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

Valor Retornado

Retorna o numero de campos em um conjunto de resultados resource em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo de mysql_num_fields()

```php
<?php
$result = mysql_query("SELECT id,email FROM people WHERE id = '42'");
if (!$result) {
    echo 'Não foi possível executar a consulta: ' . mysql_error();
    exit;
}

/* retorna 2 porque id,email === dois campos */
echo mysql_num_fields($result);
?>
```

Notas

> **Nota** :
>
> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado: **mysql_numfields()**

Veja também

- mysql_select_db() - Seleciona um banco de dados MySQL
- mysql_query() - Envia uma consulta MySQL
- mysql_fetch_field() - Obtém informações de coluna de um resultado e retorna como um objeto
- mysql_num_rows() - Obtém o número de linhas em um resultado

mysql_num_rows

mysql_num_rows

(PHP 4, PHP 5)

mysql_num_rows — Obtém o número de linhas em um resultado

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_num_rows()
> - mysqli_stmt_num_rows()
> - PDOStatement::rowCount()

Descrição

function **mysql_num_rows**(resource `$result`): int|false

Obtém o numero de linhas de um conjunto de resultados. Este comando é valido apenas para comandos como SELECT ou SHOW que realmente retornam um conjunto de resultados. Para obter o numero de linhas afetadas por uma consulta INSERT, UPDATE, REPLACE ou DELETE, use mysql_affected_rows().

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

Valor Retornado

O numero de linhas em um conjunto de resultados em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo mysql_num_rows()

```php
<?php

$link = mysql_connect("localhost", "mysql_user", "mysql_password");
mysql_select_db("database", $link);

$result = mysql_query("SELECT * FROM table1", $link);
$num_rows = mysql_num_rows($result);

echo "$num_rows Rows\n";
```

```
?>
```

Notas

> **Nota** :
>
> Se você usar mysql_unbuffered_query(), **mysql_num_rows()** não irá retornar o valor correto até que todas as linhas no conjunto de resultado tenham sido obtidas.

> **Nota** :
>
> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado: **mysql_numrows()**

Veja também

- mysql_affected_rows() - Obtém o número de linhas afetadas na operação anterior do MySQL
- mysql_connect() - Abre uma conexão a um servidor MySQL
- mysql_data_seek() - Move o ponteiro interno do resultado
- mysql_select_db() - Seleciona um banco de dados MySQL
- mysql_query() - Envia uma consulta MySQL

mysql_pconnect

mysql_pconnect

(PHP 4, PHP 5)

mysql_pconnect — Abre uma conexão persistente a um servidor MySQL

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_connect() com `p:` host prefix
> - PDO::__construct() com **`PDO::ATTR_PERSISTENT`** como uma opção de driver

Descrição

```
function mysql_pconnect(
    string $server = ini_get("mysql.default_host"),
    string $username = ini_get("mysql.default_user"),
    string $password = ini_get("mysql.default_password"),
    int $client_flags = 0
): resource
```

Estabelece uma conexão persistente com o servidor MySQL.

mysql_pconnect() funciona muito parecido com mysql_connect() com duas diferenças principais.

Primeiro, ao conectar,a função irá primeiro tentar encontrar uma conexão (persistente) que já esteja aberta com o mesmo servidor, nome de usuário e senha. Se for encontrada, um identificador para ela será retornado ao invés de ser aberta uma nova conexão.

Segundo, a conexão ao servidor SQL não será fechada quando a execução do script terminar. Ao invés disso, a conexão permanecerá aberta para uso futuro (mysql_close() não irá fechar conexões estabelecidas por **mysql_pconnect()**).

Este tipo de conexão portanto é chamado de 'persistente'.

Parâmetros

`server`

O servidor MySQL. Também pode incluir um numero de porta, por exemplo, "servidor:porta" ou um caminho para um soquete local, por exemplo, ":/caminho/para/soquete" para o servidor local.

Se a diretiva do PHP mysql.default_host não estiver definida (padrão), o valor padrão será 'localhost:3306'

`username`

O nome de usuário. O Valor padrão é o nome do usuário proprietário do processo no servidor.

`password`

A senha. O valor padrão é uma senha vazia.

`client_flags`

O parâmetro `client_flags` pode ser uma combinação das seguintes constantes: 128 (habilita uso de `LOAD DATA LOCAL`), **`MYSQL_CLIENT_SSL`**, **`MYSQL_CLIENT_COMPRESS`**, **`MYSQL_CLIENT_IGNORE_SPACE`** ou **`MYSQL_CLIENT_INTERACTIVE`**.

Valor Retornado

Retorna um identificador de conexão persistente MySQL em caso de sucesso, ou **`false`** em caso de falha.

Notas

> **Nota** :
>
> Note que este tipo de conexão funciona somente se a versão de módulo do PHP estiver sendo usada. Consulte a seção sobre Conexões Persistentes com Banco de Dados para mais informações.

> **Aviso**
>
> Usar conexões persistentes pode requerer alguns ajustes nas configurações do Apache e do MySQL para assegurar que o número máximo de conexões permitidas pelo MySQL não seja excedido.

Veja também

- mysql_connect() - Abre uma conexão a um servidor MySQL
- Conexões Persistentes com Banco de Dados

mysql_ping

mysql_ping

(PHP 4 >= 4.3.0, PHP 5)

mysql_ping — Pinga uma conexão com o servidor ou reconecta se não houver conexão

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_ping()

Descrição

function **mysql_ping**(resource `$link_identifier` = NULL): bool

Confere se a conexão com o servidor está funcionando ou não. Se ela tiver caído, tenta realizar uma conexão automática. Esta função pode ser usada em scripts que permanecem inativos por um longo tempo, para verificar se o servidor fechou ou não a conexão e reconectar se necessário.

> **Nota** :
>
> A reconexão automática está desabilitada por padrão em versões do Mysql >= 5.0.3.

Parâmetros

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível **`E_WARNING`** será gerado.

Valor Retornado

Retorna **`true`** se a conexão com o servidor MySQL estiver funcionando, caso contrário retorna **`false`**.

Examples

Exemplo #1 Exemplo mysql_ping()

```php
<?php
set_time_limit(0);

$conn = mysql_connect('localhost', 'usuario', 'senha');
$db   = mysql_select_db('mydb');

/* Supondo que esta consulta levará muito tempo */
$result = mysql_query($sql);
if (!$result) {
    echo 'Consulta #1 falhou, saindo.';
    exit;
}
```

```php
/* Certifique-se de que a conexão ainda esteja ativa; caso contrário, tente reconectar */
if (!mysql_ping($conn)) {
    echo 'Conexão perdida, saindo após consulta #1';
    exit;
}
mysql_free_result($result);

/* Então a conexão ainda está ativa, vamos executar outra consulta */
$result2 = mysql_query($sql2);
?>
```

Veja também

- mysql_thread_id() - Retorna o ID da thread atual
- mysql_list_processes() - Lista os processos MySQL

mysql_query

mysql_query

(PHP 4, PHP 5)

mysql_query — Envia uma consulta MySQL

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_query()
> - PDO::query()

Descrição

function **mysql_query**(string `$query`, resource `$link_identifier` = NULL): mixed

mysql_query() envia uma única consulta (consultas múltiplas não são suportadas) ao banco de dados atualmente ativo no servidor associado ao `link_identifier` especificado.

Parâmetros

`query`

A consulta SQL.

A consulta não deve terminar com um ponto e vírgula. Os dados dentro da consulta devem ser adequadamente escapados.

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

Para comandos SELECT, SHOW, DESCRIBE, EXPLAIN e outras instruções que retornam resultados, **mysql_query()** retorna um resource em caso de sucesso ou `false` em caso de erro.

Para outros tipos de instruções SQL, UPDATE, DELETE, DROP, etc, **mysql_query()** retorna `true` em caso de sucesso ou `false` em caso de erro.

O recurso de resultado retornado pode ser passado para mysql_fetch_array() e para outras funções para lidar com tabelas de resultados e para acessar os dados retornados.

Use mysql_num_rows() para obter quantas linhas foram retornadas para uma instrução SELECT ou mysql_affected_rows() para obter quantas linhas foram afetadas por uma instrução DELETE, INSERT, REPLACE ou UPDATE.

mysql_query() também falhará e retornará `false` se o usuário não tiver permissões para acessar a(s) tabela(s) referenciada(s) pela consulta.

Examples

Exemplo #1 Consulta Inválida

A seguinte consulta é sintaticamente inválida, assim **mysql_query()** falha e retorna `false`.

```php
<?php
$result = mysql_query('SELECT * WHERE 1=1');
if (!$result) {
    die('Consulta inválida: ' . mysql_error());
}

?>
```

Exemplo #2 Consulta Válida

A seguinte consulta é válida, assim **mysql_query()** retorna um resource.

```php
<?php
// Isto poderia ser fornecido pelo usuário, por exemplo
$firstname = 'fred';
$lastname  = 'fox';

// Formula a Consulta
// Esta é a melhor forma de realizar uma consulta SQL
// Para mais exemplo, consulte mysql_real_escape_string()
$query = sprintf("SELECT firstname, lastname, address, age FROM friends
    WHERE firstname='%s' AND lastname='%s'",
    mysql_real_escape_string($firstname),
    mysql_real_escape_string($lastname));

// Realiza a Consulta
$result = mysql_query($query);

// Verifica o resultado
// Isto mostra a consulta real enviada ao MySQL, e o erro. Útil para depuração.
if (!$result) {
    $message  = 'Consulta inválida: ' . mysql_error() . "\n";
    $message .= 'Consulta completa: ' . $query;
```

```
    die($message);
}

// Usa o resultado
// Tentar mostrar $result não permitirá acesso à informação no recurso
// Uma das funções de resultados da extensão mysql precisa ser usada
// Consulte também mysql_result(), mysql_fetch_array(), mysql_fetch_row(), etc.
while ($row = mysql_fetch_assoc($result)) {
    echo $row['firstname'];
    echo $row['lastname'];
    echo $row['address'];
    echo $row['age'];
}

// Libera os recursos associados com o conjunto de resultados
// Isto é feito automaticamente no final do script
mysql_free_result($result);
?>
```

Veja também

- mysql_connect() - Abre uma conexão a um servidor MySQL
- mysql_error() - Retorna o texto da mensagem de erro da operação MySQL anterior
- mysql_real_escape_string() - Escapa os caracteres especiais em uma string para uso em uma instrução SQL
- mysql_result() - Retorna dados do resultado
- mysql_fetch_assoc() - Obtém uma linha do resultado como um array associativo
- mysql_unbuffered_query() - Envia uma consulta SQL ao MySQL sem buscar e armazenar em buffer as linhas de resultados

mysql_real_escape_string

mysql_real_escape_string

(PHP 4 >= 4.3.0, PHP 5)

mysql_real_escape_string — Escapa os caracteres especiais em uma string para uso em uma instrução SQL

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_real_escape_string()
> - PDO::quote()

Descrição

function **mysql_real_escape_string**(string `$unescaped_string`, resource `$link_identifier` = NULL): string

Escapa caracteres especiais do parâmetro `unescaped_string`, levando em conta o conjunto de caracteres atual da conexão, de forma que seja seguro inseri-la na função mysql_query(). Se forem inseridos dados binários, esta função precisa ser usada.

mysql_real_escape_string() chama a função mysql_real_escape_string da biblioteca do MySQL, que prefixa com barras invertidas os seguintes caracteres: `\x00`, `\n`, `\r`, `\`, `'`, `"` e `\x1a`.

Esta função precisa sempre ser usada (com poucas exceções) para tornar os dados seguros antes de enviá-los em uma consulta ao MySQL.

> **Cuidado**
>
> **Segurança: o conjunto de caracteres padrão**
>
> O conjunto de caracteres precisa ser definido no nível do servidor ou com a função mysql_set_charset() da API para que afete a função **mysql_real_escape_string()**. Consulte a seção de conceitos sobre conjuntos de caracteres para mais informação.

Parâmetros

`unescaped_string`

A string a ser escapada.

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

Retorna a string escapada ou `false` em caso de erro.

Erros/Exceções

Executar esta função sem uma conexão MySQL presente emitirá um erro de nível `E_WARNING`. Esta função deve ser executada somente com uma conexão MySQL válida presente.

Examples

Exemplo #1 Exemplo de **mysql_real_escape_string()**

```php
<?php
// Connect
$link = mysql_connect('mysql_host', 'mysql_user', 'mysql_password')
    OR die(mysql_error());

// Query
$query = sprintf("SELECT * FROM users WHERE user='%s' AND password='%s'",
            mysql_real_escape_string($user),
            mysql_real_escape_string($password));
?>
```

Exemplo #2 **mysql_real_escape_string()** requer um exemplo de conexão

Este exemplo demonstra o que acontece se uma conexão MySQL não estiver presente ao chamar esta função.

```php
<?php
// Ainda não foi feita conexão ao MySQL

$lastname  = "O'Reilly";
```

```php
$_lastname = mysql_real_escape_string($lastname);

$query = "SELECT * FROM actors WHERE last_name = '$_lastname'";

var_dump($_lastname);
var_dump($query);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Warning: mysql_real_escape_string(): No such file or directory in /this/test/script.php on line 5
Warning: mysql_real_escape_string(): A link to the server could not be established in
/this/test/script.php on line 5

bool(false)
string(41) "SELECT * FROM actors WHERE last_name = ''"
```

Exemplo #3 Um exemplo de Ataque de Injeção SQL

```php
<?php
// Não verificamos $_POST['password'], ela pode conter qualquer coisa que o usuário quiser! Por
exemplo:
$_POST['username'] = 'aidan';
$_POST['password'] = "' OR ''='";

// Consulta o banco de dados para verificar se existe algum usuário correspondente
$query = "SELECT * FROM users WHERE user='{$_POST['username']}' AND password='{$_POST['password']}'";
mysql_query($query);

// Isto significa que a consulta enviada ao MySQL seria:
echo $query;
?>
```

A consulta enviada ao MySQL:

```
SELECT * FROM users WHERE user='aidan' AND password='' OR ''=''
```

Isto permitiria que qualquer pessoa fizesse login sem uma senha válida.

Notas

> **Nota** :
>
> Uma conexão MySQL é requerida antes que **mysql_real_escape_string()** seja usada, caso contrário um erro de nível `E_WARNING` será gerado e `false` é retornado. Se `link_identifier` não estiver definido, a últimoa

conexão MySQL será usada.

> **Nota** :
>
> Se esta função não for usada para escapar dados, a consulta ficará vulnerável a Ataques de Injeção SQL.

> **Nota** :
>
> **mysql_real_escape_string()** não escapa `%` e _. Estes caracteres são coringas no MySQL se combinados com `LIKE`, `GRANT` ou `REVOKE`.

Veja também

- mysql_set_charset() - Define o conjunto de caracteres do cliente
- mysql_client_encoding() - Retorna o nome do conjunto de caracteres

mysql_result

mysql_result

(PHP 4, PHP 5)

mysql_result — Retorna dados do resultado

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_data_seek() em conjunto com mysqli_field_seek() e mysqli_fetch_field()
> - PDOStatement::fetchColumn()

Descrição

function **mysql_result**(resource `$result`, int `$row`, mixed `$field` = 0): string

Retorna o conteúdo de uma célula do resultado MySQL.

Ao trabalhar com conjuntos grandes de resultados, deve ser considerado o uso de uma das funções que retornam uma linha inteira (especificadas abaixo). Como estas funções retornam o conteúdo de múltiplas células em uma chamada de função, elas são MUITO mais rápidas que **mysql_result()**. Observe também que especificar um índice numérico para o argumento de campo (field) é muito mais rápido que especificar um argumento com nome de campo ou nome_tabela.nome_de_campo.

Parâmetros

`result`

O resource do resultado que está sendo avaliado. Este resultado vem de uma chamada a mysql_query().

`row`

O número da linha do resultado que está sendo recuperado. Os números das linhas começam em `0`.

`field`

O nome ou posição do campo que está sendo recuperado.

Pode ser a posição do campo, o nome do campo ou o nome da tabela mais um ponto mais o nome do campo (nome_tabela.nome_campo). Se o nome da coluna tiver sido apelidado ('select foo as bar from...'), o apelido deve ser usado no lugar do nome da coluna. Se indefinido, o primeiro campo será recuperado.

Valor Retornado

O conteúdo de uma célula de um conjunto de resultado MySQL em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de **mysql_result()**

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não foi possível conectar: ' . mysql_error());
}
if (!mysql_select_db('database_name')) {
    die('Não foi possível selecionar o banco de dados: ' . mysql_error());
}
$result = mysql_query('SELECT name FROM work.employee');
if (!$result) {
    die('Não foi possível consultar:' . mysql_error());
}
echo mysql_result($result, 2); // mostra o nome do terceiro empregado (employee)

mysql_close($link);
?>
```

Notas

> **Nota** :
>
> Chamadas a **mysql_result()** não devem ser misturadas com chamadas a outras funções que lidam com o conjunto de resultados.

Veja também

- mysql_fetch_row() - Obtém uma linha como um array numérico
- mysql_fetch_array() - Obtém uma linha de resultado como um array associativo, numérico, ou ambos
- mysql_fetch_assoc() - Obtém uma linha do resultado como um array associativo
- mysql_fetch_object() - Obtém o resultado de uma linha como um objeto

mysql_select_db

mysql_select_db

(PHP 4, PHP 5)

mysql_select_db — Seleciona um banco de dados MySQL

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_select_db()
> - PDO::__construct() (parte da dsn)

Descrição

function **mysql_select_db**(string `$database_name`, resource `$link_identifier` = NULL): bool

Define o banco de dados ativo atual no servidor associado ao identificador de conexão especificado. Cada chamada subsequente a mysql_query() será feita no banco de dados ativo.

Parâmetros

`database_name`

O nome do banco de dados que será selecionado.

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível **`E_WARNING`** será gerado.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo de **mysql_select_db()**

```php
<?php

$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Não conectado : ' . mysql_error());
}

// faz de 'foo' o banco de dados atual
$db_selected = mysql_select_db('foo', $link);
if (!$db_selected) {
    die ('Não foi possível usar foo : ' . mysql_error());
}
?>
```

Notas

> **Nota** :

> Para compatibilidade com versões anteriores, o seguinte apelido descontinuado pode ser usado: **mysql_selectdb()**

Veja também

- mysql_connect() - Abre uma conexão a um servidor MySQL
- mysql_pconnect() - Abre uma conexão persistente a um servidor MySQL
- mysql_query() - Envia uma consulta MySQL

mysql_set_charset

mysql_set_charset

(PHP 5 >= 5.2.3)

mysql_set_charset — Define o conjunto de caracteres do cliente

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_set_charset()
> - PDO: Adicionar `charset` à string de conexão, por exemplo `charset=utf8`

Descrição

function **mysql_set_charset**(string `$charset`, resource `$link_identifier` = NULL): bool

Define o conjunto de caracteres padrão para a conexão atual.

Parâmetros

`charset`

Um nome válido de conjunto de caracteres.

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível **`E_WARNING`** será gerado.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Notas

> **Nota** :
>
> Esta função requer o MySQL 5.0.7 ou mais recente.

> **Nota** :
>
> Esta é a maneira preferida de alterar o conjunto de caracteres. Usar mysql_query() para configurá-lo (como em `SET NAMES utf8`) não é recomendado. Consulte a seção conceitos do conjunto de caracteres MySQL para obter mais informações.

Veja também

- Configurando conjuntos de caracteres no MySQL
- » Lista de conjuntos de caracteres suportados pelo MySQL
- mysql_client_encoding() - Retorna o nome do conjunto de caracteres

mysql_stat

mysql_stat

(PHP 4 >= 4.3.0, PHP 5)

mysql_stat — Obtém o status atual do sistema

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_stat()
> - PDO::getAttribute() com `attribute` igual a **`PDO::ATTR_SERVER_INFO`**

Descrição

function **mysql_stat**(resource `$link_identifier` = NULL): string

mysql_stat() retorna o status atual do servidor.

Parâmetros

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível **`E_WARNING`** será gerado.

Valor Retornado

Retorna uma string com o status de tempo de atividade, threads, consultas, tabelas abertas, tabelas de liberação e consultas por segundo. Para obter uma lista completa de outras variáveis de status, deve ser usado o comando SQL `SHOW STATUS`. Se `link_identifier` for inválido, **`null`** é retornado.

Examples

Exemplo #1 Exemplo de mysql_stat()

```php
<?php
$link   = mysql_connect('localhost', 'mysql_user', 'mysql_password');
$status = explode('  ', mysql_stat($link));
print_r($status);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => Uptime: 5380
    [1] => Threads: 2
    [2] => Questions: 1321299
    [3] => Slow queries: 0
    [4] => Opens: 26
    [5] => Flush tables: 1
    [6] => Open tables: 17
    [7] => Queries per second avg: 245.595
)
```

Exemplo #2 Exemplo alternativo a mysql_stat()

```php
<?php
$link   = mysql_connect('localhost', 'mysql_user', 'mysql_password');
$result = mysql_query('SHOW STATUS', $link);
while ($row = mysql_fetch_assoc($result)) {
    echo $row['Variable_name'] . ' = ' . $row['Value'] . "\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
back_log = 50
basedir = /usr/local/
bdb_cache_size = 8388600
bdb_log_buffer_size = 32768
bdb_home = /var/db/mysql/
bdb_max_lock = 10000
bdb_logdir =
bdb_shared_data = OFF
bdb_tmpdir = /var/tmp/
...
```

Veja também

- mysql_get_server_info() - Obtém informações do servidor MySQL
- mysql_list_processes() - Lista os processos MySQL

mysql_tablename

mysql_tablename

(PHP 4, PHP 5)

mysql_tablename — Obtém o nome da tabela do campo

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - Consulta SQL: `SHOW TABLES`

Descrição

function **mysql_tablename**(resource `$result`, int `$i`): string|false

Recupera o nome da tabela a partir de um resultado informado em `result`.

Esta função foi descontinuada. É preferível usar mysql_query() para emitir uma instrução SQL `SHOW TABLES [FROM db_name] [LIKE 'pattern']` em seu lugar.

Parâmetros

`result`

Um ponteiro de resultados do tipo resource que é retornado por mysql_list_tables().

`i`

O índice como um número inteiro (número da linha/tabela)

Valor Retornado

O nome da tabela em caso de sucesso ou **`false`** em caso de falha.

Use a função **mysql_tablename()** para percorrer este ponteiro de resultado, ou qualquer função para tabelas de resultados, como mysql_fetch_array().

Examples

Exemplo #1 Exemplo de mysql_tablename()

```php
<?php
mysql_connect("localhost", "mysql_user", "mysql_password");
$result = mysql_list_tables("mydb");
$num_rows = mysql_num_rows($result);
for ($i = 0; $i < $num_rows; $i++) {
    echo "Tabela: ", mysql_tablename($result, $i), "\n";
}
```

```php
mysql_free_result($result);
?>
```

Notas

> **Nota** :
>
> A função mysql_num_rows() pode ser usada para determinar o número de tabelas no ponteiro de resultados.

Veja também

- mysql_list_tables() - Lista as tabelas em um banco de dados MySQL
- mysql_field_table() - Obtém o nome da tabela na qual o campo especificado esta
- mysql_db_name() - Obtém o nome do banco de dados a partir da chamada à mysql_list_dbs

mysql_thread_id

mysql_thread_id

(PHP 4 >= 4.3.0, PHP 5)

mysql_thread_id — Retorna o ID da thread atual

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - mysqli_thread_id()

Descrição

function **mysql_thread_id**(resource `$link_identifier` = NULL): int|false

Recupera o ID da thread atual. Se a conexão for perdida e uma reconexão com mysql_ping() for executada, o ID da thread será alterado. Isso significa que é melhor recuperar o ID da thread apenas quando necessário.

Parâmetros

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

O ID da thread em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de mysql_thread_id()

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
$thread_id = mysql_thread_id($link);
if ($thread_id){
    printf("ID da thread atual é %d\n", $thread_id);
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
ID da thread atual é 73
```

Veja também

- mysql_ping() - Pinga uma conexão com o servidor ou reconecta se não houver conexão
- mysql_list_processes() - Lista os processos MySQL

mysql_unbuffered_query

mysql_unbuffered_query

(PHP 4 >= 4.0.6, PHP 5)

mysql_unbuffered_query — Envia uma consulta SQL ao MySQL sem buscar e armazenar em buffer as linhas de resultados

> **Aviso**
>
> Esta extensão foi descontinuada a partir do PHP 5.5.0 e foi removida no PHP 7.0.0. Em vez disso, as extensões MySQLi ou PDO_MySQL devem ser usadas. Veja também o guia MySQL: escolhendo uma API. Alternativas a esta função incluem:
>
> - Consulte: Consultas com e sem buffer

Descrição

function **mysql_unbuffered_query**(string `$query`, resource `$link_identifier` = NULL): resource

mysql_unbuffered_query() envia a consulta SQL `query` para o MySQL sem buscar e armazenar automaticamente as linhas de resultados como mysql_query() faz. Isso economiza uma quantidade considerável de memória com consultas SQL que produzem grandes conjuntos de resultados, e o conjunto de resultados já pode ser processado imediatamente após a primeira linha ter sido recuperada, pois não será necessário esperar até que a consulta SQL completa seja executada. Para usar **mysql_unbuffered_query()** enquanto múltiplas conexões de banco de dados estão abertas, o parâmetro opcional `link_identifier` deve ser especificado para identificar qual conexão deve ser usada.

Parâmetros

`query`

A consulta SQL a ser executada.

Dados dentro da consulta devem ser adequadamente escapados.

`link_identifier`

A conexão MySQL. Se o identificador da conexão não for especificado, a última conexão aberta por mysql_connect() será usada. Se não houver uma conexão anterior, haverá uma tentativa de criar uma como se mysql_connect() tivesse sido chamada sem argumentos. Se nenhuma conexão for encontrada ou estabelecida, um erro de nível `E_WARNING` será gerado.

Valor Retornado

Para instruções SELECT, SHOW, DESCRIBE ou EXPLAIN, **mysql_unbuffered_query()** retorna um resource em caso de sucesso, ou `false` em caso de erro.

Para outros tipos de instruções SQL, UPDATE, DELETE, DROP, etc., **mysql_unbuffered_query()** retorna `true` em caso de sucesso ou `false` em caso de erro.

Notas

> **Nota** :
>
> Os benefícios de **mysql_unbuffered_query()** têm um custo: mysql_num_rows() e mysql_data_seek() não podem ser usadas em um conjunto de resultados retornado de **mysql_unbuffered_query()** , até que todas as linhas sejam buscadas. Também é necessário buscar todas as linhas de resultados de uma consulta SQL sem buffer antes de enviar uma nova consulta SQL ao MySQL, usando o mesmo `link_identifier`.

Veja também

- mysql_query() - Envia uma consulta MySQL

Mysqlnd

Introdução

O Driver Nativo MySQL é um substituto para a biblioteca cliente MySQL (libmysqlclient). Driver Nativo MySQL faz parte das fontes oficiais do PHP a partir do PHP 5.3.0.

As extensões de banco de dados MySQL, `mysqli` e PDO MYSQL se comunicam com o servidor MySQL. No passado, isso era feito pela extensão usando os serviços fornecidos pela Biblioteca Cliente MySQL. As extensões foram compiladas na Biblioteca Cliente MySQL para usar seu protocolo cliente-servidor.

Com o Driver Nativo MySQL agora existe uma alternativa, pois as extensões do banco de dados MySQL podem ser compiladas para usar o Driver Nativo MySQL em vez da Biblioteca Cliente MySQL.

O Driver Nativo MySQL é escrito em C como uma extensão do PHP.

Visão Geral

O que o mysqlnd não é

Embora o Driver Nativo MySQL seja escrito como uma extensão PHP, é importante observar que ele não fornece uma nova API para o programador PHP. As APIs do programador para conectividade de banco de dados MySQL

são fornecidas pela extensão MySQL, `mysqli` e PDO MYSQL. Essas extensões agora podem usar os serviços do Driver Nativo MySQL para se comunicar com o servidor MySQL. Portanto, não se deve pensar no Driver Nativo MySQL como uma API.

Por que usá-lo?

A utilização do Driver Nativo MySQL oferece uma série de vantagens em relação à utilização da Biblioteca Cliente MySQL.

A antiga biblioteca cliente MySQL foi escrita pela MySQL AB (agora Oracle Corporation) e, portanto, foi lançada sob a licença MySQL. Isso acabou fazendo com que o suporte ao MySQL fosse desativado por padrão no PHP. Porém, o Driver Nativo MySQL foi desenvolvido como parte do projeto PHP, sendo portanto liberado sob a licença PHP. Isso elimina problemas de licenciamento que foram problemáticos no passado.

Além disso, no passado, era necessário construir as extensões de banco de dados MySQL com uma cópia da biblioteca cliente MySQL. Isso normalmente significava que era necessário ter o MySQL instalado em uma máquina onde o código-fonte PHP estivesse sendo compilado. Além disso, quando o aplicativo PHP estava em execução, as extensões do banco de dados MySQL chamavam o arquivo da biblioteca do cliente MySQL em tempo de execução, portanto, o arquivo precisava estar instalado no sistema. Com o Driver Nativo MySQL isso não acontece mais, pois está incluído como parte da distribuição padrão. Portanto, não é mais necessário o MySQL instalado para construir o PHP ou executar aplicações de banco de dados PHP.

Como o Driver Nativo MySQL é escrito como uma extensão do PHP, ele está fortemente acoplado ao funcionamento do PHP. Isso leva a ganhos de eficiência, principalmente no que diz respeito ao uso de memória, já que o driver utiliza o sistema de gerenciamento de memória PHP. Ele também suporta o limite de memória do PHP. Usar o Driver Nativo MySQL leva a um desempenho comparável ou melhor do que usar a Biblioteca Cliente MySQL, garantindo sempre o uso mais eficiente da memória. Um exemplo de eficiência de memória é o fato de que ao utilizar a Biblioteca Cliente MySQL, cada linha é armazenada na memória duas vezes, enquanto no Driver Nativo MySQL cada linha é armazenada apenas uma vez na memória.

> **Nota** : **Reportando o uso da memória**
>
> Como o Driver Nativo MySQL utiliza o sistema de gerenciamento de memória PHP, seu uso de memória pode ser rastreado com memory_get_usage(). Isso não é possível com a libmysqlclient porque ela usa a função C malloc().

Special features

O Driver Nativo MySQL também fornece alguns recursos especiais não disponíveis quando as extensões de banco de dados MySQL utilizam a Biblioteca Cliente MySQL. Esses recursos especiais estão listados abaixo:

- Conexões persistentes melhoradas
- A função especial mysqli_fetch_all()
- Chamadas para estatísticas de desempenho: mysqli_get_client_stats(), mysqli_get_connection_stats()

O recurso de estatísticas de desempenho pode ser muito útil na identificação de gargalos de desempenho.

O Driver Nativo MySQL também permite conexões persistentes quando utilizado com a extensão `mysqli`.

Suporte a SSL

Driver Nativo MySQL suporta SSL.

Suporte a Protocolo com Compressão

Driver Nativo MySQL suporta o protocolo cliente-servidor com compressão. A extensão `ext/mysqli`, se configurada para usar Driver Nativo MySQL, também pode tomar vantagem deste recurso. Observe que `PDO_MYSQL` *NÃO* suporta compressão quando usada junto com mysqlnd.

Suporte a Tubos Nomeados

Tubos nomeados podem ser usado para conexão em ambientes Windows.

Instalação

Instalação em Unix

Para usar o Driver Nativo MySQL, o PHP precisa ser construído especificando que as extensões do banco de dados MySQL são compiladas com suporte ao MySQL Native Driver. Isso é feito através de opções de configuração antes de compilar o código-fonte do PHP.

Por exemplo, para construir a extensão MySQL, `mysqli` e PDO MYSQL utilizando o Driver Nativo MySQL, seria dado o seguinte comando:

```
./configure --with-mysql=mysqlnd \
--with-mysqli=mysqlnd \
--with-pdo-mysql=mysqlnd \
[other options]
```

Instalação no Windows

Nas distribuições oficiais do PHP para Windows, o Driver Nativo MySQL está habilitado por padrão, portanto, nenhuma configuração adicional é necessária para usá-lo. Todas as extensões de banco de dados MySQL usarão o Driver Nativo MySQL neste caso.

Suporte ao plug-in de autenticação SHA-256

O Driver Nativo MySQL requer que a funcionalidade OpenSSL do PHP esteja carregada e habilitada para conexão ao MySQL através de contas que utilizam o Plugin de Autenticação MySQL SHA-256. Por exemplo, o PHP pode ser configurado usando:

```
./configure --with-mysql=mysqlnd \
--with-mysqli=mysqlnd \
--with-pdo-mysql=mysqlnd \
--with-openssl
[other options]
```

No Autotools, o suporte SSL estendido no `mysqlnd` é habilitado implicitamente ao construir junto com a extensão `openssl` usando a opção de configuração **--with-openssl**. Ao compilar sem a extensão `openssl`, a opção de configuração **--with-mysqlnd-ssl** pode ser usada para ativar explicitamente o suporte SSL estendido.

Registro de Alterações

Versão	Descrição
8.4.0	A opção de configuração **--with-mysqlnd-ssl** do Autotools foi adicionada para ativar explicitamente o suporte SSL estendido ao compilar sem a extensão `openssl`.

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de configuração do Driver Nativo MySQL

Nome	Padrão	Modificável	Registro de Alterações
mysqlnd.collect_statistics	"1"	INI_SYSTEM	
mysqlnd.collect_memory_statistics	"0"	INI_SYSTEM	
mysqlnd.debug	""	INI_SYSTEM	
mysqlnd.log_mask	0	INI_ALL	
mysqlnd.mempool_default_size	16000	INI_ALL	
mysqlnd.net_read_timeout	"86400"	INI_ALL	Antes do PHP 7.2.0 o padrão era "31536000" e a mutabilidade era INI_SYSTEM
mysqlnd.net_cmd_buffer_size	"4096"	INI_SYSTEM	
mysqlnd.net_read_buffer_size	"32768"	INI_SYSTEM	
mysqlnd.sha256_server_public_key	""	INI_PERDIR	
mysqlnd.trace_alloc	""	INI_SYSTEM	
mysqlnd.fetch_data_copy	0	INI_ALL	Removida a partir do PHP 8.1.0

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`mysqlnd.collect_statistics` bool

Permite a coleta de várias estatísticas de clientes que podem ser acessadas através de mysqli_get_client_stats() e mysqli_get_connection_stats() e também são mostradas na seção `mysqlnd` da saída da função phpinfo().

Esta configuração ativa todas as estatísticas do Driver Nativo MySQL exceto aquelas relacionadas ao gerenciamento de memória.

`mysqlnd.collect_memory_statistics` bool

Permite a coleta de várias estatísticas de clientes que podem ser acessadas através de mysqli_get_client_stats() e mysqli_get_connection_stats() e também são mostradas na seção `mysqlnd` da saída da função phpinfo().

Esta definição de configuração ativa as estatísticas de gerenciamento de memória dentro do conjunto geral de estatísticas do Driver Nativo MySQL.

`mysqlnd.debug` string

Grava a comunicação de todas as extenões que usam `mysqlnd` para o arquivo de registro especificado.

O formato da diretiva é `mysqlnd.debug = "opção1[,parâmetro_opção1][:opção2[,parâmetro_opção2]]"`.

As opções para a string de formato são as seguintes:

- A[,arquivo] - Acrescenta saída de rastreamento ao arquivo especificado. Também garante que os dados sejam gravados após cada gravação. Isso é feito fechando e reabrindo o arquivo de

rastreamento (isso é lento). Isso ajuda a garantir um arquivo de registro completo caso o aplicativo falhe.

- a[,arquivo] - Acrescenta saída de rastreamento ao arquivo especificado.
- d - Ativa a saída de macros DBUG_<N> para o estado atual. Pode ser seguido por uma lista de palavras-chave que seleciona a saída apenas para as macros DBUG com essa palavra-chave. Uma lista vazia de palavras-chave implica em saída para todas as macros.
- f[,funções] - Limita as ações do depurador à lista de funções especificada. Uma lista vazia de funções implica em que todas as funções são selecionadas.
- F - Marca cada linha de saída do depurador com o nome do arquivo de origem que contém a macro que causa a saída.
- i - Marca cada linha de saída do depurador com o PID do processo atual.
- L - Marca cada linha de saída do depurador com o nome do número da linha do arquivo de origem da macro que causa a saída.
- n - Marca cada linha de saída do depurador com a profundidade de aninhamento da função atual.
- o[,arquivo] - Similar a a[,arquivo] mas sobrescreve o arquivo antigo, e não anexa.
- O[,arquivo] - Similar a A[,arquivo] mas sobrescreve o arquivo antigo, e não anexa.
- t[,N] - Ativa o rastreamento de fluxo de controle de função. A profundidade máxima de aninhamento é especificada por N e o padrão é 200.
- x - Este opção ativa a criação de perfil de código.
- m - Rastreia chamadas relacionadas à alocação e desalocação de memória.

Example:

```
d:t:x:O,/tmp/mysqlnd.trace
```

> **Nota** :
>
> Este recurso está disponível apenas com uma compilação de depuração do PHP.

`mysqlnd.log_mask` int

Define quais consultas serão registradas. O padrão é 0, que desativa o registro. Defina usando um número inteiro e não com constantes PHP. Por exemplo, um valor de 48 (16 + 32) registrará consultas lentas que usam 'nenhum índice bom' (SERVER_QUERY_NO_GOOD_INDEX_USED = 16) ou nenhum índice (SERVER_QUERY_NO_INDEX_USED = 32). Um valor de 2.043 (1 + 2 + 8 + ... + 1.024) registrará todos os tipos de consulta lenta.

Os tipos são os seguintes: SERVER_STATUS_IN_TRANS=1, SERVER_STATUS_AUTOCOMMIT=2, SERVER_MORE_RESULTS_EXISTS=8, SERVER_QUERY_NO_GOOD_INDEX_USED=16, SERVER_QUERY_NO_INDEX_USED=32, SERVER_STATUS_CURSOR_EXISTS=64, SERVER_STATUS_LAST_ROW_SENT=128, SERVER_STATUS_DB_DROPPED=256, SERVER_STATUS_NO_BACKSLASH_ESCAPES=512 e SERVER_QUERY_WAS_SLOW=1024.

`mysqlnd.mempool_default_size` int

Tamanho padrão do pool de memória mysqlnd, que é usado pelos conjuntos de resultados.

`mysqlnd.net_read_timeout` int

`mysqlnd` e a Biblioteca Cliente MySQL, `libmysqlclient` usam APIs de rede diferentes. `mysqlnd` usa fluxos PHP, enquanto `libmysqlclient` usa seus próprio encapsulador em torno das chamadas de rede de nível operacional. O PHP, por padrão, define um tempo limite de leitura de 60s para fluxos. Isso é definido via *php.ini*, `default_socket_timeout`. Esse padrão se aplica a todos os fluxos que não definem nenhum outro valor de tempo limite. `mysqlnd` não define nenhum outro valor e, portanto, conexões de consultas de longa execução podem ser desconectadas após `default_socket_timeout` segundos, resultando em uma mensagem de erro "2006 - MySQL Server has gone away". A biblioteca cliente MySQL define um tempo limite padrão de 24 * 3600 segundos (1 dia) e espera que outros tempos limite ocorram, como tempos limites de TCP/IP. `mysqlnd` agora usa o mesmo tempo limite muito longo. O valor é configurável através de uma nova configuração *php.ini*: `mysqlnd.net_read_timeout`. `mysqlnd.net_read_timeout` é usado por qualquer extensão (`ext/mysql`, `ext/mysqli`, `PDO_MySQL`) que usa `mysqlnd`. `mysqlnd` diz aos fluxos PHP para usar `mysqlnd.net_read_timeout`. Favor observar que pode haver diferenças sutis entre `MYSQL_OPT_READ_TIMEOUT` da Biblioteca Cliente MySQL e fluxos PHP, por exemplo `MYSQL_OPT_READ_TIMEOUT` está documentado para funcionar apenas para conexões TCP/IP e, antes do MySQL 5.1.2, apenas para Windows. Os fluxos PHP podem não ter essa limitação. Por favor, verifique a documentação dos fluxos, em caso de dúvida.

`mysqlnd.net_cmd_buffer_size` int

`mysqlnd` aloca um buffer interno de comando/rede de `mysqlnd.net_cmd_buffer_size` (no *php.ini*) bytes para cada conexão. Se um comando do protocolo MySQL cliente-servidor, por exemplo, `COM_QUERY` (consulta "normal"), não couber no buffer, `mysqlnd` aumentará o buffer ao tamanho necessário para enviar o comando. Sempre que o buffer for estendido para uma conexão, `command_buffer_too_small` será incrementado em um.

Se `mysqlnd` tiver que aumentar o buffer além do tamanho inicial de `mysqlnd.net_cmd_buffer_size` bytes para quase todas as conexões, você deve considerar aumentar o tamanho padrão para evitar realocações.

O tamanho do buffer padrão é 4.096 bytes, que é o menor valor possível.

O valor também pode ser definido usando `mysqli_options(link, MYSQLI_OPT_NET_CMD_BUFFER_SIZE, size)`.

`mysqlnd.net_read_buffer_size` int

Tamanho máximo do bloco de leitura em bytes ao ler o corpo de um pacote de comando MySQL. O protocolo cliente-servidor MySQL encapsula todos os seus comandos em pacotes. Os pacotes consistem em um pequeno cabeçalho e um corpo com a carga útil real. O tamanho do corpo está codificado no cabeçalho. `mysqlnd` lê o corpo em pedaços de `MIN(header.size, mysqlnd.net_read_buffer_size)` bytes. Se o corpo de um pacote for maior que `mysqlnd.net_read_buffer_size` bytes, `mysqlnd` terá que chamar `read()` múltiplas vezes.

O valor também pode ser definido usando `mysqli_options(link, MYSQLI_OPT_NET_READ_BUFFER_SIZE, size)`.

`mysqlnd.sha256_server_public_key` string

Relacionado ao plug-in de autenticação SHA-256. Arquivo com a chave RSA pública do servidor MySQL.

Os clientes podem omitir a configuração de uma chave RSA pública, especificar a chave através desta configuração do PHP ou definir a chave em tempo de execução usando mysqli_options(). Se nenhum

arquivo de chave RSA pública for fornecido pelo cliente, a chave será trocada como parte do procedimento de autenticação padrão do plug-in de autenticação SHA-256.

`mysqlnd.trace_alloc` string

`mysqlnd.fetch_data_copy` int

Impõe cópia de conjuntos de resultados dos buffers internos do conjunto de resultados em variáveis PHP em vez de usar a referência padrão e a lógica de cópia na gravação. Por favor, veja as notas de implementação de gerenciamento de memória para mais detalhes.

Copiar conjuntos de resultados em vez de fazer referência a variáveis PHP permite liberar a memória ocupada pelas variáveis PHP anteriormente. Dependendo do código da API do usuário, das consultas reais do banco de dados e do tamanho de seus conjuntos de resultados, isso pode reduzir o consumo de memória do mysqlnd.

Não defina se estiver usando PDO_MySQL. PDO_MySQL ainda não foi atualizado para suportar o novo modo de busca.

> **Nota** :
>
> Removido a partir do PHP 8.1.0

Incompatibilidades

O Driver Nativo MySQL é na maioria dos casos compatível com a Biblioteca Cliente MySQL (`libmysql`). Esta seção documenta incompatibilidades entre estas bibliotecas.

- Valores do tipo de dados `bit` são retornados como strings binárias (por exemplo, "\0" ou "\x1F") com `libmysql` e como strings decimais (por exemplo, "0" ou "31 ") com `mysqlnd`. Se for desejado que o código seja compatível com ambas as bibliotecas, então os campos de bits devem ser sempre retornados como números do MySQL com uma consulta como esta: `SELECT bit + 0 FROM table`.

Conexões Persistentes

Usando Conexões Persistentes

Se `mysqli` for usada com `mysqlnd`, quando uma conexão persistente for criada ela gera uma chamada `COM_CHANGE_USER` (`mysql_change_user()`) no servidor. Isto assegura que a re-autenticação da conexão aconteça.

Como existe uma certa sobrecarga com a chamada `COM_CHANGE_USER`, é possível desligar isto em tempo de compilação. O reúso de uma conexão persistente irá gerar uma chamada `COM_PING` (`mysql_ping`) para simplesmente testar se a conexão é reutilizável.

A geração de `COM_CHANGE_USER` pode ser desligada com a opção de compilação `MYSQLI_NO_CHANGE_USER_ON_PCONNECT`. Por exemplo:

```
shell# CFLAGS="-DMYSQLI_NO_CHANGE_USER_ON_PCONNECT" ./configure --with-mysql=/usr/local/mysql/ --with-mysqli=/usr/local/mysql/bin/mysql_config --with-pdo-mysql=/usr/local/mysql/bin/mysql_config --enable-debug && make clean && make -j6
```

Ou alternativamente:

```
shell# export CFLAGS="-DMYSQLI_NO_CHANGE_USER_ON_PCONNECT"
shell# configure --whatever-option
shell# make clean
shell# make
```

Observe que somente `mysqli` sobre `mysqlnd` usa `COM_CHANGE_USER`. Outras compinações de extensão-driver usam `COM_PING` no uso inicial de uma conexão persistente.

Estatísticas

O Driver Nativo MySQL contém suporte para coleta de estatísticas sobre a comunicação entre o cliente e o servidor. As estatísticas recolhidas são de dois tipos principais:

- Estatísticas de cliente
- Estatísticas de conexão

Ao usar a extensão mysqli, essas estatísticas poderão ser obtidas através de duas chamadas de API:

- mysqli_get_client_stats()
- mysqli_get_connection_stats()

> **Nota** :
>
> As estatísticas são agregadas entre todas as extensões que utilizam o Driver Nativo MySQL. Por exemplo, se a extensão mysqli e o driver PDO MySQL estiverem ambos configurados para usar MySQLnd, as chamadas de função de mysqli e chamadas de métodos do PDO afetarão as estatísticas.
>
> Não há como saber o quanto uma determinada chamada de API de qualquer extensão que tenha sido compilada contra o Driver Nativo MySQL tenha impactado uma determinada estatística.

Obtendo estatísticas

Estatísticas do cliente podem ser obtidas chamando a função mysqli_get_client_stats()

Estatísticas de conexão podem ser obtidas chamando a função mysqli_get_connection_stats().

Ambas as funções retornam um array associativo, onde o nome da estatística é a chave para o dado estatístico correspondente.

Estatísticas do Driver Nativo MySQL

A maior parte das estatíticas são associadas a uma conexão mas algumas são associadas ao processo, e neste caso isto será mencionado no texto.

As seguintes estatísticas são produzidas pelo Driver Nativo MySQL:

Estatísticas Relacionadas à Rede

`bytes_sent`

: Número de bytes enviados do PHP para o servidor MySQL.

`bytes_received`

Número de bytes recebidos do servidor MySQL.

`packets_sent`

Número de pacotes enviados pelo protocolo Cliente Servidor do MySQL.

`packets_received`

Número de pacotes recebidos pelo protocolo Cliente Servidor do MySQL.

`protocol_overhead_in`

Sobrecarga do protocolo Cliente Servidor do MySQL em bytes para o tráfego de entrada. Atualmente apenas o Cabeçalho do Pacote (4 bytes) é considerado como sobrecarga. `protocol_overhead_in = packets_received * 4`

`protocol_overhead_out`

Sobrecarga do protocolo Cliente Servidor do MySQL em bytes para o tráfego de saída. Atualmente apenas o Cabeçalho do Pacote (4 bytes) é considerado como sobrecarga. `protocol_overhead_out = packets_received * 4`

`bytes_received_ok_packet`

Tamanho total em bytes de pacotes OK recebidos pelo protocolo Cliente Servidor do MySQL. Pacotes OK podem conter uma mensagem de status. O comprimento da mensagem de status pode variar e portanto o tamanho de um pacote OK não é fixo.

> **Nota** :
>
> O tamanho total em bytes inclui o tamanho do pacote de cabeçalho. (4 bytes, consulte sobre sobrecarga do protocolo)

`packets_received_ok`

Número de pacotes OK recebidos pelo protocolo Cliente Servidor do MySQL.

`bytes_received_eof_packet`

Tamanho total em bytes de pacotes EOF recebidos pelo protocolo Cliente Servidor do MySQL. EOF pode variar em tamanho dependendo da versão do servidor. Além disso, EOF pode transportar uma mensagem de erro.

> **Nota** :
>
> O tamanho total em bytes inclui o tamanho do pacote de cabeçalho. (4 bytes, consulte sobre sobrecarga do protocolo)

`packets_received_eof`

Número de pacotes EOF recebidos pelo protocolo Cliente Servidor do MySQL.

Como acontece com outras estatísticas de pacotes, o número de pacotes será incrementado mesmo se o PHP não receber o pacote esperado mas, por exemplo, receber uma mensagem de erro.

`bytes_received_rset_header_packet`

Tamanho total em bytes dos pacotes de cabeçalho do conjunto de resultados recebido pelo protocolo Cliente Servidor do MySQL. O tamanho dos pacotes varia dependendo da carga (`LOAD LOCAL INFILE`, `INSERT`, `UPDATE`, `SELECT`, mensagem de erro).

> **Nota** :
>
> O tamanho total em bytes inclui o tamanho do pacote de cabeçalho. (4 bytes, consulte sobre sobrecarga do protocolo)

`packets_received_rset_header`

Número de pacotes de cabeçalho de conjunto de resultados recebidos pelo protocolo Cliente Servidor do MySQL.

`bytes_received_rset_field_meta_packet`

Tamanho total em bytes de pacotes de metadados (informação de campo) de conjunto de resultados recebidos pelo protocolo Cliente Servidor do MySQL. É evidente que o tamanho varia com os campos no conjunto de resultados. O pacotes também pode transportar um pacote de erro ou de EOF em caso de COM_LIST_FIELDS.

> **Nota** :
>
> O tamanho total em bytes inclui o tamanho do pacote de cabeçalho. (4 bytes, consulte sobre sobrecarga do protocolo)

`packets_received_rset_field_meta`

Número de pacotesde metadados (informação de campo) de conjunto de resultados recebidos pelo protocolo Cliente Servidor do MySQL.

`bytes_received_rset_row_packet`

Tamanho total em bytes dos pacotes de dados de registros do conjunto de resultados recebidos pelo protocolo Cliente Servidor do MySQL. O pacote também pode transportar uma mensagem de erro ou de EOF. Pode-se computar o número de pacotes de erro e de EOF subtraindo-se `rows_fetched_from_server_normal` e `rows_fetched_from_server_ps` de `bytes_received_rset_row_packet`.

> **Nota** :
>
> O tamanho total em bytes inclui o tamanho do pacote de cabeçalho. (4 bytes, consulte sobre sobrecarga do protocolo)

`packets_received_rset_row`

Número de pacotes de dados de registros do conjunto de resultados recebidos pelo protocolo Cliente Servidor do MySQL.

`bytes_received_prepare_response_packet`

Tamanho total em bytes dos pacotes de Inicialização OK para Instruções Preparadas (pacotes de inicialização de instrução preparada) recebidos pelo protocolo Cliente Servidor do MySQL. O pacote também pode transportar um erro. O tamanho do pacote depende da versão MySQL.

> **Nota** :
>
> O tamanho total em bytes inclui o tamanho do pacote de cabeçalho. (4 bytes, consulte sobre sobrecarga do protocolo)

`packets_received_prepare_response`

Número de pacotes de inicialização de instruções preparadas recebidos pelo protocolo Cliente Servidor do MySQL.

`bytes_received_change_user_packet`

Tamanho total em bytes de pacotes COM_CHANGE_USER recebidos pelo protocolo Cliente Servidor do MySQL. O pacote também pode transportar um erro ou um EOF.

> **Nota** :
>
> O tamanho total em bytes inclui o tamanho do pacote de cabeçalho. (4 bytes, consulte sobre sobrecarga do protocolo)

`packets_received_change_user`

Número de pacotes COM_CHANGE_USER recebidos pelo protocolo Cliente Servidor do MySQL.

`packets_sent_command`

Número de comandos do protocolo Cliente Servidor do MySQL enviados pelo PHP ao MySQL.

Não há como saber quais foram os comandos específicos e quantos deles foram enviados.

`bytes_received_real_data_normal`

Número de bytes dos dados reais buscados pelo cliente PHP vindos do `mysqlnd` usando o protocolo de texto.

Este é o tamanho dos dados reais contidos nos conjuntos de resultados que não se originaram de instruções preparadas e que foram buscados pelo cliente PHP.

Observe que embora um conjunto de resultados completo possa ter sido puxado do MySQL pelo `mysqlnd`, esta estatística apenas conta dados reais puxados a partir do `mysqlnd` pelo cliente PHP.

A seguir é apresentado um exemplo de sequência de código que irá incrementar o valor:

```
$mysqli = new mysqli();
$res = $mysqli->query("SELECT 'abc'");
$res->fetch_assoc();
$res->close();
```

Cada operação de busca irá incrementar o valor.

Entretanto, a estatística não será incrementada se o conjunto de resultados estiver sujeito a buffer no lado do cliente mas não for buscado, como no exemplo a seguir:

```
$mysqli = new mysqli();
$res = $mysqli->query("SELECT 'abc'");
$res->close();
```

`bytes_received_real_data_ps`

Número de bytes dos dados reais buscados pelo cliente PHP vindos do `mysqlnd` usando o protocolo de instruções preparadas.

Este é o tamanho dos dados reais contidos nos conjuntos de resultados que se originaram de instruções preparadas e que foram buscados pelo cliente PHP.

O valor não será incrementado se o conjunto de resultados não for subsequentemente lido pelo cliente PHP.

Observe que embora um conjunto de resultados completo possa ter sido puxado do MySQL pelo `mysqlnd`, esta estatística apenas conta dados reais puxados a partir do `mysqlnd` pelo cliente PHP.

Consulte também `bytes_received_real_data_normal`.

Estatísticas Relacionadas ao Conjunto de Resultados

`result_set_queries`

Número de consultas que geraram um conjunto de resultados. Exemplos de consultas que geram um conjunto de resultados: `SELECT`, `SHOW`.

A estatística não será incrementada se ocorrer um erro ao ler o pacote de cabeçalho do conjunto de resultados.

> **Nota** :
>
> Esta estatística pode ser usada como uma medição indireta do número de consultas que o PHP enviou ao MySQL. Isto pode ajudar a identificar um cliente que causa uma carga alta no banco de dados.

`non_result_set_queries`

Número de consultas que não geraram um conjunto de resultados. Exemplos de consultas que não geram um conjunto de resultados: `INSERT`, `UPDATE`, `LOAD DATA`.

A estatística não será incrementada se ocorrer um erro ao ler o pacote de cabeçalho do conjunto de resultados.

> **Nota** :
>
> Esta estatística pode ser usada como uma medição indireta do número de consultas que o PHP enviou ao MySQL. Isto pode ajudar a identificar um cliente que causa uma carga alta no banco de dados.

`no_index_used`

Número de consultas que geraram um conjunto de resultados mas não usaram um índice. (Consulte também a opção de início do mysqld `--log-queries-not-using-indexes`).

> **Nota** :
>
> Essas consultas podem ser reportadas através de uma exceção com `mysqli_report(MYSQLI_REPORT_INDEX);`. É possível reportá-las também através de um alerta com `mysqli_report(MYSQLI_REPORT_INDEX ^ MYSQLI_REPORT_STRICT);`.

`bad_index_used`

Número de consultas que geraram um conjunto de resultados e não usaram um índice bom. (Consulte também a opção de início do mysqld `--log-slow-queries`).

> **Nota** :
>
> Essas consultas podem ser reportadas através de uma exceção com `mysqli_report(MYSQLI_REPORT_INDEX);`. É possível reportá-las também através de um alerta com `mysqli_report(MYSQLI_REPORT_INDEX ^ MYSQLI_REPORT_STRICT);`.

`slow_queries`

Instruções SQL que levaram mais de `long_query_time` segundos para execução e requereram que pelo menos `min_examined_row_limit` registros fossem examinados.

> **Cuidado**
>
> Não reportada através de mysqli_report().

`buffered_sets`

Número de conjuntos de resultados em buffer retornados por consultas normais (isto é, não por via de instrução preparada).

Exemplos de chamadas API que armazenarão conjuntos de resultados em buffer no cliente: mysqli_query(), mysqli_store_result(), mysqli_stmt_get_result()

`unbuffered_sets`

Número de conjuntos de resultados sem buffer retornados por consultas normais (isto é, não por via de instrução preparada).

Exemplos de chamadas API que não armazenarão conjuntos de resultados em buffer no cliente: mysqli_use_result()

`ps_buffered_sets`

Número de conjuntos de resultados em buffer retornados por instruções preparadas.

Exemplos de chamadas API que armazenarão conjuntos de resultados em buffer no cliente: mysqli_stmt_store_result()

`ps_unbuffered_sets`

Número de conjuntos de resultados sem buffer retornados por instruções preparadas.

Por padrão, instruções preparadas não são armazenadas em buffer, portanto a maioria das instruções preparadas será considerada nesta estatísitica.

`flushed_normal_sets`

Número de conjuntos de resultados retornados por consultas normais (isto é, não por via de instrução preparada) com dados não lidos que foram silenciosamente descarregados.

> **Nota** :
>
> Os descarregamento ocorre somente com conjuntos de resultados sem buffer.
>
> Conjuntos de resultados sem buffer devem ser buscados integralmente antes que uma nova consulta possa ser executada na conexão, caso contrário o MySQL lançará um erro. Se a aplicação não buscar todos os registros de um conjunto de resultados sem buffer, o mysqlnd não busca implicitamente o conjunto de resultados para limpar a linha de dados.
>
> Consulta também `rows_skipped_normal`, `rows_skipped_ps`.
>
> Algumas causas possíveis para um descarregamento implícito:
>
> - Aplicação cliente com falhas
> - Cliente parou de ler depois de ter encontrado o que estava procurando mas fez o MySQL calcular mais registros que o necessário
> - Aplicação cliente parou inesperadamente

`flushed_ps_sets`

Número de conjuntos de resultados de instruções preparadas com dados não lidos que foram silenciosamente descarregados.

> **Nota** :
>
> Os descarregamento ocorre somente com conjuntos de resultados sem buffer.
>
> Conjuntos de resultados sem buffer devem ser buscados integralmente antes que uma nova consulta possa ser executada na conexão, caso contrário o MySQL lançará um erro. Se a aplicação não buscar todos os registros de um conjunto de resultados sem buffer, o mysqlnd não busca implicitamente o conjunto de resultados para limpar a linha de dados.
>
> Consulta também `rows_skipped_normal`, `rows_skipped_ps`.
>
> Algumas causas possíveis para um descarregamento implícito:
>
> - Aplicação cliente com falhas
> - Cliente parou de ler depois de ter encontrado o que estava procurando mas fez o MySQL calcular mais registros que o necessário
> - Aplicação cliente parou inesperadamente

`ps_prepared_never_executed`

Número de intruções preparadas mas nunca executadas.

`ps_prepared_once_executed`

Número de intruções preparadas executadas apenas uma vez.

`rows_fetched_from_server_normal`
`rows_fetched_from_server_ps`

Número total de registros do conjunto de resultados buscados do servidor. Isto inclui os registros que não foram lidos pelo cliente mas foram buscados implicitamente devido a conjuntos de resultados sem buffer descarregados.

Consulte também `packets_received_rset_row`.

`rows_buffered_from_client_normal`

Número total de registros em buffer originários de uma consulta normal.

Este é o número de registros que foram buscados do MySQL e armazenados em buffer no cliente.

Exemplos de consultas que irão armazenar resultados em buffer:

- mysqli_query()
- mysqli_store_result()

`rows_buffered_from_client_ps`

O mesmo que `rows_buffered_from_client_normal`, porém para instruções preparadas.

`rows_fetched_from_client_normal_buffered`

Número total de registros buscados pelo cliente a partir de um conjunto de resultados em buffer criado por uma consulta normal.

`rows_fetched_from_client_ps_buffered`

Número total de registros buscados pelo cliente a partir de um conjunto de resultados em buffer criado por uma instrução preparada.

`rows_fetched_from_client_normal_unbuffered`

Número total de registros buscados pelo cliente a partir de um conjunto de resultados sem buffer criado por uma consulta normal.

`rows_fetched_from_client_ps_unbuffered`

Número total de registros buscados pelo cliente a partir de um conjunto de resultados sem buffer criado por uma instrução preparada.

`rows_fetched_from_client_ps_cursor`

Número total de registros buscados pelo cliente a partir de um cursor criado por uma instrução preparada.

`rows_skipped_normal`
`rows_skipped_ps`

Reservada para uso futuro (atualmente não suportada).

`copy_on_write_saved`
`copy_on_write_performed`

Esta é uma estatística cujo escopo está no nível de processo.

Com o mysqlnd, variáveis retornadas pelas extensões apontam para os buffers de resultados da rede interna do mysqlnd. Se os dados não foram modificados, os dados buscados são mantidos apenas uma vez na memória. Entretanto, qualquer modificação nos dados irá requerer que o mysqlnd realize uma operação de cópia-na-escrita.

`explicit_free_result`
`implicit_free_result`

Esta é uma estatística cujo escopo está no nível de conexão e de processo.

Número total de conjuntos de resultados liberados.

`proto_text_fetched_null`

Número total de colunas do tipo `MYSQL_TYPE_NULL` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_null`

Número total de colunas do tipo `MYSQL_TYPE_NULL` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_bit`

Número total de colunas do tipo `MYSQL_TYPE_BIT` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_bit`

Número total de colunas do tipo `MYSQL_TYPE_BIT` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_tinyint`

Número total de colunas do tipo `MYSQL_TYPE_TINY` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_tinyint`

Número total de colunas do tipo `MYSQL_TYPE_TINY` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_short`

Número total de colunas do tipo `MYSQL_TYPE_SHORT` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_short`

Número total de colunas do tipo `MYSQL_TYPE_SHORT` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_int24`

Número total de colunas do tipo `MYSQL_TYPE_INT24` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_int24`

Número total de colunas do tipo `MYSQL_TYPE_INT24` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_int`

Número total de colunas do tipo `MYSQL_TYPE_LONG` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_int`

Número total de colunas do tipo `MYSQL_TYPE_LONG` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_bigint`

Número total de colunas do tipo `MYSQL_TYPE_LONGLONG` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_bigint`

Número total de colunas do tipo `MYSQL_TYPE_LONGLONG` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_decimal`

Número total de colunas dos tipos `MYSQL_TYPE_DECIMAL` ou `MYSQL_TYPE_NEWDECIMAL` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_decimal`

Número total de colunas dos tipos `MYSQL_TYPE_DECIMAL` ou `MYSQL_TYPE_NEWDECIMAL` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_float`

Número total de colunas do tipo `MYSQL_TYPE_FLOAT` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_float`

Número total de colunas do tipo `MYSQL_TYPE_FLOAT` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_double`

Número total de colunas do tipo `MYSQL_TYPE_DOUBLE` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_double`

Número total de colunas do tipo `MYSQL_TYPE_DOUBLE` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_date`

Número total de colunas do tipo `MYSQL_TYPE_DATE` ou `MYSQL_TYPE_NEWDATE` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_date`

Número total de colunas do tipo `MYSQL_TYPE_DATE` ou `MYSQL_TYPE_NEWDATE` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_year`

Número total de colunas do tipo `MYSQL_TYPE_YEAR` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_year`

Número total de colunas do tipo `MYSQL_TYPE_YEAR` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_time`

Número total de colunas do tipo `MYSQL_TYPE_TIME` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_time`

Número total de colunas do tipo `MYSQL_TYPE_TIME` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_datetime`

Número total de colunas do tipo `MYSQL_TYPE_DATETIME` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_datetime`

Número total de colunas do tipo `MYSQL_TYPE_DATETIME` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_timestamp`

Número total de colunas do tipo `MYSQL_TYPE_TIMESTAMP` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_timestamp`

Número total de colunas do tipo `MYSQL_TYPE_TIMESTAMP` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_string`

Número total de colunas dos tipos `MYSQL_TYPE_STRING`, `MYSQL_TYPE_VARSTRING` ou `MYSQL_TYPE_VARCHAR` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_string`

Número total de colunas dos tipos `MYSQL_TYPE_STRING`, `MYSQL_TYPE_VARSTRING` ou `MYSQL_TYPE_VARCHAR` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_blob`

Número total de colunas dos tipos `MYSQL_TYPE_TINY_BLOB`, `MYSQL_TYPE_MEDIUM_BLOB`, `MYSQL_TYPE_LONG_BLOB` ou `MYSQL_TYPE_BLOB` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_blob`

Número total de colunas dos tipos `MYSQL_TYPE_TINY_BLOB`, `MYSQL_TYPE_MEDIUM_BLOB`, `MYSQL_TYPE_LONG_BLOB` ou `MYSQL_TYPE_BLOB` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_enum`

Número total de colunas do tipo `MYSQL_TYPE_ENUM` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_enum`

Número total de colunas do tipo `MYSQL_TYPE_ENUM` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_set`

Número total de colunas do tipo `MYSQL_TYPE_SET` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_set`

Número total de colunas do tipo `MYSQL_TYPE_SET` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_geometry`

Número total de colunas do tipo `MYSQL_TYPE_GEOMETRY` buscadas por uma consulta normal (protocolo de texto MySQL).

`proto_binary_fetched_geometry`

Número total de colunas do tipo `MYSQL_TYPE_GEOMETRY` buscadas por uma instrução preparada (protocolo binário MySQL).

`proto_text_fetched_other`

Número total de colunas dos tipos `MYSQL_TYPE_*` não listadas previamente e buscadas por uma consulta normal (protocolo de texto MySQL).

> **Nota** :
>
> Em teoria, deveria ser sempre `0`.

`proto_binary_fetched_other`

Número total de colunas dos tipos `MYSQL_TYPE_*` não listadas previamente e buscadas por uma instrução preparada (protocolo binário MySQL).

> **Nota** :
>
> Em teoria, deveria ser sempre `0`.

Estatísticas Relacionadas a Conexões

`connect_success`

Número total de tentativas bem sucedidas de conexões.

> **Nota** :
>
> `connect_success` contém a soma de tentativas bem sucedidas de conexões persistentes e não persistentes. Portanto, o número de tentativas bem sucedidas de conexões não persistentes é `connect_success - pconnect_success`.

`pconnect_success`

Número total de tentativas bem sucedidas de conexões persistentes.

`connect_failure`

Número total de tentativas falhadas de conexões.

`reconnect`

Esta é uma estatística cujo escopo está no nível de processo.

`active_connections`

Esta é uma estatística cujo escopo está no nível de processo.

Número total de conexões ativas, persistentes e não persistentes.

> **Nota** :
>
> Número total de conexões ativas não persistentes é `active_connections - active_persistent_connections`.

`active_persistent_connections`

Esta é uma estatística cujo escopo está no nível de processo.

Número total de conexões ativas persistentes.

`explicit_close`

Número total de conexões fechadas explicitamente.

Exemplo #1 Exemplos de trechos de código que causam um fechamento explícito

- ```
  $link = new mysqli(/* ... */);
  $link->close(/* ... */);
  ```
- ```
  $link = new mysqli(/* ... */);
  $link->connect(/* ... */);
  ```

`implicit_close`

Número total de conexões fechadas implicitamente.

Exemplo #2 Exemplos de trechos de código que causam um fechamento implícito

- ```
  $link = new mysqli(/* ... */);
  $link->real_connect(/* ... */);
  ```
- `unset($link)`
- Conexão persistente: conexão em fila foi criada com real_connect e podem haver opções desconhecidas definidas - fecha implicitamente para evitar o retorno de uma conexão com opções desconhecidas
- Conexão persistente: ping/change_user falha e a extensão mysqli fecha a conexão
- Fim de execução de script: fecha conexões que não foram fechadas pelo usuário

`disconnect_close`

Falhas de conexão indicadas pela chamada API C `mysql_real_connect` durante uma tentativa de se estabalecer uma conexão.

`in_middle_of_command_close`

Esta é uma estatística cujo escopo está no nível de processo.

Uma conexão foi fechada no meio de uma execução de comando (conjuntos de resultados não buscados, após enviar uma consulta e antes de receber uma resposta, durante a busca de dados, enquanto dados são transferidos com LOAD DATA).

> **Aviso**
>
> A menos que consultas assíncronas sejam usadas, isto só deve acontecer se a aplicação PHP terminar inesperadamente e o PHP fechar a conexão automaticamente.

`init_command_executed_count`

Número total de execuções do comando init. Por exemplo: `mysqli_options(MYSQLI_INIT_COMMAND , $value)`.

O número total de execuções bem sucedidas é `init_command_executed_count - init_command_failed_count`.

`init_command_failed_count`

Número total de comandos init falhados.
  ```

Estatísticas Relacionadas aos Comandos `COM_*`

`com_quit`
`com_init_db`
`com_query`
`com_field_list`
`com_create_db`
`com_drop_db`
`com_refresh`
`com_shutdown`
`com_statistics`
`com_process_info`
`com_connect`
`com_process_kill`
`com_debug`
`com_ping`
`com_time`
`com_delayed_insert`
`com_change_user`
`com_binlog_dump`
`com_table_dump`
`com_connect_out`
`com_register_slave`
`com_stmt_prepare`
`com_stmt_execute`
`com_stmt_send_long_data`
`com_stmt_close`
`com_stmt_reset`
`com_stmt_set_option`
`com_stmt_fetch`
`com_daemon`

Número total de tentativas de envio de um certo comando `COM_*` do PHP para o MySQL.

As estatísticas são incrementadas após verificação da linha de dados e imediatamente antes do envio do pacote correspondente do protocolo Cliente Servidor do MySQL.

> **Cuidado**
>
> Se o MySQLnd falhar ao enviar o pacote pela rede, a estatística não será decrementada. Em caso de falha, o MySQLnd emite um alerta do PHP "Error while sending %s packet. PID=%d."

Exemplo #3 Exemplos de uso

- Verificar se o PHP envia certos comandos ao MySQL, por exemplo, verificar se o cliente envia `COM_PROCESS_KILL`
- Calcular o número médio de execuções de instruções preparadas comparando `COM_EXECUTE` com `COM_PREPARE`
- Verificar se o PHP executou alguma instrução SQL não preparada observando se `COM_QUERY` é igual a zero

- Identificar scripts PHP que executam um número excessivo de instruções SQL, verificando `COM_QUERY` e `COM_EXECUTE`

Estatísticas Miscelâneas

`explicit_stmt_close`
`implicit_stmt_close`

Esta é uma estatística cujo escopo está no nível de processo.

Número total de instruções preparadas fechadas.

> **Nota** :
>
> Uma instrução preparada é sempre fechada explicitamente. O único momento em que ela é fechaada implicitamente é quando sua preparação falha.

`mem_emalloc_count`
`mem_emalloc_ammount`
`mem_ecalloc_count`
`mem_ecalloc_ammount`
`mem_realloc_count`
`mem_realloc_ammount`
`mem_efree_count`
`mem_malloc_count`
`mem_malloc_ammount`
`mem_calloc_count`
`mem_calloc_ammount`
`mem_ealloc_count`
`mem_ealloc_ammount`
`mem_free_count`

Esta é uma estatística cujo escopo está no nível de processo.

Chamadas de gerenciamento de memória.

`command_buffer_too_small`

Número de extensões de buffer de comandos de rede ao enviar comandos do PHP para o MySQL.

MySQLnd aloca um buffer interno de comando/rede de mysqlnd.net_cmd_buffer_size bytes para cada conexão.

Se um comando do protocolo Cliente Servidor do MySQL, por exemplo `COM_QUERY` (consulta normal), não couber no buffer, o MySQLnd aumentará o buffer para o tamanho necessário para envio do comando. Sempre que o buffer for estendido para uma conexão, `command_buffer_too_small` será incrementada em uma unidade.

Se o MySQLnd tiver que aumentar o buffer para além do seu tamanho inicial de mysqlnd.net_cmd_buffer_size bytes para quase todas as conexões, deve ser considerado aumento do tamanho padrão para evitar realocações.

`connection_reused`

O número total de vezes em que uma conexão persistente foi reutilizada.

Notas

Esta seção fornece uma coleção de notas miscelâneas sobre o uso do Driver Nativo MySQL.

- Usar `mysqlnd`significa usar fluxos PHP para conectividades subjacente. Para `mysqlnd`, a documentação de fluxos do PHP (Fluxos) deve ser consultada sobre detalhes como configurações de limites de tempo, e não a documentação para a Biblioteca Cliente do MySQL.

Gerenciamento de Memória

Introdução

O Driver Nativo MySQL gerencia memória diferente da Biblioteca Cliente MySQL. As bibliotecas diferem na forma como a memória é alocada e liberada, como a memória é alocada em pedaços durante a leitura dos resultados do MySQL, quais opções de depuração e desenvolvimento existem e como os resultados lidos do MySQL são vinculados às variáveis de usuário do PHP.

As notas a seguir pretendem ser uma introdução e um resumo aos usuários interessados em compreender o Driver Nativo MySQL em nível de código C.

Funções de gerenciamento de memória usadas

Toda a alocação e desalocação de memória é feita usando as funções de gerenciamento de memória do PHP. Portanto, o consumo de memória do mysqlnd pode ser rastreado usando chamadas de API PHP, como memory_get_usage(). Como a memória é alocada e liberada usando o gerenciamento de memória do PHP, as alterações podem não se tornar imediatamente visíveis no nível do sistema operacional. O gerenciamento de memória do PHP atua como um proxy que pode atrasar a liberação de memória para o sistema. Devido a isso, é difícil comparar o uso de memória do Driver Nativo MySQL e da Biblioteca Cliente MySQL. A Biblioteca Cliente MySQL está usando as chamadas de gerenciamento de memória do sistema operacional diretamente, portanto, os efeitos podem ser observados imediatamente no nível do sistema operacional.

Qualquer limite de memória imposto pelo PHP também afeta o Driver Nativo MySQL. Isso pode causar erros de falta de memória ao buscar conjuntos de resultados grandes que excedem o tamanho da memória restante disponibilizada pelo PHP. Como a Biblioteca Cliente MySQL não usa funções de gerenciamento de memória PHP, ela não está em conformidade com nenhum limite de memória PHP definido. Se estiver usando a Biblioteca Cliente MySQL, dependendo do modelo de implantação, o consumo de memória do processo PHP pode crescer além do limite de memória do PHP. Mas também os scripts PHP podem processar conjuntos de resultados maiores, pois partes da memória alocada para armazenar os conjuntos de resultados estão além do controle do mecanismo PHP.

As funções de gerenciamento de memória do PHP são invocadas pelo Driver Nativo MySQL através de um encapsulador leve. Entre outros, o encapsulador facilita a depuração.

Lidando com conjuntos de resultados

Os vários servidores MySQL e as diversas APIs do cliente diferenciam entre conjuntos de resultados com buffer e sem buffer. Conjuntos de resultados sem buffer são transferidos linha por linha do MySQL para o cliente à medida que o cliente itera sobre os resultados. Os resultados armazenados em buffer são buscados integralmente pela biblioteca cliente antes de serem repassados ao cliente.

O Driver Nativo MySQL está utilizando fluxos PHP para a comunicação da rede com o Servidor MySQL. Os resultados enviados pelo MySQL são buscados dos buffers da rede de fluxos do PHP para o buffer de resultados do mysqlnd. O buffer resultante é composto de zvals. Numa segunda etapa os resultados são disponibilizados

para o script PHP. Esta transferência final do buffer de resultados para variáveis PHP impacta o consumo de memória e é principalmente perceptível ao usar conjuntos de resultados armazenados em buffer.

Por padrão o Driver Nativo MySQL tenta evitar armazenar resultados em buffer duas vezes na memória. Os resultados são mantidos apenas uma vez nos buffers de resultados internos e seus zvals. Quando os resultados são buscados em variáveis PHP pelo script PHP, as variáveis farão referência aos buffers de resultados internos. Os resultados da consulta ao banco de dados não são copiados e mantidos na memória apenas uma vez. Caso o usuário modifique o conteúdo de uma variável que contém os resultados do banco de dados, uma cópia na gravação deverá ser executada para evitar a alteração do buffer de resultados interno referenciado. O conteúdo do buffer não deve ser modificado porque o usuário pode decidir ler o conjunto de resultados uma segunda vez. O mecanismo de cópia na gravação é implementado usando uma lista de gerenciamento de referência adicional e o uso de contadores de referência zval padrão. A cópia na gravação também deve ser feita se o usuário ler um conjunto de resultados em variáveis PHP e liberar um conjunto de resultados antes que as variáveis sejam desdefinidas.

De modo geral, esse padrão funciona bem para scripts que leem um conjunto de resultados uma vez e não modificam variáveis que contêm resultados. Sua principal desvantagem é a sobrecarga de memória causada pelo gerenciamento de referência adicional que vem principalmente do fato de que as variáveis do usuário que contêm resultados não podem ser totalmente liberadas até que o gerenciamento de referência do mysqlnd pare de referenciá-las. O Driver Nativo MySQL remove a referência às variáveis do usuário quando o conjunto de resultados é liberado ou uma cópia na gravação é executada. Um observador verá o consumo total de memória crescer até que o conjunto de resultados seja liberado. Use as estatísticas para verificar se um script libera conjuntos de resultados explicitamente ou se o driver faz liberações implícitas e, portanto, a memória é usada por um tempo maior que o necessário. As estatísticas também ajudam a ver quantas operações de cópia na gravação ocorreram.

Um script PHP lendo muitas linhas pequenas de um conjunto de resultados em buffer usando um trecho de código igual ou equivalente a `while ($row = $res->fetch_assoc()) { ... }` pode otimizar o consumo de memória solicitando cópias em vez de referências. Embora solicitar cópias signifique manter os resultados duas vezes na memória, isso permite que o PHP libere a cópia contida em `$row` à medida que o conjunto de resultados está sendo iterado e antes de liberar o próprio conjunto de resultados. Em um servidor carregado, a otimização do uso máximo de memória pode ajudar a melhorar o desempenho geral do sistema, embora para um script individual a abordagem de cópia possa ser mais lenta devido a alocações adicionais e operações de cópia de memória.

Monitoramento e depuração

Existem diversas formas de rastrear o uso de memória do Driver Nativo MySQL. Se o objetivo é obter uma visão geral rápida de alto nível ou verificar a eficiência da memória dos scripts PHP, então verifique as estatísticas coletadas pela biblioteca. As estatísticas permitem, por exemplo, capturar instruções SQL que geram mais resultados do que são processados por um script PHP.

O log de rastreamento debug pode ser configurado para registrar chamadas de gerenciamento de memória. Isso ajuda a ver quando a memória está alocada ou liberada. No entanto, o tamanho dos blocos de memória solicitados pode não estar listado.

Algumas versões recentes do Driver Nativo MySQL apresentam a emulação de situações aleatórias de falta de memória. Este recurso deve ser usado apenas pelos desenvolvedores C da biblioteca ou pelos autores do plugin do mysqlnd. Por favor, pesquise no código-fonte as configurações correspondentes do PHP e mais detalhes. O recurso é considerado privado e pode ser modificado a qualquer momento sem aviso prévio.

API do plugin do Driver Nativo MySQL

Índice

A API de plugin do Driver Nativo MySQL é um recurso do Driver Nativo MySQL, ou `mysqlnd`. Os plugins `Mysqlnd` operam na camada entre os aplicativos PHP e o servidor MySQL. Isso é comparável ao Proxy MySQL. O Proxy MySQL opera em uma camada entre qualquer aplicativo cliente MySQL, por exemplo, um aplicativo PHP e o servidor MySQL. Os plugins `Mysqlnd` podem realizar tarefas típicas do Proxy MySQL, como balanceamento de carga, monitoramento e otimizações de desempenho. Devido à arquitetura e localização diferentes, os plugins `mysqlnd` não apresentam algumas das desvantagens do Proxy MySQL. Por exemplo, com plugins, não há um ponto único de falha, nenhum servidor proxy dedicado para se implantar e nenhuma nova linguagem de programação para aprender (Lua).

Um plugin `mysqlnd` pode ser pensado como uma extensão do `mysqlnd`. Plugins podem interceptar a maioria das funções do `mysqlnd`. As funções `mysqlnd` são chamadas pelas extensões PHP MySQL como `ext/mysql`, `ext/mysqli` e `PDO_MYSQL`. Como resultado, é possível para um plugin `mysqlnd` interceptar todas as chamadas feitas para essas extensões a partir da aplicação cliente.

Chamadas internas de função `mysqlnd` também podem ser interceptadas ou substituídas. Não há restrições na manipulação de tabelas de funções internas do `mysqlnd`. É possível configurar as coisas para que quando certas funções `mysqlnd` sejam chamadas pelas extensões que usam `mysqlnd`, a chamada seja direcionada para a função apropriada no plugin `mysqlnd`. A capacidade de manipular tabelas de funções internas do `mysqlnd` desta forma permite máxima flexibilidade para plugins.

Plugins `Mysqlnd` são na verdade extensões PHP, escritas em C, que usam a API do plugin `mysqlnd` (que está embutida no Driver Nativo MySQL, `mysqlnd`) . Os plugins podem ser 100% transparentes para aplicações PHP. Nenhuma alteração na aplicação é necessária porque os plugins operam em uma camada diferente. O plugin `mysqlnd` pode ser pensado como operando em uma camada abaixo do `mysqlnd`.

A lista a seguir representa algumas aplicações possíveis de plugins `mysqlnd`.

- Balanceamento de Carga
    - Divisão de leitura/gravação. Um exemplo disso é a extensão PECL/mysqlnd_ms (Master Slave). Esta extensão divide consultas de leitura/gravação para uma configuração de replicação.
    - Failover
    - Round-Robin, menos carregado
- Monitoramento
    - Registro de Consultas
    - Análise de Consultas
    - Auditoria de Consultas. Um exemplo disso é a extensão PECL/mysqlnd_sip (Proteção Contra Injeção SQL). Esta extensão inspeciona consultas e executa apenas aquelas permitidas de acordo com um conjunto de regras.
- Desempenho
    - Cache. Um exemplo disso é a extensão PECL/mysqlnd_qc (Cache de Consultas).
    - Redução de velocidade
    - Fragmentação. Um exemplo disso é a extensão PECL/mysqlnd_mc (Multi Conexão). Esta extensão tentará dividir uma instrução SELECT em n partes, usando SELECT ... LIMIT part_1, SELECT LIMIT

part_n. Ele envia as consultas para servidores MySQL distintos e mescla o resultado no cliente.

Plugins Disponíveis do Driver Nativo MySQL

Existem vários plug-ins do mysqlnd já disponíveis. Esses incluem:

- **PECL/mysqlnd_mc** - plugin Multi Conexão.
- **PECL/mysqlnd_ms** - plugin Master Slave.
- **PECL/mysqlnd_qc** - plugin Query Cache.
- **PECL/mysqlnd_pscache** - plugin de Cache de Manipulador de Instruções Preparadas.
- **PECL/mysqlnd_sip** - plugin de Proteção Contra Injeção SQL.
- **PECL/mysqlnd_uh** - plugin de Manipulador de Usuário.

Uma comparação de plugins mysqlnd com Proxy MySQL

Plugins `Mysqlnd` e Proxy MySQL são tecnologias diferentes que usam abordagens diferentes. Ambas são ferramentas válidas para resolver uma variedade de tarefas comuns, como balanceamento de carga, monitoramento e melhorias de desempenho. Uma diferença importante é que o Proxy MySQL funciona com todos os clientes MySQL, enquanto os plugins `mysqlnd` são específicos para aplicações PHP.

Como uma extensão PHP, um plugin `mysqlnd` é instalado no servidor de aplicativos PHP, junto com o restante do PHP. O Proxy MySQL pode ser executado no servidor de aplicativos PHP ou pode ser instalado em uma máquina dedicada para lidar com vários servidores de aplicações PHP.

A implantação do Proxy MySQL no servidor de aplicativos tem duas vantagens:

1. Nenhum ponto único de falha
2. Fácil de escalonar horizontalmente (escalonar por cliente)

O proxy MySQL (e os plugins `mysqlnd`) podem resolver facilmente problemas que de outra forma exigiriam alterações em aplicações existentes.

No entanto, o Proxy MySQL tem algumas desvantagens:

- Proxy MySQL é um novo componente e uma nova tecnologia para dominar e implantar.
- O Proxy MySQL requer conhecimento da linguagem de script Lua.

O proxy MySQL pode ser customizado com programação C e Lua. Lua é a linguagem de script preferida do Proxy MySQL. Para a maioria dos especialistas em PHP, Lua é uma nova linguagem para se aprender. Um plugin `mysqlnd` pode ser escrito em C. Também é possível escrever plugins em PHP usando » PECL/mysqlnd_uh.

O proxy MySQL é executado como um daemon - um processo em segundo plano. O proxy MySQL pode recuperar decisões anteriores, pois todo o estado pode ser retido. Entretanto, um plugin `mysqlnd` está vinculado ao ciclo de vida baseado em solicitações do PHP. O Proxy MySQL também pode compartilhar resultados computados únicos entre vários servidores de aplicativos. Um plugin `mysqlnd` precisaria armazenar dados em um meio persistente para poder fazer isso. Outro daemon precisaria ser usado para essa finalidade, como o Memcache. Isso dá ao Proxy MySQL uma vantagem neste caso.

O proxy MySQL funciona sobre o protocolo de fio. Com o Proxy MySQL é necessário analisar e fazer engenharia reversa do protocolo MySQL Client Server. As ações são limitadas àquelas que podem ser alcançadas através da

manipulação do protocolo de comunicação. Se o protocolo de fio for alterado (o que acontece muito raramente), os scripts do proxy MySQL também precisarão ser alterados.

Os plugins `Mysqlnd` funcionam sobre a API C, que espelha o cliente `libmysqlclient`. Esta API C é basicamente um encapsulador em torno do protocolo clienten-servidor MySQL, ou protocolo de fio, como às vezes é chamado. Pode-se interceptar todas as chamadas da API C. O PHP faz uso da API C, portanto pode-se conectar todas as chamadas PHP, sem a necessidade de programar no nível do protocolo de fio.

`Mysqlnd` implementa o protocolo de fio. Os plugins podem, portanto, analisar, fazer engenharia reversa, manipular e até mesmo substituir o protocolo de comunicação. No entanto, isso geralmente não é necessário.

Como os plugins permitem criar implementações que utilizam dois níveis (API C e protocolo de fio), eles têm maior flexibilidade que o Proxy MySQL. Se um plugin `mysqlnd` for implementado usando a API C, quaisquer alterações subsequentes no protocolo de fio não requerem alterações no próprio plugin.

Obtendo a API do plugin mysqlnd

A API do plugin `mysqlnd` é simplesmente parte da extensão PHP do Driver Nativo MySQL, `ext/mysqlnd`. O desenvolvimento da API do plugin `mysqlnd` começou em dezembro de 2009. Ele é desenvolvido como parte do repositório de código-fonte PHP e, como tal, está disponível ao público via Git ou por meio de downloads de snapshots de código-fonte.

Os desenvolvedores do plugin podem determinar a versão do `mysqlnd` acessando `MYSQLND_VERSION`, que é uma string no formato "mysqlnd 8.3.17" ou por meio de `MYSQLND_VERSION_ID`, que é um número inteiro como 50007. Os desenvolvedores podem calcular o número da versão da seguinte maneira:

Tabela de cálculo de MYSQLND_VERSION_ID

Versão (parte)	Exemplo
Maior*10000	5*10000 = 50000
Menor*100	0*100 = 0
Correção	7 = 7
MYSQLND_VERSION_ID	50007

Durante o desenvolvimento, os desenvolvedores devem consultar o número da versão do `mysqlnd` para compatibilidade e testes de versão, já que diversas iterações do `mysqlnd` podem ocorrer durante o tempo de vida de uma ramificação de desenvolvimento PHP com um único número de versão PHP.

Arquitetura do Plugin do Driver Nativo MySQL

Esta seção fornece uma visão geral da arquitetura do plugin `mysqlnd`.

Visão Geral do Driver Nativo MySQL

Antes de desenvolver plugins `mysqlnd`, é útil saber um pouco de como o próprio `mysqlnd` é organizado. `Mysqlnd` consiste nos seguintes módulos:

O organograma mysqlnd, por módulo

Estatísticas de Módulos	mysqlnd_statistics.c
Conexão	mysqlnd.c
Conjunto de resultados	mysqlnd_result.c

Estatísticas de Módulos	mysqlnd_statistics.c
Metadados do conjunto de resultados	mysqlnd_result_meta.c
Declaração	mysqlnd_ps.c
Rede	mysqlnd_net.c
Protocolo de fio	mysqlnd_wireprotocol.c

Paradigma de C Orientado a Objetos

No nível do código, `mysqlnd` usa um padrão C para implementar orientação a objetos.

Em C você usa uma `struct` para representar um objeto. Os membros da estrutura representam propriedades do objeto. Membros da estrutura que apontam para funções representam métodos.

Ao contrário de outras linguagens como C++ ou Java, não existem regras fixas sobre herança no paradigma do C orientado a objetos. No entanto, existem algumas convenções que precisam ser seguidas e que serão discutidas posteriormente.

O Ciclo de Vida do PHP

Ao considerar o ciclo de vida do PHP, existem dois ciclos básicos:

- Ciclo de inicialização e desligamento do mecanismo PHP
- Ciclo de requisição

Quando o mecanismo PHP for iniciado ele chamará a função de inicialização do módulo (MINIT) de cada extensão registrada. Isso permite que cada módulo configure variáveis e aloque recursos que existirão durante a vida útil do processo do mecanismo PHP. Quando o mecanismo PHP for desligado, ele chamará a função de desligamento do módulo (MSHUTDOWN) de cada extensão.

Durante a vida útil do mecanismo PHP, ele receberá diversas solicitações. Cada solicitação constitui outro ciclo de vida. Em cada solicitação, o mecanismo PHP chamará a função de inicialização de solicitação de cada extensão. A extensão pode executar qualquer configuração de variável e alocação de recursos necessária para o processamento de solicitações. À medida que o ciclo de solicitação termina, o mecanismo chama a função de desligamento de solicitação (RSHUTDOWN) de cada extensão para que ela possa realizar qualquer limpeza necessária.

Como um plugin funciona

Um plugin `mysqlnd` funciona interceptando chamadas feitas para `mysqlnd` por extensões que usam `mysqlnd`. Isto é conseguido obtendo a tabela de funções `mysqlnd`, fazendo backup dela e substituindo-a por uma tabela de funções customizada, que chama as funções do plugin conforme necessário.

O código a seguir mostra como a tabela de funções `mysqlnd` é substituída:

```
/* um lugar para armazenar a tabela de funções original */
struct st_mysqlnd_conn_methods org_methods;

void minit_register_hooks(TSRMLS_D) {
  /* tabela de funções ativas */
  struct st_mysqlnd_conn_methods * current_methods
    = mysqlnd_conn_get_methods();

  /* tabela de funções original de backup */
```

```
  memcpy(&org_methods, current_methods,
    sizeof(struct st_mysqlnd_conn_methods);

  /* instala novos métodos */
  current_methods->query = MYSQLND_METHOD(my_conn_class, query);
}
```

As manipulações da tabela de funções de conexão devem ser feitas durante a inicialização do módulo (MINIT). A tabela de funções é um recurso global compartilhado. Em um ambiente multithread, com construção de TSRM, a manipulação de um recurso global compartilhado durante o processamento da solicitação quase certamente resultará em conflitos.

> **Nota** :
>
> Não use nenhuma lógica de tamanho fixo ao manipular a tabela de funções `mysqlnd`: novos métodos podem ser adicionados ao final da tabela de funções. A tabela de funções pode mudar a qualquer momento no futuro.

Chamando métodos da classe pai

Se for feito backup das entradas originais da tabela de funções, ainda será possível chamar as entradas originais da tabela de funções - os métodos pai.

Em alguns casos, como em `Connection::stmt_init()`, é vital chamar o método pai antes de qualquer outra atividade no método derivado.

```
MYSQLND_METHOD(my_conn_class, query)(MYSQLND *conn,
  const char *query, unsigned int query_len TSRMLS_DC) {

  php_printf("my_conn_class::query(query = %s)\n", query);

  query = "SELECT 'query rewritten' FROM DUAL";
  query_len = strlen(query);

  return org_methods.query(conn, query, query_len); /* retorna com chamada ao método pai */
}
```

Estendendo propriedades

Um objeto `mysqlnd` é representado por uma estrutura C. Não é possível adicionar um membro a uma estrutura C em tempo de execução. Usuários de objetos `mysqlnd` não podem simplesmente adicionar propriedades aos objetos.

Dados arbitrários (propriedades) podem ser adicionados a um objeto `mysqlnd` usando uma função apropriada da família `mysqlnd_plugin_get_plugin_<object>_data()`. Ao alocar um objeto, `mysqlnd` reserva espaço no final do objeto para conter um ponteiro `void *` para dados arbitrários. `mysqlnd` reserva espaço para um ponteiro `void *` por plugin.

A tabela a seguir mostra como calcular a posição do ponteiro para um plugin específico:

Cálculo de ponteiro para mysqlnd

Endereço de memória	Conteúdo
0	Início da estrutura C do objeto mysqlnd

Endereço de memória	Conteúdo
n	Final da estrutura C do objeto mysqlnd
n + (m x sizeof(void*))	void* para dados do objeto do m-ésimo plugin

Se for planejada uma sub-classe de qualquer dos construtores de objeto `mysqlnd`, que é permitido, deve-se ter isto em mente!

O código a seguir mostra extensão de propriedades:

```
/* qualquer dado que se queira associar */
typedef struct my_conn_properties {
  unsigned long query_counter;
} MY_CONN_PROPERTIES;

/* id do plugin */
unsigned int my_plugin_id;

void minit_register_hooks(TSRMLS_D) {
  /* obtém ID único para o plugin */
  my_plugin_id = mysqlnd_plugin_register();
  /* recorte - consulte Estendendo Conexão: métodos */
}

static MY_CONN_PROPERTIES** get_conn_properties(const MYSQLND *conn TSRMLS_DC) {
  MY_CONN_PROPERTIES** props;
  props = (MY_CONN_PROPERTIES**)mysqlnd_plugin_get_plugin_connection_data(
    conn, my_plugin_id);
  if (!props || !(*props)) {
    *props = mnd_pecalloc(1, sizeof(MY_CONN_PROPERTIES), conn->persistent);
    (*props)->query_counter = 0;
  }
  return props;
}
```

O desenvolvedor do plugin é responsável pelo gerenciamento de memória dos dados do plugin.

O uso do alocador de memória `mysqlnd` é recomendado para dados do plugin. Essas funções são nomeadas usando a convenção: `mnd_*loc()`. O alocador `mysqlnd` possui alguns recursos úteis, como a capacidade de usar um alocador de depuração em uma compilação sem depuração.

Quando e como usar uma sub-classe

	Quando usar uma sub-classe?	Cada instância tem sua própria tabela de funções privadas?	Como usar uma sub-classe
Conexão (MYSQLND)	MINIT	Não	mysqlnd_conn_get_methods()
Conjunto de resultados (MYSQLND_RES)	MINIT ou depois	Sim	mysqlnd_result_get_methods() ou manipulação de tabela de funções de métodos de objeto
Metadados do conjunto de resultados	MINIT	Não	mysqlnd_result_metadata_get_methods()

	Quando usar uma sub-classe?	Cada instância tem sua própria tabela de funções privadas?	Como usar uma sub-classe
(MYSQLND_RES_METADATA)			
Instrução (MYSQLND_STMT)	MINIT	Não	mysqlnd_stmt_get_methods()
Rede (MYSQLND_NET)	MINIT ou depois	Sim	mysqlnd_net_get_methods() ou manipulação de tabela de funções de métodos de objeto
Protocolo de fio (MYSQLND_PROTOCOL)	MINIT ou depois	Sim	mysqlnd_protocol_get_methods() ou manipulação de tabela de funções de métodos de objeto

Não se deve manipular tabelas de funções em nenhum momento posterior ao MINIT se isso não for permitido de acordo com a tabela acima.

Algumas classes contêm um ponteiro para a tabela de funções do método. Todas as instâncias dessa classe compartilharão a mesma tabela de funções. Para evitar o caos, especialmente em ambientes com threads, tais tabelas de funções só devem ser manipuladas durante o MINIT.

Outras classes usam cópias de uma tabela de funções compartilhada globalmente. A cópia da tabela de funções de classe é criada junto com o objeto. Cada objeto usa sua própria tabela de funções. Isso dá duas opções: pode-se manipular a tabela de funções padrão de um objeto no MINIT, e também pode-se refinar os métodos de um objeto sem afetar outras instâncias da mesma classe.

A vantagem da abordagem de tabela de funções compartilhadas é o desempenho. Não há necessidade de copiar uma tabela de funções para cada objeto.

Estado do construtor

Tipo	Alocação, construção, redefinição	Pode ser modificada?	Chamadora
Conexão (MYSQLND)	mysqlnd_init()	Não	mysqlnd_connect()
Conjunto de resultados (MYSQLND_RES)	Alocação: • Connection::result_init() Redefinida e reinicializada durante: • Result::use_result() • Result::store_result	Sim, mas deve-se chamar o método pai!	• Connection::list_fields() • Statement::get_result() • Statement::prepare() (Somente metadados) • Statement::resultMetaData()
Metadados do conjunto de resultados (MYSQLND_RES_METADATA)	Connection::result_meta_init()	Sim, mas deve-se chamar o método pai!	Result::read_result_metadata()
Instrução (MYSQLND_STMT)	Connection::stmt_init()	Sim, mas deve-se	Connection::stmt_init()

Tipo	Alocação, construção, redefinição	Pode ser modificada?	Chamadora
		chamar o método pai!	
Rede (MYSQLND_NET)	mysqlnd_net_init()	Não	Connection::init()
Protocolo de fio (MYSQLND_PROTOCOL)	mysqlnd_protocol_init()	Não	Connection::init()

É altamente recomendável que não se substitua totalmente um construtor. Os construtores realizam alocações de memória. As alocações de memória são vitais para a API do plugin `mysqlnd` e para a lógica do objeto `mysqlnd`. Se não houver preocupação com avisos e houver insistência em conectar os construtores, deve-se pelo menos chamar o construtor pai antes de fazer qualquer coisa no construtor.

Independentemente de todos os avisos, pode ser útil criar sub-classes de construtores. Os construtores são o lugar perfeito para modificar as tabelas de funções de objetos com tabelas de objetos não compartilhados, como Conjunto de Resultados, Rede, Protocolo de Fio.

Estado do destruidor

Tipo	Método derivado deve chamar o pai?	Destruidor
Conexão	sim, após execução do método	free_contents(), end_psession()
Conjunto de resultados	sim, após execução do método	free_result()
Metadados do conjunto de resultados	sim, após execução do método	free()
Instrução	sim, após execução do método	dtor(), free_stmt_content()
Rede	sim, após execução do método	free()
Protocolo de fio	sim, após execução do método	free()

Os destruidores são o local apropriado para liberar propriedades, `mysqlnd_plugin_get_plugin_<object>_data()`.

Os destruidores listados podem não ser equivalentes ao método `mysqlnd` real que libera o próprio objeto. No entanto, eles são o melhor lugar possível para se conectar e liberar os dados do plugin. Tal como acontece com os construtores, pode-se substituir totalmente os métodos, mas isso não é recomendado. Se vários métodos estiverem listados na tabela acima, será necessário conectar todos os métodos listados e liberar os dados do plugin em qualquer método chamado primeiro pelo `mysqlnd`.

O método recomendado para plugins é simplesmente conectar os métodos, liberar memória e chamar a implementação pai imediatamente após isso.

A API do plugin mysqlnd

A seguir está uma lista de funções fornecidas na API do plugin `mysqlnd`:

- mysqlnd_plugin_register()
- mysqlnd_plugin_count()
- mysqlnd_plugin_get_plugin_connection_data()

- mysqlnd_plugin_get_plugin_result_data()
- mysqlnd_plugin_get_plugin_stmt_data()
- mysqlnd_plugin_get_plugin_net_data()
- mysqlnd_plugin_get_plugin_protocol_data()
- mysqlnd_conn_get_methods()
- mysqlnd_result_get_methods()
- mysqlnd_result_meta_get_methods()
- mysqlnd_stmt_get_methods()
- mysqlnd_net_get_methods()
- mysqlnd_protocol_get_methods()

Não existe uma definição formal do que é um plugin e como funciona um mecanismo de plugin.

Os componentes frequentemente encontrados em mecanismos de plugins são:

- Um gerenciador de plugin
- Uma API de plugin
- Serviços (ou módulos) de aplicação
- APIs de serviço (ou APIs de módulo) de aplicação

O conceito do plugin `mysqlnd` emprega esses recursos e, adicionalmente, desfruta de uma arquitetura aberta.

Sem Restrições

Um plugin tem acesso total ao funcionamento interno do `mysqlnd`. Não há limites ou restrições de segurança. Tudo pode ser sobrescrito para implementar algoritmos amigáveis ou hostis. É recomendado que se implante apenas plugins de uma fonte confiável.

Conforme discutido anteriormente, os plugins podem usar ponteiros livremente. Esses ponteiros não são restritos de forma alguma e podem apontar para dados de outro plugin. A aritmética de deslocamento simples pode ser usada para ler os dados de outro plugin.

É recomendado que se escreva plugins cooperativos e que sempre se chame o método pai. Os plugins devem sempre cooperar com o próprio `mysqlnd`.

Questões: um exemplo de encadeamento e cooperação

Extensão	Ponteiro mysqlnd.query()	pilha de chamadas se estiver chamando o pai
ext/mysqlnd	mysqlnd.query()	mysqlnd.query
ext/mysqlnd_cache	mysqlnd_cache.query()	1. mysqlnd_cache.query() 2. mysqlnd.query
ext/mysqlnd_monitor	mysqlnd_monitor.query()	1. mysqlnd_monitor.query()

Extensão	Ponteiro mysqlnd.query()	pilha de chamadas se estiver chamando o pai
		2. mysqlnd_cache.query() 3. mysqlnd.query

Neste cenário, um cache (`ext/mysqlnd_cache`) e um plugin de monitor (`ext/mysqlnd_monitor`) são carregados. Ambas são sub-classes de `Connection::query()`. O registro do plugin acontece em `MINIT` usando a lógica mostrada anteriormente. O PHP chama extensões em ordem alfabética por padrão. Os plugins não reconhecem uns aos outros e não definem dependências de extensão.

Por padrão, os plugins chamam a implementação pai do método de consulta em sua versão derivada do método.

Recapitulação da Extensão PHP

Esta é uma recapitulação do que acontece ao usar um plugin de exemplo, `ext/mysqlnd_plugin`, que expõe a API do plugin C `mysqlnd` ao PHP:

- Qualquer aplicação PHP MySQL tenta estabelecer uma conexão com 192.168.2.29
- A aplicação PHP usará `ext/mysql`, `ext/mysqli` ou `PDO_MYSQL`. Todas as três extensões PHP MySQL usam `mysqlnd` para estabelecer a conexão com 192.168.2.29.
- `Mysqlnd` chama seu método de conexão, que agora é uma sub-classe de `ext/mysqlnd_plugin`.
- `ext/mysqlnd_plugin` chama o gancho do espaço de usuário `proxy::connect()` registrado pelo usuário.
- O gancho do espaço de usuário altera o IP do host de conexão de 192.168.2.29 para 127.0.0.1 e retorna a conexão estabelecida por `parent::connect()`.
- `ext/mysqlnd_plugin` executa o equivalente a `parent::connect(127.0.0.1)` chamando o método `mysqlnd` original para estabelecer uma conexão.
- `ext/mysqlnd` estabelece uma conexão e retorna ao `ext/mysqlnd_plugin`. `ext/mysqlnd_plugin` também retorna.
- Qualquer que seja a extensão PHP MySQL utilizada pela aplicação, ela recebe uma conexão com 127.0.0.1. A própria extensão PHP MySQL retorna ao aplicativo PHP. E o ciclo se fecha.

Começando a construir um plugin mysqlnd

É importante lembrar que um plugin `mysqlnd` é em si uma extensão PHP.

O código a seguir mostra a estrutura básica da função MINIT que será usada no plugin `mysqlnd` típico:

```
/* my_php_mysqlnd_plugin.c */

 static PHP_MINIT_FUNCTION(mysqlnd_plugin) {
  /* globais, entradas ini, recursos, classes */

  /* registra o plugin mysqlnd */
  mysqlnd_plugin_id = mysqlnd_plugin_register();

  conn_m = mysqlnd_get_conn_methods();
  memcpy(org_conn_m, conn_m,
    sizeof(struct st_mysqlnd_conn_methods));
```

```
  conn_m->query = MYSQLND_METHOD(mysqlnd_plugin_conn, query);
  conn_m->connect = MYSQLND_METHOD(mysqlnd_plugin_conn, connect);
 }
```

```
/* my_mysqlnd_plugin.c */

 enum_func_status MYSQLND_METHOD(mysqlnd_plugin_conn, query)(/* ... */) {
  /* ... */
 }
 enum_func_status MYSQLND_METHOD(mysqlnd_plugin_conn, connect)(/* ... */) {
  /* ... */
 }
```

Análise de tarefa: do C ao espaço do usuário

```
class proxy extends mysqlnd_plugin_connection {
  public function connect($host, ...) { .. }
}
mysqlnd_plugin_set_conn_proxy(new proxy());
```

Processo:

1. PHP: usuário registra chamada do plugin
2. PHP: usuário chama qualquer API PHP MySQL para se conectar ao MySQL
3. C: ext/*mysql* chama o método de mysqlnd
4. C: mysqlnd chega em ext/mysqlnd_plugin
5. C: ext/mysqlnd_plugin
    1. Chama a função de retorno no espaço de usuário
    2. Ou o método original de `mysqlnd`, se a função de retorno no espaço de usuário não estiver definida

Deve-se fazer o seguinte:

1. Escrever uma classe "mysqlnd_plugin_connection" em C
2. Aceitar e registrar um objeto proxy através de "mysqlnd_plugin_set_conn_proxy()"
3. Chamar métodos proxy do espaço do usuário a partir do C (otimização - zend_interfaces.h)

Os métodos de objeto do espaço do usuário podem ser chamados usando `call_user_function()` ou pode-se operar em um nível mais próximo do Motor Zend e usar `zend_call_method()`.

Otimização: chamando métrodos a partir do C usando zend_call_method

O trecho de código a seguir mostra o protótipo da função `zend_call_method`, obtido de *zend_interfaces.h*.

```
 ZEND_API zval* zend_call_method(
  zval **object_pp, zend_class_entry *obj_ce,
  zend_function **fn_proxy, char *function_name,
  int function_name_len, zval **retval_ptr_ptr,
```

```
  int param_count, zval* arg1, zval* arg2 TSRMLS_DC
);
```

A API Zend suporta apenas dois argumentos. Pode ser necessário mais, por exemplo:

```
 enum_func_status (*func_mysqlnd_conn__connect)(
  MYSQLND *conn, const char *host,
  const char * user, const char * passwd,
  unsigned int passwd_len, const char * db,
  unsigned int db_len, unsigned int port,
  const char * socket, unsigned int mysql_flags TSRMLS_DC
);
```

Para contornar esse problema é necessário fazer uma cópia de `zend_call_method()` e adicionar um recurso para parâmetros adicionais. Pode-se fazer isso criando um conjunto de macros `MY_ZEND_CALL_METHOD_WRAPPER`.

Chamando o espaço de usuário do PHP

Este trecho de código mostra o método otimizado para chamar uma função de espaço do usuário a partir do C:

```
/* my_mysqlnd_plugin.c */

MYSQLND_METHOD(my_conn_class,connect)(
  MYSQLND *conn, const char *host /* ... */ TSRMLS_DC) {
  enum_func_status ret = FAIL;
  zval * global_user_conn_proxy = fetch_userspace_proxy();
  if (global_user_conn_proxy) {
    /* chama o proxy de espaço de usuário */
    ret = MY_ZEND_CALL_METHOD_WRAPPER(global_user_conn_proxy, host, /*...*/);
  } else {
    /* ou o método mysqlnd original = não faz nada, é transparente */
    ret = org_methods.connect(conn, host, user, passwd,
          passwd_len, db, db_len, port,
          socket, mysql_flags TSRMLS_CC);
  }
  return ret;
}
```

Chamando o espaço de usuário: argumentos simples

```
/* my_mysqlnd_plugin.c */

 MYSQLND_METHOD(my_conn_class,connect)(
  /* ... */, const char *host, /* ...*/) {
  /* ... */
  if (global_user_conn_proxy) {
    /* ... */
    zval* zv_host;
    MAKE_STD_ZVAL(zv_host);
    ZVAL_STRING(zv_host, host, 1);
    MY_ZEND_CALL_METHOD_WRAPPER(global_user_conn_proxy, zv_retval, zv_host /*, ...*/);
    zval_ptr_dtor(&zv_host);
    /* ... */
  }
  /* ... */
}
```

Chamando o espaço de usuário: estruturas como argumentos

```
/* my_mysqlnd_plugin.c */

MYSQLND_METHOD(my_conn_class, connect)(
  MYSQLND *conn, /* ...*/) {
  /* ... */
  if (global_user_conn_proxy) {
    /* ... */
    zval* zv_conn;
    ZEND_REGISTER_RESOURCE(zv_conn, (void *)conn, le_mysqlnd_plugin_conn);
    MY_ZEND_CALL_METHOD_WRAPPER(global_user_conn_proxy, zv_retval, zv_conn, zv_host /*, ...*/);
    zval_ptr_dtor(&zv_conn);
    /* ... */
  }
  /* ... */
}
```

O primeiro argumento de muitos métodos `mysqlnd` é um "objeto" C. Por exemplo, o primeiro argumento do método connect() é um ponteiro para `MYSQLND`. A estrutura MYSQLND representa um objeto de conexão `mysqlnd`.

O ponteiro do objeto de conexão `mysqlnd` pode ser comparado a um identificador de arquivo de E/S padrão. Como um arquivo de E/S padrão, um objeto de conexão `mysqlnd` deve ser vinculado ao espaço do usuário usando o tipo de variável de recurso PHP.

Do C para o espaço de usuário e vice-versa

```
 class proxy extends mysqlnd_plugin_connection {
  public function connect($conn, $host, ...) {
    /* "pre" gancho */
    printf("Conectando-se ao servidor = '%s'\n", $host);
    debug_print_backtrace();
    return parent::connect($conn);
  }

  public function query($conn, $query) {
    /* "pós" gancho */
    $ret = parent::query($conn, $query);
    printf("Consulta = '%s'\n", $query);
    return $ret;
  }
}
mysqlnd_plugin_set_conn_proxy(new proxy());
```

Os usuários de PHP devem ser capazes de chamar a implementação pai de um método substituído.

Como resultado de se criar sub-classes, é possível refinar apenas métodos selecionados e pode-se optar por ter ganchos "pré" ou "pós".

Classe integrada: mysqlnd_plugin_connection::connect()

```
/*  my_mysqlnd_plugin_classes.c */

 PHP_METHOD("mysqlnd_plugin_connection", connect) {
  /* ... simplificada! ... */
  zval* mysqlnd_rsrc;
  MYSQLND* conn;
```

```
  char* host; int host_len;
  if (zend_parse_parameters(ZEND_NUM_ARGS() TSRMLS_CC, "rs",
    &mysqlnd_rsrc, &host, &host_len) == FAILURE) {
    RETURN_NULL();
  }
  ZEND_FETCH_RESOURCE(conn, MYSQLND* conn, &mysqlnd_rsrc, -1,
    "Mysqlnd Connection", le_mysqlnd_plugin_conn);
  if (PASS == org_methods.connect(conn, host, /* simplificado! */ TSRMLS_CC))
    RETVAL_TRUE;
  else
    RETVAL_FALSE;
}
```

PHP Data Objects

Introdução

A extensão `PHP Data Objects` (PDO) define uma interface leve e consistente para acessar bancos de dados no PHP. Cada driver de banco de dados que implementa a interface PDO pode expor recursos específicos do banco de dados como funções de extensão comuns. Observe que não é possível executar nenhuma função de banco de dados usando apenas a extensão PDO; um driver PDO de banco de dados deve ser usado para acessar o servidor de banco de dados.

O PDO fornece uma camada de abstração de *acesso a dados*, o que significa que, independentemente de qual banco de dados estiver sendo usado, as mesmas funções serão usadas para realizar consultas e buscar dados. O PDO *não* fornece uma abstração de *banco de dados*; ele não reescreve o SQL nem emula os recursos ausentes. Uma camada de abstração completa deve ser usada se esse recurso for necessário.

O PDO vem com o PHP.

Instalação/Configuração

Índice

Instalação

Esta extensão está habilitada por padrão. Ela pode ser desabilitada usando a seguinte opção em tempo de compilação: **--disable-pdo**

Instalando o PDO em sistemas Unix

1. O PDO e o driver PDO_SQLITE estão habilitados por padrão. Pode ser preciso habilitar o driver PDO do banco de dados escolhido; consulte a documentação dos drivers PDO de bancos de dados para saber mais sobre isso.

   > **Nota** :
   >
   > Ao compilar o PDO como uma extensão compartilhada (*não recomendado*), todos os drivers PDO *devem* ser carregados *após* o próprio PDO.

2. Ao instalar o PDO como um módulo compartilhado, o arquivo php.ini precisa ser atualizado para que a extensão PDO seja carregada automaticamente quando o PHP for executado. Qualquer driver de banco de dados usado também precisará ser habilitado; certifique-se de que eles estejam listados após a linha extension=pdo, pois o PDO deve ser inicializado antes que as extensões específicas do banco de dados possam ser carregadas. Se o PDO e as extensões de banco de dados foram compilados estaticamente, esta etapa pode ser ignorada.

   ```
   extension=pdo
   ```

Pessoas usuárias do Windows

1. PDO é habilitado por padrão. Selecione os outros arquivos DLL e use a função dl() para carregá-los em tempo de execução ou habilite-os no arquivo *php.ini*. Por exemplo, isto carrega o driver PDO_SQLITE mas deixa o driver PDO_ODBC comentado:

   ```
   ;extension=pdo_odbc
   extension=pdo_sqlite
   ```

   Essas DLLs devem existir no diretório de extensões extension_dir do sistema.

> **Nota** :
>
> Lembre-se que após fazer alterações no arquivo *php.ini*, será necessário reiniciar o PHP para que as novas diretivas de configuração entrem em vigor.

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de Configuração do PDO

Nome	Padrão	Modificável	Registro de Alterações
pdo.dsn.*		Somente *php.ini*	

Aqui está uma breve explicação das diretivas de configuração.

`pdo.dsn.*` string

Define um sinônimo DSN. Veja PDO::__construct() para uma explicação completa.

Constantes predefinidas

Índice

As constantes abaixo são definidas por esta extensão e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução.

Cursores

Veja também **`PDO::ATTR_CURSOR_NAME`**.

`PDO::FETCH_ORI_NEXT` (int)

Busca a próxima linha no conjunto de resultados. Válido apenas para cursores roláveis.

`PDO::FETCH_ORI_PRIOR` (int)

Busca a linha anterior no conjunto de resultados. Válido apenas para cursores roláveis.

PDO::FETCH_ORI_FIRST (int)

Busca a primeira linha no conjunto de resultados. Válido apenas para cursores roláveis.

PDO::FETCH_ORI_LAST (int)

Busca a última linha no conjunto de resultados. Válido apenas para cursores roláveis.

PDO::FETCH_ORI_ABS (int)

Busca a linha solicitada pelo número da linha do conjunto de resultados. Válida somente para cursores roláveis.

PDO::FETCH_ORI_REL (int)

Busca a linha solicitada pela posição relativa à posição atual do cursor no conjunto de resultados. Válida somente para cursores roláveis.

PDO::CURSOR_FWDONLY (int)

Cria um objeto PDOStatement com um cursor somente para frente. Esta é a escolha de cursor padrão, pois é o padrão mais rápido e comum de acesso a dados em PHP.

PDO::CURSOR_SCROLL (int)

Cria um objeto PDOStatement com um cursor rolável. Passe as constantes **PDO::FETCH_ORI_*** para controlar as linhas buscadas do conjunto de resultados.

Outras constantes

PDO::PARAM_BOOL (int)

Representa um tipo de dado booleano.

PDO::PARAM_NULL (int)

Representa o tipo de dado SQL NULL.

PDO::PARAM_INT (int)

Representa o tipo de dados SQL INTEGER.

PDO::PARAM_STR (int)

Representa o tipo de dados SQL CHAR, VARCHAR ou outra string.

PDO::PARAM_STR_NATL (int)

Sinaliza que uma string usa o conjunto de caracteres nacional.

Disponível a partir do PHP 7.2.0

PDO::PARAM_STR_CHAR (int)

Sinaliza que uma string usa o conjunto de caracteres regular.

Disponível a partir do PHP 7.2.0

PDO::PARAM_LOB (int)

Representa o tipo de dados SQL large object.

PDO::PARAM_STMT (int)

Representa um tipo de conjunto de registros. Atualmente não suportado por nenhum driver.

PDO::PARAM_INPUT_OUTPUT (int)

Especifica que o parâmetro é um parâmetro INOUT para um procedimento armazenado. Esse valor deve ser combinado através de OR binário com uma das constantes **PDO::PARAM_***.

PDO::ATTR_AUTOCOMMIT (bool)

Se esse valor for **false**, o PDO tenta desativar o autocommit para que a conexão inicie uma transação.

A partir do PHP 8.4.0, esta constante é tipada como bool; anteriormente, era tipada como int.

PDO::ATTR_PREFETCH (int)

Definir o tamanho da pré-busca permite balancear entre velocidade e uso de memória em uma aplicação. Nem todas as combinações de banco de dados e drivers suportam a definição do tamanho do pré-busca. Um tamanho de pré-busca maior resulta em um aumento de desempenho com o custo de um uso maior de memória.

O driver PDO_PGSQL, por sua vez, trata esse atributo como uma opção: a partir do PHP 8.5.0, o valor `0` habilita a busca lenta (linha única); consulte a documentação do driver PDO_PGSQL para obter detalhes.

PDO::ATTR_TIMEOUT (int)

Define o valor de tempo limite em segundos para comunicações com o banco de dados.

PDO::ATTR_ERRMODE (int)

Consulte a seção Erros e tratamento de erros para obter mais informações sobre este atributo.

PDO::ATTR_SERVER_VERSION (int)

Este é um atributo somente leitura; ele retornará informações sobre a versão do servidor de banco de dados ao qual o PDO está conectado.

PDO::ATTR_CLIENT_VERSION (int)

Este é um atributo somente leitura; ele retornará informações sobre a versão das bibliotecas de cliente que o driver PDO está usando.

PDO::ATTR_SERVER_INFO (int)

Este é um atributo somente leitura; ele retornará algumas informações meta sobre o servidor de banco de dados ao qual o PDO está conectado.

PDO::ATTR_CONNECTION_STATUS (int)

PDO::ATTR_CASE (int)

Força os nomes das colunas para um caso específico definido pelas constantes **PDO::CASE_***.

PDO::ATTR_CURSOR_NAME (int)

Obtém ou define o nome a ser usado para um cursor. Mais útil ao usar cursores roláveis e atualizações posicionadas.

`PDO::ATTR_CURSOR` (int)

Seleciona o tipo de cursor. O PDO atualmente suporta `PDO::CURSOR_FWDONLY` ou `PDO::CURSOR_SCROLL`. A menos que haja necessidade de cursores roláveis, deve-se usar o modo de cursor `PDO::CURSOR_FWDONLY`.

`PDO::ATTR_DRIVER_NAME` (int)

Retorna o nome do driver.

Exemplo #1 using `PDO::ATTR_DRIVER_NAME`

```
<?php
 if ($db->getAttribute(PDO::ATTR_DRIVER_NAME) == 'mysql') {
   echo "Running on mysql; doing something mysql specific here\n";
 }
 ?>
```

`PDO::ATTR_ORACLE_NULLS` (int)

Converte strings vazias em valores SQL NULL em buscas de dados.

`PDO::ATTR_PERSISTENT` (int)

Solicita uma conexão persistente, em vez de criar uma nova conexão. Consulte Conexões e Gerenciamento de Conexões para obter mais informações sobre este atributo.

`PDO::ATTR_STATEMENT_CLASS` (int)

Define o nome da classe e os argumentos do construtor para a extensão do objeto PDOStatement. O primeiro elemento do array é o nome da classe, e o segundo elemento é um array de argumentos do construtor.

`PDO::ATTR_FETCH_CATALOG_NAMES` (int)

Adiciona o nome do catálogo contenedor a cada nome de coluna retornado no conjunto de resultados. O nome do catálogo e o nome da coluna são separados por um caractere de ponto (.). O suporte deste atributo é no nível do driver; pode não ser suportado pelo driver em uso.

`PDO::ATTR_FETCH_TABLE_NAMES` (int)

Adiciona o nome da tabela contenedora a cada nome de coluna retornado no conjunto de resultados. O nome da tabela e o nome da coluna são separados por um caractere de ponto (.). O suporte deste atributo é no nível do driver; pode não ser suportado pelo driver em uso.

`PDO::ATTR_STRINGIFY_FETCHES` (int)

Força todos os valores buscados (exceto `null`) a serem tratados como strings. Valores `null` permanecem inalterados a menos que `PDO::ATTR_ORACLE_NULLS` esteja definida para `PDO::NULL_TO_STRING`.

`PDO::ATTR_MAX_COLUMN_LEN` (int)

Define o comprimento máximo do nome da coluna.

`PDO::ATTR_DEFAULT_FETCH_MODE` (int)

PDO::ATTR_EMULATE_PREPARES (bool)

Se deve habilitar ou desabilitar a emulação de instruções preparadas. Alguns drivers não suportam instruções preparadas nativas ou têm suporte apenas limitado a elas. Quando definido como **true**, o PDO sempre emula instruções preparadas; quando definido como **false**, ele usa as instruções preparadas nativas do driver.

A partir do PHP 8.4.0, esta constante é tipada como bool; anteriormente, era tipada como int.

PDO::ATTR_DEFAULT_STR_PARAM (int)

Define o tipo de parâmetro de string padrão, que pode ser uma das constantes **PDO::PARAM_STR_NATL** ou **PDO::PARAM_STR_CHAR**.

Disponível a partir do PHP 7.2.0.

PDO::ERRMODE_SILENT (int)

Não gera um erro ou exceção se ocorrer um erro. É esperado que o próprio desenvolvedor verifique explicitamente os erros. Anteriormente ao PHP 8.0.0, este era o modo padrão. Consulte Erros e tratamento de erros para mais informações sobre este atributo.

PDO::ERRMODE_WARNING (int)

Emite uma mensagem PHP **E_WARNING** se ocorrer um erro. Consulte Erros e tratamento de erros para mais informações sobre este atributo.

PDO::ERRMODE_EXCEPTION (int)

Lança uma PDOException se ocorrer um erro. Este é o modo padrão a partir do PHP 8.0.0. Consulte Erros e tratamento de erros para mais informações sobre este atributo.

PDO::CASE_NATURAL (int)

Deixa os nomes das colunas como retornados pelo driver do banco de dados.

PDO::CASE_LOWER (int)

Força os nomes das colunas em minúsculas.

PDO::CASE_UPPER (int)

Força os nomes das colunas em maiúsculas.

PDO::NULL_NATURAL (int)

PDO::NULL_EMPTY_STRING (int)

PDO::NULL_TO_STRING (int)

PDO::ERR_NONE (string)

Corresponde a SQLSTATE `'00000'`, significando que a instrução SQL foi emitida com sucesso sem erros ou avisos. Esta constante é uma conveniência ao verificar PDO::errorCode() ou PDOStatement::errorCode() para determinar se ocorreu um erro. De qualquer forma, isto geralmente é determinado pela verificação do código de retorno do método que gerou a condição de erro.

PDO::PARAM_EVT_ALLOC (int)

Evento de alocação

PDO::PARAM_EVT_FREE (int)

Evento de desalocação

PDO::PARAM_EVT_EXEC_PRE (int)

Evento acionado antes da execução de uma instrução preparada.

PDO::PARAM_EVT_EXEC_POST (int)

Evento acionado após a execução de uma instrução preparada.

PDO::PARAM_EVT_FETCH_PRE (int)

Evento acionado antes de buscar um resultado de um conjunto de resultados.

PDO::PARAM_EVT_FETCH_POST (int)

Evento acionado após buscar um resultado de um conjunto de resultados.

PDO::PARAM_EVT_NORMALIZE (int)

Evento acionado durante o registro de parâmetro vinculado, permitindo ao driver normalizar o nome do parâmetro.

PDO::SQLITE_DETERMINISTIC (int)

> **Aviso**
>
> Esta constante foi *DESCONTINUADA* a partir do PHP 8.5.0. Use **Pdo\Sqlite::DETERMINISTIC** em seu lugar.

Especifica que uma função criada com PDO::sqliteCreateFunction() é determinística, ou seja, ela sempre retorna o mesmo resultado dados os mesmos inputs dentro de uma única instrução SQL. (Disponível a partir do PHP 7.1.4.)

Modos de Busca

Consulte as constantes de cursores para as constantes `PDO::FETCH_ORI_*`.

Modos de Busca Básicos

Modo de Busca	Resumo
PDO::FETCH_DEFAULT	Especifica que o modo de busca padrão deve ser usado.
PDO::FETCH_ASSOC	Array indexado somente pelo nome da coluna.
PDO::FETCH_BOTH (Padrão)	Array indexado tanto pelo nome quanto pelo número da coluna.
PDO::FETCH_NAMED	Variante de **PDO::FETCH_ASSOC** que retém colunas duplicadas.
PDO::FETCH_NUM	Array indexado somente pelo número da coluna.
PDO::FETCH_COLUMN	Uma única coluna.
PDO::FETCH_KEY_PAIR	Pares de chave-valor, indexados pela primeira coluna.
PDO::FETCH_FUNC	Usa uma função para criar o valor de retorno. (PDOStatement::fetchAll() somente)

Modo de Busca	Resumo
`PDO::FETCH_OBJ`	Objeto anônimo (stdClass).
`PDO::FETCH_CLASS`	Um objeto de uma classe especificada.

Opções de PDO::FETCH_CLASS

Estes modos são usados para implementar opções ao usar `PDO::FETCH_CLASS`.

Modo de busca	Resumo
`PDO::FETCH_CLASSTYPE`	Usa a primeira coluna como o nome da classe.
`PDO::FETCH_PROPS_LATE`	Chama o construtor antes de definir as propriedades.
`PDO::FETCH_SERIALIZE`	Usa dados serializados do PHP. Descontinuado a partir do PHP 8.1.0.

Modos de resultado único

Os modos a seguir não podem ser usados com PDOStatement::fetchAll().

Modo de busca	Resumo
`PDO::FETCH_BOUND`	Vincula valores a variáveis especificadas.
`PDO::FETCH_INTO`	Atualiza um objeto existente.
`PDO::FETCH_LAZY`	Busca lenta via PDORow para acessos como array e objetos.

Sinalizadores de comportamento especiais para PDOStatement::fetchAll()

Os seguintes modos especiais para múltiplos resultados funcionam apenas com PDOStatement::fetchAll() e não funcionam com alguns outros modos de busca. Consulte a documentação completa para obter detalhes.

Modo de busca	Resumo
`PDO::FETCH_GROUP`	Os resultados estão agrupados pela primeira coluna.
`PDO::FETCH_UNIQUE`	Os resultados são indexados (exclusivamente) pela primeira coluna.

Tratamento de nomes de colunas duplicados

É possível que os resultados contenham várias colunas com o mesmo nome. Por exemplo, ao unir duas tabelas que contêm uma coluna com o mesmo nome.

Como as estruturas do PHP, como arrays e objetos, não suportam múltiplas chaves ou propriedades com o mesmo nome, o array ou objeto retornado conterá apenas 1 dos valores com o mesmo nome.

O valor retornado para um determinado nome duplicado deve ser considerado indefinido.

Para evitar esse problema, nomeie as colunas explicitamente usando um sinônimo. Por exemplo:

```
SELECT table1.created_at AS t1_created_at,
       table2.created_at AS t2_created_at
FROM table1
JOIN table2 ON table1.table2id = table2.id
```

Consulte também `PDO::FETCH_NAMED`, `PDO::ATTR_FETCH_TABLE_NAMES` e `PDO::ATTR_FETCH_CATALOG_NAMES`.

Definindo o modo de busca padrão

É possível definir o modo de busca padrão para todas as consultas usando `PDO::ATTR_DEFAULT_FETCH_MODE` com PDO::__construct() ou PDO::setAttribute().

É possível definir o modo de busca padrão para uma instrução específica usando PDOStatement::setFetchMode(). Isso afeta a reutilização como uma instrução preparada e a iteração (usando `foreach`).

> **Cuidado**
>
> PDOStatement::setAttribute() não pode ser usado para definir o modo de busca padrão. Ele aceita apenas atributos específicos do driver e ignora silenciosamente os atributos que não são reconhecidos.

PDO::FETCH_DEFAULT (int)

Disponível a partir do PHP 8.0.7.

Este é um valor especial que usa o modo de busca padrão atual para um PDOStatement. É particularmente útil como o valor padrão para parâmetros de método ao estender PDOStatement para uso com `PDO::ATTR_STATEMENT_CLASS`.

Este valor não pode ser usado com `PDO::ATTR_DEFAULT_FETCH_MODE`.

PDO::FETCH_ASSOC (int)

`PDO::FETCH_ASSOC` retorna um array indexado apenas pelo nome da coluna.

```php
<?php
$stmt = $pdo->query("SELECT userid, name, country FROM users");
$row = $stmt->fetch(\PDO::FETCH_ASSOC);
print_r($row);
```

O exemplo acima produzirá:

```
Array
(
    [userid] => 104
    [name] => Chris
    [country] => Ukraine
)
```

PDO::FETCH_BOTH (int)

Este é o modo de busca padrão.

`PDO::FETCH_BOTH` retorna uma matriz indexada tanto pelo número da coluna quanto pelo nome. Isso significa que cada valor retornado é duplicado para cada linha de resultado.

O número da coluna começa em 0 e é determinado pela ordem da coluna de resultado na consulta, não (por exemplo) pela ordem em que as colunas são definidas na tabela.

> **Nota** :
>
> Não é recomendável usar o índice numérico de coluna, pois ele pode mudar quando a consulta for alterada ou quando o esquema da tabela for alterado ao usar `SELECT *`.

> **Nota** :
>
> O número de entradas indexadas por nome pode não corresponder ao número de entradas indexadas por número nos casos em que várias colunas retornadas usam o mesmo nome.

```php
<?php
$stmt = $pdo->query("SELECT userid, name, country FROM users");
$row = $stmt->fetch(\PDO::FETCH_BOTH);
print_r($row);
```

O exemplo acima produzirá:

```
Array
(
    [id] => 104,
    [0] => 104,
    [name] => Chris,
    [1] => Chris,
    [country] => Ukraine,
    [2] => Ukraine
)
```

PDO::FETCH_NAMED (int)

`PDO::FETCH_NAMED` retorna resultados no mesmo formato que `PDO::FETCH_ASSOC` exceto que, quando várias colunas usam o mesmo nome, todos os valores são retornados como uma lista.

Para obter mais informações sobre o tratamento de nomes de colunas duplicados e alternativas, consulte a seção tratamento de nomes duplicados acima.

A ordem em que os valores duplicados são retornados deve ser considerada indefinida. Não há como saber de onde cada valor veio.

```php
<?php
$stmt = $pdo->query(
    "SELECT users.*, referrer.name
     FROM users
     LEFT JOIN users AS referrer ON users.referred_by = referrer.userid
     WHERE userid = 109"
);
$row = $stmt->fetch(\PDO::FETCH_NUM);
print_r($row);
```

O exemplo acima produzirá:

```
Array
(
    [userid] => 109
    [name] => Array
        (
            [0] => Toni
            [1] => Chris
        )
    [country] => Germany
    [referred_by] = 104
)
```

PDO::FETCH_NUM (int)

`PDO::FETCH_NUM` retorna um array indexado apenas pelo número da coluna. O número da coluna começa em 0 e é determinado pela ordem da coluna de resultado na consulta, não (por exemplo) pela ordem em que as colunas são definidas na tabela.

> **Nota** :
>
> Não é recomendável usar o índice de coluna numérica, pois ele pode mudar quando a consulta for alterada ou quando o esquema da tabela for alterado ao usar `SELECT *`.

```php
<?php
$stmt = $pdo->query("SELECT userid, name, country FROM users");
$row = $stmt->fetch(\PDO::FETCH_NUM);
print_r($row);
```

O exemplo acima produzirá:

```
Array
(
    [0] => 104
    [1] => Chris
    [2] => Ukraine
)
```

PDO::FETCH_COLUMN (int)

`PDO::FETCH_COLUMN` retorna valores de uma única coluna. Use o segundo argumento de PDOStatement::setFetchMode() ou PDOStatement::fetchAll() para especificar qual coluna será retornada.

Se a coluna especificada não existir, será lançado um ValueError.

```php
<?php
$stmt = $pdo->query("SELECT name, country FROM users LIMIT 3");
$row = $stmt->fetchAll(\PDO::FETCH_COLUMN);
print_r($row);

$stmt = $pdo->query("SELECT name, country FROM users LIMIT 3");
$row = $stmt->fetchAll(\PDO::FETCH_COLUMN, 1);
print_r($row);
```

O exemplo acima produzirá:

```
Array
(
    [0] => Chris
    [1] => Jamie
    [2] => Robin
)

Array
(
    [0] => Ukraine
    [1] => England
    [2] => Germany
)
```

PDO::FETCH_KEY_PAIR (int)

`PDO::FETCH_KEY_PAIR` retorna pares de valores, indexados pela primeira coluna. Os resultados devem conter apenas 2 colunas. Este modo de busca só faz sentido com PDOStatement::fetchAll().

> **Nota** :
>
> Se a primeira coluna não for única, valores serão perdidos. Quais valores serão perdidos devem ser considerados indefinidos.

```php
<?php
$stmt = $pdo->query("SELECT name, country FROM users LIMIT 3");
$row = $stmt->fetchAll(\PDO::FETCH_KEY_PAIR);
print_r($row);
```

O exemplo acima produzirá:

```
Array
(
    [Chris] => Ukraine
    [Jamie] => England
    [Robin] => Germany
)
```

PDO::FETCH_FUNC (int)

Especifique uma função para criar o valor retornado. Este modo só pode ser usado com PDOStatement::fetchAll().

A função recebe os valores como parâmetros. Não há como recuperar o nome da coluna à qual um determinado valor estava associado. É crucial garantir que a ordem das colunas na consulta corresponda à esperada pela função.

> **Nota** :
>
> Os efeitos de `PDO::FETCH_GROUP` e `PDO::FETCH_UNIQUE` são aplicados aos resultados antes da função ser chamada.

```php
<?php
function valueCreator($col1, $col2, $col3)
{
    return [
        'col1' => $col1,
        'col2' => strtoupper($col2),
        'col3' => $col3,
        'customKey' => 'customValue',
    ];
}

$stmt = $pdo->query("SELECT userid, name, country FROM users LIMIT 3");
$row = $stmt->fetchAll(\PDO::FETCH_FUNC, valueCreator(...));
print_r($row);
```

O exemplo acima produzirá:

```
Array
(
    [0] => Array
        (
            [col1] => 104
            [col2] => SAM
            [col3] => Ukraine
            [customKey] => customValue
        )

    [1] => Array
        (
            [col1] => 105
            [col2] => JAMIE
            [col3] => England
            [customKey] => customValue
        )

    [2] => Array
        (
```

```
            [col1] => 107
            [col2] => ROBIN
            [col3] => Germany
            [customKey] => customValue
        )

)
```

PDO::FETCH_OBJ (int)

`PDO::FETCH_OBJ` retorna um objeto stdClass.

Consulte também PDOStatement::fetchObject() e `PDO::FETCH_CLASS`.

```php
<?php
$stmt = $pdo->query("SELECT userid, name, country FROM users");
$row = $stmt->fetch(\PDO::FETCH_OBJ);
print_r($row);
```

O exemplo acima produzirá:

```
stdClass Object
(
    [userid] => 104
    [name] => Chris
    [country] => Ukraine
)
```

PDO::FETCH_CLASS (int)

Retorna um objeto de uma classe especificada. Para comportamentos adicionais, consulte os sinalizadores de opções: sinalizadores de opções.

Se uma propriedade com o nome de uma coluna retornada não existir, ela será declarada dinamicamente. Esse comportamento foi descontinuado e causará um erro a partir do PHP 9.0.

Consulte também PDOStatement::fetchObject().

```php
<?php
class TestEntity
{
    public $userid;

    public $name;

    public $country;

    public $referred_by_userid;

    public function __construct()
```

```
    {
        print "Construtor chamado com ". count(func_get_args()) ." argumentos\n";
        print "Propriedades definidas quando o construtor foi chamado? "
            . (isset($this->name) ? 'Sim' : 'Não') . "\n";
    }
}

$stmt = $db->query(
    "SELECT userid, name, country, referred_by_userid FROM users"
);
$stmt->setFetchMode(PDO::FETCH_CLASS, TestEntity::class);
$result = $stmt->fetch();
var_dump($result);
```

O exemplo acima produzirá algo semelhante a:

```
Construtor chamado com 0 argumentoss
Propriedades definidas quando o construtor foi chamado? Sim
object(TestEntity)#3 (4) {
  ["userid"]=>
  int(104)
  ["name"]=>
  string(5) "Chris"
  ["country"]=>
  string(7) "Ukraine"
  ["referred_by_userid"]=>
  NULL
}
```

PDO::FETCH_CLASSTYPE (int)

Este modo de busca só pode ser usado em combinação com **PDO::FETCH_CLASS** (e suas outras opções).

Quando este modo de busca é usado, o PDO usará a primeira coluna retornada como o nome da classe a ser retornada.

Se a classe especificada não for encontrada, um objeto stdClass será retornado, sem aviso ou erro.

```
<?php
class TestEntity
{
    public $userid;

    public $name;

    public $country;

    public $referred_by_userid;

    public function __construct()
    {
        print "Construtor chamado com ". count(func_get_args()) ." argumentos\n";
        print "Propriedades definidas quando o construtor foi chamado? "
```

```
            . (isset($this->name) ? 'Sim' : 'Não') . "\n";
    }
}

$stmt = $db->query(
    "SELECT 'TestEntity', userid, name, country, referred_by_userid FROM users"
);
$stmt->setFetchMode(PDO::FETCH_CLASS | PDO::FETCH_CLASSTYPE);
$result = $stmt->fetch();
var_dump($result);
```

O exemplo acima produzirá algo semelhante a:

```
Construtor chamado com 0 argumentos
Propriedades definidas quando o construtor foi chamado? Sim
object(TestEntity)#3 (4) {
  ["userid"]=>
  int(104)
  ["name"]=>
  string(5) "Chris"
  ["country"]=>
  string(7) "Ukraine"
  ["referred_by_userid"]=>
  NULL
}
```

PDO::FETCH_PROPS_LATE (int)

Este modo de busca só pode ser usado em combinação com **PDO::FETCH_CLASS** (e suas outras opções).

Quando este modo de busca é usado, o construtor será chamado antes que as propriedades sejam definidas.

```
<?php
class TestEntity
{
    public $userid;

    public $name;

    public $country;

    public $referred_by_userid;

    public function __construct()
    {
        print "Construtor chamado com ". count(func_get_args()) ." argumentos\n";
        print "Propriedades definidas quando o construtor foi chamado? "
            . (isset($this->name) ? 'Sim' : 'Não') . "\n";
    }
}

$stmt = $db->query(
    "SELECT userid, name, country, referred_by_userid FROM users"
```

```
);
$stmt->setFetchMode(PDO::FETCH_CLASS | PDO::FETCH_PROPS_LATE, TestEntity::class);
$result = $stmt->fetch();
var_dump($result);
```

O exemplo acima produzirá algo semelhante a:

```
Construtor chamado com 0 argumentos
Propriedades definidas quando o construtor foi chamado? Não
object(TestEntity)#3 (4) {
  ["userid"]=>
  int(104)
  ["name"]=>
  string(5) "Chris"
  ["country"]=>
  string(7) "Ukraine"
  ["referred_by_userid"]=>
  NULL
}
```

PDO::FETCH_SERIALIZE (int)

> **Aviso**
>
> Este recurso foi *DESCONTINUADO* a partir do PHP 8.1.0. O uso deste recurso é fortemente desencorajado.

Este modo de busca só pode ser usado em combinação com **PDO::FETCH_CLASS** (e suas outras opções).

Quando este modo de busca é utilizado, a classe especificada deve ser Serializable.

> **Cuidado**
>
> Este recurso não suporta uma string que contenha um objeto serializado completo (com serialize()).

> **Cuidado**
>
> Este modo de busca não chama o construtor.

```
<?php
class TestEntity implements Serializable
{
    public $userid;

    public $name;

    public $country;

    public $referred_by_userid;

    public function __construct()
    {
```

```
        print "Construtor chamado com " . count(func_get_args()) . " argumentos\n";
        print "Propriedades definidas quando o construtor é chamado? "
            . (isset($this->name) ? 'Sim' : 'Não') . "\n";
    }

    public function serialize()
    {
        return join(
            "|",
            [$this->userid, $this->name, $this->country, $this->referred_by_userid]
        );
    }

    public function unserialize(string $data)
    {
        $parts = explode("|", $data);
        $this->userid = (int) $parts[0];
        $this->name = $parts[1];
        $this->country = $parts[2];

        $refId = $parts[3];
        $this->referred_by_userid = ($refId === "" ? null : (int) $refId);
    }
}

print "Definindo o registro (construtor chamado manualmente):\n";
$db->exec(
    "CREATE TABLE serialize (
        sdata TEXT
    )"
);

$origObj = new TestEntity();
$origObj->userid = 200;
$origObj->name = 'Seri';
$origObj->country = 'Syria';
$origObj->referred_by_userid = null;

$insert = $db->prepare("INSERT INTO serialize (sdata) VALUES (:sdata)");
$insert->execute(['sdata' => $origObj->serialize()]);

print "\nRecuperando o resultado:\n";
$query = "SELECT sdata FROM serialize";
$stmt = $db->query($query);
// NOTA: O construtor nunca é chamado!
$stmt->setFetchMode(PDO::FETCH_CLASS | PDO::FETCH_SERIALIZE, TestEntity::class);
$result = $stmt->fetch();
var_dump($result);
```

O exemplo acima produzirá algo semelhante a:

```
Deprecated: TestEntity implements the Serializable interface, which is deprecated. Implement
__serialize() and __unserialize() instead (or in addition, if support for old PHP versions is
necessary) in Standard input code on line 2
Definindo o registro (construtor chamado manualmente):
Construtor chamado com 0 argumentos
```

```
Propriedades definidas quando o construtor é chamado? Não

Recuperando o resultado:
Deprecated: PDOStatement::setFetchMode(): The PDO::FETCH_SERIALIZE mode is deprecated in Standard
input code on line 58

Deprecated: PDOStatement::fetch(): The PDO::FETCH_SERIALIZE mode is deprecated in Standard input
code on line 59
object(TestEntity)#5 (4) {
  ["userid"]=>
  int(200)
  ["name"]=>
  string(4) "Seri"
  ["country"]=>
  string(5) "Syria"
  ["referred_by_userid"]=>
  NULL
}
```

PDO::FETCH_BOUND (int)

Este modo de busca não pode ser usado com PDOStatement::fetchAll().

Este modo de busca não retorna um resultado diretamente, mas associa valores a variáveis especificadas com PDOStatement::bindColumn(). O método de busca chamado retorna **`true`**.

> **Nota** :
>
> Ao usar instruções preparadas, para funcionar corretamente, as variáveis devem ser vinculadas após a execução da consulta.

```php
<?php
$query = "SELECT users.userid, users.name, users.country, referrer.name
    FROM users
    LEFT JOIN users AS referrer ON users.referred_by_userid = referrer.userid";
$stmt = $db->prepare($query);
$stmt->execute();

$stmt->bindColumn('userid', $userId);
$stmt->bindColumn('name', $name);
$stmt->bindColumn('country', $country);
// Vincule pela posição da coluna para resolver nomes de coluna duplicados
// Para evitar que isso cause problemas se a consulta for alterada, use um sinônimo SQL.
// Por exemplo: referrer.name AS referrer_name
$stmt->bindColumn(4, $referrerName);

while ($stmt->fetch(\PDO::FETCH_BOUND)) {
    print join("\t", [$userId, $name, $country, ($referrerName ?? 'NULL')]) . "\n";
}
```

O exemplo acima produzirá:

```
104     Chris   Ukraine NULL
105     Jamie   England NULL
107     Robin   Germany Chris
108     Sean    Ukraine NULL
109     Toni    Germany NULL
110     Toni    Germany NULL
```

PDO::FETCH_INTO (int)

Este modo de busca não pode ser usado com PDOStatement::fetchAll().

Este modo de busca atualiza as propriedades do objeto especificado. O objeto é retornado em caso de sucesso.

Se uma propriedade com o nome de uma coluna retornada não existir, ela será declarada dinamicamente. Esse comportamento está obsoleto e causará um erro a partir do PHP 9.0.

Propriedades precisam ser `public` e não podem ser `readonly`.

> **Cuidado**
>
> Não há como alterar o objeto a ser atualizado sem usar PDOStatement::setFetchMode() entre a recuperação de cada registro.

```php
<?php

class TestEntity
{
    public $userid;

    public $name;

    public $country;

    public $referred_by_userid;
}

$obj = new TestEntity();

$stmt = $db->query("SELECT userid, name, country, referred_by_userid FROM users");

$stmt->setFetchMode(\PDO::FETCH_INTO, $obj);
$result = $stmt->fetch();

var_dump($result);
```

O exemplo acima produzirá algo semelhante a:

```
object(TestEntity)#3 (4) {
  ["userid"]=>
  int(104)
  ["name"]=>
  string(5) "Chris"
```

```
  ["country"]=>
  string(7) "Ukraine"
  ["referred_by_userid"]=>
  NULL
}
```

PDO::FETCH_LAZY (int)

Este modo de busca não pode ser usado com PDOStatement::fetchAll().

Este modo de busca retorna um objeto PDORow que fornece acesso a valores tanto no formato de array quanto no formato de objeto (ou seja, combina o comportamento de `PDO::FETCH_BOTH` e `PDO::FETCH_OBJ`), recuperados de forma lenta.

Isso pode fornecer acesso eficiente em termos de memória (no lado do PHP) a resultados não armazenados em buffer no servidor de banco de dados. Se o PDO usa buffer no lado do cliente para resultados depende do driver específico do banco de dados usado (e de sua configuração).

> **Cuidado**
>
> PDORow retornará `null` sem qualquer erro ou aviso ao acessar propriedades ou chaves que não estejam definidas. Isso pode dificultar a detecção e depuração de erros como erros de digitação ou consultas que não retornam os dados esperados.

> **Cuidado**
>
> O objeto PDORow retornado é atualizado cada vez que um resultado é recuperado.

```php
<?php
$stmt = $db->query("SELECT userid, name, country, referred_by_userid FROM users");
$result = $stmt->fetch(\PDO::FETCH_LAZY);

print "ID: ". $result[0] ."\n";
print "Nome: {$result->name}\n";
print "País: " . $result['country'] ."\n";
// Retorna NULL. Nenhum aviso ou erro é emitido.
print "Não existe: " . var_export($result->does_not_exist, true) . "\n";

$differentResult = $stmt->fetch(\PDO::FETCH_LAZY);
// O PDORow recuperado anteriormente agora aponta para o resultado recém-recuperado.
print "ID: ". $result[0] ."\n";
```

O exemplo acima produzirá:

```
ID: 104
Nome: Chris
País: Ukraine
Não existe: NULL
ID: 105
```

PDO::FETCH_GROUP (int)

PDO::FETCH_GROUP retorna listas de arrays associativos, indexados por uma coluna (não única). Este modo de busca funciona apenas com PDOStatement::fetchAll().

Quando combinado com **PDO::FETCH_UNIQUE**, ambos os modos usarão a mesma coluna, tornando a combinação desses modos inútil.

Esta busca deve ser combinada com uma das seguintes constantes: **PDO::FETCH_ASSOC**, **PDO::FETCH_BOTH**, **PDO::FETCH_NAMED**, **PDO::FETCH_NUM**, **PDO::FETCH_COLUMN** ou **PDO::FETCH_FUNC**.

Se nenhum modo de busca da lista acima for fornecido, o modo de busca padrão atual para o PDOStatement será usado.

```php
<?php
$stmt = $pdo->query("SELECT country, userid, name FROM users");
$row = $stmt->fetchAll(\PDO::FETCH_GROUP | \PDO::FETCH_ASSOC);
print_r($row);
```

O exemplo acima produzirá:

```
Array
(
    [Ukraine] => Array
        (
            [0] => Array
                (
                    [userid] => 104
                    [name] => Chris
                )

            [1] => Array
                (
                    [userid] => 108
                    [name] => Sean
                )

        )
    [England] => Array
        (
            [0] => Array
                (
                    [userid] => 105
                    [name] => Jamie
                )

        )

    [Germany] => Array
        (
            [0] => Array
                (
                    [userid] => 107
```

```
                    [name] => Robin
                )

            [1] => Array
                (
                    [userid] => 109
                    [name] => Toni
                )
        )
)
```

No exemplo acima, pode ser observado que a primeira coluna é omitida do array de cada linha, estando disponível apenas como chave. Ela pode ser incluída repetindo a coluna, como no exemplo a seguir:

```php
<?php
$stmt = $pdo->query("SELECT country, userid, name, country FROM users");
$row = $stmt->fetchAll(\PDO::FETCH_GROUP | \PDO::FETCH_ASSOC);
print_r($row);
```

O exemplo acima produzirá:

```
Array
(
    [Ukraine] => Array
        (
            [0] => Array
                (
                    [userid] => 104
                    [name] => Chris
                    [country] => Ukraine
                )

            [1] => Array
                (
                    [userid] => 108
                    [name] => Sean
                    [country] => Ukraine
                )

        )
    [England] => Array
        (
            [0] => Array
                (
                    [userid] => 105
                    [name] => Jamie
                    [country] => England
                )

        )

    [Germany] => Array
        (
```

```
            [0] => Array
                (
                    [userid] => 107
                    [name] => Robin
                    [country] => Germany
                )

            [1] => Array
                (
                    [userid] => 109
                    [name] => Toni
                    [country] => Germany
                )
        )
)
```

PDO::FETCH_UNIQUE (int)

PDO::FETCH_UNIQUE usa a primeira coluna para indexar registros, retornando 1 registro por valor de índice. Este modo de busca funciona apenas com PDOStatement::fetchAll().

Quando combinado com **PDO::FETCH_GROUP**, ambos os modos usarão a mesma coluna, tornando a combinação desses modos inútil.

Esta busca deve ser combinada com uma das seguintes constantes **PDO::FETCH_ASSOC**, **PDO::FETCH_BOTH**, **PDO::FETCH_NAMED**, **PDO::FETCH_NUM**, **PDO::FETCH_COLUMN** ou **PDO::FETCH_FUNC**.

Se nenhum modo de busca da lista acima for fornecido, o modo de busca padrão atual para o PDOStatement será usado.

Quando usado com uma coluna que se sabe ser única (como o ID do registro), este modo oferece a capacidade de retornar rapidamente resultados indexados por esse valor.

> **Nota** :
>
> Se a primeira coluna não for única, valores serão perdidos. Quais valores serão perdidos devem ser considerados indefinidos.

> **Cuidado**
>
> A filtragem de registros deve ser feita em SQL sempre que possível. O banco de dados usará índices para otimizar esse processo e retornar apenas os registros necessários. Selecionar mais registros do que o necessário do banco de dados pode aumentar significativamente o uso de memória e o tempo de consulta para conjuntos de resultados maiores.

```php
<?php
$stmt = $pdo->query("SELECT userid, name, country FROM users LIMIT 3");
$row = $stmt->fetchAll(\PDO::FETCH_UNIQUE | \PDO::FETCH_ASSOC);
print_r($row);
```

O exemplo acima produzirá:

```
Array
(
    [104] => Array
        (
            [name] => Chris
            [country] => Ukraine
        )

    [105] => Array
        (
            [name] => Jamie
            [country] => England
        )

    [107] => Array
        (
            [name] => Robin
            [country] => Germany
        )

)
```

No exemplo acima, pode ser observado que a primeira coluna é omitida do array de cada linha, estando disponível apenas como chave. Ela pode ser incluída repetindo a coluna, como no exemplo a seguir:

```php
<?php
$stmt = $pdo->query("SELECT userid, userid, name, country FROM users LIMIT 3");
$row = $stmt->fetchAll(\PDO::FETCH_UNIQUE | \PDO::FETCH_ASSOC);
print_r($row);
```

O exemplo acima produzirá:

```
Array
(
    [104] => Array
        (
            [userid] => 104
            [name] => Chris
            [country] => Ukraine
        )

    [105] => Array
        (
            [userid] => 105
            [name] => Jamie
            [country] => England
        )

    [107] => Array
        (
            [userid] => 107
```

```
            [name] => Robin
            [country] => Germany
        )

)
```

Conexões e gerenciamento de conexões

As conexões são estabelecidas criando instâncias da classe base PDO. Não importa qual driver você deseja usar; você sempre usa o nome da classe PDO. O construtor aceita parâmetros para especificar a fonte do banco de dados (conhecida como DSN) e opcionalmente para o nome de usuário e senha (se houver).

Exemplo #1 Conectando-se ao MySQL

```php
<?php
$dbh = new PDO('mysql:host=localhost;dbname=test', $user, $pass);
?>
```

Se houver algum erro de conexão, um objeto `PDOException` será lançado. Você pode capturar a exceção se quiser lidar com a condição de erro, ou pode optar por deixá-la para um manipulador de exceção global do aplicativo que você configurou via set_exception_handler().

Exemplo #2 Tratando erros de conexão

```php
<?php
try {
    $dbh = new PDO('mysql:host=localhost;dbname=test', $user, $pass);
} catch (PDOException $e) {
    // tentar reconectar após algum intervalo, por exemplo
}
```

> **Aviso**
>
> Assim como qualquer outra exceção, PDOException pode ser capturada explicitamente, via uma instrução `catch`, ou implicitamente através de set_exception_handler(). Caso contrário, o comportamento padrão de converter uma exceção não capturada em um **`E_FATAL_ERROR`** ocorrerá. O erro fatal conterá uma rastreabilidade que pode vazar detalhes da conexão. Portanto, a opção *php.ini* `display_errors` deve ser definida como `0` em um servidor de produção.

Após a conexão bem-sucedida com o banco de dados, uma instância da classe PDO é retornada para seu script. A conexão permanece ativa pelo tempo de vida desse objeto PDO. Para fechar a conexão, você precisa destruir o objeto garantindo que todas as referências restantes a ele sejam deletadas - você faz isso atribuindo `null` à variável que mantém o objeto. Se você não fizer isso explicitamente, o PHP fechará automaticamente a conexão quando seu script terminar.

> **Nota** :

> Se ainda houver outras referências a esta instância PDO (como de uma instância PDOStatement, ou de outras variáveis referenciando a mesma instância PDO), estas também devem ser removidas (por exemplo, atribuindo `null` à variável que referencia a instância PDOStatement).

Exemplo #3 Fechando uma conexão

```php
<?php
$dbh = new PDO('mysql:host=localhost;dbname=test', $user, $pass);
// use a conexão aqui
$sth = $dbh->query('SELECT * FROM foo');

// e agora terminamos; feche-a
$sth = null;
$dbh = null;
?>
```

Muitas aplicações web se beneficiarão com a realização de conexões persistentes com servidores de banco de dados. As conexões persistentes não são fechadas no final do script, mas são armazenadas em cache e reutilizadas quando outro script solicita uma conexão usando as mesmas credenciais. O cache de conexão persistente permite evitar a sobrecarga de estabelecer uma nova conexão toda vez que um script precisa se comunicar com um banco de dados, resultando em uma aplicação web mais rápida.

Exemplo #4 Conexões persistentes

```php
<?php
$dbh = new PDO('mysql:host=localhost;dbname=test', $user, $pass, array(
    PDO::ATTR_PERSISTENT => true
));
?>
```

O valor da opção `PDO::ATTR_PERSISTENT` é convertido para bool (ativar/desativar conexões persistentes), a menos que seja uma string não numérica, caso em que permite o uso de vários pools de conexões persistentes. Isso é útil se diferentes conexões usam configurações incompatíveis, por exemplo, valores diferentes de `PDO::MYSQL_ATTR_USE_BUFFERED_QUERY`.

> **Nota** :
>
> Se desejar usar conexões persistentes, você deve definir `PDO::ATTR_PERSISTENT` no array de opções do driver passadas para o construtor PDO. Se definir este atributo com PDO::setAttribute() após a instanciação do objeto, o driver não usará conexões persistentes.

> **Aviso**
>
> O PDO não realiza nenhuma limpeza de conexões persistentes. Tabelas temporárias, bloqueios, transações e outras alterações de estado podem permanecer do uso anterior da conexão, causando problemas inesperados. Consulte a seção Conexões de Banco de Dados Persistentes para obter mais informações.

> **Nota** :
>
> Se estiver usando o driver PDO ODBC e suas bibliotecas ODBC suportarem Pool de Conexões ODBC (unixODBC e Windows são duas que suportam; pode haver mais), então é recomendável que você não use

> conexões PDO persistentes e, em vez disso, deixe o cache de conexão para a camada de Pool de Conexões ODBC. O Pool de Conexões ODBC é compartilhado com outros módulos no processo; se o PDO for instruído a armazenar em cache a conexão, então essa conexão nunca seria retornada ao pool de conexões ODBC, resultando em conexões adicionais sendo criadas para atender a esses outros módulos.

Transações e confirmação automática (auto-commit)

Agora que você está conectado via PDO, é necessário entender como o PDO gerencia transações antes de começar a emitir consultas. Se você nunca encontrou transações antes, elas oferecem 4 recursos principais: Atomicidade, Consistência, Isolamento e Durabilidade (ACID). Em termos simples, qualquer trabalho realizado em uma transação, mesmo que seja feito em etapas, é garantido de ser aplicado ao banco de dados com segurança e sem interferência de outras conexões, quando é confirmado. O trabalho transacional também pode ser desfeito automaticamente a seu pedido (desde que você ainda não tenha confirmado), o que facilita o tratamento de erros em seus scripts.

As transações são tipicamente implementadas "salvando" seu lote de alterações para serem aplicadas de uma vez; isso tem o efeito colateral agradável de melhorar drasticamente a eficiência dessas atualizações. Em outras palavras, transações podem tornar seus scripts mais rápidos e potencialmente mais robustos (ainda é necessário usá-los corretamente para obter esse benefício).

Infelizmente, nem todos os bancos de dados suportam transações, então o PDO precisa rodar no que é conhecido como modo "auto-commit" quando você abre a conexão pela primeira vez. O modo auto-commit significa que cada consulta que você executa tem sua própria transação implícita, se o banco de dados suportar, ou nenhuma transação se o banco de dados não suportar transações. Se você precisar de uma transação, você deve usar o método PDO::beginTransaction() para iniciá-la. Se o driver subjacente não suportar transações, uma PDOException será lançada (independentemente das suas configurações de tratamento de erro: essa é sempre uma condição de erro grave). Uma vez que você está em uma transação, você pode usar PDO::commit() ou PDO::rollBack() para finalizá-la, dependendo do sucesso do código que você executou durante a transação.

> **Aviso**
>
> O PDO verifica apenas as capacidades de transação no nível do driver. Se certas condições de tempo de execução significarem que transações estão indisponíveis, PDO::beginTransaction() ainda retornará **`true`** sem erro se o servidor de banco de dados aceitar a solicitação para iniciar uma transação.
>
> Um exemplo disso seria tentar usar transações em tabelas MyISAM em um banco de dados MySQL.

> **Aviso**
>
> *Confirmações Implícitas com Instruções DDL:* Alguns bancos de dados emitem automaticamente um `COMMIT` implícito quando uma instrução de Linguagem de Definição de Banco de Dados (DDL), como `DROP TABLE` ou `CREATE TABLE`, é executada dentro de uma transação. Isto significa que quaisquer modificações feitas na mesma transação serão *confirmadas automaticamente* e não poderão ser revertidas.
>
> `MySQL` e `Oracle` são exemplos de bancos de dados que exibem este comportamento.

Exemplo #1 Exemplo de Confirmação Implícita

```php
<?php
$pdo->beginTransaction();
$pdo->exec("INSERT INTO users (name) VALUES ('Rasmus')");
```

```
$pdo->exec("CREATE TABLE test (id INT PRIMARY KEY)"); // aqui ocorre um COMMIT implícito
$pdo->rollBack(); // Isto NÃO irá desfazer as instruções INSERT/CREATE no MySQL ou no Oracle
?>
```

Melhores Práticas: Evite executar instruções DDL dentro de transações quando usar banco de dados que têm este comportamento. Se necessário, separe as operações DDL da lógica das transações.

Quando o script termina ou quando uma conexão está prestes a ser fechada, se você tiver uma transação pendente, o PDO automaticamente a reverterá. Esta é uma medida de segurança para ajudar a evitar inconsistências nos casos em que o script termina inesperadamente--se você não confirmou explicitamente a transação, então é assumido que algo deu errado, então o rollback é executado para a segurança de seus dados.

> **Aviso**
>
> O rollback automático só acontece se você iniciar a transação via PDO::beginTransaction(). Se você emitir manualmente uma consulta que inicia uma transação, o PDO não tem como saber sobre isso e assim não pode revertê-la se algo ruim acontecer.

Exemplo #2 Executando um lote em uma transação

No exemplo a seguir, vamos assumir que estamos criando um conjunto de entradas para um novo funcionário, que recebeu um número de ID 23. Além de inserir os dados básicos para essa pessoa, também precisamos registrar seu salário. É bastante simples fazer duas atualizações separadas, mas ao envolvê-las nas chamadas PDO::beginTransaction() e PDO::commit(), estamos garantindo que ninguém mais poderá ver essas alterações até que elas estejam completas. Se algo der errado, o bloco catch reverte todas as alterações feitas desde que a transação foi iniciada e, em seguida, imprime uma mensagem de erro.

```
<?php
try {
  $dbh = new PDO('odbc:SAMPLE', 'db2inst1', 'ibmdb2',
      array(PDO::ATTR_PERSISTENT => true));
  echo "Connected\n";
} catch (Exception $e) {
  die("Não foi possível conectar: " . $e->getMessage());
}

try {
  $dbh->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

  $dbh->beginTransaction();
  $dbh->exec("insert into staff (id, first, last) values (23, 'Joe', 'Bloggs')");
  $dbh->exec("insert into salarychange (id, amount, changedate)
      values (23, 50000, NOW())");
  $dbh->commit();

} catch (Exception $e) {
  $dbh->rollBack();
  echo "Falha: " . $e->getMessage();
}
?>
```

Você não está limitado a fazer atualizações em uma transação; você também pode emitir consultas complexas para extrair dados e possivelmente usar essas informações para construir mais atualizações e consultas; enquanto a transação estiver ativa, você tem a garantia de que ninguém mais pode fazer alterações enquanto

você está no meio do seu trabalho. Para ler mais sobre transações, consulte a documentação fornecida pelo seu servidor de banco de dados.

Declarações preparadas e procedimentos armazenados

Muitos dos bancos de dados mais maduros suportam o conceito de declarações preparadas. O que são elas? Elas podem ser consideradas como uma espécie de modelo compilado para o SQL que uma aplicação deseja executar, que pode ser personalizado usando parâmetros variáveis. As declarações preparadas oferecem dois benefícios principais:

- A consulta só precisa ser analisada (ou preparada) uma vez, mas pode ser executada várias vezes com os mesmos ou diferentes parâmetros. Quando a consulta é preparada, o banco de dados irá analisar, compilar e otimizar seu plano para executar a consulta. Para consultas complexas, esse processo pode levar tempo suficiente para diminuir perceptivelmente o desempenho de uma aplicação se houver necessidade de repetir a mesma consulta muitas vezes com diferentes parâmetros. Ao usar uma declaração preparada, a aplicação evita repetir o ciclo de análise/compilação/otimização. Isso significa que declarações preparadas usam menos recursos e, portanto, são mais rápidas.
- Os parâmetros das declarações preparadas não precisam ser citados; o driver lida automaticamente com isso. Se uma aplicação usar exclusivamente declarações preparadas, o desenvolvedor pode ter certeza de que nenhuma injeção de SQL ocorrerá (no entanto, se outras partes da consulta estiverem sendo construídas com entrada não escapada, a injeção de SQL ainda é possível).

As declarações preparadas são tão úteis que são a única funcionalidade que o PDO emula para drivers que não as suportam. Isso garante que uma aplicação poderá usar o mesmo paradigma de acesso a dados, independentemente das capacidades do banco de dados.

Exemplo #1 Inserções repetidas usando declarações preparadas

Este exemplo executa uma consulta INSERT substituindo um `name` e um `value` pelos marcadores de posição nomeados.

```php
<?php
$stmt = $dbh->prepare("INSERT INTO REGISTRY (name, value) VALUES (:name, :value)");
$stmt->bindParam(':name', $name);
$stmt->bindParam(':value', $value);

// insere uma linha
$name = 'one';
$value = 1;
$stmt->execute();

// insere outra linha com valores diferentes
$name = 'two';
$value = 2;
$stmt->execute();
?>
```

Exemplo #2 Inserções repetidas usando declarações preparadas

Este exemplo executa uma consulta INSERT substituindo um `name` e um `value` pelos marcadores de posição posicionais `?`.

```php
<?php
$stmt = $dbh->prepare("INSERT INTO REGISTRY (name, value) VALUES (?, ?)");
$stmt->bindParam(1, $name);
$stmt->bindParam(2, $value);

// insere uma linha
$name = 'one';
$value = 1;
$stmt->execute();

// insere outra linha com valores diferentes
$name = 'two';
$value = 2;
$stmt->execute();
?>
```

Exemplo #3 Buscando dados usando declarações preparadas

Este exemplo busca dados com base em um valor de chave fornecido por um formulário. A entrada do usuário é automaticamente citada, portanto, não há risco de um ataque de injeção de SQL.

```php
<?php
$stmt = $dbh->prepare("SELECT * FROM REGISTRY where name = ?");
$stmt->execute([$_GET['name']]);
foreach ($stmt as $row) {
  print_r($row);
}
?>
```

Exemplo #4 Chamando um procedimento armazenado com um parâmetro de saída

Se o driver do banco de dados suportar, uma aplicação também pode vincular parâmetros de saída além de entrada. Os parâmetros de saída são normalmente usados para recuperar valores de procedimentos armazenados. Os parâmetros de saída são um pouco mais complexos de usar do que os parâmetros de entrada, pois um desenvolvedor deve saber o quão grande um determinado parâmetro pode ser quando o vincula. Se o valor for maior do que o tamanho sugerido, um erro será gerado.

```php
<?php
$stmt = $dbh->prepare("CALL sp_returns_string(?)");
$stmt->bindParam(1, $return_value, PDO::PARAM_STR, 4000);

// chamar o procedimento armazenado
$stmt->execute();

print "procedure returned $return_value\n";
?>
```

Exemplo #5 Chamando um procedimento armazenado com um parâmetro de entrada/saída

Os desenvolvedores também podem especificar parâmetros que contêm valores tanto de entrada quanto de saída; a sintaxe é semelhante aos parâmetros de saída. No próximo exemplo, a string 'hello' é passada para o procedimento armazenado e, quando ele retorna, hello é substituído pelo valor de retorno do procedimento.

```php
<?php
$stmt = $dbh->prepare("CALL sp_takes_string_returns_string(?)");
$value = 'hello';
```

```php
$stmt->bindParam(1, $value, PDO::PARAM_STR|PDO::PARAM_INPUT_OUTPUT, 4000);

// chamar o procedimento armazenado
$stmt->execute();

print "procedure returned $value\n";
?>
```

Exemplo #6 Uso inválido de marcador de posição

```php
<?php
$stmt = $dbh->prepare("SELECT * FROM REGISTRY where name LIKE '%?%'");
$stmt->execute([$_GET['name']]);

// o marcador de posição deve ser usado no lugar do valor inteiro
$stmt = $dbh->prepare("SELECT * FROM REGISTRY where name LIKE ?");
$stmt->execute(["%$_GET[name]%"]);
?>
```

Erros e manipulação de erros

O PDO oferece a escolha entre 3 estratégias diferentes de manipulação de erros, para se adequar ao estilo de cada desenvolvedor de aplicações.

- **`PDO::ERRMODE_SILENT`**

  Antes do PHP 8.0.0, este era o modo padrão. O PDO simplesmente definia o código de erro inspecionado pelos métodos PDO::errorCode() e PDO::errorInfo() tanto nos objetos de instrução quanto no banco de dados; se o erro resultasse de uma chamada no objeto de instrução, invocaria-se o método PDOStatement::errorCode() ou PDOStatement::errorInfo() nesse objeto. Se o erro resultasse de uma chamada no objeto de banco de dados, invocaria-se esses métodos no objeto de banco de dados em vez disso.

- **`PDO::ERRMODE_WARNING`**

  Além de definir o código de erro, o PDO emitirá uma mensagem de E_WARNING tradicional. Esta configuração é útil durante a depuração/teste, se você apenas quiser ver quais problemas ocorreram sem interromper o fluxo do aplicativo.

- **`PDO::ERRMODE_EXCEPTION`**

  A partir do PHP 8.0.0, este é o modo padrão. Além de definir o código de erro, o PDO lançará um PDOException e definirá suas propriedades para refletir o código de erro e o informações de erro. Esta configuração também é útil durante a depuração, pois irá efetivamente "explodir" o script no ponto do erro, muito rapidamente apontando um dedo para áreas problemáticas potenciais em seu código (lembre-se: as transações são automaticamente revertidas se a exceção faz com que o script termine).

  O modo de exceção também é útil porque você pode estruturar sua manipulação de erros de forma mais clara do que com avisos no estilo tradicional do PHP e com menos código/aninhamento do que em modo silencioso e explicitamente verificando o valor de retorno de cada chamada de banco de dados.

  Veja Exceções para mais informações sobre Exceções em PHP.

O PDO padroniza o uso de strings de código de erro SQLSTATE SQL-92; individualmente Os drivers do PDO são responsáveis por mapear seus códigos nativos para os códigos SQLSTATE apropriados. O método

PDO::errorCode() retorna um único código SQLSTATE. Se você precisar de informações mais específicas sobre um erro, o PDO também oferece um método PDO::errorInfo() que retorna um array contendo o código SQLSTATE, o código de erro específico do driver e o driver string de erro específico.

Exemplo #1 Criar uma instância do PDO e definir o modo de erro

```php
<?php
$dsn = 'mysql:dbname=testdb;host=127.0.0.1';
$user = 'dbuser';
$password = 'dbpass';

$dbh = new PDO($dsn, $user, $password);
$dbh->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

// Isso fará com que o PDO lance uma PDOException (quando a tabela não existe)
$dbh->query("SELECT wrongcolumn FROM wrongtable");
```

O exemplo acima produzirá:

```
Fatal error: Uncaught PDOException: SQLSTATE[42S02]: Base table or view not found: 1146 Table
'testdb.wrongtable' doesn't exist in /tmp/pdo_test.php:10
Stack trace:
#0 /tmp/pdo_test.php(10): PDO->query('SELECT wrongcol...')
#1 {main}
  thrown in /tmp/pdo_test.php on line 10
```

> **Nota** :
>
> PDO::__construct() sempre lançará uma PDOException se a conexão falhar independentemente de qual `PDO::ATTR_ERRMODE` estiver atualmente definido.

Exemplo #2 Criar uma instância do PDO e definir o modo de erro a partir do construtor

```php
<?php
$dsn = 'mysql:dbname=test;host=127.0.0.1';
$user = 'googleguy';
$password = 'googleguy';

$dbh = new PDO($dsn, $user, $password, array(PDO::ATTR_ERRMODE => PDO::ERRMODE_WARNING));

// Isso fará com que o PDO lance um erro de nível E_WARNING em vez de uma exceção (quando a tabela não
existe)
$dbh->query("SELECT wrongcolumn FROM wrongtable");
```

O exemplo acima produzirá:

```
Warning: PDO::query(): SQLSTATE[42S02]: Base table or view not found: 1146 Table 'test.wrongtable'
doesn't exist in
```

```
/tmp/pdo_test.php on line 9
```

Objetos Grandes (LOBs)

Em algum momento da aplicação, pode ser necessário armazenar dados "grandes" em seu banco de dados. Grande geralmente significa "cerca de 4kb ou mais", embora alguns bancos de dados possam lidar facilmente com até 32kb antes que os dados se tornem "grandes". Objetos grandes podem ser de natureza textual ou binária. O PDO permite que se trabalhe com esse tipo de dado grande usando o código de tipo **`PDO::PARAM_LOB`** nas chamadas a PDOStatement::bindParam() ou PDOStatement::bindColumn(). **`PDO::PARAM_LOB`** diz ao PDO para mapear os dados como um fluxo, para que seja possível manipulá-los usando a API de Fluxos do PHP.

Exemplo #1 Exibindo uma imagem de um banco de dados

Este exemplo vincula o LOB à variável chamada $lob e depois o envia para o navegador usando fpassthru(). Como o LOB é representado como um fluxo, funções como fgets(), fread() e stream_get_contents() podem ser usadas nele.

```php
<?php
$db = new PDO('odbc:SAMPLE', 'db2inst1', 'ibmdb2');
$stmt = $db->prepare("select contenttype, imagedata from images where id=?");
$stmt->execute(array($_GET['id']));
$stmt->bindColumn(1, $type, PDO::PARAM_STR, 256);
$stmt->bindColumn(2, $lob, PDO::PARAM_LOB);
$stmt->fetch(PDO::FETCH_BOUND);

header("Content-Type: $type");
fpassthru($lob);
?>
```

Exemplo #2 Inserindo uma imagem em um banco de dados

Este exemplo abre um arquivo e passa o identificador do arquivo para o PDO para inserir como um LOB. O PDO fará o melhor possível para obter o conteúdo do arquivo até o banco de dados da maneira mais eficiente possível.

```php
<?php
$db = new PDO('odbc:SAMPLE', 'db2inst1', 'ibmdb2');
$stmt = $db->prepare("insert into images (id, contenttype, imagedata) values (?, ?, ?)");
$id = get_new_id(); // uma função para alocar um novo ID

// suponha que estamos executando como parte de um formulário de upload de arquivo
// Mais informações podem ser encontradas na documentação do PHP

$fp = fopen($_FILES['file']['tmp_name'], 'rb');

$stmt->bindParam(1, $id);
$stmt->bindParam(2, $_FILES['file']['type']);
$stmt->bindParam(3, $fp, PDO::PARAM_LOB);

$db->beginTransaction();
$stmt->execute();
$db->commit();
?>
```

Exemplo #3 Inserindo uma imagem em um banco de dados: Oracle

O Oracle requer uma sintaxe ligeiramente diferente para inserir um lob de um arquivo. Também é essencial que você execute a inserção sob um transação, caso contrário, seu LOB recém-inserido será comprometido com um comprimento zero como parte do commit implícito que ocorre quando a consulta é executada:

```php
<?php
$db = new PDO('oci:', 'scott', 'tiger');
$stmt = $db->prepare("insert into images (id, contenttype, imagedata) " .
"VALUES (?, ?, EMPTY_BLOB()) RETURNING imagedata INTO ?");
$id = get_new_id(); // uma função para alocar um novo ID

// suponha que estamos executando como parte de um formulário de upload de arquivo
// Mais informações podem ser encontradas na documentação do PHP

$fp = fopen($_FILES['file']['tmp_name'], 'rb');

$stmt->bindParam(1, $id);
$stmt->bindParam(2, $_FILES['file']['type']);
$stmt->bindParam(3, $fp, PDO::PARAM_LOB);

$db->beginTransaction();
$stmt->execute();
$db->commit();
?>
```

PDO

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

Introdução

Representa uma conexão entre o PHP e um servidor de banco de dados.

Resumo da classe

class **PDO** {
/* Constantes */
public const int *PARAM_NULL*;
public const int *PARAM_BOOL* = 5;
public const int *PARAM_INT* = 1;
public const int *PARAM_STR* = 2;
public const int *PARAM_LOB* = 3;
public const int *PARAM_STMT* = 4;
public const int *PARAM_INPUT_OUTPUT*;
public const int *PARAM_STR_NATL*;
public const int *PARAM_STR_CHAR*;
public const int *PARAM_EVT_ALLOC*;
public const int *PARAM_EVT_FREE*;
public const int *PARAM_EVT_EXEC_PRE*;
public const int *PARAM_EVT_EXEC_POST*;
public const int *PARAM_EVT_FETCH_PRE*;
public const int *PARAM_EVT_FETCH_POST*;
public const int *PARAM_EVT_NORMALIZE*;

public const int *FETCH_DEFAULT*;
public const int *FETCH_LAZY*;
public const int *FETCH_ASSOC*;
public const int *FETCH_NUM*;
public const int *FETCH_BOTH*;
public const int *FETCH_OBJ*;
public const int *FETCH_BOUND*;
public const int *FETCH_COLUMN*;
public const int *FETCH_CLASS*;
public const int *FETCH_INTO*;
public const int *FETCH_FUNC*;
public const int *FETCH_GROUP*;
public const int *FETCH_UNIQUE*;
public const int *FETCH_KEY_PAIR*;
public const int *FETCH_CLASSTYPE*;
public const int *FETCH_SERIALIZE*;
public const int *FETCH_PROPS_LATE*;
public const int *FETCH_NAMED*;
public const int *ATTR_AUTOCOMMIT*;
public const int *ATTR_PREFETCH*;
public const int *ATTR_TIMEOUT*;
public const int *ATTR_ERRMODE*;
public const int *ATTR_SERVER_VERSION*;
public const int *ATTR_CLIENT_VERSION*;
public const int *ATTR_SERVER_INFO*;
public const int *ATTR_CONNECTION_STATUS*;
public const int *ATTR_CASE*;
public const int *ATTR_CURSOR_NAME*;
public const int *ATTR_CURSOR*;
public const int *ATTR_ORACLE_NULLS*;
public const int *ATTR_PERSISTENT*;
public const int *ATTR_STATEMENT_CLASS*;
public const int *ATTR_FETCH_TABLE_NAMES*;
public const int *ATTR_FETCH_CATALOG_NAMES*;
public const int *ATTR_DRIVER_NAME*;
public const int *ATTR_STRINGIFY_FETCHES*;
public const int *ATTR_MAX_COLUMN_LEN*;
public const int *ATTR_EMULATE_PREPARES*;
public const int *ATTR_DEFAULT_FETCH_MODE*;
public const int *ATTR_DEFAULT_STR_PARAM*;
public const int *ERRMODE_SILENT*;
public const int *ERRMODE_WARNING*;
public const int *ERRMODE_EXCEPTION*;
public const int *CASE_NATURAL*;
public const int *CASE_LOWER*;
public const int *CASE_UPPER*;
public const int *NULL_NATURAL*;
public const int *NULL_EMPTY_STRING*;
public const int *NULL_TO_STRING*;
public const string *ERR_NONE*;
public const int *FETCH_ORI_NEXT*;

```
public const int FETCH_ORI_PRIOR;
public const int FETCH_ORI_FIRST;
public const int FETCH_ORI_LAST;
public const int FETCH_ORI_ABS;
public const int FETCH_ORI_REL;
public const int CURSOR_FWDONLY;
public const int CURSOR_SCROLL;
/* Métodos */
public function __construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
public function beginTransaction(): bool
public function commit(): bool
public static function connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
public function errorCode(): ?string
public function errorInfo(): array
public function exec(string $statement): int|false
public function getAttribute(int $attribute): mixed
public static function getAvailableDrivers(): array
public function inTransaction(): bool
public function lastInsertId(?string $name = null): string|false
public function prepare(string $query, array $options = []): PDOStatement|false
public function query(string $query, ?int $fetchMode = null): PDOStatement|false
public function query(string $query, ?int $fetchMode = PDO::FETCH_COLUMN, int $colno): PDOStatement|false
public function query(
    string $query,
    ?int $fetchMode = PDO::FETCH_CLASS,
    string $classname,
    array $constructorArgs
): PDOStatement|false
public function query(string $query, ?int $fetchMode = PDO::FETCH_INTO, object $object): PDOStatement|false
public function quote(string $string, int $type = PDO::PARAM_STR): string|false
public function rollBack(): bool
public function setAttribute(int $attribute, mixed $value): bool
}
```

Registro de Alterações

Versão	Descrição
8.4.0	As constantes de classe agora são tipadas.

Índice

PDO::beginTransaction

PDO::beginTransaction

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::beginTransaction — Inicia uma transação

Descrição

public function **PDO::beginTransaction**(): bool

Desativa o modo de envio automático. Enquanto o modo de envio automático estiver desativado, modificações feitas no banco de dados por meio da instância do objeto PDO não serão enviadas até que você finalize a transação chamando PDO::commit(). Chamar PDO::rollBack() reverterá todas as alterações no banco de dados e retornará a conexão para o modo de envio automático.

Alguns bancos de dados, incluindo o MySQL, emitem automaticamente um COMMIT implícito quando uma instrução de linguagem de definição de banco de dados (DDL) como DROP TABLE ou CREATE TABLE é emitida em uma transação. O COMMIT implícito impedirá que você reverta quaisquer outras alterações dentro do limite da transação.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Lança uma PDOException se já houver uma transação iniciada ou se o driver não suportar transações.

> **Nota** :
>
> Uma exceção será lançada mesmo quando o atributo `PDO::ATTR_ERRMODE` não for `PDO::ERRMODE_EXCEPTION`.

Examples

Exemplo #1 Reverter uma transação

O exemplo a seguir inicia uma transação e emite duas instruções que modificam o banco de dados antes de reverter as alterações. No MySQL, no entanto, a instrução DROP TABLE envia automaticamente a transação de modo que nenhuma das alterações na transação é revertida.

```php
<?php
/* Inicia uma transação, desligando o envio automático */
$dbh->beginTransaction();

/* Altera esquema e dados do banco */
$sth = $dbh->exec("DROP TABLE fruit");
$sth = $dbh->exec("UPDATE dessert
    SET name = 'hamburger'");

/* Reconhece o engano e desfaz o envio das alterações */
```

```php
$dbh->rollBack();

/* A conexão do banco agora está de volta o modo de envio automático */
?>
```

Veja também

- PDO::commit() - Envia uma transação
- PDO::rollBack() - Reverte uma transação
- Transações e envio automático

PDO::commit

PDO::commit

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::commit — Envia uma transação

Descrição

public function **PDO::commit**(): bool

Envia uma transação, retornando a conexão com o banco de dados para o modo de envio automático até que a próxima chamada a PDO::beginTransaction() inicie uma nova transação.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Lança uma PDOException se não houver transação ativa.

> **Nota** :
>
> Uma exceção será lançada mesmo quando o atributo `PDO::ATTR_ERRMODE` não for `PDO::ERRMODE_EXCEPTION`.

Examples

Exemplo #1 Enviando uma transação básica

```php
<?php
/* Inicia uma transação, desligando o envio automático */
$dbh->beginTransaction();

/* Insere múltiplos registros na base do tudo-ou-nada */
$sql = 'INSERT INTO fruit
    (name, colour, calories)
    VALUES (?, ?, ?)';

$sth = $dbh->prepare($sql);
```

```
foreach ($fruits as $fruit) {
    $sth->execute(array(
        $fruit->name,
        $fruit->colour,
        $fruit->calories,
    ));
}

/* Envia as alterações */
$dbh->commit();

/* A conexão do banco agora está de volta o modo de envio automático */
?>
```

Exemplo #2 Enviando uma transação DDL

```
<?php
/* Inicia uma transação, desligando o envio automático */
$dbh->beginTransaction();

/* Altera esquema do banco */
$sth = $dbh->exec("DROP TABLE fruit");

/* Envia as alterações */
$dbh->commit();

/* A conexão do banco agora está de volta o modo de envio automático */
?>
```

> **Nota** :
>
> Nem todos os bancos de dados permitirão que as transações operem em instruções DDL: alguns gerarão erros, enquanto outros (incluindo o MySQL) enviarão automaticamente a transação após a primeira instrução DDL ter sido encontrada.

Veja também

- PDO::beginTransaction() - Inicia uma transação
- PDO::rollBack() - Reverte uma transação
- Transações e envio automático

PDO::connect

PDO::connect

(PHP 8 >= 8.4.0)

PDO::connect — Conecta a um banco de dados e retorna uma subclasse PDO para drivers que o suportam

Descrição

public static function **PDO::connect**(
  string $dsn,
  ?string $username = null,

```
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
```

Cria uma instância de uma subclasse PDO para o banco de dados sendo conectado, se existir, caso contrário retorna uma instância genérica de PDO.

Parâmetros

`dsn`

O nome da fonte de dados, ou DSN, que contém a informação requerida para conexão ao banco de dados.

Em geral, um DSN consiste no nome do driver PDO, seguido por dois-pontos, seguido pela sintaxe de conexão específica do driver PDO. Maiores informações estão disponíveis na documentação específica do driver PDO.

O parâmetro `dsn` suporta três métodos diferentes para especificar os argumentos requeridos para criação de uma conexão de banco de dados:

Invocação do driver

`dsn` contém o DSN completo.

Invocação de URI

`dsn` consiste em **`uri:`** seguido por uma URI que define a localização de um arquivo contendo a string DSN. A URI pode especificar um arquivo local ou uma URL remota.

```
uri:file:///caminho/para/arquivodsn
```

Criação de apelido

`dsn` consiste em um nome (`name`) que é mapeado para `pdo.dsn.name` no *php.ini* definindo a string DSN.

> **Nota** :
>
> O apelido deve ser definido no *php.ini*, e não no *.htaccess* ou no *httpd.conf*

`username`

O nome de usuário para a string DSN. Este parâmetro é opcional para alguns drivers PDO.

`password`

A senha para a srting DSN. Este parâmetro é opcional para alguns drivers PDO.

`options`

Um array de chave=>valor de opções de conexão específicas do driver.

Valor Retornado

Retorna uma instância de uma subclasse PDO para o driver PDO correspondente se existir, ou uma instância genérica de PDO.

Erros/Exceções

Uma PDOException é lançada se a tentativa de conexão ao banco de dados requisitado falhar, independente de qual `PDO::ATTR_ERRMODE` estiver definido.

Veja também

- Pdo\Dblib
- Pdo\Firebird
- Pdo\Mysql
- Pdo\Odbc
- Pdo\Pgsql
- Pdo\Sqlite
- PDO::__construct() - Cria uma instância PDO representando uma conexão a um banco de dados

PDO::__construct

PDO::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::__construct — Cria uma instância PDO representando uma conexão a um banco de dados

Descrição

```
public function PDO::__construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
```

Cria uma instância PDO para representar uma conexão ao banco de dados requisitado.

Parâmetros

`dsn`

O nome da fonte de dados, ou DSN, que contém a informação requerida para conexão ao banco de dados.

Em geral, um DSN consiste no nome do driver PDO, seguido por dois-pontos, seguido pela sintaxe de conexão específica do driver PDO. Maiores informações estão disponíveis na documentação específica do driver PDO.

O parâmetro `dsn` suporta três métodos diferentes para especificar os argumentos requeridos para criação de uma conexão de banco de dados:

Invocação do driver

`dsn` contém o DSN completo.

Invocação de URI

`dsn` consiste em **`uri:`** seguido por uma URI que define a localização de um arquivo contendo a string DSN. A URI pode especificar um arquivo local ou uma URL remota.

`uri:file:///caminho/para/arquivodsn`

Criação de apelido

`dsn` consiste em um nome (`name`) que é mapeado para `pdo.dsn.name` no *php.ini* definindo a string DSN.

> **Nota** :
>
> O apelido deve ser definido no *php.ini*, e não no *.htaccess* ou no *httpd.conf*

`username`

O nome de usuário para a string DSN. Este parâmetro é opcional para alguns drivers PDO.

`password`

A senha para a srting DSN. Este parâmetro é opcional para alguns drivers PDO.

`options`

Um array de chave=>valor de opções de conexão específicas do driver.

Erros/Exceções

Uma PDOException é lançada se a tentativa de conexão ao banco de dados requisitado falhar, independente de qual **`PDO::ATTR_ERRMODE`** estiver definido.

Examples

Exemplo #1 Cria uma instância PDO através de invocação de driver

```php
<?php

$dsn = 'mysql:dbname=testdb;host=127.0.0.1';
$user = 'dbuser';
$password = 'dbpass';

$dbh = new PDO($dsn, $user, $password);

?>
```

Exemplo #2 Cria uma instância PDO através de invocação de URI

O exemplo a seguir assume que o arquivo */usr/local/dbconnect* existe com permissões de leitura pelo PHP. O arquivo contém o DSN PDO para conexão a um banco de dados DB2 através do driver PDO_ODBC:

```
odbc:DSN=SAMPLE;UID=john;PWD=mypass
```

O script PHP pode então criar uma conexão ao banco de dados simplesmente passando o parâmetro `uri:` e apontando para a URI do arquivo:

```php
<?php

$dsn = 'uri:file:///usr/local/dbconnect';
$user = '';
$password = '';

$dbh = new PDO($dsn, $user, $password);

?>
```

Exemplo #3 Cria uma instância PDO usando um apelido

O exemplo a seguir assume que o *php.ini* contém a seguinte entrada para habilitar uma conexão ao banco de dados MySQL usando somente o apelido `mydb`:

```
[PDO]
pdo.dsn.mydb="mysql:dbname=testdb;host=localhost"
```

```php
<?php

$dsn = 'mydb';
$user = '';
$password = '';

$dbh = new PDO($dsn, $user, $password);

?>
```

PDO::errorCode

PDO::errorCode

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::errorCode — Busca o SQLSTATE associado com a última operação no identificador do banco de dados

Descrição

public function **PDO::errorCode**(): ?string

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um SQLSTATE, um identificador alfanumérico de cinco caracteres definido no padrão ANSI SQL-92. Resumidamente, um SQLSTATE consiste em um valor de classe de dois caracteres seguido por um valor de subclasse de três caracteres. Um valor de classe de '01' indica um alerta e é acompanhado por um código de retorno de SQL_SUCCESS_WITH_INFO. Valores de classe diferentes de '01', exceto pela classe 'IM', indicam um erro. A classe 'IM' é específica para alertas e erros que derivam da implementação do próprio PDO (ou talvez do

ODBC, se estiver em uso). O valor de subclasse '000' em qualquer classe indica que não há subclasse para aquele SQLSTATE.

PDO::errorCode() somente busca códigos de erro para operações realizadas diretamente no identificador do banco de dados. Se for criado um objeto PDOStatement através de PDO::prepare() ou PDO::query() e for invocado um erro no identificador da instrução, **PDO::errorCode()** não refletirá esse erro. Deve-se chamar PDOStatement::errorCode() para retornar o código de erro para uma operação realizada em um identificador de instrução em particular.

Retorna `null` se nenhuma operação foi executada no identificador do banco de dados.

Examples

Exemplo #1 Buscando um código SQLSTATE

```php
<?php
/* Provoca um erro -- a tabela OSSOS não existe */
$dbh->exec("INSERT INTO ossos(cranio) VALUES ('lucy')");

echo "\nPDO::errorCode(): ", $dbh->errorCode();
?>
```

O exemplo acima produzirá:

```
PDO::errorCode(): 42S02
```

Veja também

- PDO::errorInfo() - Busca informação de erro estendida associada com a última operação no identificador do banco de dados
- PDOStatement::errorCode() - Busca o SQLSTATE associado com a última operação no objeto da instrução
- PDOStatement::errorInfo() - Busca informação de erro estendida associada com a última operação no identificador da instrução

PDO::errorInfo

PDO::errorInfo

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::errorInfo — Busca informação de erro estendida associada com a última operação no identificador do banco de dados

Descrição

public function **PDO::errorInfo**(): array

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

PDO::errorInfo() retorna um array de informações de erro sobre a última operação realizada por este identificador de banco de dados. O array consiste em pelo menos os campos a seguir:

Elemento	Informação
0	Código de erro SQLSTATE (um identificador alfanumérico de cinco caracteres definido no padrão ANSI SQL).
1	Código de erro específico do driver.
2	Mensagem de erro específica do driver.

> **Nota** :
>
> Se o código de erro SQLSTATE não estiver definido ou se não houver erro específico do driver, os elementos após o elemento 0 serão definidos para `null`.

PDO::errorInfo() somente busca informação de erro para operações realizadas diretamente no identificador do banco de dados. Se for criado um objeto PDOStatement através de PDO::prepare() ou PDO::query() e for invocado um erro no identificador da instrução, **PDO::errorInfo()** não refletirá o erro do identificador da instrução. Deve-se chamar PDOStatement::errorInfo() para retornar a informação de erro para uma operação realizada em um identificador de instrução em particular.

Examples

Exemplo #1 Mostrando campos de errorInfo() para uma conexão PDO_ODBC a um banco de dados DB2

```php
<?php
/* Provoca um erro -- sintaxe SQL incorreta */
$stmt = $dbh->prepare('bogus sql');
if (!$stmt) {
    echo "\nPDO::errorInfo():\n";
    print_r($dbh->errorInfo());
}
?>
```

O exemplo acima produzirá:

```
PDO::errorInfo():
Array
(
    [0] => HY000
    [1] => 1
    [2] => near "bogus": syntax error
)
```

Veja também

- PDO::errorCode() - Busca o SQLSTATE associado com a última operação no identificador do banco de dados
- PDOStatement::errorCode() - Busca o SQLSTATE associado com a última operação no objeto da instrução
- PDOStatement::errorInfo() - Busca informação de erro estendida associada com a última operação no identificador da instrução

PDO::exec

PDO::exec

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::exec — Executa uma instrução SQL e retorna o número de linhas afetadas

Descrição

public function **PDO::exec**(string `$statement`): int|false

PDO::exec() executa uma instrução SQL em uma única chamada de função, retornando o número de linhas afetadas pela instrução.

PDO::exec() não retorna resultados de uma instrução SELECT. Para uma instrução SELECT que você só precisa emitir uma vez durante seu programa, considere a emissão de PDO::query(). Para uma instrução que você precisa emitir várias vezes, prepare um objeto PDOStatement com PDO::prepare() e emita a instrução com PDOStatement::execute().

Parâmetros

`statement`

A instrução SQL para preparar e executar.

Os dados dentro da consulta devem ser adequadamente escapados.

Valor Retornado

PDO::exec() retorna o número de linhas que foram modificadas ou excluídas pela instrução SQL que você emitiu. Se nenhuma linha for afetada, **PDO::exec()** retorna `0`.

> **Aviso**
>
> Esta função pode retornar o valor booleano `false`, mas também pode retornar um valor não booleano que pode ser avaliado como `false`. Leia a seção sobre Booleanos para mais informações. Use o operador `===` para testar o valor retornado por esta função.

O exemplo a seguir depende incorretamente do valor de retorno de **PDO::exec()**, em que uma instrução que afetou 0 linhas resulta em uma chamada para die():

```php
<?php
$db->exec() or die(print_r($db->errorInfo(), true)); // incorreto
?>
```

Erros/Exceções

Emite um erro de nível `E_WARNING` se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_WARNING`.

Lança uma exceção PDOException se o atributo **PDO::ATTR_ERRMODE** estiver definido como **PDO::ERRMODE_EXCEPTION**.

Examples

Exemplo #1 Emitindo uma instrução DELETE

Conte o número de linhas excluídas por uma instrução DELETE sem cláusula WHERE.

```php
<?php
$dbh = new PDO('odbc:sample', 'db2inst1', 'ibmdb2');

/* Apaga todas as linhas da tabela FRUIT */
$count = $dbh->exec("DELETE FROM fruit");

/* Retorna o número de linhas apagadas */
print "Apagada(s) $count linha(s).\n";
?>
```

O exemplo acima produzirá:

```
Apagada(s) 1 linha(s).
```

Veja também

- PDO::prepare() - Prepara uma instrução para execução e retorna um objeto de instrução
- PDO::query() - Prepara e executa uma instrução SQL sem marcadores
- PDOStatement::execute() - Executa ums instrução preparada

PDO::getAttribute

PDO::getAttribute

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDO::getAttribute — Recuperar um atributo da conexão com o banco de dados

Descrição

public function **PDO::getAttribute**(int `$attribute`): mixed

Esta função retorna o valor de um atributo da conexão com o banco de dados. Para recuperar atributos de PDOStatement consulte PDOStatement::getAttribute().

Note que algumas combinações de banco de dados e drivers podem não suportar todos os atributos da conexão com o banco de dados.

Parâmetros

`attribute`

Uma das constantes `PDO::ATTR_*`. Os atributos genéricos que se aplicam a conexões com o banco de dados são os seguintes:

- `PDO::ATTR_CASE`
- `PDO::ATTR_DEFAULT_FETCH_MODE`
- `PDO::ATTR_DRIVER_NAME`
- `PDO::ATTR_ERRMODE`
- `PDO::ATTR_ORACLE_NULLS`
- `PDO::ATTR_PERSISTENT`
- `PDO::ATTR_STATEMENT_CLASS`
- `PDO::ATTR_STRINGIFY_FETCHES`

Alguns drivers podem fazer uso de atributos específicos adicionais. Note que atributos específicos de drivers *não devem* ser usados com outros drivers.

Valor Retornado

Uma chamada com sucesso retornará o valor do atributo PDO solicitado. Uma chamada sem sucesso retornará `null`.

Erros/Exceções

PDO::getAttribute() pode lançar uma PDOException quando o driver subjacente não suportar o `attribute` requerido.

Registro de Alterações

Versão	Descrição
8.4.0	Agora é possível obter o valor do atributo `PDO::ATTR_STRINGIFY_FETCHES`.

Examples

Exemplo #1 Recuperando atributos de uma conexão com o banco de dados

```php
<?php
$conn = new PDO('odbc:sample', 'db2inst1', 'ibmdb2');
$attributes = array(
    "AUTOCOMMIT", "ERRMODE", "CASE", "CLIENT_VERSION", "CONNECTION_STATUS",
    "ORACLE_NULLS", "PERSISTENT", "PREFETCH", "SERVER_INFO", "SERVER_VERSION",
    "TIMEOUT"
);

foreach ($attributes as $val) {
    echo "PDO::ATTR_$val: ";
    echo $conn->getAttribute(constant("PDO::ATTR_$val")) . "\n";
}
?>
```

Veja também

- PDO::setAttribute() - Define um atributo
- PDOStatement::getAttribute() - Recupera um atributo da instrução
- PDOStatement::setAttribute() - Define um atributo na instrução

PDO::getAvailableDrivers

PDO::getAvailableDrivers

pdo_drivers

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 1.0.3)

PDO::getAvailableDrivers -- pdo_drivers — Retorna um array com os drivers PDO disponíveis

Descrição

public static function **PDO::getAvailableDrivers**(): array
function **pdo_drivers**(): array

Esta função retorna todos os drivers PDO disponíveis que podem ser usados no parâmetro DSN de PDO::__construct().

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

PDO::getAvailableDrivers() retorna um array com os drivers PDO disponíveis. Se nenhum driver estiver disponível então o retorno será um array vazio.

Examples

Exemplo #1 Exemplo de PDO::getAvailableDrivers()

```php
<?php
print_r(PDO::getAvailableDrivers());
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array
(
    [0] => mysql
    [1] => sqlite
)
```

PDO::inTransaction

PDO::inTransaction

(PHP 5 >= 5.3.3, Bundled pdo_pgsql, PHP 7, PHP 8)

PDO::inTransaction — Verifica se está dentro de uma transação

Descrição

public function **PDO::inTransaction**(): bool

Verifica se uma transação está atualmente ativa no driver. Este método funciona somente para drivers de banco de dados que suportam transações.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` se uma transação estiver ativa, e `false` caso contrário.

PDO::lastInsertId

PDO::lastInsertId

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::lastInsertId — Retorna o ID da última linha ou valor de sequência inserido

Descrição

public function **PDO::lastInsertId**(?string `$name = null`): string|false

Retorna o ID da última linha inserida, ou o último valor de um objeto de sequência, dependendo do driver subjacente. Por exemplo, PDO_PGSQL permite que o nome de qualquer objeto de sequência seja especificado para o parâmetro `name`.

> **Nota** :
>
> Este método pode não retornar um resultado significativo ou consistente em diferentes drivers PDO, pois o banco de dados subjacente pode nem mesmo suportar a noção de campos ou sequências auto-incrementais.

Parâmetros

`name`

Nome do objeto de sequência do qual o ID deve ser retornado.

Valor Retornado

Se um nome de sequência não foi especificado para o parâmetro `name`, **PDO::lastInsertId()** retornará uma string representando o ID da última linha que foi inserida no banco de dados.

Se um nome de sequência foi especificado para o parâmetro `name`, **PDO::lastInsertId()** retornará uma string representando o último valor recuperado do objeto de sequência especificado.

Se o driver PDO não suportar este recurso, **PDO::lastInsertId()** dispara um SQLSTATE `IM001`.

Erros/Exceções

Emite um erro de nível `E_WARNING` se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_WARNING`.

Lança uma exceção PDOException se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_EXCEPTION`.

PDO::prepare

PDO::prepare

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PHP 8,PECL pdo >= 0.1.0)

PDO::prepare — Prepara uma instrução para execução e retorna um objeto de instrução

Descrição

public function **PDO::prepare**(string `$query`, array `$options = []`): PDOStatement|false

Prepara uma instrução SQL para ser executada pelo método PDOStatement::execute(). O modelo de instrução pode conter zero ou mais marcadores de parâmetros nomeados (:name) ou pontos de interrogação (?) para os quais os valores reais serão substituídos quando a instrução for executada. Os marcadores de parâmetros nomeados e de ponto de interrogação não podem ser usados no mesmo modelo de instrução; apenas um ou outro estilo de parâmetro. Use esses parâmetros para vincular qualquer entrada do usuário, não inclua a entrada do usuário diretamente na consulta.

Deve-se incluir um marcador de parâmetro exclusivo para cada valor que se deseja passar para a instrução ao chamar PDOStatement::execute(). Um marcador de parâmetro nomeado com o mesmo nome não pode ser usado mais de uma vez em uma instrução preparada, a menos que o modo de emulação esteja ativado.

> **Nota** :
>
> Os marcadores de parâmetro podem representar apenas um literal de dados completo. Nenhuma parte do literal, nem palavra-chave, nem identificador, nem qualquer parte arbitrária da consulta pode ser vinculada usando parâmetros. Por exemplo, não se pode vincular múltiplos valores a um único parâmetro na cláusula IN() de uma instrução SQL.

Chamar **PDO::prepare()** e PDOStatement::execute() para instruções que serão emitidas diversas vezes com valores de parâmetros diferentes otimiza o desempenho da aplicação, permitindo que o driver negocie cache no lado do cliente e/ou do servidor do plano de consulta e metainformações. Além disso, chamar **PDO::prepare()** e PDOStatement::execute() ajuda a evitar ataques de injeção de SQL, eliminando a necessidade de inserir aspas e escapar manualmente dos parâmetros.

O PDO irá emular instruções preparadas/parâmetros vinculados para drivers que não os suportam nativamente, e também pode reescrever marcadores de parâmetros nomeados ou no estilo ponto de interrogação para algo mais apropriado, se o driver suportar um estilo, mas não o outro.

> **Nota** :
>
> Antes do PHP 8.4.0, o analisador usado para instruções preparadas emuladas e para reescrever parâmetros nomeados ou no estilo de ponto de interrogação suportava escapes de barra invertida não padrão para aspas simples e duplas. Isso significava que aspas de término imediatamente precedidas por uma barra invertida não eram reconhecidas como tal, o que poderia resultar na detecção incorreta de parâmetros, fazendo com que a instrução preparada falhasse quando fosse executada. Uma solução alternativa era não usar preparações emuladas para essas consultas SQL e evitar a reescrita de parâmetros usando um estilo de parâmetro que fosse suportado nativamente pelo driver.

A partir do PHP 7.4.0, os pontos de interrogação podem ser escapados duplicando-os. Isso significa que a string `??` será traduzida para `?` ao enviar a consulta ao banco de dados.

A partir do PHP 8.4.0, O PDO usa analisadores específicos do driver para localizar marcadores de parâmetros, de forma que os marcadores que aparecem dentro de strings literais ou comentários não são confundidos como marcadores de parâmetro. O analisador padrão reconhece literais de string entre aspas simples e duplas, assim como comentários de duas barras e comentários no estilo C não aninhados. Drivers individuais podem fornecer um analisador que reconheça sintaxe adicional ou diferente; consulte a documentação específica do driver para mais detalhes.

Parâmetros

`query`

Este deve ser um modelo de instrução SQL válido para o servidor de banco de dados de destino.

`options`

Este array contém um ou mais pares chave=>valor para definir valores de atributos para o objeto PDOStatement que esse método retorna. Normalmente usa-se isso para definir o valor `PDO::ATTR_CURSOR` como `PDO::CURSOR_SCROLL` para solicitar um cursor rolável. Alguns drivers têm opções específicas que podem ser definidas no momento da preparação.

Valor Retornado

Se o servidor de banco de dados preparar a instrução com sucesso, **PDO::prepare()** retornará um objeto PDOStatement. Se o servidor de banco de dados não conseguir preparar a instrução com sucesso, **PDO::prepare()** retornará `false` ou emitirá uma exceção PDOException (dependendo do tratamento de erros).

> **Nota** :
>
> Instruções preparadas emuladas não se comunicam com o servidor de banco de dados, portanto **PDO::prepare()** não verifica a instrução.

Erros/Exceções

Emite um erro de nível `E_WARNING` se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_WARNING`.

Lança uma exceção PDOException se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_EXCEPTION`.

Examples

Exemplo #1 Modelo de instrução SQL com parâmetros nomeados

```php
<?php
/* Executa uma instrução preparada passando um array de valores */
$sql = 'SELECT name, colour, calories
    FROM fruit
    WHERE calories < :calories AND colour = :colour';
$sth = $dbh->prepare($sql, [PDO::ATTR_CURSOR => PDO::CURSOR_FWDONLY]);
$sth->execute(['calories' => 150, 'colour' => 'red']);
$red = $sth->fetchAll();
/* Chaves de arrays podem ser prefizadas com dois-pontos ":" também (opcional) */
$sth->execute([':calories' => 175, ':colour' => 'yellow']);
$yellow = $sth->fetchAll();
?>
```

Exemplo #2 Modelo de instrução SQL com parâmetros de ponto de interrogação

```php
<?php
/* Executa uma instrução preparada passando um array de valores */
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < ? AND colour = ?');
$sth->execute([150, 'red']);
$red = $sth->fetchAll();
$sth->execute([175, 'yellow']);
$yellow = $sth->fetchAll();
?>
```

Exemplo #3 Modelo de instrução SQL com ponto de interrogação escapado

```php
<?php
/* nota: isso só é válido em bancos de dados PostgreSQL */
$sth = $dbh->prepare('SELECT * FROM issues WHERE tag::jsonb ?? ?');
$sth->execute(['feature']);
$featureIssues = $sth->fetchAll();
$sth->execute(['performance']);
$performanceIssues = $sth->fetchAll();
?>
```

Veja também

- PDO::exec() - Executa uma instrução SQL e retorna o número de linhas afetadas
- PDO::query() - Prepara e executa uma instrução SQL sem marcadores
- PDOStatement::execute() - Executa ums instrução preparada

PDO::query

PDO::query

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDO::query — Prepara e executa uma instrução SQL sem marcadores

Descrição

public function **PDO::query**(string `$query`, ?int `$fetchMode` = `null`): PDOStatement|false
public function **PDO::query**(string `$query`, ?int `$fetchMode` = PDO::FETCH_COLUMN, int `$colno`): PDOStatement|false
public function **PDO::query**(
  string `$query`,
  ?int `$fetchMode` = PDO::FETCH_CLASS,
  string `$classname`,
  array `$constructorArgs`
): PDOStatement|false
public function **PDO::query**(string `$query`, ?int `$fetchMode` = PDO::FETCH_INTO, object `$object`): PDOStatement|false

PDO::query() prepara e executa uma instrução SQL em uma única chamada de função, retornando a instrução como um objeto PDOStatement.

Para uma consulta que precisa ser emitida diversas vezes, obtém-se melhor desempenho se for preparado um objeto PDOStatement usando PDO::prepare() e emitida a instrução com múltiplas chamadas a PDOStatement::execute().

Se não forem buscados todos os dados em um conjunto de resultados antes de emitir a próxima chamada a **PDO::query()**, a chamada poderá falhar. Deve-se chamar PDOStatement::closeCursor() para liberar os recursos do banco de dados associados ao objeto PDOStatement antes de se emitir a próxima chamada a **PDO::query()**.

> **Nota** :
>
> Se a `query` contiver marcadores, a instrução deverá ser preparada e executada separadamente usando os métodos PDO::prepare() e PDOStatement::execute().

Parâmetros

`query`

A instrução SQL a ser preparada e executada.

Se o SQL contiver marcadores, PDO::prepare() e PDOStatement::execute() deverão ser usados. Alternativamente, o SQL pode ser preparado manualmente antes de chamar **PDO::query()**, com os dados formatados corretamente usando PDO::quote() se o driver suportar.

`fetchMode`

O modo de busca padrão para o PDOStatement retornado. Deve ser uma das constantes `PDO::FETCH_*`.

Se este argumento for passado para a função, os argumentos restantes serão tratados como se PDOStatement::setFetchMode() tivesse sido chamado no objeto de instrução resultante. Os argumentos subsequentes variam dependendo do modo de busca selecionado.

Valor Retornado

Retorna um objeto PDOStatement ou **`false`** em caso de falha.

Erros/Exceções

Emite um erro de nível **`E_WARNING`** se o atributo **`PDO::ATTR_ERRMODE`** estiver definido como **`PDO::ERRMODE_WARNING`**.

Lança uma exceção PDOException se o atributo **`PDO::ATTR_ERRMODE`** estiver definido como **`PDO::ERRMODE_EXCEPTION`**.

Examples

Exemplo #1 SQL sem marcadores pode ser executado usando PDO::query()

```php
<?php
$sql = 'SELECT name, color, calories FROM fruit ORDER BY name';
foreach ($conn->query($sql) as $row) {
    print $row['name'] . "\t";
    print $row['color'] . "\t";
    print $row['calories'] . "\n";
}
?>
```

O exemplo acima produzirá:

```
apple   red     150
banana  yellow  250
kiwi    brown   75
lemon   yellow  25
orange  orange  300
pear    green   150
watermelon      pink    90
```

Veja também

- PDO::exec() - Executa uma instrução SQL e retorna o número de linhas afetadas
- PDO::prepare() - Prepara uma instrução para execução e retorna um objeto de instrução
- PDOStatement::execute() - Executa ums instrução preparada

PDO::quote

PDO::quote

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.1)

PDO::quote — Inclui aspas em uma string para uso em uma consulta

Descrição

public function **PDO::quote**(string `$string`, int `$type` = `PDO::PARAM_STR`): string|false

PDO::quote() coloca aspas na string de entrada e escapa caracteres especiais dentro da string de entrada, usando um estilo de aspas (ex.: simples ou duplas) apropriado ao driver subjacente.

Se esta função estiver sendo usada para construir instruções SQL, é *fortemente* recomendado usar PDO::prepare() para preparar instruções SQL com parâmetros vinculados em vez de usar **PDO::quote()** para interpolar a entrada do usuário em uma instrução SQL. Instruções preparadas com parâmetros vinculados não são apenas mais portáveis, mais convenientes e imunes à injeção de SQL, mas geralmente são muito mais rápidas de executar do que consultas interpoladas, já que tanto o servidor quanto o cliente podem armazenar em cache uma forma compilada da consulta.

Nem todos os drivers PDO implementam esse método (principalmente PDO_ODBC). Considere usar instruções preparadas.

> **Cuidado**
>
> **Segurança: o conjunto de caracteres padrão**
>
> O conjunto de caracteres deve ser definido no nível do servidor ou na própria conexão do banco de dados (dependendo do driver) para afetar o **PDO::quote()**. Consulte a documentação específica do driver para obter mais informações.

Parâmetros

`string`

A string que será envolvida por aspas.

`type`

Fornece uma dica sobre o tipo de dados para drivers que possuem estilos de escape alternativos. Por exemplo, **`PDO_PARAM_LOB`** dirá ao driver para escapar dados binários.

Valor Retornado

Retorna uma string entre aspas que é teoricamente segura para passar em uma instrução SQL. Retorna `false` se o driver não suportar inserção de aspas dessa forma.

Examples

Exemplo #1 Inserindo aspas em uma string normal

```php
<?php
$conn = new PDO('sqlite:/home/lynn/music.sql3');

/* String simples */
$string = 'Teste';
print "String sem aspas: $string\n";
print "String com aspas: " . $conn->quote($string) . "\n";
?>
```

O exemplo acima produzirá:

```
String sem aspas: Teste
String com aspas: 'Teste'
```

Exemplo #2 Inserindo aspas em uma string perigosa

```php
<?php
$conn = new PDO('sqlite:/home/lynn/music.sql3');

/* String perigosa */
$string = 'String \' perversa';
print "String sem aspas: $string\n";
print "String com aspas:" . $conn->quote($string) . "\n";
?>
```

O exemplo acima produzirá:

```
String sem aspas: String ' perversa
String com aspas: 'String '' perversa'
```

Exemplo #3 Inserindo aspas em uma string complexa

```php
<?php
$conn = new PDO('sqlite:/home/lynn/music.sql3');

/* String complexa */
$string = "St'ri''ng \"com'\"plexa";
print "String sem aspas: $string\n";
print "String com aspas: " . $conn->quote($string) . "\n";
?>
```

O exemplo acima produzirá:

```
String sem aspas: St'ri''ng "com'"plexa
String com aspas: 'St''ri''''ng "com''"plexa'
```

Veja também

- PDO::prepare() - Prepara uma instrução para execução e retorna um objeto de instrução
- PDOStatement::execute() - Executa ums instrução preparada

PDO::rollBack

PDO::rollBack

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::rollBack — Reverte uma transação

Descrição

public function **PDO::rollBack**(): bool

Reverte a transação atual, iniciada por PDO::beginTransaction().

Se o banco de dados foi definido para o modo de confirmação automática, esta função restaurará o modo de confirmação automática após a reversão da transação.

Alguns bancos de dados, incluindo MySQL, emitem automaticamente um COMMIT implícito quando uma instrução DDL (linguagem de definição de banco de dados), como DROP TABLE ou CREATE TABLE, é emitida em uma transação. O COMMIT implícito impedirá que quaisquer outras alterações sejam revertidas dentro do limite da transação.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Lança uma exceção PDOException se não houver transação ativa.

> **Nota** :
>
> Uma exceção será lançada mesmo quando o atributo `PDO::ATTR_ERRMODE` não for `PDO::ERRMODE_EXCEPTION`.

Examples

Exemplo #1 Revertendo uma transação

O exemplo a seguir inicia uma transação e emite duas instruções que modificam o banco de dados antes de reverter as alterações. No MySQL, entretanto, a instrução DROP TABLE confirma automaticamente a transação de forma que nenhuma das alterações na transação é revertida.

```php
<?php
/* Inicia uma transação, desligando a auto-confirmação */
$dbh->beginTransaction();

/* Altera esquema e dados do bando */
$sth = $dbh->exec("DROP TABLE fruta");
$sth = $dbh->exec("UPDATE sobremesa
    SET nome = 'hamburger'");

/* Reconhece o erro e reverte as alterações */
$dbh->rollBack();

/* A conexão do banco agora está de volta no modo de auto-confirmação */
?>
```

Veja também

- PDO::beginTransaction() - Inicia uma transação
- PDO::commit() - Envia uma transação
- Transações e auto-confirmação

PDO::setAttribute

PDO::setAttribute

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::setAttribute — Define um atributo

Descrição

public function **PDO::setAttribute**(int `$attribute`, mixed `$value`): bool

Define um atributo no identificador do banco de dados. Alguns atributos genéricos disponíveis estão listados abaixo; alguns drivers podem fazer uso de atributos adicionais específicos do driver. Observe que os atributos específicos do driver *não devem* ser usadoscom outros drivers.

`PDO::ATTR_CASE`

Força os nomes das colunas para uma combinação específica de maiúsculas/minúsculas. Pode ser um dos seguintes valores:

`PDO::CASE_LOWER`

Força os nomes das colunas para minúsculas.

`PDO::CASE_NATURAL`

Deixa os nomes das colunas como retornados pelo driver do banco de dados.

`PDO::CASE_UPPER`

Força os nomes das colunas para maiúsculas.

`PDO::ATTR_ERRMODE`

Modo de relatório de erros do PDO. Pode ser um dos seguintes valores:

`PDO::ERRMODE_SILENT`

Define somente códigos de erro.

`PDO::ERRMODE_WARNING`

Emite diagnósticos `E_WARNING`.

`PDO::ERRMODE_EXCEPTION`

Lança exceções PDOException.

`PDO::ATTR_ORACLE_NULLS`

> **Nota** :
>
> Este atributo está disponível em todos os drivers, não somente no Oracle.

Determina se e como strings `null` e vazias devem ser convertidas. Pode ser um dos seguintes valores:

`PDO::NULL_NATURAL`

Nenhuma conversão acontece.

`PDO::NULL_EMPTY_STRING`

Strings vazias são convertidas para `null`.

`PDO::NULL_TO_STRING`

`null` é convertido para uma string vazia.

`PDO::ATTR_STRINGIFY_FETCHES`

Controla se os valores buscados (exceto `null`) são convertidos para strings. Aceita um valor do tipo bool: `true` para habilitar e `false` para desabilitar (padrão). Valores `null` permanecem inalterados a menos que

`PDO::ATTR_ORACLE_NULLS` esteja definida para `PDO::NULL_TO_STRING`.

`PDO::ATTR_STATEMENT_CLASS`

Defina a classe de instrução fornecida pelo usuário derivada de PDOStatement. Requer `array(string classname, array(mixed constructor_args))`.

> **Cuidado**
>
> Não pode ser usado com instâncias persistentes de PDO.

`PDO::ATTR_TIMEOUT`

Especifica a duração do tempo limite em segundos. Recebe um valor do tipo int.

> **Nota** :
>
> Nem todos os drivers suportam esta opção e seu significado pode variar de driver para driver. Por exemplo, o SQLite irá esperar até esse valor de tempo antes de desistir de obter uma trava para escrita, mas outros drivers podem interpretar isso como intervalo de tempo limite de conexão ou de leitura.

`PDO::ATTR_AUTOCOMMIT`

> **Nota** :
>
> Disponível apenas para drivers OCI, Firebird e MySQL.

Define se cada instrução deve ser confirmada automaticamente. Recebe um valor do tipo bool: `true` para habilitar e `false` para desabilitar. Por padrão, `true`.

`PDO::ATTR_EMULATE_PREPARES`

> **Nota** :
>
> Disponível apenas para drivers OCI, Firebird e MySQL.

Define se a emulação de instruções preparadas será habilitada ou desabilitada. Alguns drivers não oferecem suporte nativo a instruções preparadas ou têm suporte limitado para elas. Se definido como `true` O PDO sempre emulará instruções preparadas, caso contrário, o PDO tentará usar instruções preparadas nativas. Caso o driver não consiga preparar com êxito a instrução atual, o PDO sempre voltará a emular a instrução preparada.

`PDO::MYSQL_ATTR_USE_BUFFERED_QUERY`

> **Nota** :
>
> Disponível apenas para o driver MySQL.

Define se consultas com buffer devem ser usadas. Recebe um valor do tipo bool: `true` para habilitar e `false` para desabilitar. Por padrão, `true`.

`PDO::ATTR_DEFAULT_FETCH_MODE`

Define o modo de busca padrão. Uma descrição dos modos e de como utilizá-los está disponível na documentação do método PDOStatement::fetch().

Parâmetros

`attribute`

O atributo a modificar.

`value`

O valor para o qual o `attribute` será definido, pode requerer um tipo específico dependendo do atributo.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Veja também

- PDO::getAttribute() - Recuperar um atributo da conexão com o banco de dados
- PDOStatement::getAttribute() - Recupera um atributo da instrução
- PDOStatement::setAttribute() - Define um atributo na instrução

PDOStatement

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 1.0.0)

Introdução

Representa uma instrução preparada e, após a execução da instrução, um conjunto de resultados associado.

Resumo da classe

```
class PDOStatement implements IteratorAggregate {
/* Propriedades */
public string $queryString;
/* Métodos */
public function bindColumn(
    string|int $column,
    mixed &$var,
    int $type = PDO::PARAM_STR,
    int $maxLength = 0,
    mixed $driverOptions = null
): bool
public function bindParam(
    string|int $param,
    mixed &$var,
    int $type = PDO::PARAM_STR,
    int $maxLength = 0,
```

```
    mixed $driverOptions = null
): bool
public function bindValue(string|int $param, mixed $value, int $type = PDO::PARAM_STR): bool
public function closeCursor(): bool
public function columnCount(): int
public function debugDumpParams(): ?bool
public function errorCode(): ?string
public function errorInfo(): array
public function execute(?array $params = null): bool
public function fetch(int $mode = PDO::FETCH_DEFAULT, int $cursorOrientation = PDO::FETCH_ORI_NEXT, int
$cursorOffset = 0): mixed
public function fetchAll(int $mode = PDO::FETCH_DEFAULT): array
public function fetchAll(int $mode = PDO::FETCH_COLUMN, int $column): array
public function fetchAll(int $mode = PDO::FETCH_CLASS, string $class, ?array $constructorArgs): array
public function fetchAll(int $mode = PDO::FETCH_FUNC, callable $callback): array
public function fetchColumn(int $column = 0): mixed
public function fetchObject(?string $class = "stdClass", array $constructorArgs = []): object|false
public function getAttribute(int $name): mixed
public function getColumnMeta(int $column): array|false
public function getIterator(): Iterator
public function nextRowset(): bool
public function rowCount(): int
public function setAttribute(int $attribute, mixed $value): bool
public function setFetchMode(int $mode): true
public function setFetchMode(int $mode = PDO::FETCH_COLUMN, int $colno): true
public function setFetchMode(int $mode = PDO::FETCH_CLASS, string $class, ?array $constructorArgs = null): true
public function setFetchMode(int $mode = PDO::FETCH_INTO, object $object): true
}
```

Propriedades

queryString

String de consulta usada.

Registro de Alterações

Versão	Descrição
8.0.0	**PDOStatement** agora implementa IteratorAggregate em vez de Traversable.

Índice

PDOStatement::bindColumn

PDOStatement::bindColumn

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::bindColumn — Vincula uma coluna a uma variável PHP

Descrição

```
public function PDOStatement::bindColumn(
    string|int $column,
    mixed &$var,
    int $type = PDO::PARAM_STR,
    int $maxLength = 0,
    mixed $driverOptions = null
): bool
```

PDOStatement::bindColumn() faz com que uma variável em particular seja vinculada a uma coluna informada, no conjunto de resultados de uma consulta. Cada chamada a PDOStatement::fetch() ou PDOStatement::fetchAll() irá atualizar todas as variáveis que estejam vinculadas a colunas.

> **Nota** :
>
> Como as informações sobre a coluna não estão sempre disponíveis ao PDO até que a instrução seja executada, aplicações portáteis devem chamar esta função *após* PDOStatement::execute().
>
> Entretanto, para que seja possível vincular uma coluna LOB como um fluxo ao usar o *driver PgSQL*, aplicações devem chamar este método *antes* de chamada a PDOStatement::execute(), caso contrário o OID de objeto grande será retornado como um inteiro.

Parâmetros

`column`

Número da coluna (índices iniciando em 1) ou nome da coluna no conjunto de resultados. Se o nome da coluna estiver sendo usado, o nome deve ter as maiúsculas/minúsculas correspondentes à coluna, como retornada pelo driver.

`var`

Nome da variável PHP à qual a coluna será vinculada.

`type`

Tipo de dado do parâmetro, especificado pelas constantes `PDO::PARAM_*` .

`maxLength`

Um dica para pré-alocação.

`driverOptions`

Um ou mais parâmetros opcionais para o driver.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Emite um erro de nível **`E_WARNING`** se o atributo **`PDO::ATTR_ERRMODE`** estiver definido como **`PDO::ERRMODE_WARNING`**.

Lança uma exceção PDOException se o atributo **`PDO::ATTR_ERRMODE`** estiver definido como **`PDO::ERRMODE_EXCEPTION`**.

Examples

Exemplo #1 Vinculando a saída do conjunto de resultados a variáveis PHP

Vincular colunas no conjunto de resultados a variáveis PHP é uma maneira eficaz de fazer com que os dados contidos em cada linha estejam imediatamente disponíveis para a aplicação. O exemplo a seguir demonstra como o PDO permite que se vincule e se recupere colunas com uma variedade de opções e com padrões inteligentes.

```php
<?php
$stmt = $dbh->prepare('SELECT name, colour, calories FROM fruit');
$stmt->execute();

/* Vincula por número da coluna */
$stmt->bindColumn(1, $name);
$stmt->bindColumn(2, $colour);

/* Vincula por nome da coluna */
$stmt->bindColumn('calories', $cals);

while ($stmt->fetch(PDO::FETCH_BOUND)) {
    print $name . "\t" . $colour . "\t" . $cals . "\n";
}
```

O exemplo acima produzirá algo semelhante a:

```
apple   red     150
banana  yellow  175
kiwi    green   75
orange  orange  150
mango   red     200
strawberry      red     25
```

Veja também

- PDOStatement::execute() - Executa ums instrução preparada
- PDOStatement::fetch() - Busca a próxima linha de um resultado
- PDOStatement::fetchAll() - Busca as linhas remanescentes de um conjunto de resultados
- PDOStatement::fetchColumn() - Retorna uma única coluna do próximo registro de um resultado

PDOStatement::bindParam

PDOStatement::bindParam

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::bindParam — Vincula um parâmetro ao nome de variável especificado

Descrição

```
public function PDOStatement::bindParam(
    string|int $param,
    mixed &$var,
    int $type = PDO::PARAM_STR,
    int $maxLength = 0,
    mixed $driverOptions = null
): bool
```

Vincula uma variável PHP a um marcador nomeado ou de ponto de interrogação correspondente na instrução SQL que foi usada para preparar a instrução. Ao contrário de PDOStatement::bindValue(), a variável é vinculada como uma referência e só será avaliada no momento em que PDOStatement::execute() for chamado.

A maioria dos parâmetros são parâmetros de entrada, ou seja, parâmetros que são usadossomente leitura para construir a consulta (mas podem, mesmo assim, ser convertidos de acordo com o `type`). Alguns drivers suportam a invocação de procedimentos armazenados que retornam dados como parâmetros de saída, e alguns também como parâmetros de entrada/saída que tanto enviam dados quanto são atualizados para recebê-los.

Parâmetros

`param`

Identificador de parâmetro. Para uma instrução preparada usando marcadores nomeados, este será um nome de parâmetro no formato *:name*. Para uma instrução preparada usando marcadores de pontos de interrogação, esta será a posição indexada em 1 do parâmetro.

`var`

Nome da variável PHP a ser vinculada ao parâmetro da instrução SQL.

`type`

Tipo explícito de dados para o parâmetro usando as constantes `PDO::PARAM_*`. Para retornar um parâmetro INOUT de um procedimento armazenado, deve-se usar o operador binário OR para definir os bits **`PDO::PARAM_INPUT_OUTPUT`** para o parâmetro `type`.

`maxLength`

Comprimento do tipo de dados. Para indicar que um parâmetro é um parâmetro OUT de um procedimento armazenado, deve-se definir explicitamente o comprimento. Significativo apenas quando o parâmetro `type` é **`PDO::PARAM_INPUT_OUTPUT`**.

`driverOptions`

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Emite um erro de nível **`E_WARNING`** se o atributo **`PDO::ATTR_ERRMODE`** estiver definido como **`PDO::ERRMODE_WARNING`**.

Lança uma exceção PDOException se o atributo **`PDO::ATTR_ERRMODE`** estiver definido como **`PDO::ERRMODE_EXCEPTION`**.

Examples

Exemplo #1 Executando uma instrução preparada com marcadores nomeados

```php
<?php
/* Executa uma instrução preparada vinculando variáveis PHP */
$calories = 150;
$colour = 'red';
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < :calories AND colour = :colour');
$sth->bindParam('calories', $calories, PDO::PARAM_INT);
/* Os nomes podem ser prefixados com dois pontos ":" também (opcional) */
$sth->bindParam(':colour', $colour, PDO::PARAM_STR);
$sth->execute();
?>
```

Exemplo #2 Executando uma instrução preparada com marcadores com pontos de interrogação

```php
<?php
/* Executa uma instrução preparada vinculando variáveis PHP */
$calories = 150;
$colour = 'red';
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < ? AND colour = ?');
$sth->bindParam(1, $calories, PDO::PARAM_INT);
$sth->bindParam(2, $colour, PDO::PARAM_STR);
$sth->execute();
?>
```

Exemplo #3 Chamando um procedimento armazenado com um parâmetro INOUT

```php
<?php
/* Chamando um procedimento armazenado com um parâmetro INOUT */
$colour = 'red';
$sth = $dbh->prepare('CALL puree_fruit(?)');
$sth->bindParam(1, $colour, PDO::PARAM_STR|PDO::PARAM_INPUT_OUTPUT, 12);
$sth->execute();
print "Após fazer um purê com a fruta, a cor é: $colour";
?>
```

Veja também

- PDO::prepare() - Prepara uma instrução para execução e retorna um objeto de instrução
- PDOStatement::execute() - Executa ums instrução preparada
- PDOStatement::bindValue() - Vincula um valor a um parâmetro

PDOStatement::bindValue

PDOStatement::bindValue

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 1.0.0)

PDOStatement::bindValue — Vincula um valor a um parâmetro

Descrição

public function **PDOStatement::bindValue**(string|int `$param`, mixed `$value`, int `$type` = **`PDO::PARAM_STR`**): bool

Vincula um valor a um marcador, nomeado ou de ponto de interrogação, na instrução SQL que foi usada para preparar a instrução.

Parâmetros

`param`

Identificador do parâmetro. Para uma instrução preparada que usa marcadores nomeados, será um nome de parâmetro na forma *:name*. Para uma instrução preparada que usa marcadores pontos de interrogação, será a posição indexada do parâmetro, iniciando em 1.

`value`

O valor a ser vinculado ao parâmetro.

`type`

Tipo de dado explícito para o parâmetro, usando as constantes `PDO::PARAM_*` .

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Emite um erro de nível **`E_WARNING`** se o atributo **`PDO::ATTR_ERRMODE`** estiver definido como **`PDO::ERRMODE_WARNING`**.

Lança uma exceção PDOException se o atributo **`PDO::ATTR_ERRMODE`** estiver definido como **`PDO::ERRMODE_EXCEPTION`**.

Examples

Exemplo #1 Executando uma instrução preparada com marcadores nomeados

```php
<?php
/* Executando uma instrução preparada vinculando variáveis do PHP */
$calories = 150;
$colour = 'red';
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < :calories AND colour = :colour');

/* Define um valor para o parâmetro usando seu nome */
$sth->bindValue('calories', $calories, PDO::PARAM_INT);
/* Opcionalmente, os nomes de parâmetros podem ser prefixados com dois pontos ":" */
$sth->bindValue(':colour', $colour, PDO::PARAM_STR);
$sth->execute();
?>
```

Exemplo #2 Executando uma instrução preparada com marcadores de ponto de interrogação

```php
<?php
/* Executando uma instrução preparada vinculando variáveis do PHP */
$calories = 150;
$colour = 'red';
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
```

```
    WHERE calories < ? AND colour = ?');
$sth->bindValue(1, $calories, PDO::PARAM_INT);
$sth->bindValue(2, $colour, PDO::PARAM_STR);
$sth->execute();
?>
```

Veja também

- PDO::prepare() - Prepara uma instrução para execução e retorna um objeto de instrução
- PDOStatement::execute() - Executa ums instrução preparada
- PDOStatement::bindParam() - Vincula um parâmetro ao nome de variável especificado

PDOStatement::closeCursor

PDOStatement::closeCursor

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.9.0)

PDOStatement::closeCursor — Fecha o cursor, permitindo que a instrução seja executada novamente

Descrição

public function **PDOStatement::closeCursor**(): bool

PDOStatement::closeCursor() libera a conexão ao servidor de forma que outras instruções SQL possam ser emitidas, mas deixa a instrução em um estado que permite que ela seja executada novamente.

Este método é útil para drivers de banco de dados que não suportam a execução de um objeto PDOStatement quando um object PDOStatement executado anteriormente ainda tem registros não recebidos. Se o driver do banco de dados sofrer desta limitação, o problema pode se manifestar em um erro de "fora-de-sequência".

PDOStatement::closeCursor() é implementado como um método opcional específico do driver (permitindo máxima eficiência), ou como o método PDO genérico se nenhuma função específica do driver estiver instalada. O método PDO genérico é semanticamente o mesmo que escrever o seguinte código no script PHP:

```
<?php
do {
    while ($stmt->fetch())
        ;
    if (!$stmt->nextRowset())
        break;
} while (true);
?>
```

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Erros/Exceções

Emite um erro de nível `E_WARNING` se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_WARNING`.

Lança uma exceção PDOException se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_EXCEPTION`.

Examples

Exemplo #1 Um exemplo de PDOStatement::closeCursor()

No exemplo a seguire, o objeto PDOStatement *$stmt* retorna múltiplos registros mas a aplicação busca apenas a primeira linha, deixando o objeto PDOStatement em um estado com linhas não recebidas. Para assegurar que a aplicação irá funcionar com todos os drivers de banco de dados, o autor insere uma chamada a **PDOStatement::closeCursor()** na variável *$stmt* antes de executar o objeto PDOStatement *$otherStmt*.

```php
<?php
/* Cria um objeto PDOStatement */
$stmt = $dbh->prepare('SELECT foo FROM bar');

/* Cria um segundo objeto PDOStatement */
$otherStmt = $dbh->prepare('SELECT foobaz FROM foobar');

/* Executa a primeira instrução */
$stmt->execute();

/* Busca somente a primeira linha do resultado */
$stmt->fetch();

/* A chamada a closeCursor() pode ser requerida por alguns drivers */
$stmt->closeCursor();

/* Agora pode-se executar a segunda instrução */
$otherStmt->execute();
?>
```

Veja também

- PDOStatement::execute() - Executa ums instrução preparada

PDOStatement::columnCount

PDOStatement::columnCount

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDOStatement::columnCount — Retorna o número de colunas no conjunto de resultados

Descrição

public function **PDOStatement::columnCount**(): int

Use **PDOStatement::columnCount()** para retornar o número de colunas no conjunto de resultados representado pelo objeto PDOStatement.

Se o objeto PDOStatement foi retornado por PDO::query(), a contagem de colunas estará disponível imediatamente.

Se o objeto PDOStatement foi retornado por PDO::prepare(), uma contagem precisa de colunas não estará disponível até que o método PDOStatement::execute() seja chamado.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número de colunas no conjunto de resultados representado pelo objeto PDOStatement, mesmo se o conjunto de resultados estiver vazio. Se não houver conjunto de resultados, **PDOStatement::columnCount()** retornará `0`.

Erros/Exceções

Emite um erro de nível `E_WARNING` se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_WARNING`.

Lança uma exceção PDOException se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_EXCEPTION`.

Examples

Exemplo #1 Contando colunas

Este exemplo demonstra como **PDOStatement::columnCount()** opera com e sem um conjunto de resultados.

```php
<?php
$dbh = new PDO('odbc:sample', 'db2inst1', 'ibmdb2');

$sth = $dbh->prepare("SELECT name, colour FROM fruit");

/* Conta o número de colunas no conjunto de resultados (não existente) */
$colcount = $sth->columnCount();
print "Antes de execute(), o resultado tem $colcount colunas (deve ser 0)\n";

$sth->execute();

/* Conta o número de colunas no conjunto de resultados */
$colcount = $sth->columnCount();
print "Depois de execute(), o resultado tem $colcount colunas (devem ser 2)\n";

?>
```

O exemplo acima produzirá:

```
Antes de execute(), o resultado tem 0 colunas (deve ser 0)
Depois de execute(), o resultado tem 2 colunas (devem ser 2)
```

Veja também

- PDO::prepare() - Prepara uma instrução para execução e retorna um objeto de instrução
- PDOStatement::execute() - Executa ums instrução preparada
- PDOStatement::rowCount() - Retorna o número de registros afetados pela última instrução SQL

PDOStatement::debugDumpParams

PDOStatement::debugDumpParams

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.9.0)

PDOStatement::debugDumpParams — Despeja uma instrução preparada SQL

Descrição

public function **PDOStatement::debugDumpParams**(): ?bool

Despeja a informação contida em uma instrução preparada diretamente na saída. Fornece a consulta `SQL` em uso, o número de parâmetros usados (`Params`), a lista de parâmetros com seus nomes de chave ou posição, seus nomes, suas posições na consulta (se for suportado pelo driver PDO, caso contrário será -1), tipo (`param_type`) como um inteiro, e um valor booleano `is_param`.

Esta é uma função de depuração, que despeja os dados diretamente na saída padrão.

> **Dica**
>
> Assim como qualquer coisa que envia seu resultado diretamente para o navegador, as funções de controle de saída podem ser usadas para capturar a saída desta função e salvá-la em uma string, por exemplo.

Esta função somente despejará os parâmetros na instrução no momento do despejo. Parâmetros extras não são armazenados na instrução, e não serão mostrados.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `null`, ou `false` em caso de erro.

Registro de Alterações

Versão	Descrição
7.2.0	**PDOStatement::debugDumpParams()** agora retorna o SQL enviado ao banco de dados, incluindo a consulta completa, bruta (incluindo os marcadores substituídos por seus valores vinculados). Observe que isto só estará disponível se as instrução preparadas emuladas estiverem habilitadas.

Examples

Exemplo #1 Exemplo de PDOStatement::debugDumpParams() com parâmetros nomeados

```php
<?php
/* Executa uma instrução preparada vinculando variáveis PHP */
$calories = 150;
$colour = 'red';
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < :calories AND colour = :colour');
$sth->bindParam(':calories', $calories, PDO::PARAM_INT);
$sth->bindValue(':colour', $colour, PDO::PARAM_STR, 12);
```

```php
$sth->execute();

$sth->debugDumpParams();

?>
```

O exemplo acima produzirá:

```
SQL: [96] SELECT name, colour, calories
    FROM fruit
    WHERE calories < :calories AND colour = :colour
Params:  2
Key: Name: [9] :calories
paramno=-1
name=[9] ":calories"
is_param=1
param_type=1
Key: Name: [7] :colour
paramno=-1
name=[7] ":colour"
is_param=1
param_type=2
```

Exemplo #2 Exemplo de PDOStatement::debugDumpParams() com parâmetros anônimos

```php
<?php

/* Executa uma instrução preparada vinculando variáveis PHP */
$calories = 150;
$colour = 'red';
$name = 'apple';

$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < ? AND colour = ?');
$sth->bindParam(1, $calories, PDO::PARAM_INT);
$sth->bindValue(2, $colour, PDO::PARAM_STR);
$sth->execute();

$sth->debugDumpParams();

?>
```

O exemplo acima produzirá:

```
SQL: [82] SELECT name, colour, calories
    FROM fruit
    WHERE calories < ? AND colour = ?
Params:  2
```

```
Key: Position #0:
paramno=0
name=[0] ""
is_param=1
param_type=1
Key: Position #1:
paramno=1
name=[0] ""
is_param=1
param_type=2
```

Veja também

- PDO::prepare() - Prepara uma instrução para execução e retorna um objeto de instrução
- PDOStatement::bindParam() - Vincula um parâmetro ao nome de variável especificado
- PDOStatement::bindValue() - Vincula um valor a um parâmetro

PDOStatement::errorCode

PDOStatement::errorCode

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::errorCode — Busca o SQLSTATE associado com a última operação no objeto da instrução

Descrição

public function **PDOStatement::errorCode**(): ?string

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Idêntico a PDO::errorCode(), exceto que **PDOStatement::errorCode()** somente recupera códigos de erro para operações realizadas com objetos PDOStatement.

Examples

Exemplo #1 Recuperando o código SQLSTATE

```php
<?php
/* Provoca um erro -- a tabela OSSOS não existe */
$err = $dbh->prepare('SELECT cranio FROM ossos');
$err->execute();

echo "\nPDOStatement::errorCode(): ";
print $err->errorCode();
?>
```

O exemplo acima produzirá:

```
PDOStatement::errorCode(): 42S02
```

Veja também

- PDO::errorCode() - Busca o SQLSTATE associado com a última operação no identificador do banco de dados
- PDO::errorInfo() - Busca informação de erro estendida associada com a última operação no identificador do banco de dados
- PDOStatement::errorInfo() - Busca informação de erro estendida associada com a última operação no identificador da instrução

PDOStatement::errorInfo

PDOStatement::errorInfo

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::errorInfo — Busca informação de erro estendida associada com a última operação no identificador da instrução

Descrição

public function **PDOStatement::errorInfo**(): array

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

PDOStatement::errorInfo() retorna um array de informações de erro sobre a última operação realizada por este objeto de instrução. O array consiste de pelo menos os campos a seguir:

Elemento	Informação
0	Código de erro SQLSTATE (um identificador alfanumérico de cinco caracteres definido no padrão ANSI SQL).
1	Código de erro específico do driver.
2	Mensagem de erro específica do driver.

Examples

Exemplo #1 Mostrando campos de errorInfo() para uma conexão PDO_ODBC a um banco de dados DB2

```php
<?php
/* Provoca um erro -- a tabela OSSOS não existe */
$sth = $dbh->prepare('SELECT cranio FROM ossos');
$sth->execute();

echo "\nPDOStatement::errorInfo():\n";
$arr = $sth->errorInfo();
```

```
print_r($arr);
?>
```

O exemplo acima produzirá:

```
PDOStatement::errorInfo():
Array
(
    [0] => 42S02
    [1] => -204
    [2] => [IBM][CLI Driver][DB2/LINUX] SQL0204N  "LUCY.OSSOS" is an undefined name.  SQLSTATE=42704
)
```

Veja também

- PDO::errorCode() - Busca o SQLSTATE associado com a última operação no identificador do banco de dados
- PDO::errorInfo() - Busca informação de erro estendida associada com a última operação no identificador do banco de dados
- PDOStatement::errorCode() - Busca o SQLSTATE associado com a última operação no objeto da instrução

PDOStatement::execute

PDOStatement::execute

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::execute — Executa ums instrução preparada

Descrição

public function **PDOStatement::execute**(?array `$params = null`): bool

Executa a instrução preparada. Se a instrução preparada inclui marcadores de parâmetros:

- PDOStatement::bindParam() e/ou PDOStatement::bindValue() devem ser chamadas para vincular variáveis ou valores (respectivamente) aos marcadores de parâmetros. Variáveis vinculadas passam seus valores como entrada e recebem o valor de saída, se houver, de seus marcadores de parâmetros associados
- ou um array de valores de parâmetros, de entrada somente, deve ser passado

Parâmetros

`params`

Um array de valores com tantos elementos quantos forem os parâmetros vinculados na instrução SQL sendo executada. Todos os valores são tratados como **`PDO::PARAM_STR`**.

Valores múltiplos não podem ser vinculados a um parâmetro único; por exemplo, não é permitido vincular dois valores a um único parâmetro nomeado em uma cláusula IN().

Vincular mais valores que o especificado não é possível; se existirem mais chaves em `params` que no SQL especificado em PDO::prepare(), a instrução falhará e um erro será emitido.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Emite um erro de nível **`E_WARNING`** se o atributo **`PDO::ATTR_ERRMODE`** estiver definido como **`PDO::ERRMODE_WARNING`**.

Lança uma exceção PDOException se o atributo **`PDO::ATTR_ERRMODE`** estiver definido como **`PDO::ERRMODE_EXCEPTION`**.

Examples

Exemplo #1 Executa uma instrução preparada com variável e valor vinculados

```php
<?php
/* Executa uma instrução preparada vinculando uma variável e um valor */
$calorias = 150;
$cor = 'ver';
$sth = $dbh->prepare('SELECT nome, cor, calorias
    FROM frutas
    WHERE calorias < :calorias AND cor LIKE :cor');
$sth->bindParam('calorias', $calorias, PDO::PARAM_INT);
/* Nomes podem ser prefixados com dois-pontos ":" também (opcional) */
$sth->bindValue(':cor', "%$cor%");
$sth->execute();
?>
```

Exemplo #2 Executa uma instrução preparada com um array de valores nomeados

```php
<?php
/* Executa uma instrução preparada passando um array de valores de inserção */
$calorias = 150;
$cor = 'vermelha';
$sth = $dbh->prepare('SELECT nome, cor, calorias
    FROM frutas
    WHERE calorias < :calorias AND cor = :cor');
$sth->execute(array('calorias' => $calorias, 'cor' => $cor));
/* Chaves de array podem ser prefixadas com dois-pontos ":" também (opcional) */
$sth->execute(array(':calorias' => $calorias, ':cor' => $cor));
?>
```

Exemplo #3 Executa uma instrução preparada com um array de valores posicionais

```php
<?php
/* Executa uma instrução preparada passando um array de valores de inserção */
$calorias = 150;
$cor = 'vermelha';
$sth = $dbh->prepare('SELECT nome, cor, calorias
    FROM frutas
    WHERE calorias < ? AND cor = ?');
$sth->execute(array($calorias, $cor));
?>
```

Exemplo #4 Executa uma instrução preparada com variáveis vinculadas a marcadores posicionais

```php
<?php
/* Executa uma instrução preparada vinculando variáveis PHP */
$calorias = 150;
$cor = 'vermelha';
$sth = $dbh->prepare('SELECT nome, cor, calorias
    FROM frutas
    WHERE calorias < ? AND cor = ?');
$sth->bindParam(1, $calorias, PDO::PARAM_INT);
$sth->bindParam(2, $cor, PDO::PARAM_STR, 12);
$sth->execute();
?>
```

Exemplo #5 Executa uma instrução preparada usando array para cláusula IN

```php
<?php
/* Executa uma instrução preparada usando um array de valores para uma cláusula IN */
$params = array(1, 21, 63, 171);
/* Cria uma string para marcadores de parâmetros preenchidas com o número de parâmetros */
$place_holders = '?' . str_repeat(', ?', count($params) - 1);

/*
    Isto prepara a instrução com marcadores anônimos suficientes para cada valor
    no array $params. Os valores do array $params são então vinculados aos
    marcadores na instrução preparada quando ele é executada.
    Isto não é a mesma coisa que usar PDOStatement::bindParam() já que isso
    requer uma referência à variável. PDOStatement::execute() apenas vincula
    por valor.
*/
$sth = $dbh->prepare("SELECT id, nome FROM contatos WHERE id IN ($place_holders)");
$sth->execute($params);
?>
```

Notas

> **Nota** :
>
> Alguns drivers requerem que o cursor seja fechado antes da execução da próxima instrução.

Veja também

- PDO::prepare() - Prepara uma instrução para execução e retorna um objeto de instrução
- PDOStatement::bindParam() - Vincula um parâmetro ao nome de variável especificado
- PDOStatement::fetch() - Busca a próxima linha de um resultado
- PDOStatement::fetchAll() - Busca as linhas remanescentes de um conjunto de resultados
- PDOStatement::fetchColumn() - Retorna uma única coluna do próximo registro de um resultado

PDOStatement::fetch

PDOStatement::fetch

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::fetch — Busca a próxima linha de um resultado

Descrição

public function **PDOStatement::fetch**(int `$mode` = **`PDO::FETCH_DEFAULT`**, int `$cursorOrientation` = **`PDO::FETCH_ORI_NEXT`**, int `$cursorOffset` = 0): mixed

Busca uma linha de um resultado associado com um objeto PDOStatement object. O parâmetro `mode` determina como o PDO retorna a linha.

Parâmetros

`mode`

Controla como a próxima linha será retornada ao chamador. Este valor precisa ser uma das constantes `PDO::FETCH_*`, padronizado para o valor de `PDO::ATTR_DEFAULT_FETCH_MODE` (cujo padrão é `PDO::FETCH_BOTH`).

- `PDO::FETCH_ASSOC`: retorna um array indexado pelo nome da coluna como retornada no resultado
- `PDO::FETCH_BOTH` (padrão): retorna um array indexado tanto por nome de coluna quanto por número de coluna indexado em 0, como retornado no resultado
- `PDO::FETCH_BOUND`: retorna **`true`** e atribui os valores das colunas no conjunto de resultados às variáveis PHP às quais eles foram vinculados com o método PDOStatement::bindColumn()
- `PDO::FETCH_CLASS`: retorna uma nova instância da classe solicitada. O objeto é inicializado mapeando as colunas do conjunto de resultados para propriedades na classe. Isso ocorre antes do construtor ser chamado, permitindo que as propriedades sejam preenchidas independentemente de sua visibilidade ou se estão marcadas como `readonly`. Se uma propriedade não existir na classe, o método mágico __set() será invocado se existir; caso contrário, será criada uma propriedade pública dinâmica. Entretanto, quando **`PDO::FETCH_PROPS_LATE`** também é fornecido, o construtor é chamado *antes* de as propriedades serem preenchidas. Se `mode` incluir **`PDO::FETCH_CLASSTYPE`** (por exemplo, `PDO::FETCH_CLASS | PDO::FETCH_CLASSTYPE`), o nome da classe é determinado a partir do valor da primeira coluna.
- `PDO::FETCH_INTO`: atualiza uma instância existente da classe solicitada, mapeando as colunas do conjunto de resultados às propriedades nomeadas na classe
- `PDO::FETCH_LAZY`: combina `PDO::FETCH_BOTH` e `PDO::FETCH_OBJ`, e retorna um objeto PDORow que cria os nomes das propriedades do objeto conforme elas são acessadas.
- `PDO::FETCH_NAMED`: retorna um array com a mesma forma de `PDO::FETCH_ASSOC`, exceto que se houver múltiplas colunas com o mesmo nome, o valor referenciado pela chave será um array de todos os valores na linha que tiverem esse nome de coluna
- `PDO::FETCH_NUM`: retorna um array indexado por número de coluna como retornado no conjunto de resultados, iniciando na coluna 0
- `PDO::FETCH_OBJ`: retorna um objeto anônimo com nomes de propriedades que correspondem ao nomes de colunas retornados no conjunto de resultados
- `PDO::FETCH_PROPS_LATE`: quando usado com `PDO::FETCH_CLASS`, o construtor da classe é chamado antes que as propriedades sejam atribuídas com os respectivos valores das colunas.

`cursorOrientation`

Para um objeto PDOStatement representando um cursor navegável, este valor determina que linha será retornada ao chamador. Este valor precisa ser uma das constantes `PDO::FETCH_ORI_*`, sendo o padrão igual

a `PDO::FETCH_ORI_NEXT`. Para requisitar um cursor navegável para o objeto PDOStatement, o atributo `PDO::ATTR_CURSOR` precisa ser definido para `PDO::CURSOR_SCROLL` ao preparar a instrução SQL com PDO::prepare().

`cursorOffset`

Para um objeto PDOStatement representando um cursor navegável para o qual o parâmetro `cursorOrientation` esteja definido para `PDO::FETCH_ORI_ABS`, este valor especifica o número absoluto da linha no conjunto de resultados que deve ser buscada.

Para um objeto PDOStatement representando um cursor navegável para o qual o parâmetro `cursorOrientation` esteja definido para `PDO::FETCH_ORI_REL`, este valor especifica a linha a ser buscada relativa à posição do cursor antes do método **PDOStatement::fetch()** ser chamado.

Valor Retornado

O valor de retorno desta função em caso de sucesso depende do tipo de busca. Em todos os casos, `false` é retornado em caso de falha ou se não houver mais linhas.

Erros/Exceções

Emite um erro de nível `E_WARNING` se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_WARNING`.

Lança uma exceção PDOException se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_EXCEPTION`.

Examples

Exemplo #1 Buscando linhas usando estilos de busca diferentes

```php
<?php
$sth = $dbh->prepare("SELECT name, colour FROM fruit");
$sth->execute();

/* Exercita estilos de PDOStatement::fetch */
print "PDO::FETCH_ASSOC: ";
print "Retorna a próxima linha como um array indexado por nome de coluna\n";
$result = $sth->fetch(PDO::FETCH_ASSOC);
print_r($result);
print "\n";

print "PDO::FETCH_BOTH: ";
print "Retorna a próxima linha como um array indexado por nome de coluna e número\n";
$result = $sth->fetch(PDO::FETCH_BOTH);
print_r($result);
print "\n";

print "PDO::FETCH_LAZY: ";
print "Retorna a próxima linha como um objeto PDORow com nomes de colunas como propriedades\n";
$result = $sth->fetch(PDO::FETCH_LAZY);
print_r($result);
print "\n";

print "PDO::FETCH_OBJ: ";
print "Retorna a próxima linha como um objeto anônimo com nomes de coluna como propriedades\n";
$result = $sth->fetch(PDO::FETCH_OBJ);
print $result->name;
```

```
print "\n";
?>
```

O exemplo acima produzirá:

```
PDO::FETCH_ASSOC: Retorna a próxima linha como um array indexado por nome de coluna
Array
(
    [name] => apple
    [colour] => red
)

PDO::FETCH_BOTH: Retorna a próxima linha como um array indexado por nome de coluna e número
Array
(
    [name] => banana
    [0] => banana
    [colour] => yellow
    [1] => yellow
)

PDO::FETCH_LAZY: Retorna a próxima linha como um objeto PDORow com nomes de colunas como
propriedades
PDORow Object
(
    [name] => orange
    [colour] => orange
)

PDO::FETCH_OBJ: Retorna a próxima linha como um objeto anônimo com nomes de coluna como propriedades
kiwi
```

Exemplo #2 Buscando linhas com um cursor navegável

```
<?php
function readDataForwards($dbh) {
    $sql = 'SELECT hand, won, bet FROM mynumbers ORDER BY BET';
    $stmt = $dbh->prepare($sql, array(PDO::ATTR_CURSOR => PDO::CURSOR_SCROLL));
    $stmt->execute();
    while ($row = $stmt->fetch(PDO::FETCH_NUM, PDO::FETCH_ORI_NEXT)) {
        $data = $row[0] . "\t" . $row[1] . "\t" . $row[2] . "\n";
        print $data;
    }
}
function readDataBackwards($dbh) {
    $sql = 'SELECT hand, won, bet FROM mynumbers ORDER BY bet';
    $stmt = $dbh->prepare($sql, array(PDO::ATTR_CURSOR => PDO::CURSOR_SCROLL));
    $stmt->execute();
    $row = $stmt->fetch(PDO::FETCH_NUM, PDO::FETCH_ORI_LAST);
    do {
        $data = $row[0] . "\t" . $row[1] . "\t" . $row[2] . "\n";
        print $data;
```

```
    } while ($row = $stmt->fetch(PDO::FETCH_NUM, PDO::FETCH_ORI_PRIOR));
}

print "Lendo para a frente:\n";
readDataForwards($conn);

print "Lendo para trás:\n";
readDataBackwards($conn);
?>
```

O exemplo acima produzirá:

```
Reading forwards:
21    10    5
16    0     5
19    20    10

Reading backwards:
19    20    10
16    0     5
21    10    5
```

Exemplo #3 Ordem de construção

Quando objetos são buscados através de `PDO::FETCH_CLASS`, as propriedades do objeto são atribuídas primeiro, e depois o construtor da classe é invocado. Entretanto, quando `PDO::FETCH_PROPS_LATE` também for informada, esta ordem é revertida, isto, é, primeiro o construtor é chamado, e depois as propriedades são atribuídas.

```
<?php
class Person
{
    private $name;

    public function __construct()
    {
        $this->tell();
    }

    public function tell()
    {
        if (isset($this->name)) {
            echo "Eu sou {$this->name}.\n";
        } else {
            echo "Eu não tenho um nome ainda.\n";
        }
    }
}

$sth = $dbh->query("SELECT * FROM people");
$sth->setFetchMode(PDO::FETCH_CLASS, 'Person');
$person = $sth->fetch();
$person->tell();
```

```php
$sth->setFetchMode(PDO::FETCH_CLASS|PDO::FETCH_PROPS_LATE, 'Person');
$person = $sth->fetch();
$person->tell();
?>
```

O exemplo acima produzirá algo semelhante a:

```
Eu sou Alice.
Eu sou Alice.
Eu não tenho um nome ainda.
Eu sou Bob.
```

Veja também

- PDO::prepare() - Prepara uma instrução para execução e retorna um objeto de instrução
- PDOStatement::execute() - Executa ums instrução preparada
- PDOStatement::fetchAll() - Busca as linhas remanescentes de um conjunto de resultados
- PDOStatement::fetchColumn() - Retorna uma única coluna do próximo registro de um resultado
- PDOStatement::fetchObject() - Busca o próximo registro e retorna-o como um objeto
- PDOStatement::setFetchMode() - Define o modo de carga de dados para esta instrução

PDOStatement::fetchAll

PDOStatement::fetchAll

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::fetchAll — Busca as linhas remanescentes de um conjunto de resultados

Descrição

public function **PDOStatement::fetchAll**(int `$mode` = PDO::FETCH_DEFAULT): array
public function **PDOStatement::fetchAll**(int `$mode` = PDO::FETCH_COLUMN, int `$column`): array
public function **PDOStatement::fetchAll**(int `$mode` = PDO::FETCH_CLASS, string `$class`, ?array `$constructorArgs`): array
public function **PDOStatement::fetchAll**(int `$mode` = PDO::FETCH_FUNC, callable `$callback`): array

Parâmetros

`mode`

> Controla o conteúdo do array retornado como documentado em PDOStatement::fetch(). O padrão é **`PDO::ATTR_DEFAULT_FETCH_MODE`** (cujo padrão é **`PDO::FETCH_BOTH`**).
>
> Para retornar um array que consiste em todos os valores de uma única coluna do conjunto de resultados, especifique **`PDO::FETCH_COLUMN`**. Pode-se especificar uma coluna qualquer com o parâmetro `column`.
>
> Para indexar o array resultante pelo valor de uma determinada coluna (em vez de números consecutivos), coloque o nome desta coluna primeiro na lista de colunas no SQL e use **`PDO::FETCH_UNIQUE`**. Esta coluna deve conter apenas valores exclusivos ou alguns dados serão perdidos.

Para agrupar resultados na forma de um array tridimensional indexado por valores de uma coluna especificada, coloque o nome desta coluna primeiro na lista de colunas no SQL e use `PDO::FETCH_GROUP`.

Para agrupar resultados na forma de um array bidimensional, use a operação OR binária com `PDO::FETCH_GROUP` e `PDO::FETCH_COLUMN`. Os resultados serão agrupados pela primeira coluna, com o valor do elemento do array sendo um array de lista das entradas correspondentes da segunda coluna.

A lista a seguir apresenta parâmetros dinâmicos que dependem do modo de busca. Eles não podem ser usados com parâmetros nomeados.

`column`

Usado com `PDO::FETCH_COLUMN`. Retorna a coluna indicada com índice iniciando em 0.

`class`

Usado com `PDO::FETCH_CLASS`. Retorna instâncias da classe especificada, mapeando colunas de cada linha a propriedades nomeadas na classe.

`constructorArgs`

Argumentos de construtor de classe customizado quando o parâmetro `mode` for `PDO::FETCH_CLASS`.

`callback`

Usado com `PDO::FETCH_FUNC`. Retorna os resultados da chamada à função especificada, usando cada coluna do registro como parâmetro na chamada.

Valor Retornado

PDOStatement::fetchAll() retorna um array contendo todas os registros remanescentes no conjunto de resultados O array representa cada registro como um array de valores de coluna ou um objeto com propriedades correspondentes a cada nome de coluna. Um array vazio é retornado se não houver resultados na busca.

Usar este método para burcar conjuntos de resultados grandes resultará em uma alta demanda no sistema e possivelmente em recursos de rede. Ao invés de recuperar todos os dados e manipulá-los no PHP, deve ser considerado usar o servidor de banco de dados para manipular os conjuntos de resultados. Por exemplo, usando as cláusulas WHERE e ORDER BY no SQL para restringir resultados antes de recuperá-los e processá-los com o PHP.

Erros/Exceções

Emite um erro de nível `E_WARNING` se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_WARNING`.

Lança uma exceção PDOException se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_EXCEPTION`.

Registro de Alterações

Versão	Descrição
8.0.0	Este método agora sempre retornará um array, enquanto antes poderia retornar `false` em caso de falha.

Examples

Exemplo #1 Buscando todos os registros remanescentes em um conjunto de resultados

```php
<?php
$sth = $dbh->prepare("SELECT name, colour FROM fruit");
$sth->execute();

/* Busca todos os registros remanescentes em um conjunto de resultados */
print "Busca todos os registros remanescentes em um conjunto de resultados:\n";
$result = $sth->fetchAll();
print_r($result);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Busca todos os registros remanescentes em um conjunto de resultados:
Array
(
    [0] => Array
        (
            [name] => apple
            [0] => apple
            [colour] => red
            [1] => red
        )

    [1] => Array
        (
            [name] => pear
            [0] => pear
            [colour] => green
            [1] => green
        )

    [2] => Array
        (
            [name] => watermelon
            [0] => watermelon
            [colour] => pink
            [1] => pink
        )

)
```

Exemplo #2 Buscando todos os valores de uma única coluna de um resultado

O exemplo a seguir demonstra como retornar todos os valores de uma única coluna de um resultado, mesmo que a instrução SQL em si possa retornar múltiplas colunas por registro.

```php
<?php
$sth = $dbh->prepare("SELECT name, colour FROM fruit");
$sth->execute();

/* Busca todos os valores da primeira coluna */
$result = $sth->fetchAll(PDO::FETCH_COLUMN, 0);
```

```
var_dump($result);
?>
```

O exemplo acima produzirá algo semelhante a:

```
Array(3)
(
    [0] =>
    string(5) => apple
    [1] =>
    string(4) => pear
    [2] =>
    string(10) => watermelon
)
```

Exemplo #3 Agrupando todos os valores por uma única coluna

O exemplo a seguir demonstra como retornar um array associativo agrupado pelos valores da coluna especificada no resultado. O array contém três chaves: valores `apple` e `pear` são retornados como arrays que contêm duas cores diferentes, enquanto `watermelon` é retornado como um array qye contém apenas uma cor.

```
<?php
$insert = $dbh->prepare("INSERT INTO fruit(name, colour) VALUES (?, ?)");
$insert->execute(array('apple', 'green'));
$insert->execute(array('pear', 'yellow'));

$sth = $dbh->prepare("SELECT name, colour FROM fruit");
$sth->execute();

/* Agrupa valores pela primeira coluna */
var_dump($sth->fetchAll(PDO::FETCH_COLUMN|PDO::FETCH_GROUP));
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(3) {
  ["apple"]=>
  array(2) {
    [0]=>
    string(5) "green"
    [1]=>
    string(3) "red"
  }
  ["pear"]=>
  array(2) {
    [0]=>
    string(5) "green"
    [1]=>
    string(6) "yellow"
```

```
  }
  ["watermelon"]=>
  array(1) {
    [0]=>
    string(5) "pink"
  }
}
```

Exemplo #4 Instanciando uma classe para cada resultado

O exemplo a seguir demonstra o comportamento do estilo de busca **PDO::FETCH_CLASS**.

```php
<?php
class fruit {
    public $name;
    public $colour;
}

$sth = $dbh->prepare("SELECT name, colour FROM fruit");
$sth->execute();

$result = $sth->fetchAll(PDO::FETCH_CLASS, "fruit");
var_dump($result);
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(3) {
  [0]=>
  object(fruit)#1 (2) {
    ["name"]=>
    string(5) "apple"
    ["colour"]=>
    string(5) "green"
  }
  [1]=>
  object(fruit)#2 (2) {
    ["name"]=>
    string(4) "pear"
    ["colour"]=>
    string(6) "yellow"
  }
  [2]=>
  object(fruit)#3 (2) {
    ["name"]=>
    string(10) "watermelon"
    ["colour"]=>
    string(4) "pink"
  }
  [3]=>
  object(fruit)#4 (2) {
    ["name"]=>
```

```
    string(5) "apple"
    ["colour"]=>
    string(3) "red"
  }
  [4]=>
  object(fruit)#5 (2) {
    ["name"]=>
    string(4) "pear"
    ["colour"]=>
    string(5) "green"
  }
}
```

Exemplo #5 Chamando uma função para cada resultado

O exemplo a seguir demonstra o comportamento do estilo de busca `PDO::FETCH_FUNC`.

```php
<?php
function fruit($name, $colour) {
    return "{$name}: {$colour}";
}

$sth = $dbh->prepare("SELECT name, colour FROM fruit");
$sth->execute();

$result = $sth->fetchAll(PDO::FETCH_FUNC, "fruit");
var_dump($result);
?>
```

O exemplo acima produzirá algo semelhante a:

```
array(3) {
  [0]=>
  string(12) "apple: green"
  [1]=>
  string(12) "pear: yellow"
  [2]=>
  string(16) "watermelon: pink"
  [3]=>
  string(10) "apple: red"
  [4]=>
  string(11) "pear: green"
}
```

Veja também

- PDO::query() - Prepara e executa uma instrução SQL sem marcadores
- PDOStatement::fetch() - Busca a próxima linha de um resultado

- PDOStatement::fetchColumn() - Retorna uma única coluna do próximo registro de um resultado
- PDO::prepare() - Prepara uma instrução para execução e retorna um objeto de instrução
- PDOStatement::setFetchMode() - Define o modo de carga de dados para esta instrução

PDOStatement::fetchColumn

PDOStatement::fetchColumn

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.9.0)

PDOStatement::fetchColumn — Retorna uma única coluna do próximo registro de um resultado

Descrição

public function **PDOStatement::fetchColumn**(int `$column` = 0): mixed

Retorna uma única coluna do próximo registro de um resultado ou `false` se não houver mais registros.

> **Nota** :
>
> **PDOStatement::fetchColumn()** não deve ser usado para colunas do tipo booleano, já que é impossível distinguir um valor de `false` em relação à inexistência de mais registros a se recuperar. Use PDOStatement::fetch() em seu lugar.

Parâmetros

`column`

Número da coluna que se deseja recuperar do registro, com índice iniciando em 0. Se nenhum valor for informado, **PDOStatement::fetchColumn()** busca a primeira coluna.

Valor Retornado

PDOStatement::fetchColumn() retorna uma única coluna do próximo registro do resultado ou `false` se não houver mais registros.

> **Aviso**
>
> Não há como retornar outra coluna do mesmo registro se o método **PDOStatement::fetchColumn()** for usado para recuperar dados.

Erros/Exceções

Emite um erro de nível `E_WARNING` se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_WARNING`.

Lança uma exceção PDOException se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_EXCEPTION`.

Examples

Exemplo #1 Retorna a primeira coluna do próximo registro

```
<?php
$sth = $dbh->prepare("SELECT name, colour FROM fruit");
```

```php
$sth->execute();

print "Busca a primeira coluna da primeira linha do resultado:\n";
$result = $sth->fetchColumn();
print "name = $result\n";

print "Busca a segunda coluna da segunda linha do resultado:\n";
$result = $sth->fetchColumn(1);
print "colour = $result\n";
?>
```

O exemplo acima produzirá:

```
Busca a primeira coluna da primeira linha do resultado:
name = lemon
Busca a segunda coluna da segunda linha do resultado:
colour = red
```

Veja também

- PDO::query() - Prepara e executa uma instrução SQL sem marcadores
- PDOStatement::fetch() - Busca a próxima linha de um resultado
- PDOStatement::fetchAll() - Busca as linhas remanescentes de um conjunto de resultados
- PDO::prepare() - Prepara uma instrução para execução e retorna um objeto de instrução
- PDOStatement::setFetchMode() - Define o modo de carga de dados para esta instrução

PDOStatement::fetchObject

PDOStatement::fetchObject

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.4)

PDOStatement::fetchObject — Busca o próximo registro e retorna-o como um objeto

Descrição

public function **PDOStatement::fetchObject**(?string `$class` = "stdClass", array `$constructorArgs` = []): object|false

Busca o próximo registro e retorna-o como um objeto. Esta função é uma alternativa a PDOStatement::fetch() com estilo **`PDO::FETCH_CLASS`** ou **`PDO::FETCH_OBJ`**.

Quando um objeto é buscado, suas propriedades são atribuídas a partir dos respectivos valores de colunas, e depois seu construtor é invocado.

Parâmetros

`class`

Nome da classe a ter um objeto criado.

`constructorArgs`

Elementos deste array são passados para o construtor.

Valor Retornado

Retorna uma instância da classe solicitada com nomes de propriedades que correspondem aos nomes de colunas ou `false` em caso de falha.

Erros/Exceções

Emite um erro de nível `E_WARNING` se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_WARNING`.

Lança uma exceção PDOException se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_EXCEPTION`.

Veja também

- PDOStatement::fetch() - Busca a próxima linha de um resultado

PDOStatement::getAttribute

PDOStatement::getAttribute

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDOStatement::getAttribute — Recupera um atributo da instrução

Descrição

public function **PDOStatement::getAttribute**(int `$name`): mixed

Obtém um atributo da instrução. Atualmente, não existem atributos genéricos mas apenas específicos de drivers:

- `PDO::ATTR_CURSOR_NAME` (específico do Firebird e do ODBC): Obtém o nome do cursor para `UPDATE ... WHERE CURRENT OF`.
- `Pdo\Pgsql::ATTR_RESULT_MEMORY_SIZE` (específico do PostgreSQL, disponível a partir do PHP 8.4.0): Obtém a quantidade de memória, em bytes, alocada para o resultado da consulta.

Observe que atributos específicos de drivers *não podem* ser usados com outros drivers.

Parâmetros

`name`

O atributo a ser consultado.

Valor Retornado

Retorna o valor do atributo.

Veja também

- PDO::getAttribute() - Recuperar um atributo da conexão com o banco de dados
- PDO::setAttribute() - Define um atributo
- PDOStatement::setAttribute() - Define um atributo na instrução

PDOStatement::getColumnMeta

PDOStatement::getColumnMeta

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDOStatement::getColumnMeta — Retorna metadados para uma coluna no resultado

Descrição

public function **PDOStatement::getColumnMeta**(int `$column`): array|false

Retorna metadados para uma coluna no resultado, indexada a partir de 0, como um array associativo.

> **Aviso**
>
> Alguns drivers podem não implementar o método **PDOStatement::getColumnMeta()**, já que ele é opcional. Entretanto, todos os drivers PDO documentados neste manual implementam este método.

Parâmetros

`column`

A coluna no resultado, indexada a partir de 0.

Valor Retornado

Retorna um array associativo contendo os seguintes valores que representam os metadados para uma única coluna:

Metadados de coluna

Nome	Valor
`native_type`	O tipo de dados nativo do PHP usado para representar o valor da coluna.
`driver:decl_type`	O tipo de dados SQL usado para representar o valor da coluna no banco de dados. Se a coluna for resultado de uma função, este valor não será retornado por **PDOStatement::getColumnMeta()**.
`flags`	Quaisquer opções configuradas para esta coluna.
`name`	O nome desta coluna como retornado pelo banco de dados.
`table`	O nome da tabela desta coluna como retornado pelo banco de dados.
`len`	O comprimento desta coluna. Normalmente `-1` para tipos que não sejam números com decimais.
`precision`	A precisão numérica desta coluna. Normalmente `0` para tipos que não sejam números decimais.
`pdo_type`	O tipo desta coluna como representado pelas constantes `PDO::PARAM_*`.

Retorna `false` se a coluna requisitada não existir no resultado ou se não existir resultado.

Examples

Exemplo #1 Recuperando metadados de coluna

O exemplo a seguir mostra o resultado da recuperação de metadados para uma única coluna gerada por uma função (COUNT) em um driver PDO_SQLITE.

```php
<?php
$select = $DB->query('SELECT COUNT(*) FROM fruit');
$meta = $select->getColumnMeta(0);
var_dump($meta);
?>
```

O exemplo acima produzirá:

```
array(6) {
  ["native_type"]=>
  string(7) "integer"
  ["flags"]=>
  array(0) {
  }
  ["name"]=>
  string(8) "COUNT(*)"
  ["len"]=>
  int(-1)
  ["precision"]=>
  int(0)
  ["pdo_type"]=>
  int(2)
}
```

Veja também

- PDOStatement::columnCount() - Retorna o número de colunas no conjunto de resultados
- PDOStatement::rowCount() - Retorna o número de registros afetados pela última instrução SQL

PDOStatement::getIterator

PDOStatement::getIterator

(PHP 8)

PDOStatement::getIterator — Obtém o iterador do conjunto de resultados

Descrição

public function **PDOStatement::getIterator**(): Iterator

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

PDOStatement::nextRowset

PDOStatement::nextRowset

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDOStatement::nextRowset — Avança para o próximo conjunto de registros em um identificados de instrução com múltiplos conjuntos de registros

Descrição

public function **PDOStatement::nextRowset**(): bool

Alguns servidores de bancos de dados suportam procedimentos armazenados que retornam mais de um conjunto de registros (também conhecido como conjunto de resultados). **PDOStatement::nextRowset()** habilita o acesso ao segundo conjunto de registros, e subsequentes, associados com um objeto PDOStatement. Cada conjunto de registros pode ter um conjunto diferente de colunas em relação ao conjunto anterior.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Buscando múltiplos conjuntos de registros de um procedimento armazenado

O exemplo a seguir mostra como chamar um procedimento armazenado, `MULTIPLE_ROWSETS`, que retorna três conjuntos de registros. É usado um laço do-while para chamar o método **PDOStatement::nextRowset()** até que ele retorne `false` e termine o laço quando não houver mais conjuntos de registros disponíveis.

```
<?php
$sql = 'CALL multiple_rowsets()';
$stmt = $conn->query($sql);
$i = 1;
do {
    $rowset = $stmt->fetchAll(PDO::FETCH_NUM);
    if ($rowset) {
        printResultSet($rowset, $i);
    }
    $i++;
} while ($stmt->nextRowset());

function printResultSet(&$rowset, $i) {
    print "Conjunto de resultados $i:\n";
    foreach ($rowset as $row) {
        foreach ($row as $col) {
            print $col . "\t";
        }
        print "\n";
    }
    print "\n";
}
?>
```

O exemplo acima produzirá:

```
Conjunto de resultados 1:
apple    red
banana   yellow

Conjunto de resultados 2:
orange   orange    150
banana   yellow    175

Conjunto de resultados 3:
lime     green
apple    red
banana   yellow
```

Veja também

- PDOStatement::columnCount() - Retorna o número de colunas no conjunto de resultados
- PDOStatement::execute() - Executa ums instrução preparada
- PDOStatement::getColumnMeta() - Retorna metadados para uma coluna no resultado
- PDO::query() - Prepara e executa uma instrução SQL sem marcadores

PDOStatement::rowCount

PDOStatement::rowCount

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::rowCount — Retorna o número de registros afetados pela última instrução SQL

Descrição

public function **PDOStatement::rowCount**(): int

PDOStatement::rowCount() retorna o número de registros afetados pela última instrução DELETE, INSERT ou UPDATE executada pelo objeto `PDOStatement` correspondente.

Para instruções que produzem conjuntos de resultados, como `SELECT`, o comportamento é indefinido e pode ser diferente para cada driver. Alguns bancos de dados podem retornar o número de registros produzidos por essa instrução (por exemplo, MySQL em modo buffer), mas este comportamento não é garantido para todos os bancos de dados e não deve ser uma fonte confiável para aplicações portáveis.

> **Nota** :
>
> Este método retorna "0" (zero) com o driver PostgreSQL ao definir o atributo de instrução **`PDO::ATTR_CURSOR`** para **`PDO::CURSOR_SCROLL`**.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o número de registros.

Erros/Exceções

Emite um erro de nível `E_WARNING` se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_WARNING`.

Lança uma exceção PDOException se o atributo `PDO::ATTR_ERRMODE` estiver definido como `PDO::ERRMODE_EXCEPTION`.

Examples

Exemplo #1 Retorna o número de registros apagados

PDOStatement::rowCount() retorna o número de registros afetados pelas instruções DELETE, INSERT ou UPDATE.

```php
<?php
/* Apaga todos os registros da tabela FRUIT */
$del = $dbh->prepare('DELETE FROM fruit');
$del->execute();

/* Retorna o número de registros apagados */
print "Número retornado de registros apagados:\n";
$count = $del->rowCount();
print "$count registros apagados.\n";
?>
```

O exemplo acima produzirá algo semelhante a:

```
Número retornado de registros apagados:
9 registros apagados.
```

Exemplo #2 Contando registros retornados por uma instrução SELECT

Para a maioria dos bancos de dados, o método **PDOStatement::rowCount()** não retorna o número de registros afetados por uma instrução SELECT. Ao invés dele, o método PDO::query() deve ser usado para emitir uma instrução SELECT COUNT(*) com os mesmos predicados da instrução SELECT requerida, e então PDOStatement::fetchColumn() deve ser usado para recuperar o número de registros correspondentes.

```php
<?php
$sql = "SELECT COUNT(*) FROM fruit WHERE calories > 100";
$res = $conn->query($sql);
$count = $res->fetchColumn();

print "Há " .  $count . " registros correspondentes.";
```

O exemplo acima produzirá algo semelhante a:

```
Há 2 registros correspondentes.
```

Veja também

- PDOStatement::columnCount() - Retorna o número de colunas no conjunto de resultados
- PDOStatement::fetchColumn() - Retorna uma única coluna do próximo registro de um resultado
- PDO::query() - Prepara e executa uma instrução SQL sem marcadores

PDOStatement::setAttribute

PDOStatement::setAttribute

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDOStatement::setAttribute — Define um atributo na instrução

Descrição

public function **PDOStatement::setAttribute**(int `$attribute`, mixed `$value`): bool

Define um atributo na instrução. Atualmente não há atributos genericos definidos, apenas atributos específicos do driver:

- `PDO::ATTR_CURSOR_NAME` (Específico do Firebird e do ODBC): Define o nome do cursor para `UPDATE ... WHERE CURRENT OF`.

Note que atributos específicos de drivers *não devem* ser usados com outros drivers.

Parâmetros

`attribute`

O atibuto a modificar.

`value`

O valor para definir o `attribute`, pode requerer um tipo específico dependendo do atributo.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Veja também

- PDO::getAttribute() - Recuperar um atributo da conexão com o banco de dados
- PDO::setAttribute() - Define um atributo
- PDOStatement::getAttribute() - Recupera um atributo da instrução

PDOStatement::setFetchMode

PDOStatement::setFetchMode

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDOStatement::setFetchMode — Define o modo de carga de dados para esta instrução

Descrição

public function **PDOStatement::setFetchMode**(int `$mode`): true
public function **PDOStatement::setFetchMode**(int `$mode` = PDO::FETCH_COLUMN, int `$colno`): true
public function **PDOStatement::setFetchMode**(int `$mode` = PDO::FETCH_CLASS, string `$class`, ?array `$constructorArgs` = **`null`**): true
public function **PDOStatement::setFetchMode**(int `$mode` = PDO::FETCH_INTO, object `$object`): true

Parâmetros

`mode`

O modo de carga precisa ser uma das constantes **`PDO::FETCH_*`**.

`colno`

Número da coluna.

`class`

Nome da classe.

`constructorArgs`

Argumentos do construtor.

`object`

Objeto.

Valor Retornado

Sempre retorna **`true`**.

Registro de Alterações

Versão	Descrição
8.4.0	O tipo do retorno agora é **`true`**; anteriormente, era bool.

Examples

Exemplo #1 Definindo um modo de carga

O seguinte exemplo demonstra como o **PDOStatement::setFetchMode()** altera o modo padrão de carga em um objeto PDOStatement.

```php
<?php
$stmt = $dbh->query('SELECT name, colour, calories FROM fruit');
$stmt->setFetchMode(PDO::FETCH_NUM);
foreach ($stmt as $row) {
    print $row[0] . "\t" . $row[1] . "\t" . $row[2] . "\n";
}
```

O exemplo acima produzirá algo semelhante a:

```
apple   red     150
banana  yellow  250
orange  orange  300
kiwi    brown   75
lemon   yellow  25
pear    green   150
```

PDORow

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 1.0.0)

Introdução

Representa uma linha de um conjunto de resultados retornado pelo PDOStatement::fetch() chamado com o modo de recuperação `PDO::FETCH_LAZY`.

Objetos desta classe não podem ser instanciados e não são serializáveis.

O objeto **PDORow** permite acesso aos dados retornados como se tanto o modo `PDO::FETCH_OBJ` quanto o modo `PDO::FETCH_BOTH` fossem usados. Isso significa que os dados retornados podem ser acessados como propriedades de objeto, e como um array indexado tanto pelo nome da coluna quanto pelo número de deslocamento da coluna.

> **Cuidado**
>
> Acesso a uma propriedade indefinida retorna `null` sem emitir um aviso.

Resumo da classe

final class **PDORow** {
/* Propriedades */
public string *$queryString*;
}

Propriedades

queryString

String de consulta usada pelo PDOStatement que retornou o objeto **PDORow**.

Erros/Exceções

Gera um Error ao tentar escrever ou unset() em qualquer propriedade.

PDOException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introdução

Representa um erro levantado pelo PDO. Não se deve lançar uma **PDOException** de dentro do código do usuário. Veja Exceções para mais informações sobre Exceções em PHP.

Resumo da classe

class **PDOException** extends RuntimeException {
/* Propriedades */
protected int|string *$code*;
public ?array *$errorInfo* = null;
/* Propriedades herdadas */
protected string *$message* = "";
private string *$string* = "";
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Métodos herdados */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

Propriedades

errorInfo

Corresponde a PDO::errorInfo() ou PDOStatement::errorInfo()

code

Código de erro `SQLSTATE`. Use Exception::getCode() para acessá-lo.

Drivers do PDO

Índice

Os seguintes drivers atualmente implementam a interface PDO:

Nome do Driver	Bancos de Dados Suportados
PDO_CUBRID	Cubrid
PDO_DBLIB	FreeTDS / Microsoft SQL Server / Sybase
PDO_FIREBIRD	Firebird
PDO_IBM	IBM DB2

Nome do Driver	Bancos de Dados Suportados
PDO_INFORMIX	IBM Informix Dynamic Server
PDO_MYSQL	MySQL 3.x/4.x/5.x/8.x
PDO_OCI	Oracle Call Interface
PDO_ODBC	ODBC v3 (IBM DB2, unixODBC e ODBC win32)
PDO_PGSQL	PostgreSQL
PDO_SQLITE	SQLite 3 e SQLite 2
PDO_SQLSRV	Microsoft SQL Server / SQL Azure

Driver PDO do CUBRID

Introdução

PDO_CUBRID é um driver que implementa a interface PHP Data Objects (PDO) para permitir acesso do PHP a bancos de dados CUBRID.

> **Nota** :
>
> A versão atual do PDO_CUBRID não suporta conexão persistente agora.

Instalação

Para construir a extensão PDO_CUBRID, o SGBD CUBRID deve estar instalado no mesmo sistema que o PHP. O PDO_CUBRID é uma extensão » PECL, então siga as instruções em Instalação das extensões PECL para instalar a extensão PDO_CUBRID. Emita o comando **configure** para apontar para a localização do diretório base do CUBRID da seguinte forma:

```
$ ./configure --with-pdo-cubrid=/path/to/CUBRID[,shared]
```

O comando **configure** utiliza por padrão o valor da variável de ambiente *CUBRID*.

Uma DLL para esta extensão PECL não está disponível no momento. Veja também a seção Compilando no Windows. Para informações detalhadas sobre a instalação no Linux e Windows manualmente, leia o arquivo build-guide.html no pacote PECL CUBRID para referência.

Recursos do PDO_CUBRID

Cursores Roláveis

PDO_CUBRID suporta cursores roláveis. O tipo de cursor padrão é somente para frente, e você pode usar a opção do driver em PDO::prepare() para mudar o tipo de cursor.

Timeout

PDO_CUBRID suporta definição de tempo limite de execução de declarações SQL; Você pode usar PDO::setAttribute() para definir o valor do timeout.

Modo de Autocommit e Transação

PDO_CUBRID suporta tanto o modo de autocommit quanto transações, e o modo de autocommit está habilitado por padrão. Você pode usar PDO::setAttribute() para mudar seu estado.

Se você usar PDO::beginTransaction() para iniciar uma transação, ela desabilitará automaticamente o modo de autocommit e restaurará após PDO::commit() ou PDO::rollBack().

> **Nota** :
>
> Antes de desabilitar o modo de autocommit, qualquer instrução pendente é automaticamente processada.

Múltiplas Declarações SQL

PDO_CUBRID suporta Múltiplas declarações SQL. declarações SQL são separadas por ponto e vírgula (;).

Informações de Esquema

PDO_CUBRID implementa PDO::cubrid_schema() para obter informações de esquema.

LOBs

PDO_CUBRID suporta tipo de dado BLOB/CLOB. O LOB no PDO é representado como um fluxo, de forma que LOBs possam ser inseridos através de vinculação a um fluxo, e obtidos pela leitura de um fluxo retornado pelo CUBRID PDO. Por exemplo:

Exemplo #1 Inserir LOBs no CUBRID PDO

```
<?php
$fp = fopen('lob_test.png', 'rb');

$sql_stmt = "INSERT INTO lob_test(name, content) VALUES('lob_test.png', ?)";

$stmt = $dbh->prepare($sql_stmt);
$ret = $stmt->bindParam(1, $fp, PDO::PARAM_LOB);
$ret = $stmt->execute();
?>
```

Exemplo #2 Buscar LOBs no CUBRID PDO

```
<?php
$sql_stmt = "SELECT content FROM lob_test WHERE name='lob_test.png'";

$stmt = $dbh->prepare($sql_stmt);
$stmt->execute();
$result = $stmt->fetch(PDO::FETCH_NUM);

header("Content-Type: image/png");
fpassthru($result[0]);
?>
```

Meta-dados de Coluna

O PDOStatement::getColumnMeta() no CUBRID PDO retornará um array associativo contendo os seguintes valores:

- type
- name
- table
- def
- precision
- scale
- not_null
- auto_increment
- unique_key
- multiple_key
- primary_key
- foreign_key
- reverse_index
- reverse_unique

Tipo de Dados de Coleção

PDO_CUBRID suporta tipo de dados SET/MULTISET/SEQUENCE. Se o tipo de dados não for especificado, o padrão será char. Por exemplo:

Exemplo #3 Inserir conjunto no CUBRID PDO com tipo de dado padrão.

```
<?php
$conn_str ="cubrid:dbname=demodb;host=localhost;port=33000";
$cubrid_pdo = new PDO($conn_str, 'dba', '');

$cubrid_pdo->exec("DROP TABLE if exists test_tbl");
$cubrid_pdo->exec("CREATE TABLE test_tbl (col_1 SET(VARCHAR))");

$sql_stmt_insert = "INSERT INTO test_tbl VALUES (?);";
$stmt = $cubrid_pdo->prepare($sql_stmt_insert);
$data = array("abc","def","ghi");
$ret = $stmt->bindParam(1, $data, PDO::PARAM_NULL);
$ret = $stmt->execute();
var_Dump($ret);
?>
```

Exemplo #4 Especificar tipo de dado ao inserir conjunto no CUBRID PDO

```
<?php
$conn_str ="cubrid:dbname=demodb;host=localhost;port=33000";
$cubrid_pdo = new PDO($conn_str, 'dba', '');

$cubrid_pdo->exec("DROP TABLE if exists test_tbl");
$cubrid_pdo->exec("CREATE TABLE test_tbl (col_1 SET(int))");

$sql_stmt_insert = "INSERT INTO test_tbl VALUES (?);";
$stmt = $cubrid_pdo->prepare($sql_stmt_insert);
$data = array(1,2,3,4);
$ret = $stmt->bindParam(1, $data, 0,0,"int");
$ret = $stmt->execute();
var_Dump($ret);
?>
```

Tipos de Dados Vinculados CUBRID para o quinto parâmetro de PDOStatement::bindParam():

- CHAR
- STRING
- NCHAR
- VARNCHAR
- BIT
- VARBIT
- NUMERIC
- NUMBER
- INT
- SHORT
- BIGINT
- MONETARY
- FLOAT
- DOUBLE
- DATE
- TIME
- DATETIME
- TIMESTAMP

Constantes predefinidas

As constantes abaixo são definidas por este driver e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução. Além disso, estas constantes específicas do driver só devem ser usadas se este driver estiver sendo usado. Usar atributos específicos de um driver com outro driver pode resultar em comportamento inesperado. PDO::getAttribute() pode ser usada para obter o atributo `PDO::ATTR_DRIVER_NAME` para verificar o driver, se o código puder ser executado com vários drivers.

As seguintes constantes podem ser usadas ao definir o atributo do banco de dados. Elas podem ser passadas para PDO::getAttribute() ou PDO::setAttribute().

PDO::CUBRID attribute flags

Constante	Descrição
`PDO::CUBRID_ATTR_ISOLATION_LEVEL`	Nível de isolamento de transação para a conexão do banco de dados.
`PDO::CUBRID_ATTR_LOCK_TIMEOUT`	Tempo limite de transação em segundos.
`PDO::CUBRID_ATTR_MAX_STRING_LENGTH`	Somente leitura. O comprimento máximo da string para bit, varbit, char, varchar, nchar, tipos de dados nchar ao usar a API PDO CUBRID.

As seguintes constantes podem ser usadas ao definir o nível de isolamento da transação. Elas podem ser passadas para PDO::getAttribute() ou retornadas por PDO::setAttribute().

Flags de nível de isolamento do PDO::CUBRID

Constante	Descrição
`PDO::TRAN_COMMIT_CLASS_UNCOMMIT_INSTANCE`	O nível de isolamento mais baixo (1). Uma leitura suja, não repetível ou leitura fantasma pode ocorrer para a tupla e uma leitura não repetível pode ocorrer para a tabela também.
`PDO::TRAN_COMMIT_CLASS_COMMIT_INSTANCE`	Um nível de isolamento relativamente baixo (2). Uma leitura suja não ocorre, mas uma leitura não repetível ou fantasma pode ocorrer.

Constante	Descrição
PDO::TRAN_REP_CLASS_UNCOMMIT_INSTANCE	O isolamento padrão do CUBRID (3). Uma leitura suja, não repetível ou leitura fantasma pode ocorrer para a tupla, mas uma leitura repetível é garantida para a tabela.
PDO::TRAN_REP_CLASS_COMMIT_INSTANCE	Um nível de isolamento relativamente baixo (4). Uma leitura suja não ocorre, mas uma leitura não repetível ou fantasma pode ocorrer.
PDO::TRAN_REP_CLASS_REP_INSTANCE	Um nível de isolamento relativamente alto (5). Uma leitura suja ou não repetível não ocorre, mas uma leitura fantasma pode.
PDO::TRAN_SERIALIZABLE	O mais alto nível de isolamento (6). Problemas relacionados à concorrência (por exemplo, leitura suja, leitura não repetível, leitura fantasma, etc.) não ocorrem.

As seguintes constantes podem ser usadas ao obter informações de esquema. Elas podem ser passadas para PDO::cubrid_schema().

PDO::CUBRID schema flags

Constante	Descrição
PDO::CUBRID_SCH_TABLE	Obtém o nome e o tipo da tabela no CUBRID.
PDO::CUBRID_SCH_VIEW	Obtém o nome e o tipo da visão no CUBRID.
PDO::CUBRID_SCH_QUERY_SPEC	Obtém a definição da consulta da visão.
PDO::CUBRID_SCH_ATTRIBUTE	Obtém os atributos da coluna da tabela.
PDO::CUBRID_SCH_TABLE_ATTRIBUTE	Obtém os atributos da tabela.
PDO::CUBRID_SCH_METHOD	Obtém o método de instância. O método de instância é um método chamado por uma instância de classe. É usado com mais frequência do que o método de classe porque a maioria das operações é executada na instância.
PDO::CUBRID_SCH_TABLE_METHOD	Obtém o método de classe. O método de classe é um método chamado por um objeto de classe. Geralmente é usado para criar uma nova instância de classe ou inicializá-la. Também é usado para acessar ou atualizar atributos de classe.
PDO::CUBRID_SCH_METHOD_FILE	Obtém as informações do arquivo onde o método da tabela está definido.
PDO::CUBRID_SCH_SUPER_TABLE	Obtém o nome e o tipo da tabela que herda atributos.
PDO::CUBRID_SCH_SUB_TABLE	Obtém o nome e o tipo da tabela que herda atributos desta tabela.
PDO::CUBRID_SCH_CONSTRAINT	Obtém as restrições (constraints) da tabela.
PDO::CUBRID_SCH_TRIGGER	Obtém os disparadores (triggers) da tabela.
PDO::CUBRID_SCH_TABLE_PRIVILEGE	Obtém informações de privilégio da tabela.
PDO::CUBRID_SCH_COL_PRIVILEGE	Obtém informações de privilégio da coluna.
PDO::CUBRID_SCH_DIRECT_SUPER_TABLE	Obtém a tabela super direta da tabela.
PDO::CUBRID_SCH_PRIMARY_KEY	Obtém a chave primária da tabela.
PDO::CUBRID_SCH_IMPORTED_KEYS	Obtém chaves importadas da tabela.
PDO::CUBRID_SCH_EXPORTED_KEYS	Obtém chaves exportadas da tabela.

Constante	Descrição
PDO::CUBRID_SCH_CROSS_REFERENCE	Obtém relação de referência de duas tabelas.

Índice

PDO_CUBRID DSN

PDO_CUBRID DSN

(PECL PDO_CUBRID >= 8.3.0.0001)

PDO_CUBRID DSN — Conectando-se a bancos de dados CUBRID

Descrição

O Nome da Fonte de Dados (DSN) PDO_CUBRID é composto pelos seguintes elementos, delimitados por ponto e vírgula:

DSN prefix

O prefixo DSN é **cubrid:**.

`host`

O nome do host no qual o servidor de banco de dados reside.

`port`

A porta na qual o servidor de banco de dados está em execução.

`dbname`

O nome do banco de dados.

Notas

> **Nota** :
>
> Ao estabelecer a conexão com o CUBRID, você deve fornecer nome de usuário e senha, exceto o DSN.

Examples

Exemplo #1 Exemplos de DSN PDO_CUBRID

O exemplo a seguir mostra um DSN PDO_CUBRID para se conectar a um banco de dados CUBRID:

```
cubrid:host=localhost;port=33000;dbname=demodb
```

PDO::cubrid_schema

PDO::cubrid_schema

(PECL PDO_CUBRID >= 8.3.0.0001)

PDO::cubrid_schema — Obtém as informações do esquema solicitadas

Descrição

public function **PDO::cubrid_schema**(int `$schema_type`, string `$table_name = ?`, string `$col_name = ?`): array

Esta função é usada para obter as informações do esquema solicitadas do banco de dados. Deve ser informado o nome da tabela em `table_name`, se for necessário obter informações sobre determinada tabela e o nome da coluna em `col_name`, se a intenção for obter informações sobre determinada coluna (pode ser usado apenas com PDO:: CUBRID_SCH_COL_PRIVILEGE).

O resultado desta função é retornado como um array bidimensional (coluna (array associativo) * linha (array numérico)). As tabelas a seguir mostram os tipos de esquema e a estrutura de colunas do array de resultados a ser retornada com base no tipo de esquema.

Composição de Resultado para Cada Tipo

Esquema	Número da Coluna	Nome da Coluna	Valor
PDO::CUBRID_SCH_TABLE	1	NAME	
	2	TYPE	0:system table 1:view 2:table
PDO::CUBRID_SCH_VIEW	1	NAME	
	2	TYPE	1:view
PDO::CUBRID_SCH_QUERY_SPEC	1	QUERY_SPEC	
PDO::CUBRID_SCH_ATTRIBUTE / PDO::CUBRID_SCH_TABLE_ATTRIBUTE	1	ATTR_NAME	
	2	DOMAIN	
	3	SCALE	
	4	PRECISION	
	5	INDEXED	1:indexed
	6	NOT NULL	1:not null
	7	SHARED	1:shared
	8	UNIQUE	1:unique
	9	DEFAULT	
	10	ATTR_ORDER	base:1
	11	CLASS_NAME	
	12	SOURCE_CLASS	
	13	IS_KEY	1:key
PDO::CUBRID_SCH_METHOD / PDO::CUBRID_SCH_TABLE_METHOD	1	NAME	
	2	RET_DOMAIN	
	3	ARG_DOMAIN	
PDO::CUBRID_SCH_METHOD_FILE	1	METHOD_FILE	
PDO::CUBRID_SCH_SUPER_TABLE / PDO::CUBRID_SCH_DIRECT_SUPER_TABLE / PDO::CUBRID_SCH_SUB_TABLE	1	CLASS_NAME	

Esquema	Número da Coluna	Nome da Coluna	Valor
	2	TYPE	0:system table 1:view 2:table
PDO::CUBRID_SCH_CONSTRAINT	1	TYPE	0:unique 1:index 2:reverse unique 3:reverse index
	2	NAME	
	3	ATTR_NAME	
	4	NUM_PAGES	
	5	NUM_KEYS	
	6	PRIMARY_KEY	1:primary key
	7	KEY_ORDER	base:1
PDO::CUBRID_SCH_TRIGGER	1	NAME	
	2	STATUS	
	3	EVENT	
	4	TARGET_CLASS	
	5	TARGET_ATTR	
	6	ACTION_TIME	
	7	ACTION	
	8	PRIORITY	
	9	CONDITION_TIME	
	10	CONDITION	
PDO::CUBRID_SCH_TABLE_PRIVILEGE / PDO::CUBRID_SCH_COL_PRIVILEGE	1	CLASS_NAME / ATTR_NAME	
	2	PRIVILEGE	
	3	GRANTABLE	
PDO::CUBRID_SCH_PRIMARY_KEY	1	CLASS_NAME	
	2	ATTR_NAME	
	3	KEY_SEQ	base:1
	4	KEY_NAME	
PDO::CUBRID_SCH_IMPORTED_KEYS / PDO::CUBRID_SCH_EXPORTED_KEYS / PDO::CUBRID_SCH_CROSS_REFERENCE	1	PKTABLE_NAME	
	2	PKCOLUMN_NAME	
	3	FKTABLE_NAME	base:1
	4	FKCOLUMN_NAME	
	5	KEY_SEQ	base:1
	6	UPDATE_ACTION	0:cascade 1:restrict 2:no

Esquema	Número da Coluna	Nome da Coluna	Valor
			action 3:set null
	7	DELETE_ACTION	0:cascade 1:restrict 2:no action 3:set null
	8	FK_NAME	
	9	PK_NAME	

Parâmetros

`schema_type`

Tipo de esquema a ser conhecido.

`table_name`

Tabela cujo esquema deseja-se conhecer.

`col_name`

Coluna cujo esquema deseja-se conhecer.

Valor Retornado

Array contendo informações do esquema, em caso de sucesso;

FALSE, em caso de falha.

Examples

Exemplo #1 Um exemplo de PDO::cubrid_schema()

Este exemplo mostra como obter a chave primária e as chaves estrangeiras da tabela 'game'.

```
<?php
$pk_list = $dbh->cubrid_schema(PDO::CUBRID_SCH_PRIMARY_KEY, "game");
print_r($pk_list);

$fk_list = $dbh->cubrid_schema(PDO::CUBRID_SCH_IMPORTED_KEYS, "game");
print_r($fk_list);
?>
```

O exemplo acima produzirá:

```
Resultado:
Array
(
    [0] => Array
        (
            [CLASS_NAME] => game
            [ATTR_NAME] => athlete_code
            [KEY_SEQ] => 3
```

```
            [KEY_NAME] => pk_game_host_year_event_code_athlete_code
        )

    [1] => Array
        (
            [CLASS_NAME] => game
            [ATTR_NAME] => event_code
            [KEY_SEQ] => 2
            [KEY_NAME] => pk_game_host_year_event_code_athlete_code
        )

    [2] => Array
        (
            [CLASS_NAME] => game
            [ATTR_NAME] => host_year
            [KEY_SEQ] => 1
            [KEY_NAME] => pk_game_host_year_event_code_athlete_code
        )

)
Array
(
    [0] => Array
        (
            [PKTABLE_NAME] => athlete
            [PKCOLUMN_NAME] => code
            [FKTABLE_NAME] => game
            [FKCOLUMN_NAME] => athlete_code
            [KEY_SEQ] => 1
            [UPDATE_RULE] => 1
            [DELETE_RULE] => 1
            [FK_NAME] => fk_game_athlete_code
            [PK_NAME] => pk_athlete_code
        )

    [1] => Array
        (
            [PKTABLE_NAME] => event
            [PKCOLUMN_NAME] => code
            [FKTABLE_NAME] => game
            [FKCOLUMN_NAME] => event_code
            [KEY_SEQ] => 1
            [UPDATE_RULE] => 1
            [DELETE_RULE] => 1
            [FK_NAME] => fk_game_event_code
            [PK_NAME] => pk_event_code
        )

)
```

Driver PDO do MS SQL Server

Introdução

PDO_DBLIB é um driver que implementa a interface para Objetos de Dados PHP Data Objects (PDO) para permitir o acesso do PHP aos bancos de dados Microsoft SQL Server e Sybase através da biblioteca FreeTDS.

Esta extensão não está mais disponível no Windows.

No Windows, deve ser usado o SqlSrv, um driver alternativo para MS SQL está disponível na Microsoft: » http://msdn.microsoft.com/en-us/sqlserver/ff657782.aspx .

Caso não seja possível usar o SqlSrv, pode-se usar o driver PDO_ODBC para conectar-se aos bancos de dados Microsoft SQL Server e Sybase, pois o DB-LIB nativo do Windows é antigo, inseguro para threads e não é mais suportado pela Microsoft.

Índice

PDO_DBLIB DSN

PDO_DBLIB DSN

(PECL PDO_DBLIB >= 0.9.0)

PDO_DBLIB DSN — Conectando-se aos bancos de dados Microsoft SQL Server e Sybase

Descrição

O Nome da Fonte de Dados (DSN) PDO_DBLIB é composto pelos seguintes elementos:

Prefixo DSN

> O prefixo DSN é **`sybase:`** se o PDO_DBLIB foi vinculado às bibliotecas Sybase ct-lib, **`mssql:`** se o PDO_DBLIB foi vinculado às bibliotecas do Microsoft SQL Server ou **`dblib :`** se o PDO_DBLIB foi vinculado às bibliotecas FreeTDS.

`host`

> O nome do servidor onde o banco de dados reside. O padrão é 127.0.0.1.

`dbname`

> O nome do banco de dados.

`charset`

> O conjunto de caracteres do cliente.

`appname`

> O nome da aplicação (usado nos processos do sistema). O padrão é "PHP Generic DB-lib" ou "PHP freetds".

`secure`

> Atualmente não utilizado.

Examples

Exemplo #1 Exemplos de DSN PDO_DBLIB

Os exemplos a seguir mostram um DSN PDO_DBLIB para conexão com bancos de dados Microsoft SQL Server e Sybase:

```
mssql:host=localhost;dbname=testdb
sybase:host=localhost;dbname=testdb
dblib:host=localhost;dbname=testdb
```

Pdo\Dblib

(PHP 8 >= 8.4.0)

Introdução

Uma subclasse PDO representando uma conexão que usa o driver PDO DBLib.

Resumo da classe

namespace **Pdo**;
class **Dblib** extends PDO {
/* Constantes herdadas */
public const int *PDO::PARAM_NULL*;
public const int *PDO::PARAM_BOOL* = 5;
public const int *PDO::PARAM_INT* = 1;
public const int *PDO::PARAM_STR* = 2;
public const int *PDO::PARAM_LOB* = 3;
public const int *PDO::PARAM_STMT* = 4;
public const int *PDO::PARAM_INPUT_OUTPUT*;
public const int *PDO::PARAM_STR_NATL*;
public const int *PDO::PARAM_STR_CHAR*;
public const int *PDO::PARAM_EVT_ALLOC*;
public const int *PDO::PARAM_EVT_FREE*;
public const int *PDO::PARAM_EVT_EXEC_PRE*;
public const int *PDO::PARAM_EVT_EXEC_POST*;
public const int *PDO::PARAM_EVT_FETCH_PRE*;
public const int *PDO::PARAM_EVT_FETCH_POST*;
public const int *PDO::PARAM_EVT_NORMALIZE*;
public const int *PDO::FETCH_DEFAULT*;
public const int *PDO::FETCH_LAZY*;
public const int *PDO::FETCH_ASSOC*;
public const int *PDO::FETCH_NUM*;
public const int *PDO::FETCH_BOTH*;
public const int *PDO::FETCH_OBJ*;
public const int *PDO::FETCH_BOUND*;
public const int *PDO::FETCH_COLUMN*;
public const int *PDO::FETCH_CLASS*;
public const int *PDO::FETCH_INTO*;
public const int *PDO::FETCH_FUNC*;
public const int *PDO::FETCH_GROUP*;
public const int *PDO::FETCH_UNIQUE*;
public const int *PDO::FETCH_KEY_PAIR*;
public const int *PDO::FETCH_CLASSTYPE*;
public const int *PDO::FETCH_SERIALIZE*;

```
public const int PDO::FETCH_PROPS_LATE;
public const int PDO::FETCH_NAMED;
public const int PDO::ATTR_AUTOCOMMIT;
public const int PDO::ATTR_PREFETCH;
public const int PDO::ATTR_TIMEOUT;
public const int PDO::ATTR_ERRMODE;
public const int PDO::ATTR_SERVER_VERSION;
public const int PDO::ATTR_CLIENT_VERSION;
public const int PDO::ATTR_SERVER_INFO;
public const int PDO::ATTR_CONNECTION_STATUS;
public const int PDO::ATTR_CASE;
public const int PDO::ATTR_CURSOR_NAME;
public const int PDO::ATTR_CURSOR;
public const int PDO::ATTR_ORACLE_NULLS;
public const int PDO::ATTR_PERSISTENT;
public const int PDO::ATTR_STATEMENT_CLASS;
public const int PDO::ATTR_FETCH_TABLE_NAMES;
public const int PDO::ATTR_FETCH_CATALOG_NAMES;
public const int PDO::ATTR_DRIVER_NAME;
public const int PDO::ATTR_STRINGIFY_FETCHES;
public const int PDO::ATTR_MAX_COLUMN_LEN;
public const int PDO::ATTR_EMULATE_PREPARES;
public const int PDO::ATTR_DEFAULT_FETCH_MODE;
public const int PDO::ATTR_DEFAULT_STR_PARAM;
public const int PDO::ERRMODE_SILENT;
public const int PDO::ERRMODE_WARNING;
public const int PDO::ERRMODE_EXCEPTION;
public const int PDO::CASE_NATURAL;
public const int PDO::CASE_LOWER;
public const int PDO::CASE_UPPER;
public const int PDO::NULL_NATURAL;
public const int PDO::NULL_EMPTY_STRING;
public const int PDO::NULL_TO_STRING;
public const string PDO::ERR_NONE;
public const int PDO::FETCH_ORI_NEXT;
public const int PDO::FETCH_ORI_PRIOR;
public const int PDO::FETCH_ORI_FIRST;
public const int PDO::FETCH_ORI_LAST;
public const int PDO::FETCH_ORI_ABS;
public const int PDO::FETCH_ORI_REL;
public const int PDO::CURSOR_FWDONLY;
public const int PDO::CURSOR_SCROLL;
/* Constantes */
public const int ATTR_CONNECTION_TIMEOUT;
public const int ATTR_QUERY_TIMEOUT;
public const bool ATTR_STRINGIFY_UNIQUEIDENTIFIER;
public const int ATTR_VERSION;
public const int ATTR_TDS_VERSION;
public const int ATTR_SKIP_EMPTY_ROWSETS;
public const bool ATTR_DATETIME_CONVERT;
/* Métodos herdados */
```

```
public function PDO::__construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
public function PDO::beginTransaction(): bool
public function PDO::commit(): bool
public static function PDO::connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
public function PDO::errorCode(): ?string
public function PDO::errorInfo(): array
public function PDO::exec(string $statement): int|false
public function PDO::getAttribute(int $attribute): mixed
public static function PDO::getAvailableDrivers(): array
public function PDO::inTransaction(): bool
public function PDO::lastInsertId(?string $name = null): string|false
public function PDO::prepare(string $query, array $options = []): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = null): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_COLUMN, int $colno):
PDOStatement|false
public function PDO::query(
    string $query,
    ?int $fetchMode = PDO::FETCH_CLASS,
    string $classname,
    array $constructorArgs
): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_INTO, object $object):
PDOStatement|false
public function PDO::quote(string $string, int $type = PDO::PARAM_STR): string|false
public function PDO::rollBack(): bool
public function PDO::setAttribute(int $attribute, mixed $value): bool
}
```

Constantes predefinidas

Pdo\Dblib::ATTR_CONNECTION_TIMEOUT

Pdo\Dblib::ATTR_QUERY_TIMEOUT

Pdo\Dblib::ATTR_STRINGIFY_UNIQUEIDENTIFIER

A partir do PHP 8.4.0, esta constante é tipada como bool; anteriormente, era tipada como int.

Pdo\Dblib::ATTR_VERSION

Pdo\Dblib::ATTR_TDS_VERSION

Pdo\Dblib::ATTR_SKIP_EMPTY_ROWSETS

`Pdo\Dblib::ATTR_DATETIME_CONVERT`

Esse atributo de conexão controla o formato das strings para tipos datetime. Quando definido como `false`, o PDO_DBLIB retornará um tipo datetime como uma string no formato em que o SQL Server o retorna (ou seja, `"2017-10-27 10:22:44"`). Quando `true`, o PDO_DBLIB converterá o tipo datetime em uma string usando um formato definido pelo usuário ou pela localidade, conforme especificado no arquivo *locales.conf* do FreeTDS. Por padrão, esse atributo é `false`.

A partir do PHP 8.4.0, esta constante é tipada como bool; anteriormente, era tipada como int.

Driver PDO do Firebird

Introdução

PDO_FIREBIRD é um driver que implementa a interface aos Objetos de Dados PHP (PDO) para permitir acesso do PHP a bancos de dados Firebird.

Instalação

Use **--with-pdo-firebird[=DIR]** para instalar a extensão PDO Firebird, onde o `[=DIR]` opcional é o diretório base de instalação do Firebird.

```
$ ./configure --with-pdo-firebird
```

A partir do PHP 8.4.0, esta extensão utiliza as APIs C++ do Firebird e, portanto, requer um compilador C++ e deve ser compilada contra o fbclient 3.0 ou superior.

Constantes predefinidas

As constantes abaixo são definidas por este driver e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução. Além disso, estas constantes específicas do driver só devem ser usadas se este driver estiver sendo usado. Usar atributos específicos de um driver com outro driver pode resultar em comportamento inesperado. PDO::getAttribute() pode ser usada para obter o atributo `PDO::ATTR_DRIVER_NAME` para verificar o driver, se o código puder ser executado com vários drivers.

> **Aviso**
>
> As constantes listadas abaixo foram *DESCONTINUADAS* a partir do PHP 8.5.0. Use as constantes correspondentes de Pdo\Firebird no lugar delas.

`PDO::FB_ATTR_DATE_FORMAT` (int)

Sinônimo de `Pdo\Firebird::ATTR_DATE_FORMAT`.

`PDO::FB_ATTR_TIME_FORMAT` (int)

Sinônimo de `Pdo\Firebird::ATTR_TIME_FORMAT`.

`PDO::FB_ATTR_TIMESTAMP_FORMAT` (int)

Sinônimo de `Pdo\Firebird::ATTR_TIMESTAMP_FORMAT`.

Índice

PDO_FIREBIRD DSN

PDO_FIREBIRD DSN

(PECL PDO_FIREBIRD >= 0.1.0)

PDO_FIREBIRD DSN — Conectando ao banco de dados Firebird

Descrição

O Nome da Fonte de Dados (DSN) PDO_FIREBIRD é composto pelos seguintes elementos:

DSN prefix

O prefixo DSN é **`firebird:`**.

`dbname`

O nome do banco de dados.

`charset`

O conjunto de caracteres.

`role`

O nome da função SQL.

`dialect`

O dialeto do banco de dados; `1` ou `3`. Se não for especificado, o dialeto será padronizado como `3`. Disponível a partir do PHP 7.4.0.

Examples

Exemplo #1 Exemplo de DSN PDO_FIREBIRD DSN com caminho

O exemplo a seguir mostra um DSN PDO_FIREBIRD para conexão com bancos de dados Firebird:

```
firebird:dbname=/caminho/para/DATABASE.FDB
```

Exemplo #2 Exemplo de DSN PDO_FIREBIRD com porta e caminho

O exemplo a seguir mostra um DSN PDO_FIREBIRD para conexão a um banco de dados Firebird usando nome de servidor, porta e caminho:

```
firebird:dbname=servidor/porta:/caminho/para/DATABASE.FDB
```

Exemplo #3 Exemplo de DSN PDO_FIREBIRD com localhost e caminho para Employee.fdb no sistema Debian

O exemplo a seguir mostra um DSN PDO_FIREBIRD para conexão a um banco de dados Firebird employee.fdb usando localhost:

```
firebird:dbname=localhost:/var/lib/firebird/2.5/data/employee.fdb
```

Exemplo #4 DSN PDO_FIREBIRD para conectar-se a um banco de dados de dialeto 1

O exemplo a seguir mostra um DSN PDO_FIREBIRD para conexão a um banco de dados Firebird teste.fdb que foi criado usando o dialeto 1. Isso só é suportado a partir do PHP 7.4.0.

```
firebird:dbname=localhost:/var/lib/firebird/2.5/data/teste.fdb;charset=utf-8;dialect=1
```

Pdo\Firebird

(PHP 8 >= 8.4.0)

Introdução

Uma subclasse PDO representando uma conexão que usa o driver PDO Firebird.

Resumo da classe

namespace **Pdo**;
class **Firebird** extends PDO {
/* Constantes herdadas */
public const int *PDO::PARAM_NULL*;
public const int *PDO::PARAM_BOOL* = 5;
public const int *PDO::PARAM_INT* = 1;
public const int *PDO::PARAM_STR* = 2;
public const int *PDO::PARAM_LOB* = 3;
public const int *PDO::PARAM_STMT* = 4;
public const int *PDO::PARAM_INPUT_OUTPUT*;
public const int *PDO::PARAM_STR_NATL*;
public const int *PDO::PARAM_STR_CHAR*;
public const int *PDO::PARAM_EVT_ALLOC*;
public const int *PDO::PARAM_EVT_FREE*;
public const int *PDO::PARAM_EVT_EXEC_PRE*;
public const int *PDO::PARAM_EVT_EXEC_POST*;
public const int *PDO::PARAM_EVT_FETCH_PRE*;
public const int *PDO::PARAM_EVT_FETCH_POST*;
public const int *PDO::PARAM_EVT_NORMALIZE*;
public const int *PDO::FETCH_DEFAULT*;
public const int *PDO::FETCH_LAZY*;
public const int *PDO::FETCH_ASSOC*;
public const int *PDO::FETCH_NUM*;
public const int *PDO::FETCH_BOTH*;
public const int *PDO::FETCH_OBJ*;
public const int *PDO::FETCH_BOUND*;
public const int *PDO::FETCH_COLUMN*;
public const int *PDO::FETCH_CLASS*;
public const int *PDO::FETCH_INTO*;
public const int *PDO::FETCH_FUNC*;
public const int *PDO::FETCH_GROUP*;
public const int *PDO::FETCH_UNIQUE*;
public const int *PDO::FETCH_KEY_PAIR*;

```
public const int PDO::FETCH_CLASSTYPE;
public const int PDO::FETCH_SERIALIZE;
public const int PDO::FETCH_PROPS_LATE;
public const int PDO::FETCH_NAMED;
public const int PDO::ATTR_AUTOCOMMIT;
public const int PDO::ATTR_PREFETCH;
public const int PDO::ATTR_TIMEOUT;
public const int PDO::ATTR_ERRMODE;
public const int PDO::ATTR_SERVER_VERSION;
public const int PDO::ATTR_CLIENT_VERSION;
public const int PDO::ATTR_SERVER_INFO;
public const int PDO::ATTR_CONNECTION_STATUS;
public const int PDO::ATTR_CASE;
public const int PDO::ATTR_CURSOR_NAME;
public const int PDO::ATTR_CURSOR;
public const int PDO::ATTR_ORACLE_NULLS;
public const int PDO::ATTR_PERSISTENT;
public const int PDO::ATTR_STATEMENT_CLASS;
public const int PDO::ATTR_FETCH_TABLE_NAMES;
public const int PDO::ATTR_FETCH_CATALOG_NAMES;
public const int PDO::ATTR_DRIVER_NAME;
public const int PDO::ATTR_STRINGIFY_FETCHES;
public const int PDO::ATTR_MAX_COLUMN_LEN;
public const int PDO::ATTR_EMULATE_PREPARES;
public const int PDO::ATTR_DEFAULT_FETCH_MODE;
public const int PDO::ATTR_DEFAULT_STR_PARAM;
public const int PDO::ERRMODE_SILENT;
public const int PDO::ERRMODE_WARNING;
public const int PDO::ERRMODE_EXCEPTION;
public const int PDO::CASE_NATURAL;
public const int PDO::CASE_LOWER;
public const int PDO::CASE_UPPER;
public const int PDO::NULL_NATURAL;
public const int PDO::NULL_EMPTY_STRING;
public const int PDO::NULL_TO_STRING;
public const string PDO::ERR_NONE;
public const int PDO::FETCH_ORI_NEXT;
public const int PDO::FETCH_ORI_PRIOR;
public const int PDO::FETCH_ORI_FIRST;
public const int PDO::FETCH_ORI_LAST;
public const int PDO::FETCH_ORI_ABS;
public const int PDO::FETCH_ORI_REL;
public const int PDO::CURSOR_FWDONLY;
public const int PDO::CURSOR_SCROLL;
/* Constantes */
public const int ATTR_DATE_FORMAT;
public const int ATTR_TIME_FORMAT;
public const int ATTR_TIMESTAMP_FORMAT;
public const int TRANSACTION_ISOLATION_LEVEL;
public const int READ_COMMITTED;
public const int REPEATABLE_READ;
```

```
public const int SERIALIZABLE;
public const int WRITABLE_TRANSACTION;
/* Métodos */
public static function getApiVersion(): int
/* Métodos herdados */
public function PDO::__construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
public function PDO::beginTransaction(): bool
public function PDO::commit(): bool
public static function PDO::connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
public function PDO::errorCode(): ?string
public function PDO::errorInfo(): array
public function PDO::exec(string $statement): int|false
public function PDO::getAttribute(int $attribute): mixed
public static function PDO::getAvailableDrivers(): array
public function PDO::inTransaction(): bool
public function PDO::lastInsertId(?string $name = null): string|false
public function PDO::prepare(string $query, array $options = []): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = null): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_COLUMN, int $colno):
PDOStatement|false
public function PDO::query(
    string $query,
    ?int $fetchMode = PDO::FETCH_CLASS,
    string $classname,
    array $constructorArgs
): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_INTO, object $object):
PDOStatement|false
public function PDO::quote(string $string, int $type = PDO::PARAM_STR): string|false
public function PDO::rollBack(): bool
public function PDO::setAttribute(int $attribute, mixed $value): bool
}
```

Constantes predefinidas

`Pdo\Firebird::ATTR_DATE_FORMAT`

Define o formato de data.

`Pdo\Firebird::ATTR_TIME_FORMAT`

Define o formato de horário.

`Pdo\Firebird::ATTR_TIMESTAMP_FORMAT`

Define o formato de timestamp.

`Pdo\Firebird::TRANSACTION_ISOLATION_LEVEL`

Atributo para definir o nível de isolamento da transação. Pode ser uma das constantes `Pdo\Firebird::READ_COMMITTED`, `Pdo\Firebird::REPEATABLE_READ` ou `Pdo\Firebird::SERIALIZABLE`.

`Pdo\Firebird::READ_COMMITTED`

Opção que denota que o nível de isolamento de transação ANSI é de confirmação de leitura. Este é o comportamento padrão.

`Pdo\Firebird::REPEATABLE_READ`

Opção que denota que o nível de isolamento de transação ANSI é de leitura repetível. Corresponde ao nível de isolamento "snapshot" do Firebird.

`Pdo\Firebird::SERIALIZABLE`

Opção que denota que o nível de isolamento de transação ANSI é serializável. Corresponde ao nível de isolamento "snapshot table stability" do Firebird.

`Pdo\Firebird::WRITABLE_TRANSACTION`

Atributo booleano usado para alternar o modo de acesso de transação entre `READ ONLY` e `READ WRITE`. Por padrão é `true`, que indica `READ WRITE`.

Índice

Pdo\Firebird::getApiVersion

Pdo\Firebird::getApiVersion

(PHP 8 >= 8.4.0)

Pdo\Firebird::getApiVersion — Obtém a versão da API

Descrição

public static function **Pdo\Firebird::getApiVersion**(): int

Retorna a versão da API do Firebird como definida pela constante C **`FB_API_VER`** em *ibase.h*.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna a API do FireBird como um int.

Driver PDO da IBM

Introdução

PDO_IBM é um driver que implementa a interface aos Objetos de Dados PHP (PDO) para permitir acesso do PHP a bancos de dados IBM.

Instalação

Para construir a extensão PDO_IBM, o DB2 Client v9.1 ou posterior deve ser instalado no mesmo sistema que o PHP. O DB2 Client pode ser obtido do » Site de Desenvolvimento de Aplicativos da IBM.

> **Nota** : **Notas**
>
> O DB2 Client v9.1 ou posterior suporta acesso direto aos servidores DB2 para Linux, UNIX e Windows v8 e v9.1.
>
> O DB2 Client v9.1 também suporta acesso a servidores DB2 UDB para i5 e DB2 UDB para z/OS usando o » produto DB2 Connect adquirido separadamente.

PDO_IBM é uma extensão » PECL, portanto siga as instruções em Instalação das extensões PECL para instalar a extensão PDO_IBM. Execute o comando **configure** para apontar o caminho dos arquivos de cabeçalho e bibliotecas do DB2 Client, conforme a seguir:

```
bash$ ./configure --with-pdo-ibm=/caminho/para/sqllib[,shared]
```

O comando **configure** tem como padrão o valor da variável de ambiente *DB2DIR*.

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de Configuração do PDO_IBM

Nome	Padrão	Modificável	Registro de Alterações
pdo_ibm.i5_dbcs_alloc	"0"	`INI_SYSTEM`	Adicionado no PDO_IBM 1.5.0
pdo_ibm.i5_override_ccsid	"0"	`INI_SYSTEM`	Adicionado no PDO_IBM 1.5.0

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`pdo_ibm.i5_dbcs_alloc` int

Esta opção afeta a estratégia de alocação de buffer interno no IBM i. Por padrão, esta opção é 0. Quando esta opção é definida, os buffers são alocados com um tamanho muito maior, caso o banco de dados se engane sobre o tamanho dos caracteres ao converter entre codificações. Esta opção usa seis vezes mais memória para buffers (para considerar as maiores sequências UTF-8 possíveis), mas pode ser necessária se forem retornados dados truncados.

- 0 - Buffers de tamanho mínimo são alocados.
- 1 - Buffers de tamanho maior são alocados.

`pdo_ibm.i5_override_ccsid` int

O CCSID ASCII a ser usado para conversão de EBCDIC no IBM i. Definir como 1208 usará UTF-8. Por padrão, é 0, que selecionará o CCSID da tarefa ASCII padrão.

Para saber mais sobre CCSIDs no IBM i, consulte a » documentação da IBM.

Índice

PDO_IBM DSN

PDO_IBM DSN

(PECL PDO_IBM >= 0.9.0)

PDO_IBM DSN — Conectando-se a bancos de dados IBM

Descrição

O Nome da Fonte de Dados (DSN) PDO_IBM é baseado no DSN IBM CLI. Os principais componentes do DSN PDO_IBM são:

DSN prefix

O prefixo DSN é **`ibm:`**.

DSN

O DSN pode ser qualquer um dos seguintes:

- a) Configuração da fonte de dados usando *db2cli.ini* ou *odbc.ini*
- b) Nome do banco de dados catalogado, ou seja, apelido do banco de dados no catálogo do cliente DB2
- c) String de conexão completa no seguinte formato: `DRIVER={IBM DB2 ODBC DRIVER};DATABASE=database;HOSTNAME=hostname;PORT=port;PROTOCOL=TCPIP;UID=username;PWD=password;` onde os parâmetros representam os seguintes valores:

  `database`

  O nome do banco de dados.

  `hostname`

  O nome ou endereço do servidor de banco de dados.

  `port`

  A porta TCP/IP na qual o banco de dados está atendendo requisições.

  `username`

  O nome de usuário com o qual será feita a conexão ao banco de dados.

  `password`

  A senha com a qual será feita a conexão ao banco de dados.

Examples

Exemplo #1 Exemplo de DSN PDO_IBM usando *db2cli.ini*

O exemplo a seguir mostra um DSN PDO_IBM para conexão com um banco de dados DB2 catalogado como DB2_9 em *db2cli.ini*:

```
$db = new PDO("ibm:DSN=DB2_9", "", "");

[DB2_9]
Database=db_teste
Protocol=tcpip
Hostname=11.22.33.444
Servicename=56789
```

Exemplo #2 Exemplo de DSN PDO_IBM usando uma string de conexão

O exemplo a seguir mostra um DSN PDO_IBM para conexão com um banco de dados DB2 denominado **`testdb`** usando a sintaxe da string de conexão CLI do DB2.

```
$db = new PDO("ibm:DRIVER={IBM DB2 ODBC DRIVER};DATABASE=testdb;" .
  "HOSTNAME=11.22.33.444;PORT=56789;PROTOCOL=TCPIP;", "usuario_teste", "senha_teste");
```

Driver PDO do Informix

Introdução

PDO_INFORMIX é um driver que implementa a interface aos Objetos de Dados PHP (PDO) para permitir acesso do PHP a bancos de dados Informix.

Instalação

Para construir a extensão PDO_INFORMIX, o Informix Client SDK 2.81 UC1 ou superior deve ser instalado no mesmo sistema que o PHP. O Informix Client SDK está disponível no » site de suporte do IBM Informix.

PDO_INFORMIX é uma extensão » PECL, portanto devem ser seguidas as instruções em Instalação das extensões PECL para instalá-la. Execute o comando **configure** para apontar o caminho dos arquivos de cabeçalho e bibliotecas do Informix Client SDK, conforme a seguir:

```
bash$ ./configure --with-pdo-informix=/caminho/para/SDK[,shared]
```

O comando **configure** tem como padrão o valor da variável de ambiente *INFORMIXDIR*.

Cursores roláveis

PDO_INFORMIX suporta cursores roláveis; no entanto, eles não estão habilitados por padrão. Para ativar o suporte ao cursor rolável, deve-se definir **`ENABLESCROLLABLECURSORS=1`** nas configurações de conexão ODBC correspondentes em *odbc.ini* ou passar a opção **`EnableScrollableCursors=1`** na string de conexão DSN.

Índice

PDO_INFORMIX DSN

PDO_INFORMIX DSN

(PECL PDO_INFORMIX >= 0.1.0)

PDO_INFORMIX DSN — Conectando-se a bancos de dados Informix

Descrição

O Nome da Fonte de Dados (DSN) do PDO_INFORMIX é baseado na string DSN ODBC do Informix. Detalhes sobre como configurar um DSN ODBC do Informix estão disponíveis no » Centro de informações do Informix Dynamic Server. Os principais componentes do DSN PDO_INFORMIX são:

DSN prefix

O prefixo DSN é **`informix:`**.

DSN

O DSN pode ser uma configuração de fonte de dados usando *odbc.ini* ou uma » string de conexão completa.

Examples

Exemplo #1 Exemplo de DSN PDO_INFORMIX usando *odbc.ini*

O exemplo a seguir mostra um DSN PDO_INFORMIX para conexão a um banco de dados Informix catalogado como Infdrv33 em *odbc.ini*:

```
$db = new PDO("informix:DSN=Infdrv33", "", "");
```

```
[ODBC Data Sources]
Infdrv33=INFORMIX 3.3 32-BIT

[Infdrv33]
Driver=/opt/informix/csdk_2.81.UC1G2/lib/cli/iclis09b.so
Description=INFORMIX 3.3 32-BIT
Database=common_db
LogonID=usuario_teste
pwd=senha_teste
Servername=ids_server
DB_LOCALE=en_US.819
OPTIMIZEAUTOCOMMIT=1
ENABLESCROLLABLECURSORS=1
```

Exemplo #2 Exemplo de DSN PDO_INFORMIX usando uma string de conexão

O exemplo a seguir mostra um DSN PDO_INFORMIX para conexão a um banco de dados Informix denominado **`common_db`** usando a sintaxe de string de conexão do Informix.

```
$db = new PDO("informix:host=host.domain.com; service=9800;
    database=common_db; server=ids_server; protocol=onsoctcp;
    EnableScrollableCursors=1", "usuario_teste", "senha_teste");
```

Driver PDO do MySQL

Introdução

PDO_MYSQL é um driver que implementa a interface PHP Data Objects (PDO) para habilitar o acesso do PHP aos bancos de dados MySQL.

PDO_MYSQL utiliza as preparações emuladas por padrão.

MySQL 8

Ao executar uma versão de PHP anterior a 7.1.16, ou 7.2.4, defina o plugin padrão de senhas do Servidor MySQL 8 para *mysql_native_password* ou poderão surgir erros similares a *The server requested authentication method unknown to the client [caching_sha2_password]* mesmo quando *caching_sha2_password* não seja usado.

Isto acontece porque o padrão do MySQL 8 é caching_sha2_password, um plugin que não é reconhecido pelas versões antigas do PHP (mysqlnd). Portanto, altere o plugin configurando `default_authentication_plugin=mysql_native_password` no arquivo *my.cnf*. O plugin *caching_sha2_password* é totalmente suportado a partir do PHP 7.4.4. Para versões anteriores do PHP, a extensão mysql_xdevapi suporta o plugin.

> **Aviso**
>
> Cuidado: Alguns tipos de tabelas do MySQL (motores de armazenamento) não suportam transações. Ao escrever código para um banco de dados transacional onde o tipo de tabela não suporta transações, o MySQL irá entender que a transação foi inicializada com sucesso. Além disso, qualquer consulta DDL irá, de forma implícita, executar o "commit" para qualquer transação que esteja pendente.

> **Nota** :
>
> O driver MySQL não suporta adequadamente a opção `PDO::PARAM_INPUT_OUTPUT` através de PDOStatement::bindParam(); embora tais parâmetros possam ser usados, eles não são atualizados (isto é, a saída real é ignorada).

Instalação

As distribuições Unix comuns incluem versões binárias do PHP que podem ser instaladas. Embora estas versões binárias são tipicamente compiladas com suporte a extensões MySQL, as bibliotecas das extensões em si podem precisar ser instaladas usando um pacote adicional. Verifique o gerenciador de pacote que vem com a distribuição sendo usada para disponibilidade desse pacote.

Por exemplo, no Ubuntu o pacote `php5-mysql` instala as extensões PHP ext/mysql, ext/mysqli e PDO_MYSQL. No CentOS, o pacote `php-mysql` também instala estas três extensões.

Alternativamente, o usuário pode compilar esta extensão por conta própria. Compilar o PHP direto da fonte permite especificar as extensões a serem utilizadas, assim como as bibliotecas clientes para cada extensão.

Ao compilar, use a opção **--with-pdo-mysql[=DIR]** para instalar a extensão PDO MySQL, onde o `[=DIR]` opcional é a localização da biblioteca base do MySQL. Mysqlnd é a biblioteca principal. Para detalher sobre a escolha de biblioteca, consulte a seção Escolhendo uma biblioteca MySQL.

Opcionalmente, **--with-mysql-sock[=DIR]** define a localização do ponteiro do soquete Unix MySQL para todas as extensões MySQL, incluindo a PDO_MYSQL. Se não especificada, as localizações padrão são pesquisadas.

Optionalmente, **--with-zlib-dir[=DIR]** é usada para definir o caminho da instalação da zlib.

```
$ ./configure --with-pdo-mysql --with-mysql-sock=/var/mysql/mysql.sock
```

O suporte a SSL é habilitado usando as constantes `Pdo\Mysql::ATTR_SSL_*` apropriadas, que é o equivalente a chamar a » função mysql_ssl_set() da API C do MySQL. Além disso, o SSL não pode ser habilitado com PDO::setAttribute() porque a conexão já existirá. Consulte também a documentação do MySQL sobre » conexão ao MySQL com SSL.

Constantes predefinidas

As constantes abaixo são definidas por este driver e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução. Além disso, estas constantes específicas do driver só devem ser usadas se este driver estiver sendo usado. Usar atributos específicos de um driver com outro driver pode resultar em comportamento inesperado. PDO::getAttribute() pode ser usada para obter o atributo `PDO::ATTR_DRIVER_NAME` para verificar o driver, se o código puder ser executado com vários drivers.

> **Aviso**
>
> As constantes listadas abaixo foram *DESCONTINUADAS* a partir do PHP 8.5.0. Use as constantes correspondentes de Pdo\Mysql no lugar delas.

`PDO::MYSQL_ATTR_USE_BUFFERED_QUERY` (int)

Sinônimo de `Pdo\Mysql::ATTR_USE_BUFFERED_QUERY`.

`PDO::MYSQL_ATTR_LOCAL_INFILE` (int)

Sinônimo de `Pdo\Mysql::ATTR_LOCAL_INFILE`

`PDO::MYSQL_ATTR_LOCAL_INFILE_DIRECTORY` (int)

Sinônimo de `Pdo\Mysql::ATTR_LOCAL_INFILE_DIRECTORY`. Disponível a partir do PHP 8.1.0.

`PDO::MYSQL_ATTR_INIT_COMMAND` (int)

Sinônimo de `Pdo\Mysql::ATTR_INIT_COMMAND`.

`PDO::MYSQL_ATTR_READ_DEFAULT_FILE` (int)

Sinônimo de `Pdo\Mysql::ATTR_READ_DEFAULT_FILE`.

`PDO::MYSQL_ATTR_READ_DEFAULT_GROUP` (int)

Sinônimo de `Pdo\Mysql::ATTR_READ_DEFAULT_GROUP`.

`PDO::MYSQL_ATTR_MAX_BUFFER_SIZE` (int)

Sinônimo de `Pdo\Mysql::ATTR_MAX_BUFFER_SIZE`.

`PDO::MYSQL_ATTR_DIRECT_QUERY` (bool)

Sinônimo de `PDO::ATTR_EMULATE_PREPARES`.

A partir do PHP 8.4.0, esta constante é tipada como bool; anteriormente, era tipada como int.

`PDO::MYSQL_ATTR_FOUND_ROWS` (int)

Sinônimo de `Pdo\Mysql::ATTR_FOUND_ROWS`.

`PDO::MYSQL_ATTR_IGNORE_SPACE` (int)

Sinônimo de `Pdo\Mysql::ATTR_IGNORE_SPACE`.

`PDO::MYSQL_ATTR_COMPRESS` (int)

Sinônimo de `Pdo\Mysql::ATTR_COMPRESS`.

`PDO::MYSQL_ATTR_SERVER_PUBLIC_KEY` (int)

Sinônimo de `Pdo\Mysql::ATTR_SERVER_PUBLIC_KEY`.

`PDO::MYSQL_ATTR_SSL_CA` (int)

Sinônimo de `Pdo\Mysql::ATTR_SSL_CA`.

`PDO::MYSQL_ATTR_SSL_CAPATH` (int)

Sinônimo de `Pdo\Mysql::ATTR_SSL_CAPATH`.

`PDO::MYSQL_ATTR_SSL_CERT` (int)

Sinônimo de `Pdo\Mysql::ATTR_SSL_CERT`.

`PDO::MYSQL_ATTR_SSL_CIPHER` (int)

Sinônimo de `Pdo\Mysql::ATTR_SSL_CIPHER`.

`PDO::MYSQL_ATTR_SSL_KEY` (int)

Sinônimo de `Pdo\Mysql::ATTR_SSL_KEY`.

`PDO::MYSQL_ATTR_SSL_VERIFY_SERVER_CERT` (int)

Sinônimo de `Pdo\Mysql::ATTR_SSL_VERIFY_SERVER_CERT`. Disponível a partir do PHP 7.0.18 e PHP 7.1.4.

`PDO::MYSQL_ATTR_MULTI_STATEMENTS` (int)

Sinônimo de `Pdo\Mysql::ATTR_MULTI_STATEMENTS`.

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de Configuração do PDO_MYSQL

Nome	Padrão	Modificável
pdo_mysql.default_socket	"/tmp/mysql.sock"	`INI_SYSTEM`
pdo_mysql.debug	NULL	`INI_SYSTEM`

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`pdo_mysql.default_socket` string

Define um soquete de domínio Unix. Este valor também pode ser definido no momento da compilação se um soquete de domínio puder ser encontrado durante o comando "configure". Esta configuração ini é apenas para sistemas baseados em Unix.

`pdo_mysql.debug` bool

Habilita a depuração para o PDO_MYSQL. Esta configuração está disponível somente quando o PDO_MYSQL tiver sido compilado com a mysqlnd e em modo de depuração PDO.

Índice

PDO_MYSQL DSN

PDO_MYSQL DSN

(PECL PDO_MYSQL >= 0.1.0)

PDO_MYSQL DSN — Conectando a bancos de dados MySQL

Descrição

O Data Source Name (DSN) do PDO_MYSQL é composto dos seguintes elementos:

Prefixo DSN

O prefixo DSN é **`mysql:`**.

`host`

O nome do servidor onde está o banco de dados.

`port`

O número da porta onde o servidor do banco de dados está escutando.

`dbname`

O nome do banco de dados.

`user`

Nome do usuário para a conexão. Um nome de usuário informado como segundo argumento ao construtor PDO tem precedência sobre o especificado no DSN.

`password`

Senha para a conexão. Uma senha informada como terceiro argumento ao construtor PDO tem precedência sobre a especificada no DSN.

`unix_socket`

O soquete Unix do MySQL (não deve ser usado com `host` ou `port`).

`charset`

O conjunto de caracteres. Veja a documentação sobre conjunto de caracteres para mais informação.

Examples

Exemplo #1 Exemplos de DSN do PDO_MYSQL

O exemplo a seguir demonstra um DSN do PDO_MYSQL para conexão com banco de dados MySQL:

```
mysql:host=localhost;dbname=testdb
```

Exemplos mais completos:

```
mysql:host=localhost;port=3307;dbname=testdb
mysql:unix_socket=/tmp/mysql.sock;dbname=testdb
```

Notas

> **Nota** : **Somente em Unix:**
>
> Quando o nome do servidor é definido como `"localhost"`, a conexão ao servidor é feita através de um soquete de domínio. Se o PDO_MYSQL tiver sido compilado usando libmysqlclient, a localização do soquete é a mesma localização da compilação da libmysqlclient. Se o PDO_MYSQL foi compilado usando mysqlnd, um soquete padrão pode ser definido através da configuração pdo_mysql.default_socket.

Pdo\Mysql

(PHP 8 >= 8.4.0)

Introdução

Um subclasse PDO representando uma conexão que usa o driver PDO MySQL.

Este driver oferece suporte a um analisador de consulta SQL dedicado para o dialeto MySQL. Ele pode lidar com o seguinte:

- Literais entre aspas simples e duplas com duplicação e barra invertida como mecanismos de escape
- Literais entre acentos graves com duplicação como mecanismo de escape
- Dois travessões, comentários em estilo C e comentários em hash.

Resumo da classe

class **Pdo\Mysql** extends PDO {
/* Constantes herdadas */
public const int *PDO::PARAM_NULL*;
public const int *PDO::PARAM_BOOL* = 5;
public const int *PDO::PARAM_INT* = 1;
public const int *PDO::PARAM_STR* = 2;
public const int *PDO::PARAM_LOB* = 3;
public const int *PDO::PARAM_STMT* = 4;
public const int *PDO::PARAM_INPUT_OUTPUT*;
public const int *PDO::PARAM_STR_NATL*;
public const int *PDO::PARAM_STR_CHAR*;
public const int *PDO::PARAM_EVT_ALLOC*;
public const int *PDO::PARAM_EVT_FREE*;
public const int *PDO::PARAM_EVT_EXEC_PRE*;

public const int *PDO::PARAM_EVT_EXEC_POST*;
public const int *PDO::PARAM_EVT_FETCH_PRE*;
public const int *PDO::PARAM_EVT_FETCH_POST*;
public const int *PDO::PARAM_EVT_NORMALIZE*;
public const int *PDO::FETCH_DEFAULT*;
public const int *PDO::FETCH_LAZY*;
public const int *PDO::FETCH_ASSOC*;
public const int *PDO::FETCH_NUM*;
public const int *PDO::FETCH_BOTH*;
public const int *PDO::FETCH_OBJ*;
public const int *PDO::FETCH_BOUND*;
public const int *PDO::FETCH_COLUMN*;
public const int *PDO::FETCH_CLASS*;
public const int *PDO::FETCH_INTO*;
public const int *PDO::FETCH_FUNC*;
public const int *PDO::FETCH_GROUP*;
public const int *PDO::FETCH_UNIQUE*;
public const int *PDO::FETCH_KEY_PAIR*;
public const int *PDO::FETCH_CLASSTYPE*;
public const int *PDO::FETCH_SERIALIZE*;
public const int *PDO::FETCH_PROPS_LATE*;
public const int *PDO::FETCH_NAMED*;
public const int *PDO::ATTR_AUTOCOMMIT*;
public const int *PDO::ATTR_PREFETCH*;
public const int *PDO::ATTR_TIMEOUT*;
public const int *PDO::ATTR_ERRMODE*;
public const int *PDO::ATTR_SERVER_VERSION*;
public const int *PDO::ATTR_CLIENT_VERSION*;
public const int *PDO::ATTR_SERVER_INFO*;
public const int *PDO::ATTR_CONNECTION_STATUS*;
public const int *PDO::ATTR_CASE*;
public const int *PDO::ATTR_CURSOR_NAME*;
public const int *PDO::ATTR_CURSOR*;
public const int *PDO::ATTR_ORACLE_NULLS*;
public const int *PDO::ATTR_PERSISTENT*;
public const int *PDO::ATTR_STATEMENT_CLASS*;
public const int *PDO::ATTR_FETCH_TABLE_NAMES*;
public const int *PDO::ATTR_FETCH_CATALOG_NAMES*;
public const int *PDO::ATTR_DRIVER_NAME*;
public const int *PDO::ATTR_STRINGIFY_FETCHES*;
public const int *PDO::ATTR_MAX_COLUMN_LEN*;
public const int *PDO::ATTR_EMULATE_PREPARES*;
public const int *PDO::ATTR_DEFAULT_FETCH_MODE*;
public const int *PDO::ATTR_DEFAULT_STR_PARAM*;
public const int *PDO::ERRMODE_SILENT*;
public const int *PDO::ERRMODE_WARNING*;
public const int *PDO::ERRMODE_EXCEPTION*;
public const int *PDO::CASE_NATURAL*;
public const int *PDO::CASE_LOWER*;
public const int *PDO::CASE_UPPER*;
public const int *PDO::NULL_NATURAL*;

```
public const int PDO::NULL_EMPTY_STRING;
public const int PDO::NULL_TO_STRING;
public const string PDO::ERR_NONE;
public const int PDO::FETCH_ORI_NEXT;
public const int PDO::FETCH_ORI_PRIOR;
public const int PDO::FETCH_ORI_FIRST;
public const int PDO::FETCH_ORI_LAST;
public const int PDO::FETCH_ORI_ABS;
public const int PDO::FETCH_ORI_REL;
public const int PDO::CURSOR_FWDONLY;
public const int PDO::CURSOR_SCROLL;
/* Constantes */
public const int ATTR_USE_BUFFERED_QUERY;
public const int ATTR_LOCAL_INFILE;
public const int ATTR_LOCAL_INFILE_DIRECTORY;
public const int ATTR_INIT_COMMAND;
public const int ATTR_MAX_BUFFER_SIZE;
public const int ATTR_READ_DEFAULT_FILE;
public const int ATTR_READ_DEFAULT_GROUP;
public const int ATTR_COMPRESS;
public const int ATTR_DIRECT_QUERY;
public const int ATTR_FOUND_ROWS;
public const int ATTR_IGNORE_SPACE;
public const int ATTR_MULTI_STATEMENTS;
public const int ATTR_SERVER_PUBLIC_KEY;
public const int ATTR_SSL_KEY;
public const int ATTR_SSL_CERT;
public const int ATTR_SSL_CA;
public const int ATTR_SSL_CAPATH;
public const int ATTR_SSL_CIPHER;
public const int ATTR_SSL_VERIFY_SERVER_CERT;
/* Métodos */
public function getWarningCount(): int
/* Métodos herdados */
public function PDO::__construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
public function PDO::beginTransaction(): bool
public function PDO::commit(): bool
public static function PDO::connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
public function PDO::errorCode(): ?string
public function PDO::errorInfo(): array
public function PDO::exec(string $statement): int|false
```

```
public function PDO::getAttribute(int $attribute): mixed
public static function PDO::getAvailableDrivers(): array
public function PDO::inTransaction(): bool
public function PDO::lastInsertId(?string $name = null): string|false
public function PDO::prepare(string $query, array $options = []): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = null): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_COLUMN, int $colno):
PDOStatement|false
public function PDO::query(
    string $query,
    ?int $fetchMode = PDO::FETCH_CLASS,
    string $classname,
    array $constructorArgs
): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_INTO, object $object):
PDOStatement|false
public function PDO::quote(string $string, int $type = PDO::PARAM_STR): string|false
public function PDO::rollBack(): bool
public function PDO::setAttribute(int $attribute, mixed $value): bool
}
```

Constantes predefinidas

`Pdo\Mysql::ATTR_USE_BUFFERED_QUERY`

Por padrão, todas as instruções são executadas em modo buffer. Se este atributo estiver definido como `false` em um objeto **Pdo\Mysql**, o driver MySQL usará o modo sem buffer.

Exemplo #1 Configurando o modo sem buffer do MySQL

```php
<?php
$pdo = new Pdo\Mysql("mysql:host=localhost;dbname=world", 'my_user', 'my_password');
$pdo->setAttribute(PDO::MYSQL_ATTR_USE_BUFFERED_QUERY, false);

$unbufferedResult = $pdo->query("SELECT Name FROM City");
foreach ($unbufferedResult as $row) {
    echo $row['Name'] . PHP_EOL;
}
?>
```

`Pdo\Mysql::ATTR_LOCAL_INFILE`

Habilita `LOAD LOCAL INFILE`.

> **Nota** :
>
> Só pode ser usado no array `driver_options` ao construir um novo identificador de banco de dados.

`Pdo\Mysql::ATTR_LOCAL_INFILE_DIRECTORY`

Permite restringir o carregamento de LOCAL DATA a arquivos localizados neste diretório designado.

> **Nota** :

> Só pode ser usado no array `driver_options` ao construir um novo identificador de banco de dados.

Pdo\Mysql::ATTR_INIT_COMMAND

Comando a ser executado ao conectar-se ao servidor MySQL. Será reexecutado automaticamente ao reconectar.

> **Nota** :
>
> Só pode ser usado no array `driver_options` ao construir um novo identificador de banco de dados.

Pdo\Mysql::ATTR_READ_DEFAULT_FILE

Lê as opções do arquivo de opções nomeado em vez de *my.cnf*.

> **Nota** :
>
> Esta opção não está disponível se mysqlnd for usado, porque o mysqlnd não lê os arquivos de configuração do mysql.

Pdo\Mysql::ATTR_READ_DEFAULT_GROUP

Lê opções do grupo nomeado de *my.cnf* ou do arquivo especificado com **Pdo\Mysql::ATTR_READ_DEFAULT_FILE**.

> **Nota** :
>
> Esta opção não está disponível se mysqlnd for usado, porque o mysqlnd não lê os arquivos de configuração do mysql.

Pdo\Mysql::ATTR_COMPRESS

Ativa a compactação de comunicação de rede.

Pdo\Mysql::ATTR_DIRECT_QUERY

Sinônimo de **PDO::ATTR_EMULATE_PREPARES**.

Pdo\Mysql::ATTR_FOUND_ROWS

Retorna o número de linhas encontradas (correspondidas), não o número de linhas alteradas.

> **Nota** :
>
> Só pode ser usado no array `driver_options` ao construir um novo identificador de banco de dados.

Pdo\Mysql::ATTR_IGNORE_SPACE

Permite espaços após nomes de funções SQL. Torna todos os nomes de funções SQL palavras reservadas.

> **Nota** :
>
> Só pode ser usado no array `driver_options` ao construir um novo identificador de banco de dados.

`Pdo\Mysql::ATTR_MAX_BUFFER_SIZE`

Tamanho máximo do buffer. O padrão é 1 MiB.

> **Nota** :
>
> Esta constante não é suportada quando compilada no mysqlnd.

`Pdo\Mysql::ATTR_MULTI_STATEMENTS`

Desativa a execução de múltiplas consultas em PDO::prepare() e PDO::query() quando definido como **`false`**.

> **Nota** :
>
> Só pode ser usado no array `driver_options` ao construir um novo identificador de banco de dados.

`Pdo\Mysql::ATTR_SERVER_PUBLIC_KEY`

Arquivo de chave pública RSA usado com autenticação baseada em SHA-256.

> **Nota** :
>
> Só pode ser usado no array `driver_options` ao construir um novo identificador de banco de dados.

`Pdo\Mysql::ATTR_SSL_KEY`

O caminho do arquivo para a chave SSL.

> **Nota** :
>
> Só pode ser usado no array `driver_options` ao construir um novo identificador de banco de dados.

`Pdo\Mysql::ATTR_SSL_CERT`

O caminho do arquivo para o certificado SSL.

> **Nota** :
>
> Só pode ser usado no array `driver_options` ao construir um novo identificador de banco de dados.

`Pdo\Mysql::ATTR_SSL_CA`

O caminho do arquivo para a autoridade de certificação SSL.

> **Nota** :
>
> Só pode ser usado no array `driver_options` ao construir um novo identificador de banco de dados.

Pdo\Mysql::ATTR_SSL_CAPATH

O caminho do arquivo para o diretório que contém os certificados CA SSL confiáveis, que são armazenados no formato PEM.

> **Nota** :
>
> Só pode ser usado no array `driver_options` ao construir um novo identificador de banco de dados.

Pdo\Mysql::ATTR_SSL_CIPHER

Uma lista de uma ou mais cifras permitidas para uso na criptografia SSL, em um formato compreendido pelo OpenSSL. Por exemplo, `DHE-RSA-AES256-SHA:AES128-SHA`.

> **Nota** :
>
> Só pode ser usado no array `driver_options` ao construir um novo identificador de banco de dados.

Pdo\Mysql::ATTR_SSL_VERIFY_SERVER_CERT

Fornece uma maneira de desabilitar a verificação do certificado SSL do servidor.

> **Nota** :
>
> Esta opção está disponível apenas com mysqlnd.

> **Nota** :
>
> Só pode ser usado no array `driver_options` ao construir um novo identificador de banco de dados.

Índice

Pdo\Mysql::getWarningCount

Pdo\Mysql::getWarningCount

(PHP 8 >= 8.4.0)

Pdo\Mysql::getWarningCount — Retorna o número de alertas da última consulta executada

Descrição

public function **Pdo\Mysql::getWarningCount**(): int

Retorna o número de alertas da última consulta executada.

> **Nota** :
>
> Para recuperar mensagens de alerta, o seguinte comando SQL pode ser usado: `SHOW WARNINGS [limit row_count]`, onde "row_count" é o número máximo de mensagens a serem retornadas.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna um int representando o número de avisos gerados pela última consulta.

Examples

Exemplo #1 Exemplo de Pdo\Mysql::getWarningCount()

```php
<?php

$conn = PDO::connect("mysql:host=localhost;dbname=test;charset=utf8mb4", 'user', 'password');

$conn->query('SELECT 42/0');
if ($conn->getWarningCount() > 0) {
    $result = $conn->query("SHOW WARNINGS");
    $row = $result->fetch();
    printf("%s (%d): %s\n", $row[0], $row[1], $row[2]);
}

?>
```

O exemplo acima produzirá:

```
Warning (1365): Division by 0
```

Driver PDO do Microsoft SQL Server

Introdução

PDO_SQLSRV é um driver que implementa a interface aos Objetos de Dados PHP (PDO) para permitir acesso do PHP a bancos de dados MS SQL Server (iniciando com o SQL Server 2005) e SQL Azure.

Instalação

A versão mais recente do driver está disponível aqui: » SQLSRV. O código-fonte do driver está hospedado em um » repositório público.

Para mais informações sobre requisitos do sistema, consulte » Requisitos de Sistema para o SQLSRV.

No Windows, a extensão PDO_SQLSRV é habilitado pela adição dos arquivos DLL apropriados ao diretório de extensões do PHP e da entrada correspondente no arquivo *php.ini*.

No Linux e no macOS, a extensão PDO_SQLSRV pode ser instalada usando o » PECL. Consulte o » tutorial de instalação para detalhes.

Constantes predefinidas

As constantes abaixo são definidas por este driver e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução. Além disso, estas constantes específicas do driver só devem ser usadas se este driver estiver sendo usado. Usar atributos específicos de um driver com outro driver pode resultar em comportamento inesperado. PDO::getAttribute() pode ser usada para obter o atributo **`PDO::ATTR_DRIVER_NAME`** para verificar o driver, se o código puder ser executado com vários drivers.

`PDO::SQLSRV_TXN_READ_UNCOMMITTED` (int)

Essa constante é um valor aceitável para a chave TransactionIsolation do DSN SQLSRV. Esta constante define o nível de isolamento da transação para a conexão como Read Uncommitted.

`PDO::SQLSRV_TXN_READ_COMMITTED` (int)

Essa constante é um valor aceitável para a chave TransactionIsolation do DSN SQLSRV. Esta constante define o nível de isolamento da transação para a conexão como Leitura Confirmada.

`PDO::SQLSRV_TXN_REPEATABLE_READ` (int)

Essa constante é um valor aceitável para a chave TransactionIsolation do DSN SQLSRV. Esta constante define o nível de isolamento da transação para a conexão como Leitura Repetível.

`PDO::SQLSRV_TXN_SNAPSHOT` (int)

Essa constante é um valor aceitável para a chave TransactionIsolation do DSN SQLSRV. Esta constante define o nível de isolamento da transação para a conexão como Snapshot.

`PDO::SQLSRV_TXN_SERIALIZABLE` (int)

Essa constante é um valor aceitável para a chave TransactionIsolation do DSN SQLSRV. Esta constante define o nível de isolamento da transação para a conexão como Serializável.

`PDO::SQLSRV_ENCODING_BINARY` (int)

Especifica que os dados são enviados/recuperados como um fluxo de bytes brutos de/para o servidor sem realizar codificação ou tradução. Essa constante pode ser passada para PDOStatement::setAttribute, PDO::prepare, PDOStatement::bindColumn e PDOStatement::bindParam.

`PDO::SQLSRV_ENCODING_SYSTEM` (int)

Especifica que os dados são enviados/recuperados de/para o servidor como caracteres de 8 bits, conforme especificado na página de código da localidade do Windows definida no sistema. Quaisquer caracteres multibyte ou caracteres que não sejam mapeados nesta página de códigos serão substituídos por um caractere de ponto de interrogação (?) de byte único. Essa constante pode ser passada para PDOStatement::setAttribute, PDO::setAttribute, PDO::prepare, PDOStatement::bindColumn e PDOStatement::bindParam.

`PDO::SQLSRV_ENCODING_UTF8` (int)

Especifica que os dados são enviados/recuperados de/para o servidor na codificação UTF-8. Esta é a codificação padrão. Essa constante pode ser passada para PDOStatement::setAttribute, PDO::setAttribute, PDO::prepare, PDOStatement::bindColumn e PDOStatement::bindParam.

PDO::SQLSRV_ENCODING_DEFAULT (int)

Especifica que os dados são enviados/recuperados de/para o servidor de acordo com PDO::SQLSRV_ENCODING_SYSTEM se especificado durante a conexão. A codificação da conexão é usada se especificada em uma instrução de preparação. Essa constante pode ser passada para PDOStatement::setAttribute, PDO::setAttribute, PDO::prepare, PDOStatement::bindColumn e PDOStatement::bindParam.

PDO::SQLSRV_ATTR_QUERY_TIMEOUT (int)

Um número inteiro não negativo que representa o período de tempo limite, em segundos. Zero (0) é o padrão e significa que não há tempo limite. Essa constante pode ser passada para PDOStatement::setAttribute, PDO::setAttribute e PDO::prepare.

PDO::SQLSRV_ATTR_DIRECT_QUERY (int)

Indica que uma instrução deve ser executada diretamente, sem estar preparada. Essa constante pode ser passada para PDO::setAttribute e PDO::prepare. Para obter mais informações, consulte » Execução de instrução direta e preparada.

Índice

PDO_SQLSRV DSN

PDO_SQLSRV DSN

(PECL pdo_sqlsrv >= 2.0.1)

PDO_SQLSRV DSN — Conectando-se aos bancos de dados MS SQL Server e SQL Azure

Descrição

O Nome da Fonte de Dados (DSN) PDO_SQLSRV é composto pelos seguintes elementos:

DSN prefix

O prefixo DSN é **sqlsrv:**.

APP

O nome da aplicação usada no rastreamento.

ConnectionPooling

Especifica se a conexão é atribuída a partir de um grupo de conexões (1 ou **true**) ou não (0 ou **false**).

Database

O nome do banco de dados.

Encrypt

Especifica se a comunicação com o SQL Server é criptografada (1 ou `true`) ou não criptografada (0 ou `false`).

`Failover_Partner`

Especifica o servidor e a instância do espelho do banco de dados (se habilitado e configurado) a serem usados quando o servidor primário estiver indisponível.

`LoginTimeout`

Especifica o número de segundos de espera antes de falhar a tentativa de conexão.

`MultipleActiveResultSets`

Desativa ou ativa explicitamente o suporte para Conjuntos de Resultados Ativos Múltiplos (MARS).

`QuotedId`

Especifica se devem ser usadas regras SQL-92 para identificadores entre aspas (1 ou `true`) ou regras Transact-SQL legadas (0 ou `false`).

`Server`

O nome do servidor de banco de dados.

`TraceFile`

Especifica o caminho do arquivo usado para dados de rastreamento.

`TraceOn`

Especifica se o rastreamento ODBC está habilitado (1 ou `true`) ou desabilitado (0 ou `false`) para a conexão que está sendo estabelecida.

`TransactionIsolation`

Especifica o nível de isolamento da transação. Os valores aceitos para esta opção são PDO::SQLSRV_TXN_READ_UNCOMMITTED, PDO::SQLSRV_TXN_READ_COMMITTED, PDO::SQLSRV_TXN_REPEATABLE_READ, PDO::SQLSRV_TXN_SNAPSHOT e PDO::SQLSRV_TXN_SERIALIZABLE.

`TrustServerCertificate`

Especifica se o cliente deve confiar (1 ou `true`) ou rejeitar (0 ou `false`) um certificado de servidor autoassinado.

`WSID`

Especifica o nome do computador para rastreamento.

Examples

Exemplo #1 Exemplos de DSN PDO_SQLSRV

O exemplo a seguir mostra como se conectar a um banco de dados MS SQL Server especificado:

```
$c = new PDO("sqlsrv:Server=localhost;Database=db_teste", "NomeUsuario", "Senha");
```

O exemplo a seguir mostra como conectar-se a um banco de dados MS SQL Server em uma porta especificada:

```
$c = new PDO("sqlsrv:Server=localhost,1521;Database=db_teste", "NomeUsuario", "Senha");
```

O exemplo a seguir mostra como se conectar a um banco de dados SQL Azure com ID de servidor 12345abcde. Observe que quando há uma conexão ao SQL Azure com PDO, o nome de usuário será NomeUsuario@12345abcde (NomeUsuario@IdServidor).

```
$c = new PDO("sqlsrv:Server=12345abcde.database.windows.net;Database=db_teste",
"NomeUsuario@12345abcde", "Senha");
```

Driver PDO do Oracle

Instalação

Se o banco de dados Oracle estiver na mesma máquina que o PHP, o software de banco de dados já contém as bibliotecas necessárias. Quando o PHP estiver em uma máquina diferente, use as bibliotecas gratuitas » Oracle Instant Client. Para obter detalhes, consulte a seção Requisitos do OCI8.

PHP 8.4

Esta extensão foi movida para o repositório » PECL e não é mais distribuída com o PHP desde a versão 8.4.0

Informações para instalar esta extensão PECL podem ser encontradas no capítulo Instalação de Extensões PECL do manual. Informações adicionais, como novas versões, downloads, arquivos fontes, informações do mantenedor e um CHANGELOG, podem ser encontradas aqui: » https://pecl.php.net/package/PDO_OCI.

PHP < 8.4

Use **--with-pdo-oci[=DIR]** para instalar a extensão PDO Oracle OCI, onde o `[=DIR]` opcional é o diretório base do Oracle. O padrão de `[=DIR]` é a variável de ambiente *$ORACLE_HOME*.

Use **--with-pdo-oci=instantclient,prefix,version** para um SDK do Oracle Instant Client, onde o prefixo e a versão são configurados.

```
// Usando $ORACLE_HOME
$ ./configure --with-pdo-oci

// Usando OIC para Linux com RPMs 10.2.0.3 e prefixo /usr
$ ./configure --with-pdo-oci=instantclient,/usr,10.2.0.3
```

Constantes predefinidas

As constantes abaixo são definidas por este driver e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução. Além disso, estas constantes específicas do driver só devem ser usadas se este driver estiver sendo usado. Usar atributos específicos de um driver com outro driver pode resultar em comportamento inesperado. PDO::getAttribute() pode ser usada para obter o atributo **`PDO::ATTR_DRIVER_NAME`** para verificar o driver, se o código puder ser executado com vários drivers.

`PDO::OCI_ATTR_ACTION` (int)

Fornece uma maneira de especificar a ação na sessão do banco de dados.

Isto existe a partir do PHP 7.2.16 e 7.3.3

PDO::OCI_ATTR_CLIENT_INFO (int)

Fornece uma maneira de especificar as informações do cliente na sessão do banco de dados.

Isto existe a partir do PHP 7.2.16 e 7.3.3

PDO::OCI_ATTR_CLIENT_IDENTIFIER (int)

Fornece uma maneira de especificar o identificador do cliente na sessão do banco de dados.

Isto existe a partir do PHP 7.2.16 e 7.3.3

PDO::OCI_ATTR_MODULE (int)

Fornece uma maneira de especificar o módulo na sessão do banco de dados.

Isto existe a partir do PHP 7.2.16 e 7.3.3

Índice

PDO_OCI DSN

PDO_OCI DSN

(PECL PDO_OCI >= 0.1.0)

PDO_OCI DSN — Conectando-se a bancos de dados Oracle

Descrição

O Nome da Fonte de Dados (DSN) PDO_OCI é composto pelos seguintes elementos:

DSN prefix

O prefixo DSN é **`oci:`**....

`dbname` (Oracle Instant Client)

O URI para a conexão do Oracle Instant Client assume o formato `dbname=//hostname:port-number/database`. Se for feita uma conexão um banco de dados definido em *tnsnames.ora*, apenas o nome do banco de dados deve ser usado: `dbname=database`.

`charset`

O conjunto de caracteres do lado do cliente para o identificador do ambiente atual.

Examples

Exemplo #1 Exemplos de DSN PDO_OCI

Os exemplos a seguir mostram um DSN PDO_OCI para conexão com bancos de dados Oracle:

```
// Conecta a um banco de dados definido em tnsnames.ora
oci:dbname=mydb
```

```
// Conecta usando o Oracle Instant Client
oci:dbname=//localhost:1521/mydb
```

Driver PDO do ODBC e do DB2

Introdução

PDO_ODBC é um driver que implementa a interface aos Objetos de Dados PHP (PDO) para permitir acesso do PHP a bancos de dados através de drivers ODBC ou através da biblioteca IBM DB2 Call Level Interface (DB2 CLI). O PDO_ODBC atualmente suporta três diferentes "sabores" de drivers de banco de dados:

ibm-db2

> Suporta acesso aos servidores IBM DB2 Universal Database, Cloudscape e Apache Derby por meio do cliente DB2 express-C gratuito.

unixODBC

> Suporta acesso a servidores de banco de dados por meio do gerenciador de driver unixODBC e dos próprios drivers ODBC do banco de dados.

generic

> Oferece uma opção de compilação para gerenciadores de driver ODBC que não são explicitamente suportados por PDO_ODBC.

No Windows, *php_pdo_odbc.dll* deve estar habilitado como extensão no *php.ini*. Ele está vinculado ao Windows ODBC Driver Manager para que o PHP possa se conectar a qualquer banco de dados catalogado como um DSN de sistema.

Instalação

PDO_ODBC em sistemas UNIX

1. PDO_ODBC está incluído no código-fonte do PHP. Pode-se compilar a extensão PDO_ODBC como um módulo estático ou compartilhado usando os seguintes comandos **configure**.

   ibm_db2

   ```
   ./configure --with-pdo-odbc=ibm-db2,/opt/IBM/db2/V8.1/
   ```

   Para construir o PDO_ODBC com o tipo ibm-db2, é necessário ter os cabeçalhos de desenvolvimento do aplicativo DB2 instalados previamente na mesma máquina em que o PDO_ODBC estiver sendo compilado. Os cabeçalhos de desenvolvimento de aplicativos DB2 são uma opção instalável nos servidores DB2 e também estão disponíveis como parte do DB2 Application Development Client, gratuitamente no » site do IBM developerWorks.

   Se não for informado um local para as bibliotecas e cabeçalhos do DB2 para o comando **configure**, o PDO_ODBC usará o caminho padrão */home/db2inst1/sqllib*.

   unixODBC

```
./configure --with-pdo-odbc=unixODBC,/usr/local
```

Se não for informado um local para as bibliotecas e cabeçalhos unixODBC para o comando **configure**, o PDO_ODBC usará o caminho padrão */usr/local*.

generic

```
./configure --with-pdo-odbc=generic,/usr/local,libname,ldflags,cflags
```

Constantes predefinidas

As constantes abaixo são definidas por este driver e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução. Além disso, estas constantes específicas do driver só devem ser usadas se este driver estiver sendo usado. Usar atributos específicos de um driver com outro driver pode resultar em comportamento inesperado. PDO::getAttribute() pode ser usada para obter o atributo `PDO::ATTR_DRIVER_NAME` para verificar o driver, se o código puder ser executado com vários drivers.

`PDO_ODBC_TYPE` (string)

Descreve a biblioteca ODBC vinculada à extensão PDO_ODBC. Os valores possíveis incluem `unixODBC`, `iODBC` ou `generic`.

> **Aviso**
>
> As constantes listadas abaixo foram *DESCONTINUADAS* a partir do PHP 8.5.0. Use as constantes correspondentes de Pdo\Odbc no lugar delas.

`PDO::ODBC_ATTR_USE_CURSOR_LIBRARY` (int)

Sinônimo de `Pdo\Odbc::ATTR_USE_CURSOR_LIBRARY`.

`PDO::ODBC_SQL_USE_IF_NEEDED` (int)

Sinônimo de `Pdo\Odbc::SQL_USE_IF_NEEDED`.

`PDO::ODBC_SQL_USE_DRIVER` (int)

Sinônimo de `Pdo\Odbc::SQL_USE_DRIVER`.

`PDO::ODBC_SQL_USE_ODBC` (int)

Sinônimo de `Pdo\Odbc::SQL_USE_ODBC`.

`PDO::ODBC_ATTR_ASSUME_UTF8` (bool)

Sinônimo de `Pdo\Odbc::ATTR_ASSUME_UTF8`.

Configurações de Execução

O comportamento destas funções é afetado pelas configurações do *php.ini*.

Opções de Configuração do PDO_ODBC

Nome	Padrão	Modificável	Registro de Alterações
pdo_odbc.connection_pooling	"strict"	INI_ALL	
pdo_odbc.db2_instance_name	NULL	INI_SYSTEM	Este recurso descontinuado *será* certamente *removido* no futuro.

Para mais detalhes e definições dos modos INI_*, consulte os Onde uma configuração deve ser definida.

Aqui está uma breve explicação das diretivas de configuração.

`pdo_odbc.connection_pooling` string

Informa se as conexões ODBC devem ser agrupadas. Pode ser `"strict"`, `"relaxed"` ou `"off"` (igual a `""`). O parâmetro descreve quão rigoroso o gerenciador de conexões deve ser ao comparar parâmetros de conexão com conexões agrupadas existentes. **`strict`** é o padrão recomendado e resultará no uso de conexões em cache somente quando todos os parâmetros de conexão tiverem correspondência exata. **`relaxed`** resultará no uso de conexões em cache quando parâmetros de conexão semelhantes forem usados. Isso pode resultar no aumento do uso do cache, correndo o risco de sangrar informações de conexão entre (por exemplo) servidores virtuais.

Esta configuração só pode ser alterada no arquivo *php.ini* e afeta todo o processo; quaisquer outros módulos carregados no processo que usem as mesmas bibliotecas ODBC também serão afetados, incluindo a extensão ODBC unificada.

> **Aviso**
>
> A correspondência **`relaxed`** não deve ser usada em um servidor compartilhado, por motivos de segurança.

> **Dica**
>
> Esta configuração deve ser mantida na configuração padrão **`strict`** a menos que exista um bom motivo para alterá-la.

`pdo_odbc.db2_instance_name` string

Se você o PDO_ODBC for compilado usando o tipo `db2`, esta configuração definirá o valor da variável de ambiente DB2INSTANCE nos sistemas operacionais Linux e UNIX para o nome especificado da instância do DB2. Isso permite que o PDO_ODBC resolva o local das bibliotecas DB2 e estabeleça conexões catalogadas com bancos de dados DB2.

Esta configuração só pode ser alterada no arquivo *php.ini* e afeta todo o processo; quaisquer outros módulos carregados no processo que usem as mesmas bibliotecas ODBC também serão afetados, incluindo a extensão ODBC unificada.

Esta configuração não tem efeito no Windows.

Índice

PDO_ODBC DSN

PDO_ODBC DSN

(PECL PDO_ODBC >= 0.1.0)

PDO_ODBC DSN — Conectando-se a bancos de dados ODBC ou DB2

Descrição

O Nome da Fonte de Dados (DSN) PDO_ODBC é composto pelos seguintes elementos:

Prefixo DSN

> O prefixo DSN é **`odbc:`**. Se estiver sendo feita uma conexão a um banco de dados catalogado no gerenciador de driver ODBC ou no catálogo do DB2, pode-se anexar o nome catalogado do banco de dados ao DSN.

DSN

> O nome do banco de dados conforme catalogado no gerenciador de driver ODBC ou no catálogo do DB2. Como alternativa, pode-se fornecer uma string de conexão ODBC completa para conectar-se a um banco de dados, conforme descrito em » http://www.connectionstrings.com/.

`UID`

> O nome do usuário para a conexão. Se for especificado o nome de usuário no DSN, o PDO ignorará o valor do argumento do nome de usuário no construtor do PDO.

`PWD`

> A senha do usuário para a conexão. Se for especificada a senha no DSN, o PDO ignorará o valor do argumento de senha no construtor do PDO.

Registro de Alterações

Versão	Descrição
8.4.0	Ao passar uma string vazia para o argumento de senha no construtor PDO, `pwd` não era incluído na string de conexão criada até agora, mas o comportamento foi alterado para incluí-lo como uma string vazia. Passando **`null`** para o argumento de senha no construtor PDO resulta no mesmo comportamento de antes.
8.4.0	Alterado o comportamento para ignorar o argumento do nome de usuário e o argumento da senha no construtor PDO separadamente quando o DSN contém `uid` ou `pwd`. Anteriormente, se incluído apenas `uid` ou `pwd` no DSN, tanto o argumento do nome de usuário quanto o argumento da senha no construtor PDO eram ignorados.

Examples

Exemplo #1 Exemplo de PDO_ODBC DSN (gerenciador de driver ODBC)

O exemplo a seguir mostra um DSN PDO_ODBC para conexão a um banco de dados ODBC catalogado como testdb no gerenciador de driver ODBC:

```
odbc:testdb
```

Exemplo #2 Exemplo de DSN PDO_ODBC (conexão não catalogada do IBM DB2)

O exemplo a seguir mostra um DSN PDO_ODBC para conexão a um banco de dados IBM DB2 denominado **`SAMPLE`** usando a sintaxe DSN ODBC completa:

```
odbc:DRIVER={IBM DB2 ODBC
DRIVER};HOSTNAME=localhost;PORT=50000;DATABASE=SAMPLE;PROTOCOL=TCPIP;UID=db2inst1;PWD=ibmdb2;
```

Exemplo #3 Exemplo de PDO_ODBC DSN (conexão não catalogada do Microsoft Access)

O exemplo a seguir mostra um DSN PDO_ODBC para conexão a um banco de dados Microsoft Access armazenado em **`C:\db.mdb`** usando a sintaxe DSN ODBC completa:

```
odbc:Driver={Microsoft Access Driver (*.mdb)};Dbq=C:\\db.mdb;Uid=Admin
```

Pdo\Odbc

(PHP 8 >= 8.4.0)

Introdução

Uma subclasse PDO representando uma conexão que usa o driver PDO ODBC.

Resumo da classe

namespace **Pdo**;
class **Odbc** extends PDO {
/* Constantes herdadas */
public const int *PDO::PARAM_NULL*;
public const int *PDO::PARAM_BOOL* = 5;
public const int *PDO::PARAM_INT* = 1;
public const int *PDO::PARAM_STR* = 2;
public const int *PDO::PARAM_LOB* = 3;
public const int *PDO::PARAM_STMT* = 4;
public const int *PDO::PARAM_INPUT_OUTPUT*;
public const int *PDO::PARAM_STR_NATL*;
public const int *PDO::PARAM_STR_CHAR*;
public const int *PDO::PARAM_EVT_ALLOC*;
public const int *PDO::PARAM_EVT_FREE*;
public const int *PDO::PARAM_EVT_EXEC_PRE*;
public const int *PDO::PARAM_EVT_EXEC_POST*;
public const int *PDO::PARAM_EVT_FETCH_PRE*;
public const int *PDO::PARAM_EVT_FETCH_POST*;
public const int *PDO::PARAM_EVT_NORMALIZE*;
public const int *PDO::FETCH_DEFAULT*;
public const int *PDO::FETCH_LAZY*;
public const int *PDO::FETCH_ASSOC*;
public const int *PDO::FETCH_NUM*;
public const int *PDO::FETCH_BOTH*;
public const int *PDO::FETCH_OBJ*;
public const int *PDO::FETCH_BOUND*;
public const int *PDO::FETCH_COLUMN*;
public const int *PDO::FETCH_CLASS*;
public const int *PDO::FETCH_INTO*;
public const int *PDO::FETCH_FUNC*;
public const int *PDO::FETCH_GROUP*;
public const int *PDO::FETCH_UNIQUE*;

```
public const int PDO::FETCH_KEY_PAIR;
public const int PDO::FETCH_CLASSTYPE;
public const int PDO::FETCH_SERIALIZE;
public const int PDO::FETCH_PROPS_LATE;
public const int PDO::FETCH_NAMED;
public const int PDO::ATTR_AUTOCOMMIT;
public const int PDO::ATTR_PREFETCH;
public const int PDO::ATTR_TIMEOUT;
public const int PDO::ATTR_ERRMODE;
public const int PDO::ATTR_SERVER_VERSION;
public const int PDO::ATTR_CLIENT_VERSION;
public const int PDO::ATTR_SERVER_INFO;
public const int PDO::ATTR_CONNECTION_STATUS;
public const int PDO::ATTR_CASE;
public const int PDO::ATTR_CURSOR_NAME;
public const int PDO::ATTR_CURSOR;
public const int PDO::ATTR_ORACLE_NULLS;
public const int PDO::ATTR_PERSISTENT;
public const int PDO::ATTR_STATEMENT_CLASS;
public const int PDO::ATTR_FETCH_TABLE_NAMES;
public const int PDO::ATTR_FETCH_CATALOG_NAMES;
public const int PDO::ATTR_DRIVER_NAME;
public const int PDO::ATTR_STRINGIFY_FETCHES;
public const int PDO::ATTR_MAX_COLUMN_LEN;
public const int PDO::ATTR_EMULATE_PREPARES;
public const int PDO::ATTR_DEFAULT_FETCH_MODE;
public const int PDO::ATTR_DEFAULT_STR_PARAM;
public const int PDO::ERRMODE_SILENT;
public const int PDO::ERRMODE_WARNING;
public const int PDO::ERRMODE_EXCEPTION;
public const int PDO::CASE_NATURAL;
public const int PDO::CASE_LOWER;
public const int PDO::CASE_UPPER;
public const int PDO::NULL_NATURAL;
public const int PDO::NULL_EMPTY_STRING;
public const int PDO::NULL_TO_STRING;
public const string PDO::ERR_NONE;
public const int PDO::FETCH_ORI_NEXT;
public const int PDO::FETCH_ORI_PRIOR;
public const int PDO::FETCH_ORI_FIRST;
public const int PDO::FETCH_ORI_LAST;
public const int PDO::FETCH_ORI_ABS;
public const int PDO::FETCH_ORI_REL;
public const int PDO::CURSOR_FWDONLY;
public const int PDO::CURSOR_SCROLL;
/* Constantes */
public const int ATTR_USE_CURSOR_LIBRARY;
public const int ATTR_ASSUME_UTF8;
public const int SQL_USE_IF_NEEDED;
public const int SQL_USE_DRIVER;
public const int SQL_USE_ODBC;
```

```
/* Métodos herdados */
public function PDO::__construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
public function PDO::beginTransaction(): bool
public function PDO::commit(): bool
public static function PDO::connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
public function PDO::errorCode(): ?string
public function PDO::errorInfo(): array
public function PDO::exec(string $statement): int|false
public function PDO::getAttribute(int $attribute): mixed
public static function PDO::getAvailableDrivers(): array
public function PDO::inTransaction(): bool
public function PDO::lastInsertId(?string $name = null): string|false
public function PDO::prepare(string $query, array $options = []): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = null): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_COLUMN, int $colno):
PDOStatement|false
public function PDO::query(
    string $query,
    ?int $fetchMode = PDO::FETCH_CLASS,
    string $classname,
    array $constructorArgs
): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_INTO, object $object):
PDOStatement|false
public function PDO::quote(string $string, int $type = PDO::PARAM_STR): string|false
public function PDO::rollBack(): bool
public function PDO::setAttribute(int $attribute, mixed $value): bool
}
```

Constantes predefinidas

`Pdo\Odbc::ATTR_USE_CURSOR_LIBRARY`

Esta opção controla se a biblioteca de cursores ODBC é usada. A biblioteca de cursores ODBC oferece suporte a alguns recursos ODBC avançados (por exemplo, cursores roláveis em bloco), que podem não ser implementados pelo driver. Os seguintes valores são suportados:

`Pdo\Odbc::SQL_USE_IF_NEEDED`

Usa a biblioteca de cursores ODBC quando necessário. Este é o padrão.

`Pdo\Odbc::SQL_USE_DRIVER`

Nunca usa a biblioteca de cursores ODBC.

`Pdo\Odbc::SQL_USE_ODBC`

Sempre usa a biblioteca de cursores ODBC.

`Pdo\Odbc::ATTR_ASSUME_UTF8`

Somente Windows. Se `true`, os dados de caracteres codificados em UTF-16 (`CHAR`, `VARCHAR` e `LONGVARCHAR`) serão convertidos para UTF-8 ao ler ou gravar dados no banco de dados. Se `false` (o padrão), a conversão da codificação de caracteres pode ser feita pelo driver.

Driver PDO do PostgreSQL

Introdução

PDO_PGSQL é um driver que implementa a interface PHP Data Objects (PDO) para permitir o acesso do PHP aos bancos de dados PostgreSQL.

Tipos de Recurso

Esta extensão define um recurso de fluxo retornado por Pdo\Pgsql::lobOpen().

Instalação

Use **--with-pdo-pgsql[=DIR]** para instalar a extensão PDO PostgreSQL, onde o opcional `[=DIR]` é o diretório base de instalação do PostgreSQL ou o caminho para *pg_config*.

```
$ ./configure --with-pdo-pgsql
```

Esta extensão requer libpq (a biblioteca cliente C do PostgreSQL). A partir do PHP 8.4.0, libpq 10.0 ou superior é exigido.

Constantes predefinidas

As constantes abaixo são definidas por este driver e só estarão disponíveis quando a extensão tiver sido compilada no PHP ou tiver sido carregada dinamicamente em tempo de execução. Além disso, estas constantes específicas do driver só devem ser usadas se este driver estiver sendo usado. Usar atributos específicos de um driver com outro driver pode resultar em comportamento inesperado. PDO::getAttribute() pode ser usada para obter o atributo `PDO::ATTR_DRIVER_NAME` para verificar o driver, se o código puder ser executado com vários drivers.

`PDO::PGSQL_ATTR_DISABLE_PREPARES` (int)

Sinônimo de `Pdo\Pgsql::ATTR_DISABLE_PREPARES`. A partir do PHP 8.5.0, esta constante está descontinuada.

`PDO::PGSQL_TRANSACTION_IDLE` (int)

Equivalente a `Pdo\Pgsql::TRANSACTION_IDLE`. A partir do PHP 8.5.0, esta constante está descontinuada, pois não tem efeito.

`PDO::PGSQL_TRANSACTION_ACTIVE` (int)

Equivalente a **Pdo\Pgsql::TRANSACTION_ACTIVE**. A partir do PHP 8.5.0, esta constante está descontinuada, pois não tem efeito.

PDO::PGSQL_TRANSACTION_INTRANS (int)

Equivalente a **Pdo\Pgsql::TRANSACTION_INTRANS**. A partir do PHP 8.5.0, esta constante está descontinuada, pois não tem efeito.

PDO::PGSQL_TRANSACTION_INERROR (int)

Equivalente a **Pdo\Pgsql::TRANSACTION_INERROR**. A partir do PHP 8.5.0, esta constante está descontinuada, pois não tem efeito.

PDO::PGSQL_TRANSACTION_UNKNOWN (int)

Equivalente a **Pdo\Pgsql::TRANSACTION_UNKNOWN**. A partir do PHP 8.5.0, esta constante está descontinuada, pois não tem efeito.

Notas gerais

> **Nota** :
>
> As colunas `bytea` são retornadas como recursos de fluxo. Consulte Objetos Grandes (LOBs) para entender como é a leitura destes valores e como vincular dados com **PDO::PARAM_LOB**.

Índice

PDO_PGSQL DSN

PDO_PGSQL DSN

(PHP 5 >= 5.1.0, PHP 7, PECL PDO_PGSQL >= 0.1.0)

PDO_PGSQL DSN — Conectando-se a bancos de dados PostgreSQL

Descrição

O nome da fonte de dados (DSN) PDO_PGSQL é composto pelos seguintes elementos, delimitados por espaços ou ponto e vírgula:

DSN prefix

O prefixo DSN é **pgsql:**.

`host`

O nome do host no qual o servidor de banco de dados reside.

`port`

A porta na qual o servidor de banco de dados está sendo executado.

`dbname`

O nome do banco de dados.

`user`

O nome do usuário para a conexão. Se você especificar o nome do usuário no DSN, o PDO ignorará o valor do argumento do nome do usuário no construtor do PDO.

`password`

A senha do usuário para a conexão. Se você especificar a senha no DSN, o PDO ignorará o valor do argumento de senha no construtor do PDO.

`sslmode`

O modo SSL. Os valores aceitos são `disable`, `allow`, `prefer`, `require`, `verify-ca` e `verify-full`; seus significados estão listados na » Documentação do PostgreSQL. Muitos serviços hospedados do PostgreSQL requerem uma conexão criptogradada, portanto `require` ou um modo mais estrito é necessário para conexão a eles.

> **Nota** :
>
> A partir do PHP 8.4.0, as credenciais especificadas no DSN (`user` e `password`) têm prioridade sobre os argumentos correspondentes passados para o construtor PDO. Em versões anteriores, os argumentos do construtor PDO tinham precedência.

> **Nota** :
>
> Todos os pontos e vírgulas na string DSN são substituídos por espaços, porque o PostgreSQL espera esse formato. Isso implica que pontos e vírgulas em qualquer um dos componentes (por exemplo, `password` ou `dbname`) não são suportados.

Examples

Exemplo #1 Exemplos de DSN PDO_PGSQL

O exemplo a seguir mostra um DSN PDO_PGSQL para conexão a um banco de dados PostgreSQL:

```
pgsql:host=localhost;port=5432;dbname=testdb;user=bruce;password=mypass
```

O exemplo a seguir mostra um DSN PDO_PGSQL para conexão a um banco de dados PostgreSQL via soquete unix */tmp/.s.PGSQL.5432*:

```
pgsql:host=/tmp;port=5432;dbname=testdb;user=bruce;password=mypass
```

PDO::pgsqlCopyFromArray

PDO::pgsqlCopyFromArray

(PHP 5 >= 5.3.3, PHP 7, PHP 8)

PDO::pgsqlCopyFromArray — Sinônimo de Pdo\Pgsql::copyFromArray()

> **Aviso**

> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

```
#[\Deprecated]
public function PDO::pgsqlCopyFromArray(
    string $tableName,
    array|Traversable $rows,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
```

Este método é um apelido para: Pdo\Pgsql::copyFromArray().

Registro de Alterações

Versão	Descrição
8.5.0	O parâmetro `rows` agora aceita um Traversable; anteriormente somente um array era aceito.

PDO::pgsqlCopyFromFile

PDO::pgsqlCopyFromFile

(PHP 5 >= 5.3.3, PHP 7, PHP 8)

PDO::pgsqlCopyFromFile — Sinônimo de Pdo\Pgsql::copyFromFile()

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

```
#[\Deprecated]
public function PDO::pgsqlCopyFromFile(
    string $tableName,
    string $filename,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
```

Este método é um apelido para: Pdo\Pgsql::copyFromFile().

PDO::pgsqlCopyToArray

PDO::pgsqlCopyToArray

(PHP 5 >= 5.3.3, PHP 7, PHP 8)

PDO::pgsqlCopyToArray — Sinônimo de Pdo\Pgsql::copyToArray()

> **Aviso**

> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

```
#[\Deprecated]
public function PDO::pgsqlCopyToArray(
    string $tableName,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): array|false
```

Este método é um apelido para: Pdo\Pgsql::copyToArray().

PDO::pgsqlCopyToFile

PDO::pgsqlCopyToFile

(PHP 5 >= 5.3.3, PHP 7, PHP 8)

PDO::pgsqlCopyToFile — Sinônimo de Pdo\Pgsql::copyToFile()

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

```
#[\Deprecated]
public function PDO::pgsqlCopyToFile(
    string $tableName,
    string $filename,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
```

Este método é um apelido para: Pdo\Pgsql::copyToFile().

PDO::pgsqlGetNotify

PDO::pgsqlGetNotify

(PHP 5 >= 5.6.0, PHP 7, PHP 8)

PDO::pgsqlGetNotify — Sinônimo de Pdo\Pgsql::getNotify()

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
public function **PDO::pgsqlGetNotify**(int `$fetchMode` = PDO::FETCH_DEFAULT, int `$timeoutMilliseconds` = 0): array|false

Este método é um apelido para: Pdo\Pgsql::getNotify().

PDO::pgsqlGetPid

PDO::pgsqlGetPid

(PHP 5 >= 5.6.0, PHP 7, PHP 8)

PDO::pgsqlGetPid — Sinônimo de Pdo\Pgsql::getPid()

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
public function **PDO::pgsqlGetPid**(): int

Este método é um apelido para: Pdo\Pgsql::getPid().

PDO::pgsqlLOBCreate

PDO::pgsqlLOBCreate

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL pdo_pgsql >= 1.0.2)

PDO::pgsqlLOBCreate — Sinônimo de Pdo\Pgsql::lobCreate()

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
public function **PDO::pgsqlLOBCreate**(): string

Este método é um apelido para: Pdo\Pgsql::lobCreate().

PDO::pgsqlLOBOpen

PDO::pgsqlLOBOpen

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL pdo_pgsql >= 1.0.2)

PDO::pgsqlLOBOpen — Sinônimo de Pdo\Pgsql::lobOpen()

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
public function **PDO::pgsqlLOBOpen**(string `$oid`, string `$mode` = "rb"): resource|false

Este método é um apelido para: Pdo\Pgsql::lobOpen().

PDO::pgsqlLOBUnlink

PDO::pgsqlLOBUnlink

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL pdo_pgsql >= 1.0.2)

PDO::pgsqlLOBUnlink — Sinônimo de Pdo\Pgsql::lobUnlink()

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

#[\Deprecated]
public function **PDO::pgsqlLOBUnlink**(string `$oid`): bool

Este método é um apelido para: Pdo\Pgsql::lobUnlink().

Pdo\Pgsql

(PHP 8 >= 8.4.0)

Introdução

Uma subclasse PDO representando uma conexão que usa o driver PDO PostgreSQL.

Este driver oferece suporte a um analisador de consulta SQL dedicado para o dialeto PostgreSQL. Ele pode lidar com o seguinte:

- Literais entre aspas simples e duplas, com duplicação como mecanismo de escape
- Literais de string de "escape" estilo C
- Literais de string entre cifrões
- Comentários com dois traços e estilo C (não aninhados).
- Suporte para `??` como sequência de escape para o operador `?`.

> **Nota** :
>
> Como este analisador reconhece literais de string entre cifrões, os sinais de interrogação dentro deles não são tratados como marcadores de posição de parâmetro e não precisam ser escapados. A partir do PHP 8.4.0, escapar sinais de interrogação como `??` dentro de uma string entre cifrões foi descontinuado.

Resumo da classe

namespace **Pdo**;

```
class Pgsql extends PDO {
/* Constantes herdadas */
public const int PDO::PARAM_NULL;
public const int PDO::PARAM_BOOL = 5;
public const int PDO::PARAM_INT = 1;
public const int PDO::PARAM_STR = 2;
public const int PDO::PARAM_LOB = 3;
public const int PDO::PARAM_STMT = 4;
public const int PDO::PARAM_INPUT_OUTPUT;
public const int PDO::PARAM_STR_NATL;
public const int PDO::PARAM_STR_CHAR;
public const int PDO::PARAM_EVT_ALLOC;
public const int PDO::PARAM_EVT_FREE;
public const int PDO::PARAM_EVT_EXEC_PRE;
public const int PDO::PARAM_EVT_EXEC_POST;
public const int PDO::PARAM_EVT_FETCH_PRE;
public const int PDO::PARAM_EVT_FETCH_POST;
public const int PDO::PARAM_EVT_NORMALIZE;
public const int PDO::FETCH_DEFAULT;
public const int PDO::FETCH_LAZY;
public const int PDO::FETCH_ASSOC;
public const int PDO::FETCH_NUM;
public const int PDO::FETCH_BOTH;
public const int PDO::FETCH_OBJ;
public const int PDO::FETCH_BOUND;
public const int PDO::FETCH_COLUMN;
public const int PDO::FETCH_CLASS;
public const int PDO::FETCH_INTO;
public const int PDO::FETCH_FUNC;
public const int PDO::FETCH_GROUP;
public const int PDO::FETCH_UNIQUE;
public const int PDO::FETCH_KEY_PAIR;
public const int PDO::FETCH_CLASSTYPE;
public const int PDO::FETCH_SERIALIZE;
public const int PDO::FETCH_PROPS_LATE;
public const int PDO::FETCH_NAMED;
public const int PDO::ATTR_AUTOCOMMIT;
public const int PDO::ATTR_PREFETCH;
public const int PDO::ATTR_TIMEOUT;
public const int PDO::ATTR_ERRMODE;
public const int PDO::ATTR_SERVER_VERSION;
public const int PDO::ATTR_CLIENT_VERSION;
public const int PDO::ATTR_SERVER_INFO;
public const int PDO::ATTR_CONNECTION_STATUS;
public const int PDO::ATTR_CASE;
public const int PDO::ATTR_CURSOR_NAME;
public const int PDO::ATTR_CURSOR;
public const int PDO::ATTR_ORACLE_NULLS;
public const int PDO::ATTR_PERSISTENT;
public const int PDO::ATTR_STATEMENT_CLASS;
public const int PDO::ATTR_FETCH_TABLE_NAMES;
```

```
public const int PDO::ATTR_FETCH_CATALOG_NAMES;
public const int PDO::ATTR_DRIVER_NAME;
public const int PDO::ATTR_STRINGIFY_FETCHES;
public const int PDO::ATTR_MAX_COLUMN_LEN;
public const int PDO::ATTR_EMULATE_PREPARES;
public const int PDO::ATTR_DEFAULT_FETCH_MODE;
public const int PDO::ATTR_DEFAULT_STR_PARAM;
public const int PDO::ERRMODE_SILENT;
public const int PDO::ERRMODE_WARNING;
public const int PDO::ERRMODE_EXCEPTION;
public const int PDO::CASE_NATURAL;
public const int PDO::CASE_LOWER;
public const int PDO::CASE_UPPER;
public const int PDO::NULL_NATURAL;
public const int PDO::NULL_EMPTY_STRING;
public const int PDO::NULL_TO_STRING;
public const string PDO::ERR_NONE;
public const int PDO::FETCH_ORI_NEXT;
public const int PDO::FETCH_ORI_PRIOR;
public const int PDO::FETCH_ORI_FIRST;
public const int PDO::FETCH_ORI_LAST;
public const int PDO::FETCH_ORI_ABS;
public const int PDO::FETCH_ORI_REL;
public const int PDO::CURSOR_FWDONLY;
public const int PDO::CURSOR_SCROLL;
/* Constantes */
public const int ATTR_DISABLE_PREPARES;
public const int ATTR_RESULT_MEMORY_SIZE;
public const int TRANSACTION_IDLE;
public const int TRANSACTION_ACTIVE;
public const int TRANSACTION_INTRANS;
public const int TRANSACTION_INERROR;
public const int TRANSACTION_UNKNOWN;
/* Métodos */
public function copyFromArray(
    string $tableName,
    array|Traversable $rows,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
public function copyFromFile(
    string $tableName,
    string $filename,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
public function copyToArray(
    string $tableName,
    string $separator = "\t",
```

```
    string $nullAs = "\\\\N",
    ?string $fields = null
): array|false
public function copyToFile(
    string $tableName,
    string $filename,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
public function escapeIdentifier(string $input): string
public function getNotify(int $fetchMode = PDO::FETCH_DEFAULT, int $timeoutMilliseconds = 0): array|false
public function getPid(): int
public function lobCreate(): string|false
public function lobOpen(string $oid, string $mode = "rb"): resource|false
public function lobUnlink(string $oid): bool
public function setNoticeCallback(?callable $callback): void
/* Métodos herdados */
public function PDO::__construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
public function PDO::beginTransaction(): bool
public function PDO::commit(): bool
public static function PDO::connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
public function PDO::errorCode(): ?string
public function PDO::errorInfo(): array
public function PDO::exec(string $statement): int|false
public function PDO::getAttribute(int $attribute): mixed
public static function PDO::getAvailableDrivers(): array
public function PDO::inTransaction(): bool
public function PDO::lastInsertId(?string $name = null): string|false
public function PDO::prepare(string $query, array $options = []): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = null): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_COLUMN, int $colno):
PDOStatement|false
public function PDO::query(
    string $query,
    ?int $fetchMode = PDO::FETCH_CLASS,
    string $classname,
    array $constructorArgs
): PDOStatement|false
```

public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_INTO, object $object): PDOStatement|false
public function PDO::quote(string $string, int $type = PDO::PARAM_STR): string|false
public function PDO::rollBack(): bool
public function PDO::setAttribute(int $attribute, mixed $value): bool
}

Constantes predefinidas

Pdo\Pgsql::ATTR_DISABLE_PREPARES

Envia a consulta e os parâmetros ao servidor juntos em uma única chamada, evitando a necessidade de criar uma instrução preparada nomeada separadamente. Se a consulta for executada apenas uma vez, isso poderá reduzir a latência, evitando uma viagem de ida e volta desnecessária ao servidor.

Pdo\Pgsql::ATTR_RESULT_MEMORY_SIZE

Retorna a quantidade de memória, em bytes, alocada para a instância especificada do resultado da consulta PDOStatement ou `null` se não existirem resultados antes da consulta ser executada.

PDO::ATTR_PREFETCH

A partir do PHP 8.5.0, definir este atributo como `0` habilita a busca lenta (linha única): as linhas são recuperadas do servidor uma de cada vez à medida que são buscadas, em vez de armazenar em buffer todo o conjunto de resultados na memória antes da primeira chamada a PDOStatement::fetch(). Isso reduz o uso de memória para conjuntos grandes de resultados. Qualquer outro valor mantém o comportamento padrão de buffer.

Pode ser definido para cada conexão com PDO::setAttribute(), ou por instrução através das opções de driver PDO::prepare() ou PDO::query().

> **Cuidado**
>
> No modo lento, uma conexão pode ter apenas uma instrução ativa por vez. A execução de outra instrução descarta silenciosamente quaisquer linhas não lidas da anterior; nenhum erro é gerado.

Pdo\Pgsql::TRANSACTION_IDLE

> **Aviso**
>
> Esta constante não tem efeito e foi descontinuada a partir do PHP 8.5.0.

Pdo\Pgsql::TRANSACTION_ACTIVE

> **Aviso**
>
> Esta constante não tem efeito e foi descontinuada a partir do PHP 8.5.0.

Pdo\Pgsql::TRANSACTION_INTRANS

> **Aviso**
>
> Esta constante não tem efeito e foi descontinuada a partir do PHP 8.5.0.

`Pdo\Pgsql::TRANSACTION_INERROR`

> **Aviso**
>
> Esta constante não tem efeito e foi descontinuada a partir do PHP 8.5.0.

`Pdo\Pgsql::TRANSACTION_UNKNOWN`

> **Aviso**
>
> Esta constante não tem efeito e foi descontinuada a partir do PHP 8.5.0.

Índice

Pdo\Pgsql::copyFromArray

Pdo\Pgsql::copyFromArray

(PHP 8 >= 8.4.0)

Pdo\Pgsql::copyFromArray — Copia dados de um array PHP para uma tabela

Descrição

```
public function Pdo\Pgsql::copyFromArray(
    string $tableName,
    array|Traversable $rows,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
```

Copia dados do array `rows` para a tabela `tableName` usando `separator` como delimitador de campos e uma lista de campos no parâmetro `fields`.

Parâmetros

`tableName`

String contendo o nome da tabela.

`rows`

Um array indexado (ou Traversable) de strings com campos separados por `separator`.

`separator`

Delimitador usado para separar campos em uma entrada do array `rows`.

`nullAs`

Como interpretar valores SQL `NULL`.

`fields`

Lista de campos a serem inseridos.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Registro de Alterações

Versão	Descrição
8.5.0	`rows` agora aceira um Traversable; anteriormente somente um array era aceito.

Examples

Exemplo #1 Exemplo de **Pdo\Pgsql::copyFromArray()**

Cada elemento de `rows` é um registro cujos campos são conectados por `separator` (uma tabulação por padrão).

```php
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->exec('CREATE TABLE fruits (id int, name text, qty int)');

$rows = [
    "1\tapple\t10",
    "2\tbanana\t20",
    "3\tcherry\t30",
];
$db->copyFromArray('fruits', $rows);

foreach ($db->query('SELECT * FROM fruits ORDER BY id') as $row) {
    echo "{$row['id']} {$row['name']} {$row['qty']}\n";
}
?>
```

O exemplo acima produzirá:

```
1 apple 10
2 banana 20
3 cherry 30
```

Veja também

- Pdo\Pgsql::copyToArray() - Copia dados da tabela do banco de dados para o array PHP
- Pdo\Pgsql::copyFromFile() - Copia dados do arquivo para a tabela
- Pdo\Pgsql::copyToFile() - Copia dados da tabela para arquivo

Pdo\Pgsql::copyFromFile

Pdo\Pgsql::copyFromFile

(PHP 8 >= 8.4.0)

Pdo\Pgsql::copyFromFile — Copia dados do arquivo para a tabela

Descrição

public function **Pdo\Pgsql::copyFromFile**(
  string $tableName,
  string $filename,
  string $separator = "\t",
  string $nullAs = "\\\\N",
  ?string $fields = **null**
): bool

Copia dados do arquivo especificado por `filename` na tabela `tableName` usando `separator` como delimitador de campos e lista de campos no parâmetro `fields`.

Parâmetros

`tableName`

String contendo o nome da tabela.

`filename`

Nome do arquivo que contém os dados a serem importados.

`separator`

Delimitador usado para separar campos em uma entrada do array `rows`.

`nullAs`

Como interpretar valores SQL `NULL`.

`fields`

Lista de campos a serem inseridos.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Se `filename` não puder ser aberto para leitura, a falha é reportada através da manipulação do erro da conexão (consulte **`PDO::ATTR_ERRMODE`**); com **`PDO::ERRMODE_EXCEPTION`**, uma PDOException é lançada.

Examples

Exemplo #1 Exemplo de **Pdo\Pgsql::copyFromFile()**

O arquivo tem um registro por linha com campos conectados por `separator` (uma tabulação por padrão).

```php
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->exec('CREATE TABLE fruits (id int, name text, qty int)');

file_put_contents('/tmp/fruits.tsv', "1\tapple\t10\n2\tbanana\t20\n");
$db->copyFromFile('fruits', '/tmp/fruits.tsv');
```

```
echo $db->query('SELECT count(*) FROM fruits')->fetchColumn(), "\n";
?>
```

O exemplo acima produzirá:

```
2
```

Veja também

- Pdo\Pgsql::copyToFile() - Copia dados da tabela para arquivo
- Pdo\Pgsql::copyFromArray() - Copia dados de um array PHP para uma tabela
- Pdo\Pgsql::copyToArray() - Copia dados da tabela do banco de dados para o array PHP

Pdo\Pgsql::copyToArray

Pdo\Pgsql::copyToArray

(PHP 8 >= 8.4.0)

Pdo\Pgsql::copyToArray — Copia dados da tabela do banco de dados para o array PHP

Descrição

public function **Pdo\Pgsql::copyToArray**(
    string `$tableName`,
    string `$separator` = "\t",
    string `$nullAs` = "\\\\N",
    ?string `$fields` = **`null`**
): array|false

Copia dados de `tableName` para um array usando `separator` como delimitador de campos e lista de campos no parâmetro `fields`.

Parâmetros

`tableName`

String contendo o nome da tabela.

`separator`

Delimitador usado para separar campos em uma entrada do array `rows`.

`nullAs`

Como interpretar valores SQL `NULL`.

`fields`

Lista de campos a serem exportados.

Valor Retornado

Retorna um array de linhas, ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de **Pdo\Pgsql::copyToArray()**

Cada elemento retornado é um registro com os campos conectados por `separator` e uma nova linha no final.

```
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->exec('CREATE TABLE fruits (id int, name text, qty int)');
$db->exec("INSERT INTO fruits VALUES (1, 'apple', 10), (2, 'banana', 20)");

$rows = $db->copyToArray('fruits');
var_export($rows);
?>
```

O exemplo acima produzirá:

```
array (
  0 => '1	apple	10
',
  1 => '2	banana	20
',
)
```

Veja também

- Pdo\Pgsql::copyFromArray() - Copia dados de um array PHP para uma tabela
- Pdo\Pgsql::copyFromFile() - Copia dados do arquivo para a tabela
- Pdo\Pgsql::copyToFile() - Copia dados da tabela para arquivo

Pdo\Pgsql::copyToFile

Pdo\Pgsql::copyToFile

(PHP 8 >= 8.4.0)

Pdo\Pgsql::copyToFile — Copia dados da tabela para arquivo

Descrição

public function **Pdo\Pgsql::copyToFile**(
  string `$tableName`,
  string `$filename`,
  string `$separator` = "\t",
  string `$nullAs` = "\\N",

```
    ?string $fields = null
): bool
```

Copia dados da tabela para o arquivo especificado por `filename` usando `separator` como delimitador de campos e lista de campos no parâmetro `fields`.

Parâmetros

`tableName`

String contendo o nome da tabela.

`filename`

Nome do arquivo para exportar dados.

`separator`

Delimitador usado para separar campos em uma entrada do array `rows`.

`nullAs`

Como interpretar valores SQL `NULL`.

`fields`

Lista de campos a serem exportados.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Erros/Exceções

Se `filename` não puder ser aberto para escrita ou não puder ser gravado, a falha é reportada através da manipulação do erro de conexão (consulte **`PDO::ATTR_ERRMODE`**); com **`PDO::ERRMODE_EXCEPTION`** uma PDOException é lançada.

Examples

Exemplo #1 Exemplo de **Pdo\Pgsql::copyToFile()**

A tabela é gravada em `filename`, um registro por linha com os campos conectados por `separator`.

```php
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->exec('CREATE TABLE fruits (id int, name text, qty int)');
$db->exec("INSERT INTO fruits VALUES (1, 'apple', 10), (2, 'banana', 20)");

$db->copyToFile('fruits', '/tmp/fruits.tsv');
echo file_get_contents('/tmp/fruits.tsv');
?>
```

O exemplo acima produzirá:

```
1       apple   10
2       banana  20
```

Veja também

- Pdo\Pgsql::copyFromFile() - Copia dados do arquivo para a tabela
- Pdo\Pgsql::copyFromArray() - Copia dados de um array PHP para uma tabela
- Pdo\Pgsql::copyToArray() - Copia dados da tabela do banco de dados para o array PHP

Pdo\Pgsql::escapeIdentifier

Pdo\Pgsql::escapeIdentifier

(PHP 8 >= 8.4.0)

Pdo\Pgsql::escapeIdentifier — Escapa uma string para uso como identificador SQL

Descrição

public function **Pdo\Pgsql::escapeIdentifier**(string `$input`): string

Escapa uma string para uso como um identificador SQL, como uma tabela, coluna ou nome de função. Isso é útil quando um identificador fornecido pelo usuário pode conter caracteres especiais que de outra forma não seriam interpretados como parte do identificador pelo analisador SQL ou quando o identificador pode conter caracteres em maiúsculas que precisam ser preservadas.

Parâmetros

`input`

Uma string contendo texto a ser escapado.

Valor Retornado

Uma string contendo os dados escapados.

Examples

Exemplo #1 Exemplo de Pdo\Pgsql::escapeIdentifier()

```php
<?php
$pdo = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);

$unescapedTableName = 'UnescapedTableName';
$pdo->exec("CREATE TABLE $unescapedTableName ()");

$escapedTableName = $pdo->escapeIdentifier('EscapedTableName');
$pdo->exec("CREATE TABLE $escapedTableName ()");

$statement = $pdo->query(
    "SELECT relname FROM pg_stat_user_tables WHERE relname ilike '%tablename'"
);

var_export($statement->fetchAll(PDO::FETCH_COLUMN, 0));
```

```php
$tableNameWithSymbols = 'Table-Name-With-Symbols';
$pdo->exec("CREATE TABLE $tableNameWithSymbols ()");
?>
```

O exemplo acima produzirá algo semelhante a:

```
array (
  0 => 'unescapedtablename',
  1 => 'EscapedTableName',
)
Fatal error: Uncaught PDOException: SQLSTATE[42601]: Syntax error: 7 ERROR:  syntax error at or near
"Table"
LINE 1: CREATE TABLE Table-Name-With-Symbols ()
```

Veja também

- PDO::quote() - Inclui aspas em uma string para uso em uma consulta

Pdo\Pgsql::getNotify

Pdo\Pgsql::getNotify

(PHP 8 >= 8.4.0)

Pdo\Pgsql::getNotify — Recebe notificação assíncrona

Descrição

public function **Pdo\Pgsql::getNotify**(int `$fetchMode` = **`PDO::FETCH_DEFAULT`**, int `$timeoutMilliseconds` = 0): array|false

Retorna um conjunto de resultados que representa uma notificação assíncrona pendente.

Parâmetros

`fetchMode`

O formato como o conjunto de resultados deve ser retornado, uma das seguintes constantes:

- **`PDO::FETCH_DEFAULT`**
- **`PDO::FETCH_BOTH`**
- **`PDO::FETCH_ASSOC`**
- **`PDO::FETCH_NUM`**

`timeoutMilliseconds`

O tempo de espera por uma resposta, em milissegundos.

Valor Retornado

Se uma notificação estiver pendente, retorna uma única linha, caso contrário retorna **`false`**. Alinha tem um campo `message` (o nome do canal) e um campo `pid` (o ID do processo do backend que envia a notificação). Se a

notificação traz uma carga não vazia, a linha também tem um campo `payload`. Com **`PDO::FETCH_NUM`**, estes campos estão nos índices `0`, `1` e `2`.

Erros/Exceções

Uma exceção ValueError será lançada se `fetchMode` não for uma das constantes **`PDO::FETCH_*`** válidas.

Uma exceção ValueError será lançada se `timeoutMilliseconds` for menor que `0`.

Um **`E_WARNING`** é gerado quando `timeoutMilliseconds` é maior que o valor que pode estar contido em um inteiro com sinal de 32 bits; nesse caso, será o valor máximo de um inteiro com sinal de 32 bits.

Examples

Exemplo #1 Exemplo de Pdo\Pgsql::getNotify()

Inscreva em um canal com `LISTEN` e então leia a próxima notificação pendente com um tempo limite de um segundo.

```php
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$db->exec('LISTEN messages');
$db->exec("NOTIFY messages, 'hello'");

$notification = $db->getNotify(PDO::FETCH_ASSOC, 1000);
var_export($notification);
?>
```

O exemplo acima produzirá algo semelhante a:

```
array (
  'message' => 'messages',
  'pid' => 1928,
  'payload' => 'hello',
)
```

Veja também

- PDO::query() - Prepara e executa uma instrução SQL sem marcadores
- PDOStatement::fetch() - Busca a próxima linha de um resultado
- PDOStatement::fetchAll() - Busca as linhas remanescentes de um conjunto de resultados

Pdo\Pgsql::getPid

Pdo\Pgsql::getPid

(PHP 8 >= 8.4.0)

Pdo\Pgsql::getPid — Obtém o PID do processo subjacente que trata esta conexão

Descrição

public function **Pdo\Pgsql::getPid**(): int

Retorna o PID do processo subjacente que trata esta conexão. Observe que o PID pertence a um processo em execução no host do servidor de banco de dados, não no host local.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o PID como um int.

Examples

Exemplo #1 Exemplo de Pdo\Pgsql::getPid()

```php
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
echo $db->getPid();
?>
```

O exemplo acima produzirá algo semelhante a:

```
12345
```

Veja também

- pg_get_pid() - Obtém o ID do processo do backend

Pdo\Pgsql::lobCreate

Pdo\Pgsql::lobCreate

(PHP 8 >= 8.4.0)

Pdo\Pgsql::lobCreate — Cria um novo objeto grande

Descrição

public function **Pdo\Pgsql::lobCreate**(): string|false

Pdo\Pgsql::lobCreate() cria um objeto grande e retorna o OID que se refere a ele. Ele pode ser aberto para ler ou gravar dados com Pdo\Pgsql::lobOpen().

O OID pode ser armazenado em colunas do tipo OID e usado para fazer referência ao objeto grande, sem fazer com que a linha cresça arbitrariamente. O objeto grande continuará no banco de dados até ser removido chamando Pdo\Pgsql::lobUnlink().

Objetos grandes são complicados de usar. Na verdade, é necessário que Pdo\Pgsql::lobUnlink() seja chamado antes de excluir a última linha que faz referência ao OID em todo o banco de dados; caso contrário, objetos grandes não referenciados permanecerão no servidor indefinidamente. Além disso, objetos grandes não possuem controles de acesso. Uma alternativa é o tipo de coluna bytea, que pode ter até 1 GB de tamanho, e esse tipo de coluna gerencia de forma transparente o armazenamento para obter o tamanho de linha ideal.

> **Nota** :
>
> Esta função e todas as manipulações de objeto grande devem ser chamadas e executadas dentro de uma transação.

Parâmetros

Esta função não possui parâmetros.

Valor Retornado

Retorna o OID do objeto grande recém-criado em caso de sucesso, ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de **Pdo\Pgsql::lobCreate()**

Este exemplo cria um novo objeto grande e copia nele o conteúdo de um arquivo. O OID é então armazenado em uma tabela.

```
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->beginTransaction();
$oid = $db->lobCreate();
$stream = $db->lobOpen($oid, 'w');
$local = fopen($filename, 'rb');
stream_copy_to_stream($local, $stream);
$local = null;
$stream = null;
$stmt = $db->prepare("INSERT INTO BLOBS (ident, oid) VALUES (?, ?)");
$stmt->execute([$some_id, $oid]);
$db->commit();
?>
```

Veja também

- Pdo\Pgsql::lobOpen() - Abre um fluxo de objetos grandes existente
- Pdo\Pgsql::lobUnlink() - Exclui o objeto grande
- pg_lo_create() - Cria um objeto grande
- pg_lo_open() - Abra um objeto grande

Pdo\Pgsql::lobOpen

Pdo\Pgsql::lobOpen

(PHP 8 >= 8.4.0)

Pdo\Pgsql::lobOpen — Abre um fluxo de objetos grandes existente

Descrição

public function **Pdo\Pgsql::lobOpen**(string `$oid`, string `$mode` = "rb"): resource|false

Pdo\Pgsql::lobOpen() abre um fluxo para acessar os dados referenciados por `oid`. Todas as funções usuais do sistema de arquivos, como fread(), fwrite() ou fgets() podem ser usadas para manipular o conteúdo do fluxo.

> **Nota** :
>
> Esta função e todas as manipulações de objeto grande devem ser chamadas e executadas dentro de uma transação.

Parâmetros

`oid`

Um identificador de objeto grande.

`mode`

O modo de acesso. Se `mode` contiver `w` ou `+`, o fluxo é aberto para leitura e escrita; caso contrário é aberto somente-leitura. Um sinalizador `b` (binário) não tem efeito. O padrão `"rb"` abre o fluxo para leitura.

Valor Retornado

Retorna um recurso de fluxo em caso de sucesso, ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de **Pdo\Pgsql::lobOpen()**

Seguindo o exemplo de Pdo\Pgsql::lobCreate(), este trecho de código recupera o objeto grande do banco de dados e o envia para o navegador.

```
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->beginTransaction();
$stmt = $db->prepare("SELECT oid FROM BLOBS WHERE ident = ?");
$stmt->execute([$some_id]);
$stmt->bindColumn('oid', $oid, PDO::PARAM_STR);
$stmt->fetch(PDO::FETCH_BOUND);
$stream = $db->lobOpen($oid, 'r');
header("Content-type: application/octet-stream");
fpassthru($stream);
?>
```

Veja também

- Pdo\Pgsql::lobCreate() - Cria um novo objeto grande
- Pdo\Pgsql::lobUnlink() - Exclui o objeto grande
- pg_lo_create() - Cria um objeto grande
- pg_lo_open() - Abra um objeto grande

Pdo\Pgsql::lobUnlink

Pdo\Pgsql::lobUnlink

(PHP 8 >= 8.4.0)

Pdo\Pgsql::lobUnlink — Exclui o objeto grande

Descrição

public function **Pdo\Pgsql::lobUnlink**(string `$oid`): bool

Exclui um objeto grande do banco de dados identificado pelo OID.

> **Nota** :
>
> Esta função e todas as manipulações de objeto grande devem ser chamadas e executadas dentro de uma transação.

Parâmetros

`oid`

Um identificador de objeto grande.

Valor Retornado

Retorna `true` em caso de sucesso ou `false` em caso de falha.

Examples

Exemplo #1 Exemplo de Pdo\Pgsql::lobUnlink()

Este exemplo desvincula um objeto grande do banco de dados antes de excluir a linha que o referencia. Ele usa a tabela de blobs dos exemplos de Pdo\Pgsql::lobCreate() e Pdo\Pgsql::lobOpen().

```
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->beginTransaction();
$db->lobUnlink($oid);
$stmt = $db->prepare("DELETE FROM BLOBS where ident = ?");
$stmt->execute([$some_id]);
$db->commit();
?>
```

Veja também

- Pdo\Pgsql::lobCreate() - Cria um novo objeto grande
- Pdo\Pgsql::lobOpen() - Abre um fluxo de objetos grandes existente
- pg_lo_create() - Cria um objeto grande
- pg_lo_open() - Abra um objeto grande

Pdo\Pgsql::setNoticeCallback

Pdo\Pgsql::setNoticeCallback

(PHP 8 >= 8.4.0)

Pdo\Pgsql::setNoticeCallback — Define um retorno de chamada para lidar com mensagens de alerta e de aviso geradas pelo back-end

Descrição

public function **Pdo\Pgsql::setNoticeCallback**(?callable `$callback`): void

Define um retorno de chamada para lidar com mensagens de aviso e aviso geradas pelo back-end. Isto inclui mensagens emitidas pelo próprio PostgreSQL, bem como aquelas geradas por funções SQL definidas pelo usuário usando `RAISE`. Observe que o recebimento real dessas mensagens depende da configuração de backend `client_min_messages`.

Parâmetros

`callback`

Se `null` for passado, o manipulador será redefinido para seu estado padrão.

Caso contrário, o manipulador será um retorno de chamada com a seguinte assinatura:

function handler(string `$message`): void

`message`

Uma mensagem gerada pelo backend.

Valor Retornado

Nenhum valor é retornado.

Examples

Exemplo #1 Exemplo de **Pdo\Pgsql::setNoticeCallback()**

```php
<?php
$pdo = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);

$pdo->exec('CREATE TABLE parent(id int primary key)');
$pdo->exec('CREATE TABLE child(id int references parent)');

$pdo->setNoticeCallback(function ($message) {
    echo $message;
});

$pdo->exec('TRUNCATE parent CASCADE');
?>
```

O exemplo acima produzirá algo semelhante a:

```
NOTICE:  truncate cascades to table "child"
```

Driver PDO do SQLite

Introdução

PDO_SQLITE é um driver que implementa a Interface de Objeto de Dados do PHP (PDO) para permitir o acesso a bancos de dados SQLite 3.

> **Nota** :
>
> O PDO_SQLITE permite usar strings separadas de fluxos junto com `PDO::PARAM_LOB`.

Instalação

O driver PDO PDO_SQLITE está habilitado por padrão. Para desabilitá-lo, a opção **--without-pdo-sqlite[=DIR]** pode ser usada, onde o valor opcional `[=DIR]` é o diretório de instalação do sqlite. A partir do PHP 7.4.0, a » libsqlite ≥ 3.5.0 é necessária. Anteriormente, a libsqlite que vinha com o PHP poderia ser usada e era o padrão se `[=DIR]` fosse omitido.

> **Nota** : **Configuração adicional no Windows a partir do PHP 7.4.0**
>
> Para que esta extensão funcione, existem arquivos DLL que devem estar disponíveis no *PATH* do sistema Windows. Para saber como fazer isso, veja a FAQ intitulada "Como eu adiciono o meu diretório PHP ao PATH do Windows". Embora copiar arquivos DLL da pasta do PHP para o diretório system do Windows também funcione (porque o diretório system está, por padrão, no *PATH* do sistema), isso não é recomendado. *Esta extensão requer que os seguintes arquivos estejam no PATH: libsqlite3.dll.*

Índice

DSN PDO_SQLITE

DSN PDO_SQLITE

(No version information available, might only be in Git)

DSN PDO_SQLITE — Conectando a bancos de dados SQLite

Descrição

O Data Source Name (DSN) PDO_SQLITE é composto pelos seguintes elementos:

Prefixo DSN (SQLite 3)

O prefixo DSN é **`sqlite:`**.

- Para acessar um banco de dados em disco, o caminho absoluto deve ser adicionado ao prefixo DSN.
- Para criar um banco de dados em memória, `:memory:` deve ser adicionado ao prefixo DSN.
- Se o DSN consistir apenas no prefixo DSN, um banco de dados temporário será usado e será excluído quando a conexão for encerrada.

Examples

Exemplo #1 Exemplos de DSN PDO_SQLITE

Os exemplos a seguir mostram o DSN PDO_SQLITE para conexão com bancos de dados SQLite:

```
sqlite:/caminho/para/o/arquivo/banco.sq3
sqlite::memory:
sqlite:
```

PDO::sqliteCreateAggregate

PDO::sqliteCreateAggregate

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo_sqlite >= 1.0.0)

PDO::sqliteCreateAggregate — Sinônimo de Pdo\Sqlite::createAggregate()

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

public function **PDO::sqliteCreateAggregate**(
  string `$name`,
  callable `$step`,
  callable `$finalize`,
  int `$numArgs = -1`
): bool

Este método é um apelido para: Pdo\Sqlite::createAggregate().

PDO::sqliteCreateCollation

PDO::sqliteCreateCollation

(PHP 5 >= 5.3.11, PHP 7, PHP 8)

PDO::sqliteCreateCollation — Sinônimo de Pdo\Sqlite::createCollation()

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

public function **PDO::sqliteCreateCollation**(string `$name`, callable `$callback`): bool

Este método é um apelido para: Pdo\Sqlite::createCollation().

PDO::sqliteCreateFunction

PDO::sqliteCreateFunction

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo_sqlite >= 1.0.0)

PDO::sqliteCreateFunction — Sinônimo de Pdo\Sqlite::createFunction()

> **Aviso**
>
> Esta função foi *DESCONTINUADA* a partir do PHP 8.5.0. O uso desta função é fortemente desencorajado.

Descrição

public function **PDO::sqliteCreateFunction**(
  string `$function_name`,
  callable `$callback`,
  int `$num_args` = -1,
  int `$flags` = 0
): bool

Este método é um apelido para: Pdo\Sqlite::createFunction().

Registro de Alterações

Versão	Descrição
7.1.4	O parâmetro `flags` foi adicionado.

Pdo\Sqlite

(PHP 8 >= 8.4.0)

Introdução

Uma subclasse PDO representando uma conexão que usa o driver PDO SQLite.

Este driver suporta um analisador de consulta SQL dedicado para o dialeto SQLite. Pode lidar com:

- Literais entre aspas simples, duplas, e acentos graves, com a duplicação como mecanismo de escape.
- Identificadores envolvidos por colchetes.
- Comentários com dois hífens ou no estilo C (não aninhados).

Resumo da classe

class **Pdo\Sqlite** extends PDO {
/* Constantes herdadas */
public const int *PDO::PARAM_NULL*;
public const int *PDO::PARAM_BOOL* = 5;
public const int *PDO::PARAM_INT* = 1;
public const int *PDO::PARAM_STR* = 2;
public const int *PDO::PARAM_LOB* = 3;
public const int *PDO::PARAM_STMT* = 4;
public const int *PDO::PARAM_INPUT_OUTPUT*;
public const int *PDO::PARAM_STR_NATL*;
public const int *PDO::PARAM_STR_CHAR*;
public const int *PDO::PARAM_EVT_ALLOC*;
public const int *PDO::PARAM_EVT_FREE*;
public const int *PDO::PARAM_EVT_EXEC_PRE*;
public const int *PDO::PARAM_EVT_EXEC_POST*;
public const int *PDO::PARAM_EVT_FETCH_PRE*;

public const int *PDO::PARAM_EVT_FETCH_POST*;
public const int *PDO::PARAM_EVT_NORMALIZE*;
public const int *PDO::FETCH_DEFAULT*;
public const int *PDO::FETCH_LAZY*;
public const int *PDO::FETCH_ASSOC*;
public const int *PDO::FETCH_NUM*;
public const int *PDO::FETCH_BOTH*;
public const int *PDO::FETCH_OBJ*;
public const int *PDO::FETCH_BOUND*;
public const int *PDO::FETCH_COLUMN*;
public const int *PDO::FETCH_CLASS*;
public const int *PDO::FETCH_INTO*;
public const int *PDO::FETCH_FUNC*;
public const int *PDO::FETCH_GROUP*;
public const int *PDO::FETCH_UNIQUE*;
public const int *PDO::FETCH_KEY_PAIR*;
public const int *PDO::FETCH_CLASSTYPE*;
public const int *PDO::FETCH_SERIALIZE*;
public const int *PDO::FETCH_PROPS_LATE*;
public const int *PDO::FETCH_NAMED*;
public const int *PDO::ATTR_AUTOCOMMIT*;
public const int *PDO::ATTR_PREFETCH*;
public const int *PDO::ATTR_TIMEOUT*;
public const int *PDO::ATTR_ERRMODE*;
public const int *PDO::ATTR_SERVER_VERSION*;
public const int *PDO::ATTR_CLIENT_VERSION*;
public const int *PDO::ATTR_SERVER_INFO*;
public const int *PDO::ATTR_CONNECTION_STATUS*;
public const int *PDO::ATTR_CASE*;
public const int *PDO::ATTR_CURSOR_NAME*;
public const int *PDO::ATTR_CURSOR*;
public const int *PDO::ATTR_ORACLE_NULLS*;
public const int *PDO::ATTR_PERSISTENT*;
public const int *PDO::ATTR_STATEMENT_CLASS*;
public const int *PDO::ATTR_FETCH_TABLE_NAMES*;
public const int *PDO::ATTR_FETCH_CATALOG_NAMES*;
public const int *PDO::ATTR_DRIVER_NAME*;
public const int *PDO::ATTR_STRINGIFY_FETCHES*;
public const int *PDO::ATTR_MAX_COLUMN_LEN*;
public const int *PDO::ATTR_EMULATE_PREPARES*;
public const int *PDO::ATTR_DEFAULT_FETCH_MODE*;
public const int *PDO::ATTR_DEFAULT_STR_PARAM*;
public const int *PDO::ERRMODE_SILENT*;
public const int *PDO::ERRMODE_WARNING*;
public const int *PDO::ERRMODE_EXCEPTION*;
public const int *PDO::CASE_NATURAL*;
public const int *PDO::CASE_LOWER*;
public const int *PDO::CASE_UPPER*;
public const int *PDO::NULL_NATURAL*;
public const int *PDO::NULL_EMPTY_STRING*;
public const int *PDO::NULL_TO_STRING*;

```
public const string PDO::ERR_NONE;
public const int PDO::FETCH_ORI_NEXT;
public const int PDO::FETCH_ORI_PRIOR;
public const int PDO::FETCH_ORI_FIRST;
public const int PDO::FETCH_ORI_LAST;
public const int PDO::FETCH_ORI_ABS;
public const int PDO::FETCH_ORI_REL;
public const int PDO::CURSOR_FWDONLY;
public const int PDO::CURSOR_SCROLL;
/* Constantes */
public const int DETERMINISTIC;
public const int OPEN_READONLY;
public const int OPEN_READWRITE;
public const int OPEN_CREATE;
public const int ATTR_OPEN_FLAGS;
public const int ATTR_READONLY_STATEMENT;
public const int ATTR_EXTENDED_RESULT_CODES;
/* Métodos */
public function createAggregate(
    string $name,
    callable $step,
    callable $finalize,
    int $numArgs = -1
): bool
public function createCollation(string $name, callable $callback): bool
public function createFunction(
    string $function_name,
    callable $callback,
    int $num_args = -1,
    int $flags = 0
): bool
public function loadExtension(string $name): void
public function openBlob(
    string $table,
    string $column,
    int $rowid,
    ?string $dbname = "main",
    int $flags = Pdo\Sqlite::OPEN_READONLY
): resource|false
public function setAuthorizer(?callable $callback): void
/* Métodos herdados */
public function PDO::__construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
public function PDO::beginTransaction(): bool
public function PDO::commit(): bool
```

```
public static function PDO::connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
public function PDO::errorCode(): ?string
public function PDO::errorInfo(): array
public function PDO::exec(string $statement): int|false
public function PDO::getAttribute(int $attribute): mixed
public static function PDO::getAvailableDrivers(): array
public function PDO::inTransaction(): bool
public function PDO::lastInsertId(?string $name = null): string|false
public function PDO::prepare(string $query, array $options = []): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = null): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_COLUMN, int $colno):
PDOStatement|false
public function PDO::query(
    string $query,
    ?int $fetchMode = PDO::FETCH_CLASS,
    string $classname,
    array $constructorArgs
): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_INTO, object $object):
PDOStatement|false
public function PDO::quote(string $string, int $type = PDO::PARAM_STR): string|false
public function PDO::rollBack(): bool
public function PDO::setAttribute(int $attribute, mixed $value): bool
}
```

Constantes predefinidas

Pdo\Sqlite::DETERMINISTIC

Pdo\Sqlite::OPEN_READONLY

Pdo\Sqlite::OPEN_READWRITE

Pdo\Sqlite::OPEN_CREATE

Pdo\Sqlite::ATTR_OPEN_FLAGS

Pdo\Sqlite::ATTR_READONLY_STATEMENT

Pdo\Sqlite::ATTR_EXTENDED_RESULT_CODES

Índice

Pdo\Sqlite::createAggregate

Pdo\Sqlite::createAggregate

(PHP 8 >= 8.4.0)

Pdo\Sqlite::createAggregate — Registra uma função de agregação definida pelo usuário para uso em instruções SQL

Descrição

public function **Pdo\Sqlite::createAggregate**(
    string $name,
    callable $step,
    callable $finalize,
    int $numArgs = -1
): bool

Este método é semelhante a Pdo\Sqlite::createFunction() exceto que registra funções que podem ser usadas para calcular um resultado agregado em todas as linhas de uma consulta.

A principal diferença entre este método e Pdo\Sqlite::createFunction() é que duas funções são necessárias para gerenciar a agregação.

> **Dica**
>
> Usando este método é possível substituir funções SQL nativas.

Parâmetros

`name`

O nome da função usada nas instruções SQL.

`step`

Função de retorno de chamada chamada para cada linha do conjunto de resultados. O retorno de chamada deve acumular o resultado e armazená-lo no contexto de agregação.

Esta função precisa ser definida como:

function step(
    mixed $context,
    int $rownumber,
    mixed $value,
    mixed ...$values
): mixed

`context`

`null` para a primeira linha; nas linhas subsequentes terá o valor que foi retornado anteriormente da função step; deve-se usar isso para manter o estado agregado.

`rownumber`

O número da linha atual.

`value`

O primeiro argumento passado para o agregado.

`values`

Outros argumentos passados para o agregado.

O valor de retorno desta função será usado como argumento `context` na próxima chamada das funções step ou finalize.

`finalize`

Função de retorno de chamada para agregar os dados "escalonados" de cada linha. Depois que todas as linhas forem processadas, esta função será chamada, devendo então pegar os dados do contexto de agregação e retornar o resultado. Esta função de retorno de chamada deve retornar um tipo compreendido pelo SQLite (ou seja, tipo escalar).

Esta função precisa ser definida como:

function fini(mixed `$context`, int `$rowcount`): mixed

`context`

Contém o valor de retorno da última chamada para a função step.

`rowcount`

Contém o número de linhas nas quais a agregação foi executada.

O valor de retorno desta função será usado como valor de retorno da agregação.

`numArgs`

Dica para o analisador SQLite se a função de retorno de chamada aceita um número predeterminado de argumentos.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo de Pdo\Sqlite::createAggregate()

Neste exemplo, criaremos uma função agregada personalizada chamada `max_length` que pode ser usada em consultas SQL.

Neste exemplo, estamos criando uma função agregadora, chamada `max_length`, que calculará o comprimento da string mais longa em uma das colunas da tabela. Para cada linha, a função `max_len_step` é chamada e passa um parâmetro `$context`. O parâmetro context é como qualquer outra variável PHP e pode ser configurado para conter um array ou até mesmo um object. Neste exemplo, estamos usando-o para manter o comprimento máximo que vimos até agora; se *$string* tiver um comprimento maior que o máximo atual, atualizamos o contexto para manter esse novo comprimento máximo.

Após todas as linhas terem sido processadas, o SQLite chama a função `max_len_finalize` para determinar o resultado agregado. É possível realizar algum tipo de cálculo baseado nos dados em `$context`. Neste exemplo básico o resultado foi calculado à medida que a consulta avançava, para que o valor do contexto pudesse ser retornado diretamente.

```php
<?php
$data = [
    'one',
    'two',
    'three',
```

```
    'four',
    'five',
    'six',
    'seven',
    'eight',
    'nine',
    'ten',
];
$db = new Pdo\Sqlite('sqlite::memory:');
$db->exec("CREATE TABLE strings(a)");
$insert = $db->prepare('INSERT INTO strings VALUES (?)');
foreach ($data as $str) {
    $insert->execute(array($str));
}
$insert = null;

function max_len_step($context, $row_number, $string)
{
    if (strlen($string) > $context) {
        $context = strlen($string);
    }
    return $context;
}

function max_len_finalize($context, $row_count)
{
    return $context === null ? 0 : $context;
}

$db->createAggregate('max_len', 'max_len_step', 'max_len_finalize');

var_dump($db->query('SELECT max_len(a) from strings')->fetchAll());

?>
```

> **Dica**
>
> NÃO é recomendado que se armazene uma cópia dos valores no contexto e depois processe-os no final, pois faria com que o SQLite usasse muita memória para processar a consulta - basta pensar em quanta memória seria necessárias se um milhão de linhas fossem armazenadas na memória, cada uma contendo uma string de 32 bytes de comprimento.

Veja também

- Pdo\Sqlite::createFunction() - Registra uma função definida pelo usuário para uso em instruções SQL
- Pdo\Sqlite::createCollation() - Registra uma função definida pelo usuário para uso como função de agrupamento em instruções SQL
- **sqlite_create_function()**
- **sqlite_create_aggregate()**

Pdo\Sqlite::createCollation

Pdo\Sqlite::createCollation

(PHP 8 >= 8.4.0)

Pdo\Sqlite::createCollation — Registra uma função definida pelo usuário para uso como função de agrupamento em instruções SQL

Descrição

public function **Pdo\Sqlite::createCollation**(string `$name`, callable `$callback`): bool

Este método é semelhante a Pdo\Sqlite::createFunction() exceto que ele registra funções que são usadas para agrupar strings.

Parâmetros

`name`

Nome da função de agrupamento SQL a ser criada ou redefinida.

`callback`

Função de retorno de chamada que define o comportamento de um agrupamento. Ele deve aceitar duas strings e retornar `-1`, `0` ou `1` se a primeira string for classificada antes, ordenada de forma idêntica ou ordenada depois da segunda string, respectivamente. Uma função interna que se comporta assim é strcmp().

Esta função precisa ser definida como:

function collation(string `$string1`, string `$string2`): int

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo de **Pdo\Sqlite::createCollation()**

```php
<?php
$db = new Pdo\Sqlite('sqlite::memory:');
$db->exec("CREATE TABLE test (col1 string)");
$db->exec("INSERT INTO test VALUES ('a1')");
$db->exec("INSERT INTO test VALUES ('a10')");
$db->exec("INSERT INTO test VALUES ('a2')");

$db->sqliteCreateCollation('NATURAL_CMP', 'strnatcmp');
foreach ($db->query("SELECT col1 FROM test ORDER BY col1") as $row) {
    echo $row['col1'] . "\n";
}
echo "\n";
foreach ($db->query("SELECT col1 FROM test ORDER BY col1 COLLATE NATURAL_CMP") as $row) {
    echo $row['col1'] . "\n";
}
?>
```

O exemplo acima produzirá:

```
a1
a10
a2

a1
a2
a10
```

Veja também

- Pdo\Sqlite::createFunction() - Registra uma função definida pelo usuário para uso em instruções SQL
- Pdo\Sqlite::createAggregate() - Registra uma função de agregação definida pelo usuário para uso em instruções SQL
- **sqlite_create_function()**
- **sqlite_create_aggregate()**

Pdo\Sqlite::createFunction

Pdo\Sqlite::createFunction

(PHP 8 >= 8.4.0)

Pdo\Sqlite::createFunction — Registra uma função definida pelo usuário para uso em instruções SQL

Descrição

```
public function Pdo\Sqlite::createFunction(
    string $function_name,
    callable $callback,
    int $num_args = -1,
    int $flags = 0
): bool
```

Este método permite que a função PHP seja registrada no SQLite como uma função definida pelo usuário, para que possa ser chamada em consultas SQL. A função definida pode ser usada em qualquer consulta SQL que permita chamadas de função, por exemplo `SELECT`, `UPDATE` ou gatilhos.

> **Dica**
>
> Usando este método é possível substituir funções SQL nativas.

Parâmetros

`function_name`

O nome da função usada nas instruções SQL.

`callback`

Função de retorno de chamada para lidar com a função SQL definida.

> **Nota** :
>
> As funções de retorno de chamada devem retornar um tipo compreendido pelo SQLite (ou seja, tipo escalar).

Esta função precisa ser definida como:

function callback(mixed `$value`, mixed `...$values`): mixed

`value`

O primeiro argumento passado para a função SQL.

`values`

Argumentos adicionais passados para a função SQL.

`num_args`

O número de argumentos que a função SQL utiliza. Se este parâmetro for `-1`, a função SQL poderá receber qualquer número de argumentos.

`flags`

Uma máscara de bits de opções. Atualmente, apenas **`Pdo\Sqlite::DETERMINISTIC`** é suportada, o que especifica que a função sempre retorna o mesmo resultado dadas as mesmas entradas em uma única instrução SQL.

Valor Retornado

Retorna **`true`** em caso de sucesso ou **`false`** em caso de falha.

Examples

Exemplo #1 Exemplo de **Pdo\Sqlite::createFunction()**

Neste exemplo, temos uma função que calcula a soma SHA256 de uma string e depois a inverte. Quando a instrução SQL é executada, ela retorna o valor do nome do arquivo transformado pela nossa função. Os dados retornados em *$rows* contêm o resultado processado.

A beleza desta técnica é que não há necessidade de processar o resultado usando um laço `foreach` após a consulta.

```php
<?php
function sha256_and_reverse($string)
{
    return strrev(hash('sha256', $string));
}

$db = new Pdo\Sqlite('sqlite::sqlitedb');
$db->createFunction('sha256rev', 'sha256_and_reverse', 1);
$rows = $db->query('SELECT sha256rev(filename) FROM files')->fetchAll();
?>
```

Veja também

- Pdo\Sqlite::createAggregate() - Registra uma função de agregação definida pelo usuário para uso em instruções SQL
- Pdo\Sqlite::createCollation() - Registra uma função definida pelo usuário para uso como função de agrupamento em instruções SQL
- **sqlite_create_function()**
- **sqlite_create_aggregate()**

Pdo\Sqlite::loadExtension

Pdo\Sqlite::loadExtension

(PHP 8 >= 8.4.0)

Pdo\Sqlite::loadExtension — Carrega extensão

Descrição

public function **Pdo\Sqlite::loadExtension**(string `$name`): void

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Descrição.

Parâmetros

`name`

Descrição.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

Erros `E_*`, e/ou exceções Exception.

Examples

Exemplo #1 Exemplo de **Pdo\Sqlite::loadExtension()**

Descrição.

```
<?php
echo "Exemplo de código";
?>
```

O exemplo acima produzirá:

```
Code example
```

Notas

> **Nota** :
>
> Notas.

Veja também

- **ClassName::otherMethodName()**
- **some_function()**

Pdo\Sqlite::openBlob

Pdo\Sqlite::openBlob

(PHP 8 >= 8.4.0)

Pdo\Sqlite::openBlob — Abre Blob

Descrição

```
public function Pdo\Sqlite::openBlob(
    string $table,
    string $column,
    int $rowid,
    ?string $dbname = "main",
    int $flags = Pdo\Sqlite::OPEN_READONLY
): resource|false
```

> **Aviso**
>
> Esta função não está documentada; apenas a lista de argumentos está disponível.

Descrição.

Parâmetros

`table`

Descrição.

`column`

Descrição.

`rowid`

Descrição.

`dbname`

Descrição.

`flags`

Uma das constantes `Pdo\Sqlite::OPEN_*`.

Valor Retornado

Descrição.

Erros/Exceções

Erros `E_*`, e/ou exceções Exception.

Examples

Exemplo #1 Exemplo de **Pdo\Sqlite::openBlob()**

Descrição.

```
<?php
echo "Code example";
?>
```

O exemplo acima produzirá:

```
Code example
```

Notas

> **Nota** :
>
> Notas.

Veja também

- **ClassName::otherMethodName()**
- **some_function()**

Pdo\Sqlite::setAuthorizer

Pdo\Sqlite::setAuthorizer

(PHP 8 >= 8.5.0)

Pdo\Sqlite::setAuthorizer — Configures a callback to be used as an authorizer to limit what a statement can do

Descrição

public function **Pdo\Sqlite::setAuthorizer**(?callable `$callback`): void

Sets a callback that will be called by SQLite every time an action is performed (reading, deleting, updating, etc.). This is used when preparing an SQL statement from an untrusted source to ensure that the statement does not access data it is not allowed to see or execute malicious statements that damage the database.

The authorizer is used only during the statement preparation phase. It may be called several times for a single statement: a `SELECT` or `UPDATE` query calls it for every column that would be read or updated. It runs again whenever SQLite re-prepares a statement, which may happen while the statement is being executed, for instance after the schema has changed.

The authorizer is called with up to five arguments. The arguments received are described at SQLite3::setAuthorizer().

Only a single authorizer can be in place on a database connection at a time. Each call to this method overrides the previous one. The authorizer is disabled by default, and can be disabled again by setting a `null` callback.

The callback must not modify the database connection that invoked it.

More details can be found in the » SQLite documentation.

> **Nota** :
>
> This method is the equivalent of SQLite3::setAuthorizer(), except that it returns void instead of bool.

Parâmetros

`callback`

The callable to be invoked, or `null` to disable the current authorizer callback.

It must return one of **`Pdo\Sqlite::OK`**, **`Pdo\Sqlite::DENY`**, or **`Pdo\Sqlite::IGNORE`**. When **`Pdo\Sqlite::DENY`** is returned, the statement that triggered the authorizer fails with an error stating that access is denied. When **`Pdo\Sqlite::IGNORE`** is returned for a read action, the statement is prepared so that a `null` value is substituted for the column that would have been read.

Valor Retornado

Nenhum valor é retornado.

Erros/Exceções

This method itself does not throw, but if the authorizer callback does not return an int, or returns an int which is not one of **`Pdo\Sqlite::OK`**, **`Pdo\Sqlite::DENY`**, or **`Pdo\Sqlite::IGNORE`**, the statement preparation throws a TypeError or a ValueError respectively.

Examples

Exemplo #1 **Pdo\Sqlite::setAuthorizer()** example

Only read actions are allowed on the connection. The action codes are exposed as SQLite3 class constants, which requires the SQLite3 extension to be available; their integer values may be used directly otherwise.

```php
<?php
$db = new Pdo\Sqlite('sqlite::memory:');
$db->exec('CREATE TABLE users (id, name)');
```

```php
$db->setAuthorizer(function (int $action, ...$args) {
    return match ($action) {
        SQLite3::SELECT, SQLite3::READ => Pdo\Sqlite::OK,
        default => Pdo\Sqlite::DENY,
    };
});

var_dump($db->query('SELECT name FROM users') instanceof PDOStatement);

try {
    $db->exec('DROP TABLE users');
} catch (PDOException $e) {
    echo $e->getMessage(), "\n";
}
?>
```

O exemplo acima produzirá algo semelhante a:

```
bool(true)
SQLSTATE[HY000]: General error: 23 not authorized
```

Veja também

- SQLite3::setAuthorizer() - Configura um retorno de chamada a ser usado como um autorizador para limitar o que uma declaração pode fazer